

Introduction to Computing Systems

From Bits & Gates to C/C++ & Beyond

计算机系统概论

从位与逻辑门到 C/C++ 及更远

第三版 · 中文翻译稿

Yale N. Patt Sanjay J. Patel

翻译：GPT-6 Astra

校对：王子川

联系方式：wangzichuan@mail.ustc.edu.cn

2026 年 10 月 1 日

目录

前言	xiii
本书的由来	xiii
第三版的主要变化	xiv
章节组织	xv
模拟器/调试器	xviii
本书的其他用法	xviii
为什么选择 LC-3，而不是 ARM 或 RISC-V?	xix
几点观察	xx
致谢	xxi
最后的话	xxiii
第一章 欢迎启程	1
1.1 我们想做什么	1
1.2 我们将怎样达到目标	1
1.3 贯穿全书的两个主题	2
1.3.1 抽象的概念	3
1.3.2 硬件与软件	4
1.4 计算机系统	5
1.4.1 一点历史，带来更好的视角	6
1.4.2 计算机系统的组成部分	8
1.5 两个非常重要的思想	9
1.6 作为通用计算设备的计算机	9
1.7 怎样让电子来干活?	11
1.7.1 问题的表述	11
1.7.2 算法	11
1.7.3 程序	12
1.7.4 ISA	13
1.7.5 微体系结构	14
1.7.6 逻辑电路	14
1.7.7 器件	14
习题	16
第二章 位、数据类型与运算	19
2.1 位与数据类型	19
2.1.1 作为信息单位的位	19
2.1.2 数据类型	19
2.2 整数数据类型	20
2.2.1 无符号整数	20
2.2.2 有符号整数	20
2.3 补码整数	22

2.4	二进制与十进制之间的转换	23
2.4.1	二进制转换为十进制	23
2.4.2	十进制转换为二进制	23
2.4.3	推广到带小数部分的数	24
2.5	位运算——第一部分：算术运算	25
2.5.1	加法与减法	25
2.5.2	符号扩展	25
2.5.3	溢出	26
2.6	位运算——第二部分：逻辑运算	26
2.6.1	逻辑变量	26
2.6.2	AND 运算	26
2.6.3	OR 运算	27
2.6.4	NOT 运算	27
2.6.5	XOR 运算	28
2.6.6	德摩根定律	28
2.6.7	位向量	28
2.7	其他表示	30
2.7.1	浮点数：更大的范围，较低的精度	30
2.7.2	ASCII 码	32
2.7.3	十六进制记法	32
	习题	33
第三章	数字逻辑结构	41
3.1	晶体管	41
3.2	逻辑门	42
3.2.1	NOT 门（反相器）	43
3.2.2	OR 门与 NOR 门	43
3.2.3	为什么不能简单地把 P 型晶体管接地	44
3.2.4	AND 门与 NAND 门	46
3.2.5	超过两个输入的逻辑门	46
3.3	组合逻辑电路	47
3.3.1	译码器	47
3.3.2	多路选择器	48
3.3.3	一位加法器（又称全加器）	49
3.3.4	可编程逻辑阵列（PLA）	51
3.3.5	逻辑完备性	51
3.4	基本存储元件	52
3.4.1	R-S 锁存器	52
3.4.2	门控 D 锁存器	53
3.5	存储器的概念	53
3.5.1	地址空间	54
3.5.2	寻址粒度	54
3.5.3	一个四字、每字三位的存储器	54
3.6	时序逻辑电路	55
3.6.1	一个简单例子：密码锁	57

3.6.2	状态的概念	57
3.6.3	有限状态机及其状态图	59
3.6.4	同步有限状态机	61
3.6.5	时钟	61
3.6.6	例子：危险警示牌	62
3.7	后续内容预览：LC-3 的数据通路	66
	习题	68
第四章	冯·诺依曼模型	91
4.1	基本组成部分	91
4.1.1	存储器	91
4.1.2	处理单元	93
4.1.3	输入与输出	93
4.1.4	控制单元	94
4.2	LC-3：冯·诺依曼机的一个实例	94
4.3	指令处理	96
4.3.1	指令	96
4.3.2	指令周期（不是时钟周期！）	97
4.3.3	改变执行顺序	100
4.3.4	指令周期的控制	101
4.3.5	让计算机停机（TRAP 指令）	103
4.4	我们的第一个程序：乘法算法	104
	习题	106
第五章	LC-3	111
5.1	ISA 概述	111
5.1.1	存储器组织	111
5.1.2	寄存器	111
5.1.3	指令集	113
5.1.4	操作码	113
5.1.5	数据类型	113
5.1.6	寻址方式	115
5.1.7	条件码	115
5.2	运算指令	116
5.2.1	ADD、AND 和 NOT	116
5.2.2	立即数	117
5.2.3	LEA 指令（虽然它并不真是运算指令）	118
5.3	数据传送指令	119
5.3.1	PC 相对寻址	120
5.3.2	间接寻址	120
5.3.3	Base+offset 寻址	121
5.3.4	一个例子	123
5.4	控制指令	124
5.4.1	条件分支	124
5.4.2	两种循环控制方法	126

5.4.3	JMP 指令	129
5.4.4	TRAP 指令	129
5.5	另一个例子：统计字符出现次数	130
5.6	重访数据通路	133
5.6.1	数据通路的基本部件	133
5.6.2	LC-3 的具体指令周期	135
	习题	137
第六章	程序设计	156
6.1	问题求解	156
6.1.1	系统化分解	156
6.1.2	三种结构：顺序、条件和迭代	156
6.1.3	用 LC-3 控制指令实现三种结构	157
6.1.4	重新考察第 5 章的字符计数实例	158
6.2	调试	163
6.2.1	调试操作	163
6.2.2	使用交互式调试器	164
	习题	171
第七章	汇编语言	179
7.1	汇编语言程序设计——向上一步	179
7.2	一个汇编语言程序	180
7.2.1	指令	180
7.2.2	伪操作（汇编指示）	183
7.2.3	实例：再次考察 5.5 节的字符计数程序	184
7.3	汇编过程	186
7.3.1	引言	186
7.3.2	两遍扫描	186
7.3.3	第一遍：建立符号表	187
7.3.4	第二遍：生成机器语言程序	188
7.4	单个汇编语言程序之外	188
7.4.1	可执行映像	189
7.4.2	多个目标文件	189
	习题	190
第八章	数据结构	207
8.1	子程序	207
8.1.1	调用 / 返回机制	209
8.1.2	JSR(R)——调用子程序的指令	210
8.1.3	保存和恢复寄存器	210
8.1.4	库例程	212
8.2	栈	215
8.2.1	栈——一种抽象数据类型	216
8.2.2	两个实现实例	216
8.2.3	在存储器中实现	217
8.2.4	完整实现	221

8.3	递归：运用得当时的强大技术	222
8.3.1	不好的例子之一：阶乘	223
8.3.2	Fibonacci：一个更糟的例子	227
8.3.3	迷宫：一个好的例子	230
8.4	队列	237
8.4.1	基本操作：从队首移除，向队尾插入	237
8.4.2	回绕	238
8.4.3	队列能存储多少个元素？	239
8.4.4	下溢和溢出检测	239
8.4.5	完整实现	240
8.5	字符串	241
	习题	245
第九章	I/O	254
9.1	特权、优先级与存储器地址空间	254
9.1.1	特权与优先级	254
9.1.2	存储器的组织	256
9.2	输入 / 输出	257
9.2.1	I/O 的一些基本特性	257
9.2.2	从键盘输入	259
9.2.3	向显示器输出	261
9.2.4	更完善的输入例程	263
9.2.5	再看存储器映射 I/O 的实现	265
9.3	操作系统服务例程：LC-3 陷阱例程	265
9.3.1	引言	265
9.3.2	陷阱机制	266
9.3.3	TRAP 指令	267
9.3.4	RTI 指令：将控制权交回调用程序	268
9.3.5	陷阱服务过程小结	268
9.3.6	处理 I/O 的陷阱例程	270
9.3.7	使计算机停机的陷阱例程	272
9.3.8	字符输入陷阱例程：最后再看一次	273
9.3.9	PUTS：向显示器输出字符串	275
9.4	中断与中断驱动 I/O	276
9.4.1	什么是中断驱动 I/O？	276
9.4.2	为什么需要中断驱动 I/O？	278
9.4.3	过程的两个部分	278
9.4.4	第一部分：使中断发生	278
9.4.5	第二部分：处理中断请求	281
9.4.6	一个例子	282
9.4.7	不只是 I/O 设备	285
9.5	了解中断之后，再看轮询	285
9.5.1	问题	285
9.5.2	解决办法	286
	习题	287

第十章 计算器	314
10.1 数据类型转换	314
10.1.1 例子：一个有问题的程序—— $2 + 3 = e$	315
10.1.2 输入数据：从 ASCII 到二进制	316
10.1.3 显示结果：从二进制到 ASCII	320
10.2 使用栈进行算术运算	322
10.2.1 把栈用作临时存储	322
10.2.2 一个例子	322
10.2.3 OpAdd、OpMult 和 OpNeg	324
10.3 计算器	330
10.3.1 功能	330
10.3.2 代码	332
习题	337
第十一章 C/C++ 程序设计入门	339
11.1 我们的目标	339
11.2 跨越层次之间的距离	340
11.3 高级语言程序的转换	343
11.3.1 解释	344
11.3.2 编译	344
11.3.3 优点与缺点	344
11.4 C/C++ 编程语言	344
11.4.1 C 和 C++ 的起源	344
11.4.2 我们将怎样学习 C 和 C++	345
11.4.3 编译过程	345
11.4.4 软件开发环境	347
11.5 一个简单的 C 示例	348
11.5.1 main 函数	349
11.5.2 格式、注释与风格	350
11.5.3 C 预处理器	351
11.5.4 输入与输出	352
11.6 小结	354
习题	355
第十二章 变量与运算符	357
12.1 引言	357
12.2 变量	357
12.2.1 四种基本数据类型	358
12.2.2 选择标识符	359
12.2.3 作用域：局部与全局	360
12.2.4 更多例子	362
12.3 运算符	362
12.3.1 表达式与语句	363
12.3.2 赋值运算符	364
12.3.3 算术运算符	364

12.3.4 求值次序	365
12.3.5 位运算符	366
12.3.6 关系运算符	368
12.3.7 逻辑运算符	368
12.3.8 自增与自减运算符	369
12.3.9 包含多个运算符的表达式	371
12.4 用运算符解决问题	371
12.5 把各个概念联系起来	373
12.5.1 符号表	373
12.5.2 为变量分配空间	374
12.5.3 一个综合实例	376
12.6 补充主题	379
12.6.1 基本类型的变体	379
12.6.2 字面量、常量与符号值	380
12.6.3 其他 C 运算符	381
12.7 小结	382
习题	382
第十三章 控制结构	386
13.1 引言	386
13.2 条件结构	386
13.2.1 if 语句	386
13.2.2 if-else 语句	389
13.3 迭代结构	393
13.3.1 while 语句	393
13.3.2 for 语句	396
13.3.3 do-while 语句	400
13.4 使用控制结构求解问题	402
13.4.1 问题 1: 求 π 的近似值	402
13.4.2 问题 2: 找出小于 100 的素数	405
13.4.3 问题 3: 分析电子邮件地址	408
13.5 其他 C 控制结构	410
13.5.1 switch 语句	410
13.5.2 break 和 continue 语句	412
13.5.3 示例: 简单计算器	412
13.6 小结	414
习题	414
第十四章 函数	420
14.1 引言	420
14.2 C 中的函数	420
14.2.1 带形参的函数	421
14.2.2 示例: 圆环面积	424
14.3 C 函数的实现	425
14.3.1 运行时栈	426

14.3.2 让各部分协同工作	428
14.3.3 将各部分整合起来	433
14.4 使用函数求解问题	435
14.4.1 问题 1: 大小写转换	435
14.4.2 问题 2: 勾股数组	437
14.5 小结	439
习题	439
第十五章 测试与调试	447
15.1 引言	447
15.2 错误的类型	448
15.2.1 语法错误	448
15.2.2 语义错误	449
15.2.3 算法错误	451
15.2.4 规格错误	452
15.3 测试	452
15.3.1 黑盒测试	453
15.3.2 白盒测试	453
15.4 调试	454
15.4.1 临时调试方法	454
15.4.2 源代码级调试器	455
15.5 为正确性而编程	456
15.5.1 明确规格	456
15.5.2 模块化设计	457
15.5.3 防御式编程	457
15.6 小结	458
习题	459
第十六章 指针与数组	465
16.1 引言	465
16.2 指针	465
16.2.1 声明指针变量	467
16.2.2 指针运算符	468
16.2.3 用指针传递引用	469
16.2.4 空指针	471
16.2.5 理解这些语法	471
16.2.6 一个使用指针的问题	472
16.3 数组	474
16.3.1 声明和使用数组	474
16.3.2 使用数组的例子	476
16.3.3 数组作为参数	479
16.3.4 C 中的字符串	481
16.3.5 C 中数组与指针的关系	485
16.3.6 问题求解: 插入排序	485
16.3.7 C 数组的常见陷阱	488

16.3.8 变长数组	489
16.3.9 C 中的多维数组	490
16.4 小结	492
习题	493
第十七章 递归	499
17.1 引言	499
17.2 什么是递归?	499
17.3 递归与迭代	501
17.4 汉诺塔	501
17.5 Fibonacci 数	506
17.6 二分查找	511
17.7 走出迷宫	514
17.8 小结	516
习题	518
第十八章 C 中的 I/O	525
18.1 引言	525
18.2 C 标准库	525
18.3 逐字符 I/O	526
18.3.1 I/O 流	526
18.3.2 putchar	526
18.3.3 getchar	527
18.3.4 缓冲 I/O	527
18.4 格式化 I/O	529
18.4.1 printf	529
18.4.2 scanf	530
18.4.3 可变实参列表	532
18.5 文件 I/O	533
18.6 小结	536
习题	537
第十九章 C 中的动态数据结构	539
19.1 引言	539
19.2 结构体	539
19.2.1 typedef	542
19.2.2 C 中结构体的实现	542
19.3 结构体数组	543
19.4 动态内存分配	546
19.4.1 动态大小数组	547
19.5 链表	549
19.5.1 辅助函数	550
19.5.2 向链表添加节点	553
19.5.3 从链表删除节点	558
19.5.4 数组与链表的比较	559
19.6 小结	560

习题	561
第二十章 C++ 入门	564
20.1 C++ 基础	564
20.2 从 C 到 C++	565
20.2.1 编译 C++ 代码	565
20.2.2 命名空间	567
20.2.3 输入和输出	567
20.2.4 传递引用	568
20.2.5 函数重载	568
20.2.6 动态分配	569
20.2.7 编译为机器代码	569
20.3 类	570
20.3.1 方法	570
20.3.2 访问说明符	574
20.3.3 构造函数	574
20.3.4 进阶主题	577
20.4 容器与模板	577
20.4.1 vector	578
20.4.2 模板	579
20.5 小结	580
习题	581
附录 A LC-3 指令集体系结构	583
A.1 概述	583
A.2 指令集	585
A.3 中断与异常处理	605
A.3.1 中断	605
A.3.2 异常	606
附录 B 从 LC-3 到 x86	608
B.1 LC-3 的特征与对应的 x86 特征	609
B.1.1 指令集	609
B.1.2 内存	613
B.1.3 内部状态	615
B.2 x86 指令的格式与说明	618
B.2.1 前缀	618
B.2.2 操作码	619
B.2.3 ModR/M 字节	619
B.2.4 SIB 字节	620
B.2.5 位移量	620
B.2.6 立即数	621
B.3 一个例子	622
附录 C LC-3 的微体系结构	626
C.1 概述	626

C.2	状态机	627
C.3	数据通路	630
C.4	控制结构	632
C.5	TRAP 指令	636
C.6	存储器映射 I/O	636
C.7	中断与异常控制	637
C.7.1	启动中断	639
C.7.2	从中断返回	640
C.7.3	启动异常	640
C.8	控制存储器	641
附录 D	C 程序设计语言	643
D.1	概述	643
D.2	C 语言的约定	643
D.2.1	源文件	643
D.2.2	头文件	643
D.2.3	注释	643
D.2.4	字面量	644
D.2.5	格式	646
D.2.6	关键字	646
D.3	类型	646
D.3.1	基本数据类型	646
D.3.2	类型修饰	647
D.3.3	存储类	649
D.3.4	派生类型	649
D.3.5	typedef	651
D.4	声明	651
D.4.1	变量声明	651
D.4.2	函数声明	652
D.5	运算符	653
D.5.1	赋值运算符	653
D.5.2	算术运算符	653
D.5.3	位运算符	654
D.5.4	逻辑运算符	654
D.5.5	关系运算符	655
D.5.6	自增与自减运算符	655
D.5.7	条件表达式运算符	655
D.5.8	指针、数组和结构体运算符	655
D.5.9	sizeof	657
D.5.10	求值次序	657
D.5.11	类型转换	658
D.6	表达式与语句	659
D.6.1	表达式	659
D.6.2	语句	659
D.7	控制	660

D.7.1	if	660
D.7.2	if-else	660
D.7.3	switch	661
D.7.4	while	661
D.7.5	for	662
D.7.6	do-while	662
D.7.7	break	663
D.7.8	continue	663
D.7.9	return	663
D.8	C 预处理器	664
D.8.1	宏替换	664
D.8.2	文件包含	664
D.9	部分标准库函数	665
D.9.1	I/O 函数	665
D.9.2	字符串函数	667
D.9.3	数学函数	667
D.9.4	实用函数	667
附录 E	常用表格	669
E.1	常用数值词头	669
E.2	标准 ASCII 码	670
E.3	2 的幂	671
附录 F	部分习题的解答	672
	术语表	673

前言

第三版终于问世了！首先，我们必须感谢所有一直催促我们推出第三版的人。第二版出版至今已经过去很久，书中的内容显然该过时了吧？并非如此！基础知识并不会频繁变化。我们的初衷仍是写一本把基础知识讲清楚的书，因为我们相信：只要掌握了基础，只要才智与专注程度足以应对挑战，一个人所能达到的高度就没有上限。

我们也必须为这次修订花了这么长时间而致歉。请相信，延误绝不意味着我们的热情有所减退。25年前，我们重新设计了计算领域的第一门课程，后来它发展成本书的第一版；今天，我们对计算基础的热情依然如初。事实上，我们两人一直都在定期开设这门课程。正如预料的那样，每教一次，我们都会对该教什么产生新的认识，找到新的例子和观察问题的新角度。我们希望，第三版已经体现了这些积累。

终于能写这篇前言，令人欣慰。学习过本书的学生和使用本书授课的教师给我们寄来了大量意见。许多人赞同我们的教学方法，而且这种赞同来自亲身经历：学生亲自用它学习，教师亲眼看到学生用它学习。这让我们深感欣慰。他们在来信中表现出的热情，一直激励着我们。

本书的由来

本书由 EECS 100 课程发展而来。这是密歇根大学安娜堡分校面向计算机科学、计算机工程和电气工程专业的学生的第一门计算课程，由 Kevin Compton 和本书第一作者于 1995 年秋季首次开设。

密歇根大学之所以开设 EECS 100，是因为多年来，计算机科学与工程系的教师一直对学生未能理解一些非常基本的概念感到不满。例如，学生理解指针变量时困难重重；递归似乎成了无法理解的“魔法”。

1993 年，我们认定，从高级编程语言开始教起这一传统做法存在不足。当时我们和大多数大学都在这样做。我们认为，学生之所以学不明白，是因为他们尚未理解底层基础，就被迫记忆技术细节。

由此形成了本书自底向上的方法：不断在学生已有知识之上构建新知识，只有在必要时才要求记忆。对于学习，我们过去不赞同、现在也不赞同流行的信息隐藏 (information hiding) 方法。在理解了系统如何工作之后，信息隐藏是提高效率的有用技术；但在达到这一点之前，我们坚持认为，信息隐藏会妨碍理解。因此，我们始终在已有内容之上继续构建，使一切都能与已建立的基础联系起来，不再有“魔法”。

需要指出，我们并不反对自顶向下的设计。恰恰相反，我们坚信，自顶向下是正确的设计方法。但是，理解了底层构件后如何解决设计问题，与怎样才能理解这些构件，是两件明显不同的事。简言之，我们主张自顶向下地设计，自底向上地学习，以求真正理解。

第三版的主要变化

LC-3

LC-3 ISA 仍是本书的一大特色。它足够小，只需几页就能描述清楚，我们希望读者能在很短时间内掌握；同时它又足够丰富，可以传达 ISA 所提供的核心内容。之所以叫 LC “3”，是因为我们尝试了三次才把它做好。其实是四次，不过，第二版以来对 LC-3 ISA 的两项修改，即对 LEA 和 TRAP 指令的修改，都很小，因此我们决定不把稍作修改后的 ISA 改称 LC-4。

LEA 指令不再设置条件码。它过去会设置条件码，是基于一种错误认识：既然 LEA 的全称是 Load Effective Address，就应该像 LD、LDI 和 LDR 一样设置条件码。如今我们认识到，这个理由并不合理。LD、LDI 和 LDR 从存储器向寄存器加载数据，所以条件码能提供有用信息，说明加载的值为负、为零还是为正。LEA 加载到寄存器中的是地址，条件码对此并没有实际价值。在这项修改之前编写的旧代码应当仍能正常运行。

TRAP 指令不再把返回调用程序所需的链接信息保存在 R7 中。现在，PC 和 PSR 被压入系统栈，并由陷阱例程最后执行的 RTI 指令弹出。RTI 已改称 Return from Trap or Interrupt，即“从陷阱或中断返回”。陷阱例程现在在特权内存区域（x0000 至 x2FFF）中执行。这项修改使陷阱例程可以重入。只要从 Trap Vector Table 中取得的陷阱服务例程起始地址位于特权内存区域，并把每个陷阱服务例程的最后一条指令从 RET 改为 RTI，就不会影响旧代码。

与此前一样，附录 A 给出了 LC-3 的完整规范。

加入 C++

对于如何把我们的教学方法和教材扩展到 C++，我们一直在讨论。一个顾虑是，C++ 的许多语言特性与底层相隔太远，不容易融入我们的方法。另一个顾虑是，C++ 太庞大，要作足够充分的介绍，还得再加上一千页。我们也不想完全舍弃 C，因为它实际上是系统开发和面向硬件项目的通用开发语言。

我们采用的办法，是在第 11 至第 19 章介绍 C 和 C++ 共有的核心。这些内容与第二版相近，只做了一些小更新。第 20 章承担过渡作用，我们希望它能让读者平顺地进入 C++ 的核心概念。这样就有机会考察从 C 到 C++ 的演进，并借此理解：哪些变化对于提高程序员的生产效率至关重要。

具体来说，我们把 C++ 的类作为 C 中结构体的演进来介绍。我们从编译器构造的角度讨论类，讲解如何调用方法以及构造函数的概念。我们也简要涉及继承，但把详细内容留给后续课程。

C++ 的一个重要内容，是标准模板库（Standard Template Library, STL）中的容器类，这是 C++ 中使用极为广泛的一部分。它为我们深入讨论 vector 类提供了机会，也延续了本书后半部分的一个贯穿性例子：高级语言如何支持大小可变的数组，尤其是 C 对这类数组支持不足的问题。

其他重要更新

全书每一章都做了修改，但有些章节的改动更多。应几位教师的要求，第 2 章扩展了浮点数据类型以及小数在十进制和二进制之间转换的内容。我们把德摩根定律从第 3 章移到第 2 章，因为它本质上讨论的是 AND 和 OR 函数，而非数字逻辑实现。第 3 章对状态、锁存器、触发器、有限状态机和危险警示标志这一例子的说明做了全面改写。我们认为第二版的解释还不够清楚，而这些概念又太重要，必须讲好。第 4 章也经过修订，以便更好地引入 LC-3，其中采用了一组不同的指令，进而给出第一个完整的计算机程序示例。

第 8、9、10 章的组织方式经过全面调整，以更容易理解的方式介绍基本相同的内容。尽管大部分数据结构要等到后半本书介绍 C 后才讲，我们仍认为，应当在学生刚刚脱离机器语言编程时就介绍栈、队列和字符串。这样，他们便能编写处理这些数据结构的程序，并看到这些结构实际上怎样组织在内存中。我们把子程序的讨论提前到第 8 章，因为子程序对于构造功能更丰富的程序十分重要。

我们还在第 8 章引入了递归，不过主要讨论仍放在后半本书。递归的表达能力和对递归的误用，在本科课程中都十分常见，因此我们认为值得讲两遍：先在学生深入汇编语言细节时讲一次，再在他们进入表达能力更丰富的 C 之后讲一次。

第 9 章现在集中介绍 I/O 的各个方面，包括轮询和中断驱动 I/O。第二版已经涉及特权概念，第三版则更加强调它。系统调用，即由 TRAP 指令调用的陷阱例程，也放在第 9 章。这样，第 10 章便集中成为一个综合例子：模拟计算器，把前半本书的许多内容串联起来。它需要 12 个子程序，我们逐一给出了完整细节。实现这一例子还需要栈算术和 ASCII/二进制转换，因此第 10 章也包含这两个概念。

我们按照最新的 ANSI C 标准，即 C18，重新整理了第 11 至第 19 章的全部例子。我们还增加了更多编程实例，以突出重点，并把 C 中指针、数组、递归和指向指针的指针等复杂主题讲清楚。第 16 章新增了介绍 C 中大小可变数组和多维数组的小节。

章节组织

本书分为两大部分：(a) 以 LC-3 为具体实例的计算机底层结构；(b) 高级语言编程，本书使用 C 和 C++。

LC-3

我们从理解真实计算机工作原理所需的基础开始。第 2 章介绍位以及对位进行的算术运算和逻辑运算。随后，我们开始构建理解 LC-3 所需的结构。第 3 章从 MOS 晶体管出发，一步步引导学生构建“真正的”存储器和有限状态机。

我们的真正存储器由 4 个字组成，每个字 3 位，并非如今大多数笔记本电脑中常见的 16 GB。它的完整结构可以放在一页中（图 3.20），便于学生掌握。学到这里时，学生已经接触了构建这块存储器所需的全部元件。有限状态机则用于理解计算机如何处理指令，这部分从第 4 章开始。第 4 章介绍冯·诺依曼执行模型，以及足以编写一

个 LC-3 程序的若干条 LC-3 指令。第 5 章介绍 LC-3 ISA 中其余指令的大部分。

LC-3 是一种 16 位体系结构，包含通过键盘和显示器进行的物理 I/O、向操作系统请求服务的 TRAP、依据 N、Z、P 条件码进行的条件分支、子程序调用/返回机制、最小的一组运算指令（ADD、AND 和 NOT），以及用于加载和存储的多种寻址方式（直接、间接、基址加偏移量）。

第 6 章专门讨论编程方法，即逐步求精，以及调试；第 7 章是汇编语言编程入门。我们为 LC-3 开发了模拟器和汇编器，可以在 Windows、Linux 和 MacOS 平台运行，并可从网上免费下载。

学生使用模拟器测试和调试用 LC-3 机器语言及汇编语言编写的程序。模拟器支持交互式调试，包括写入、查看、单步执行、设置断点等功能。它可以用于简单的 LC-3 机器语言和汇编语言编程作业，而这些作业对于学生掌握前十章的概念不可或缺。

我们教授汇编语言，并不是为了培养汇编语言编程专家。若目标是培养这样的程序员，这些内容应当放在高年级课程，而非大一入门课程中。第 7 章介绍汇编语言，是因为它符合本书的教学方式。按照自底向上的学习路径，学到第 7 章时，学生已经能够处理把汇编语言程序转换成 0、1 序列的过程。我们针对一个非常简单的 LC-3 汇编器，逐步讲解汇编过程。通过手工汇编，学生只需多花很少的时间，就能强化“转换”这一重要基础概念。

汇编语言还提供了便于人阅读的机器指令表示法。这对于编写第 8、9、10 章的程序，以及解释后半本书的许多内容，都特别有用。从第 11 章开始讲解 C 语句的语义时，读者理解 ADD R1, R2, R3，远比费力解读 0001001010000011 容易。

第 8 章介绍三种重要数据结构：栈、队列和字符串，并说明它们如何存放在内存中。由于 LC-3 的子程序调用/返回机制既有助于操作这些数据结构，也有助于编写程序，我们在此介绍它。我们还引入递归这一强大的构造。在学生具备更扎实的高级语言编程能力后，后半本书的第 17 章将更全面地讨论递归。这里先通过几个例子展示递归在执行时间方面带来的取舍，作为理解何时适合、何时不适合使用递归的第一步。

第 9 章讨论输入/输出，以及处理器与操作系统之间的一些基本交互。我们引入优先级和特权这两个系统环境中的核心概念。这里的 I/O 讨论全部涉及物理层面：输入使用键盘数据寄存器和状态寄存器，输出使用显示器数据寄存器和状态寄存器。我们既描述中断驱动 I/O，也描述程序控制 I/O。LC-3 模拟器同时支持这两种方式，因此学生可以编写中断驱动程序。最后，我们展示陷阱服务例程的实际 LC-3 代码；学生从第 4 章起就在通过 TRAP 指令调用它们。为了处理中断驱动 I/O 和陷阱服务例程，我们通过详细说明 RTI（Return from Trap or Interrupt）和 TRAP 指令的操作，补全 LC-3 ISA 的介绍。

前半本书以第 10 章结束。该章用一个简单计算器的综合例子，汇集学生在第 1 至第 9 章学到的许多知识。

C 和 C++ 编程

进入本书第二部分时，学生已经理解了底下各层。在介绍 C 和 C++ 编程时，每引入一个新概念，我们都会展示编译器为它生成的 LC-3 代码，以此运用已经建立的基础。

我们先讲 C，因为它提供了与 C++ 共有的基本核心。C 很适合我们自底向上的方法。它较为贴近底层，使学生可以清楚地看到软件与底层硬件之间的联系。本书着重介绍控制结构、函数、数组等基本概念。掌握这些基本编程概念后，学生只需再迈出一小步，就能学习 C++ 中对象和抽象等更高级的概念。

每次引入一个新的高级语言构造，我们都会展示编译器为它生成的 LC-3 代码。我们介绍 C 的基本构造（变量、运算符、控制和函数）、指针、数组、递归、I/O、复杂数据结构及动态分配。在 C++ 部分，我们介绍相对于 C 的一些基本改进，以及类和容器。

第 11 章以循序渐进的方式引入高级编程语言。学到这里时，学生已经大量使用过汇编语言，能够理解高级编程语言所提供的功能背后的动机。这一章还给出了一个简单的 C 程序，以此开启 C 的学习。

第 12 章讨论值、变量、常量和运算符。第 13 章介绍 C 的控制结构。我们提供了许多完整程序，让学生看到这些概念在实践中如何使用，并利用 LC-3 代码展示每一种 C 构造怎样影响更低层的机器。

第 14 章介绍 C 中的函数。学生接触的不只是函数语法，还会学习函数实际上如何执行，以及如何通过运行时栈传递参数。我们为此提供了多个例子。

第 15 章介绍代码测试技术，以及如何调试高级语言源代码，其中讨论了白盒测试和黑盒测试。

第 16 章讲解指针和数组，充分利用学生对存储器组织方式的理解。我们讨论 C 中固定大小数组和变长数组的概念，以及多维数组的分配。

第 17 章利用学生刚刚掌握的函数、栈帧和运行时栈知识讲解递归。第 18 章介绍 C 中 I/O 函数的细节，特别是流、可变长度参数列表，以及各种格式说明如何影响 C 的 I/O。这一章依赖学生此前在第 8 章接触的物理 I/O 知识。第 19 章讨论 C 的结构体、动态内存分配和链表。

第 20 章帮助学生快速开始 C++ 编程：先介绍 C++ 在 C 中的根源，再把类作为结构体的自然演进引入。我们还讨论标准模板库中容器的概念，使学生能够迅速投入有效的 C++ 编程。

在整个过程中，我们尽量通过例子强调良好的编程风格和编码方法。初学者从阅读程序示例中学到的东西，很可能并不少于从被要求学习的规则中学到的东西。与这些示例相伴的重要认识，会用页边的灯泡图标突出显示。

我们发现，指针变量这一概念（第 16 章）完全不成问题。学生学到这里时，已经分析过一块小型存储器的逻辑设计（第 3 章），对存储器有了充分认识。例如，他们知道存储单元的地址与其中存放的数据有什么区别。

递归也不再神秘，因为学到第 17 章时，学生已经接触了它的全部基础。他们理

解栈如何在机器层面工作（第 8 章）；通过 LC-3 机器语言编程，他们理解调用/返回机制，以及被调用程序与返回调用者之间为什么需要链接信息（第 8 章）。在这一基础上，通过引入运行时栈帧（第 14 章）来解释函数，就只需迈出不大的一步，参数传递、动态声明等内容的许多神秘之处也随之消失。既然函数可以调用函数，那么函数调用自身也只需再迈一小步，当然不涉及任何魔法。

模拟器/调试器

模拟器/调试器对于测试学生编写的程序有多重要，再怎么强调也不为过。我们坚信，亲手实践、检验自己的知识，是任何东西都无法替代的。写出一个无法正确运行的程序，通过测试找出原因，亲自修复错误，最后看到程序正确运行，这对学生的学习极有成就感。为此，我们完全重写了模拟器/调试器。它在 Windows、Linux 和 MacOS 上运行，无论学生使用哪种平台，提供的图形用户界面（GUI）都相同。我们还改进了模拟器对中断驱动 I/O 的支持，使学生能轻松编写中断驱动程序，并通过中断一个正在执行的低优先级程序来调用它们。学生在第一门计算课程中，就能做到这些！

本书的其他用法

我们把本书写成面向大一学生的计算入门教材。我们坚信，这种循序引导的自底向上方法，是学习计算基础的最佳方式。大量证据表明，理解计算机工作原理的学生，通常更容易掌握后来遇到的内容，包括必须使用的高级编程语言。由于一切都有道理，他们学习这些语言的规则时，也需要少得多的死记硬背。在我们看来，本书最适合用于面向学习动力特别强的大一学生的一学期课程，或用于节奏较缓的两学期系列课程。

话虽如此，我们也认识到其他人对课程安排有不同看法。因此需要补充，本书当然也能以其他方式有效使用。事实上，下面每一种安排都经过实践，并且都取得了成功。

两个学季的大一课程

这是使用本书的一种很好方式。不需要先修课程，两个学季就能比较从容地讲完全书：第一学季讲第 1 至第 10 章，第二学季讲第 11 至第 20 章。进度虽然较快，但两个学季足以完成全书内容。

一学期的第二门课程

有几所学校成功地把本书用于第二门课程；此前，学生已经在较温和的第一门课程中接触了面向对象语言编程。这样安排的理由是，学生在第一门课程接触高级语言编程后，第二门课程应以入门深度介绍数字逻辑、基本计算机组成和汇编语言编程。

学期前面三分之二的时间用于第 1 至第 10 章，后面三分之一用于第 11 至第 20 章，既教授 C 编程，也说明第一门课程中的一些“魔法”如何实现。函数、活动记录、递归、指针变量和数据结构，都是揭开神秘面纱特别有帮助的主题。由于学生已经学过编程入门课，后半本书可以推进得更快。这种模式还让先接触面向对象语言的学生学会 C；在后续高级软件课程中，他们几乎肯定会用到它。

大二的计算机组成课程

本书也曾用于大二课程，深入探讨计算机实现。整个学期用于第 1 至第 10 章，最后往往深入学习附录 C，其中给出了微程序控制的 LC-3 的完整微体系结构。不过，需要指出，本书没有涉及计算机体系结构中的一些重要思想，尤其是高速缓存、流水线和虚拟内存。采用这种方式授课的教师，宜另发讲义介绍这些主题。我们赞同它们对于学生的体系结构教育十分重要，但认为它们更适合放在后续的计算机体系结构与设计课程中。本书并不是为那一目的而写的。

为什么选择 LC-3，而不是 ARM 或 RISC-V？

有人问过我们：为什么要自行设计 LC-3 ISA，而不采用似乎已经成为大多数移动设备首选 ISA 的 ARM，或近年来引起广泛兴趣的 RISC-V？

原因有很多。首先，我们知道所选择的 ISA 是学生的第一个 ISA，而不是最后一个。从大一到毕业，学生很可能接触多个 ISA，其中多数用于商业产品，例如 ARM、RISC-V、x86 和 POWER。

然而，所有商业 ISA 都有一些不适合放入门课程的细节，学生却又必须理解这些细节才能有效使用它们。当然，我们可以选取某个现有 ISA 的子集，但这总会引发该删什么、该留什么的问题，结果并不像乍看之下那么简洁，肯定也比不上从头设计来得清晰。而且，如果学生在考试或作业中使用了子集之外的指令，还会产生问题。从教学角度看，这并不利落。

我们希望 ISA 简洁清晰，没有需要特别处理的特殊情况，并把操作码数量减到必要的最低限度。这样，学生可以把几乎全部时间用于课程的基本概念，只花很少时间处理指令集的细枝末节。LC-3 全部指令的格式可以放在一页里。附录 A 用 25 页给出了整个 LC-3 ISA 的所有细节，也就是完整的数据手册。

我们还希望，指令集虽然只有少量指令，覆盖的功能却足够广。因此，我们设计了 LC-3：它只有 15 个 4 位操作码，少到学生无需刻意记忆就能掌握。算术运算只有 ADD，而不是同时提供 ADD、SUB、MUL、DIV。逻辑运算有 AND 和 NOT，而没有 OR、XOR 等。我们也没有移位或循环移位指令。在这些情况下，缺失的操作都可以通过使用 LC-3 已有少数操作码的过程来实现。加载和存储具有三种寻址方式，各有不同用途。此外，还有条件分支、子程序调用、从陷阱或中断返回，以及系统调用 (TRAP 指令)。

事实上，操作码精简正是一个优点！它让学生切实体会到，必须用简单操作构造复杂功能，也必须使用抽象。这两点都是本书的核心概念。

更重要的是，与数百名学生交谈后，我们发现，从 LC-3 开始并不会使他们在后续课程中处于劣势，情况恰好相反。例如，一所学校的后续课程引入 ARM，另一所学校则引入 x86。

两所学校的学生都很庆幸自己从 LC-3 开始。由于先借助 LC-3 学会了基础概念，后来学习 ARM 或 x86 都容易得多。

几点观察

我们两人合计已经教授这门课二十多次，有以下几点体会。

理解，而不是记忆

由于课程自底向上构建，我们发现，与传统编程课程相比，学生需要记忆的那些看似任意的规则更少。学生明白规则是有道理的，因为学到某个主题时，他们已经知道该主题在更低层如何实现。这种方法也为后续设计课程做好准备，因为理解基础并从中获得洞见，是进行必要设计取舍的关键。

学生调试自己的程序

我们经常听到业界抱怨计算机科学毕业生不会编程。问题的一部分在于热心的助教：他们为学生程序贡献了过多实质性思考，使学生始终不必真正掌握编程技能。我们的做法是推动学生在不依赖助教（TA）的情况下完成工作。这一方面来自自底向上的方法，它尽量减少记忆，让学生在已知基础上继续学习；另一方面来自模拟器，学生从编写第一个程序起就使用它。学生写第一个程序时就开始学习调试，并从一开始就必须借助模拟器的调试工具使程序正确运行。模拟器与教学内容的组织顺序相结合，使学生真正调试自己的程序，而不是拿着程序向 TA 求助——后一种做法太容易变成 TA 最后替学生把程序写好。

为未来做准备：穿透保护层

在今天的系统中使用计算机，却不了解底层发生了什么的专业人员，很可能会付出代价才发现：影响解决方案效果的，不只有自己实际编写的程序。这既适用于经验丰富的程序员，也适用于经验丰富的工程师。

认真从事编程的人，如果理解高级语言语句背后发生的事情，就能写出效率更高的代码。如今，工程师需要越来越多地在设备或引脚层面与计算机系统交互，而且并不只是计算机工程师如此。当计算机用于从气象测量设备或反馈控制系统等装置采集数据时，工程师所需的知识远不止 MATLAB 编程。今天的机械、化学和航空工程师同样如此，并非只有电气工程师才需要。因此，那种由编译器替学生挡住一切底层“丑陋”细节的高级语言课程，无法很好地满足大多数工科学生的需要，更无法为他们的未来做好准备。

对后续课程的连带影响

本书的内容显然会影响后续课程能够教授什么。后续编程课程不仅可以假定学生懂 C/C++ 的语法，还可以假定他们理解语法与底层体系结构之间的关系。因此，课程可以把重点放在问题求解和更复杂的数据结构上。硬件方面，数字逻辑设计和计算机组成课程也有类似效果。学生开始学习逻辑设计时，已经知道将要掌握的逻辑电路有什么用。在计算机组成课程中，起点也远高于学生第一次听到“程序计数器”这个词时的水平。教授后续课程的教师注意到，与学生尚未学习本课程时相比，他们的理解能力有了显著提升。

致谢

第二版出版至今已有 15 年，最初整理那套最终演变为第一版的课程讲义，距今也大约 20 年了。这些年，许多人主动参与，提供帮助，采用本书，阅读草稿，提出改进意见，等等。若列出所有人的名字，很容易写满许多页。我们感谢每一位帮助过我们的人，并由衷感激他们的贡献。

不过，有些人我们必须在这里特别致谢。对于那些给予巨大帮助，却因篇幅限制而未能提及的人，我们深表歉意。

首先是 Kevin Compton 教授。1993 年，我们在他主持的密歇根大学课程委员会会议上首次提出本书的理念，他从那时起就相信这一理念。1995 至 1997 年间，Kevin 共同讲授了密歇根大学最初四次开设的这门课程。他对编程方法的见解独立于具体语言的语法，为初学者提供了坚实基础。若没有 Kevin 的影响，密歇根大学的这门课程和本书都会逊色许多。

在寻求对本书当前版本及未来发展的建议和洞见时，有几位同事与学生是我们主要请教的对象。自第二版以来，Wen-mei Hwu、Veynu Narasiman、Steve Lumetta、Matt Frank、Faruk Guvinilir、Chirag Sakuja、Siavash Zanganeh、Stephen Pruett、Jim Goodman 和 Soner Onder 对我们尤为重要。

从第一位编辑 Betsy Jones 开始，McGraw-Hill 一直是这项工作的重要伙伴。我们曾把第一版书稿寄给几家出版社，并很高兴地收到许多积极回应。不过，有一位编辑格外突出。Betsy 立即对我们进行了了解；在确认之后，她坚定地支持我们想要实现的目标。在前两版的整个过程中，她的投入与热情，以及第一位开发编辑 Michelle Flomenhoft 的投入与热情，都令我们深为感激。两版之间相隔 15 年，确实很久，McGraw-Hill 的合作团队也已经焕然一新。最近，我们有幸与许多新同事合作，特别是 Thomas Scaife 博士、Suzy Bainbridge、Heather Ervolino 和 Jeni McAtee。

本书受益于许多大学教师提出的大量意见，有些来自正式书稿评审，有些来自电子邮件或技术会议中的交谈。我们衷心感谢以下教授的建议：

Jim Goodman, Wisconsin and Aukland; Soner Onder Michigan Tech; Vijay Pai, Purdue; Richard Johnson, Western New Mexico; Tore Larsen, Tromso; Greg Byrd, NC State; Walid Najjar, UC Riverside; Sean Joyce, Heidelberg College; James Boettler, South Carolina State; Steven Zeltmann, Arkansas; Mike McGregor, Alberta; David Lilja, Minnesota; Eric Thompson, Colorado, Denver; Brad Hutchings,

Brigham Young; Carl D. Crane III, Florida; Nat Davis, Virginia Tech; Renee Elio, University of Alberta; Kelly Flangan, BYU; George Friedman, UIUC; Franco Fummi, Universita di Verona; Dale Grit, Colorado State; Thor Guisrud, Stavanger College; Dave Kaeli, Northeastern; Rasool Kenarangui, UT, Arlington; Joel Kraft, Case Western Reserve; Wei-Ming Lin, UT, San Antonio; Roderick Loss, Montgomery College; Ron Meleshko, Grant MacEwan Community College; Andreas Moshovos, Toronto; Tom Murphy, The Citadel; Murali Narayanan, Kansas State; Carla Purdy, Cincinnati; T. N. Rajashekhara, Camden County College; Nello Scarabottolo, Universita degli Studi di Milano; Robert Schaefer, Daniel Webster College; Tage Stabell Kuloe, University of Tromsø; Jean-Pierre Steger, Burgdorf School of Engineering; Bill Sverdlik, Eastern Michigan; John Trono, St. Michael's College; Murali Varansi, University of South Florida; Montanez Wade, Tennessee State; Carl Wick, US Naval Academy; Robert Crisp, Arkansas; Allen Tannenbaum, Georgia Tech; Nickolas Jovanovic, Arkansas–Little Rock; Dean Brock, North Carolina–Asheville; Amar Raheja, Cal State–Pomona; Dayton Clark, Brooklyn College; William Yurcik, Illinois State; Jose Delgado-Frias, Washington State; Peter Drexel, Plymouth State; Mahmoud Manzoul, Jackson State; Dan Connors, Colorado; Massoud Ghyam, USC; John Gray, UMass–Dartmouth; John Hamilton, Auburn; Alan Rosenthal, Toronto; and Ron Taylor, Wright State.

我们一直有幸得到热情、知识丰富且关心学生的 TA 的帮助。他们不断向我们提出挑战，并提供有用的见解，帮助我们把内容解释得更清楚。名单同样很长，如今已超过 100 人。几乎所有人都很出色，不过我们仍想提到其中帮助特别大的几位：

Stephen Pruett, Siavash Zangeneh, Meiling Tang, Ali Fakhrzadehgan, Sabee Grewal, William Hoenig, Matthew Normyle, Ben Lin, Ameya Chaudhari, Nikhil Garg, Lauren Guckert, Jack Koenig, Saijel Mokashi, Sruti Nuthalapati, Faruk Guvenilir, Milad Hashemi, Aater Suleman, Chang Joo Lee, Bhargavi Narayanasetty, RJ Harden, Veynu Narasiman, Eiman Ebrahimi, Khubaib, Allison Korczynski, Pratyusha Nidamaluri, Christopher Wiley, Cameron Davison, Lisa de la Fuente, Phillip Duran, Jose Joao, Rustam Miftakhutdinov, Nady Obeid, Linda Bigelow, Jeremy Carillo, Aamir Hasan, Basit Sheik, Erik Johnson, Tsung-Wei Huang, Matthew Potok, Chun-Xun Lin, Jianxiong Gao, Danny Kim, and Iou-Jen Liu.

一些以前的学生、同事和朋友审阅了本书的章节。我们特别感谢 Rich Belgard、Alba Cristina de Melo、Chirag Sakhuja、Sabee Grewal、Pradip Bose 和 Carlos Villavieja 为此提供的帮助。他们的见解使内容更加易读。

我们很高兴地得到了得克萨斯大学奥斯汀分校和伊利诺伊大学厄巴纳-香槟分校同事们的热情回应。他们使用本书授课，并与我们分享见解：得克萨斯大学的 Derek Chiou、Jacob Abraham、Ramesh Yerraballi、Nina Telang 和 Aater Suleman；伊利诺伊大学的 Yuting Chen、Sayan Mitra、Volodymyr Kindratenko、Yih-Chun Hu、Seth Hutchinson、Steve Lumetta、Juan Jaramillo、Pratik Lahiri 和 Wen-mei Hwu。谢谢各位。

还有一些人以不同且往往独特的方式作出了贡献。限于篇幅，我们无法详述，只能列出姓名并道一声感谢：Amanda、Bryan 和 Carissa Hwu, Mateo Valero、Rich Belgard、Aman Aulakh、Minh Do、Milena Milenkovic、Steve Lumetta、Brian Evans、Jon Valvano、Susan Kornfield、Ed DeFranco、Evan Gsell、Tom Conte、Dave Nagle、Bruce Shriver、Bill Sayle、Dharma Agarwal、David Lilja 和 Michelle Chapman。

最后，请允许第一作者稍稍说一点个人感受：本书旨在帮助学生打牢基础。我们坚信，一旦基础建立起来，学生就能走到其才智和精力所能到达的地方。这一目标，

是教会我如何成为教授的 William K. Linvill 教授灌输给我的。距离我坐在他的课堂里已经超过 50 年，但他树立的榜样，我至今仍十分珍视。

最后的话

我们希望，你会喜欢使用本书第三版教学或学习。不过，正如前两版前言中所说，我们清楚地知道，本书的当前版本始终是一项尚在推进的工作。我们两人都欢迎你对任何方面提出意见。你可以通过电子邮件 patt@ece.utexas.edu 和 sjp@illinois.edu 联系我们。我们期待你的来信。

Yale N. Patt
Sanjay J. Patel
2019 年 9 月

第一章 欢迎启程

1.1 我们想做什么

欢迎阅读《从位与逻辑门到 C 及更远》。我们希望在接下来的 xxx 页中，带你走进计算的世界。在这一过程中，我们有一个高于其他一切的目标：让你清楚地看到，计算并没有什么魔法。计算机是一个确定性系统（deterministic system）：每次我们以相同方式、在相同位置“敲它一下”，只要起始状态相同，就会得到同样的响应。计算机不是电子天才；相反，硬要这样比喻的话，它倒像个电子笨蛋，只会严格按照我们的吩咐行事，并没有自己的主意。

看似极其复杂的系统，实际上不过是大量极其简单的部件按系统方式连接起来的集合。本书的任务，就是介绍这些简单部件，然后一步一步构建起你所熟知的“计算机”。就像盖房子，我们从底部开始，先打地基，再一层层往上建，逐渐接近大多数人印象中那台完整的计算机。每增加一层，我们都会解释在做什么，并把新概念与底层结构联系起来。我们的目标是：学完之后，你不仅能运用 C 这样的计算机语言中的复杂特性编写程序，还能理解计算机内部的底层究竟发生了什么。

1.2 我们将怎样达到目标

我们首先会在第 2 章说明：计算机处理的任何信息，都表示为由 0 和 1 组成的序列。也就是说，我们把所有信息编码成 0、1 序列。例如，字母 a 的一种常用编码是 01100001；十进制数 35 的一种编码是 00100011。我们将学习如何对这样编码的信息进行操作。

当我们熟悉了用 0、1 编码表示信息，以及对这些表示进行运算（例如加法）之后，就开始说明计算机如何工作。从第 3 章起，我们会看到，计算机是一种电子设备，由电压驱动、通过导线相互连接的电子部件组成。计算机中的每根导线，在任一时刻都处于高电压或低电压。为了表示 0 和 1，我们不必具体规定电压有多高；只关心相对于 0 V，它是否足够高，能被认作 1。也就是说，相对于 0 V，有没有一个足够大的电压，决定了它表示 0 还是 1。

第 3 章会介绍组成当今微处理器——现代计算机核心——的晶体管如何工作。接着，我们会看到晶体管如何组合成更大的结构：有些执行加法等运算，有些保存信息，供以后使用。第 4 章把这些更大的结构组合成冯·诺依曼机，这是一种描述计算机工作方式的基本模型。我们也会开始研究一台简单的计算机 LC-3，并在第 5 章继续学习。LC-3 是 Little Computer 3 的缩写。我们其实从 LC-1 开始，但又尝试了两次，才算把它做好——至少我们自己这样认为！LC-3 具备你可能听说过的微处理器的所有重要特征，例如 1981 年第一批 IBM PC 使用的 Intel 8088、1984 年 Macintosh 使用的 Motorola 68000、2003 年 PC 高性能微处理器的优选之一 Pentium IV，以及今天笔记本

和台式机采用的 Intel Core 处理器 I3、I5、I7，甚至如今大多数智能手机中的 ARM 微处理器。也就是说，LC-3 具备这些“真正”微处理器的重要特征，却没有复杂到妨碍你理解它。

理解 LC-3 如何工作后，下一步就是给它编程：先使用它自己的语言（第 5、6 章），再使用一种更便于人使用的语言——汇编语言（第 7 章）。第 8 章介绍比单个数字更复杂的信息表示，即栈、队列和字符串，并说明如何实现它们。第 9 章讨论如何把信息送入 LC-3（输入）和从中取出（输出），也讨论操作系统为计算机用户提供的服务。前半本书以第 10 章的一个大型例子结束：模拟计算器，也就是今天大多数智能手机上都有的一种应用。

后半本书（第 11 至第 20 章）把注意力转向高级编程概念，并通过 C 和 C++ 介绍这些概念。高级语言把底层硬件的细节抽象起来，使程序员能够更有效地开发复杂软件。C 和 C++ 尤其提供了丰富且便于程序员使用的构造，同时又足够贴近硬件，让我们可以考察代码如何经过转换，在底下各层执行。我们的目标是让你能使用这些语言的核心部分编写简短、简单的程序，同时理解代码要在底层硬件执行，需要经历哪些转换。

我们先讲 C 的基本主题，例如变量和运算符（第 12 章）、控制结构（第 13 章）和函数（第 14 章）。你会发现，它们都是前半本书概念的直接延伸。随后，第 15 至第 19 章介绍能让代码更强大的编程概念：测试与调试（第 15 章）、C 中的指针和数组（第 16 章）、递归（第 17 章）、C 中的输入与输出（第 18 章），以及 C 中的数据结构（第 19 章）。

第 20 章专门介绍 C++，我们将其视为 C 的演进。由于 C++ 最初被定义为 C 的超集，第 11 至第 19 章的许多概念都可以直接对应到 C++。我们会介绍 C++ 的一些核心思想，正是这些思想帮助它成为开发实际软件时最受欢迎的语言之一。第 20 章就是我们的 C++ 入门。

几乎在所有情况下，我们都尽量把 C 和 C++ 的高级构造与底层 LC-3 联系起来，使你理解：在 C 或 C++ 程序中使用某个构造时，你究竟要求计算机做什么。

1.3 贯穿全书的两个主题

有两个主题贯穿本书。过去，身为教授的我们曾把它们视为理所当然，以为每个人都认识到其价值，也经常向工程和计算机专业学生强调它们。然而，我们越来越清楚，从一开始就有必要把这两点明确说出来。因此，这里先把它们列出：(a) 抽象的概念；(b) 不要在头脑中割裂硬件与软件。这两个主题对于你成长为有能力的工程师或计算机科学家的价值，远远不止帮助你理解计算机怎样工作、怎样编程。

无论你从事数学、物理、任何工程领域还是商业，抽象都处于所学知识及其运用的核心。很难想到哪个知识领域不依赖抽象这一重要概念。

把硬件与软件割裂开来的错误观念，则与你继续学习和使用计算机直接相关。

下面依次讨论这两个主题。

1.3.1 抽象的概念

抽象的运用随处可见。我们坐进出租车，对司机说“请送我去机场”，就是在使用抽象。如果有必要，我们大概也能逐段给司机指路：“沿这条街开十个街区，然后左转。”到了那里，再说：“现在沿这条街开五个街区，然后右转。”如此继续。你知道细节，但直接让司机送你去机场显然快得多。

甚至“沿这条街开十个街区……”这句话，还能继续分解成如何使用油门、方向盘，以及如何留意其他车辆和行人等指示。

抽象（abstraction）是一种简化人与系统交互方式的技术：去掉那些对于有效交互并非必要的细节。抽象能力能显著提高效率，让我们在更高层次处理问题，关注关键方面，把组成它的各个概念暂时放在背景中。它使我们更有效地利用时间和思考能力，也使我们在一切实细都正常工作时，不必陷入细节。

不过，这背后有一个前提：一切细都正常工作。倘若并非如此呢？这时，要成功解决问题，抽象能力就必须与还原抽象的能力结合起来。有人称之为解构（deconstruct），也就是从抽象回到其组成部分的能力。

我们想到了两个故事。

第一个故事发生在很久以前，第一作者在盛夏最炎热的时候驾车穿越 Arizona。当时他住在 California 的 Palo Alto，那里几乎全年气候温和。他知道出发前应该把车送到修理工那里，请对方检查冷却系统。“冷却系统”就是这里的抽象。他没有掌握的是，适用于 Palo Alto 的冷却系统，并不一定能应付 Arizona 夏季沙漠的环境。结果，他在 Arizona 的 Deer Lodge——一个只有 3 名居民的地方——等了两天，等人把气缸垫运来。

第二个故事据说发生在发电技术的早期，真实性或许无从考证。General Electric 的一台大型发电机出了问题，大家不知怎么办。发电机正面有许多仪表，显示大量信息；还有许多螺钉，操作员可以顺时针或逆时针旋转。在这一整面仪表和螺钉的后面，某个部件出了故障，却没人知道如何处理。据说，他们请来了一位电力工业早期的泰斗。他看了看仪表，听了一分钟声音，然后从口袋里拿出小螺丝刀，把一颗螺钉逆时针转了 35 度，问题立刻消失。他提交了一张没有细目的 1000 美元账单，这在当时是一大笔钱。财务主管觉得两分钟工作收费这么多不太妥当，要求进一步说明。于是收到了一张新账单：

将一颗螺钉逆时针旋转 35 度：0.75 美元。

知道该转哪颗螺钉以及转多少：999.25 美元。

两个故事表达的是同一个意思：把事物作为抽象来处理更有效率，没有必要时，谁也不想陷入细节。只要没有意外，我们这样做就没有问题。如果没有去 Arizona，“冷却系统”这个抽象就足够了；如果发电机从不出故障，就不需要那位电力工程大师更深层的理解。

我们将会看到，现代计算机由晶体管组成。晶体管组合成逻辑“门”，这一抽象让我们可以用 0 和 1 思考，而不必考虑晶体管上的电压变化。逻辑电路则是对多个门的组合所作的进一步抽象。用门设计逻辑电路时，不去考虑每个门的内部细节，效率

会高得多。反复考虑这些细节只会拖慢设计过程。我们希望把门当作一个部件；但如果逻辑电路不能正常工作，查看门的内部结构，判断其工作方式是否造成了问题，往往很有帮助。

设计复杂的计算机应用程序时，无论是新的电子表格软件、文字处理系统还是计算机游戏，我们都希望把所用的每个部件当作抽象。如果在不必要的时候还去思考每个部件的细节，这种干扰很容易使整个工作永远做不完。但是，当部件组合起来出现问题时，仔细检查每个部件的细节，通常有助于找到原因。

抽象能力是最重要的技能。在我们看来，只要能保证一切有效工作，就应尽量保持较高的抽象层次。本书的方法就是不断提高抽象层次。我们用晶体管描述逻辑门；理解了逻辑门这一抽象后，就不再从晶体管的角度思考。接着用门构建更大的结构；理解了这些更大的抽象后，就不再从门的角度思考。

要点 抽象使我们处理各种情况时都更有效率。只要一切正常，即使不了解抽象之下是什么，我们也能有效工作。因此，不应轻视抽象；相反，我们应当重视它，因为它使我们更有效率。

事实上，如果永远不必把某个部件与其他东西组合成更大的系统，而且这个部件永远不会出问题，那么只在抽象层面理解它，完全没有关系。

但是，如果必须把多个部件组合成更大的系统，就要注意：不要让这些部件的抽象成为自己理解的最深层。如果不了解抽象之下的组成部分，我们就只能寄希望于它们无需干预也能协同工作。一旦它们无法协同，而我们又不能深入抽象之下，就会束手无策。我们应当避免陷入这种境地。

1.3.2 硬件与软件

许多计算机科学家和工程师自称“做硬件的”或“做软件的”。他们所说的硬件，通常是物理计算机以及与它有关的全部规范；软件则通常是程序，包括 Android、ChromeOS、Linux、Windows 等操作系统，Access、MongoDB、Oracle、DB-terrific 等数据库系统，以及 Facebook、Chrome、Excel、Word 等应用程序。这种说法暗示，一个人对其中一方面了解很多，对另一方面却所知甚少，而且往往进一步暗示：只精通硬件或软件中的一方面，对另一方一无所知，也没有关系。仿佛硬件——计算机及其实际工作方式——与软件——指挥计算机执行任务的程序——之间有一堵高墙，人只需安心待在墙的一侧即可。

抽象的力量使我们“通常”可以在某个层次工作，而不必时刻考虑底下各层。这是好事，能提高效率。但是，如果我们对底层一无所知，那么在必须利用底层细微特征的时候，也就无从利用。

这并不是说，你必须在较低抽象层次工作，放弃抽象带来的效率提升。恰恰相反，我们鼓励你在所能采用的最高抽象层次工作。不过，如果与此同时还能把底层放在心里，你会发现自己能把工作做得更好。

在学习和实践计算时，我们希望你这样看待硬件和软件：它们是计算系统两个部分的名称；只有设计者同时考虑两者的能力和局限，系统才能发挥最佳效果。

微处理器设计者如果理解将要在其设计上执行的程序的需求，就能设计出更有效

的微处理器。例如，几年前 Intel、AMD、ARM 等主要微处理器厂商认识到，未来很大一部分程序会包含视频片段，例如电子邮件、电子游戏和完整电影中的视频。他们意识到，必须让这些程序高效执行。因此，如今大多数微处理器都具有专门处理视频片段的硬件能力。Intel 定义了额外的指令，最初称为 MMX 指令集，并为其开发了专门硬件。Motorola、IBM 和 Apple 也做了基本相同的事，产生了 AltiVec 指令集及其硬件支持。

软件设计者也有类似情况。大型程序的设计者如果理解执行程序任务的硬件的能力和局限，就能比不了解硬件的人设计出运行效率更高的程序。几乎所有大型软件系统都要完成一项重要任务，叫作排序 (sorting)，即把一组项目按某种次序排列。词典中的单词按字母顺序排列；班级学生的成绩常根据期末考试成绩，按数值次序评定。要把一组项目排好序，可以编写许多原理上完全不同的程序。世界顶尖计算机科学家之一 Donald Knuth，在《计算机程序设计艺术》第 3 卷中用了 391 页讨论这项任务。哪一种排序程序最好，往往很大程度上取决于软件设计者对底层硬件特征了解多少。

要点 我们相信，无论你倾向于从事计算机硬件还是软件工作，掌握两者都会使你更有能力。本书就是要引导你迈上同时掌握硬件与软件的道路。深入分析某个概念时，我们有时不会明确重申这一点，但它们确实能够相互阐明。

学习 C 的数据类型这一软件概念时 (第 12 章)，你会理解计算机有限的字长这一硬件概念，如何影响我们对数据类型的认识。

学习 C 的函数时 (第 14 章)，你能把函数调用规则与使这些规则成为必要的硬件实现联系起来。

学习递归这一强大的算法手段时 (第 8 章初步介绍，第 17 章深入讨论)，你能把它与硬件联系起来。如果愿意花时间这样做，你就会更清楚地理解：何时值得付出递归执行一个过程所带来的额外时间。

学习 C 的指针变量时 (第 16 章)，存储器知识能帮助你更深入地理解指针提供了什么，尤其是何时应使用它、何时应避免使用它。

学习 C 的数据结构时 (第 19 章)，存储器知识能帮助你更好地理解：要高效操作内存中的实际结构，需要做些什么。

我们知道，前面五个短段落中的大多数术语，你现在可能还不熟悉。没有关系，学期结束时可以再读这一页。此刻需要知道的是，软件中的重要主题与硬件中的主题有着深刻联系。我们认为，同时关注两者，掌握其中任何一方都会更容易。

最重要的是，如果求解者同时具备硬件和软件能力，多数计算问题就能得到更好的解决。

1.4 计算机系统

前面几页已经使用了二十多次“计算机”这个词。虽然没有明确说明，我们指的是由软件与硬件组成的系统：软件，也就是计算机程序，指挥并规定信息的处理；硬件则响应软件的要求，实际处理信息。所谓“实际处理”，是指在硬件中真正执行完成任务所必需的加法、乘法等操作。对这种硬件，更准确的称呼是中央处理器 (central

processing unit, CPU), 也可简称处理器或微处理器。本书主要讨论处理器, 以及由处理器执行的程序。

1.4.1 一点历史, 带来更好的视角

在深入了解处理器及其软件如何工作之前, 我们应该稍作停留, 看看计算机问世以来这段相对短暂的时间里, 计算行业在性能上取得的巨大而前所未有的飞跃。毕竟, 最早的计算机直到 20 世纪 40 年代才出现。ENIAC (Electronic Numerical Integrator and Calculator) 就是其中之一, 它是一台可以通过重新编程来完成不同任务的通用电子计算机, 由 Presper Eckert 及其同事于 1943 至 1945 年在宾夕法尼亚大学设计并建造。它包含超过 17000 个真空管, 大约高 8 英尺、宽 100 多英尺、深 3 英尺, 占地约 300 平方英尺, 重 30 吨, 运行需要 140 kW 功率。图 1.1 展示了三位操作员通过插拔连接线和操作开关为 ENIAC 编程的情景。

大约 40 年后, 经历了许多计算机公司和计算机产品的发展, Burroughs A 系列于 20 世纪 80 年代初诞生。组成它的大约十几块 18 英寸电路板中的一块, 如图 1.2 所示。每块板包含 50 个或更多集成电路封装。它的占地由 300 平方英尺降到约 50 至 60 平方英尺, 重量由 30 吨降到约 1 吨, 运行功率则由 140 kW 降到约 25 kW。

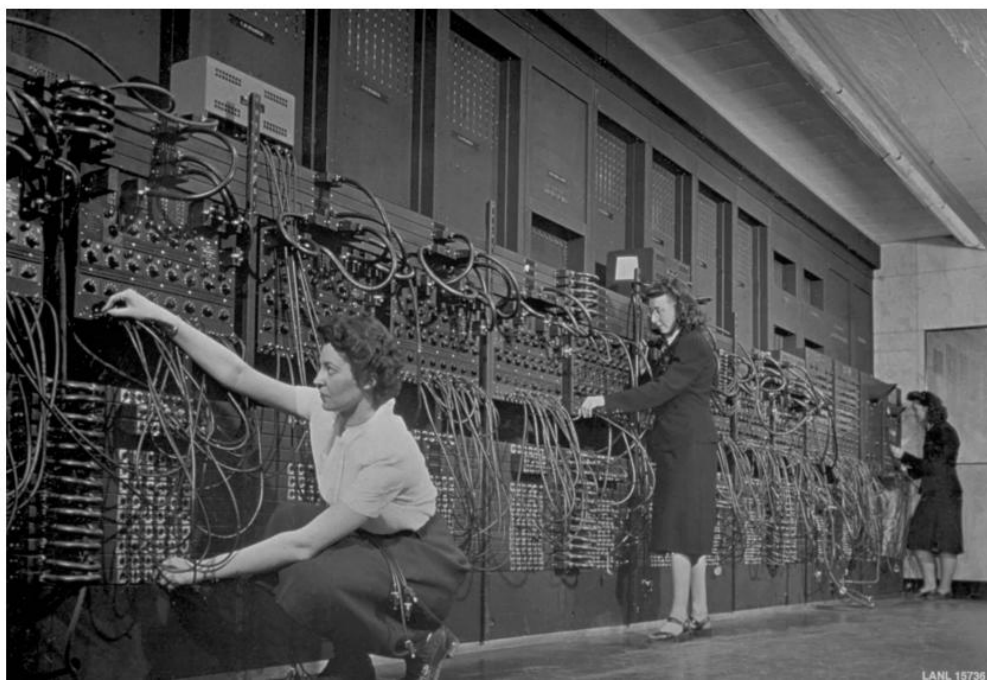


图 1.1 ENIAC, 于 1943—1945 年在宾夕法尼亚大学设计并建造。© Historical/Getty Images

再快进约 30 年, 我们看到, 今天的许多计算机已经出现在桌面上 (图 1.3)、笔记本电脑中 (图 1.4), 以及最近的智能手机中 (图 1.5)。它们的相对重量和能源需求大幅下降, 处理信息的速度则大幅提升。据我们估计, 今天一部智能手机的计算能力, 即用它计算有多快, 是 ENIAC 的四百多万倍!

构成现代数字计算机的集成电路封装, 也取得了惊人的进步。图 1.6 展示了如今的一种微处理器。第一款微处理器是 1971 年的 Intel 4004, 包含 2300 个晶体管, 工作

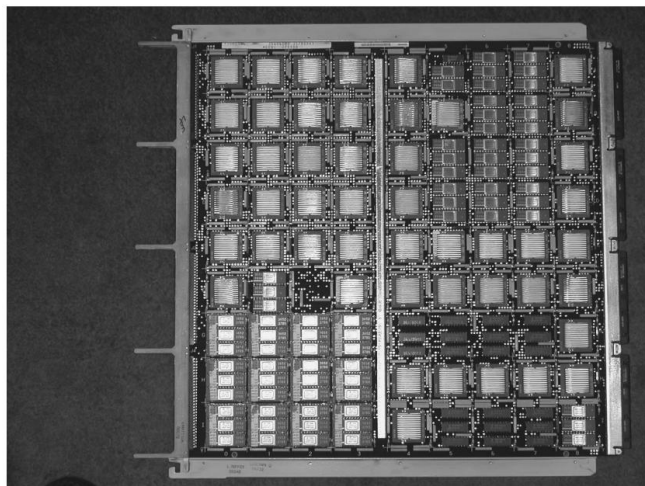


图 1.2 20 世纪 80 年代的一块处理器电路板。图片由 Emilio Salguerio 提供。



图 1.3 台式电脑 © Joby Sessions/Future/REX/Shutterstock



图 1.4 笔记本电脑 © Rob Monk/Future/REX/Shutterstock



图 1.5 智能手机 © Oleksiy Maksymenko/imageBROKER/REX/Shutterstock

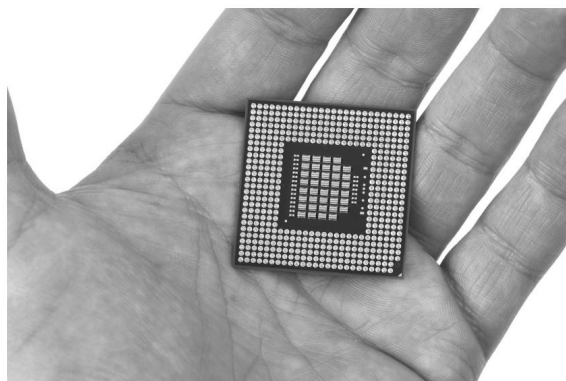


图 1.6 微处理器 © Peter Gudella/Shutterstock

频率为 106 kHz。到 1992 年，Intel Pentium 微处理器的这些数字已经达到 310 万个晶体管 and 66 MHz，两项参数都提高了约 1000 倍。如今的微处理器包含超过 50 亿个晶体管，工作频率可超过 4 GHz，两项参数又大约提高了 1000 倍。

自 1971 年以来，晶体管数量和微处理器工作频率都提高了一百万倍，这产生了重大影响。每次操作所用时间只有 1971 年的百万分之一，意味着如今的微处理器可以在当年完成一件事的时间里做一百万件事。单芯片上的晶体管数量增加到一百多万倍，则意味着今天可以同时完成比 1971 年多得多的事情。

所有这些带来的结果是：今天的计算机似乎能够理解人类语言，例如英语、西班牙语和汉语；也似乎能够识别人脸。许多人把这看作人工智能的魔法。随着深入了解计算机怎样工作，我们会发现，许多看似神奇的能力，实际上来自极其简单、毫无智能的操作被执行得惊人地快，而且许多操作可以同时进行。

1.4.2 计算机系统的组成部分

大多数人说“计算机”时，指的通常不只是负责执行软件指令的处理器，也就是 CPU。他们通常指的是共同组成计算机系统的一组部件。如今，这样的系统往往是一台笔记本电脑（见图 1.4），再加上许多附加设备。

计算机系统除了处理器，一般还包括：用于键入命令的键盘；用于定位菜单项的鼠标、小键盘或操纵杆；用于显示计算机系统产生的信息的显示器；用于暂时保存信息的存储器；即使关机后也能长期保存信息各种磁盘和 USB 存储设备；与其他设备的连接，例如连接打印机以取得这些信息的纸质副本；以及用户希望执行的程序集合，也就是软件。

这些部件都帮助用户完成工作。例如，没有打印机，用户就得手工抄下显示器上的内容；没有鼠标、小键盘或操纵杆，用户就得逐条键入命令，而不能只用这些设备定位选择。

因此，在开始这段旅程时，虽然我们的重点是占据不到 1 平方英寸硅片中很小一部分的 CPU，以及使 CPU 按我们意愿行事的软件，但也要注意：我们使用的计算机系统还包含许多其他部件。

1.5 两个非常重要的思想

在结束第一章之前，我们希望你理解两个非常重要的思想。它们处于计算的核心。

思想 1：只要给予足够的时间和足够的存储空间，所有计算机——无论最大还是最小、最快还是最慢、最贵还是最便宜——能够计算的东西都完全相同。也就是说，快计算机能做的事，慢计算机也能做，只不过做得更慢。只要较便宜的计算机能访问足够的存储空间，较昂贵的计算机就无法算出某种便宜计算机根本算不出的东西。为了不断扩大存储空间，你可能每次用完存储空间后都得去商店再买一些。所有计算机能做的事情完全相同，有些做得更快，但没有哪台能比另一台做更多种事情。

思想 2：我们用英语或其他人类语言描述问题，但实际解决问题的，却是计算机内部运动的电子。因此，必须把问题从人类语言转换成影响电子流动的电压。这种转换实际上是一系列系统化的转换，它们在过去 70 年间不断发展和改进，组合起来，让计算机能够完成看似十分复杂的任务。实际上，这些任务本身是简单而直接的。

本章剩余部分将讨论这两个思想。

1.6 作为通用计算设备的计算机

一本入门教材从介绍计算机如何工作开始，似乎有些奇怪。毕竟，机械工程学生先学物理，而不是汽车发动机如何工作；化学工程学生先学化学，而不是炼油厂。为什么学习计算的学生要从计算机开始呢？

答案是，计算机不同。要学习计算的基本原理，就必须研究计算机，或研究能做计算机所做事物的机器。原因与“计算机是通用计算设备”这一概念有关。下面看看这是什么意思。

现代计算机出现之前，已经存在许多种计算机器。其中一些是模拟机器（analog machine），通过测量距离、电压等物理量给出答案。例如，计算尺通过让两把按对数刻度（logarithmic scale）标记的尺子相对滑动来做乘法，用户可以从第二把尺上读出一个对数“距离”。有些早期模拟加法机则通过向秤上放砝码来工作。模拟机器的困难在于，精度很难提高。

正因如此，数字机器（digital machine）开始主导计算。数字机器通过操作一组固定且有限的数字或字母来完成计算。你熟悉指针式手表与数字手表的区别。指针式手表有时针、分针，也可能有秒针，通过指针的位置，也就是角度，显示时间；数字手表则用数字显示时间。只需增加数字位数，就能提高精度。例如，如果需要把时间测量到百分之一秒，就可以买显示 10:35.16 而不是仅显示 10:35 的手表。怎样让指针式手表精确显示到百分之一秒呢？可以做到，但恐怕得有一根很长的秒针！本书提到计算机时，始终指数字机器。

现代数字计算机出现之前，西方最常见的数字机器是加法机；世界其他地区则广泛使用另一种数字机器——算盘。数字加法机是机械或机电设备，只能完成特定种类的计算：整数加法。此外，也有能做整数乘法的数字机器，还有能把一叠打孔记录姓名的卡片按字母顺序排列的机器。这些机器的主要局限是只能执行某一种计算。如果

只有加法机却想把两个整数相乘，就还得用纸笔做些工作。

这就是计算机的不同之处。你可以告诉它如何相加、如何相乘，如何按字母顺序排列列表，或执行你想要的任何计算。想到一种新的计算时，不必购买或设计新计算机，只需给原来的计算机一组新指令，也就是一个程序，让它完成新的计算。因此，我们称计算机为通用计算设备（universal computational device）。计算机科学家相信，只要有足够的时间和存储空间，任何可计算的东西都能由计算机计算。研究计算机，就是研究一切计算的基础，学习什么是计算，以及什么能够被计算。

通用计算设备这一思想来自 Alan Turing。1937 年，Turing 提出，所有计算都可以由一种特定的机器完成，今天称为图灵机（Turing machine）。他给出了这种机器的数学描述，但没有实际建造它。数字计算机又过了几年才开始运行。Turing 更感兴趣的是解决一个哲学问题：如何定义计算。他先考察人们计算时进行的动作，例如在纸上做标记，以及在存在某些符号时按特定规则写出其他符号等。他把这些动作抽象出来，规定了一种能执行这些动作的机制，并举例说明这种机器能够做什么。一台图灵机可以把两个整数相加，另一台可以把两个整数相乘。

图 1.7 展示了能执行加法和乘法的图灵机的“黑盒”模型。每个盒子中描述了要执行的操作；参与操作的数据作为输入进入盒子，结果作为输出离开盒子。黑盒模型不提供操作具体如何执行的信息，事实上，把两个数相加或相乘都有许多实现方法。

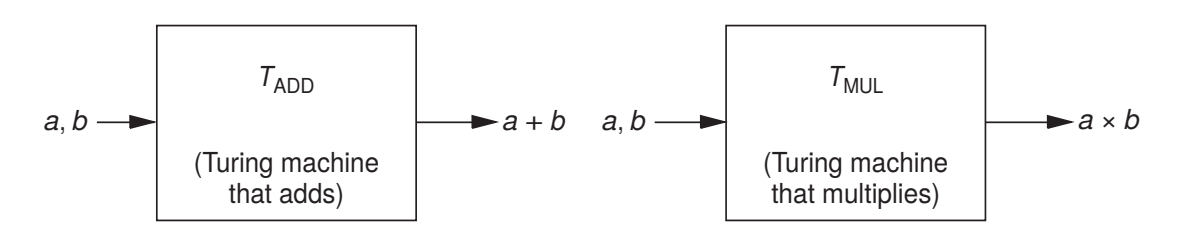


图 1.7 图灵机的黑盒模型

Turing 提出，每一种计算都能由某台图灵机完成。我们称之为图灵论题（Turing’s thesis）。尽管图灵论题从未得到证明，却有大量证据表明它是正确的。例如，我们知道，对图灵机作出的各种增强，并不会得到能计算更多东西的机器。

支持图灵论题的最好论据，或许是 Turing 本人在最初论文中给出的。他说，若要尝试构造一台比任意特定图灵机都更强大的机器，可以造一台能模拟所有图灵机的机器 U。只需向 U 描述你希望它模拟的那台图灵机，比如用于两个整数相加的机器，再给 U 输入数据，U 就会算出相应输出，在这个例子中就是输入之和。随后，Turing 证明，确实存在一台能做到这一点的图灵机。因此，即使这样的尝试，也没能找到图灵机无法计算的东西。

图 1.8 进一步说明了这一点。假设你想计算 $g \cdot (e + f)$ ，只需向 U 提供加法图灵机和乘法图灵机的描述，以及三个输入 e 、 f 、 g ，其余工作交给 U 即可。

通过规定 U，Turing 带给我们一个深刻认识：他第一次描述了计算机所做的事情。事实上，拥有所需全部存储空间的计算机，与通用图灵机（universal Turing machine），能够计算的东西完全相同。在两种情况下，你都向机器提供计算的描述及所需数据，机器则计算出相应答案。计算机和通用图灵机之所以能够计算一切可计算的东西，是因为它们可以编程。

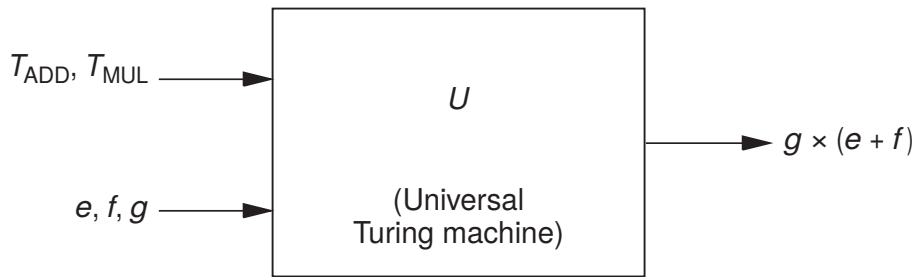


图 1.8 通用图灵机的黑盒模型

因此，大型或昂贵计算机并不能比小型、便宜计算机做更多种事情。多花钱也许能买到更快的计算机、分辨率更高的显示器，或更好的音响系统；但只要有一台小型、便宜的计算机，你就已经拥有一台通用计算设备。

1.7 怎样让电子来干活？

图 1.9 展示了要让真正干活的电子按我们的意愿行事，必须经过的过程。我们把这一过程的各个步骤称为“转换层次”（levels of transformation）。我们将看到，每个层次都有选择。如果忽略任何一个层次，就可能严重影响我们充分利用计算机系统的能力。

1.7.1 问题的表述

我们用“自然语言”（natural language）描述希望解决的问题。自然语言就是人们使用的语言，例如英语、法语、日语和意大利语等。它们在数百年使用中逐渐演变，包含许多不适合用来向计算机下达指令的特征。其中最重要的是歧义（ambiguity）。自然语言充满歧义。要推断一句话的含义，听者往往需要借助说话者的语气，或者至少依靠句子的上下文。

英语中的一个歧义例子是：“*Time flies like an arrow.*”这句话至少有三种解释，取决于情境是：（1）感叹时间过得多快；（2）参加昆虫田径比赛；（3）给昆虫村的 Dear Abby 咨询专栏写信。第一种是明喻，把时间流逝的速度比作射出的箭，即“光阴似箭”。第二种把 *time* 理解为“计时”的动词，*flies* 理解为“苍蝇”，是在告诉计时员：像一支箭那样给苍蝇计时。第三种则把 *time flies* 看作一种苍蝇的名称，与果蝇相对；*like* 表示“喜欢”，说的是这一群“时间蝇”都爱上了同一支箭。

给计算机的指令不能有这样的歧义。计算机这个电子笨蛋，只会照吩咐行事。如果一句指示有多种解释，它就不知道该遵从哪一种。

1.7.2 算法

这串转换的第一步，是把问题的自然语言描述变成算法，并在这一过程中消除自然语言中不合要求的特征。算法（algorithm）是一种逐步执行的过程，保证会终止，而且每一步都表述精确、能由计算机执行。这些性质各有对应术语。

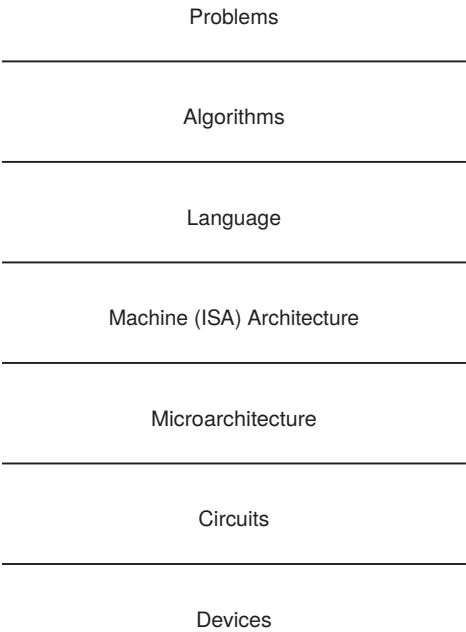


图 1.9 转换层次

我们用确定性（definiteness）表示每一步都表述精确。若一份制作美味薄煎饼的食谱要求“搅拌至有结块”，它就不具备确定性，因为“有结块”的程度并不精确。

我们用有效可计算性（effective computability）表示每一步都能由计算机执行。一个要求计算机“取出最大的素数”的过程不具备这一性质，因为根本不存在最大的素数。

我们用有穷性（finiteness）表示过程会终止。

对于每个问题，通常都有许多不同算法可以解决。某个算法可能需要的步骤最少，另一个则可能允许部分步骤并发执行。能够同时做不止一件事的计算机，往往可以在更短时间内解决问题，尽管需要执行的总步骤数可能增加。

1.7.3 程序

下一步，是用某种编程语言把算法转换成计算机程序。编程语言属于“机械语言”（mechanical language）。也就是说，它们不像自然语言那样在人类交流中演变，而是人为发明的，用来向计算机规定一串指令。因此，机械语言不会具有歧义这类使其无法用于规定计算机程序的缺陷。

编程语言有一千多种。有些为特定应用设计，例如 Fortran 用于科学计算，COBOL 用于商业数据处理。后半本书将使用 C 和 C++，这两种语言被设计用来操作底层硬件结构。

其他语言又适合其他用途。对于许多需要设计专家系统的应用，Prolog 是首选语言。多年来，LISP 一直是许多人工智能研究者的首选。Pascal 则是为教授初学者编程而发明的语言。

编程语言分为高级语言和低级语言。高级语言距离底层计算机较远，处于较高层次。理想情况下，它们独立于程序实际执行的计算机。我们称这种语言具有“机器无

关性”(machine independent)。

到目前为止提到的语言都是高级语言。低级语言则与程序执行的计算机紧密相关。通常,每种计算机都有一种这样的低级语言,称为该计算机的汇编语言。

1.7.4 ISA

下一步,是把程序翻译成承担程序工作的那台计算机的指令集。指令集体系结构(instruction set architecture, ISA)是程序与执行程序工作的底层计算机硬件之间接口的完整规范。

汽车可以帮助我们理解 ISA。对于计算机,程序表示为一串 0 和 1;在汽车类比中,对应的就是坐在驾驶座上的人。对应微处理器硬件的,则是汽车本身。汽车的“ISA”规定了驾驶者为了告诉汽车做什么必须知道的一切,以及汽车为了完成驾驶者指定的任务必须知道的一切。例如,地板上称为刹车的踏板及其功能,就是汽车“ISA”的一部分。驾驶者知道,踩下刹车,汽车就会停下;汽车则知道,当感受到驾驶者在踏板上施加压力时,其硬件必须启用停车所需的部件。汽车完整的“ISA”还包括其他踏板、方向盘、点火钥匙、挡位、雨刷等的规范。对每一项,它都规定:(a) 驾驶者要怎样操作,才能告诉汽车自己的意图;(b) 汽车又会怎样解释这些操作,从而完成指定任务。

计算机 ISA 的作用与汽车“ISA”相同,只不过双方从驾驶者和汽车,换成了指挥硬件的程序,以及执行指令的硬件。例如,考虑计算机能够执行的指令集合:它能做什么操作,又从哪里取得执行操作所需的数据。操作码(opcode)用于描述操作;操作数(operand)用于描述各个数据值。ISA 规定操作数可采用的表示,称为数据类型(data type)。数据类型是一种操作数表示,计算机能够对这种表示执行操作。ISA 还规定计算机确定操作数位置的机制,称为寻址方式(addressing mode)。

不同 ISA 规定的操作码、数据类型和寻址方式的数量各不相同。有些 ISA 只有约六个操作码,有些却多达数百个;有些只有一种数据类型,有些则有十几种;有些只有一两种寻址方式,有些则超过二十种。PC 使用的 x86 ISA 有两百多个操作码、十几种数据类型和二十多种寻址方式。

ISA 还规定了构成计算机存储器的不同存储单元数量,以及每个单元包含多少个 0 和 1。

如今有许多 ISA 在使用。最广为人知的是 x86,由 Intel 于 1979 年推出,现在 AMD 等公司也制造实现它的产品。其他 ISA 及其负责公司还包括 ARM 和 THUMB (ARM)、POWER 和 z/Architecture (IBM),以及 SPARC (Oracle)。

把高级语言(例如 C)翻译成程序所运行计算机的 ISA(例如 x86),通常由一种称为编译器(compiler)的翻译程序完成。要把 C 程序翻译成 x86 ISA,就需要一个 C 到 x86 的编译器。每一种高级语言与每一种目标 ISA 的组合,都需要相应的编译器。

从计算机专属的汇编语言到其 ISA 的翻译,则由汇编器(assembler)完成。

1.7.5 微体系结构

下一步是 ISA 的实现，称为微体系结构（microarchitecture）。前面用于解释 ISA 的汽车类比，也可以说明 ISA 与实现它的微体系结构之间的关系。汽车“ISA”描述驾驶者坐在车内时，要知道些什么，才能使汽车执行其意愿。所有汽车具有相同的 ISA。如果地板上有三个踏板，无论由哪家公司制造，中间那个总是刹车；右边那个总是油门，踩得越深，汽车就跑得越快。由于汽车只有一种 ISA，人们不必为 Buick 考一本驾照，再为 Honda 考另一本。

另一方面，汽车 ISA 的微体系结构，也就是实现，关心的是引擎盖下面发生了什么。这里，各种品牌和型号完全可以不同，取决于设计者在制造前如何权衡成本与性能。有些汽车使用盘式制动器，有些至少过去使用鼓式制动器；有些有八个气缸，有些有六个，还有些只有四个；有些带涡轮增压，有些没有。有些汽车一加仑汽油能跑 60 英里，有些能从一个加油站撑到下一个就算幸运。有些售价 6000 美元，有些则要 200000 美元。每种汽车的“微体系结构”，都是设计者对成本和性能作出取舍的结果。各品牌型号的“微体系结构”不同，也正是 Honda 出故障时应找 Honda 修理工，而不是 Buick 修理工的充分理由。

上一节列举了几家厂商的 ISA，包括 x86（Intel）、PowerPC（IBM 和 Motorola）以及 THUMB（ARM）。每种都已有许多不同的微体系结构实现。例如，x86 在 1979 年的最初实现是 8086，之后是 20 世纪 80 年代的 80286、80386 和 80486。再后来，Intel 于 2001 年推出 Pentium IV 微处理器，2015 年又推出 Skylake。这些 x86 微处理器各有自己的微体系结构。

PowerPC ISA 的情况也一样：它有十几种不同的微处理器，每一种都有自己的微体系结构。

每一种微体系结构都给计算机设计者一个机会，让他们在微处理器成本、所能提供的性能和供其运行所需的能量之间，作出不同取舍。计算机设计始终是在权衡：选择更高或更低性能、更多或更少的能量需求，以及更大或更小的成本。

1.7.6 逻辑电路

下一步是用简单逻辑电路实现微体系结构的每个组成部分。这里同样有选择，逻辑设计者必须决定怎样权衡成本和性能。例如，即使是加法这样简单的运算，也有多种逻辑电路可供选择，执行速度及相应成本各不相同。

1.7.7 器件

最后，每种基本逻辑电路都要依照所用器件工艺的要求来实现。因此，CMOS 电路不同于 NMOS 电路，而后者又不同于砷化镓电路。

要点 从问题的自然语言描述，到电子通过在不同电势之间移动来实际解决问题，中间需要许多转换。如果我们会说“电子语言”，或者电子听得懂英语，也许直接走到计算机面前，就能让电子照办。但我们不会说电子语言，电子也不会说英语，所以能做的就是使用这一系列系统化的转换。每一层都有不同的推进方式，而我们怎样选择，

决定了计算机最终的成本和性能。

本书将描述每一个转换：晶体管如何组合成逻辑电路，逻辑电路如何组合成微体系结构，微体系结构如何实现特定 ISA。本书采用的 ISA 是 LC-3。最后，我们从问题的英语描述出发，编写解决问题的 C 或 C++ 程序，并展示程序如何被翻译，也就是编译，成 LC-3 的 ISA，从而完成整个过程。

希望你享受这段旅程。

习题

- 1.1** [1] 解释第 1.5 节提出的两个重要思想中的第一个。
- 1.2** [1] 与较低级的编程语言相比,较高级的编程语言能否指示计算机计算更多种东西?
- 1.3** [1] 模拟计算机的什么困难促使计算机设计者采用数字设计?
- 1.4** [1] 指出自然语言的一项特征,说明它为什么不能用作编程语言。
- 1.5** [5] 假设有一个“黑盒”,输入两个数,输出它们的和,见图 1.10(a)。另一个盒子能把两个数相乘,见图 1.10(b)。把这些盒子连接起来,就能计算 $p \times (m + n)$, 见图 1.10(c)。假设这些盒子的数量无限,请说明如何连接它们以计算:
- $ax + b$ 。
 - 四个输入数 w 、 x 、 y 、 z 的平均值。
 - $a^2 + 2ab + b^2$ 。你能只用一个加法盒和一个乘法盒完成吗?
- 1.6** [1] 用自然语言写一句话,并给出两种不同的解释。
- 1.7** [3] 第 1.3.1 节讨论抽象时指出,只要“一切细节都正常工作”,就不必理解部件的组成;一旦出现问题,就必须能够解构这些部件,否则只能任由抽象摆布。在出租车的例子中,假设你并不了解部件,也就是根本不知道怎样去机场。借助抽象,你只告诉司机:“请送我去机场。”解释这种做法在什么情况下能提高效率,又在什么情况下可能造成十分糟糕的后果。

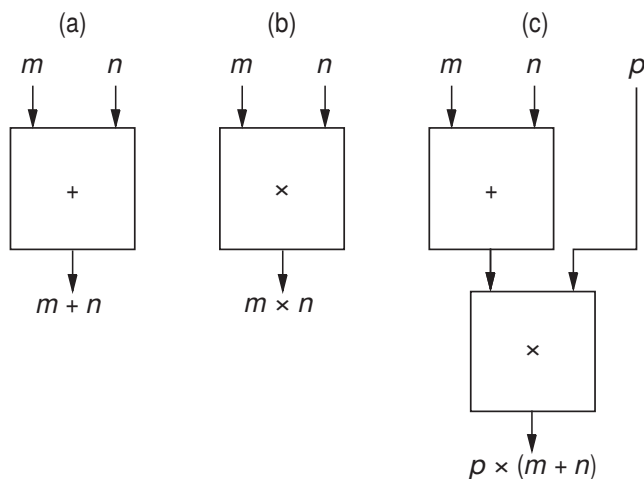


图 1.10 能够执行 (a) 加法、(b) 乘法, 以及 (c) 加法与乘法组合的“黑盒”

- 1.8** [5] John 说: “*I saw the man in the park with a telescope.*” 他是什么意思? 你能为这句话给出多少种合理解释? 请逐一列出。这句话表现出的什么性质, 使它不能作为程序中的语句?

1.9 [1] 自然语言能否表达算法？

1.10 [1] 列出算法的三个性质，并分别作简要解释。

1.11 [4] 对算法的每一个性质，举出一个不具备该性质、因此不是算法的过程。

1.12 [5] 以下 a 至 e 各项是算法吗？如果不是，它们缺少算法应有的哪些性质？

- a. 把下面矩阵的第一行加到另一行上，要求后者第一列的元素非零。（提示：列沿竖直方向，行沿水平方向。）

$$\begin{bmatrix} 1 & 2 & 0 & 4 \\ 0 & 3 & 2 & 4 \\ 2 & 3 & 10 & 22 \\ 12 & 4 & 3 & 4 \end{bmatrix}$$

- b. 为了说明素数与自然数一样多，按如下方式将每个素数与一个自然数配对：第一个素数与第一个自然数 1 配对，第二个素数与 2 配对，第三个与 3 配对，依此类推。如果最后每个素数和每个自然数都能一一配对，就证明素数与自然数一样多。
- c. 给定两个各有 20 个元素的向量，执行以下操作：取第一个向量的第一个元素，与第二个向量的第一个元素相乘；第二个元素也这样处理，依此类推。将各个乘积相加，得到点积。
- d. Lynne 和 Calvin 想决定由谁遛狗。Lynne 建议抛硬币，并从口袋里拿出一枚 25 美分硬币。Calvin 不信任 Lynne，怀疑硬币配重不均，即抛掷时可能更容易出现某种结果，于是提出以下过程，以公平地决定谁去遛狗：
1. 抛两次硬币。
 2. 若第一次正面、第二次反面，我去遛狗。
 3. 若第一次反面、第二次正面，你去遛狗。
 4. 若两次都是反面或两次都是正面，再抛两次。

Calvin 的方法是算法吗？

- e. 给定一个数，按顺序执行以下步骤：
1. 乘以 4。
 2. 加 4。
 3. 除以 2。
 4. 减 2。
 5. 除以 2。
 6. 减 1。
 7. 此时，将一个计数器加 1，以记录已经执行了一次步骤 1 至 6。然后检查减 1 后的结果。若为 0，写下执行步骤 1 至 6 的次数并停止；若不为 0，以减 1 后的结果为起点，再执行一遍这七个步骤。

- 1.13** [4] 两台计算机 A 和 B 完全相同，唯一的区别是 A 有减法指令，B 没有。两者都有加法指令，也都有能取一个值并产生其相反数的指令。A 与 B 哪一台能解决更多种问题？证明你的结论。
- 1.14** [4] 假设我们希望把一组姓名按字母顺序排列，这一操作称为排序。有一种能完成该任务的算法叫作冒泡排序。我们可以用 C 实现它，再将 C 程序编译到 x86 ISA 上执行，而 x86 ISA 可以由 Intel Pentium IV 微体系结构实现。把“冒泡排序、C 程序、x86 ISA、Pentium IV 微体系结构”这一序列称为一个转换过程。
- 假设有四种排序算法，可以使用 C、C++、Pascal、Fortran 和 COBOL 编程，并且有编译器能把每种语言翻译成 x86 或 SPARC。x86 有三种不同微体系结构，SPARC 也有三种。
- 一共有多少种转换过程？
 - 写出三个转换过程的例子。
 - 如果不是 x86 和 SPARC 各有三种微体系结构，而是 x86 有两种、SPARC 有四种，那么有多少种转换过程？
- 1.15** [7] 与较低级语言相比，使用较高级语言编程有什么优点？再指出一个缺点。
- 1.16** [1] 至少列出三项 ISA 所规定的内容。
- 1.17** [1] 简述 ISA 与微体系结构的区别。
- 1.18** [4] 一种微体系结构通常实现多少种 ISA？反过来，一种 ISA 可以有多少种微体系结构？
- 1.19** [1] 列出各个转换层次，并为每一层举一个例子。
- 1.20** [4] 图 1.9 中的转换层次也常被称为抽象层次。这样的描述合理吗？如果合理，请举例；如果不合理，请说明原因。
- 1.21** [7] 假设你去商店购买文字处理软件。软件实际上以什么形式存在？是高级编程语言形式、汇编语言形式，还是运行它的计算机的 ISA 形式？说明理由。
- 1.22** [5] 假设给你一项位于图 1.9 某一转换层次的任务，要求把它转换到紧接着的下一层。在哪一层完成这种转换最困难？为什么？
- 1.23** [5] 为什么一种 ISA 不太可能随着实现它的微体系结构更新换代而改变？例如，Intel 为什么希望确保 Pentium III 实现的 ISA 与 Pentium II 相同？提示：升级计算机或购买采用更新 CPU 的计算机时，是否必须扔掉所有旧软件？

第二章 位、数据类型与运算

2.1 位与数据类型

2.1.1 作为信息单位的位

第1章指出，计算机是一个包含多个转换层次的系统。一个用英语等自然语言描述的问题，最终实际上是由计算机各部件内部运动的电子来解决的。

计算机内部有数百万个极其微小、动作极快的器件，控制着这些电子的运动。这些器件对电路中电压的有无作出响应。它们也可以对电压的具体数值作出响应，而不只是区分有无电压，不过那样会使控制电路和检测电路变得不必要地复杂。检测电路中某一点是否存在电压，比精确测量该点的电压容易得多。

想一想家中的任意一个墙壁插座。你可以测量它所承载的确切电压，例如 120 V、115 V 或 118.6 V。不过，只判断有无电压，所需的检测电路就简单得多。例如，随意把手指插进墙壁插座就足够了。

我们用符号 1 表示存在电压，用符号 0 表示不存在电压。每个 0 或 1 都称为一个**位** (bit)，这个名称是 binary digit (二进制数字) 的缩写。回想从小就使用的数字 0、1、2、3、...、9，它们共有十个，称为十进制数字。二进制数字则只有两个：0 和 1。

更准确地说，计算机并不是区分电压的绝对不存在 (0) 和绝对存在 (1)，而是区分接近 0 的电压与明显偏离 0 的电压。例如，假设计算机预期的电压是 1.2 V 或 0 V，分别表示 1 和 0，那么 1.0 V 会被当作 1，0.2 V 会被当作 0。

一根导线只能区分两种情况，可以分别赋值为 0 和 1。要让计算机完成有用的工作，就必须能区分大量不同的值，并为每个值指定唯一的表示。把许多导线，也就是许多位，组合起来，就能做到这一点。例如，使用八位 (对应八根导线上各自的电压)，可以用 01001110 表示一个值，用 11100111 表示另一个值。八位最多能区分 256，即 2^8 个不同的事物。一般而言， k 位最多能区分 2^k 个不同的项目。这 k 位的每一种**位模式** (bit pattern) 都是一个编码，对应某个特定的项目或值。

2.1.2 数据类型

同一个值可以有多种表示方法。例如，数值“五”可以写成 5，这是你熟悉的十进制记法。也可以伸出一只手，张开全部五个手指，意思是：“数一数我伸出的手指，就知道我要表达的数了。”把这种方法写下来，就是 11111，称为一进制 (unary) 记法。罗马人还有另一种表示五的方法：字符 V。稍后我们会看到第四种记法，即二进制表示 00000101。

仅仅表示值还不够，我们还必须能够对这些值进行运算。如果计算机提供了可以对某种表示所编码的信息进行操作的运算，我们就称这种表示为一种**数据类型** (data

type)。每一种指令集体系结构 (ISA) 都有自己的一组数据类型, 以及能对这些类型进行操作的指令。本书主要使用两种数据类型: 用**补码整数** (2's complement integers) 表示需要进行算术运算的正整数和负整数; 用 ASCII 码表示需要通过键盘输入计算机或通过显示器输出的字符。下面很快就会介绍这两种类型。

信息还有其他表示方式, 事实上, 大多数计算机都支持其中的一些。还记得中学化学课上的科学记数法吗? 老师要求把十进制数 621 写成 6.21×10^2 。有些计算机以这种形式表示数, 并提供相应的运算。这种数据类型通常称为**浮点数** (floating point)。第 2.7.1 节将介绍它的表示。

2.2 整数数据类型

2.2.1 无符号整数

首先考察**无符号整数** (unsigned integer)。顾名思义, 无符号整数没有正号或负号, 只有大小。它在计算机中有许多用途。如果需要把一个任务执行指定次数, 只要统计已经执行的次数, 就可以方便地跟踪进度。无符号整数还可以标识不同的存储器位置, 就像门牌号把 Main Street 129 号和 131 号区分开一样。我可不得记得见过前面带负号的门牌号。

我们可以用一串二进制数字表示无符号整数, 采用的**位值记数法** (positional notation) 与从小使用的十进制系统很相似。以十进制数 329 为例, 虽然单独的 3 只有单独的 9 的三分之一大, 但这里的 3 所代表的值远大于 9。这是因为 3 所在的位置使它代表 $3 \times 10^2 = 300$, 而 9 代表 9×10^0 。

把十进制数字换成 0 和 1, 就可以表示二进制无符号整数。此时基数是 2, 而不是 10。例如, 若有五位可用, 前面提到的数值 5 就表示为 00101, 对应

$$0 \times 2^4 + 0 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0.$$

采用这种位值记法, k 位恰好可以表示 2^k 个整数, 范围是 0 到 $2^k - 1$ 。图 2.1 列出了 0 到 31 的五位表示。

2.2.2 有符号整数

要完成有用的算术运算, 往往还需要处理负数, 虽然并非所有情形都如此。可以把 k 位的 2^k 种不同模式分成两半, 一半用于正整数, 另一半用于负整数。这样, 五位编码不再表示 0 到 +31, 而是表示 +1 到 +15 和 -1 到 -15。这些数一共 30 个, 而 $2^5 = 32$, 还剩两个编码没有分配。自然可以把 00000 分配给 0, 这样就覆盖了 -15 到 +15 的所有整数。剩下的一个编码有不同的分配方法, 稍后就会看到。

但仍然需要决定: 哪个编码对应哪个值? 我们有 32 个编码, 应该怎样分配?

正整数采用直接的位值记法。我们有 k 位, 并希望恰好用一半编码表示正数和零, 因此把最左边的一位设为 0, 其余 $k - 1$ 位按前面的方法表示数值。于是, 五位的正整数表示为 00001 到 01111。

负整数的编码就没有这么直接了。一种办法是让最左边的位表示符号：0 表示正，1 表示负，其余各位表示大小。例如，+5 是 00101，-5 是 10101。这种表示称为**原码**（signed-magnitude）。

另一种办法是把正数表示中的每一位都反转，用所得编码表示相应的负数。这样，+5 为 00101，-5 为 11010。这种表示称为**反码**（1's complement），CDC 6600 等计算机就使用过它。

还可以任意分配负整数的编码，只要不同值使用不同编码即可。不过，计算机不仅要表示数，还要对数进行运算。选择表示方法时，应当考虑相应运算是否容易实现。事实上，原码和反码都需要相当繁复的加法硬件。计算机设计者了解实现整数加法的电路需要什么，因此选择了能够简化电路的表示，结果就是**补码**（2's complement）。今天制造的计算机几乎都采用它。图 2.1 对比了这几种表示。

位模式	无符号整数	原码	反码	补码
00000	0	0	0	0
00001	1	1	1	1
00010	2	2	2	2
00011	3	3	3	3
00100	4	4	4	4
00101	5	5	5	5
00110	6	6	6	6
00111	7	7	7	7
01000	8	8	8	8
01001	9	9	9	9
01010	10	10	10	10
01011	11	11	11	11
01100	12	12	12	12
01101	13	13	13	13
01110	14	14	14	14
01111	15	15	15	15
10000	16	-0	-15	-16
10001	17	-1	-14	-15
10010	18	-2	-13	-14
10011	19	-3	-12	-13
10100	20	-4	-11	-12
10101	21	-5	-10	-11
10110	22	-6	-9	-10
10111	23	-7	-8	-9
11000	24	-8	-7	-8
11001	25	-9	-6	-7
11010	26	-10	-5	-6
11011	27	-11	-4	-5
11100	28	-12	-3	-4
11101	29	-13	-2	-3
11110	30	-14	-1	-2
11111	31	-15	-0	-1

图 2.1 整数的四种表示

2.3 补码整数

补码的正整数表示与刚才相同。五位编码中恰好用一半表示 0 和 1 到 $2^4 - 1$ 的正整数。选择负整数编码的目标，仍然是使逻辑电路尽可能简单。

几乎所有计算机都使用同一种基本机制执行加法，称为算术逻辑单元 (ALU)。第 3、4 章会介绍它的实际结构。现在只需知道，它接收两个输入位模式，产生一个输出位模式，即两输入位模式相加的结果。它并不知道，也不关心这些位“意味着”什么，只是按照规定的规则进行运算。因此，我们希望整数的编码分配能让这种只管相加的硬件得到正确结果。

例如，把 00110 和 00101 相加，得到 01011，正好对应 $6 + 5 = 11$ 。计算从最右端开始，逐位相加，并把进位送到左边相邻的一位。我们希望负数的编码也能直接使用这套规则。

任意一个非零整数 A 与同样大小、符号相反的 $-A$ 相加，都应当等于 0。因此，既然 +5 的编码是 00101，就希望 -5 的编码与它相加得到 00000。11011 正好满足这个要求。更重要的是，从 -15 到 +15 依次查看编码，每个编码加上 00001 都应得到下一个编码。数学上可以写成

$$\begin{aligned} \text{REPRESENTATION}(\text{value} + 1) &= \text{REPRESENTATION}(\text{value}) \\ &+ \text{REPRESENTATION}(1). \end{aligned}$$

只要结果不大于 +15、不小于 -15，这个性质就足以保证二进制 ALU 正确执行加法。

由此可以确定 -1 的编码是 11111，因为

$$\begin{array}{r} 11111 \\ + 00001 \\ \hline 00000 \end{array}$$

这里忽略了最左端向外产生的进位。这是补码运算的一条一般规则：超出所用位数的进位不保留。

求一个非零数 A 的相反数的补码表示，可以分两步进行：先把 A 的每一位取反，再加 1。原因是， A 与逐位取反后的模式相加得到 11111，再加 00001 就得到 00000。这个过程也就是对位模式**取补码**。

例 2.1 如何得到 -13 的五位补码表示？先表示 $A = +13$ ：由于 $13 = 8 + 4 + 1$ ，其编码是 01101。逐位取反得到 10010，再加 1，得到 10011。检验如下：

$$\begin{array}{r} 01101 \\ + 10011 \\ \hline 00000 \end{array}$$

忽略最左端的进位后，结果确实是 0。

实际上，01101 与 10011 相加会从五位 ALU 的最左端产生一个进位；完整的六位结果是 100000。这个进位不属于五位结果，所以被忽略。

现在已经分配了 15 个正整数、15 个负整数以及 0，共 31 个值，只剩下 10000。从 $-1 = 11111$ 开始倒着数： $-2 = 11110$ ， $-3 = 11101$ ，一直到 $-15 = 10001$ 。再减 1 就得到 10000，所以把它分配给 -16 。因此，五位补码的范围是 -16 到 $+15$ 。LC-3 使用 16 位补码整数，范围是 -32768 到 $+32767$ 。

2.4 二进制与十进制之间的转换

2.4.1 二进制转换为十进制

用八位补码举例，它能表示 -128 到 $+127$ 。把八位记为 $b_7b_6b_5b_4b_3b_2b_1b_0$ ，其中 b_7 是符号位。如果符号位为 0，数为非负；把各位乘以相应的 2 的幂，再相加即可。如果符号位为 1，先取补码得到其大小，再按同样的方法转换，最后加上负号。

对于正数，各位的贡献为¹

$$b_62^6 + b_52^5 + b_42^4 + b_32^3 + b_22^2 + b_12^1 + b_02^0.$$

例 2.2 把 11000111 转换为十进制。最左边为 1，所以这是负数。取反得 00111000，加 1 得 00111001，其大小为

$$\begin{aligned} 0 \times 2^6 + 1 \times 2^5 + 1 \times 2^4 + 1 \times 2^3 + 0 \times 2^2 + 0 \times 2 + 1 \\ = 32 + 16 + 8 + 1 = 57. \end{aligned}$$

因此，原数为 -57 。

2.4.2 十进制转换为二进制

先考虑正整数。若二进制数的最低位为 0，这个数就是偶数；最低位为 1，则为奇数。因为除最低位以外，各位的权值都是 2 的倍数，所以最低位决定奇偶性。利用这一点，就可以从右向左逐位求出二进制表示。

以 105 为例。它是奇数，所以 $b_0 = 1$ 。从 105 中减去 1 得 104，再除以 2 得 52。这相当于把原来各位的权值都降低一位：

$$\begin{aligned} 105 &= b_62^6 + b_52^5 + b_42^4 + b_32^3 + b_22^2 + b_12 + b_0, \\ 104 &= b_62^6 + b_52^5 + b_42^4 + b_32^3 + b_22^2 + b_12, \\ 52 &= b_62^5 + b_52^4 + b_42^3 + b_32^2 + b_22 + b_1. \end{aligned}$$

52 是偶数，因此没有乘以大于 1 的 2 的幂的那一项 b_1 必须为 0。重复“减去最右位，

¹译注：原文的这套写法没有单独处理最小补码值。八位的 10000000 表示 -128 ，同样用八位取补码后仍为 10000000；此时应把所得模式按无符号大小 128 解释，不能只计算低七位。

再除以 2，检查新值奇偶性”的过程，依次得到

$$\begin{aligned} 26 &= b_6 2^4 + b_5 2^3 + b_4 2^2 + b_3 2 + b_2, & b_2 &= 0, \\ 13 &= b_6 2^3 + b_5 2^2 + b_4 2 + b_3, & b_3 &= 1, \\ 6 &= b_6 2^2 + b_5 2 + b_4, & b_4 &= 0, \\ 3 &= b_6 2 + b_5, & b_5 &= 1, \\ 1 &= b_6 2^0, & b_6 &= 1. \end{aligned}$$

加上符号位 0，结果为 01101001。

归纳这个过程：从待转换整数的大小 N 开始，若 N 为奇数，写下 1，否则写下 0；从 N 中减去刚写下的位值，再除以 2。反复进行，直到 $N = 0$ 。写下的位依次是从最低位到最高位。若原数为正，最左端补上符号位 0；若原数为负，先在其大小的表示前补 0，再取补码。

2.4.3 推广到带小数部分的数

位值记法也可以延伸到小数点右边。二进制小数点右侧各位的权值依次为 2^{-1} 、 2^{-2} 、 2^{-3} 、 2^{-4} ，也就是 $1/2$ 、 $1/4$ 、 $1/8$ 、 $1/16$ 。因此

$$(0.1011)_2 = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4} = 0.6875.$$

反过来，把十进制小数转换为二进制，可以反复乘以 2，并取出所得结果的整数部分。以 0.421 为例，设

$$0.421 = b_{-1} 2^{-1} + b_{-2} 2^{-2} + b_{-3} 2^{-3} + b_{-4} 2^{-4} + \dots$$

转换整数时，通过除以 2、检查奇偶性来确定 2^0 的系数；转换小数时，则乘以 2，根据左侧是否大于等于 1 来确定 2^0 的系数。你能看出为什么吗？具体过程为

$$\begin{aligned} 0.842 &= b_{-1} 2^0 + b_{-2} 2^{-1} + b_{-3} 2^{-2} + b_{-4} 2^{-3} + \dots, \\ 1.684 &= b_{-2} 2^0 + b_{-3} 2^{-1} + b_{-4} 2^{-2} + \dots, \\ 0.684 &= b_{-3} 2^{-1} + b_{-4} 2^{-2} + \dots, \\ 1.368 &= b_{-3} 2^0 + b_{-4} 2^{-1} + \dots, \\ 0.368 &= b_{-4} 2^{-1} + \dots, \\ 0.736 &= b_{-4} 2^0 + \dots. \end{aligned}$$

0.842 的整数部分为 0，因此 $b_{-1} = 0$ 。继续乘以 2 得 1.684，所以 $b_{-2} = 1$ ；减去 1 留下 0.684。再乘以 2 得 1.368，所以 $b_{-3} = 1$ ；减去 1 留下 0.368。再乘以 2 得 0.736，所以 $b_{-4} = 0$ 。

这个过程可以一直继续，直到累得不想再算，或者剩余部分变成 0；后一种情况下，后续各位全为 0。只保留四位小数时，得到 0.0110。它是原数的四位二进制小数近似值。

2.5 位运算——第一部分：算术运算

2.5.1 加法与减法

二进制加法与十进制加法类似，都从右向左逐位相加。区别在于，二进制只有 0 和 1，两个 1 相加就需要向左进位： $1 + 1 = 10$ 。

例 2.3 计算 $11 + 3$ 。11 的五位补码是 01011，3 是 00011，因此

$$\begin{array}{r} 01011 \\ + 00011 \\ \hline 01110 \end{array}$$

结果 01110 就是十进制的 14。

减去一个数，可以通过加上它的相反数来实现。因此，先对减数取补码，再相加即可。

例 2.4 计算 $14 - 9$ 。14 为 01110，9 为 01001。把 9 取补码，得到 -9 的表示 10111，于是

$$\begin{array}{r} 01110 \\ + 10111 \\ \hline 00101 \end{array}$$

丢弃最高位之外的进位，结果为 00101，即 5。

例 2.5 把一个数与它自身相加，相当于把它乘以 2。在结果仍能用现有位数表示的前提下，这也相当于把各位向左移动一位，并在最右端补 0。例如，59 的八位表示为 00111011，左移一位得到 01110110，即 118。

这是因为

$$2(b_62^6 + b_52^5 + \cdots + b_12 + b_0) = b_62^7 + b_52^6 + \cdots + b_12^2 + b_02.$$

每一位的权值都增加一倍，也就是移到左边相邻的位置。

2.5.2 符号扩展

同一个数可以用不同的位数表示。例如，5 的六位表示是 000101，16 位表示是 0000000000000101。在左端增加 0 不改变正数的值，就像支票上的 \$456.78 与 \$0000456.78 表示同一金额一样。

负数怎么办？-5 的六位补码是 111011，16 位补码是 1111111111111011。这里需要在左边补 1，而不是 0。

例如，要把用 16 位表示的 13 (0000000000001101) 与用六位表示的 -5 (111011) 相加，必须先使位数相同。如果在后者左边补 0，它就变成了 59，相加得到 72；如果补 1，则得到正确结果 0000000000001000，即 8。

这种在最左端复制符号位、增加位数而保持数值不变的操作，称为**符号扩展**(sign-extension)，常记作 SEXT。正数扩展 0，负数扩展 1。

2.5.3 溢出

表示数的位数有限，能表示的范围也有限。如果运算的数学结果超出这个范围，就发生了**溢出**（overflow）。

想象一辆汽车只有五位十进制里程表。在显示 99992 时再行驶 100 英里，里程表显示的会是 00092，而不是 100092，因为它最多只能显示 99999。产生的进位没有地方保存。我在波士顿开车时，有时倒能从车身上的锈迹看出那个丢失的进位！较新的汽车使用六位里程表，要接近一百万英里才会绕回零。

里程表就是无符号算术的例子：增加的里程总是正数。例如，显示 000129 时再行驶 50 英里，就显示 000179。溢出就是从最高位产生了向外的进位。

补码溢出的判断不同。例如，用五位计算 $9 + 11$ ：

$$\begin{array}{r} 01001 \\ + 01011 \\ \hline 10100 \end{array}$$

两个正数相加，结果却被解释成负数。这是因为真实结果 20 超过了五位补码的最大正数 15。类似地，-12 的表示 10100 与 -6 的表示 11010 相加，得到 01110，看上去成了正数。真实结果 -18 小于五位补码的最小值 -16，也发生了溢出。

两个异号的补码整数相加不会溢出。为什么？习题 2.25 请你解释这一点。

2.6 位运算——第二部分：逻辑运算

2.6.1 逻辑变量

除了把一组位看作一个数，还可以把每一位看作一个**逻辑变量**（logical variable）。逻辑变量有两个值：0 和 1，通常分别解释为“假”和“真”。逻辑运算的名称来自这种用法，不过它们在计算机中的用途远不止判断真假。大多数 ALU 都提供一些基本逻辑运算。

2.6.2 AND 运算

AND（与）运算有两个输入，只有两个输入都为 1 时，输出才为 1。可以用**真值表**（truth table）列出每种输入组合及其输出。对于 n 个输入，真值表有 $n + 1$ 列和 2^n 行：前 n 列为输入，最后一列为输出。两输入 AND 的表如下。

A	B	$A \text{ AND } B$
0	0	0
0	1	0
1	0	0
1	1	1

对两个多位模式进行 AND 运算时，每一对对应位独立运算。

例 2.6 设 $a = 0011101001101001$, $b = 0101100100100001$, 则

$$\begin{array}{r} a \quad 0011101001101001 \\ b \quad 0101100100100001 \\ \hline a \text{ AND } b \quad 0001100000100001 \end{array}$$

例如, $a_0 = b_0 = 1$, 所以结果的第 0 位为 1; $a_6 = 1$ 而 $b_6 = 0$, 所以结果的第 6 位为 0。

例 2.7 AND 可以用来提取一个位模式中的特定位。若只想保留八位模式 A 的最低两位, 可以让它与 00000011 做 AND。这个用于挑选各位的模式称为**掩码** (mask)。例如, 01010110 与它做 AND 得到 00000010; 11111100 与它做 AND 得到 00000000。无论 A 是什么, 结果只能为 0、1、2 或 3。

2.6.3 OR 运算

OR (或) 运算只要至少一个输入为 1, 输出就为 1。

A	B	$A \text{ OR } B$
0	0	0
0	1	1
1	0	1
1	1	1

例 2.8 对例 2.6 中相同的两个模式做 OR:

$$\begin{array}{r} a \quad 0011101001101001 \\ b \quad 0101100100100001 \\ \hline a \text{ OR } b \quad 0111101101101001 \end{array}$$

$a_0 = b_0 = 1$, 因此结果的第 0 位为 1; $a_6 = 1$ 、 $b_6 = 0$, 结果的第 6 位仍为 1。

这种 OR 也称为包含性的或 (inclusive-OR), 因为两个输入同时为 1 时, 它也输出 1。稍后会看到另一种“或”。

2.6.4 NOT 运算

NOT (非) 只有一个输入, 称为一元运算。它把 0 变成 1, 把 1 变成 0, 因此也称取反或反相。

A	NOT A
0	1
1	0

例如, 0011101001101001 的 NOT 为 1100010110010110。

2.6.5 XOR 运算

XOR（异或）在且仅在两个输入不同时输出 1。其真值表为

A	B	$A \text{ XOR } B$
0	0	0
0	1	1
1	0	1
1	1	0

例 2.9 还是前面两个模式：

$$\begin{array}{r}
 a \quad 0011101001101001 \\
 b \quad 0101100100100001 \\
 \hline
 a \text{ XOR } b \quad 0110001101001000
 \end{array}$$

XOR 的全称是 exclusive-OR，意思是排他性的或。它与 OR 的区别在于：当两个输入都为 1 时，XOR 输出 0，而 OR 输出 1。

例 2.10 假设要判断两个位模式是否完全相同。XOR 只有在对应的两位相同时才输出 0，因此，两个位模式逐位 XOR 的结果全为 0，当且仅当它们完全相同。

2.6.6 德摩根定律

AND 与 OR 之间有两个著名关系，称为**德摩根定律** (DeMorgan's laws)。图 2.2 展示其中一个：(a) 先将 A 、 B 各自取反，再送入 AND，最后把结果取反；(b) 展示 $A = 0, B = 1$ 的情况；(c) 用真值表列出四种输入组合的行为。注意， A 的 NOT 记为 $\bar{A}$ 。这个关系可以写成

$$\text{NOT}((\text{NOT } A) \text{ AND } (\text{NOT } B)) = A \text{ OR } B.$$

也就是 $\overline{\bar{A} \text{ AND } \bar{B}} = A \text{ OR } B$ 。用日常语言说，“并非两者都为假”等价于“至少有一个为真”。如果将 OR 的两个输入取反，再将输出取反，会得到类似的关系吗？

2.6.7 位向量

前面已经对两个 m 位模式进行逐位 AND、OR，以及对一个模式进行 NOT，各位单独、独立地处理。还使用了 m 位掩码，通过为各位选择 0 或 1，保留感兴趣的位，忽略其他位。

一组位中每一位的逻辑值独立于其他位，这种表示称为**位向量** (bit vector)。它便于表示某种性质的有无：有些位表示具有该性质，另一些位表示不具有。

位向量最常见的用途就是位掩码。例 2.7 只关心第 1、0 位，不关心第 7 到第 2 位；与 00000011 做 AND，结果只能是 00000000、00000001、00000010 或 00000011。这里掩码各位所表达的性质是“我们是否关心这一位”。

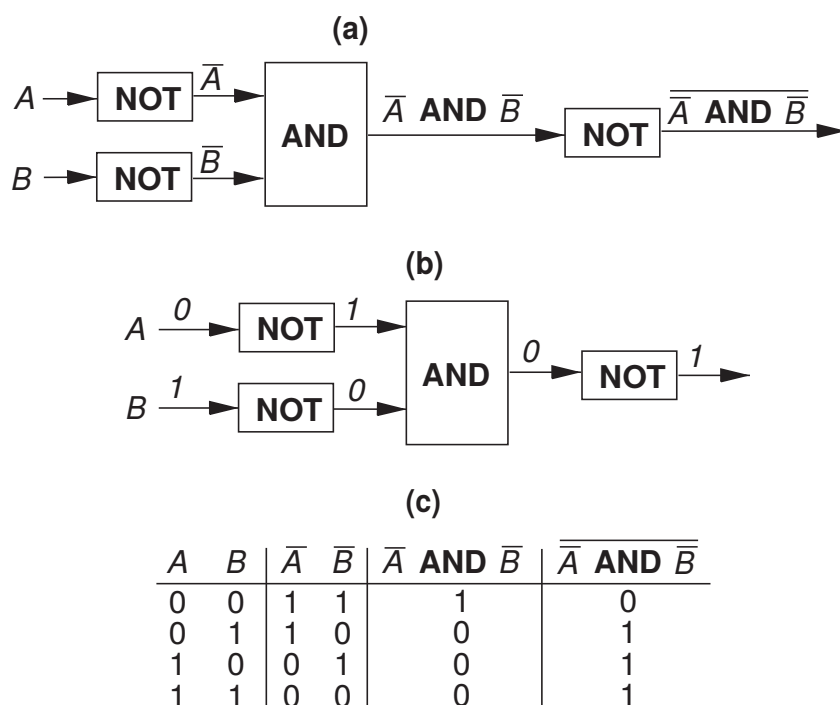


图 2.2 德摩根定律

例如，要同时检查一个 16 位补码整数的符号和奇偶性，可以使用以下掩码做 AND：

1000000000000001.

结果只能是以下四种：

0000000000000000
 0000000000000001
 1000000000000000
 1000000000000001

这四种情况分别对应非负偶数、非负奇数、负偶数和负奇数。

位向量也可以跟踪资源的忙闲状态，例如工厂里的机器或出租车公司的车辆。让每一位对应一个单元，1 表示空闲、可用，0 表示忙碌。

例 2.11 设八个单元的状态记录在名为 BUSYNESS 的八位向量中，当前值为 11000010。这表示单元 7、6 和 1 空闲，其余单元忙碌。

如果把一项任务交给单元 7，就应把第 7 位清为 0。与 01111111 做 AND，得到 01000010。如果随后单元 5 完成任务，就应把第 5 位置为 1。与 00100000 做 OR，得到 01100010。

2.7 其他表示

到目前为止，我们主要考察了整数和逻辑变量。下面简要介绍浮点数与 ASCII，并介绍一种方便书写位模式的十六进制记法。十六进制本身并不是一种新的数据类型。

2.7.1 浮点数：更大的范围，较低的精度

16 位补码用一位表示符号，其余 15 位提供数值的精度，总共能区分 2^{16} 个值。可是，我们有时需要表示非常大或非常小的数。例如，阿伏伽德罗常数约为 6.022×10^{23} 。这里需要很大的范围，却可能只需要 6022 这四个有效十进制数字的精度。

科学记数法把有效数字与数量级分开。浮点表示也采用这个思想：把一些位分配给精度，把另一些位分配给范围。多数 ISA 规定了不止一种浮点类型，其中一种通常称为 float，共有 32 位。图 2.3 给出大多数计算机采用的 IEEE 754 格式：一位符号位 S 、八位指数域 (exponent) 和 23 位小数域 (fraction)。

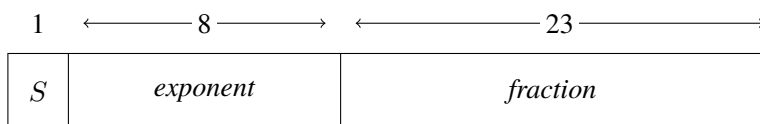


图 2.3 32 位浮点表示

2.7.1.1 规格化形式

对于指数域在 1 到 254 之间的位模式，所表示的数为

$$N = (-1)^S \times 1.\text{fraction} \times 2^{\text{exponent}-127}.$$

指数域为 0 或 255 的情况另有含义，稍后介绍。

对照十进制科学记数法 $+6.022 \times 10^{23}$ ：它有一个符号、小数点前的一位非零数字、若干后续有效数字，以及 10 的某次幂。二进制**规格化形式** (normalized form) 也要求小数点前有一位非零数字，但二进制的非零数字只能是 1。因此，这个开头的 1 不必存储，可以由格式隐含给出。23 位小数域加上隐含的 1，一共提供 24 位二进制精度。

符号位 $S = 0$ 表示正数， $S = 1$ 表示负数。指数采用**偏置表示**，也称 excess-127：先把存储的八位指数当作无符号整数，再减去 127，得到实际指数。例如，10000110 是 134，对应指数 +7；00000111 是 7，对应指数 -120。最大的普通指数域 254 对应 +127，最小的普通指数域 1 对应 -126。

例 2.12 位模式 00111101100000000000000000000000 可以分成

$$\underbrace{0}_S \quad \underbrace{01111011}_{\text{exponent}} \quad \underbrace{000000000000000000000000}_{\text{fraction}}.$$

符号为正，指数域为 123，小数域全为 0，因此表示 $+1.0 \times 2^{123-127} = 2^{-4} = 1/16$ 。

例 2.13 把 $-6\frac{5}{8}$ 表示成 32 位浮点数。先写成二进制：

$$-6\frac{5}{8} = -(110.101)_2 = -(1 \times 2^2 + 1 \times 2 + 0 + 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3}).$$

规格化后为 -1.10101×2^2 。符号位为 1；实际指数为 2，所以指数域为 $2 + 127 = 129$ ，即 10000001；小数域为 10101，后面补 18 个 0。最终得到

1 10000001 101010000000000000000000.

例 2.14 再看三个例子：

```
0 10000011 001010000000000000000000
1 10000010 001010000000000000000000
0 11111110 111111111111111111111111
```

第一个的指数域为 131，对应实际指数 4。因此它是 $+1.00101 \times 2^4 = (10010.1)_2 = 18.5$ 。

第二个的符号为负，指数域为 130，对应实际指数 3。因此它是 $-1.00101 \times 2^3 = -(1001.01)_2 = -9.25$ 。

第三个是这种格式能表示的最大有限正数：有效数字中的各位全为 1，实际指数为 127，数值略小于 2^{128} 。

2.7.1.2 无穷大

指数域全为 1、小数域全为 0 时，表示无穷大。符号位为 0 表示正无穷大，为 1 表示负无穷大。

2.7.1.3 次正规数

最小的正规化数是 $1.000 \dots 0 \times 2^{-126}$ 。为了表示更接近 0 的数，IEEE 754 还提供**次正规数**（subnormal number）。当指数域全为 0 时，不再使用隐含的开头 1，而采用

$$N = (-1)^S \times 0.\text{fraction} \times 2^{-126}.$$

最大的正次正规数是 $0.111 \dots 1 \times 2^{-126} = 2^{-126} - 2^{-149}$ ，其中小数域有 23 个 1；最小的正次正规数只有小数域最右边一位为 1，数值为 $2^{-23} \times 2^{-126} = 2^{-149}$ 。你能看出为什么最大值恰好是 $2^{-126} - 2^{-149}$ 吗？

例 2.15 位模式

```
0 00000000 000010000000000000000000
```

表示正次正规数。小数域的第 5 位为 1，因而数值是 $2^{-5} \times 2^{-126} = 2^{-131}$ 。

浮点运算的许多细节超出了入门课程的范围。这里希望你理解的是：通过把位分配给指数和有效数字，可以表示极大或极小的数，代价是在给定位数下牺牲一定的精度。

2.7.2 ASCII 码

另一种重要表示是设备厂商普遍约定采用的字符交换代码，用于在计算机主处理单元与输入、输出设备之间传送字符。它是称为 ASCII 的八位编码，即 American Standard Code for Information Interchange（美国信息交换标准代码）。如果键盘、计算机和显示器来自三个不同厂商，共同采用 ASCII 就能大大简化它们之间的接口。

例如，字符 3 的编码是 00110011，字符 2 是 00110010，小写 e 是 01100101，ENTER 对应 00001101。附录 E 的图 E.2 列出了 ASCII 码。敲击键盘时，对应的八位编码被存储起来，供计算机使用。它存在哪里、怎样进入计算机，将在第 9 章讨论。

大写与小写字母也有不同的编码。例如，大写 E 是 01000101，小写 e 是 01100101。按下字母键时是否同时按住 Shift，会影响送出的字符编码。显示器收到某个编码后，就显示相应字符。

2.7.3 十六进制记法

最后介绍一种便于人阅读和书写位模式的记法。它并不改变数据的含义，也不是一种独立的数据类型。LC-3 的位模式通常有 16 位。看一眼 0011110101101110，把它遮住，再试着准确写下来，你就会明白为什么需要更方便的记法。

把 16 位分成四组，每组四位，可以把原来的位值表达式重新组合为

$$\begin{aligned}\sum_{i=0}^{15} a_i 2^i &= (a_{15} 2^3 + a_{14} 2^2 + a_{13} 2 + a_{12}) 2^{12} \\ &\quad + (a_{11} 2^3 + a_{10} 2^2 + a_9 2 + a_8) 2^8 \\ &\quad + (a_7 2^3 + a_6 2^2 + a_5 2 + a_4) 2^4 \\ &\quad + (a_3 2^3 + a_2 2^2 + a_1 2 + a_0) \\ &= h_3 16^3 + h_2 16^2 + h_1 16 + h_0.\end{aligned}$$

每个 h_i 的值都在 0 到 15 之间。因此，我们可以改用以 16 为基数的位值记法，称为**十六进制**（hexadecimal）。所需的 16 个数字符号是 0 到 9，以及 A、B、C、D、E、F，后六个分别代表 10 到 15。

例如，0011 1101 0110 1110 四组分别是 3、D、6、E，所以整个模式可以写成 3D6E。这比原来的 16 位更容易记忆。每个十六进制数字恰好对应四个二进制位。顺便想一想：如果 E92F 表示一个 16 位补码整数，它是正数还是负数？你是怎么知道的？

十六进制虽然可以用于以 16 为基数的算术运算，但主要是为了方便人使用。无论一串二进制位表示整数、浮点数、ASCII 字符序列还是位向量，都可以这样简写。数字个数减少为原来的四分之一，每个数字改用 0 到 F，而不是 0 或 1，通常就能显著减少复制长位串时的错误。

习题

- 2.1** 给定 n 位，一共有多少种不同的位组合？
- 2.2** 英文字母表有 26 个字母。为每个字母分配唯一的位模式，最少需要多少位？如果还要区分这 26 个字母的大小写，需要多少位？
- 2.3** a. 假设班里约有 400 名学生，要给每人分配唯一的位模式，最少需要多少位？
b. 在不增加每人编码位数的情况下，还可以接收多少名学生？
- 2.4** 给定 n 位，可以表示多少个无符号整数？这些整数的范围是什么？
- 2.5** 每个数使用五位，分别写出 7 和 -7 的反码、原码和补码表示。
- 2.6** 写出 -32 的六位补码表示。
- 2.7** 制作一张表，列出全部四位补码数及其十进制值。
- 2.8** a. 八位补码能表示的最大正数是什么？分别用二进制和十进制作答。
b. 八位补码能表示的绝对值最大的负数是什么？分别用二进制和十进制作答。
c. n 位补码能表示的最大正数是什么？
d. n 位补码能表示的绝对值最大的负数是什么？
- 2.9** 用二进制补码表示阿伏伽德罗常数 6.02×10^{23} ，需要多少位？
- 2.10** 把下列二进制补码数转换为十进制。
a. 1010
b. 01011010
c. 11111110
d. 0011100111010011
- 2.11** 把下列十进制数转换为八位二进制补码。
a. 102
b. 64
c. 33
d. -128
e. 127
- 2.12** 若补码数的最后一位为 0，则该数为偶数。如果最后两位都是 0，例如 01100，还能得出什么结论？

2.13 在不改变数值的前提下，把下列补码数转换为八位补码。

- a. 1010
- b. 011001
- c. 1111111000
- d. 01

2.14 将下列位模式相加，结果保留二进制形式。

- a. 1011 + 0001
- b. 0000 + 1010
- c. 1100 + 0011
- d. 0101 + 0110
- e. 1111 + 0001

2.15 例 2.5 说明，二进制数左移一位相当于乘以 2。右移一位相当于什么运算？

2.16 分别用八位二进制和十进制写出以下加法的结果。每一问都使用第 2.5.1 节介绍的标准二进制加法。

- a. 将 7 的反码与 -7 的反码相加。
- b. 将 7 的原码与 -7 的原码相加。
- c. 将 7 的补码与 -7 的补码相加。

2.17 将下列二进制补码数相加，并把答案也写成十进制。

- a. 01 + 1011
- b. 11 + 01010101
- c. 0101 + 110
- d. 01 + 10

2.18 将下列无符号二进制数相加，并把答案也写成十进制。

- a. 01 + 1011
- b. 11 + 01010101
- c. 0101 + 110
- d. 01 + 10

2.19 分别用八位、16 位和 32 位补码表示 -27。这说明了补码符号扩展的什么性质？

2.20 下列各数都是四位补码。哪些运算会溢出？将操作数和结果转换为十进制，说明理由。

- a. $1100 + 0011$
- b. $1100 + 0100$
- c. $0111 + 0001$
- d. $1000 - 0001$
- e. $0111 + 1001$

2.21 两个补码数相加时，什么条件表明发生了溢出？

2.22 构造两个 16 位补码整数，使它们的和发生溢出。

2.23 两个无符号数相加时，什么条件表明发生了溢出？

2.24 构造两个 16 位无符号整数，使它们的和发生溢出。

2.25 为什么一个负补码数与一个正补码数相加绝不会溢出？

2.26 要用补码表示 -64 。

- a. 最少需要多少位？
- b. 使用这么多位，能表示的最大正数是什么？用十进制和二进制作答。
- c. 使用这么多位，能表示的最大无符号数是什么？用十进制和二进制作答。

2.27 LC-3 是一台 16 位机器。它将下列两个补码数相加：

$$0101010101010101 + 0011100111001111,$$

得到 1000111100100100 。这里有问题吗？如果有，是什么问题？如果没有，为什么？

2.28 AND 运算的输出在什么情况下等于 1？

2.29 填写下面的一位 AND 运算真值表。

X	Y	$X \text{ AND } Y$
0	0	
0	1	
1	0	
1	1	

2.30 计算下列各式，用二进制写出结果。

- a. $01010111 \text{ AND } 11010111$
- b. $101 \text{ AND } 110$
- c. $11100000 \text{ AND } 10110100$
- d. $00011111 \text{ AND } 10110100$

e. $(0011 \text{ AND } 0110) \text{ AND } 1101$

f. $0011 \text{ AND } (0110 \text{ AND } 1101)$

2.31 OR 运算的输出在什么情况下等于 1?

2.32 填写下面的一位 OR 运算真值表。

X	Y	$X \text{ OR } Y$
0	0	
0	1	
1	0	
1	1	

2.33 计算下列各式。

a. $01010111 \text{ OR } 11010111$

b. $101 \text{ OR } 110$

c. $11100000 \text{ OR } 10110100$

d. $00011111 \text{ OR } 10110100$

e. $(0101 \text{ OR } 1100) \text{ OR } 1101$

f. $0101 \text{ OR } (1100 \text{ OR } 1101)$

2.34 计算下列各式。

a. $\text{NOT}(1011) \text{ OR } \text{NOT}(1100)$

b. $\text{NOT}(1000 \text{ AND } (1100 \text{ OR } 0101))$

c. $\text{NOT}(\text{NOT}(1101))$

d. $(0110 \text{ OR } 0000) \text{ AND } 1111$

2.35 例 2.11 中的掩码有什么用途?

2.36 根据例 2.11 回答以下问题。

a. 应使用什么掩码和什么运算来表示机器 2 忙碌?

b. 应使用什么掩码和什么运算来表示机器 2 和 6 不再忙碌? 注意, 只需一次运算。

c. 应使用什么掩码和什么运算来表示所有机器都忙碌?

d. 应使用什么掩码和什么运算来表示所有机器都空闲?

e. 利用本章介绍的运算, 设计一个过程, 把机器 2 的状态位提取到符号位。例如, **BUSYNESS** 为 01011100 时输出 10000000, 为 01110011 时输出 00000000。一般而言, 输入为 $b_7b_6b_5b_4b_3b_2b_1b_0$, 输出为 $b_20000000$ 。提示: 把一个位模式与自身 ADD, 会发生什么?

- 2.37** n 和 m 都是四位补码, s 是它们相加所得的四位结果。如何只用第 2.6 节的逻辑运算判断加法是否溢出? 设计一个过程, 以 n 、 m 、 s 为输入; 未溢出时输出 0000, 溢出时输出 1000。
- 2.38** n 和 m 都是四位无符号数, s 是它们相加所得的四位结果。如何只用第 2.6 节的逻辑运算判断加法是否溢出? 设计一个过程, 以 n 、 m 、 s 为输入; 未溢出时输出 0000, 溢出时输出 1000。
- 2.39** 写出以下十进制数的 IEEE 浮点表示。
- 3.75
 - $-55\frac{23}{64}$
 - 3.1415927
 - 64000
- 2.40** 写出下列 IEEE 浮点数对应的十进制值。
- 0 10000000 000000000000000000000000
 - 1 10000011 000100000000000000000000
 - 0 11111111 000000000000000000000000
 - 1 10000000 100100000000000000000000
- 2.41**
- IEEE 标准的 32 位浮点数允许的最大指数是什么?
 - 允许的最小指数是什么?
- 2.42** 一名程序员编写了一个把两个数相加的程序。运行时发现, 5 加 8 的结果却是字符 m。解释程序为什么会出现这种错误。
- 2.43** 将下面的 ASCII 码转换为字符串, 每八位解释为一个 ASCII 字符。
- x48656c6c6f21
 - x68454c4c4f21
 - x436f6d70757465727321
 - x4c432d32
- 2.44** 用什么运算可以把 3 的二进制表示 0000 0011 转换成字符 3 的 ASCII 表示 0011 0011? 如果是 4 呢? 任意一个十进制数字呢?
- 2.45** 把下列无符号二进制数转换为十六进制。
- 1101 0001 1010 1111
 - 001 1111
 - 1
 - 1110 1101 1011 0010

2.46 把下列十六进制数转换为二进制。

- a. x10
- b. x801
- c. xF731
- d. x0F1E2D
- e. xBCAD

2.47 把下列以十六进制书写的补码数转换为十进制。

- a. xF0
- b. x7FF
- c. x16
- d. x8000

2.48 把下列十进制数转换为补码的十六进制表示。

- a. 256
- b. 111
- c. 123 456 789
- d. -44

2.49 完成下列加法。相应的 16 位二进制数采用补码表示，请用十六进制给出答案。

- a. x025B + x26DE
- b. x7D96 + xF0A0
- c. xA397 + xA35D
- d. x7D96 + x7412
- e. 对 c、d 两问的结果，还能说些什么？

2.50 完成下列逻辑运算，用十六进制写出答案。

- a. x5478 AND xFDEA
- b. xABCD OR x1234
- c. NOT ((NOT (xDEFA)) AND (NOT (xFFFF)))
- d. x00FF XOR x325C

2.51 下列各项的十六进制表示是什么？

- a. 25 675。
- b. 采用 IEEE 754 浮点标准的 675.625，即 $675\frac{5}{8}$ 。

c. ASCII 字符串 Hello。

2.52 考虑两个十六进制数 x434F4D50 和 x55544552。把它们分别解释为下列五种数据类型时，各代表什么值？

	x434F4D50	x55544552
无符号二进制整数		
反码		
补码		
IEEE 754 浮点数		
ASCII 字符串		

2.53 根据下列等式填写真值表。第一行已填好作为示例。

$$Q_1 = \text{NOT}(A \text{ AND } B),$$
$$Q_2 = \text{NOT}((\text{NOT } A) \text{ AND } (\text{NOT } B)).$$

A	B	Q_1	Q_2
0	0	1	0
0	1		
1	0		
1	1		

用另一种方式表示 Q_2 。

2.54 根据下列等式填写真值表。第一行已填好作为示例。

$$Q_1 = \text{NOT}((\text{NOT } X) \text{ OR } (X \text{ AND } Y \text{ AND } Z)),$$
$$Q_2 = \text{NOT}((Y \text{ OR } Z) \text{ AND } (X \text{ AND } Y \text{ AND } Z)).$$

X	Y	Z	Q_1	Q_2
0	0	0	0	1
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

2.55 我们已经用过基数为 2 的二进制和基数为 16 的十六进制。现在考虑无符号四进制（quad），其数字只能为 0、1、2、3。

a. 三个四进制数字能表示的最大无符号十进制值是什么？

- b. n 个四进制数字能表示的最大无符号十进制值是什么？提示：答案应是 n 的函数。
 - c. 将两个无符号四进制数 023 和 221 相加。
 - d. 十进制数 42 的四进制表示是什么？
 - e. 无符号四进制数 123.3 的二进制表示是什么？
 - f. 用 IEEE 浮点格式表示无符号四进制数 123.3。
 - g. 一个黑盒以 m 个四进制数字为输入，输出一个四进制数字。它最多可以实现多少种不同的函数？
- 2.56** 定义一种新的八位浮点格式：一位符号位、四位采用 excess-7（偏置为 7）编码的指数域，以及三位小数域。如果 xE5 是这种格式的一个数的位模式，它的数值是多少？用十进制作答。

第三章 数字逻辑结构

第 1 章说过，计算机是由数量极大的简单结构构成的。例如，Intel 在 2016 年推出的 Broadwell-E5 微处理器包含超过 70 亿个晶体管；IBM 在 2017 年推出的 Power9 则包含 80 亿个。本章将解释 MOS 晶体管作为逻辑元件如何工作，展示怎样把它们连接成逻辑门，再把逻辑门连接成构建计算机所需的更大部件。第 4 章将把这些部件连接起来，组成一台计算机。

不过，先从晶体管开始。

3.1 晶体管

今天，大多数计算机——更准确地说，构成计算机核心的大多数微处理器——都是用 MOS 晶体管制造的。MOS 是 metal-oxide semiconductor（金属氧化物半导体）的缩写。它们的电学性质远远超出本课程的讨论范围，处于我们最低的抽象层次之下。这意味着，如果晶体管出了问题，我们就只能听天由命了。不过，在这门课中，晶体管不太可能给我们带来麻烦。

仍然有必要知道，MOS 晶体管有 P 型和 N 型两种。从逻辑上看，它们的工作方式都很像墙壁开关。

图 3.1 是最基本的电路，由三部分组成：电源（在美国，是进入住宅的 120 V 电源；在世界上大多数其他地区，是 220 V 电源）、墙壁开关，以及接入插座的灯。灯要发光，就必须有电子流动；电子要流动，就必须存在一条从电源经过灯再返回电源的闭合回路。拨动开关，接通或断开回路，就能使灯亮或灭。

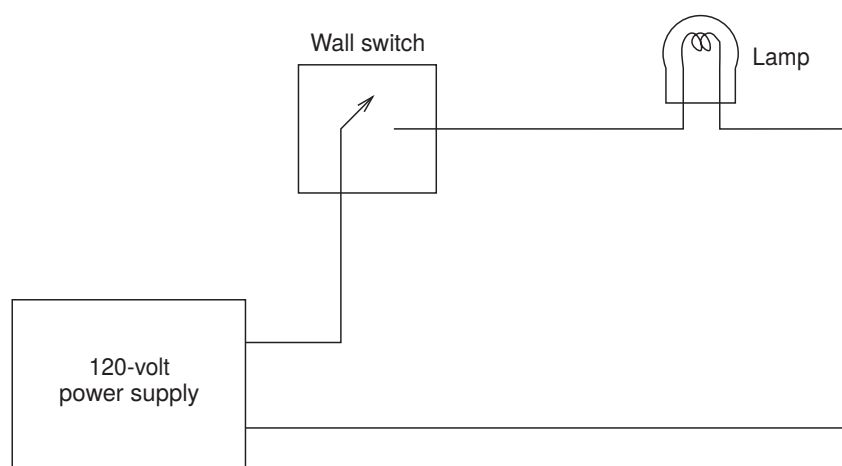


图 3.1 一个用墙壁开关控制的简单电路

可以用 N 型或 P 型 MOS 晶体管代替墙壁开关，接通或断开电路。图 3.2(a) 是 N 型晶体管的示意图，(b) 展示它接入电路的情形。晶体管有三个端子，分别称为栅极

(gate)、**源极** (source) 和**漏极** (drain)。本课程不关心源极和漏极名称的由来，而关心下面的性质：N 型晶体管的栅极接上 1.2 V 时，源极和漏极之间就像接了一根导线，电学上称为**短路**；栅极接 0 V 时，源极和漏极之间的连接断开，称为**开路**。

图 3.2 中，晶体管与电池、灯泡组成电路。栅极为 1.2 V 时，晶体管像导线一样接通回路，灯泡发光；栅极为 0 V 时，晶体管使回路断开，灯泡不亮。

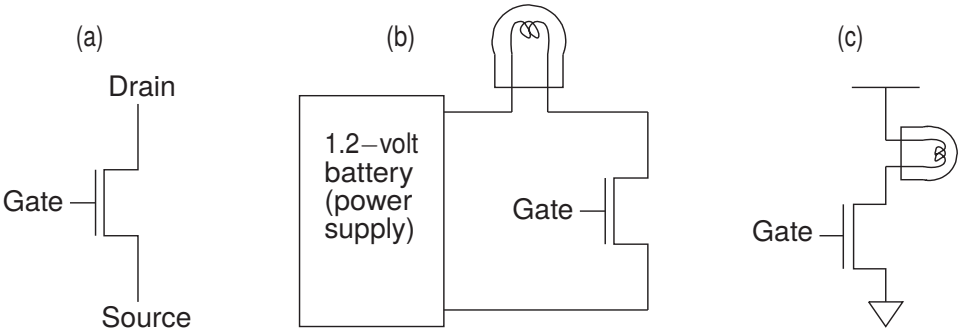


图 3.2 N 型 MOS 晶体管

图 3.2(c) 是 (b) 的简写。电气工程师通常只画出电源的端点，而不反复画出整个电源和完整回路。大家都知道电源本身会把回路闭合，所以通常省略它。

P 型晶体管恰好相反。图 3.3 是其示意图。栅极为 0 V 时，它大体上像一根导线，使电路闭合；栅极为 1.2 V 时，它表现为开路。由于 P 型与 N 型晶体管的行为互补，同时使用两者的电路称为 CMOS 电路，即 complementary metal-oxide semiconductor（互补金属氧化物半导体）电路。

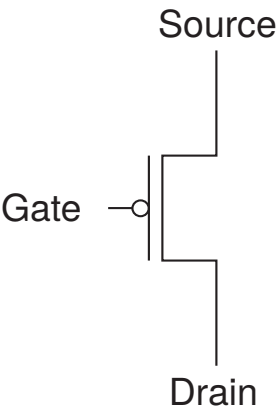


图 3.3 P 型 MOS 晶体管

3.2 逻辑门

比晶体管高一个层次的是逻辑门，即用单个 MOS 晶体管构成的基本逻辑结构。第 2 章研究了 AND、OR、NOT 函数的行为，现在来构造实现这些函数的晶体管电路。对应的电路分别称为 AND 门、OR 门和 NOT 门。

3.2.1 NOT 门（反相器）

图 3.4 是计算机中最简单的逻辑结构，由一个 P 型和一个 N 型 MOS 晶体管构成。(a) 是电路示意图；(b) 展示输入为 0 V 时的行为：P 型晶体管相当于短路，N 型晶体管相当于开路，因此输出接到 1.2 V。反过来，输入为 1.2 V 时，P 型晶体管开路，N 型晶体管短路，输出接地，即 0 V。

表 (c) 完整描述了电路的行为。把 0 V 换成符号 0，把 1.2 V 换成符号 1，就得到表 (d)，也就是第 2 章的 NOT 真值表。换句话说，我们已经构造出实现 NOT 逻辑函数的电子电路，称为非门（NOT gate）或反相器（inverter）。

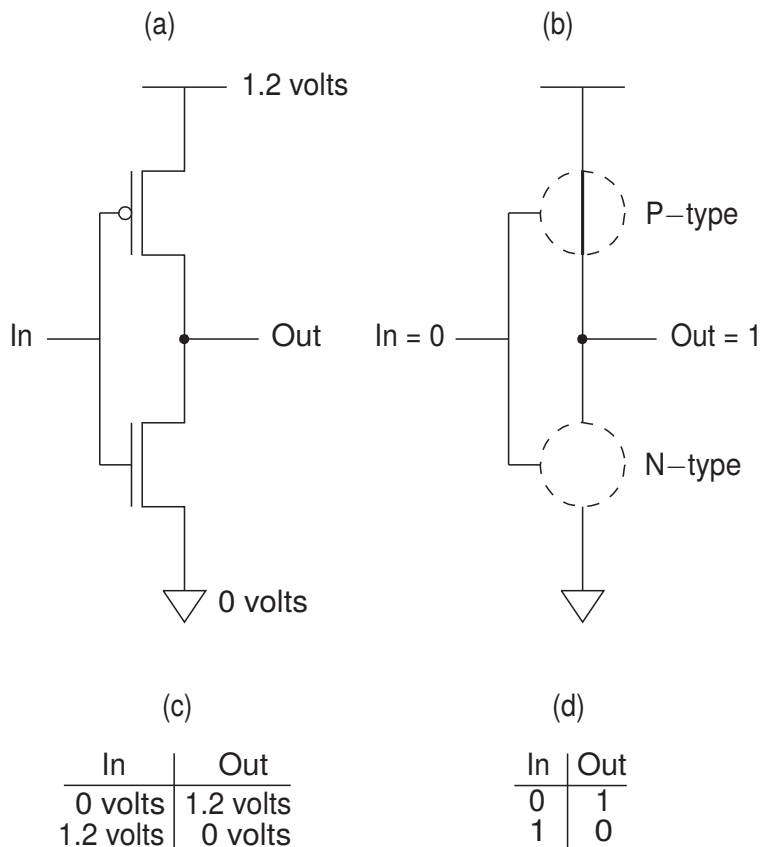


图 3.4 CMOS 反相器

3.2.2 OR 门与 NOR 门

图 3.5(a) 展示一个 NOR 门的电路，由两个 P 型和两个 N 型晶体管构成。(b) 中，A 接 0 V，B 接 1.2 V。下方的 P 型晶体管开路，使输出 C 与 1.2 V 电源断开；最左边的 N 型晶体管则像导线一样把 C 接到 0 V。

如果 A、B 都为 0 V，两个 P 型晶体管都导通，C 接到 1.2 V。这里没有歧义，因为两个 N 型晶体管都开路，C 与地断开。

只要 A 或 B 为 1.2 V，对应的 P 型晶体管就开路，足以切断 C 与 1.2 V 电源的连接。同时，对应 N 型晶体管的栅极为 1.2 V，足以使它导通，把 C 接地。

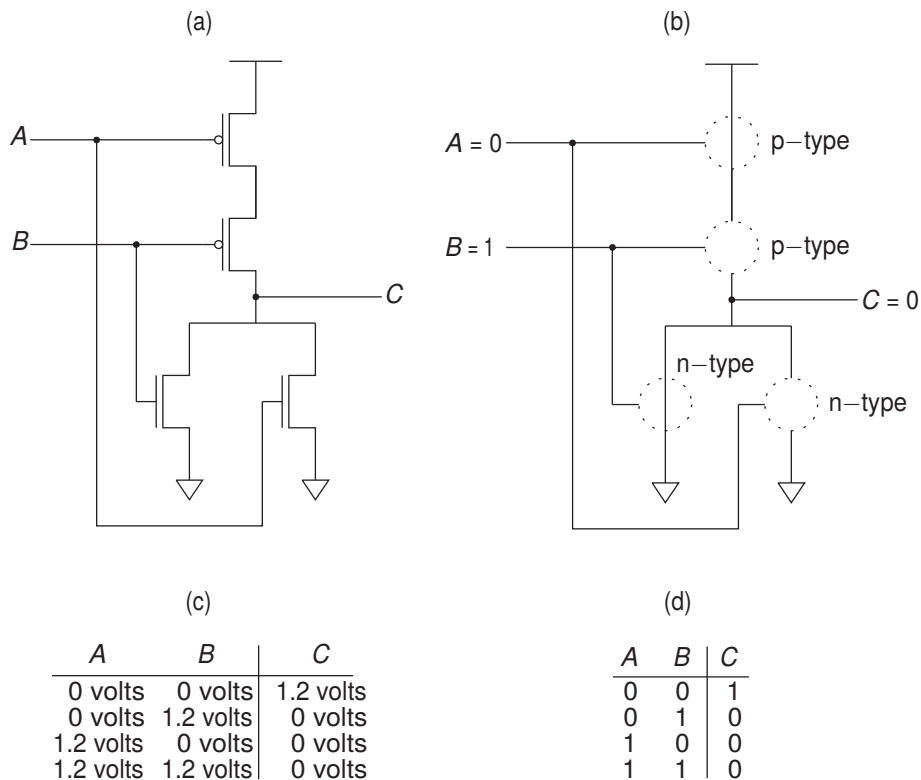


图 3.5 NOR 门

图 3.5(c) 列出四种输入电压组合 (0, 0)、(0, 1.2)、(1.2, 0)、(1.2, 1.2) 下的完整行为。换成逻辑值，就得到 (d) 的真值表。输出 C 恰好是第 2 章 OR 函数的反：NOT-OR，通常缩写为 NOR。实现它的电路称为**或非门**（NOR gate）。

如图 3.6(a)，在 NOR 门的输出端再接一个反相器，输出 D 就是 OR。因此，整个电路是**或门**（OR gate）。(b) 展示 A = 0、B = 1 时的行为，(c) 是它的真值表。

3.2.3 为什么不能简单地把 P 型晶体管接地

一些机灵的同学看过“先 NOR、再反相”的 OR 门实现后，会问：为什么不直接按图 3.7(a) 连接？从逻辑上看很诱人，只需四个晶体管，而不是六个。不幸的是，晶体管的电学性质使这种做法行不通。

当 P 型晶体管接 1.2 V、N 型晶体管接地时，按照这里的电路模型，导通晶体管上没有电压降，因此能像图 3.5 那样，随输入得到 0 V 或 1.2 V 的输出。但如果把 P 型晶体管接地、N 型晶体管接 1.2 V，其电学特性会造成约 0.5 V 的传输电压降。于是图 3.7 的电路在 A = B = 0 时输出 0.5 + 0.5 = 1.0 V，其他情况下输出 1.2 - 0.5 = 0.7 V。

图 3.7(b) 因而列出实际电压，而不是 0 和 1。这个电路虽然看起来可行，却会用 1 V 表示逻辑 0，用 0.7 V 表示逻辑 1。这可不是我们想要的 OR 门！

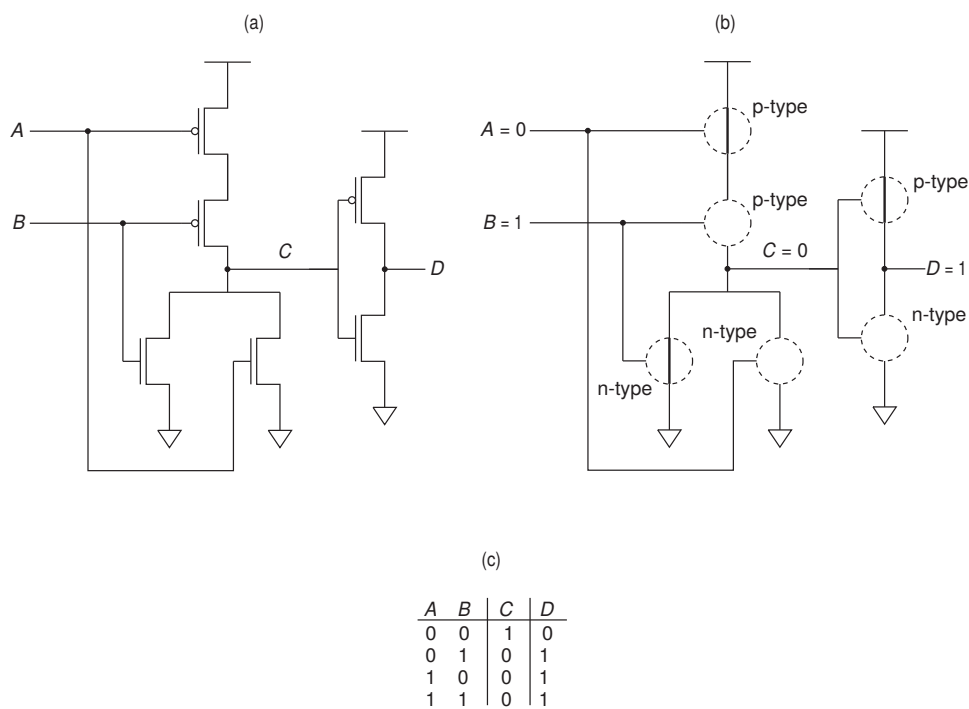


图 3.6 OR 门

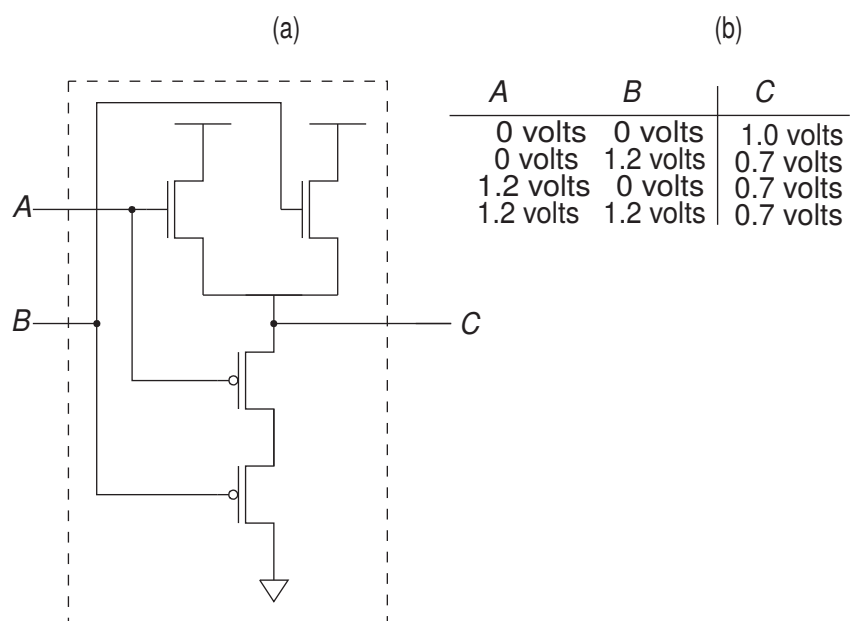


图 3.7 一个 OR 门（其实不是!）

3.2.4 AND 门与 NAND 门

图 3.8 展示一个 AND 门。只要 A 或 B 为 0 V ， C 就直接接到 1.2 V 电源。 C 为 1.2 V ，使栅极接 C 的 N 型晶体管导通，把 D 接地。因此，只要有一个输入为 0 V ，输出 D 就为 0 V 。

这里同样没有歧义：至少一个输入为 0 V ，意味着栅极接 A 或 B 的两个 N 型晶体管中至少一个开路，所以 C 与地断开；而 C 为 1.2 V ，又使栅极接 C 的 P 型晶体管开路，所以 D 与 1.2 V 电源断开。

反之，如果 A 、 B 都为 1.2 V ，对应的 P 型晶体管都开路，N 型晶体管都像导线一样，把 C 接地。 C 为 0 V ，使最右边的 P 型晶体管导通， D 因而为 1.2 V 。

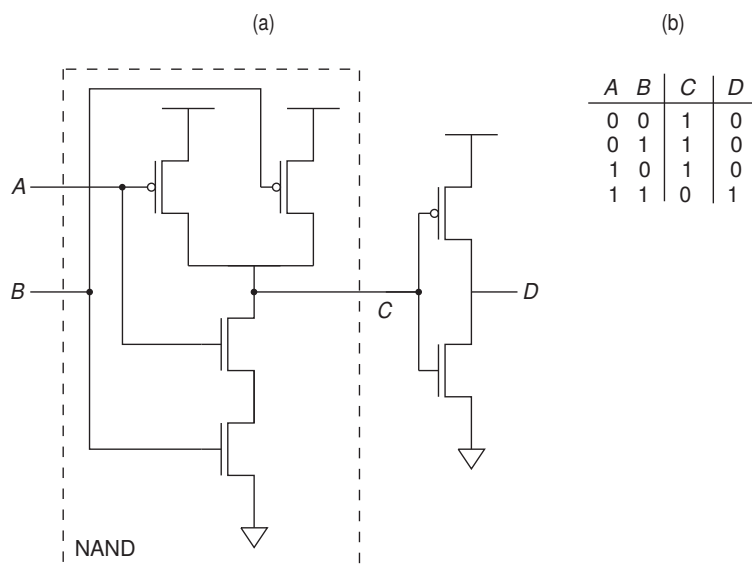


图 3.8 AND 门

图 3.8(b) 的真值表汇总了这些行为，表明整个电路是与门 (AND gate)。虚线框内、以 C 为输出的部分则是 NOT-AND，通常缩写为 NAND，称为与非门。

这些门在数字逻辑电路和数字计算机中非常常见。例如，Intel 的 Skylake 微处理器含有数十亿个反相器。为方便起见，我们用图 3.9 的标准符号表示它们。反相器、NAND 门和 NOR 门符号中的小圆圈表示取反，即 NOT。接下来将提高抽象层次，不再逐个画出晶体管，而使用这些门符号。

3.2.5 超过两个输入的逻辑门

AND、OR、NAND、NOR 的概念可以推广到更多输入。例如，可以构造三输入 AND 门或四输入 OR 门。 n 输入 AND 门只有在全部 n 个输入都为 1 时才输出 1；只要其中一个为 0，输出就是 0。 n 输入 OR 门只要有一个输入为 1，就输出 1；只有全部输入都为 0，输出才为 0。

图 3.10(a) 给出三输入 AND 门的真值表，(b) 给出其符号。你能画出三输入 AND 门的晶体管电路吗？四输入 AND 门呢？四输入 OR 门呢？

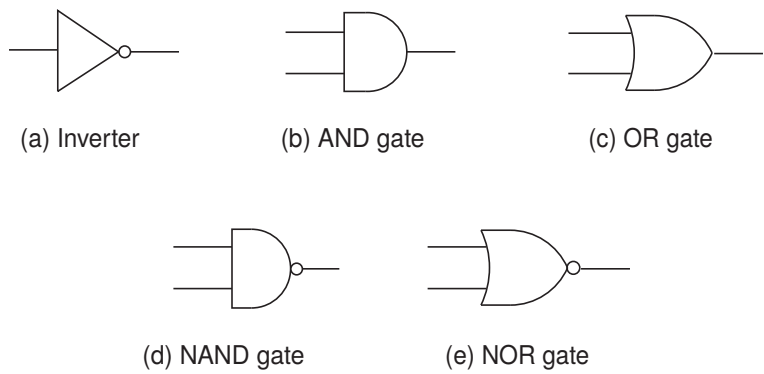


图 3.9 基本逻辑门

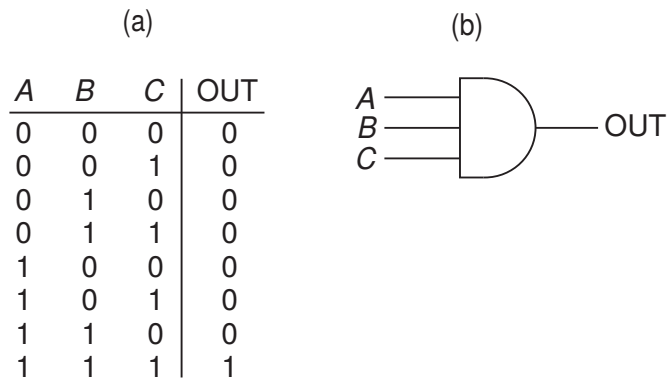


图 3.10 三输入 AND 门

3.3 组合逻辑电路

理解基本逻辑门之后，下一步是构建计算机微体系结构中的一些重要部件。

逻辑结构从根本上可分为两类：存储信息的和不存储信息的。第 3.4、3.5、3.6 节讨论前者，本节讨论后者。这些结构有时称为判决元件，通常称为**组合逻辑结构**：输出完全由此刻施加的输入值组合决定，与过去的历史无关，因为组合逻辑电路内部根本不能存储信息。

下面考察三个有用的组合逻辑电路：译码器、多路选择器和一位加法器。

3.3.1 译码器

图 3.11 给出两输入**译码器**（decoder）的门级实现。译码器的输出中恰有一个为 1，其余全为 0；为 1 的那一根输出线，对应被检测到的输入模式。一般来说，译码器有 n 个输入、 2^n 个输出。我们说检测到输入模式的那根输出线被**置为有效**（asserted）：它为 1，其他输出线为 0。

图中 A 、 B 的四种输入组合，每一种在任一时刻都恰好使一个输出为 1。在 (b) 中，输入为 10，第三根输出线有效。

译码器可用于确定如何解释位模式。第 5 章将看到，LC-3 每条指令要执行什么工作，由指令中的四位**操作码**决定。一个 4 线—16 线译码器就是识别指令操作的简单

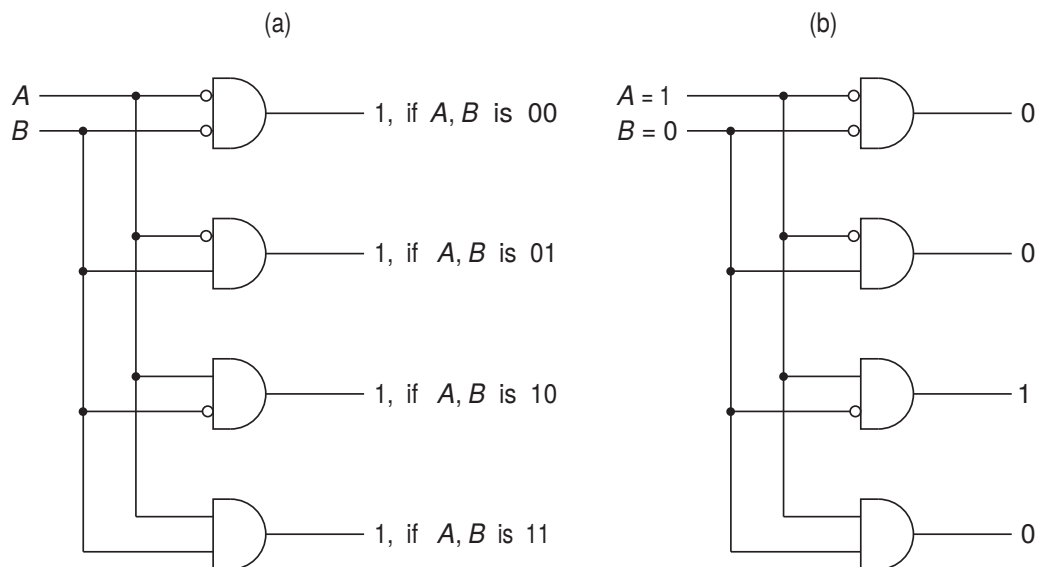


图 3.11 两输入译码器

组合逻辑结构。

3.3.2 多路选择器

图 3.12(a) 给出两输入**多路选择器** (multiplexer, 通常简称 mux) 的门级实现。mux 从输入 A 、 B 中选择一个, 连接到输出; 选择信号 S 决定选择哪一个。

假设 $S = 0$, 如 (b)。AND 门只有全部输入为 1 才输出 1, 因此最右边 AND 门的输出为 0; 最左边 AND 门的输出则等于 A : $A = 0$ 时为 0, $A = 1$ 时为 1。右边 AND 门的 0 不影响 OR 门的结果, 所以输出 C 恰好等于左边 AND 门的输出, 也就等于 A 。

若 $S = 1$, 则是 B 与 1 做 AND, 使 OR 门的输出等于 B 。因此, C 总是连接到 A 或 B , 具体是哪一个取决于选择线 S 。我们说 S 选择了要送到输出 C 的输入源。(c) 是 mux 的标准符号。

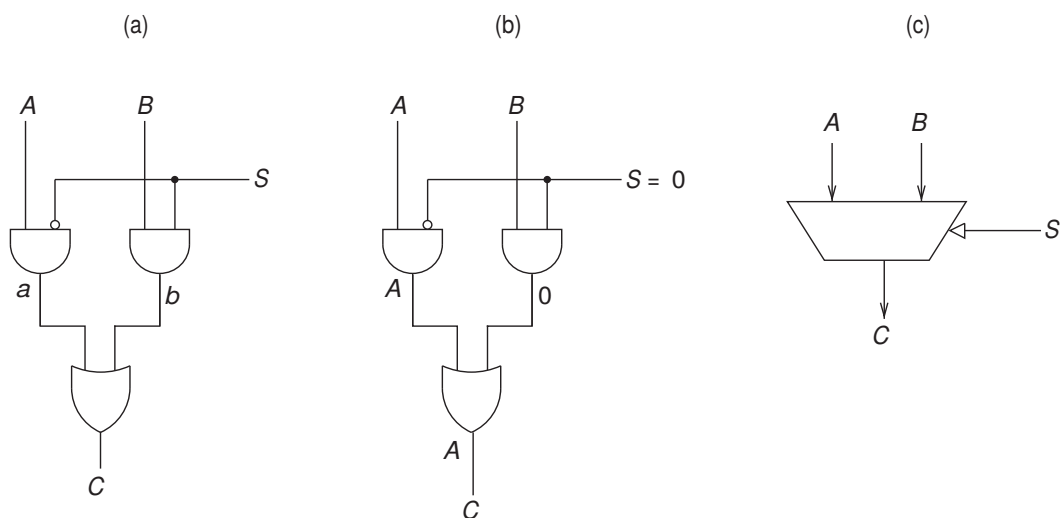


图 3.12 2 选 1 多路选择器

一般而言，mux 有 2^n 个数据输入和 n 根选择线。图 3.13(a) 是四输入 mux 的门级实现，需要两根选择线；(b) 是标准符号。你能构造八输入 mux 的门级表示吗？它需要几根选择线？

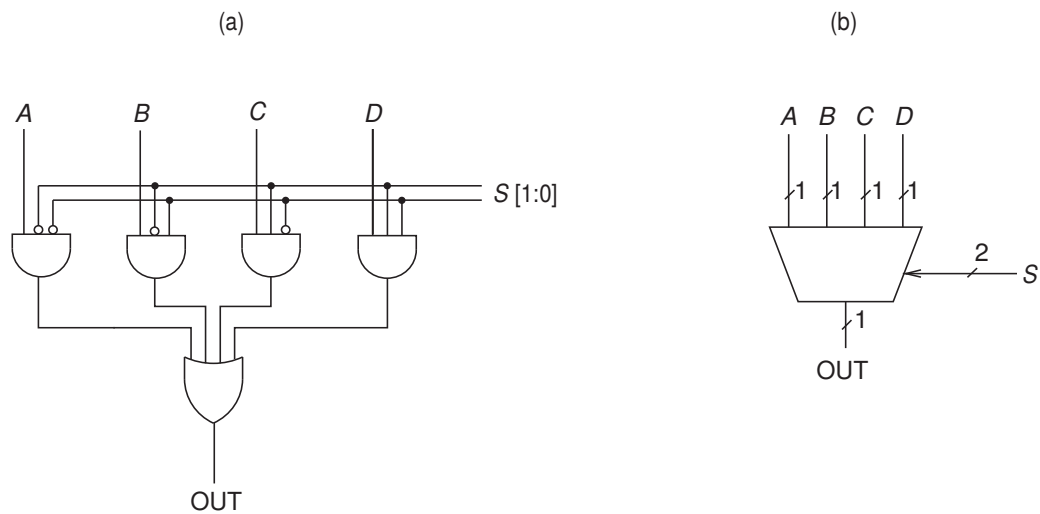


图 3.13 四输入多路选择器

3.3.3 一位加法器（又称全加器）

回想第 2 章的二进制加法。简单算法与十进制加法一样，从右向左逐列进行：把两个数在这一列的数字与前一列的进位相加，产生这一列的和，以及送往下一列的进位。区别只是二进制逢二进一，而十进制逢十进一。

图 3.14 描述两个 n 位操作数在某一列上的加法。要相加的三位是操作数的 A_i 、 B_i 和来自前一列的进位 C_i ；两个结果是和位 S_i 与送往下一列的进位 C_{i+1} 。如果只有一位为 1，和为 1、进位为 0；如果有两位为 1，和为 0、进位为 1；如果三位都为 1，总和为 3，二进制中对应和位 1、进位 1。

A_i	B_i	C_i	C_{i+1}	S_i
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

图 3.14 一位加法器的真值表

图 3.15 是它的门级实现。每个 AND 门恰好对八种输入组合中的一种输出 1。产生 C_{i+1} 的 OR 门，应在真值表规定进位为 1 的那些组合下输出 1，所以把相应 AND

门的输出接到这个 OR 门。产生 S_i 的 OR 门也采用相同办法，连接到真值表规定和位为 1 的那些组合。

输入组合 000 的两个输出都为 0，因此对应的 AND 门不需要接到任何一个 OR 门。

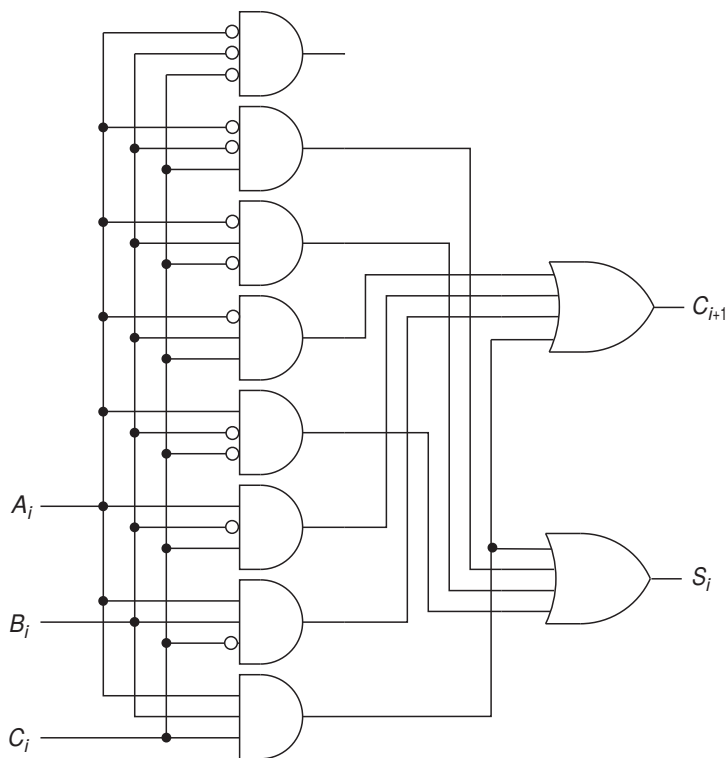


图 3.15 一位加法器的门级描述

图 3.16 用四个一位加法器构成两个四位数的加法电路。第 i 列的输出进位是第 $i + 1$ 列的输入。若要把两个 16 位数相加，只需连接 16 个一位加法器。

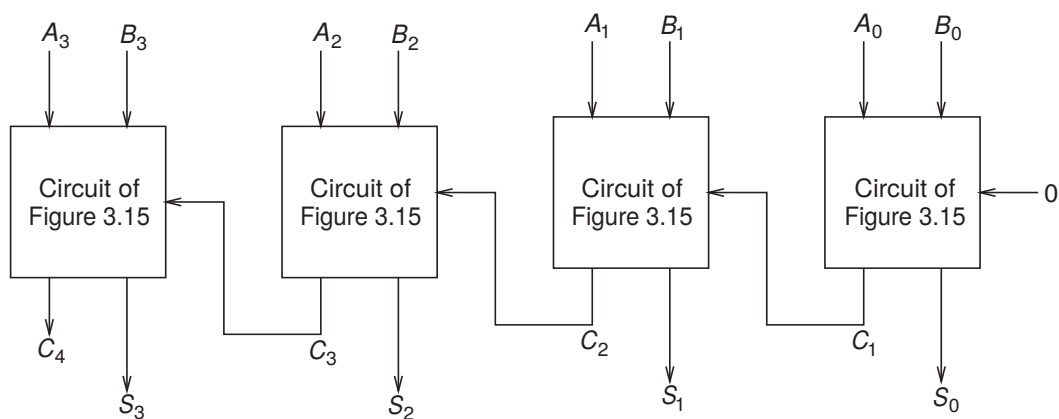


图 3.16 将两个四位二进制数相加的电路

历史上，图 3.15 这种有 A_i 、 B_i 、 C_i 三个输入和 S_i 、 C_{i+1} 两个输出的结构，通常称为**全加器**（full adder），以区别于**半加器**（half adder）。区别在于进位输入。图 3.16 最右列的输入进位是 0，所以 S_0 和 C_1 只依赖 A_0 、 B_0 两个输入；只处理这两个输入

的电路称为半加器。其余列依赖全部三个输入，称为全加器。我们更喜欢“一位加法器”这个名称，因为它更直接地说明每一列在做什么。

3.3.4 可编程逻辑阵列 (PLA)

图 3.17 是一种很常见的构件，可以实现任意一组逻辑函数，称为**可编程逻辑阵列** (programmable logic array, PLA)。它由一组 AND 门组成的 AND 阵列，以及后接的一组 OR 门组成的 OR 阵列构成。

AND 门的数量对应真值表的输入组合数，也就是行数。对于 n 输入逻辑函数，这里的 PLA 使用 2^n 个 n 输入 AND 门。图中有 2^3 个三输入 AND 门，对应三个逻辑输入变量。OR 门的数量对应所需逻辑函数的数量，也就是真值表的输出列数。

实现算法很简单：如果真值表某一行在某个输出列上为 1，就把对应 AND 门的输出连接到对应 OR 门的输入。“可编程”指的就是这些连接：通过配置 AND 阵列到 OR 阵列的连接，实现所需函数。

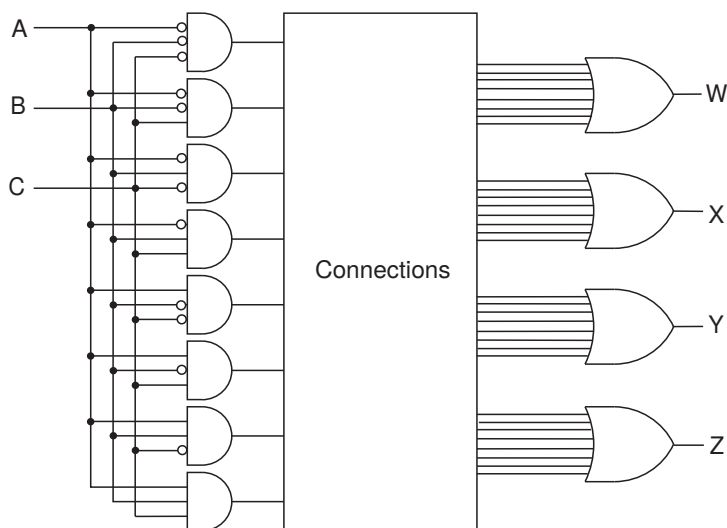


图 3.17 可编程逻辑阵列

图 3.15 用七个 AND 门连接两个 OR 门，因为要实现三个输入变量的两个函数——和与进位。图 3.17 则可以通过适当连接，实现三个变量的任意四个函数。对每个函数，只需把输出应为 1 的那些输入组合对应的 AND 门接到一个 OR 门即可。例如，按图 3.14 的两个输出列配置输出为 W、X 的两个 OR 门，就能实现一位加法器。

3.3.5 逻辑完备性

在结束组合逻辑电路的讨论前，还要介绍一个重要性质：**逻辑完备性** (logical completeness)。第 3.3.4 节表明，任意逻辑函数都可以用 PLA 实现，而 PLA 只包含 AND、OR、NOT 门。因此，只要这些门足够多，就能实现任意真值表规定的逻辑函数，不需要其他种类的门。我们说集合 {AND, OR, NOT} 是逻辑完备的。打个比方，给你一桶 AND 门、一桶 OR 门和一桶 NOT 门，就足以实现任意所需真值表；桶也许得很大，但无需别种门。

有没有某一种两输入门，单独就具有逻辑完备性？例如 NAND 门呢？提示：能否用 NAND 门实现 NOT？如果能，能否把 NAND 后接 NOT 来实现 AND？再能否只用 AND 和 NOT 实现 OR？如果都能做到，那么一桶 NAND 门就足以实现任意真值表所描述的函数，NAND 单独就是逻辑完备的。

3.4 基本存储元件

第 3.3 节开头把逻辑结构分为存储信息和不存储信息的两类。已经讨论的译码器、mux 和全加器都不存储信息。现在来讨论能够存储信息的结构。

3.4.1 R-S 锁存器

R-S 锁存器 (R-S latch) 是一个简单的存储元件，能存储一位信息，即 0 或 1。它有多种实现方法，最简单的一种如图 3.18：两个两输入 NAND 门交叉连接，每个门的输出接到另一个门的一个输入；剩余输入 S 、 R 通常保持为逻辑 1。

R-S 这个名称来自旧有术语：使锁存器存入 1 叫 set（置位），使它存入 0 叫 reset（复位），因此称为 R-S。

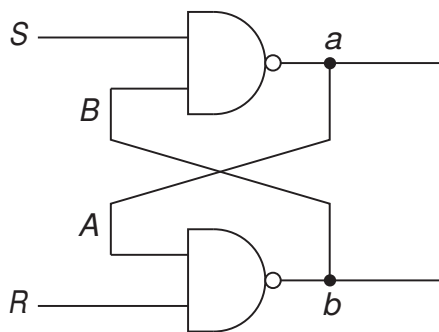


图 3.18 R-S 锁存器

静止状态 锁存器正在保存 0 或 1、没有任何操作试图改变它时，称为**静止状态** (quiescent state)。此时 $S = R = 1$ 。图中的 a 是当前存储的值，也称为锁存器的输出。

先假设 $a = 1$ ，于是 $A = 1$ 。由于静止状态下 $R = 1$ ，下方 NAND 门输出 $b = 0$ ；因此 $B = 0$ ，又使上方 NAND 门输出 $a = 1$ 。只要 S 、 R 保持为 1，电路状态就不变，锁存器继续存储 1。

反之，假设 $a = 0$ ，则 $A = 0$ ，使 $b = 1$ ，进而 $B = 1$ 。结合 $S = 1$ ，得到 $a = 0$ 。同样，只要 S 、 R 都保持为 1，状态就不变，此时锁存器存储 0。

把锁存器置为 1 或 0 保持 $R = 1$ ，让 S 短暂变为 0，就可以存入 1。保持 $S = 1$ ，让 R 短暂变为 0，就可以存入 0。为使 R-S 锁存器正常工作，绝不能让 S 、 R 同时为 0。

我们用 set 表示把变量设为某个值，例如“设为 0”或“设为 1”；也常用 clear（清零）专门表示设为 0。

让 S 短暂为 0，会使 $a = 1$ ，进而 $A = 1$ 。由于 R 也是 1，得到 $b = 0$ ，进而 $B = 0$ ，使 a 继续为 1。此后把 S 恢复为 1，为什么不影响 a ？因为 $B = 0$ ，而 NAND 门只要一个输入为 0，就必定输出 1。因此， S 恢复为 1 后，锁存器仍会长期保存 1。同理，让 R 短暂为 0，就能将锁存器清零。

如果允许 S 、 R 同时为 0，输出 a 、 b 就都会为 1；之后的最终状态将取决于构成逻辑门的晶体管的电学性质，而不是所执行的逻辑。这种情况下，电学性质如何决定最终状态，只好留待以后学期讨论了。

最后，数字电路刚通电时，锁存器可能处于 0 或 1 中的任一状态。究竟是哪一种并不重要，因为我们总要先把它明确设为 0 或 1，然后才使用其中的信息。

3.4.2 门控 D 锁存器

要使锁存器有用，就必须控制何时置位、何时清零。一个简单办法是采用门控锁存器。

图 3.19 的门控 D 锁存器 (gated D latch) 由 R-S 锁存器外加两个 NAND 门构成。只有 WE 有效，即 WE 为 1 时，才允许把 D 的值存入。WE 是 write enable (写使能) 的缩写。

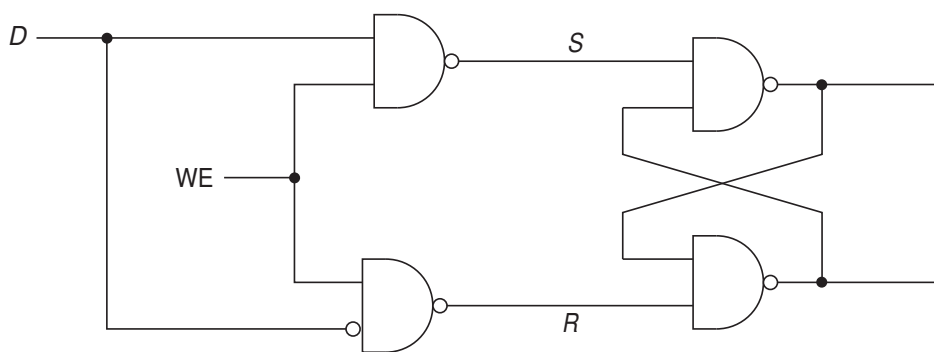


图 3.19 门控 D 锁存器

WE 无效，即为 0 时， S 、 R 都为 1，因此 R-S 锁存器原有的值保持不变。WE 短暂为 1 时， S 、 R 中恰有一个变成 0，取决于 D ：如果 $D = 1$ ，则 $S = 0$ ；如果 $D = 0$ ，下方 NAND 门的两个输入都为 1，使 $R = 0$ 。前者把锁存器置为 1，后者把它置为 0，所以存入的恰好是 D 。WE 恢复为 0 后， S 、 R 都恢复为 1，存入的值就保留下来。

3.5 存储器的概念

现在，我们已经具备介绍电子数字计算机中最重要结构之一——存储器——所需的全部工具。第 4 章将说明它在计算机处理信息的基本框架中的位置；本书后续内容，以及你今后使用计算机的经历，都会不断体现存储器的重要性。

存储器包含许多个位置，通常数量很大。每个位置都能唯一标识，并能存储一个值。每个位置的唯一标识称为地址 (address)，每个位置存储的位数称为寻址粒度 (addressability)。

例如，笔记本电脑广告可能写着“配备 2 gigabytes 存储器”，通常简写成 2 GB 或 2 Giga。按照下面介绍的含义，这表示有约二十亿个存储位置，每个位置存储一个字节。

3.5.1 地址空间

全部可唯一标识的位置的总数称为存储器的**地址空间**（address space）。例如，2 GB 存储器包含约二十亿个可唯一标识的位置。

“二十亿”只是近似值。计算机中的其他信息用 0 和 1 表示，存储位置也用二进制地址标识。有 n 位地址，就能唯一标识 2^n 个位置。十位给出 1024 个位置，约为一千；20 位给出 2^{20} 个位置，约为一百万；30 位给出 2^{30} 个位置，约为十亿。

本书沿用以 kilo 表示 2^{10} 、mega 表示 2^{20} 、giga 表示 2^{30} 的习惯。因此，2 giga 对应 31 位地址能够标识的位置数，地址空间为 2^{31} ，精确值是 2 147 483 648，而不是 2 000 000 000，尽管口头上常简称“二十亿”。

3.5.2 寻址粒度

寻址粒度是每个存储位置存放的位数。2 GB 存储器有 2 147 483 648 个位置，每个位置可存一个字节，即八位。大多数存储器是**按字节寻址**的。这有历史原因：许多计算机最初用于数据处理，而一次键盘字符输入对应一个八位 ASCII 码，正如第 2 章所学。按字节寻址时，每个 ASCII 字符占一个存储位置。能够唯一标识每个字节，就容易单独修改已存储的信息。

许多专为大规模科学计算设计的计算机则按 64 位寻址，因为科学计算中的数经常采用 64 位浮点表示。第 2 章介绍过浮点类型。既然常用的数需要 64 位，那么让每个可唯一标识的位置恰好存放一个这样的数，是一种合理设计。

3.5.3 一个四字、每字三位的存储器

图 3.20 的存储器大小为 $2^2 \times 3$ 位，即地址空间有四个位置、寻址粒度为三位。选择四个位置之一需要两位地址，记为 $A[1:0]$ ；每个位置存储的三位数据记为 $D[2:0]$ 。

这里的 $A[\text{high}:\text{low}]$ 和 $D[\text{high}:\text{low}]$ 表示位编号从右向左递增，最右边为第 0 位。 $[\text{high}:\text{low}]$ 表示一段共 $\text{high} - \text{low} + 1$ 位的序列，high 是最左边、最高位的编号，low 是最右边、最低位的编号。

访问存储器需要对地址译码。地址译码器以 $A[1:0]$ 为输入，使四个输出中对应被访问位置的那一根**字线**（word line）有效。图中每一行对应一个三位字，所以称为字线。

读取时，施加地址使所需字线有效。每一位先与自己的字线做 AND，再与其他字中相同位位置的结果做 OR。一次只有一根字线有效，因此这实际上构成了 mux：译码器的输出为各条位线提供选择作用。所需的字就从 $D[2:0]$ 读出。

图 3.21 展示读取位置 3 的过程。地址 3 的编码为 11，译码后最下面的字线有效，

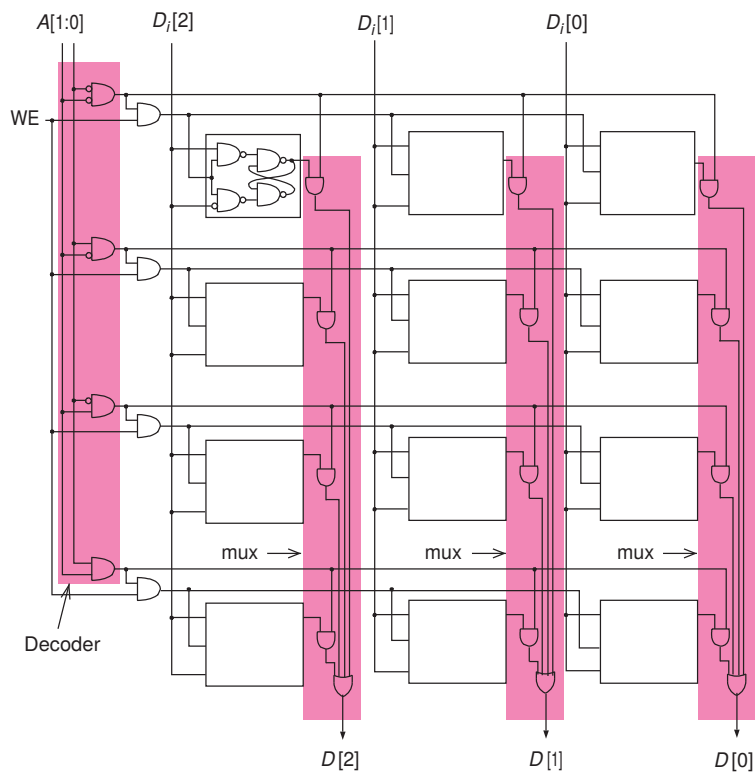


图 3.20 一个 $2^2 \times 3$ 位存储器

其余三根为 0。位置 3 存储 101，这三位分别与有效字线相与，产生 101，送到三个输出 OR 门。其他字线无效，所以 OR 门的其他输入都为 0。最终 $D[2:0]=101$ ，即读出了位置 3 的内容。

写入采用类似方法。把 $A[1:0]$ 送到地址译码器，使正确的字线有效；同时使 WE 有效，就可以把 $D[2:0]$ 的三位写入该字线对应的三个门控锁存器。

3.6 时序逻辑电路

第 3.3 节讨论了处理信息、作出判决的数字逻辑结构，其输出只依赖当前输入，例如 mux、译码器和全加器。这些组合逻辑电路没有“过去”的概念，也无法存储此前发生过的任何事情。第 3.4、3.5 节则介绍了能够存储信息的结构：基本存储元件和一个简单的 $2^2 \times 3$ 位存储器。

本节把两者结合起来，讨论既能处理信息、又能存储信息的结构。它们作出判决时，不仅依赖当前输入，还依赖过去发生过的事情，称为**时序逻辑电路**（sequential logic circuit）。它们与组合逻辑电路的区别，就在于具有能够保留历史信息的存储元件。

图 3.22 是时序逻辑电路的框图。注意其中的存储元件：输出可以同时依赖当前输入和元件中存储的值，后者反映过去的历史。

时序逻辑电路用于实现一类非常重要的机制——**有限状态机**（finite state machine, FSM）。几乎所有工程领域都会使用有限状态机，例如电气、机械和航空系统的控制

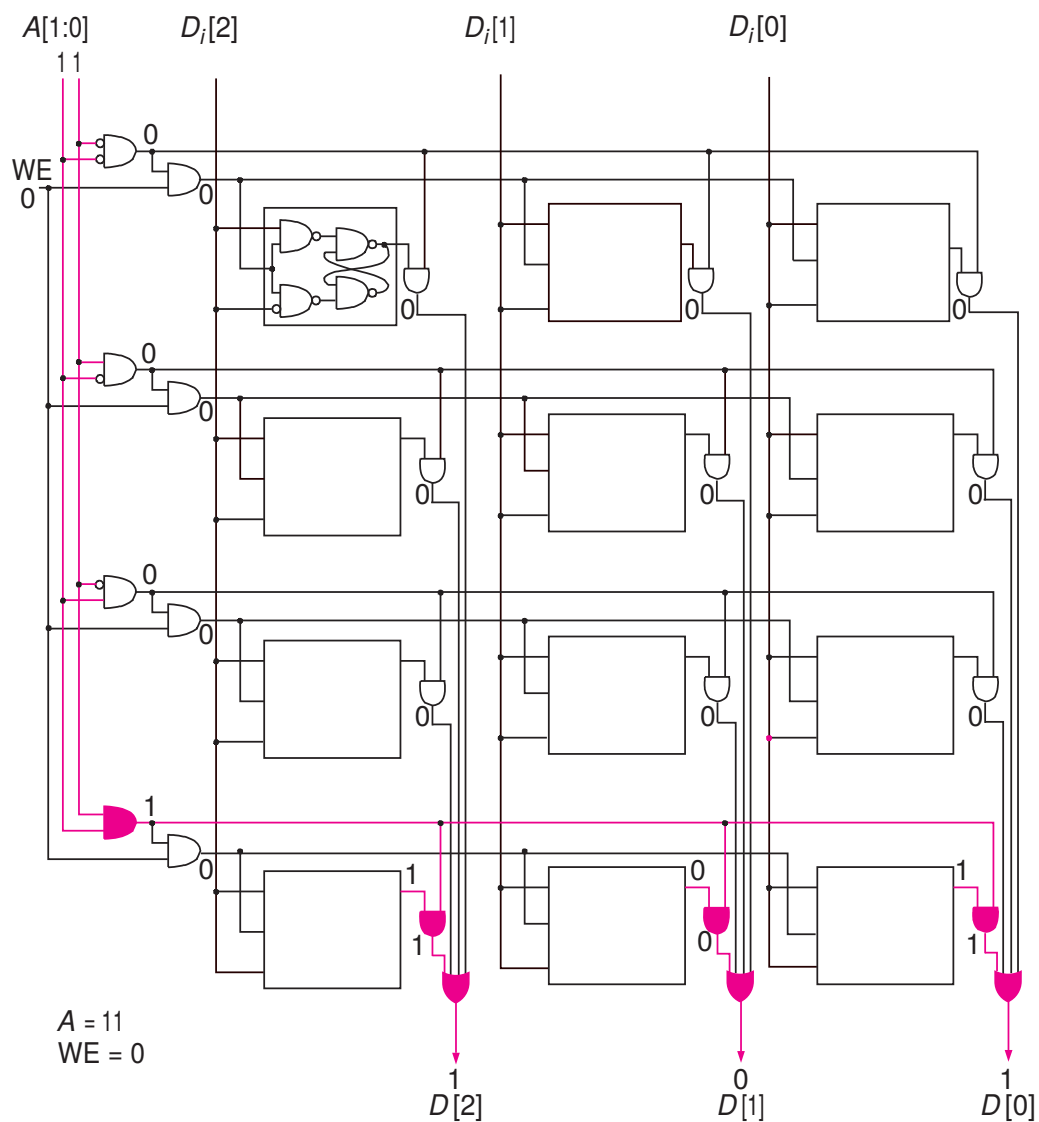
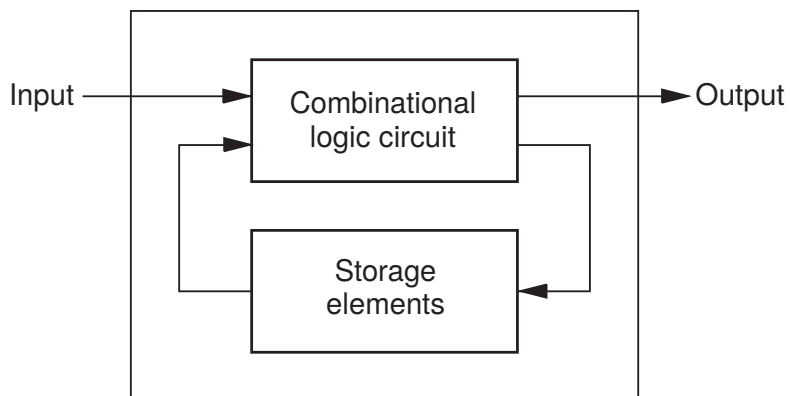
图 3.21 读取 $2^2 \times 3$ 位存储器中的位置 3

图 3.22 时序逻辑电路框图

器。交通灯控制器决定显示红、黄还是绿，要依据当前亮着的灯（历史信息），以及道路感应线、记录当前灯已亮多久的定时器、监测交通的光学设备等传感器输入。第4章介绍冯·诺依曼模型时会看到，有限状态机处于计算机的核心，控制着计算机的信息处理过程。

3.6.1 一个简单例子：密码锁

一个简单例子可以说明组合逻辑与时序逻辑的区别。假设要锁住自行车，又不想随身带钥匙，常见办法是使用密码锁：记住一组“组合”就可以开锁。图 3.23 展示两种常见形式。

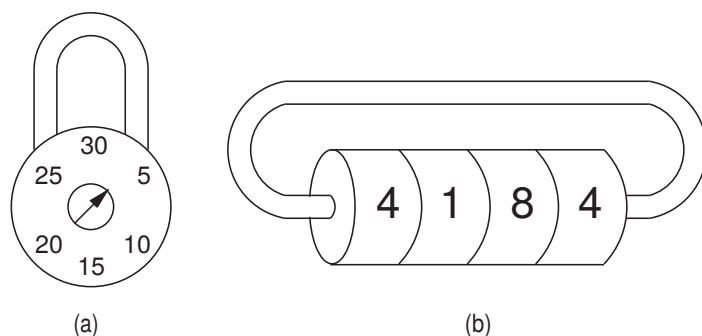


图 3.23 两种密码锁

图 (a) 的转盘圆周上均匀标有 0 到 30。假设密码为 R13-L22-R3，开锁时先向右（顺时针）转两整圈，再继续转到 13；然后向左（逆时针）转一整圈，再继续转到 22；最后向右转到 3，锁便打开。

关键是转动顺序。如果先向右转两圈停在 22，再向左转一圈停在 13，最后向右停在 3，锁不会开。虽然最后都停在 3，R22-L13-R3 也使用了与正确密码相同的三个数，但锁会保存先前的转动信息，依据历史操作是否构成正确序列决定开不开。

图 (b) 通常有四个轮，每轮标有 0 到 9。当四个数字正确对齐时，锁就打开。在这里，密码只是四个数字的组合，开不开与各轮过去怎么转完全无关，只取决于当前的四个数字。这是组合结构的简单例子。

有趣的是，日常英语把两者都叫 combination lock（组合锁）。严格说，只有 (b) 才是“组合”锁，(a) 更适合叫“时序”锁！

3.6.2 状态的概念

图 3.23(a) 的锁要正确工作，就必须跟踪开锁之前的操作序列，区分 R13-L22-R3 与所有其他序列。例如，R22-L13-R3 和 R10-L22-R3 都不能开锁。它必须识别以下几种相关情况：

- A. 锁未打开，尚未完成任何相关的正确操作。
- B. 锁未打开，刚完成 R13。

- C. 锁未打开，刚依次完成 R13、L22。
- D. 刚依次完成 R13、L22、R3，锁已打开。

我们把这四种情况标为 A、B、C、D，称为锁的**状态**（state）。状态是计算机工程乃至几乎所有工程领域中的重要概念。一个机制，更一般地说一个系统的状态，就是明确描述其全部相关项目的一份快照。

要点 系统的状态，是在某一时刻对系统全部相关要素所作的快照。

这把锁共有四种状态：已经打开（D）；或者尚未打开，已完成零个（A）、一个（B）、两个（C）正确操作。这涵盖了所有可能情况。为什么恰好需要四个状态？完成一次 R_n 或 L_n 操作后，你能想出一种 A、B、C、D 都不能描述、必须增加第五个状态快照吗？

许多熟悉的系统都可以用状态描述。篮球比赛的状态可以由记分牌表示。图 3.24 中，Texas 73 分，Oklahoma 68 分，下半场还剩 7 分 38 秒，进攻计时还剩 14 秒，Texas 控球，两队各有四次全队犯规。这就是比赛某一时刻的快照。若 12 秒后 Texas 投中一个两分球，新状态的记分牌将显示 Texas 75、Oklahoma 68，比赛还剩 7 分 26 秒，进攻计时恢复为 25 秒，球权归 Oklahoma。

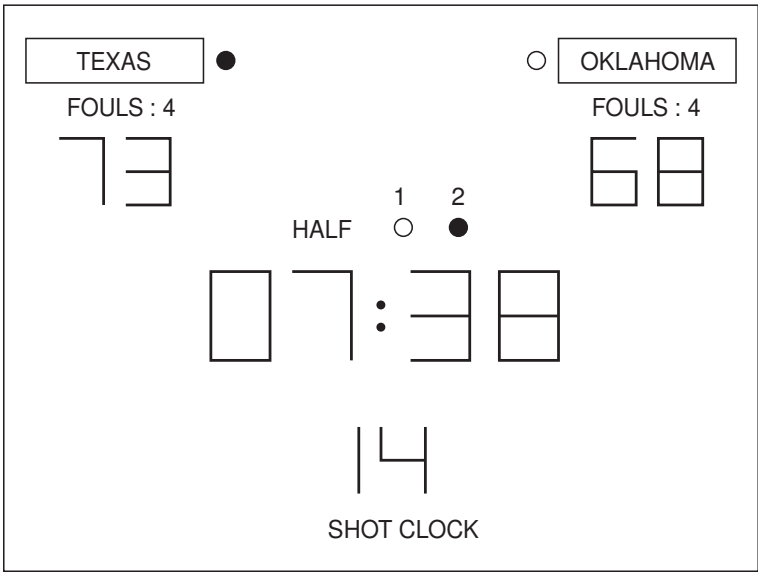


图 3.24 一个状态的例子

井字棋也可以用状态描述。两个人，或者这里的人与计算机，轮流在九个空格中放置符号：人放 X，计算机放 O，人先走。谁先让自己的三个符号沿横、竖或对角线连成一条直线，谁就获胜。这里把每次计算机请人走下一步时的棋盘快照作为状态。

图 3.25(a) 是双方都没走时的初始状态。(b) 是一种可能的第二步前状态：人先把 X 放在左上角，计算机把 O 放在正中。(c) 是一种可能的第三步前状态：人把第二个 X 放在右上角，计算机把第二个 O 放在上方中间。

最后再看一台很老的饮料机：一瓶饮料卖 15 美分，只收 5 美分和 10 美分硬币，而且不能找零。状态可以用已经投入的金额，以及锁是否打开、能否拿走瓶子来描述。

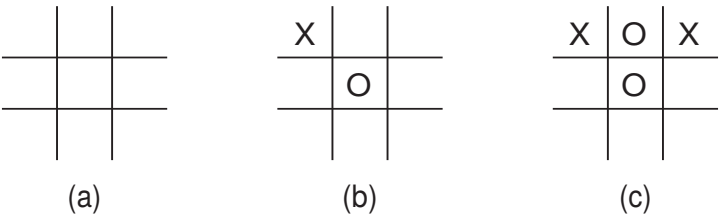


图 3.25 井字棋机器的三个状态

这里只有三种状态：A，锁已打开，可以拿走（或已经拿走!）饮料；B，锁未开，已投入 5 美分；C，锁未开，已投入 10 美分。

3.6.3 有限状态机及其状态图

状态是系统在某一时刻全部相关部分的快照，其他时刻则可能处于其他状态。前面介绍了密码锁、篮球比赛、井字棋机器和 15 美分一瓶的老式饮料机。这些系统的行为都可以用有限状态机规定，并用**状态图**（state diagram）表示。

有限状态机包含五个要素：

- 1. 有限个状态；
- 2. 有限个外部输入；
- 3. 有限个外部输出；
- 4. 对全部状态转换的明确规定；
- 5. 对每个外部输出值由什么决定的明确规定。

状态集合包含系统可能处于的全部情形；每个状态转换说明从一个状态到另一个状态需要什么条件。下面分别考察这四个系统。

密码锁 状态图是一种方便的有限状态机表示。图 3.26 中的四个圆分别对应 A、B、C、D，当前处于哪一个，取决于从任意初始情况走向开锁已经取得多少进展。

外部输入是 R13、L22、R3，以及向右转到非 13、向左转到非 22、向右转到非 3 的情况。外部输出是“打开”或“未打开”，一个逻辑变量就够了。A、B、C 状态下锁关闭，D 状态下锁打开。

所有转换由箭头明确给出，专业术语叫**弧**（arc）。箭头指出从哪个状态出发、到哪个状态结束，分别称为**当前状态**（current state）和**下一状态**（next state）。这个密码锁有八个状态转换，每个转换都标有触发它的输入。例如，R13 使 A 转到 B。

一个当前状态通常有多条通往下一状态的弧，发生哪一次转换取决于当前状态与当前外部输入的组合。例如，处于 B 且输入为 L22 时，下一状态是 C；处于 B 而输入不是 L22 时，下一状态是 A。

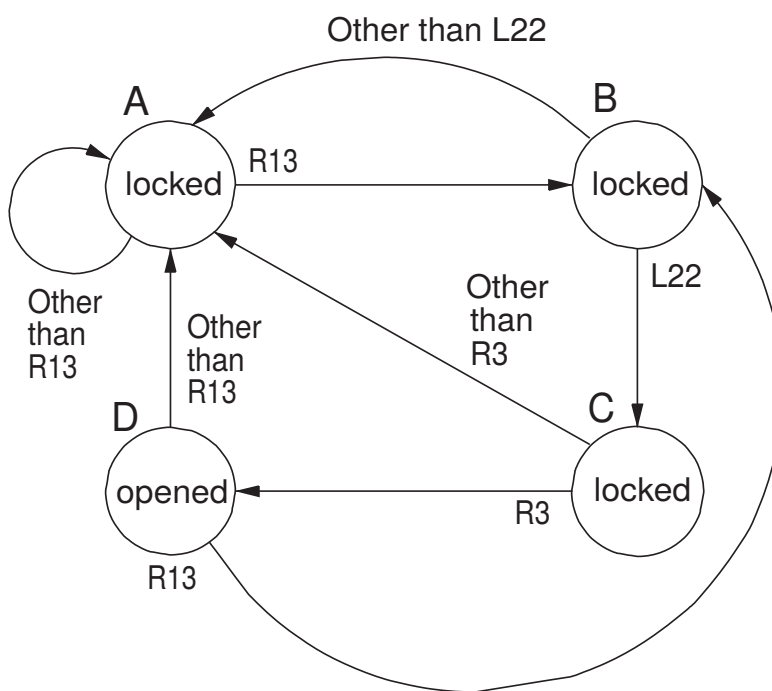


图 3.26 图 3.23(a) 中密码锁的状态图

系统输出也可以同时依赖当前状态和当前外部输入。不过，对这把锁来说，输出只由状态决定：A、B、C 表示“锁住”，D 表示“打开”。本书研究的系统，其输出值都由当前状态单独规定。

一台很老的饮料机 图 3.27 是饮料机的状态图。它只有三个状态：已投入 5 美分、已投入 10 美分、已投入至少 15 美分。投入 5 或 10 美分硬币是导致转换的输入。输出只与状态有关：B、C 下金额不够，锁关闭；A 下至少投够了 15 美分，锁打开，可以取走饮料。

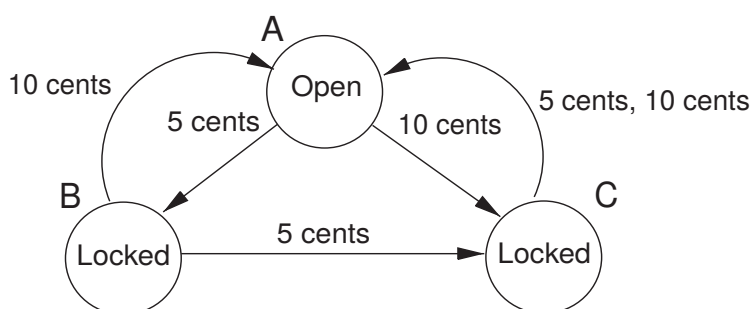


图 3.27 饮料机的状态图

篮球比赛 同样可以为比赛画状态图，每个状态是记分牌的一种可能配置。裁判吹哨或球权转移等事件导致状态转换，前面已展示 Texas 投中两分球的例子。显然，比赛的状态数量会非常庞大。

不过，与任意两状态间都画弧的数量相比，合法转换很少。例如，不可能用一次输入把 Texas 68、Oklahoma 67 变成 Texas 75、Oklahoma 91。输入是上次转换后场上发生的事件，例如 Texas 得两分、Oklahoma 得三分、Texas 抢断、Oklahoma 抢下 Texas 投篮后的篮板等。

输出是比赛结果，有三种：比赛仍在进行、Texas 胜、Oklahoma 胜。能否从 Texas 30、Oklahoma 28 的状态画一条弧到双方 30 平？能否存在这样两个比分分别为 30-28 和 30-30 的状态，却不能在它们之间画弧？

井字棋机器 在人机对局中，每个状态是提示人把 X 放入空格时的棋盘。图 3.25(a) 到 (b) 的转换，是人把 X 放左上、计算机随后把 O 放正中；(b) 到 (c) 则是人把 X 放右上、计算机随后把 O 放上中。

九个格子各可为 X、O 或空，因此状态数少于 3^9 ；实际上，由于规则的各种限制，会远少于这个上限。输入有九种，对应人可以放 X 的九个格子。输出有三种：比赛进行中、人获胜、计算机获胜。

3.6.4 同步有限状态机

到目前为止，状态转换在事件发生时才发生，并没有固定时间安排。例如，向饮料机投入 5 美分后，可以再等 10 秒，也可以等 10 分钟，再投下一枚。机器不会抱怨；没到 15 美分就不出饮料，但会耐心等待。连续输入之间没有固定间隔，前面四个系统都如此。我们称它们为**异步**系统，因为没有什么机制同步规定状态转换必须何时发生。

但几乎没有计算机这样工作。计算机通常是**同步**的：状态转换按相同、固定的时间单位依次发生，由同步有限状态机控制。控制计算机的具体转换留到第 4 章以后，本章先设计一个更简单、同样由同步有限状态机控制的交通控制器。

上述异步状态机与控制计算机的同步状态机有一个共同点：一次执行一个状态转换，逐步接近目标。密码锁在操作正确时逐步走向打开；饮料机逐步走向让你喝到饮料；计算机则通过处理人们编写的程序，逐步走向问题的解答。

3.6.5 时钟

同步有限状态机每经过相同的固定时间间隔，就从当前状态转到下一状态。**时钟**电路承担了控制这种同步行为的部分责任。

时钟电路产生一个信号，通常就叫时钟，其电压在 0 V 和某个规定的固定电压之间交替。用数字逻辑的语言说，就是在 0 和 1 之间交替。图 3.28 展示时钟信号随时间的变化。每一段重复出现的相同间隔称为**时钟周期** (clock cycle)：从一次 0 到 1 的跳变开始，到下一次 0 到 1 的跳变结束。

从第 5 章开始会看到，计算机每个时钟周期都可以完成一部分有用的工作。说一台笔记本以 2 GHz 运行，就是说每秒有二十亿个时钟周期，每周期只有半纳秒，因而每秒完成二十亿份这样的工作。同步有限状态机每周期进行一次状态转换。

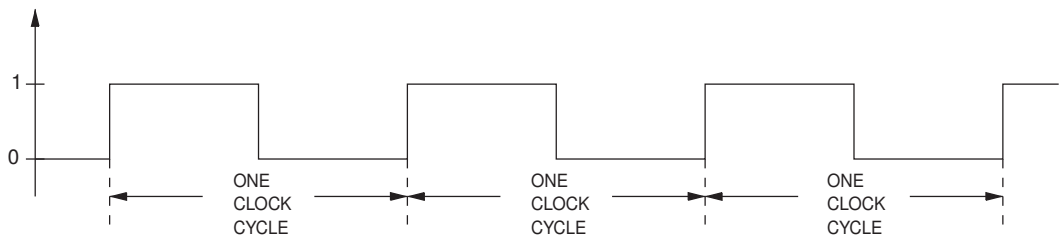


图 3.28 时钟信号

下面用交通信号控制器展示时钟怎样逐周期控制状态转换。在电子电路实现中，从一个状态到下一个状态的转换发生在每个时钟周期的开始。

3.6.6 例子：危险警示牌

许多电气、机械和航空系统由同步有限状态机控制。本节设计一个交通危险警示牌所需的完整控制逻辑。图 3.29 是放在公路上的标志，写着 DANGER, MOVE RIGHT，意思是“危险，向右行驶”。它有五盏灯，编号为 1 到 5。

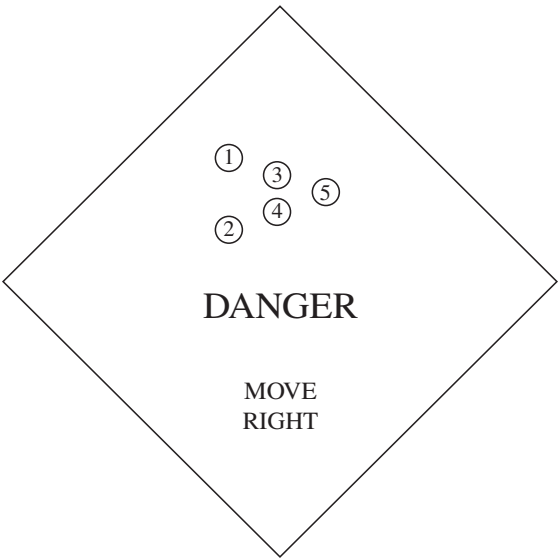


图 3.29 交通危险警示牌

这里的同步有限状态机，也就是控制器，要指挥这五盏灯闪烁，提醒司机向右移动。控制器带一个开关。开关打开时，第一个时间单位所有灯灭；下一个时间单位灯 1、2 亮；再下一个时间单位灯 1、2、3、4 亮；然后五盏全亮。接着重复：全灭，1、2 亮，1、2、3、4 亮，依此类推。每个时间单位为一秒。接近标志的司机会清楚地看到向右移动的指示。开关保持打开，灯就在四种状态间循环；开关关闭，所有灯熄灭并保持熄灭。

控制器的状态图 图 3.30 给出四个状态，分别对应四种亮灯情况。输出，即每盏灯是否亮，由当前状态决定。开关打开（输入为 1）时，每隔一秒转到下一状态；开关关闭（输入为 0）时，下一状态总是 A，即“全灭”。

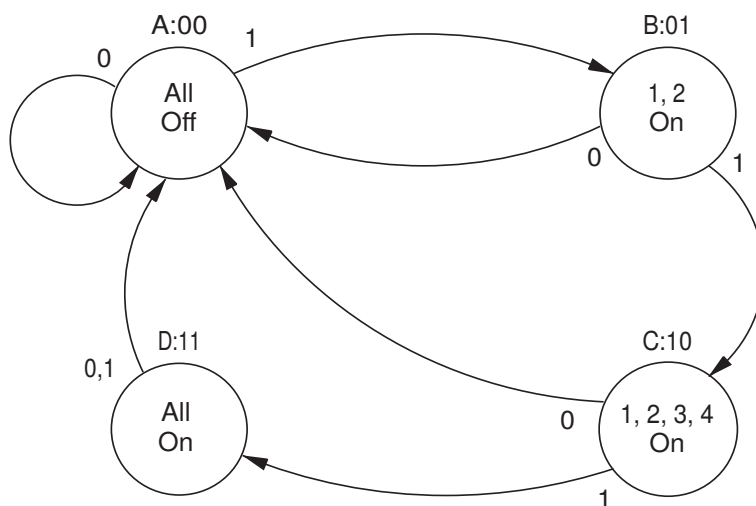


图 3.30 危险警示牌控制器的状态图

控制器的时序逻辑电路 图 3.22 是通用框图，图 3.31 是控制这些灯的具体框图，需要注意四点。

首先，有两个外部输入：开关和时钟。开关决定在四种状态间循环，还是转到全灭的 A；时钟通过控制存储元件，控制 A 到 B、B 到 C、C 到 D、D 到 A 的转换。

其次，有两个存储元件保存状态信息。每个能存一位，两位就能标识四种状态：A 为 00，B 为 01，C 为 10，D 为 11。存储元件 2 保存高位，元件 1 保存低位。例如，元件 2 为 0、元件 1 为 1 时，控制器处于 B。

第三，组合逻辑电路 1 表明，灯的亮灭由这两个存储元件控制，即由有限状态机当前状态决定。

最后，组合逻辑电路 2 表明，下一状态取决于两个存储元件以及开关。开关打开时，它根据当前存储的状态产生下一状态。

组合逻辑 图 3.32 实现了组合逻辑电路 1 和 2。控制器需要两组输出：控制灯的外部输出，以及送往两个状态存储元件输入端的内部输出。

先看灯的控制。只需要三个输出：灯 5 由标为 V 的 AND 门控制，因为它只在状态 11 下亮；灯 3、4 由标为 X 的 OR 门控制，因为它们在 10 和 11 两个状态下亮。为什么灯 1、2 由 W 控制？参见习题 3.48。

再看决定下一状态的内部输出。下一状态为 10 或 11 时，元件 2 应在下一周期置为 1；这只发生在开关打开且当前状态为 01 或 10 时，因此它由 OR 门 Y 的输出控制。为什么元件 1 的下一状态由 OR 门 Z 控制？请结合图中的状态转换思考。

两个存储元件 为了正确工作，开关打开时，状态必须每秒只转换一次。

用门控锁存器会出现的问题 假如两个存储元件是门控 D 锁存器，会怎样？当写使能信号，也就是时钟，为 1 时，OR 门 Y 、 Z 的输出会立即改变锁存器中存储的位。新

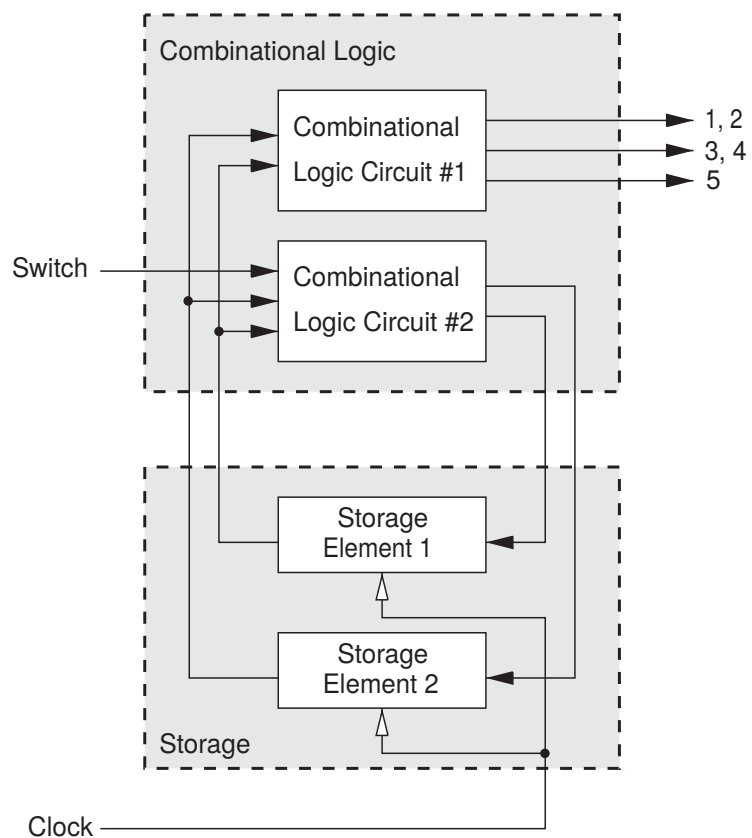


图 3.31 危险警示牌控制器的时序逻辑电路

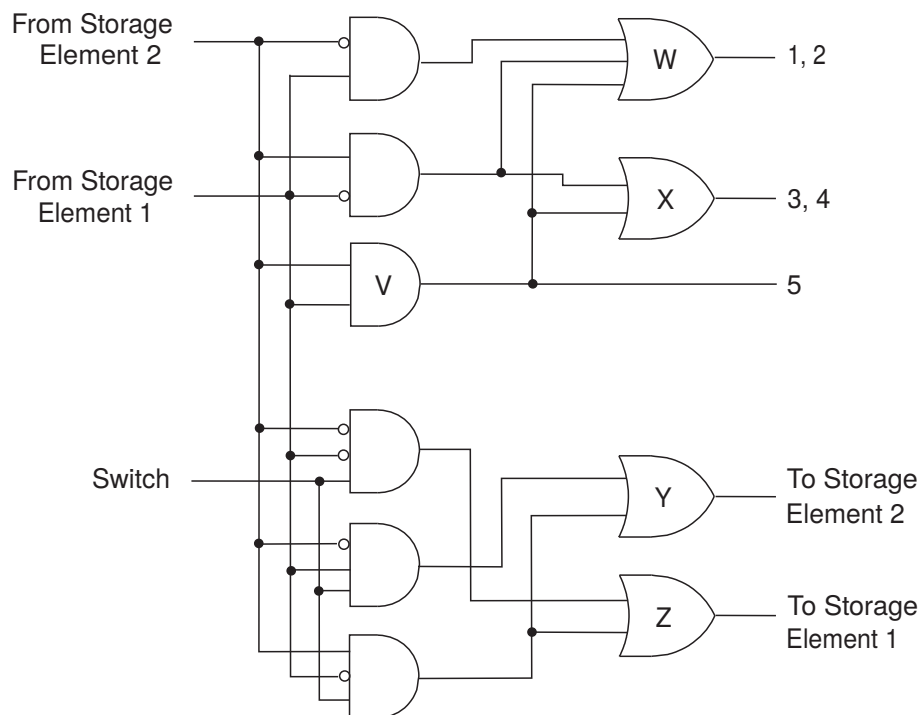


图 3.32 组合逻辑电路 1 和 2

值又改变向 Y 、 Z 提供输入的三个 AND 门的输入，产生新的 OR 输出；这些输出又写入锁存器，导致下一轮变化。只要写使能保持有效，这个过程就不断重复。

结果是，我们无法知道下一周期状态机究竟处于什么状态。甚至当前周期中，存储元件的值也会变化得太快，灯会不规则地闪动。

问题出在门控 D 锁存器的行为。我们希望 Y 、 Z 算出的下一状态，到当前周期结束时才生效，此前当前状态一直不变。但门控 D 锁存器在写使能有效时会立即响应输入，所以不能直接作为这里的状态存储元件。所需元件必须允许在整个当前周期读取当前状态，只在下一周期开始时写入下一状态。

用触发器解决问题 值得再强调一次：一个周期内的工作同时涉及读取和写入同一个变量；整个周期都应能读取原值，而写入应发生在周末。触发器（flip-flop）能做到这一点。

图 3.33 是主从触发器（master/slave flip-flop），由两个门控 D 锁存器构成，分别称为主锁存器和从锁存器。时钟为 0 时，主锁存器的写使能为 1；时钟为 1 时，从锁存器的写使能为 1。

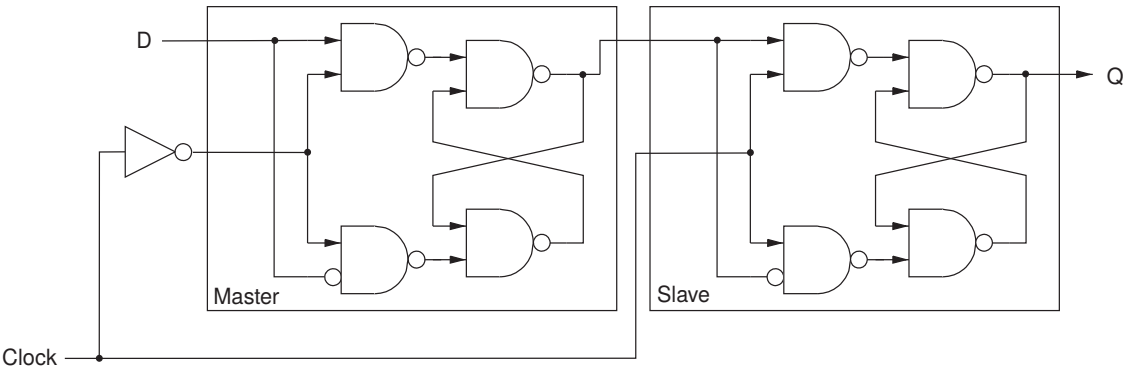


图 3.33 主从触发器

图 3.34 的时序图（timing diagram）展示它为什么能解决问题。图中时间从左向右流逝，第 n 周期从时刻 1 开始，到时刻 4 结束；第 $n + 1$ 周期从时刻 4 开始。把第 n 周期分成前半周期 A、后半周期 B，并考察标出的 1、2、3、4 四个时刻。

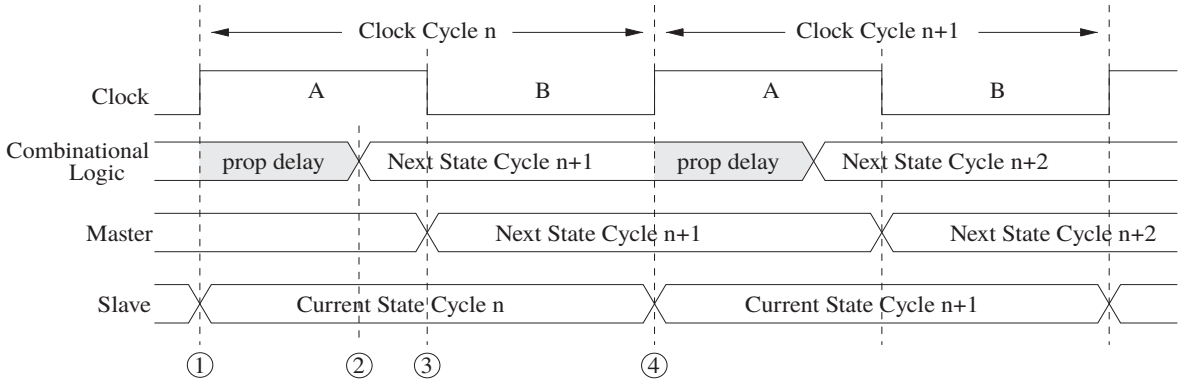


图 3.34 主从触发器的时序图

每个周期开始时,存储元件的输出来自两个从锁存器。时刻 1 起,这些值送入 AND 门,经过组合逻辑后, Y 、 Z 在时刻 2 产生下一状态。图中标出了**传播延迟**(propagation delay),即组合逻辑产生结果所需的时间。虽然下一状态在前半周期 A 内已经产生,但主锁存器的写使能为 0,不能写入。

后半周期 B 开始的时刻 3,时钟变为 0,主锁存器写使能为 1,可以写入下一状态。但此时从锁存器写使能为 0,所以它们不能接收主锁存器中的新值。

第 $n+1$ 周期开始的时刻 4,从锁存器写使能变为 1,存入前一周期组合逻辑产生的下一状态,它便成为第 $n+1$ 周期的当前状态。与此同时,主锁存器写使能变为 0,其内容不能再变。因此,虽然从锁存器写使能为 1,看到的主锁存器内容却保持不变,它们自己也就不再变化。

因此,从锁存器在整个周期内保存系统当前状态,并为组合逻辑中的六个 AND 门提供输入。它们只在周期开始时存入上一周期算出的下一状态,之后整个周期保持不变:前半周期 A,主锁存器不能变;后半周期 B,从锁存器写使能为 0,自己不能变。

与此同时,主锁存器在后半周期 B 可以保存组合逻辑产生的下一状态,但要到下一周期开始,才能把它送进从锁存器,作为新周期的状态。

3.7 后续内容预览: LC-3 的数据通路

第 5 章将规定一台称为 LC-3 的计算机,你将有机会编写在它上面执行的程序。第 3 章的最后,来看看图 3.35 的 LC-3 数据通路。

数据通路包含计算机核心中共同处理信息的全部逻辑结构。此刻,这张图大概有些吓人,但不用担心,你还不必分析它,那是第 5 章的事。这里展示它,是为了让你发现:计算机的许多基本构件已经很熟悉了。例如,图中有五个 MUX,你已经知道它们如何工作;还有一个加法器(用内部带加号的 ALU 符号表示)和一个 ALU,你也知道它们如何用门构成。

还有一个尚未明确介绍的元件:**寄存器**。寄存器就是一组 n 个触发器,共同存储一个 n 位值。图中的 PC、IR、MAR、MDR 都是 16 位寄存器,各存 16 位信息。REG FILE 包含八个寄存器,每个存 16 位。每个触发器存一位,所以每个这样的寄存器需要 16 个触发器。图中还有 N、Z、P 三个一位寄存器,各只需一个触发器。寄存器可以具有任何所需宽度,只取决于待存值需要多少位。

一种实现办法是使用主从触发器。图 3.36 用四个主从触发器构成四位寄存器。通常需要触发器而非锁存器,因为往往既要在整个周期读取寄存器,又要在同一周期末存入新值。图中,当前周期存储的值是 $Q_3Q_2Q_1Q_0$;周期末写入 $D_3D_2D_1D_0$ 。

图 3.35 的箭头代表把值从一个结构传送到另一个结构的导线。多数箭头上有一条短斜线,旁边的数字表示导线数量,也就是传送的位数。例如,PC 到 ADDR1MUX 一个输入端的箭头,表示传送 16 位。

第 5 章会解释,为什么必须按图中的方式连接这些部件,才能执行 LC-3 程序。现在,只需享受认出这些熟悉部件的感觉。第 4、5 章将再次提高抽象层次,把它们组

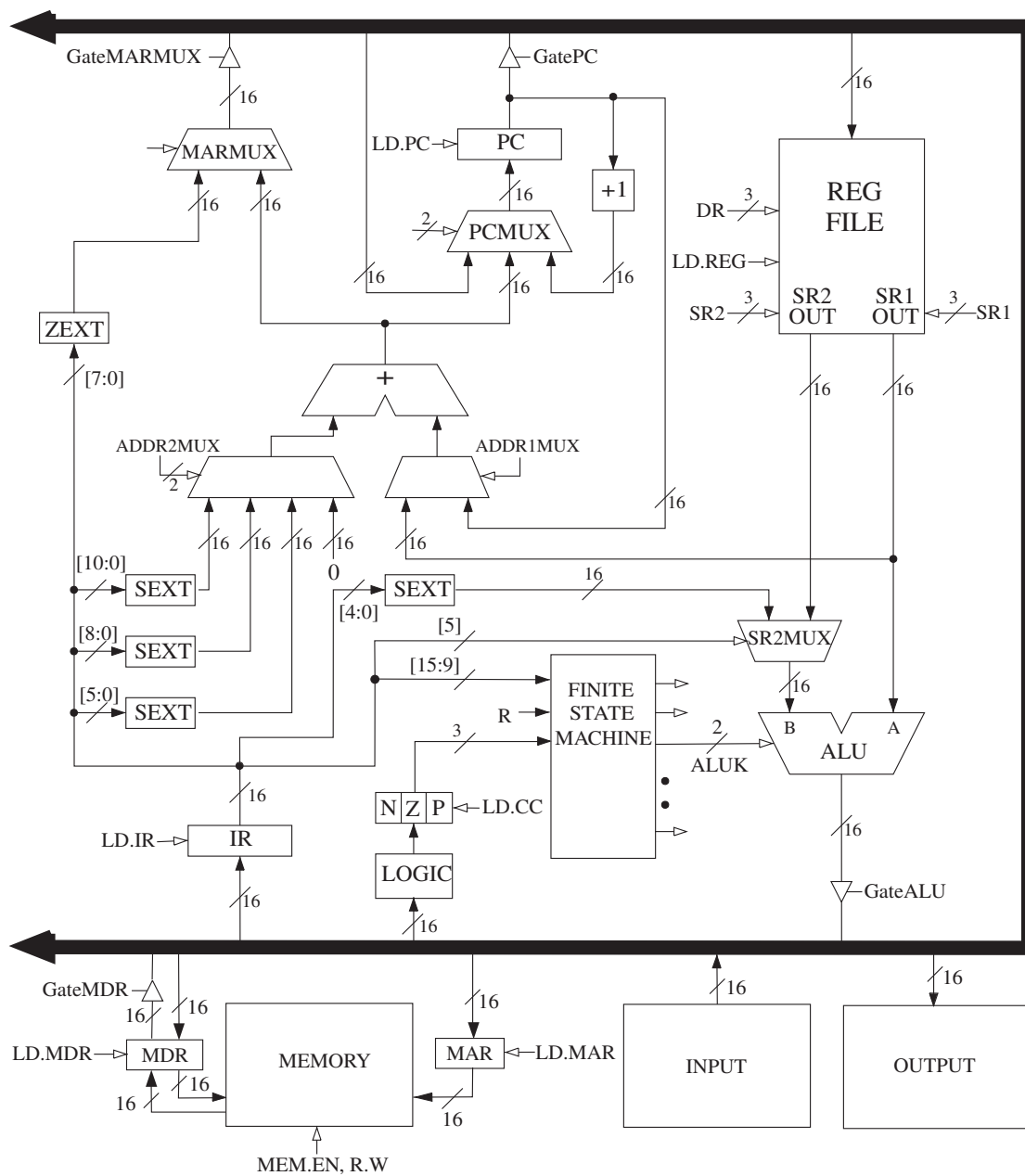


图 3.35 LC-3 计算机的数据通路

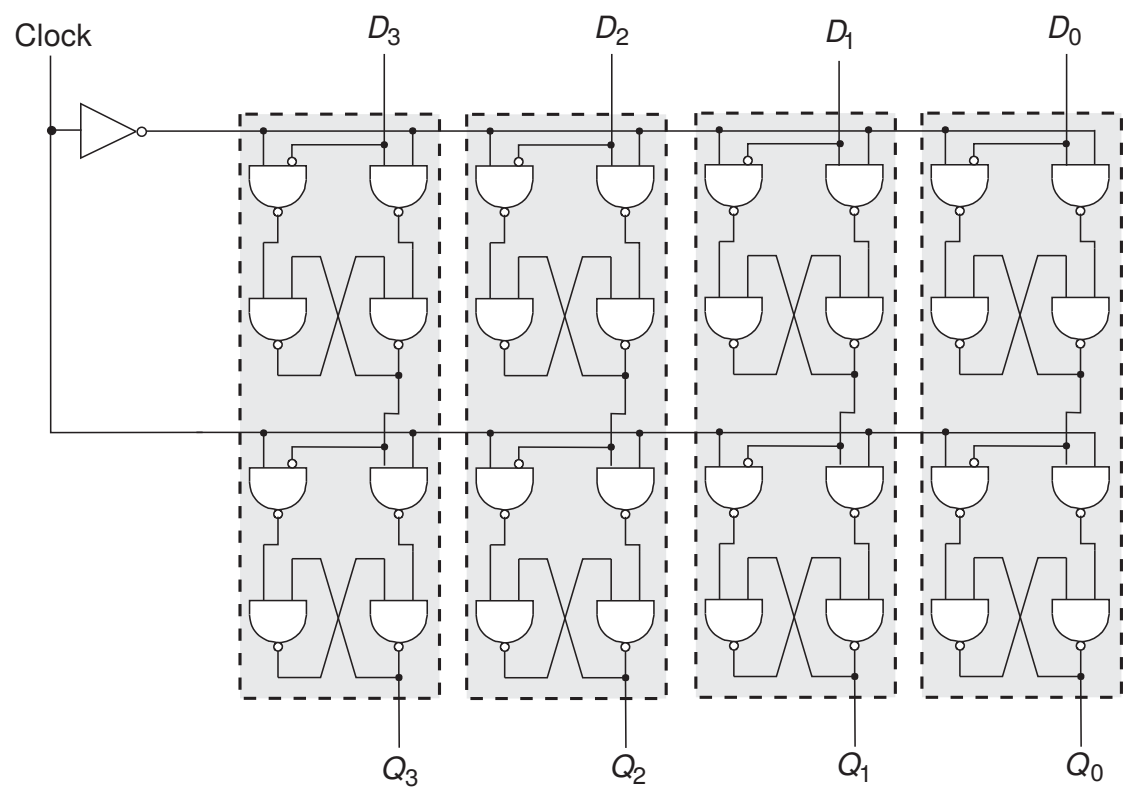


图 3.36 四位寄存器

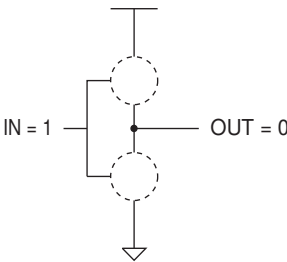
成一台能够工作的计算机。

习题

3.1 在下表中填写各种晶体管表现为开路还是闭合电路。

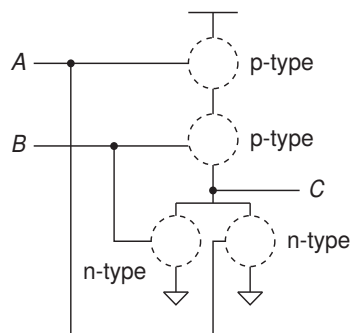
	N 型	P 型
栅极 = 1		
栅极 = 0		

3.2 在下图缺失处画上导线或保持断开，使输入 IN 为逻辑 1 时，输出 OUT 为逻辑 0。



3.3 两输入 AND 和两输入 OR 都是两输入逻辑函数。一共可能有多少种不同的两输入逻辑函数？

3.4 在下图缺失处画上导线或保持断开，使输出 C 为逻辑 1。再描述一组使 C 为逻辑 0 的输入，按这组输入对应的状态补全电路。



3.5 为图 3.37 的晶体管级电路完成真值表。

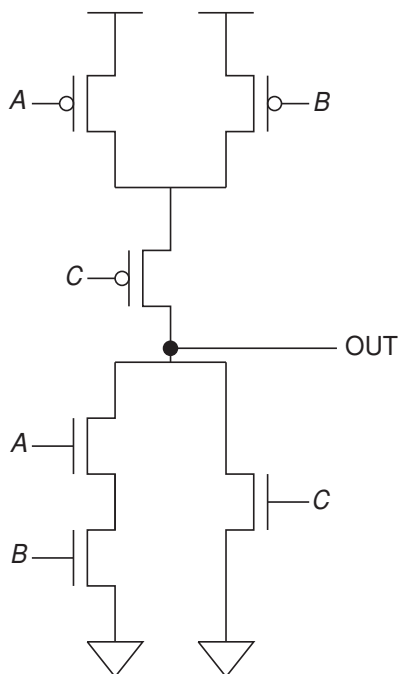


图 3.37 习题 3.5 的电路图

3.6 根据图 3.38 的晶体管级电路填写真值表。用 A 、 B 表示 Z 。

A	B	C	D	Z
0	0			
0	1			
1	0			
1	1			

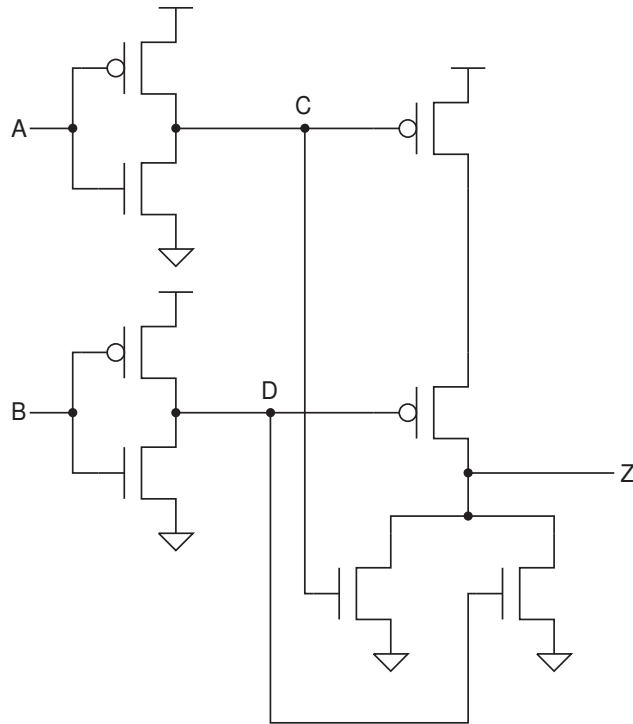
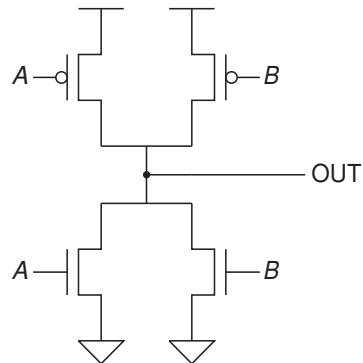
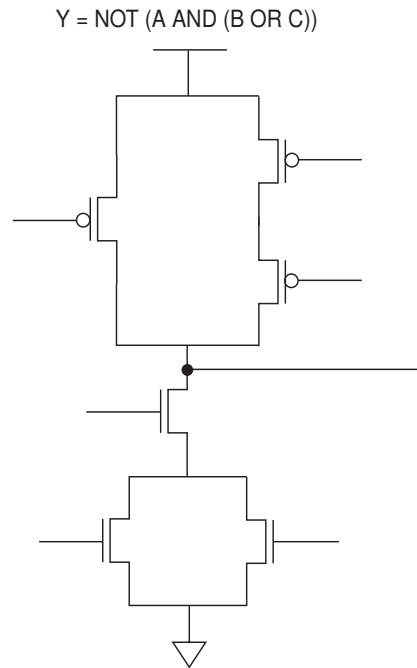


图 3.38 习题 3.6 的电路图

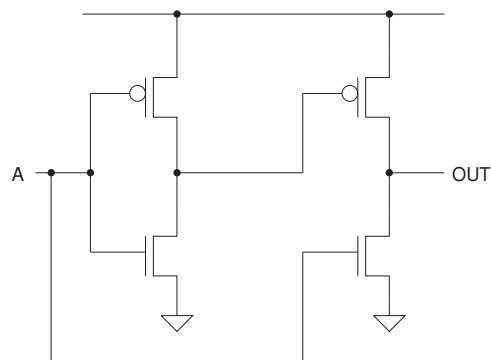
3.7 下列电路有一个重大缺陷。你能指出来吗？提示：对全部输入组合分析电路。



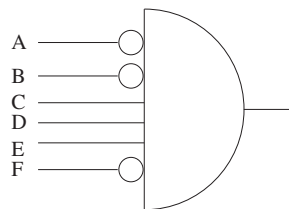
3.8 下列晶体管电路实现图中给出的逻辑等式。为所有晶体管标出输入。



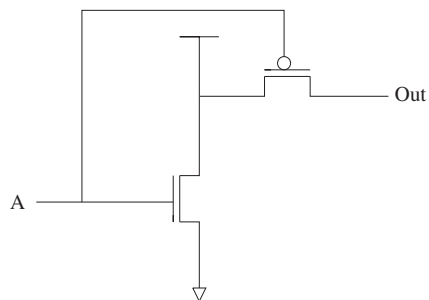
3.9 下列晶体管电路有什么作用？



3.10 A 、 B 、 C 、 D 、 E 、 F 取什么值时，这个六输入 AND 门的输出为 1？

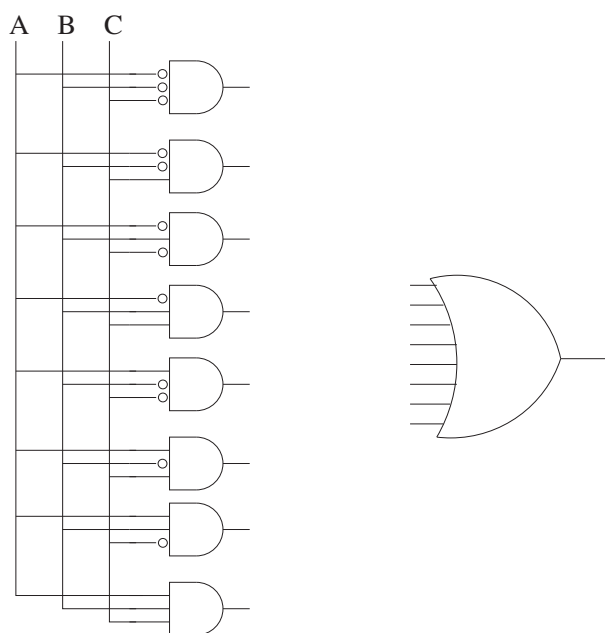


3.11 一名学生知道反相器由一个 P 型和一个 N 型晶体管组成，却接错了线，如下图。 $A = 0$ 时 Out 为多少？ $A = 1$ 时呢？

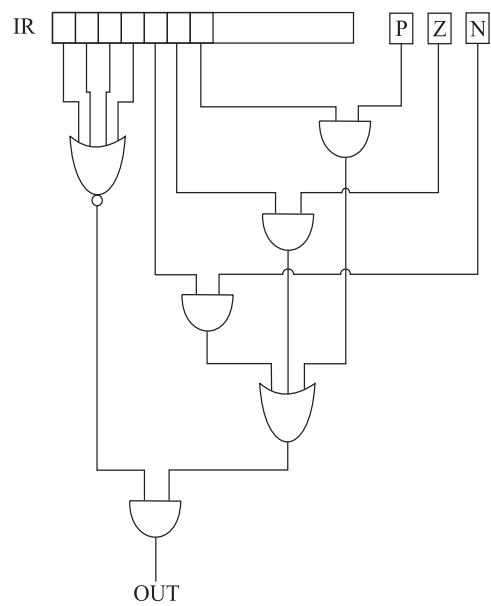


3.12 下列真值表描述一个函数。请为所给逻辑实现补上适当连接。

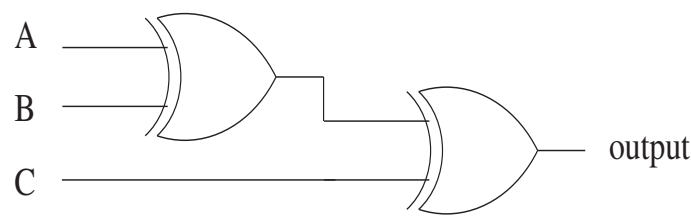
A	B	C	Out
0	0	0	1
0	0	1	1
0	1	0	0
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



3.13 下列逻辑图产生逻辑值 OUT。OUT 为 0 或 1 分别说明什么？



3.14 下列电路由两个 XOR 门构成。写出输出的真值表。



A	B	C	output
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

3.15 填写逻辑表达式 NOT (NOT (A) OR NOT (B)) 的真值表。哪一种单个逻辑门具有相同的真值表？

A	B	NOT (NOT (A) OR NOT (B))
0	0	
0	1	
1	0	
1	1	

3.16 填写两输入 NOR 门的真值表。

A	B	$A \text{ NOR } B$
0	0	
0	1	
1	0	
1	1	

- 3.17** a. 画出三输入 AND 门和三输入 OR 门的晶体管级电路图。请扩展图 3.6(a) 和 3.7(a) 中的设计。
- b. 对 a 中电路, 用导线或断开状态替换晶体管, 表示下列输入下的电路行为:
- (1) $A = 1, B = 0, C = 0$;
 - (2) $A = 0, B = 0, C = 0$;
 - (3) $A = 1, B = 1, C = 1$ 。

3.18 仿照图 3.11(a), 画出三输入译码器的门级图。对每个输出, 写出它为 1 时的输入条件。

3.19 五输入译码器有多少根输出线?

3.20 16 输入多路选择器有多少根输出线? 有多少根选择线?

3.21 A 、 B 是四位无符号二进制数, 分别为 0111 和 1011。使用图 3.15 的全加器逐位计算和 S , 填写下表。将 A 、 B 的十进制值相加并与 S 比较, 以检查答案。两者相同吗? 为什么?

C_{in}					0
A	0	1	1	1	
B	1	0	1	1	
S					
C_{out}					

3.22 根据下列真值表, 使用第 3.3.4 节的实现算法构造门级逻辑电路。

A	B	C	Z
0	0	0	1
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	0

3.23 a. 给定四个输入 A 、 B 、 C 、 D 和一个输出 Z , 构造一张至少在七种输入组合下输出 1 的真值表。这张表有多少行?

b. 使用第 3.3.4 节的算法，实现这张真值表的门级电路。

3.24 用 AND、OR、NOT 门实现下列函数。输入为 A 、 B ，输出为 F 。

- a. 仅当 $A = 0, B = 1$ 时， $F = 1$ 。
- b. 仅当 $A = 1, B = 0$ 时， $F = 1$ 。
- c. 利用 a、b 的答案实现一个一位加法器，其真值表如下。

A	B	Sum
0	0	0
0	1	1
1	0	1
1	1	0

- d. 能否只用四份 c 中的电路构成四位加法器，正确地将两个四位数相加？如果不能，缺少什么信息？提示： $A = B = 1$ 时和为 0，丢失了什么信息？

3.25 图 3.39 的电路 1 有输入 A 、 B 、 C ，图 3.40 的电路 2 有输入 A 、 B ，两者都有输出 D 。它们的行为有一个根本区别，是什么？提示：两个电路的输入 A 都从 0 变为 1 时，会发生什么？

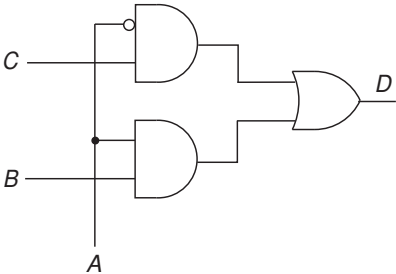


图 3.39 习题 3.25 的逻辑电路 1

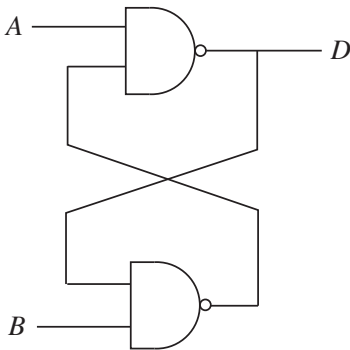


图 3.40 习题 3.25 的逻辑电路 2

3.26 实现下列真值表的门级电路，再根据门级结构画出晶体管电路，并验证晶体管电路符合真值表。

in_0	in_1	$f(in_0, in_1)$
0	0	1
0	1	0
1	0	1
1	1	1

3.27 一个字节有八位，四位称为一个半字节（nibble）。若按字节寻址的存储器使用 14 位地址，一共能存储多少个半字节？

3.28 只用 2 选 1 mux 实现一个 4 选 1 mux，正确连接全部端子。它应有四个输入、两个控制信号和一个输出。写出真值表。

3.29 根据图 3.41 的电路，填写输出 Z 的真值表。

A	B	C	Z
0	0	0	
0	0	1	
0	1	0	
0	1	1	
1	0	0	
1	0	1	
1	1	0	
1	1	1	

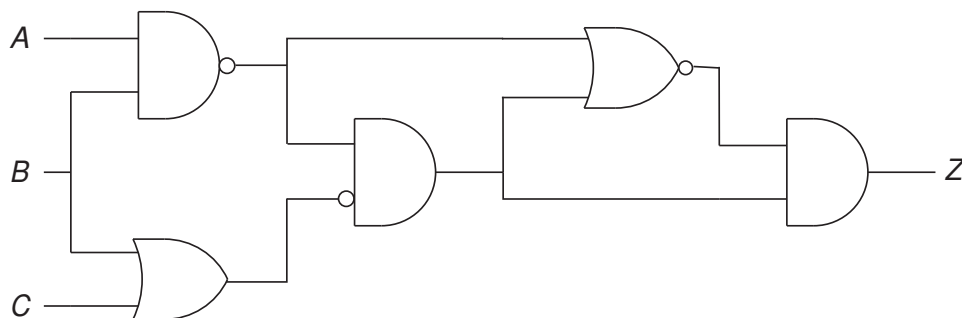


图 3.41 习题 3.29 的电路图

- 3.30** a. 图 3.42 的逻辑电路出现在许多处理器中，每个方框都是全加器。导线 X 上的值起什么作用？ $X = 0$ 与 $X = 1$ 时，输出有什么不同？
- b. 构造一个加法/减法电路，根据 X 的值计算 $A + B$ 或 $A - B$ 。提示：把图 3.42 作为构件。

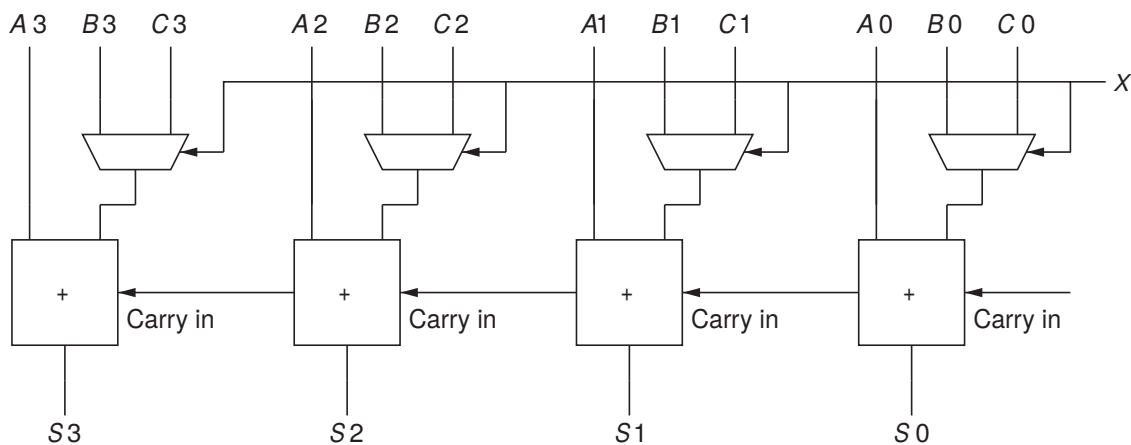
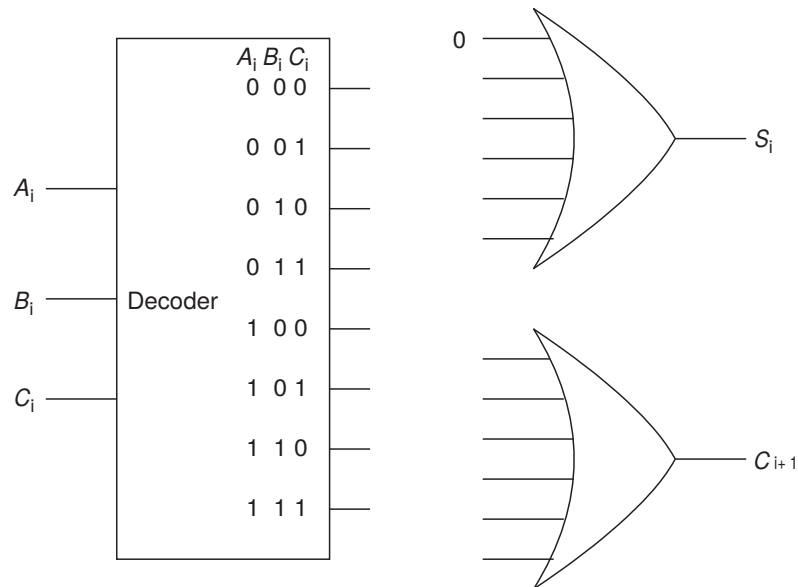


图 3.42 习题 3.30 的电路图

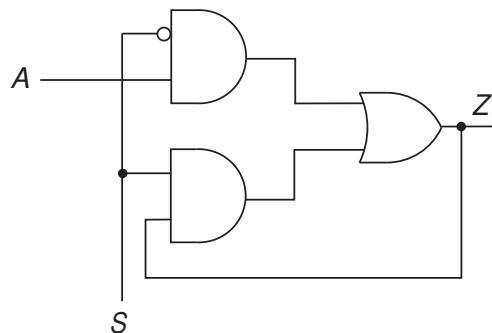
3.31 假设逻辑结构的速度取决于输入到输出所经过的最大逻辑门数量，并假设 NOT、AND、OR 各算一个门延迟。例如，图 3.11 两输入译码器的传播延迟为 2，因为某些输入经过两个门。

- 图 3.12 两输入 mux 的传播延迟是多少？
- 图 3.15 一位全加器呢？
- 图 3.16 四位加法器呢？
- 若把四位加法器扩展到 32 位呢？

3.32 加法器由逐位的“片”组成，根据操作数位 A 、 B 和输入进位，产生和位与输出进位，这样的元件叫全加器。现在有一个 3 线—8 线译码器和两个六输入 OR 门，如下图。能否连接成一个全加器？如果可以，请完成连接。提示：不用的 OR 输入可以接 0，不会影响其他输入。



3.33 根据下图回答问题。



- 选择线 $S = 0$ 时，输入 A 的每种值分别对应什么输出 Z ？
- 若 S 从 0 变成 1，输出会怎样？
- 这个逻辑电路是存储元件吗？

3.34 设计过二进制加法器后，现在设计一个两位乘两位的无符号二进制乘法器。输入为 $A[1:0]$ 和 $B[1:0]$ ，输出是它们的积 Y ，标准记法为

$$Y = A[1:0] \times B[1:0].$$

- 两位 $A[1:0]$ 能表示的最大值是什么？
- 两位 $B[1:0]$ 能表示的最大值是什么？
- Y 的最大可能值是什么？
- 表示这个最大值需要多少位？
- 写出乘法器的四输入真值表，输入为 $A[1]$ 、 $A[0]$ 、 $B[1]$ 、 $B[0]$ 。
- 根据真值表，只用 AND、OR、NOT 门实现输出的第三位 $Y[2]$ 。

3.35 一个 16 位寄存器中原来存有一个值，随后向它写入 $\times 75A2$ 。原来的值还能恢复吗？

3.36 一个比较器有两个一位输入 A 、 B 和三个一位输出 G （大于）、 E （等于）、 L （小于）。 $A > B$ 时 $G = 1$ ，否则为 0； $A = B$ 时 $E = 1$ ，否则为 0； $A < B$ 时 $L = 1$ ，否则为 0。参见图 3.43、3.44。

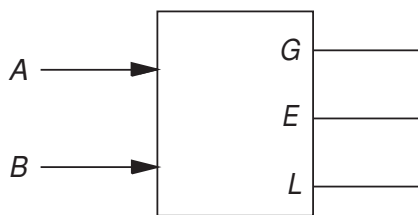


图 3.43 习题 3.36 的比较器

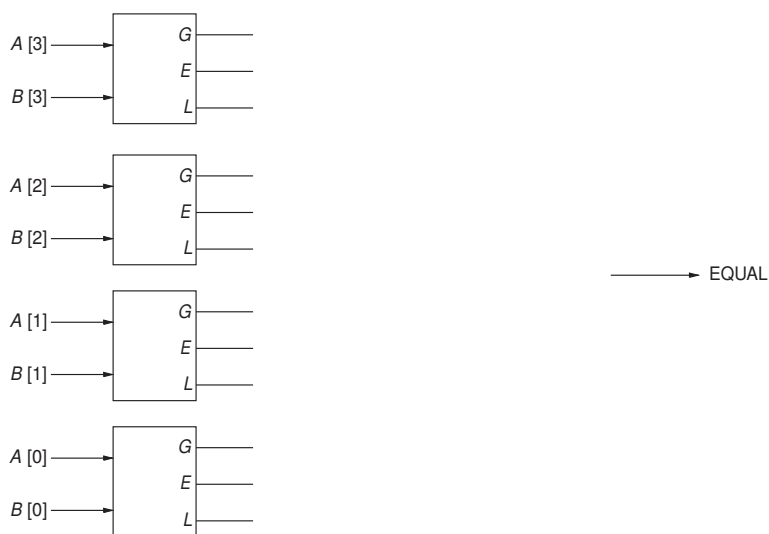


图 3.44 习题 3.36 的电路图

- 填写一位比较器的真值表。

A	B	G	E	L
0	0			
0	1			
1	0			
1	1			

- b. 用 AND、OR、NOT 门实现 G 、 E 、 L 。
 - c. 以一位比较器为基本构件，构造一个四位相等判断电路。当 $A[3:0] = B[3:0]$ 时，输出 EQUAL 为 1，否则为 0。
- 3.37** 一台计算机每个可寻址位置存储八字节，访问一个位置需要三位地址。总存储容量是多少字节？
- 3.38** 区分存储器地址与寻址粒度。
- 3.39** 参见图 3.21 中具有四个位置的 $2^2 \times 3$ 位存储器。
- a. 读取第四个位置时， $A[1:0]$ 与 WE 应取什么值？
 - b. 若把位置数从 4 增加到 60，需要多少根地址线？改变后的寻址粒度是多少？
 - c. 假设程序计数器的最小宽度恰好等于寻址这 60 个位置所需的最少位数，那么不改变程序计数器宽度，还能增加多少个位置？程序计数器是 CPU 内部的一个特殊寄存器，第 4 章会详细介绍。
- 3.40** 对图 3.45 的存储器：
- a. 地址空间是多少？
 - b. 寻址粒度是多少？
 - c. 地址 2 中的数据是什么？

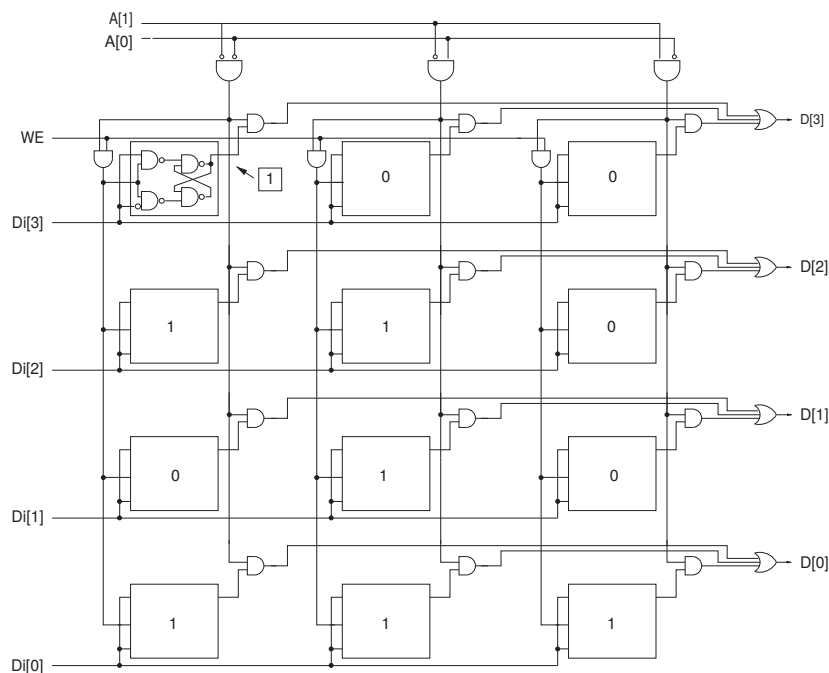
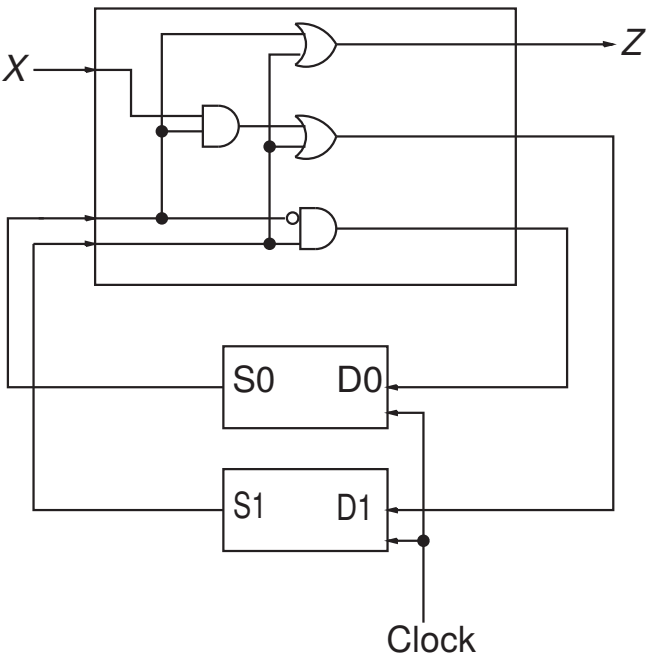


图 3.45 习题 3.40 的存储器

- 3.41** 一个存储器使用 22 位地址，每个可寻址位置存三位，一共包含多少位存储容量？
- 3.42** 一个组合逻辑电路有两个输入，过去十个周期的输入依次为 01, 10, 11, 01, 10, 11, 01, 10, 11, 01，当前输入为 10。说明过去十周期的输入对当前输出有什么影响。
- 3.43** 图 3.23(a) 的锁有第 3.6.2 节介绍的 A、B、C、D 四个状态：锁已打开为 D；尚未打开时，已经完成零个、一个、两个正确操作，分别为 A、B、C。这就包含了全部可能状态，为什么？如果存在第五个状态，它会描述这把锁的什么可能情形？
- 3.44** 回顾第 3.6.2 节。能否从 Texas 30、Oklahoma 28 的状态画一条弧到 Texas 30、Oklahoma 30 的状态？仿照图 3.24，画出这两个状态的记分牌示例。
- 3.45** 再回顾第 3.6.2 节。能否有比分为 30–28 和 30–30 的两个状态，却不能在它们之间画弧？画出这样的两个记分牌。再分别为“比赛进行中”“Texas 胜”“Oklahoma 胜”三个输出各画一个对应状态的记分牌。
- 3.46** 参照第 3.6.2 节，画出井字棋有限状态机的一部分。
- 3.47** 校园 IEEE 社团办公室出售 35 美分一罐的汽水。假设安装一个只接受 5、10、25 美分三种硬币输入的控制器。每投一枚硬币，按一次按钮确认。如果金额达到至少 35 美分，就输出汽水和应找的零钱。画出描述其行为的有限状态机。每个中间状态表示累计金额，提示：有七个这样的状态。投够钱后进入一个最终状态，输出汽水与适当找零，提示：有五个最终状态。从最终状态

投下一枚硬币时，重新开始这个过程。

- 3.48 参见图 3.32。为什么灯 1、2 由 OR 门 W 的输出控制？为什么存储元件 2 的下一状态由 OR 门 Y 的输出控制？
- 3.49 下图实现了一个有限状态机，输入为 X ，输出为 Z 。 S_1, S_0 指定当前状态， D_1, D_0 指定下一状态。



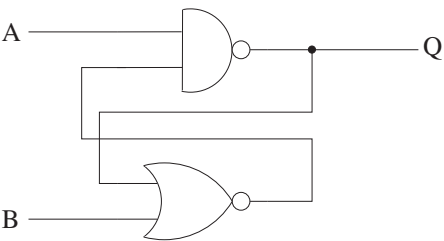
a. 补全下表。

S_1	S_0	X	$D_1 \quad D_0 \quad Z$		
0	0	0			
0	0	1			
0	1	0			
0	1	1	1	0	1
1	0	0			
1	0	1			
1	1	0			
1	1	1			

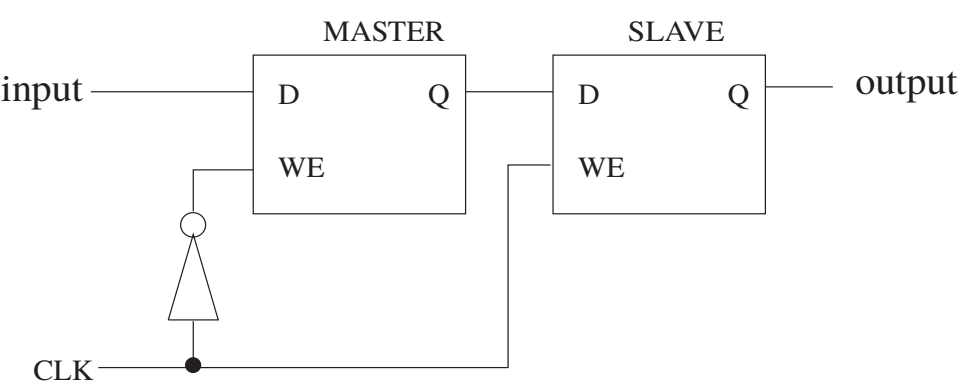
b. 根据 a 的真值表画状态图。

- 3.50 只用 NAND 门分别构造实现 AND、NOT、OR 的三个电路，证明 NAND 单独具有逻辑完备性，参见第 3.3.5 节。
- 3.51 我们已经知道，透明锁存器可以写入一位信息，写入后几乎立即就能读到。有时需要存下一位，却直到下一周期才能读到新值。具有这种性质的一种逻辑电路是_____。

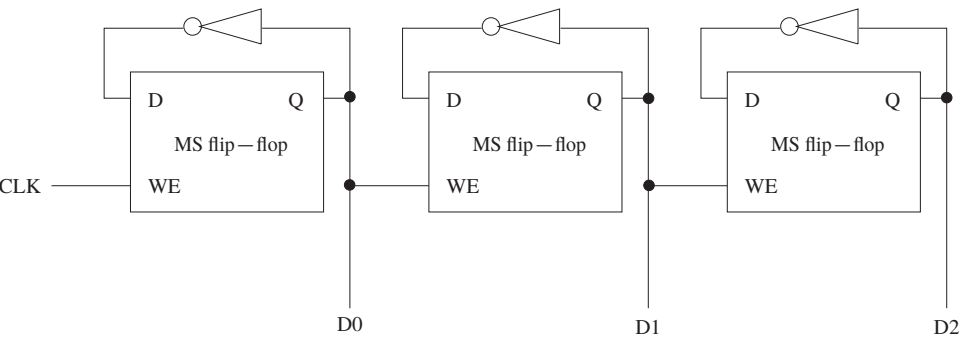
3.52 一名学生设计了下列锁存器。 A 、 B 取什么值时，它保持静止状态，即输出不变？



3.53 ★ 本章介绍的主从触发器如下。注意，时钟从 0 跳变到 1 后，输入值才出现在输出端。



下面的电路由三个这样的触发器构成。

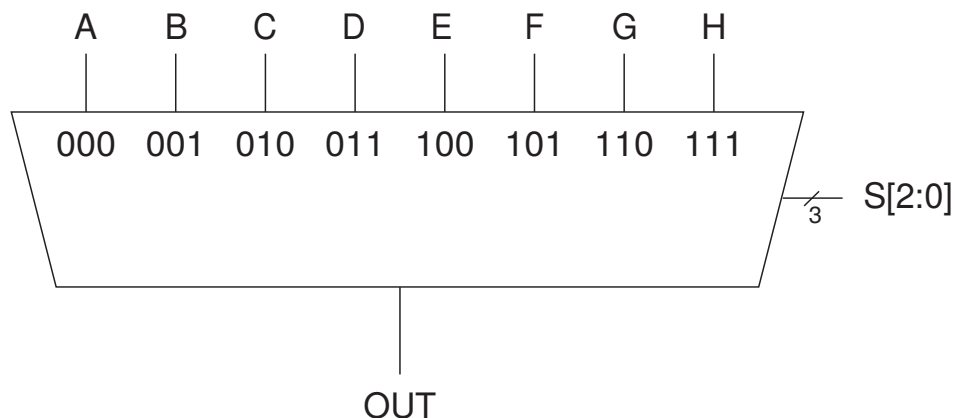


填写各时钟周期的 D_2, D_1, D_0 。

	cycle 0	cycle 1	cycle 2	cycle 3	cycle 4	cycle 5	cycle 6	cycle 7
D2	0							
D1	0							
D0	0							

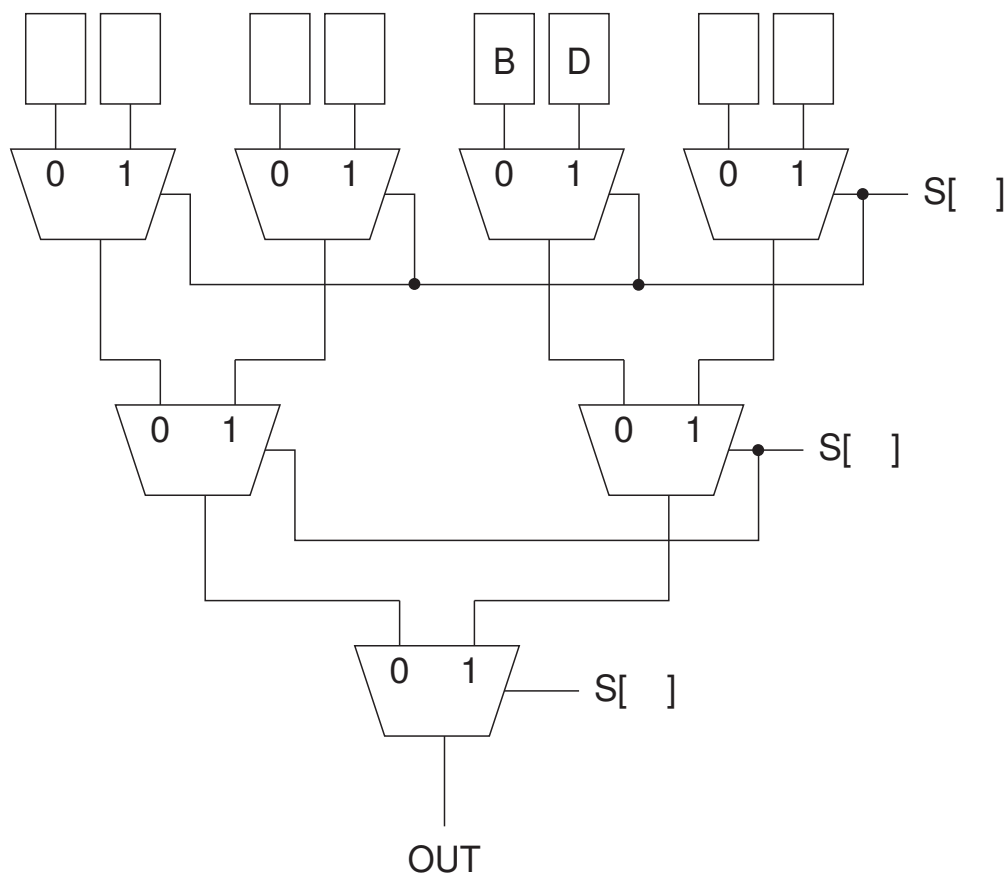
用不超过十个词说明这个电路在做什么。

3.54 ★ 下图的 8 选 1 mux 根据 $S[2:0]$ 从 A、B、C、D、E、F、G、H 中选一个输出。每个输入下方标有对应的选择值，例如 $S[2:0]=001$ 时输出 B。



可以用若干 2 选 1 mux 实现它，如下图。各 mux 输入下的 0、1 表示选择该输入时选择线应取的值。图中只标出了两个输入源，尚未标出选择位。请补全：

- 为每个 mux 的选择线标上 $S[2]$ 、 $S[1]$ 或 $S[0]$ 。
- 标出其余六个输入源，使整个电路与上面的 8 选 1 mux 行为完全一致。

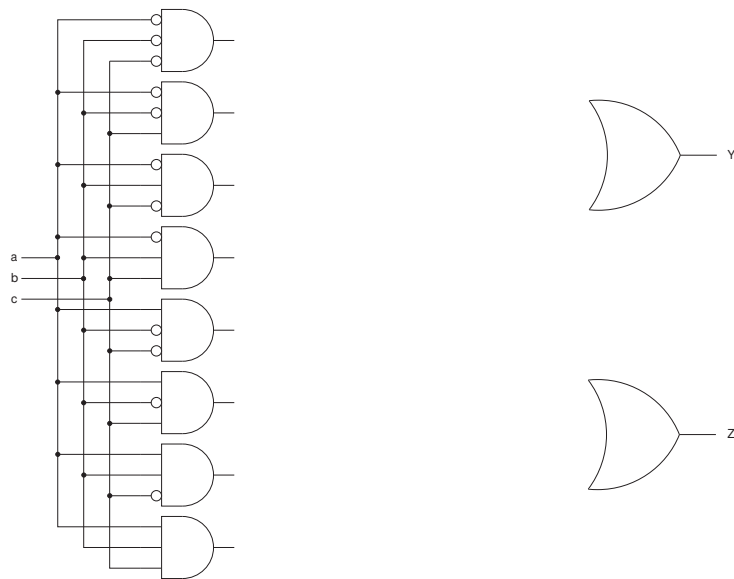


3.55 ★ 要实现两个逻辑函数 $Y(a, b, c)$ 和 $Z(a, b)$ 。当且仅当 a, b, c 中有奇数个 1 时, $Y = 1$; Z 为 a, b 的 XOR。

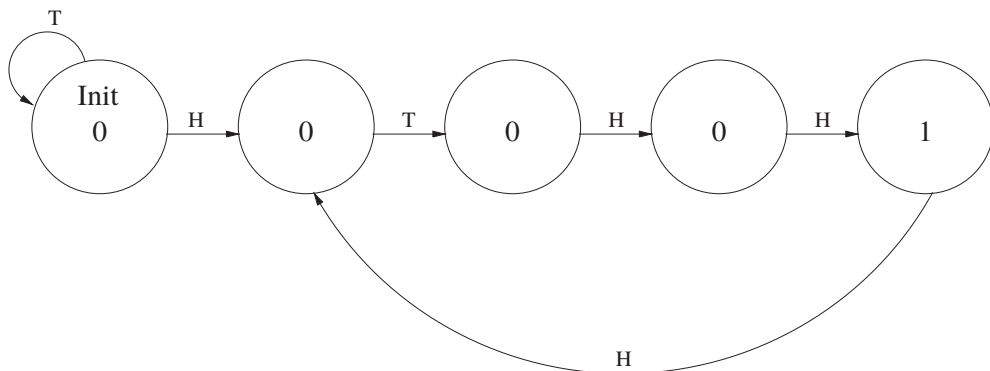
a. 填写 Y, Z 的真值表。

a	b	c	Y	Z
0	0	0		
0	0	1		
0	1	0		
0	1	1		
1	0	0		
1	0	1		
1	1	0		
1	1	1		

b. 只使用下图的一个 3 线—8 线译码器与两个 OR 门, 补上译码器输出到 OR 门输入的导线, 实现这两个函数。可假设每个 OR 门都所有所需数量的输入端。



3.56 ★ 下图是一张尚未完成的状态图。状态机接收由 H（正面）和 T（反面）组成的输入序列，每出现一次 HTHH，就输出 1。



例如，输入与输出可以逐位对齐写为

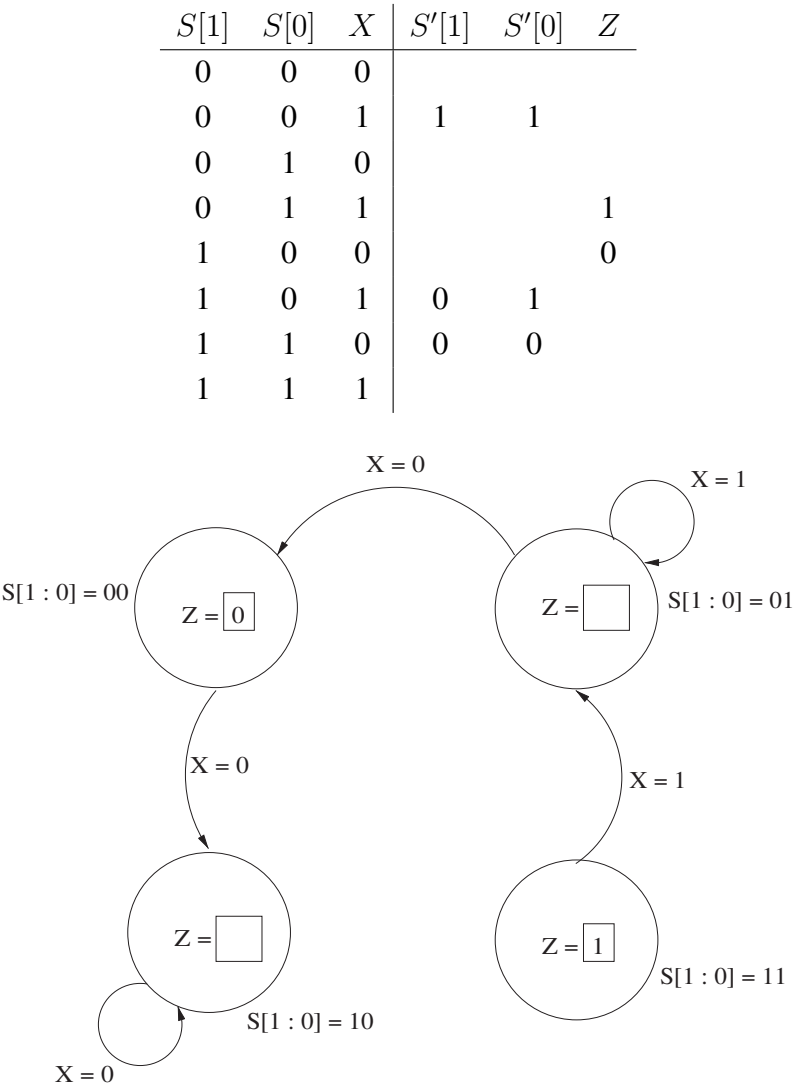
输入：H H H H H T H H T H H H H H T H H T

输出：0 0 0 0 0 0 0 1 0 0 1 0 0 0 0 0 1 0

注意，第八次抛掷的 H 同时属于两个 HTHH 序列。

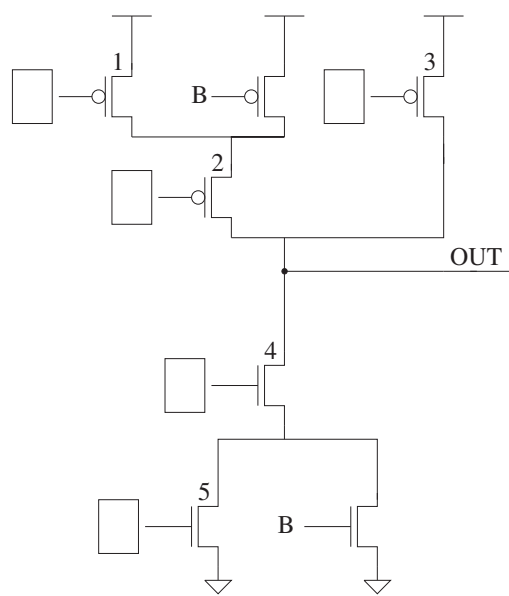
- a. 补全状态图，使其适用于任意长度的任意输入序列。
- b. 若用类似本章危险警示牌的时序逻辑电路实现，需要多少个状态变量？

3.57 ★ 下图给出一个四状态机器及其真值表，两者都缺少部分内容。状态标为 00、01、10、11，各状态内标有输出 Z (0 或 1)，输入为 X。请补全真值表和状态图。



3.58 ★ 下列晶体管电路产生所附真值表。部分晶体管的栅极输入未标出，部分输入组合的输出也未给出。请补全：所有栅极都应标为 A、B 或 C，真值表每行都应有一个 0 或 1 的输出。

注意，这个电路没有接线缺陷。对于每种输入组合，输出要么接地 (OUT=0)，要么接电池正极 (OUT=1)。



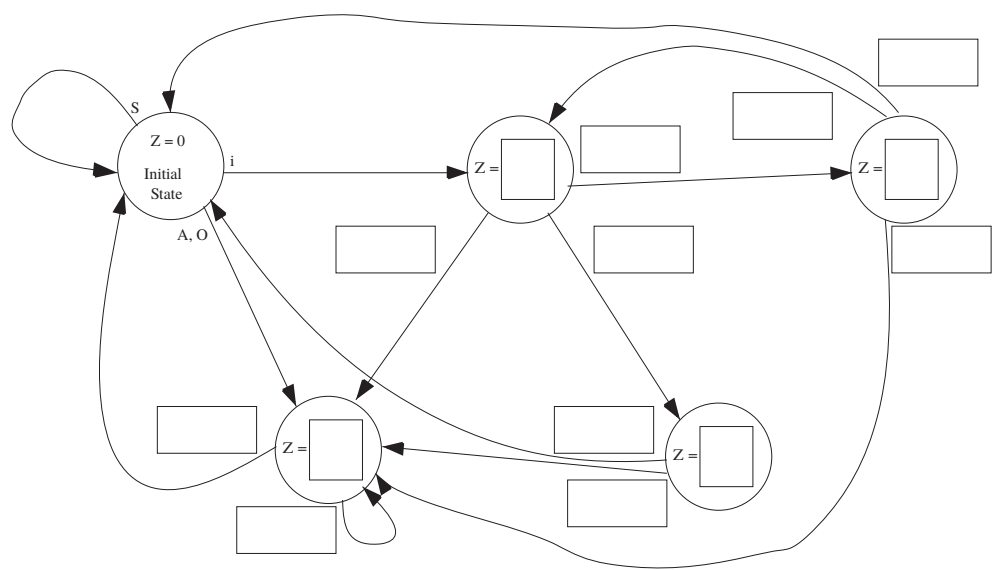
A	B	C	OUT
0	0	0	
0	0	1	
0	1	0	
0	1	1	1
1	0	0	1
1	0	1	0
1	1	0	
1	1	1	

3.59 ★ 大多数文字处理器能纠正简单的拼写和语法错误。请规定一个有限状态机，在某些情况下把误输成小写 i 的人称代词改成大写 I。例如，i think i'm in love 应改为 I think I'm in love。

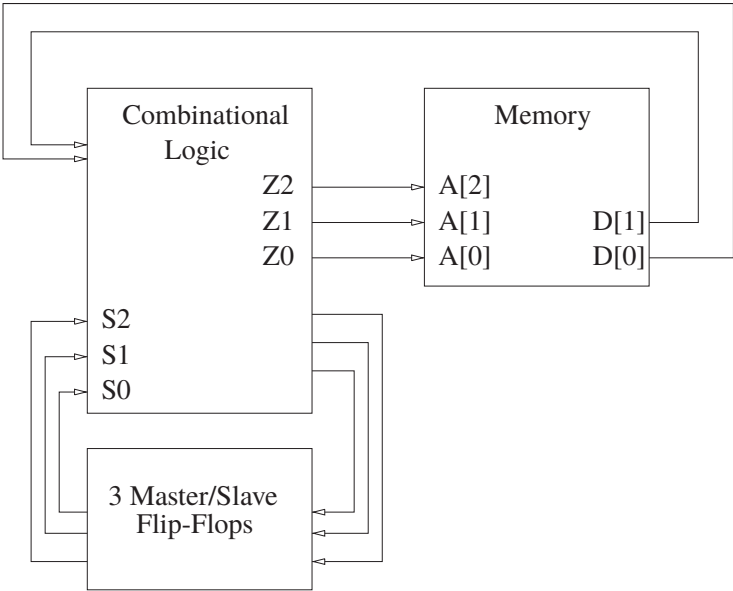
输入是标准键盘的任意字符序列。当 i 是第一个输入字符，或前面是空格，并且后面是空格或撇号时，应把它改为 I。

下面状态机有一些输入和输出尚未规定，请补全。输入来自集合 {i, A, S, O}，其中 A 表示撇号，S 表示空格，O 表示除 i、撇号、空格以外的任意字符。每个状态的输出 Z 为 0 或 1：0 表示“不做任何事”，1 表示“把最近的 i 改成 I”。

注意，这只是用状态机实现文字处理器的第一步，很多这样的替换并不能真正修正错误。例如，i' am → I' am、i'abcd → I'abcd、i'i → I'i 仍然不对。不过，毕竟迈出了第一步！



3.60 ★ 一个有限状态机连接到 $2^3 \times 2$ 位存储器，如下图。



存储器内容与下一状态转换表如下。

地址 A[2:0] 内容 D[1:0]		当前状态 S[2:0]					下一状态			
							D = 00	D = 01	D = 10	D = 11
000	11	000	001	010	110	100				
001	10	001	100	000	011	110				
010	01	010	010	100	111	010				
011	10	011	001	100	100	010				
100	01	100	110	011	011	111				
101	00	101	100	010	100	110				
110	00	110	001	110	100	010				
111	01	111	000	101	111	101				

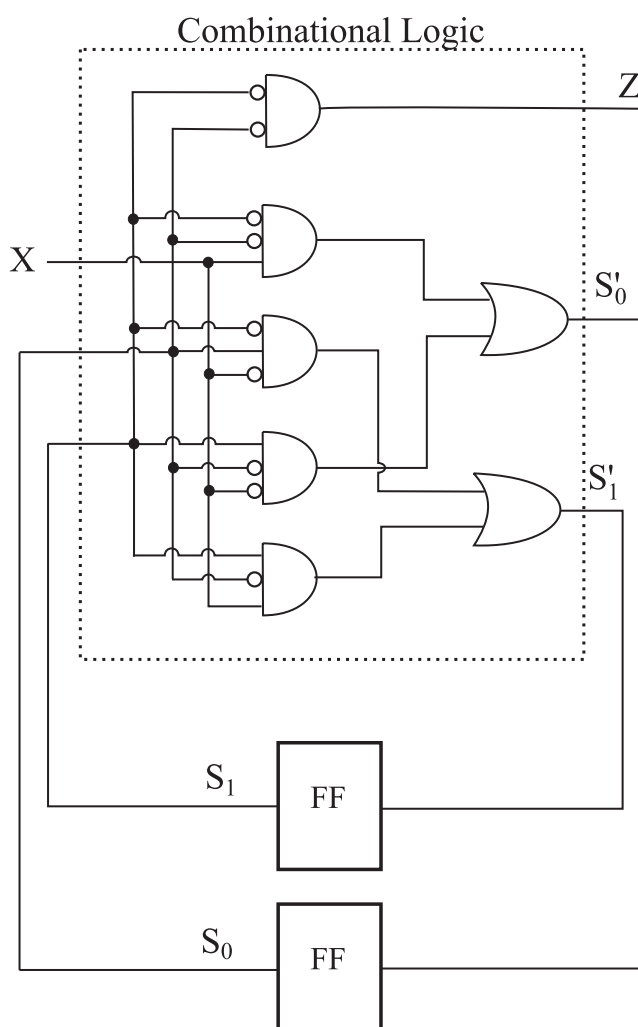
输出 Z_0, Z_1, Z_2 就是当前状态: $Z_0 = S_0, Z_1 = S_1, Z_2 = S_2$ 。周期足够长, 使一个周期内能完成: 状态机输出访问存储器, 存储器给出的值送入组合逻辑, 组合逻辑确定下一状态。

a. 补全下表。

周期	状态	数据
0	000	11
1		
2		
3		

b. 第 100 周期即将结束时, FSM 处于什么状态? 为什么?

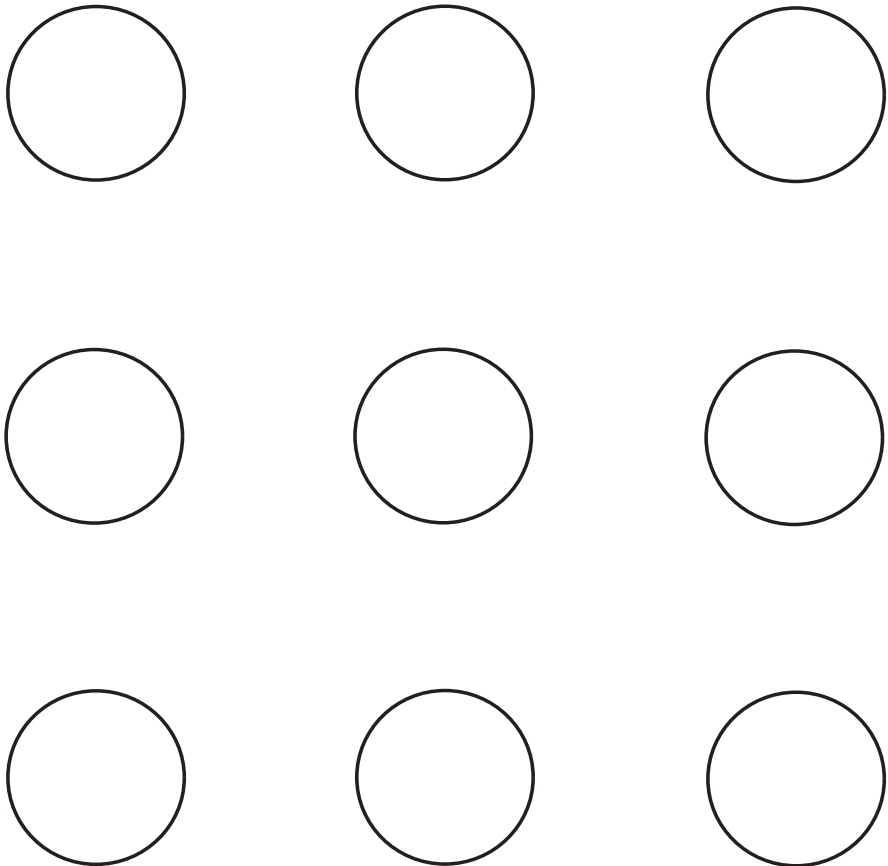
3.61 ★ 下列逻辑图是一个有限状态机。



a. 写出组合逻辑的真值表。

S_1	S_0	X	Z	S'_1	S'_0
0	0	0			
0	0	1			
0	1	0			
0	1	1			
1	0	0			
1	0	1			
1	1	0			
1	1	1			

b. 补全状态图。图中提供了九个状态，不必全部使用，只用所需数量。



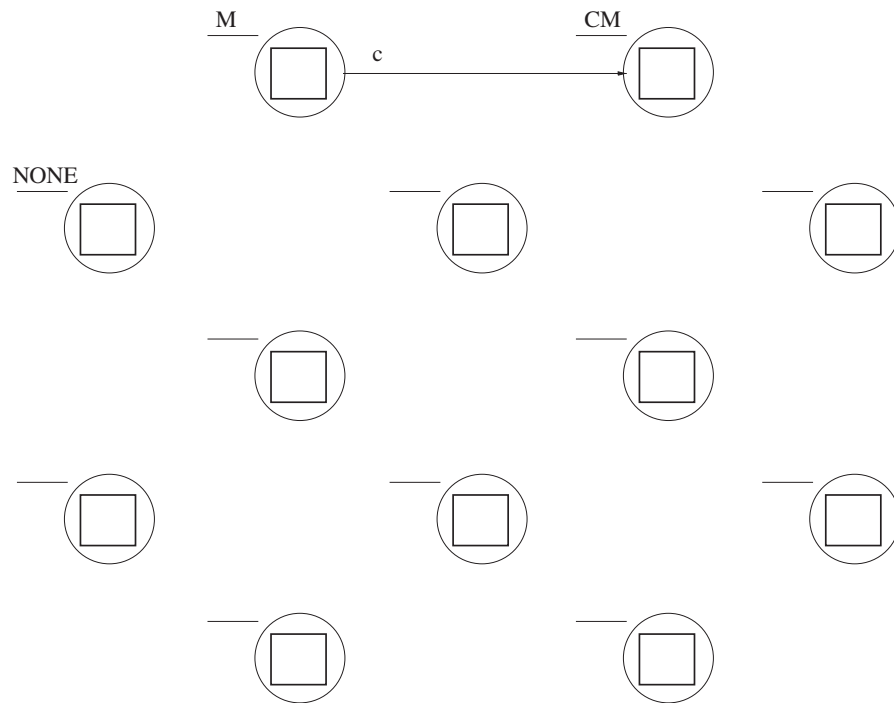
3.62 ★ 你在学习计算机 (C)、工程 (E) 和数学 (M) 三门课，每门课不时布置作业。一次不会收到多份作业；某门课的作业未完成之前，也不会收到该课的新作业。除非计算机和工程两门课同时有未完成作业，否则你必须拖延，也就是不做什么事。

设计一个有限状态机，描述待完成作业的状态，以及你是在工作还是拖延。

- a. 用 C、E、M 标出每个状态下尚未完成的作业。图中给出的状态远多于所需，只使用必要的那些。
- b. 输入有六个： $c, e, m, \bar{c}, \bar{e}, \bar{m}$ 。 c, e, m 表示收到相应课程的作业， $\bar{c}, \bar{e}, \bar{m}$ 表示完成相应作业。为每个状态与输入的组合画转换弧。例如，原先只有

数学作业未完成，随后收到计算机作业，则从 **M** 转到 **CM**，如图所示。

- c. 每个状态的输出表示你的行为：正在做作业为 1，正在拖延为 0。标出各状态的输出。



第四章 冯·诺依曼模型

现在，我们可以把抽象层次再提高一级了。我们将以第 3 章研究过的逻辑结构——包括决策元件和存储元件——为基础，构造最初于 20 世纪 40 年代提出的基本计算机模型，通常称为冯·诺依曼机。然后，我们还要用 LC-3 的 ISA 写出第一个计算机程序。

4.1 基本组成部分

要让计算机完成一项任务，我们需要两样东西：(a) 一个计算机程序，规定计算机必须做些什么才能完成任务；(b) 一台执行这项任务的计算机。

计算机程序由一组指令组成，每条指令规定计算机要执行的一项定义明确的工作。指令是计算机程序所规定的最小工作单位。也就是说，计算机要么完成一条指令所规定的工作，要么不执行这条指令；它不能只挑其中一部分来做。

John von Neumann 于 1946 年提出了一个执行计算机程序的基本计算机模型。图 4.1 展示了它的基本组成部分。我们在 von Neumann 的原图上稍作发挥，增加了一些自己的小修饰。冯·诺依曼模型由五部分组成：**存储器、处理单元、输入、输出和控制单元**。计算机程序存放在计算机的存储器中。程序执行工作所需的数据，要么已经存放在存储器中，要么从输入设备获得。程序执行的结果由输出设备提供。指令按什么次序执行，则由控制单元安排。

下面分别详细介绍这五部分。

4.1.1 存储器

回想第 3 章，我们研究过一个由逻辑门和锁存器构成的简单存储器，规模为 $2^2 \times 3$ 位。对于今天的计算机系统，更现实的存储器规模是 $2^{34} \times 8$ 位。也就是说，一个典型存储器包含 2^{34} 个不同的存储位置，每个位置能够保存八位信息。我们说，这个存储器的**地址空间**包含 2^{34} 个可唯一标识的位置，**寻址粒度**为八位。这样的存储器称为 16 gigabyte 存储器，简写为 16 GB。“16 giga”指 2^{34} 个位置，“byte”指每个位置保存的八位。之所以是 16 giga，是因为 16 等于 2^4 ，而 giga 在这里表示 2^{30} ，约为十亿； $2^4 \times 2^{30} = 2^{34}$ 。byte（字节）是八位的名称，就像 gallon（加仑）是四夸脱的名称一样。

这里再提醒一次——以后还会不断提醒——用 k 位可以唯一表示 2^k 个对象。因此，要唯一标识 2^{34} 个存储位置，每个位置都必须有一个自己的 34 位地址。第 5 章将开始完整定义 LC-3 计算机。届时我们会看到，LC-3 的存储器地址空间包含 2^{16} 个位置，寻址粒度为 16 位。

第 3 章讲过，访问存储器时，要给出希望读取或写入的位置的地址。读取某个位置的内容时，首先把该位置的地址放入**存储器地址寄存器（MAR）**，然后访问存储器。

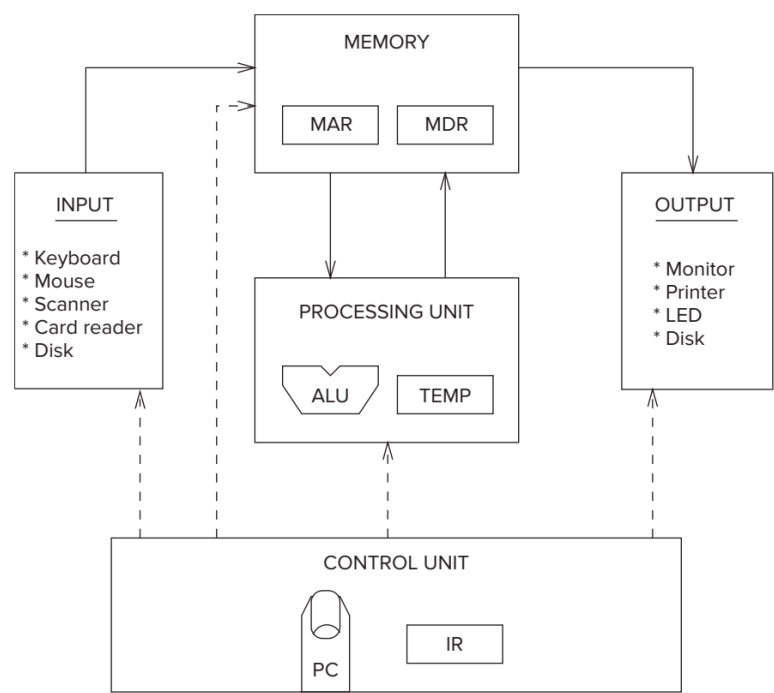


图 4.1 冯·诺依曼模型的总体框图

该地址对应位置中的信息就会进入**存储器数据寄存器**（MDR）。向某个位置写入（或存储）一个值时，首先把位置的地址写入 MAR，把待存储的值写入 MDR，然后在写使能信号有效的条件下访问存储器。MDR 中的信息就会写入 MAR 所指定的存储位置。

暂时结束对存储器的讨论之前，再强调一下一个存储位置的两个特征：它的地址，以及其中保存的内容。图 4.2 表示一个包含八个位置的存储器。左边用二进制列出了各个地址，从 0 到 7。每个位置保存八位信息。注意，值 6 存放在地址为 4 的位置，值 4 存放在地址为 6 的位置。这是两种完全不同的情况。

000	
001	
010	
011	
100	00000110
101	
110	00000100
111	

图 4.2 位置 6 中存放值 4；位置 4 中存放值 6。

最后打个比方：想一想当地邮局里的信箱。信箱号码就像存储位置的地址，每个号码都是唯一的；存储位置中的信息则像信箱里的信件。随着时间推移，信箱中的东

西会发生变化，信箱号码却保持不变。每个存储位置也是如此：其中保存的值可以改变，这个位置的存储器地址却保持不变。

4.1.2 处理单元

计算机中真正的信息处理工作由**处理单元**（processing unit）完成。现代计算机的处理单元可以包含许多精巧而复杂的功能单元，每个单元执行一种特定运算，例如除法或平方根。最简单的处理单元，也是讨论基本冯·诺依曼模型时通常想到的处理单元，是ALU。ALU是Arithmetic and Logic Unit（算术逻辑单元）的缩写，之所以这样命名，是因为它通常能够执行基本算术运算，例如ADD和SUBTRACT，以及第2章学过的基本逻辑运算，例如逐位AND、OR和NOT。第5章将介绍LC-3的ALU，它能够执行ADD、AND和NOT。本章先讨论其中的ADD和AND。

ALU通常处理大小固定的数据元素，这个大小称为计算机的**字长**（word length），这些数据元素则称为**字**（word）。例如，执行ADD时，ALU接收两个字作为输入，产生一个字——两者之和——作为输出。每种ISA都有自己的字长，取决于计算机的预期用途。

如今，大多数用于PC或工作站的微处理器的字长为64位，例如Intel的“Core”处理器；也有32位的，例如Intel的“Atom”处理器。现在，就连大多数手机微处理器也采用64位字长，例如Apple的A7至A11，以及Qualcomm的SnapDragon处理器。不过，用于成本极低的应用的微处理器，字长往往只有16位，甚至八位。

LC-3的ALU处理16位的字。我们说，LC-3的字长是16位。

计算机几乎总会在非常靠近ALU的地方提供少量存储空间。如果某个结果很快还要用于计算其他结果，就可以把它暂存在这里。例如，要计算 $(A + B) \cdot C$ ，可以先把 $A + B$ 的结果存入存储器，随后再读出来，与 C 相乘。不过，相对于ADD或MULTIPLY运算，访问存储器所需的时间很长。因此，几乎所有计算机都提供临时存储空间来保存 $A + B$ 的结果，避免在准备相乘时再花较长时间访问存储器。最常见的临时存储形式是一组寄存器，就像第3.7节介绍的寄存器一样。通常，每个寄存器的大小与ALU所处理的值相同，也就是一个字。LC-3有八个寄存器（R0、R1、...、R7），每个寄存器包含16位。

当前的微处理器通常包含32个寄存器，根据体系结构的不同，每个寄存器为32位或64位。它们与LC-3的八个16位寄存器用途相同。不过，暂存即将使用的值非常重要，因此今天许多计算机还提供一组128位的专用寄存器，以满足特殊需要。至于这些特殊需要是什么，就留待你今后学习了。

4.1.3 输入与输出

计算机要处理信息，信息首先必须进入计算机；要使用处理结果，就必须以某种形式把结果呈现在计算机之外。有许多设备可以用于输入和输出。在计算机术语中，它们统称为**外围设备**（peripherals），因为从某种意义上说，它们是处理功能的附属设备。不过，这并不意味着它们不重要。

LC-3 将使用两种最基本的输入输出设备：输入用键盘，输出用显示器。

当然，今天的计算机系统还有许多其他输入输出设备。输入设备包括鼠标、数字扫描仪，以及帮助你在商场中找路的自助终端等。输出设备包括打印机、LED 显示屏、磁盘，以及帮助你在商场中找路的自助终端等。:-) 过去，大量输入输出工作通过穿孔卡片完成。对于那些不得不搬运一箱箱卡片的人来说，幸运的是，穿孔卡片如今已经基本退出使用了。

4.1.4 控制单元

控制单元 (control unit) 就像乐队指挥，负责让计算机的其他部分协调工作。稍后逐步介绍程序执行过程时，我们会看到，控制单元既要跟踪程序执行到了哪里，也要跟踪每条指令执行到了哪一步。

为了跟踪当前执行的指令，控制单元用一个**指令寄存器**保存这条指令。为了跟踪接下来处理哪条指令，控制单元还用了一个寄存器保存下一条指令的地址。出于历史原因，这个寄存器称为**程序计数器** (program counter, 简称 PC)。其实，把它叫作**指令指针** (instruction pointer) 更合适，因为从某种意义上说，其中的内容“指向”下一条待处理的指令。有趣的是，Intel 确实把这个寄存器称为 instruction pointer，不过，这个简洁贴切的名称并没有普及开来。

4.2 LC-3: 冯·诺依曼机的一个实例

第 5 章将详细定义 LC-3，这是一台我们要深入研究的简单计算机。第 3 章已经展示过它的数据通路 (图 3.35)，第 4.1 节也介绍了其中几个结构。本节把描述 LC-3 作为一台冯·诺依曼机所需的各部分汇集起来，见图 4.3。

图 4.3 是在完整 LC-3 数据通路 (图 3.35) 的基础上，删去对于说明冯·诺依曼模型五个基本组成部分并非必需的元件而得到的。

注意，图中有两种箭头：实心和空心。实心箭头表示沿相应路径流动的数据元素，空心箭头表示控制这些数据如何被处理的控制信号。例如，处理单元中标为 ALU 的方框处理两个 16 位值，产生一个 16 位结果。两个源值和结果都是数据，因此用实心箭头表示。对两个 16 位数据元素执行什么运算，由标为 ALUK 的信号决定；它属于控制部分，因此用空心箭头表示。

存储器 (MEMORY) 包括存储元件、用于对各个位置寻址的存储器地址寄存器 MAR，以及在数据进出存储元件时保存该位置内容的存储器数据寄存器 MDR。MAR 包含 16 位，反映出 LC-3 的存储器地址空间包含 2^{16} 个位置。MDR 包含 16 位，反映出每个存储位置保存 16 位，也就是说，LC-3 按 16 位寻址。

输入与输出 (INPUT/OUTPUT) 包括键盘和显示器。最简单的键盘需要两个寄存器：**键盘数据寄存器** (KBDR)，用于保存所按按键的 ASCII 码；**键盘状态寄存器** (KBSR)，用于保存按键相关的状态信息。最简单的显示器也需要两个寄存器：**显示数据寄存器** (DDR)，用于保存待显示字符的 ASCII 码；**显示状态寄存器** (DSR)，用于保存相应的状态信息。第 9 章将详细讨论这些输入输出寄存器。

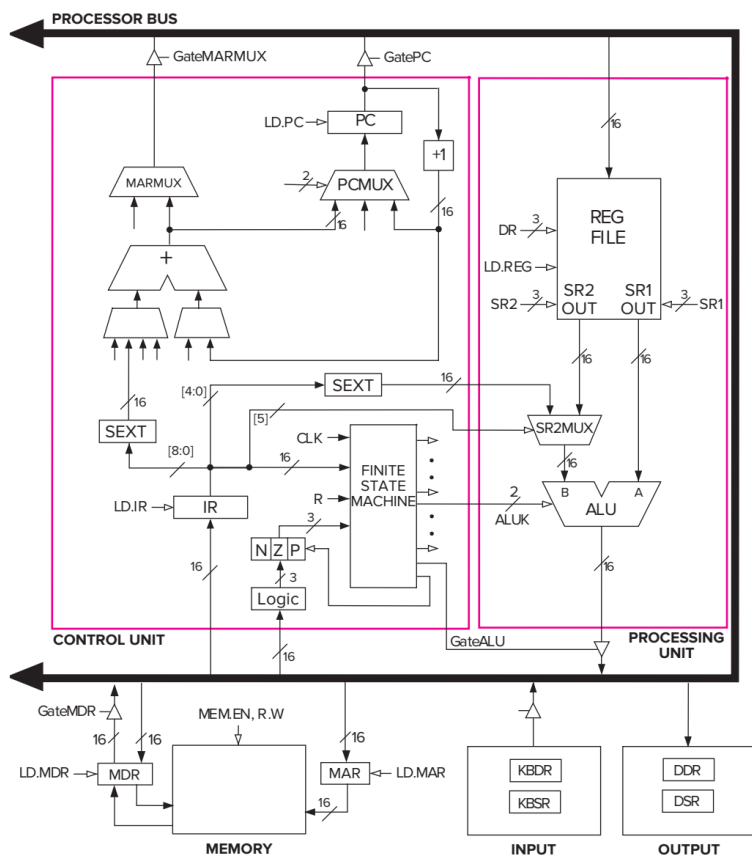


图 4.3 作为冯·诺依曼模型实例的 LC-3

处理单元（PROCESSING UNIT） 包括执行算术与逻辑运算的功能单元 ALU，以及八个寄存器（R0、...、R7）。这些寄存器保存临时值，供随后执行的指令作为操作数使用。LC-3 的 ALU 可以执行一种算术运算——加法，以及两种逻辑运算——逐位 AND 和逐位 NOT。

控制单元（CONTROL UNIT） 包括管理计算机处理活动所需的所有结构。其中最重要的是指挥全部活动的有限状态机。回想第 3.6 节介绍的有限状态机：处理工作一步一步地进行，更准确地说，是一个时钟周期接着一个时钟周期地进行。注意图 4.3 中有限状态机的 CLK 输入，它规定每个时钟周期持续多久。指令寄存器 IR 也是有限状态机的输入，因为当前处理的 LC-3 指令决定了必须执行哪些活动。程序计数器 PC 也属于控制单元，用于跟踪当前指令结束后接下来要执行的指令。

注意，图 4.3 中有限状态机的所有外部输出都用空心箭头表示。这些输出控制着整个计算机中的处理活动。例如，其中一个两位输出是 ALUK，它决定当前时钟周期中 ALU 执行哪种运算：ADD、AND 或 NOT。另一个输出是 GateALU，它决定当前时钟周期中是否把 ALU 的输出送上处理器总线。

附录 C 将完整介绍 LC-3 的一种实现，包括它的数据通路、控制部分和有限状态机。

4.3 指令处理

冯·诺依曼计算模型的核心思想是：程序和数据都以位序列的形式存放在计算机的存储器中，程序在控制单元的指挥下一次执行一条指令。

4.3.1 指令

计算机处理的最基本单位是**指令**。指令由两部分组成：**操作码**（opcode，规定做什么）和**操作数**（operands，规定对谁做）。

指令基本上分为三类：运算指令、数据传送指令和控制指令。不过，许多 ISA 还具有一些自身需要的特殊指令。**运算指令**对数据进行运算。LC-3 有三条运算指令：一条算术指令 ADD，以及两条逻辑指令 AND 和 NOT。**数据传送指令**在处理单元与存储器之间、处理单元与输入输出设备之间传送信息。LC-3 有六条数据传送指令。

控制指令用于改变指令的顺序执行。通常，接下来执行的是相邻下一个存储位置中的指令。如果程序的指令 1、2、3、4、...、10 分别位于存储位置 A、A+1、A+2、...、A+9，一般就按 1、2、3、...、10 的次序执行。不过，本章稍后会看到，有时我们希望改变这个次序。控制指令使这一点成为可能。

一条 LC-3 指令包含 16 位，也就是一个字，从左到右编号为位 [15] 至位 [0]。位 [15:12] 保存操作码，因此最多可以有 2^4 种不同操作码。实际上，我们只使用其中 15 种四位编码，留出一种供将来使用。位 [11:0] 用于确定操作数在哪里。

本章将介绍 LC-3 的 15 条指令中的五条：两条运算指令 ADD 和 AND，一条数据传送指令 LD，以及两条控制指令 BR 和 TRAP。其余十条留到第 5、8、9 章介绍。

例 4.1 ADD 指令。ADD 是一条运算指令，需要三个操作数：两个源操作数，也就是要相加的数；一个目的操作数，也就是加法完成后保存和的位置。前面说过，LC-3 的处理单元有八个寄存器，用于保存以后可能需要的数据。实际上，ADD 要求两个源操作数中至少有一个位于这些寄存器中，而且加法结果也必须放入这八个寄存器之一。由于有八个寄存器，标识一个寄存器需要三位。16 位的 LC-3 ADD 指令有以下两种形式，我们称之为**格式**（format）：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	0	0	1	0	0	0	0	1	1	0
ADD				R6			R2			R6					

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	0	0	1	0	1	0	0	1	1	0
ADD				R6			R2			imm					

两种格式中，位 [15:12] 都保存 ADD 的四位操作码 0001。位 [11:9] 标识保存结果的位置，这里是寄存器 6（R6）。位 [8:6] 标识保存一个源操作数的寄存器，这里是 R2。两种格式的区别在于位 [5] 保存的是 0 还是 1，以及这代表什么。第一种情况中，位 [5] 为 0，表示第二个源操作数在位 [2:0] 指定的寄存器中，这里是 R6。第二种情况中，位

[5] 为 1，表示把位 [4:0] 中的整数符号扩展到 16 位，作为第二个源操作数，这里是正整数 6。

因此，根据位 [5] 为 0 还是 1，刚才编码的指令分别解释为：“把寄存器 2 (R2) 的内容与寄存器 6 (R6) 的内容相加，把结果写回 R6”；或者“把 R2 的内容与正整数 6 相加，把结果写入 R6”。本章稍后编写程序时，两种格式都会用到。

例 4.2 AND 指令。AND 也是一条运算指令，它的行为与 ADD 基本相同，只有一点不同：它对两个源操作数的对应位执行逐位 AND，而不是把两个源操作数相加。例如，下面这条指令：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	0	0	1	1	1	0	0	0	0	0
AND				R2				R3				imm			

位 [15:12] 为 0101，因此这是一条 AND 指令。两个源操作数是 R3 和立即数 0。由于第二个操作数的每一位都是 0，逐位 AND 的结果也是 0，因此指令把 0 装入 R2。我们将看到，这是确保某个寄存器在处理开始时为 0 的一种方便方法，称为把 R2 **初始化为 0**。

例 4.3 LD 指令。LD 需要两个操作数。LD 是 load 的缩写，用计算机术语来说，就是“到某个存储位置，读出其中保存的值，再把这个值放入一个寄存器”。所需的两个操作数，一个是从存储器读取的值，另一个是指令处理完成后保存该值的目的寄存器。计算待读位置的地址可以采用许多公式，每一种公式称为一种**寻址方式** (addressing mode)。操作码 LD 所指定的寻址方式称为 PC+offset。第 5 章将介绍 LC-3 ISA 的其他寻址方式，它们对应着计算存储位置地址的其他公式。

16 位的 LC-3 LD 指令格式如下：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	0	1	1	0	0	0	1	1	0
LD				R2				198							

LD 的四位操作码是 0010。位 [11:9] 标识指令执行后保存所读值的寄存器。位 [8:0] 用于计算待读位置的地址。由于 LD 采用 PC+offset 寻址，计算方法是：把位 [8:0] 中的补码整数符号扩展到 16 位，再与程序计数器当前的内容相加。总之，这条指令要求计算机把 198 加到 PC 的内容上，形成一个存储位置的地址，然后把该位置的内容装入 R2。

如果位 [8:0] 为 111111001，这条指令就应解释为：“把 -7 加到 PC 的内容上，形成一个存储位置的地址。”

4.3.2 指令周期 (不是时钟周期!)

指令在控制单元的指挥下，以系统而有序的方式逐步处理。处理一条指令所需的完整步骤序列称为**指令周期** (instruction cycle)。一个指令周期包含六个依次进行的阶

段，每个阶段需要零步或多步。这里说“零步”，是因为在大多数计算机的设计中，并非每条指令都需要全部六个阶段。稍后会讨论这一点。先来看看这六个阶段：

FETCH	取指令
DECODE	译码
EVALUATE ADDRESS	计算地址
FETCH OPERANDS	取操作数
EXECUTE	执行
STORE RESULT	存储结果

下面介绍具体过程，请再次参考图 4.3 中简化的 LC-3 数据通路。

4.3.2.1 取指令 (FETCH)

取指令阶段从存储器获得下一条指令，并将其装入控制单元的指令寄存器 **IR**。回想一下，计算机程序由多条指令组成，每条指令都表示为一个位序列，而且在冯·诺依曼模型中，整个程序都存放在存储器里。要执行一条指令所规定的工作，首先必须确定它在哪里。程序计数器 **PC** 保存下一条待处理指令的地址。因此，取指令阶段包含以下步骤：

1. 首先，将 **PC** 的内容装入 **MAR**。
2. 然后访问存储器，由存储器把下一条指令放入 **MDR**。
3. 最后，将 **MDR** 的内容装入 **IR**。

现在可以进入下一个阶段，对指令进行译码。不过，当前指令执行结束、准备取下一条指令时，我们希望 **PC** 已经保存了下一条指令的地址。因此，取指令阶段还要完成另一项工作：**将 PC 加一**。这样，只要当前指令执行时没有改变 **PC**，当前指令结束后，下一条指令的取指令阶段就会把相邻下一个存储位置的内容装入 **IR**。

完整的取指令阶段如下：

步骤 1： 将 **PC** 的内容装入 **MAR**，同时将 **PC** 加一。

步骤 2： 访问存储器，将指令放入 **MDR**。

步骤 3： 将 **MDR** 的内容装入 **IR**。

每一步都在控制单元的指挥下完成，就像前面说过的，乐队中的乐器跟随指挥棒一样。指挥棒每挥动一次，对应一个**机器周期** (machine cycle)。第 4.3.5 节将说明，每个机器周期耗时一个时钟周期。事实上，我们经常交替使用这两个术语。步骤 1 需要一个时钟周期。步骤 2 可能需要一个，也可能需要多个时钟周期，取决于访问存储器要花多久。步骤 3 需要一个时钟周期。在现代数字计算机中，一个时钟周期只是极短的一瞬间。

例如，3.1 GHz 的 Intel Core i7 每秒完成 31 亿个时钟周期。换句话说，一个时钟周期为十亿分之 0.322 秒，即 0.322 纳秒。回想一下，照亮书页的灯泡以每秒 60 次的速率亮灭。这样，在灯泡完成一次亮灭的时间里，今天的计算机可以完成超过 5100 万个时钟周期！

4.3.2.2 译码 (DECODE)

译码阶段检查指令，确定它要求微体系结构做什么。回想第 3 章研究的译码器。在 LC-3 中，一个 4 线—16 线译码器识别要处理的是 16 种操作码中的哪一种，尽管其中一种并未使用。输入是四位操作码 IR[15:12]，与输入操作码对应的输出线置为有效。根据译码器哪条输出线有效，再用其余 12 位确定处理该指令还需要哪些信息。

4.3.2.3 计算地址 (EVALUATE ADDRESS)

这个阶段计算处理指令所需的存储位置地址。回想 LD 的例子：LD 把存储器中的一个值装入寄存器。在那个例子中，先把指令的位 [8:0] 符号扩展到 16 位，再与 PC 当前的内容相加，得到地址。这个计算在计算地址阶段完成。并非所有指令都需要访问存储器来读写数据。例如，已经介绍过的 LC-3 ADD 和 AND，从寄存器或指令本身取得源操作数，并把运算结果存入寄存器，因此不需要计算地址阶段。

4.3.2.4 取操作数 (FETCH OPERANDS)

这个阶段取得处理指令所需的源操作数。在 LD 的例子中，它包含两步：把计算地址阶段得到的地址装入 MAR，然后读取存储器，把源操作数放入 MDR。

在 ADD 的例子中，这个阶段从 R2 和 R6 取得源操作数。对于大多数当前的微处理器，ADD 的这一阶段可以与指令执行，也就是指令周期的第五阶段同时进行。究竟怎样通过这种方式加快指令处理，是个很有意思的话题，不过我们只能留待你今后学习了。

4.3.2.5 执行 (EXECUTE)

这个阶段完成指令规定的运算。在 ADD 的例子中，就是让 ALU 执行加法。

4.3.2.6 存储结果 (STORE RESULT)

这是指令执行的最后一个阶段，把结果写入指定的目的位置。在许多计算机中，ADD 的这项工作在执行阶段就完成了。也就是说，在包括 LC-3 在内的许多计算机中，一条 ADD 指令可以在同一个时钟周期内取得源操作数、在 ALU 中完成加法，并把结果存入目的寄存器，不需要单独的存储结果阶段。

一个指令周期结束后，控制单元开始下一条指令的指令周期，重新从取指令阶段开始。PC 在前一个指令周期中已经更新，此时保存着相邻下一个存储位置中指令的

地址，因此接下来取到的是顺序上的下一条指令。处理就这样继续，直到发生某种情况打破顺序执行，或者程序执行结束。

再强调一次：虽然指令周期包含六个阶段，但并非所有指令都需要全部六个阶段。前面已经指出，LC-3 的 ADD 不需要单独的计算地址阶段，也不需要单独的存储结果阶段；LC-3 的 LD 不需要执行阶段。不过，其他 ISA 中确实存在需要全部六个阶段的指令。

例 4.4 ADD [eax], edx 是一条 Intel x86 指令，它需要指令周期的全部六个阶段。所有指令都需要前两个阶段：取指令和译码。这条指令使用 eax 寄存器计算一个存储位置的地址（计算地址），随后读取该位置的内容（取操作数），将其与 edx 寄存器的内容相加（执行），再把结果写回原先保存第一个源操作数的存储位置（存储结果）。

4.3.3 改变执行顺序

到目前为止，所描述的都是程序按顺序执行的情况：先执行第一条指令，再执行第二条，然后是第三条，依此类推。

我们已经介绍了两类指令：ADD 和 AND 对数据进行运算，是运算指令的例子；LD 把数据从一个位置移到另一个位置，是数据传送指令的例子。第 5 章进一步研究 LC-3 时，还会看到这两类指令的其他例子。

第三类是控制指令，用于改变指令的执行顺序。例如，我们很快就会看到，有时希望先执行第一、第二、第三条指令，再执行一遍第一、第二、第三条，然后第三次执行第一、第二条，等等。我们知道，每个指令周期都从把 PC 的内容装入 MAR 开始。因此，要改变执行顺序，就必须一条指令的取指令阶段将 PC 加一之后、下一条指令的取指令阶段开始之前，改变 PC 的内容。

控制指令在执行阶段装入 PC，覆盖取指令阶段写入的那个加一后的 PC 值。这样，下一个指令周期开始时，计算机从 PC 获取待取指令的地址，得到的就是上一条指令在执行阶段装入的地址，而不是程序中顺序上的下一条指令的地址。

最常见的控制指令是**条件分支**（conditional branch，BR）。它根据前面一条指令的结果——通常就是紧接在条件分支之前执行的那条指令——决定是否改变 PC 的内容。

例 4.5 BR 指令。BR 包含三部分：操作码，位 [15:12] 为 0000；待测试的条件，位 [11:9]；以及寻址方式所使用的位 [8:0]。如果前一条指令的结果符合位 [11:9] 指定的测试条件，就用位 [8:0] 形成要装入 PC 的地址。这里采用的寻址方式，也就是确定实际地址的方法，与 LD 相同：将位 [8:0] 符号扩展到 16 位，再与 PC 当前的内容相加。

假设下面这条 BR 指令位于存储位置 x36C9：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	0	1	1	1	1	1	1	1	0	1	0
BR				条件								-6			

操作码 0000 表明这是一条条件分支指令。位 [11:9] 为 101，指定对最近一次结果所

做的测试是：该结果是否非零。第 5 章将详细介绍可以对最近一次结果进行的各种测试。现在只用这一种：结果是否非零？位 [8:0] 表示值 -6。

假设前一条执行的指令位于 x36C8，是一条 ADD，而且加法结果为 0。“非零”测试失败，因此 BR 在执行阶段不做任何事。接下来执行的是 M[x36CA] 中的指令；地址 x36CA 是 BR 的取指令阶段将 PC 加一得到的。

反过来，如果 ADD 的结果不为 0，测试成功，BR 就把 x36C4 装入 PC。这个地址由位 [8:0] 符号扩展到 16 位后的值 -6，与加一后的 PC 值 x36CA 相加得到。

因此，位于 x36C9 的 BR 之后执行的，要么是 x36CA 中的指令，要么是 x36C4 中的指令，取决于 ADD 的结果是零还是非零。

4.3.4 指令周期的控制

指令周期由一个同步有限状态机控制。图 4.4 是它的简化状态图，突出展示本章讨论过的几条 LC-3 指令。与第 3.6 节研究的有限状态机一样，每个状态对应一个机器周期的活动，需要一个时钟周期完成。表示状态的节点内部写出了该状态所控制的处理工作，弧表示向下一状态的转换。

处理从状态 1 开始。取指令阶段需要三个时钟周期，对应前面介绍的三个步骤。第一个时钟周期把 PC 的内容装入 MAR，同时将 PC 加一。为了把 PC 的内容装入 MAR（见图 4.3），有限状态机必须使 GatePC 和 LD.MAR 有效。GatePC 把 PC 接到处理器总线上。LD.MAR 是 MAR 寄存器的写使能信号，使总线的内容在当前时钟周期结束时装入 MAR。（只要相应控制信号有效，寄存器就在时钟周期结束时装入新值。）

为了将 PC 加一，再看图 4.3，有限状态机必须设置 PCMUX 的选择线，使它选中标为 +1 的方框的输出，同时使 LD.PC 有效，在当前周期结束时把 PCMUX 的输出装入 PC。

随后，有限状态机进入状态 2。在这里，从存储器读出的指令装入 MDR。

状态 3 把指令从 MDR 传送到指令寄存器 IR。这要求有限状态机使 GateMDR 和 LD.IR 有效，从而在时钟周期结束时装入 IR。至此，指令周期的取指令阶段结束。

译码阶段需要一个时钟周期。在状态 4，有限状态机利用外部输入 IR，尤其是指令的操作码位，根据 IR[15:12] 中的具体操作码，转向处理相应指令的下一状态。图中画出了从状态 4 出发的 15 条路径中的三条。随后，处理继续逐个时钟周期推进，直到指令执行完成，下一状态逻辑使有限状态机回到状态 1。

前面已经讨论过，有时不能执行顺序上的下一条指令，而必须访问其他位置取得下一条待执行指令。这样改变指令处理流程的指令称为控制指令。对于条件分支 BR，指令周期结束时，PC 保存两个地址之一：要么是在状态 1 装入的加一后的 PC，要么是把 BR 的位 [8:0] 符号扩展后加到 PC 上得到、并在状态 63 装入的新地址。PC 最终保存哪个地址，取决于对最近一次结果的测试。

附录 C 完整描述 LC-3 的实现，包括完整的状态图和数据通路。本章不深入到那个程度。这里的目的是说明，指令周期的处理过程并没有什么神秘之处。只要状态图设计完整，就能逐个时钟周期控制每个指令周期中各个阶段所需的全部步骤。由于每

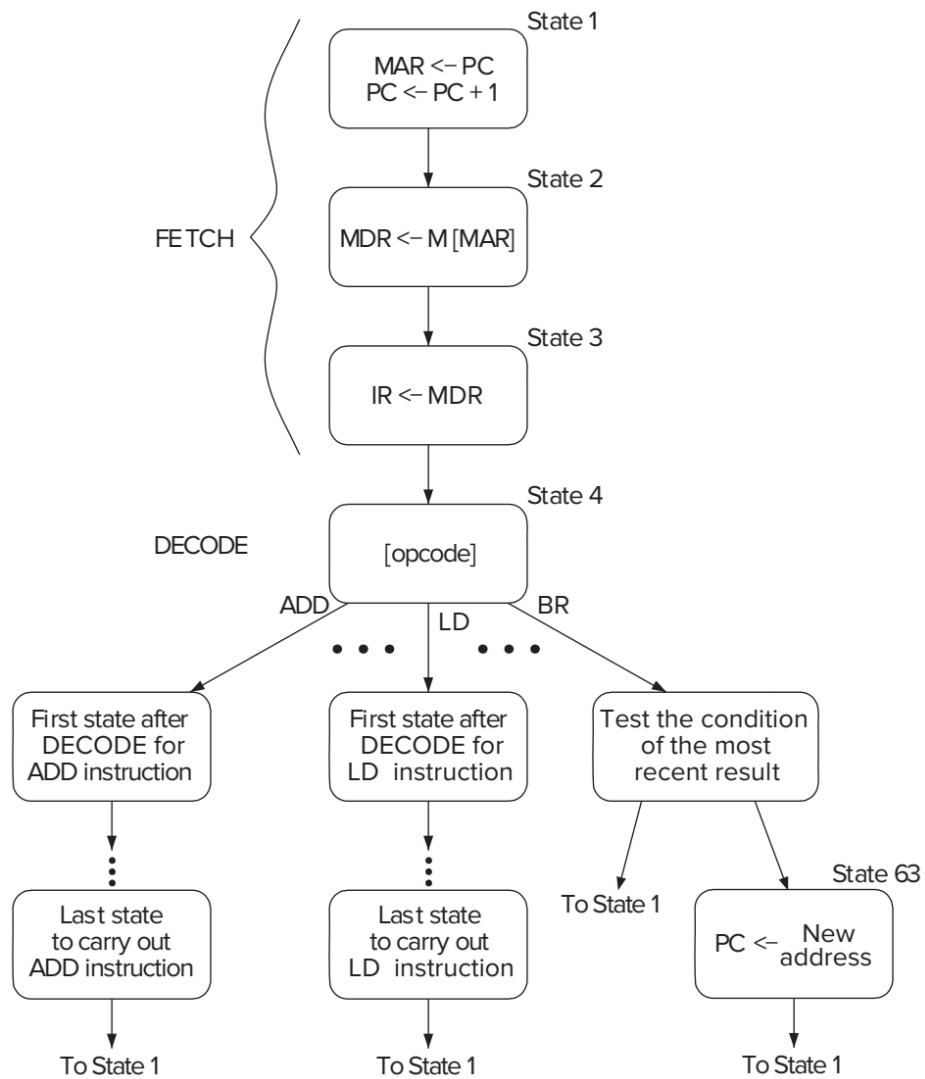


图 4.4 LC-3 的简化状态图

个指令周期结束时都会回到状态 1，这个有限状态机便能逐个时钟周期执行完整的计算机程序。

4.3.5 让计算机停机 (TRAP 指令)

按照前面的描述，计算机似乎会不断处理指令，一遍又一遍地执行指令周期，永无休止。计算机不会觉得无聊，那么，难道只能等有人拔掉插头、切断电源，这一切才会停止吗？

用户程序通常在操作系统的控制下执行。Linux、DOS、MacOS 和 Windows 都是操作系统的例子，而操作系统本身也只是计算机程序。对计算机来说，无论当前处理的是用户程序还是操作系统，指令周期都会继续。这对用户程序没有问题，因为每个用户程序都以一条控制指令结束，改变 PC，使计算机重新执行操作系统，而操作系统往往又会启动另一个用户程序。

可是，如果我们确实想停止这个可能无限延续的指令周期序列呢？回想指挥棒的比喻：它以每秒数十亿个时钟周期的速率打着拍子。要停止指令序列，就得让指挥棒停下来。我们已经多次指出，计算机内部有一个与指挥棒非常相似的部件，叫作**时钟**，它规定每个机器周期持续多久。第 3 章讲过，时钟使同步有限状态机能够继续进入下一个时钟周期。那一章中，下一个周期对应我们设计的危险警示牌的下一状态；这里，下一个周期对应指令周期的下一状态，可能是当前阶段的下一状态，也可能是下一个阶段的第一个状态。要停止指令周期，就需要停止时钟。

图 4.5(a) 是时钟电路框图，主要由时钟发生器和 RUN 锁存器组成。时钟发生器是一种晶体振荡器，即一种压电器件，你也许在物理或化学课上学过它。在这里，我们把晶体振荡器看作黑盒——回想第 1.4 节对黑盒的定义——它产生图 4.5(b) 所示的振荡电压。注意，这个电压多像指挥棒的运动。每个时钟周期，电压升到 1.2 V，然后降回 0 V。

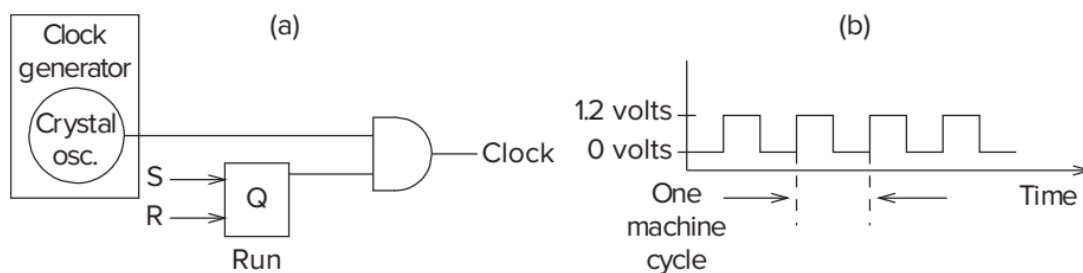


图 4.5 时钟电路及其控制

如果 RUN 锁存器处于状态 1，即 $Q = 1$ ，时钟电路的输出就与时钟发生器的输出相同。如果 RUN 锁存器处于状态 0，即 $Q = 0$ ，时钟电路的输出就是 0。

因此，要停止指令周期，只需要清零 RUN 锁存器。每台计算机都有某种办法做到这一点。一些较早的机器通过执行 HALT 指令完成。在 LC-3 和许多其他机器中，这件事由操作系统控制，第 9 章将介绍其过程。现在只需知道，如果用户程序需要操作系统帮助，就用 TRAP 指令（操作码为 1111）提出请求，并提供一个八位编码，称为

陷阱向量 (trap vector)，标识所需的帮助。八位编码 x25 告诉操作系统：程序已经执行完毕，计算机可以停止处理指令了。

思考：如果 HALT 指令能够清零 RUN 锁存器，从而停止指令周期，那么，需要什么指令把 RUN 锁存器置为 1，以重新开始指令周期？提示：这个问题有个小陷阱！

4.4 我们的第一个程序：乘法算法

现在，编写第一个程序所需的工具都齐了。我们有数据传送指令 LD，可以把数据从存储器装入寄存器；有两条运算指令 ADD 和 AND，分别进行算术运算和逐位逻辑运算；有控制指令 BR，可以把一个不同于加一后 PC 的地址装入 PC，让接下来执行的指令不再是相邻下一个存储位置中的指令；还有 TRAP 指令，也称为**系统调用** (system call)，可以请求操作系统帮助，这里用来停止计算机。有了这些，就能写出第一个程序了。

假设计算机不知道怎样把两个正整数相乘。过去，许多计算机确实如此：它们有 ADD 指令，却没有乘法指令。怎么办呢？假设我们想计算 5×4 。即使不知道怎样相乘，只要知道 $5 \times 4 = 5 + 5 + 5 + 5$ ，而计算机又有 ADD 指令，我们就能写出执行乘法的程序。只需把 5 累加四次。

图 4.6 展示了这个过程。

假设存储位置 x3007 (简记为 M[x3007]) 保存值 5，M[x3008] 保存值 4。首先，把这两个值从存储器复制到寄存器 R1 和 R2。我们准备在 R3 中累积加法结果，因此将 R3 初始化为 0。然后，把 5 加到 R3 上，并把 R2 减一，这样就能知道还需要再加几次。重复这两件事——R3 加 5，R2 减一——直到 R2 为 0。这说明已经把 5 加到 R3 上四次，计算完成，于是让计算机停机。R3 保存值 20，也就是乘法结果。

图 4.7 给出了实际的 LC-3 程序，它存放在 x3000 至 x3008。

用于跟踪下一条待执行指令的程序计数器，初始内容为 x3000。

把 M[x3007] 和 M[x3008] 中的数据传送到 R1、R2，使用的是数据传送指令 LD。执行 M[x3000] 中的 LD 时，LC-3 把偏移量 (这里为 6) 符号扩展到 16 位，与加一后的 PC 相加。由于取指令阶段已将 PC 加一，此时 PC 为 x3001。于是，计算机从 M[x3007] 取得数据，并装入 R1。M[x3001] 中的 LD 以相同方式执行。

把 R3 的内容与符号扩展后的立即数 0 做逐位 AND，再把结果装回 R3，就将 R3 初始化为 0。

随后，计算机执行 M[x3003] 和 M[x3004] 中的 ADD。M[x3003] 中的 ADD 把 R1 的内容加到 R3 的内容上，再把结果装入 R3。M[x3004] 中的 ADD 把 -1 加到 R2 的内容上，用它跟踪把 5 加到 R3 上的次数。

此时 PC 为 x3005。如果前一条指令，也就是 M[x3004] 中的指令，执行结果不为 0，M[x3005] 中的 BR 就把 x3003 装入 PC。如果前一条指令的结果为 0，BR 不做任何事，接下来执行 M[x3006] 中的指令；也就是说，加一后的 PC 为 x3006。

于是，两条 ADD 反复执行，直到 M[x3004] 中的指令产生结果 0，表明已经把 5 加了四次。最后，执行 M[x3006] 中的 TRAP，调用操作系统，让计算机停机。

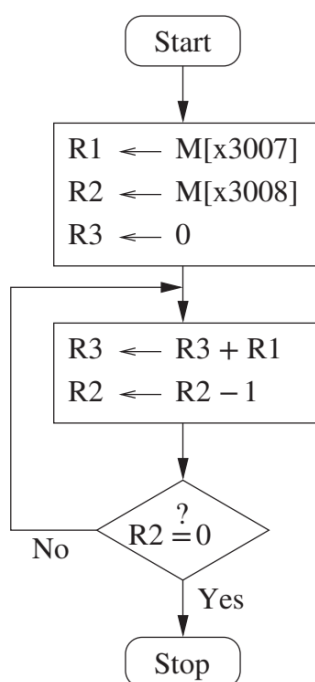


图 4.6 两个正整数相乘算法的流程图

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	0	1	0	0	0	1	0	0	0	0	0	0	1	1	0	R1 ← M[x3007]
x3001	0	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	R2 ← M[x3008]
x3002	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 ← 0
x3003	0	0	0	1	0	1	1	0	1	1	0	0	0	0	0	1	R3 ← R3+R1
x3004	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 ← R2-1
x3005	0	0	0	0	1	0	1	1	1	1	1	1	1	1	0	1	BR not-zero M[x3003]
x3006	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT
x3007	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	1	The value 5
x3008	0	0	0	0	0	0	0	0	0	0	0	0	0	1	0	0	The value 4

图 4.7 不使用乘法指令实现乘法的程序

习题

- 4.1 列出冯·诺依曼模型的五个组成部分，并说明每一部分的用途。
- 4.2 简要描述存储器与处理单元之间的接口，也就是两者如何通信。
- 4.3 “程序计数器”这个名称有什么容易误导人的地方？为什么“指令指针”更能说明它的作用？
- 4.4 什么是计算机的字长？字长怎样影响计算机能够计算什么？换句话说，结合第1章所学知识，下面的说法是否成立：字长较大的计算机能处理更多信息，因此能计算的东西比字长较小的计算机更多？
- 4.5 下表表示一个小型存储器。请据此回答问题。

地址	数据
0000	0001 1110 0100 0011
0001	1111 0000 0010 0101
0010	0110 1111 0000 0001
0011	0000 0000 0000 0000
0100	0000 0000 0110 0101
0101	0000 0000 0000 0110
0110	1111 1110 1101 0011
0111	0000 0110 1101 1001

- 位置3包含什么二进制值？位置6呢？
 - 每个位置中的二进制值可以有许多解释。我们已经知道，二进制值可以表示无符号数、补码有符号数、浮点数等。
 - 将位置0和位置1的内容解释为补码整数。
 - 将位置4的内容解释为ASCII值。
 - 将位置6和位置7的内容合起来解释为一个IEEE浮点数。位置6保存 `number[15:0]`，位置7保存 `number[31:16]`。
 - 将位置0和位置1的内容解释为无符号整数。
 - 在冯·诺依曼模型中，存储位置的内容也可以是一条指令。如果将位置0中的位模式解释为指令，它表示什么指令？
 - 二进制值还可以解释为存储器地址。假设位置5中的值是一个地址，它指向哪个位置？那个位置包含什么二进制值？
- 4.6 指令由哪两个部分组成？这两个部分分别包含什么信息？
- 4.7 假设一条32位指令采用以下格式：

OPCODE	SR	DR	IMM
--------	----	----	-----

如果有 60 种操作码和 32 个寄存器，立即数 IMM 能表示的值的范围是什么？假设 IMM 为补码值。

4.8 假设一条 32 位指令采用以下格式：

OPCODE	DR	SR1	SR2	UNUSED
--------	----	-----	-----	--------

如果有 225 种操作码和 120 个寄存器：

- 表示 OPCODE 至少需要多少位？
- 表示目的寄存器 DR 至少需要多少位？
- 指令编码中最多可以有多少个 UNUSED（未使用）位？

4.9 指令周期的取指令阶段完成两项重要工作。一项是把下一条待处理指令装入 IR，另一项是什么？

4.10 例 4.1、4.2 和 4.5 展示了 ADD、LDR 和 JMP 指令的处理过程。¹根据指令操作码的不同，PC、IR、MAR 和 MDR 会在指令周期的不同阶段被写入。请在下表每个空格中，填入会在相应阶段（列）写入相应寄存器（行）的操作码。

	取指令	译码	计算地址	取数据	执行	存储结果
PC						
IR						
MAR						
MDR						

4.11 列出指令周期的各个阶段，并简要说明每个阶段进行哪些操作。

4.12 对于 ADD、LDR 和 JMP，写出指令周期各个阶段进行的操作。

4.13 假设读写存储器需要 100 个周期，而读写寄存器只需要一个周期。分别计算 IA-32 指令 ADD [eax], edx（参见例 4.4）和 LC-3 指令 ADD R6, R2, R6 在指令周期每个阶段所需的周期数。假设每个必需阶段都耗时一个周期，除非需要访问存储器。

4.14 假设 R3 保存 x369C，描述 JMP 指令的执行过程（原书提示参见例 4.5）。²

4.15 如果 HALT 指令能够清零 RUN 锁存器，从而停止指令周期，那么，需要什么指令把 RUN 锁存器置为 1，以重新开始指令周期？

4.16 a. 如果一个机器周期为 2 纳秒，即 2×10^{-9} 秒，每秒有多少个机器周期？

¹译注：原书此处的指令名称与例题编号不一致；本章例 4.1、4.2、4.5 实际分别介绍 ADD、AND、BR，例 4.3 介绍 LD。题干的 LDR、JMP 保留原文，这两条指令在第 5 章介绍。习题 4.12 和 4.14 也涉及它们。

²译注：例 4.5 实际介绍 BR，不能直接用作 JMP 的例子；这里保留原题所问的 JMP 和寄存器数值。

- b. 如果计算机处理每条指令平均需要八个周期，而且一次从头到尾只处理一条指令，那么一秒可以处理多少条指令？
- c. 提前看看后续课程的内容：今天的微处理器加入了许多特性，以提高每秒处理的指令数。其中一种相当于计算机中的装配线。指令周期的每个阶段都用一个或多个独立逻辑部分实现。处理一条指令时，每一步接着前一步在上一个机器周期结束的地方继续。有了这个特性，每个机器周期都可以从存储器取一条指令，并在周期结束时交给译码器。下一个机器周期中，译码器对它译码，同时又取下一条指令。这就形成了一条装配线。假设指令位于存储器中的连续地址，并且没有任何事情打断顺序执行，有了这条装配线，微处理器每秒能执行多少条指令？（这条装配线称为**流水线**，你将在后续课程中学习。流水线不能始终以最大速率运行，原因有很多，有些后续课程会详细讨论这些问题。）

4.17 ★ 本题连续进行五次存储器访问。下表给出每次访问是读（load）还是写（store），以及访问结束时 MAR 和 MDR 的内容，其中一些项目未给出。为减少不必要的书写，本题把寻址粒度改为五位，而不是熟悉的 LC-3 的 16 位。

存储器操作

	R/W	MAR	MDR				
操作 1	W		1	1	1	1	0
操作 2							
操作 3	W		1	0			
操作 4							
操作 5							

下面三个表分别给出第一次访问之前、第三次访问之后和第五次访问之后，存储位置 x4000 至 x4004 的内容。同样，并非所有项目都已给出。为了使问题只有一个正确答案，增加一条不常见的限制：**MDR 只能通过读（load）访问从存储器装入数据。**

第一次访问之前

x4000	0	1	1	0	1
x4001	1	1	0	1	0
x4002	1				
x4003	1	0	1	1	0
x4004	1	1	1	1	0

第三次访问之后

x4000					0
x4001	0				0
x4002					
x4003					
x4004	1	1	1	1	0

第五次访问之后

x4000					
x4001					
x4002					
x4003	0	1	1	0	1
x4004	1	1	1	1	0

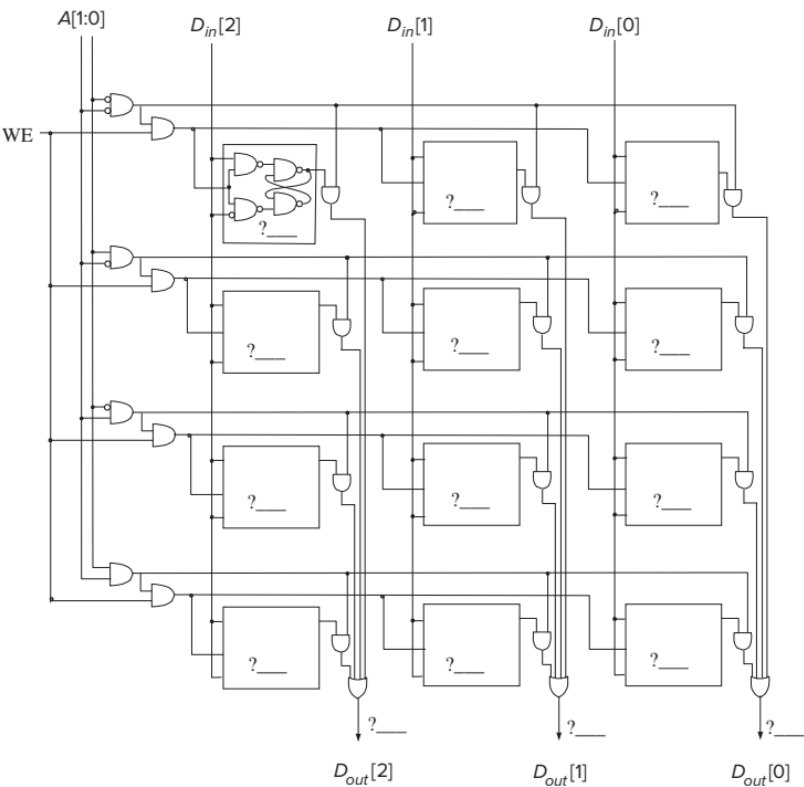
你的任务是填入所有缺失项目。

提示：你知道，写存储器时需要把地址装入 MAR，把待写入（存储）的数据装入 MDR。MDR 中的数据必须来自此前的一次读（load）访问。

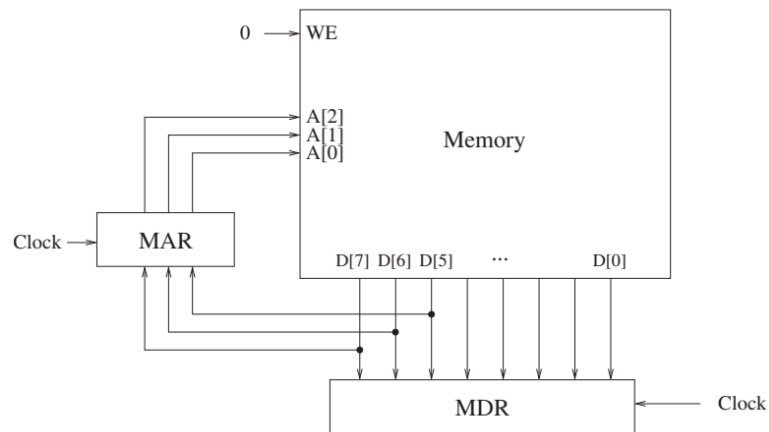
4.18 ★ 课堂上讨论过的 $2^2 \times 3$ 位存储器在连续五个时钟周期中被访问。下表给出每次访问时两位地址、一位写使能和三位数据输入信号的值。

	A[1:0]	WE	D _{in} [2 : 0]
周期 1	0 1	1	1 0 1
周期 2	1 1	0	1 1 0
周期 3	1 0	1	0 1 0
周期 4	0 1	1	0 1 1
周期 5	0 0	0	0 0 0

你的任务是填出第五个周期即将结束时，每个存储单元所保存的值，以及三条数据输出线的值。假设最初全部 12 个存储单元都保存 1。下图中每个问号 (?) 表示一个需要填写的值。



4.19 ★ 下图是一个包含八个位置的按字节寻址存储器,以及相连的 MAR 和 MDR。MAR 和 MDR 都由触发器构成,每个时钟周期开始时,根据各自输入线上的值锁存。每个周期都会发起一次存储器读操作,数据在该周期结束之前可用。



周期 1 开始之前，MAR 为 000，MDR 为 00010101，各个存储位置的内容如下。

存储位置	值
x0	01010000
x1	11110001
x2	10000011
x3	00010101
x4	11000110
x5	10101011
x6	00111001
x7	01100010

- a. 周期 1 即将结束时，MAR 和 MDR 的内容分别是什么？

MAR: MDR:

- b. 周期 4 即将结束时，MDR 的内容是什么？

MDR:

第五章 LC-3

第4章讨论了计算机的基本组成部分：存储器；处理单元及其临时存储空间，通常是一组寄存器；输入输出设备；以及指挥各个单元（包括它自己！）活动的控制单元。我们还研究了指令周期的六个阶段：取指令、译码、计算地址、取操作数、执行和存储结果，并用 LC-3 的一些部件说明有关概念。实际上，我们已经介绍了五种操作码：两条运算指令 ADD 和 AND，一条数据传送指令 LD，以及两条控制指令 BR 和 TRAP。现在可以更加深入地研究 LC-3 了。

回想第1章，ISA 是软件提出的要求与硬件实际完成的工作之间的接口。本章将介绍 LC-3 ISA 的大部分重要特性，少数内容留到第8、9章。要用 LC-3 自己的语言，也就是机器语言编写程序，就需要这些知识。

附录 A 完整描述了 LC-3 的 ISA。

5.1 ISA 概述

ISA 规定了软件必须知道的有关计算机的全部信息。换句话说，程序员用计算机自己的机器语言编程时可以使用的全部功能，都由 ISA 规定。不过，大多数人并不直接使用机器语言，而是选择 C++、Python，或者已有五十多年历史的 Fortran、COBOL 等高级语言。因此，对于希望把高级语言程序翻译成机器语言的人——编译器作者——ISA 也规定了他所需要知道的全部计算机信息。

ISA 规定存储器组织、寄存器组和指令集，包括指令集中的操作码、数据类型和寻址方式。

5.1.1 存储器组织

LC-3 的存储器地址空间包含 2^{16} ，即 65,536 个位置，寻址粒度为 16 位。这 65,536 个地址并非全部用于存储位置，不过这个问题留到第9章讨论。LC-3 通常处理的数据单位是 16 位，因此把 16 位称为一个字，并说 LC-3 是按字寻址的。

5.1.2 寄存器

从存储器取得数据通常需要远多于一个时钟周期的时间，因此，LC-3 与几乎所有计算机一样，提供了可在一个时钟周期内访问的额外临时存储位置。

最常见的形式是一组寄存器，LC-3 也采用这种形式。组中的每个寄存器称为**通用寄存器**（general purpose register, GPR）。与存储位置一样，寄存器保存以后可用于运算的信息，每个寄存器通常保存一个字，在 LC-3 中就是 16 位。

寄存器必须能够被唯一标识。LC-3 规定了八个 GPR，每个由一个三位寄存器编

号标识，分别称为 R0、R1、...、R7。图 5.1 是寄存器组某一时刻的快照，R0 至 R7 分别保存 1、3、5、7、-2、-4、-6、-8。寄存器组也称为 register file。

Register 0	(R0)	0000000000000001
Register 1	(R1)	0000000000000011
Register 2	(R2)	0000000000000101
Register 3	(R3)	0000000000000111
Register 4	(R4)	1111111111111110
Register 5	(R5)	1111111111111100
Register 6	(R6)	1111111111111010
Register 7	(R7)	1111111111111000

图 5.1 LC-3 寄存器组的快照

第 4 章讲过，把 R0 的内容与 R1 的内容相加、将结果存入 R2 的指令编码如下：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	0	1	0	0	0	0	0	0	0	0	0	1
ADD				R2			R0			R1					

ADD 的两个源操作数由位 [8:6] 和 [2:0] 指定，结果的目的位置由位 [11:9] 指定。图 5.2 是执行 ADD R2, R1, R0 之后的寄存器组。¹

Register 0	(R0)	0000000000000001
Register 1	(R1)	0000000000000011
Register 2	(R2)	0000000000000100
Register 3	(R3)	0000000000000111
Register 4	(R4)	1111111111111110
Register 5	(R5)	1111111111111100
Register 6	(R6)	1111111111111010
Register 7	(R7)	1111111111111000

图 5.2 图 5.1 的寄存器组执行 ADD 后的状态

¹译注：原书此处交换了 R0、R1 的书写顺序；上方机器码对应 ADD R2, R0, R1。由于加法满足交换律，两者产生相同结果，图中 R2 均为 4。

5.1.3 指令集

回想第4章，一条指令由操作码和操作数组成：前者说明要求计算机做什么，后者说明对什么做。指令集由操作码、数据类型和寻址方式共同定义。寻址方式确定操作数在哪里，数据类型规定操作数如何用0和1表示。

指令ADD R2, R0, R1的操作码是ADD, 使用一种寻址方式——寄存器寻址, 以及一种数据类型——补码整数。它要求计算机执行补码整数加法, 指定到哪些GPR中取得操作数, 以及把结果写入哪个GPR。

第4章还讲过, ADD也可以同时使用寄存器和立即数两种寻址方式, 这时一个操作数直接包含在指令的位[4:0]中。

图5.3列出了LC-3的全部指令、各操作码的位[15:12]编码及指令格式。其中一些在第4章已经见过, 其他许多指令将在第5.2、5.3、5.4节介绍。

5.1.4 操作码

有些ISA的操作码很多, 程序可能执行的各种任务各有相应操作码。x86 ISA就有超过200种操作码。另一些ISA的操作码集合则很小。有些ISA提供专门支持科学计算的操作码。例如, Hewlett Packard的Precision Architecture可以用一个操作码指定复合运算 $(A \cdot B) + C$, 即先乘后加, 使用A、B、C三个源操作数。另一些ISA有处理万维网上视频图像的指令。Intel曾为x86 ISA增加一批指令, 最初称为MMX, 因为它们扩展(eXtend)了ISA, 以辅助使用网络的多媒体(MultiMedia)应用。还有些ISA提供辅助操作系统工作的专用操作码。例如, 20世纪80年代流行的VAX ISA, 在切换到另一个程序之前, 可用一个操作码保存正在执行的程序的相关信息, 而其他计算机需要一长串指令才能完成。制定新ISA时, 哪些指令应当纳入、哪些应当排除, 通常是公司内部争论激烈的话题。

LC-3 ISA有15条指令, 各有唯一的操作码, 位于指令的位[15:12]。四位本来可以表示16种不同操作码, 但这里只规定15种, 编码1101尚未定义, 留给目前无法预见的未来需要。

第4章简要介绍过, 指令分为三类, 操作码也相应分为三类: 运算、数据传送和控制。运算指令处理信息; 数据传送指令在存储器与寄存器之间、寄存器或存储器与输入输出设备之间传送信息; 控制指令改变随后执行的指令序列, 使计算机能够执行相邻下一个存储位置以外的指令。

5.1.5 数据类型

第2.1.2节首次指出, 数据类型是一种信息表示方式, 而且ISA必须提供能对这种表示进行运算的操作码。同一信息在计算机中有许多表示方法, 这并不奇怪, 因为日常生活中我们也经常用不同方法表示同一信息。问一个孩子多大了, 他可能伸出三根手指, 表示三岁。如果特别早慧, 他可能写出十进制数字3。如果这个“孩子”是大学CS或CE专业的学生, 他可能写出3的16位二进制表示0000000000000011。如果学的是化学, 他可能写出 3.0×10^0 。四者表示的值都是3。

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
ADD ⁺	0001				DR			SR1			0	00		SR2		
ADD ⁺	0001				DR			SR1			1	imm5				
AND ⁺	0101				DR			SR1			0	00		SR2		
AND ⁺	0101				DR			SR1			1	imm5				
BR	0000				n	z	p	PCoffset9								
JMP	1100				000			BaseR			000000					
JSR	0100				1	PCoffset11										
JSRR	0100				0	00		BaseR			000000					
LD ⁺	0010				DR			PCoffset9								
LDI ⁺	1010				DR			PCoffset9								
LDR ⁺	0110				DR			BaseR			offset6					
LEA	1110				DR			PCoffset9								
NOT ⁺	1001				DR			SR			111111					
RET	1100				000			111			000000					
RTI	1000				000000000000											
ST	0011				SR			PCoffset9								
STI	1011				SR			PCoffset9								
STR	0111				SR			BaseR			offset6					
TRAP	1111				0000			trapvect8								
reserved	1101															

图 5.3 LC-3 全部指令的格式。注：+ 标记会修改条件码的指令。

不仅同一个数可以在不同数据类型中表示为不同位模式，同一个位模式也可以根据数据类型的不同表示不同的数。例如，16 位 0011000100110000 可以表示补码整数 12,592，也可以表示字符序列“10”的 ASCII 码，或者表示一个位向量，其中 b_{13} 、 b_{12} 、 b_8 、 b_5 和 b_4 具有该位向量所表示的性质。²

这同样不奇怪。日常生活中的一种表示也可能有多种解释，例如红灯。开车时在道路上看到红灯，意味着停车；在 Montreal Canadiens 队打冰球的 Centre Bell 看到红灯，则意味着刚有人进球。

每个操作码都按照它支持的数据类型解释操作数的位模式。例如，ADD 的硬件将操作数解释为补码整数。因此，即使程序员把 0011000100110000 放入 R3 时，心里想的是整数 10，指令 ADD R4, R3, #10 写入 R4 的也会是整数 12,602，而不是整数 20 的 ASCII 码。为什么？因为 ADD 按补码整数解释操作数，而不是按 ASCII 码解释，与生成这些数的人原本想表达什么无关。

5.1.6 寻址方式

寻址方式是规定操作数位置的机制。操作数通常可以在三个地方找到：存储器、寄存器，或指令本身。如果操作数属于指令的一部分，就称为**字面量**（literal）或**立即操作数**（immediate operand）。“字面量”强调指令中的位本身就构成操作数；“立即”强调可以直接从指令取得操作数，无需到别处查找。

LC-3 支持五种寻址方式：立即数（或字面量）寻址、寄存器寻址，以及三种存储器寻址——PC 相对寻址、间接寻址和 Base+offset 寻址。第 5.2 节将说明运算指令使用寄存器和立即数两种寻址方式，第 5.3 节将说明数据传送指令使用这五种中的四种。

5.1.7 条件码

最后介绍条件码，完成对 LC-3 ISA 的概览。LC-3 有三个一位寄存器。每当一条运算指令或装入指令执行后将值写入八个通用寄存器之一，这三个一位寄存器就会分别置位（设为 1）或清零（设为 0）。运算指令计算结果并写入 GPR，装入指令读取存储位置的内容并写入 GPR。所有运算指令和装入指令将分别在第 5.2、5.3 节介绍。

这三个寄存器名为 N、Z、P，分别对应 negative（负）、zero（零）、positive（正）。每次运算指令或装入指令写入 GPR 时，根据写入值是负、零还是正，分别设置 N、Z、P：结果为负，N 置位，Z、P 清零；结果为零，Z 置位，N、P 清零；结果为正，P 置位，N、Z 清零。

这三个一位寄存器统称**条件码**，因为它们的状态用于改变程序中指令的执行顺序。许多 ISA 使用条件码，例如 SPARC 和 x86。第 5.4 节将介绍 LC-3 如何利用它们。

²译注：原书列出的后三个位下标与给定位串不符；这里按从右向左、最低位为 b_0 的约定修正。

5.2 运算指令

5.2.1 ADD、AND 和 NOT

运算指令处理数据。常见例子包括 ADD、SUB、MUL、DIV 等算术运算，以及 AND、OR、NOT、XOR 等逻辑运算。LC-3 有三条运算指令：ADD、AND、NOT。

NOT 的操作码为 1001，是唯一执行一元运算的运算指令，也就是只需要一个源操作数。NOT 将 16 位源操作数逐位取反，把结果存入目的寄存器。源和目的都采用寄存器寻址：位 [8:6] 指定源寄存器，位 [11:9] 指定目的寄存器，位 [5:0] 必须全为 1。

若 R5 初值为 0101 0000 1111 0000，执行下列指令后，R3 的值为 1010 1111 0000 1111：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	0	1	1	0	1	1	1	1	1	1	1	1
NOT				R3				R5							

图 5.4 展示执行这条 NOT 所用的关键数据通路。NOT 是一元运算，因此只用到 ALU 的 A 输入，值来自 R5。送给 ALU 的控制信号指定执行逐位取反。ALU 的输出，也就是运算结果，存入 R3，同时设置条件码，完成 NOT 的执行。

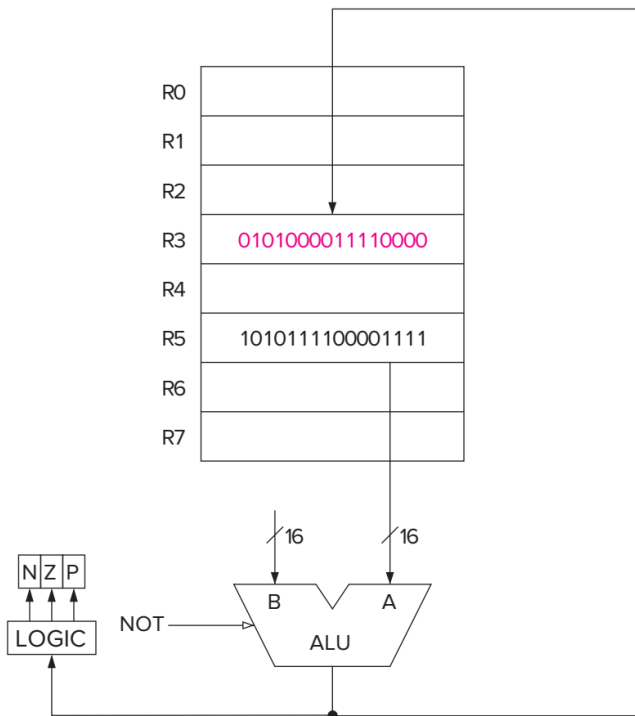


图 5.4 执行 NOT R3, R5 所涉及的数据通路

回想第 4 章，ADD（操作码 0001）和 AND（操作码 0101）执行的都是二元运算，需要两个 16 位源操作数。ADD 对它们做补码加法，AND 对对应的每一对位做逐位 AND。与 NOT 一样，ADD 和 AND 的一个源操作数及目的操作数使用寄存器寻址。位 [8:6] 指定源寄存器，位 [11:9] 指定写入结果的目的寄存器。

5.2.2 立即数

第4章也介绍过，ADD 和 AND 的第二个源操作数既可使用寄存器，也可作为立即数，由位 [5] 决定。位 [5] 为 0 时，第二个源操作数在寄存器中，由位 [2:0] 指定哪个寄存器；此时位 [4:3] 为 00，以完成指令编码。

对于下面这条 ADD，如果 R4 为 6，R5 为 -18，执行后 R1 将为 -12：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	0	0	1	1	0	0	0	0	0	1	0	1
ADD				R1			R4			R5					

位 [5] 为 1 时，第二个源操作数包含在指令中。执行 ADD 或 AND 前，先将位 [4:0] 符号扩展到 16 位，作为第二个源操作数。结果写入目的寄存器，并设置条件码，完成指令执行。图 5.5 展示执行 ADD R1, R4, #-2 所用的关键数据通路。

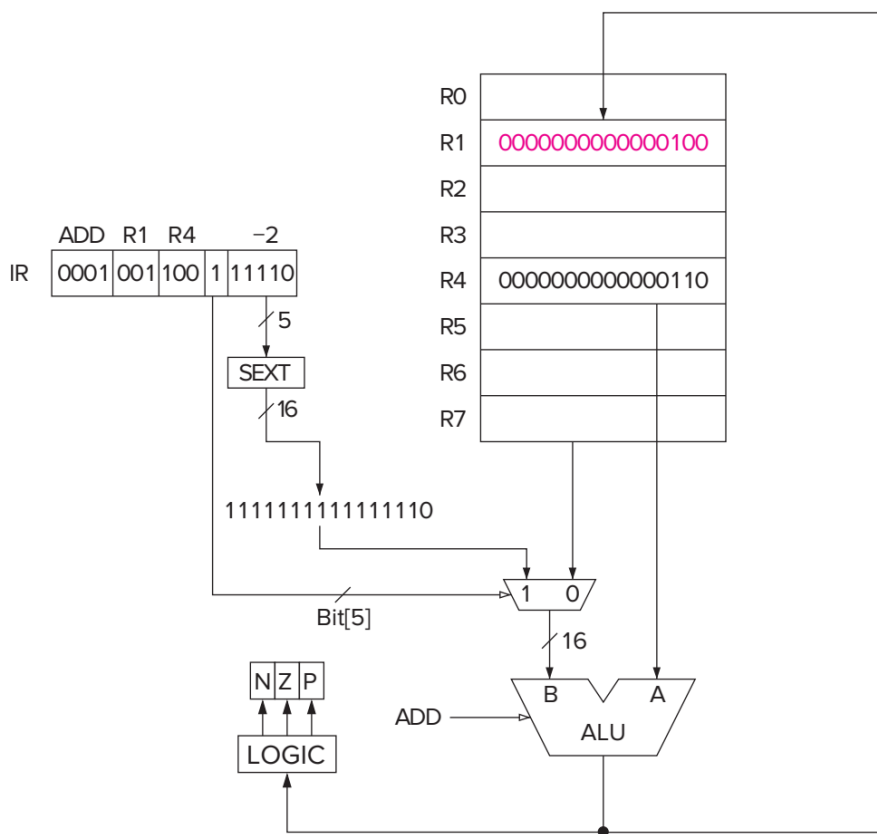


图 5.5 执行 ADD R1, R4, #-2 所涉及的数据通路。

由于 ADD、AND 的立即操作数必须装得进位 [4:0]，并非所有补码整数都能用作立即数。**思考：**哪些整数可以？

例 5.1 下面的指令做什么？

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0
AND				R2			R2			0					

答案：清零寄存器 2，即把所有位设为 0。

例 5.2 下面的指令做什么？

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	1	1	0	1	1	0	1	0	0	0	0	1
ADD				R6			R6			1					

答案：寄存器 6 加一，即 $R6 \leftarrow R6 + 1$ 。

注意，同一条指令中，一个寄存器可以同时用作源和目的。LC-3 的所有指令都允许这样使用寄存器。

例 5.3 回想一下，补码整数取负可以先逐位取反，再加一。假设 A、B 分别在 R0、R1 中，哪三条指令可以计算“A 减 B”，并把结果写入 R2？

答案：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1
NOT				R1			R1								

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	0	1	0	0	0	1	1	0	0	0	0	1
ADD				R2			R1			1					

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	1	0	1	0	0	0	0	0	0	0	1	0	
ADD				R2			R0			R2					

思考：这段指令序列还产生了什么不理想的结果？怎样轻松避免？

5.2.3 LEA 指令（虽然它并不真是运算指令）

把 LEA 归到哪一类，可以拿来争论一番——如果你没有更重要的事可做的話！它并不真正对数据进行运算，只是把一个地址装入寄存器。它显然也没有把数据从存储器传送到寄存器，更不是控制指令。总得找个地方介绍它，所以我们把它放在这里！

LEA 的操作码为 1110。它将加一后的程序计数器与符号扩展后的位 [8:0] 相加，把得到的值装入位 [11:9] 指定的寄存器。第 4 章介绍 LD 时已经见过这种地址计算方法。不过，这里不访问存储器，只把计算出的地址装入寄存器。也许 CEA，即 Compute Effective Address（计算有效地址），是更好的名称。但是，工业界许多具有该指令的微处理器都称它为 LEA，即 Load Effective Address（装入有效地址），所以我们也采用同样的缩写。

稍后会看到，如果要把一个距离当前指令很近的地址装入寄存器作为初值，LEA 很有用。

如果 x4018 中的指令是 LEA R5, #-3，且 PC 为 x4018，那么执行后 R5 将为 x4016：

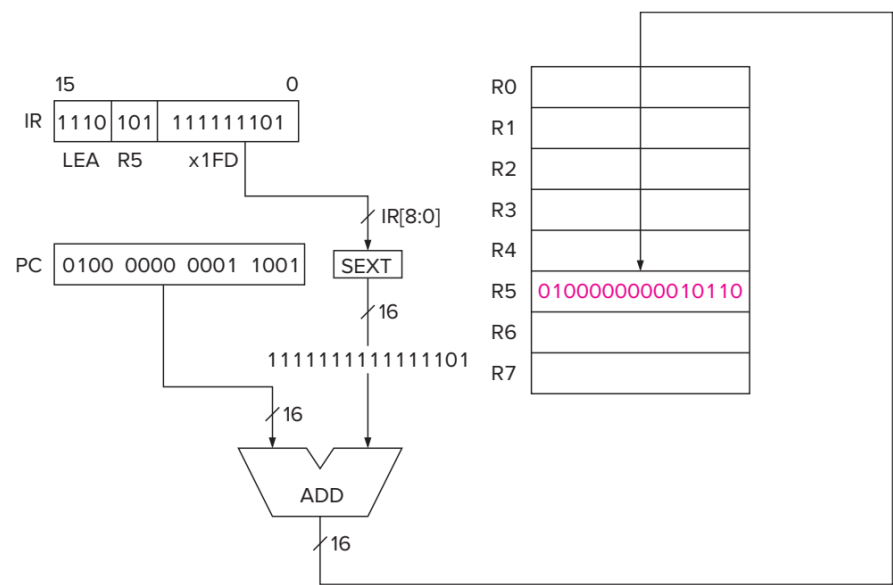


图 5.6 执行 `LEA R5, #-3` 所涉及的数据通路。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	0	1	0	1	1	1	1	1	1	1	0	1	1
LEA				R5				-3							

思考：为什么 R5 不是 x4015？

图 5.6 展示执行 `LEA` 所需的数据通路。注意，装入寄存器的值不涉及任何存储器访问，也不影响条件码。

5.3 数据传送指令

数据传送指令在通用寄存器与存储器之间、寄存器与输入输出设备之间传送信息。暂时不讨论把信息从输入设备传到寄存器、从寄存器传到输出设备的过程，那是第 9 章的重要内容。本章只讨论存储器与 GPR 之间的数据传送。

从存储器向寄存器传送信息称为**装入** (load)，从寄存器向存储器传送信息称为**存储** (store)。两种情况下，源操作数所在位置的内容都保持不变，目的位置则被源操作数覆盖，原先保存在那里的值随之丢失。

LC-3 有六条数据传送指令：`LD`、`LDR`、`LDI`、`ST`、`STR` 和 `STI`。它们的格式如下：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
opcode				DR / SR				Address Generation							

数据传送指令需要两个操作数，一个是源，即待传送的数据；一个是目的，即数据要传到的位置。其中一个位置是寄存器，另一个是存储位置或输入输出设备。本章假设第二个操作数在存储器中，第 9 章再研究它是输入输出设备的情况。

位 [11:9] 指定寄存器。如果是装入指令，`DR` 是目的 GPR，在指令周期结束时保存从存储器读出的值；如果是存储指令，`SR` 是源寄存器，保存即将写入存储器的值。

位 [8:0] 是地址生成位，提供计算第二个操作数的 16 位地址所需的信息。对于 LC-3 数据传送指令，解释位 [8:0] 有三种方式，统称为寻址方式。操作码决定如何解释这九位，也就是决定用三种寻址方式中的哪一种取得操作数地址。

5.3.1 PC 相对寻址

LD（操作码 0010）和 ST（操作码 0011）指定 PC 相对寻址。第 4 章已经讨论过这种方式。它之所以这样命名，是因为指令的位 [8:0] 指定相对于 PC 的偏移量。把位 [8:0] 符号扩展到 16 位，再加入到**加一后的 PC** 上，就得到存储器地址。“加一后的 PC”指取指令阶段结束、PC 已经自增后的内容。对于 LD，PC+offset 是待读存储位置的地址，其内容装入位 [11:9] 指定的寄存器；对于 ST，则把位 [11:9] 指定寄存器的内容写入该地址。装入指令还会根据装入值是负、零还是正，设置 N、Z、P 条件码。³

如果下面的指令位于 x4018，它会把 x3FC8 中的内容装入 R2：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	1	0	0	1	0	1	1	0	1	0	1	1	1	1
LD				R2				x1AF							

图 5.7 标出了执行 LD 的三个步骤。步骤 1，把加一后的 PC（x4019）与 IR[8:0] 符号扩展后的值 xFFAF 相加，将结果 x3FC8 装入 MAR。步骤 2，读存储器，将 x3FC8 的内容装入 MDR。假设其中保存的是 5。步骤 3，将 5 装入 R2，并设置 NZP 条件码，完成指令周期。

注意，操作数地址只能落在整个存储器中的一小段范围内：相对于 LD 或 ST 指令本身的位置，偏移范围为 -255 至 +256 个位置。这一范围由位 [8:0] 符号扩展后的值决定。如果装入指令需要访问更远的位置，就必须采用另外两种寻址方式之一。

5.3.2 间接寻址

LDI（操作码 1010）和 STI（操作码 1011）指定**间接寻址**。首先，与 LD、ST 完全一样地形成一个地址。不过，这不是待装入或存储的操作数的地址，而是**保存操作数地址的位置的地址**，因此称为“间接”。操作数本身可以位于计算机存储器的任何位置，不像 LD、ST 那样受指令位 [8:0] 所给范围的限制。与其他装入、存储指令一样，LDI 的目的寄存器和 STI 的源寄存器由位 [11:9] 指定。

如果下面的指令位于 x4A1B，且 x49E8 的内容为 x2110，执行后就会把 x2110 中的内容装入 R3：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	1	0	0	1	1	1	1	0	0	1	1	0	0	0
LDI				R3				x1CC							

图 5.8 展示相应数据通路。步骤 1，与 LD、ST 一样，把加一后的 PC（x4A1C）与 IR[8:0] 符号扩展后的值 xFFCC 相加，把结果 x49E8 装入 MAR。步骤 2，读存储器，

³译注：原书在同一段中介绍 LD、ST 后笼统提到设置条件码。根据图 5.3 和第 5.1.7 节，此处仅指 LD；ST 不修改条件码。

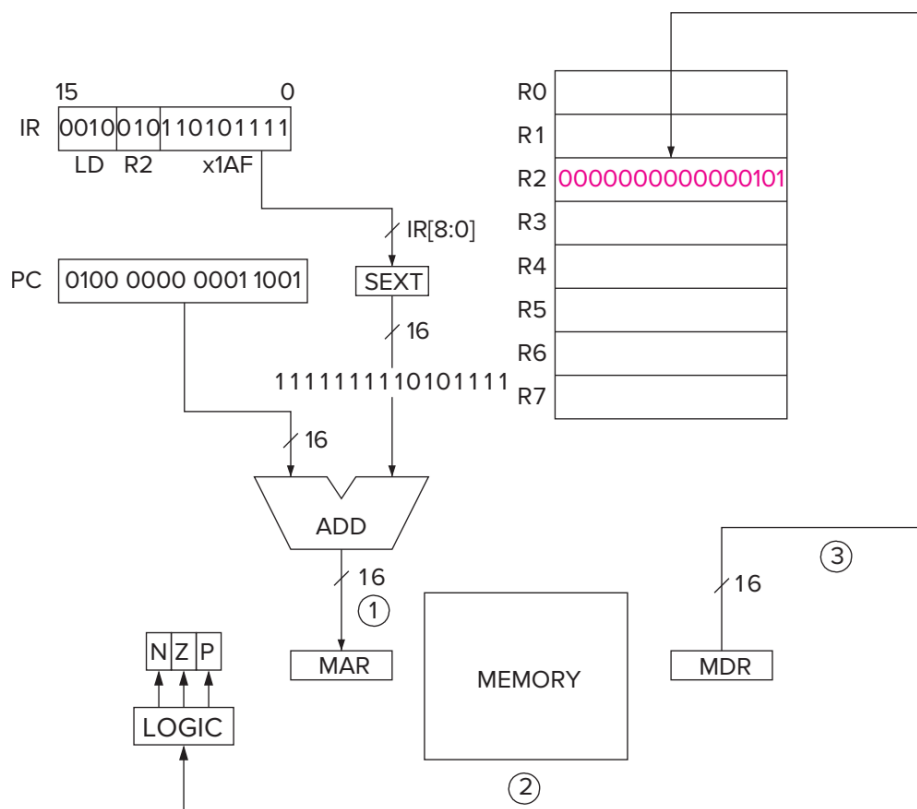


图 5.7 执行 LD R2, x1AF 所涉及的数据通路

把 x49E8 中保存的 x2110 装入 MDR。步骤 3，由于 x2110 是操作数的地址，而不是操作数本身，将它装入 MAR。步骤 4，再次读存储器，将 x2110 的内容装入 MDR。假设 x2110 中保存 -1。步骤 5，将 MDR 中的 -1 装入 R3，并设置 NZP 条件码，完成指令周期。

5.3.3 Base+offset 寻址

LDR（操作码 0110）和 STR（操作码 0111）指定 Base+offset 寻址，也就是**基址加偏移量寻址**。之所以这样命名，是因为操作数地址由基址寄存器的内容加上符号扩展后的六位偏移量得到。六位偏移量来自指令的位 [5:0]，基址寄存器由位 [8:6] 指定。

如果 R2 保存 16 位值 x2345，下面的指令会把 x2362 的内容装入 R1：

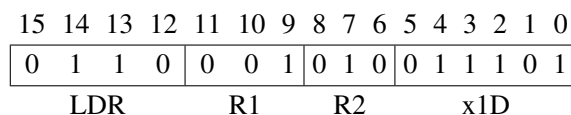
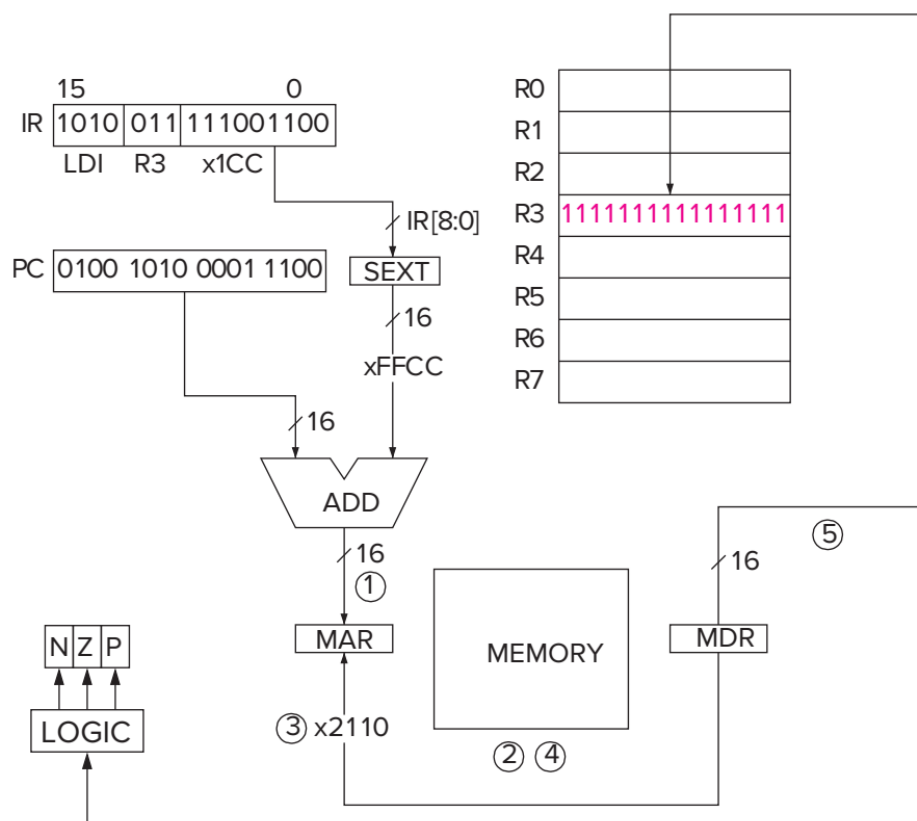
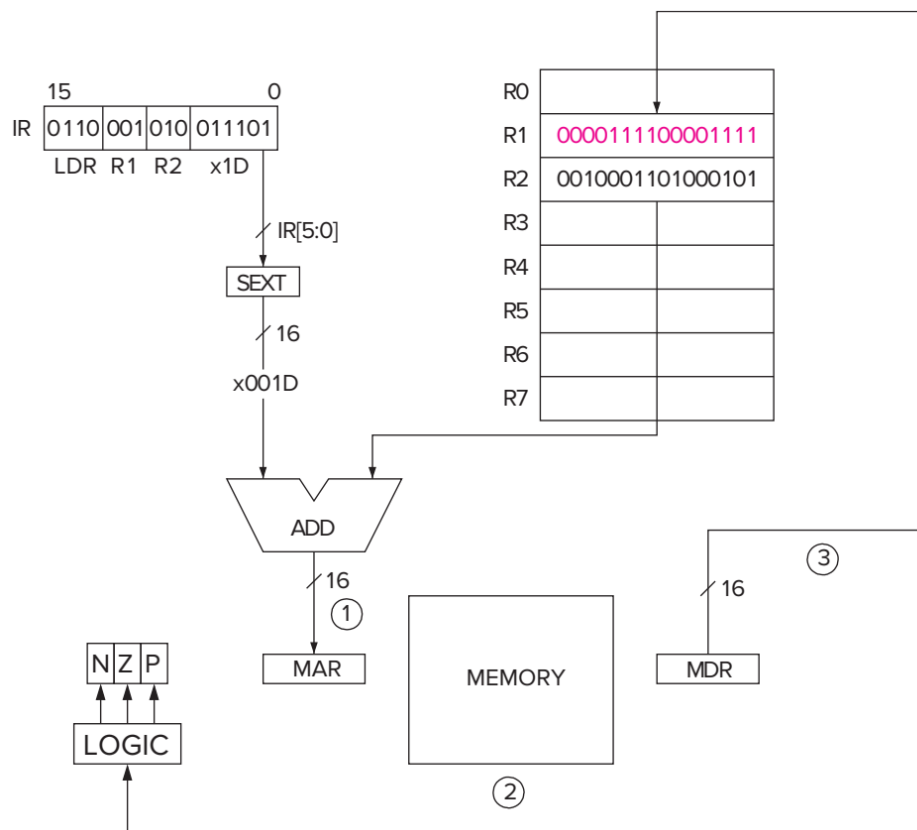


图 5.9 展示相应数据通路。首先, 把 R2 的内容 x2345 与 IR[5:0] 符号扩展后的值 x001D 相加, 将结果 x2362 装入 MAR。然后读存储器, 将 x2362 的内容装入 MDR。假设该位置保存 x0F0F。最后, 将 MDR 的内容 x0F0F 装入 R1, 并设置 NZP 条件码, 完成 LDR 的执行。

注意，Base+offset 寻址也允许操作数地址位于计算机存储器的任何位置。

图 5.8 执行 `LDI R3, x1CC` 所涉及的数据通路图 5.9 执行 `LDR R1, R2, x1D` 所涉及的数据通路

5.3.4 一个例子

用一个综合例子结束对寻址方式的讨论。假设 x30F6 至 x30FC 的内容如图 5.10 所示，PC 为 x30F6。我们将考察从 x30F6 开始的七条指令执行后产生的影响。⁴

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
x30F6	1	1	1	0	0	0	1	1	1	1	1	1	1	1	0	1
x30F7	0	0	0	1	0	1	0	0	0	1	1	0	1	1	1	0
x30F8	0	0	1	1	0	1	0	1	1	1	1	1	1	0	1	1
x30F9	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0
x30FA	0	0	0	1	0	1	0	0	1	0	1	0	0	1	0	1
x30FB	0	1	1	1	0	1	0	0	0	1	0	0	1	1	1	0
x30FC	1	0	1	0	0	1	1	1	1	1	1	1	0	1	1	1

图 5.10 说明三种寻址方式的代码片段

PC 最初指向 x30F6，因此首先执行该位置的指令。操作码 1110 是 LEA，即装入有效地址。将指令的位 [8:0] 符号扩展，得到 xFFFD，再与加一后的 PC (x30F7) 相加，结果装入位 [11:9] 指定的寄存器。所以执行结束时，R1 为 x30F4，PC 为 x30F7。

接下来执行 x30F7 中的指令。操作码 0001 是 ADD，位 [5] 为 1，因此把位 [4:0] 中的立即数符号扩展后，与位 [8:6] 指定的寄存器内容相加，再写入位 [11:9] 指定的寄存器。上一条指令已将 x30F4 写入 R1，而符号扩展后的立即数为 x000E，所以和为 x3102。执行后，R2 为 x3102，PC 为 x30F8，R1 仍为 x30F4。

然后是 x30F8。操作码 0011 指定 ST，将位 [11:9] 指定的 R2 的内容，存入 PC 相对寻址计算出的地址。加一后的 PC 为 x30F9，位 [8:0] 符号扩展后为 xFFFB，两者相加为 x30F4。因此，ST 执行结束时，M[x30F4] 保存 R2 中的值 x3102，PC 为 x30F9。

x30F9 中是带立即数 x0000 的 AND。执行结束时 R2 为 0，PC 为 x30FA。

x30FA 中的操作码 0001 指定 ADD。执行后 R2 为 5，PC 为 x30FB。

x30FB 中的操作码 0111 指定 STR。STR 与 LDR 一样，采用 Base+offset 寻址。位 [8:6] 指定基址寄存器 R1，其内容为 x30F4；位 [5:0] 符号扩展后的偏移量为 x000E，两者相加得到 x3102。STR 把位 [11:9] 指定寄存器 R2 的内容写入 x3102。由于 R2 为 5，执行结束时 M[x3102] 为 5，PC 为 x30FC。

最后是 x30FC。操作码 1010 指定 LDI，与 STI 一样采用间接寻址。先按 PC 相对寻址形成地址：位 [8:0] 符号扩展后为 xFFF7，加上已自增的 PC (x30FD)，得到 x30F4。这是保存操作数地址的位置。由于 M[x30F4] 为 x3102，操作数地址就是 x3102。LDI 读取该地址中的值 5，装入位 [11:9] 指定的 R3。执行结束时，R3 为 5，PC 为 x30FD。

⁴译注：原书此处写成“从 x30FC 开始”，与给定 PC、图 5.10 和后续逐条讲解不符；据上下文修正为 x30F6。

5.4 控制指令

控制指令改变待执行的指令序列。如果没有它们，当前指令完成后，接下来取得的总是相邻下一个存储位置中的指令，因为每个指令周期的取指令阶段都会将 PC 加一。第 4.4 节的程序已经说明，打破这种顺序往往很有用。

LC-3 有五种能打破顺序执行的操作码：条件分支、无条件跳转、子程序调用（子程序有时也称为函数）、TRAP，以及 RTI（从陷阱或中断返回）。本节主要讨论最常见的条件分支，也会介绍无条件跳转和 TRAP。TRAP 常称为服务调用，让程序员能请操作系统帮助完成自己未必完全了解的工作，例如从输入设备取得信息、向输出设备显示信息，以及停止计算机。TRAP 打断用户程序的顺序执行，转而执行操作系统中的指令序列。它如何实现这一点、有关 TRAP 的大部分细节，以及子程序调用和中断返回，留到第 8、9 章介绍。

5.4.1 条件分支

在这五种改变执行流程的指令中，只有一种会在每次执行时作出选择：继续执行顺序上的下一条，还是执行位于别处的一条。这就是条件分支 BR，操作码为 0000。

与 LC-3 的所有指令一样，BR 在取指令阶段使 PC 加一。根据程序此前指令的执行结果，BR 的执行阶段要么不做任何事，要么把希望接下来执行的指令地址装入 PC。如果什么也不做，加一后的 PC 保持不变，下一条仍是顺序上的下一条指令。

是否改变 PC，取决于先前的计算结果。这些结果反映在第 5.1.7 节介绍的条件码中。下面具体说明。条件分支的格式如下：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0000				n	z	p	PCoffset9								

位 [11]、[10]、[9] 分别对应 N、Z、P。LC-3 的三条运算指令 ADD、AND、NOT，以及三条装入指令 LD、LDI、LDR，会向 GPR 写值，并根据写入值是负、零还是正设置条件码。条件分支利用这些信息，决定是否离开 PC 在取指令阶段自增所产生的通常顺序。

第 4.4 节曾说，如果 BR 的位 [11:9] 为 101，只要上述六条指令最近一次写入寄存器的值不为 0，就打破顺序执行。现在可以解释原因了。

在 BR 的执行阶段，处理器只检查位 [11:9] 中为 1 的位所对应的条件码。注意格式中的小写 n、z、p：位 [11] 为 1，检查 N；位 [10] 为 1，检查 Z；位 [9] 为 1，检查 P。某位为 0，就不检查对应条件码。只要任一被检查的条件码为 1，就将计算地址阶段得到的地址装入 PC；如果被检查的条件码都不是 1，PC 保持不变，下个指令周期仍取得顺序上的下一条指令。计算地址阶段使用 PC 相对寻址生成目标地址。

第 4.4 节的例子中，x3004 的 ADD 将 R2 减一，结果写回 R2，同时设置条件码。x3005 的 BR 的位 [11:9] 为 101。位 [11] 为 1，因此检查表示负结果的 N；位 [9] 为 1，因此检查表示正结果的 P；位 [10] 为 0，所以不检查 Z。只要先前结果为正或负，也就是非零，就把分支计算地址阶段得到的 x3003 装入 PC。

图 4.7 的程序用 R2 跟踪把 5 加到 R3 上的次数。加法尚未全部完成时，R2 减一后的结果非零；全部完成时，结果为 0，Z 置 1，N、P 清零。此时 BR 检查到 N、P 都为 0，不改变已自增的 PC，接下来执行 x3006 中的 TRAP，请操作系统停机。

再看一个例子。假设以下指令位于 x4027，最近一次装入 GPR 的值为 0：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	0	1	0	0	1	1	0	1	1	0	0	1
BR				n	z	p	x0D9								

图 5.11 展示执行所需的数据通路。注意判断是否打破顺序执行的逻辑：三个 AND 门分别对应三个条件码。只有相应条件码为 1，且指令中的对应位要求检查它时，AND 门才输出 1。任一 AND 门输出 1，OR 门就输出 1，表示应改变顺序，将计算地址阶段求出的地址装入 PC。

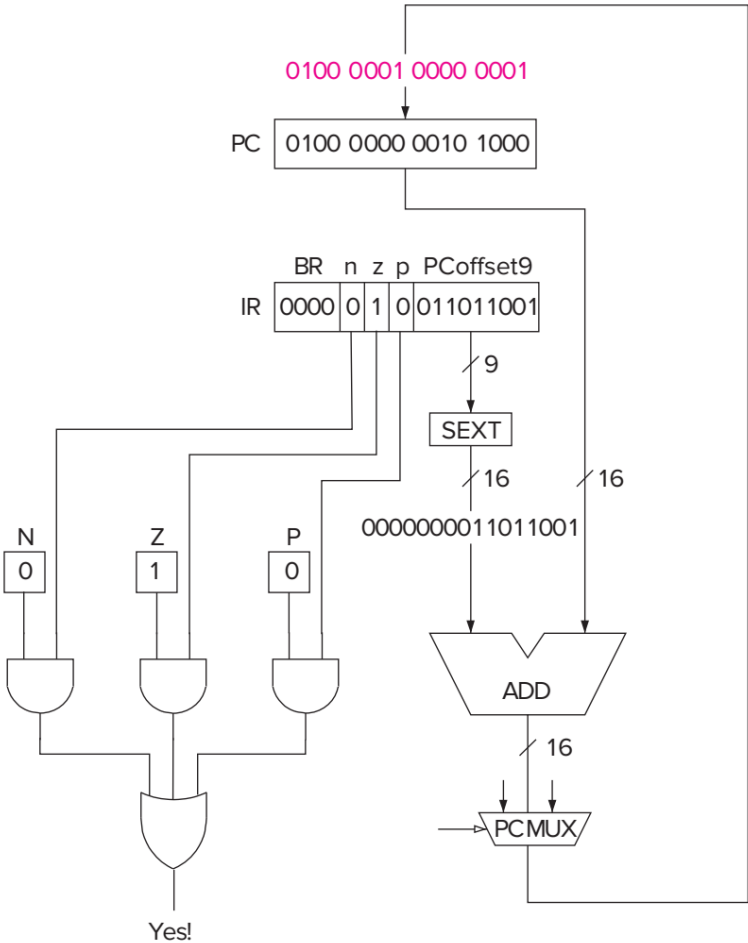


图 5.11 执行 BRz x0D9 所涉及的数据通路

对于 x4027 中的这条分支，条件成立，PC 装入 x4101，替换取指令阶段装入的 x4028。

另一个例子。如果位 [11:9] 全为 1，就检查全部三个条件码。最近一次写入寄存器的结果必定是负、零、正中的一种，没有其他选择，因此其中一个条件码必为 1。既

然全部检查，PC 就一定装入计算地址阶段得到的地址。这称为**无条件分支**，因为执行流程的改变与数据无关。

例如，位于 x507B 的下面这条指令执行后，PC 将为 x5001：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	0	0	0	1	1	1	1	1	0	0	0	0	1	0	1
BR				n z p			x185								

思考：如果 BR 的位 [11:9] 全为 0，会怎样？

5.4.2 两种循环控制方法

第 4.4 节的乘法程序反复执行一段指令，直到一个寄存器的值为 0。这段指令称为**循环体**，每执行一次称为一次**迭代**。序列末尾的 BR 控制循环体执行多少次。控制迭代次数有两种常见方法。

用计数器控制循环 假设已知 x3100 至 x310B 这 12 个位置保存整数，希望求出它们的和。图 5.12 是解决问题的算法流程图。

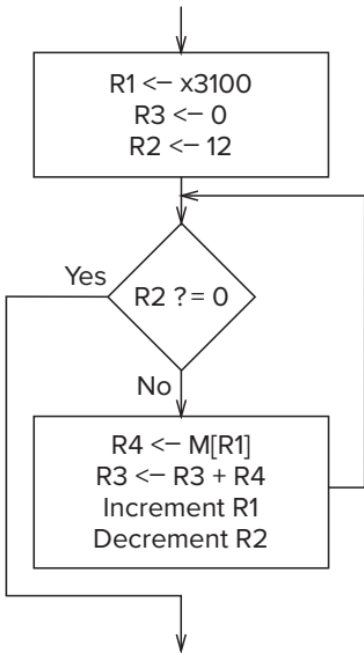


图 5.12 用计数器控制循环的整数求和算法

与所有算法一样，首先必须初始化变量，也就是设好程序执行所需变量的初值。这里有三个变量：下一个待加整数的地址，分配给 R1；当前累积的和，分配给 R3；剩余待加整数的个数，分配给 R2。把第一个整数的地址放入 R1，将 R3 初始化为 0，将 R2 初始化为 12，然后开始求和。

程序反复把这 12 个整数之一装入 R4，再加到 R3 上。每次执行 ADD 后，将 R1 加一，使它指向下一个待加数，也就是保存其地址；同时将 R2 减一，以知道还剩多

少个数。R2 变为 0 时，Z 条件码置位，就能判断工作已经完成。图 5.13 的十条指令完成这一任务。

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	1	1	1	0	0	0	1	0	1	1	1	1	1	1	1	1	R1<- 3100
x3001	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3002	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 <- 0
x3003	0	0	0	1	0	1	0	0	1	0	1	0	1	1	0	0	R2 <- 12
x3004	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	1	BRz x300A
x3005	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	R4 <- M[R1]
x3006	0	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0	R3 <- R3+R4
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 <- R1+1
x3008	0	0	0	1	0	1	0	0	1	0	1	1	1	1	1	1	R2 <- R2-1
x3009	0	0	0	0	1	1	1	1	1	1	1	1	1	0	1	0	BRnzp x3004

图 5.13 实现图 5.12 算法的程序

执行细节如下。程序从 PC=x3000 开始。x3000 的第一条指令用 x3100 初始化 R1：加一后的 PC 为 x3001，符号扩展后的 PCoffset 为 x00FF。

x3001 的指令清零 R3。R3 跟踪累积的和，因此必须从 0 开始，这称为把 SUM 初始化为零。

x3002、x3003 将 R2 初始化为 12，即待加整数的个数。每次加法后，x3008 中的指令将 R2 减一，以跟踪还有多少个数未加。

x3004 是条件分支。位 [10] 为 1，表示检查 Z。如果 Z 置位，说明 R2 刚刚减到 0，没有数需要再加，工作已经完成；如果 Z 清零，说明还有工作，继续下一次迭代。

x3005 把下一个整数装入 R4，x3006 将它加到 R3 上。

x3007、x3008 完成必要的记录更新。x3007 使 R1 加一，指向下一个整数；x3008 使 R2 减一，记录剩余整数个数，并设置条件码。

x3009 是无条件分支，位 [11:9] 全为 1，将 x3004 装入 PC。它不影响条件码，因此下一条执行的 x3004 条件分支，依据的仍是 x3008 的结果。

这值得再说一次：x3004 条件分支前面执行的是不影响条件码的 x3009，而 x3009 前面执行的是 x3008。因此，x3004 的判断依据是 x3008 将 R2 减一后设置的条件码。只要还有整数未加，x3008 的结果就大于 0，Z 清零。x3004 检查 Z，只要 Z 为 0，就不改变 PC，下个指令周期从 x3005 取指，开始下一次迭代。

所以执行序列是 x3000、x3001、x3002、x3003、x3004、x3005、x3006、x3007、x3008、x3009、x3004、x3005、x3006、x3007、x3008、x3009、x3004、x3005，等等。循环体由 x3005 至 x3009 的指令组成。R2 变为 0 时，PC 装入 x300A，程序从那里继续下一项工作。

你也许注意到，可以删去 x3004 的分支，把 x3009 的无条件分支改成测试非零的条件分支，即位 [11:9]=101，并转到目前位于 x3005 的指令。这样少执行一条指令，看起来很诱人。不过，我们建议不要这么做！图中的程序遵循第 6 章将讨论的结构化程序设计规则。这个简单例子中，捷径确实奏效，却破坏了结构化程序设计的方法。不

要养成随手走这种捷径的习惯，因为对于更大的程序，这很容易招致灾难。第 6 章会进一步讨论。

最后，还可以完全不用控制指令来计算这 12 个整数的和。仍需 x3000 的 LEA 初始化 R1，但无需 x3001 初始化累积和，也无需 x3002、x3003 初始化剩余个数。直接把 x3100 的内容装入 R3，然后反复将 R1 加一，把后续整数装入 R4，再将 R4 加到 R3 上，重复 11 次！加完第 12 个数，直接进入下一项工作，就像图 5.13 通过 x3004 的分支进入下一项工作一样。

可惜，这将得到一个 35 条指令的程序，而不是十条。若要加 100 个整数，无控制指令的版本将有 299 条指令，而使用循环仍只需十条。图 5.13 的控制指令通过打破顺序执行，使循环体这段代码得以重复使用。

用哨兵控制循环 上例使用计数器控制循环体的执行次数。预先知道要执行 12 次，就把计数器设为 12；每次执行后减一，检查是否为零。若不为零，将 PC 设回循环起点，继续下一次迭代。

第二种方法使用**哨兵** (sentinel)。如果事先不知道要迭代多少次，这种方法尤其有效。每次迭代通常处理一个值，我们就在待处理值序列后面附加一个预先知道绝不可能属于有效数据的值，作为哨兵。例如，若在对一串数求和，哨兵可以是字母 A 或 *，也就是某种不是数的东西。循环测试只需判断是否遇到了哨兵；遇到就说明完成了。

假设 x3100 至 x310B 中的值都为正，就可以用任意负数作哨兵。设 x310C 中的哨兵为 -1。相应流程图见图 5.14，程序见图 5.15。

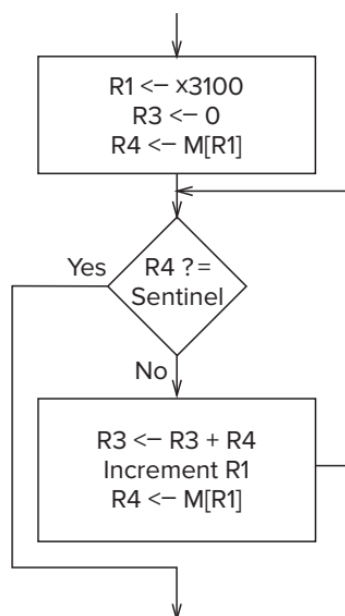


图 5.14 用哨兵控制循环的整数求和算法

与前面一样，x3000 将第一个待加值的地址装入 R1，x3001 将保存累积和的 R3 初始化为 0。

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	1	1	1	0	0	0	1	0	1	1	1	1	1	1	1	1	R1<- x3100
x3001	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3002	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	R4 <- M[R1]
x3003	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	BRn x3008
x3004	0	0	0	1	0	1	1	0	1	1	0	0	0	1	0	0	R3 <- R3+R4
x3005	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 <- R1+1
x3006	0	1	1	0	1	0	0	0	0	1	0	0	0	0	0	0	R4 <- M[R1]
x3007	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1		BRnzp x3003

图 5.15 实现图 5.14 算法的程序

x3002 将下一个存储位置的内容装入 R4。如果装入的是哨兵，N 条件码就会置位。

x3003 的条件分支检查 N。若 N=1，PC 装入 x3008，进入下一项工作。若 N=0，R4 一定包含有效的待加数。于是，x3004 将它加到 R3 上，x3005 将 R1 加一指向下一个位置，x3006 将该位置的内容装入 R4，x3007 将 x3003 装入 PC，开始下一次迭代。

5.4.3 JMP 指令

条件分支指令虽然功能强大，却有一个不便的限制：接下来执行的指令必须位于这样一个地址范围内，即把加一后的 PC 与指令位 [8:0] 给出的偏移量进行符号扩展后相加所能得到的地址范围。由于位 [8:0] 指定的是一个补码整数，条件分支之后执行的指令相对于分支指令本身，最远只能相距 +256 或 -255 个存储位置。

如果我们希望接下来执行一条距离当前指令 2000 个存储位置的指令呢？九位字段无法容纳数值 2000，因此条件分支指令无法完成这项任务。

LC-3 ISA 确实提供了一条能够完成这项任务的指令 JMP（操作码 = 1100）。

JMP 指令把指令位 [8:6] 指定的寄存器的内容装入 PC。如果下面这条 JMP 指令位于地址 x4000，

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	0	0	0	0	0	0	0	1	0	0	0	0	0	0
JMP								BaseR							

R2 中保存的值为 x6600，PC 中保存的是 x4000，那么将先执行 x4000 处的指令（即这条 JMP 指令），随后执行 x6600 处的指令。由于寄存器包含 16 位（可覆盖存储器的整个地址空间），JMP 指令对接下来执行的指令所在的位置没有限制。

5.4.4 TRAP 指令

TRAP 的工作细节留到第 9 章。不过，在那之前就需要输入输出数据，因此先在这里介绍。TRAP（操作码 1111）把 PC 改成操作系统中的一个地址，让操作系统替当前程序完成某项任务。用操作系统术语来说，TRAP 调用了一项操作系统服务。位

[7:0] 构成 **trapvector**，即八位陷阱向量，标识程序请求的服务。表 A.2 列出了本书使用的全部 LC-3 服务调用的陷阱向量。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1111				0000				trapvector8							

操作系统完成服务后，将 PC 设为 TRAP 后面那条指令的地址，程序继续执行。这样，程序可以在执行过程中请求服务，并在每次服务完成后继续处理。现在需要的服务有：

- 从键盘输入一个字符，**trapvector**=x23。
- 向显示器输出一个字符，**trapvector**=x21。
- 停止程序，**trapvector**=x25。

5.5 另一个例子：统计字符出现次数

用另一个示例程序结束对 LC-3 ISA 的介绍。假设希望从键盘输入一个字符，统计它在文件中出现的次数，最后在显示器上输出计数。为简化问题，假定任何感兴趣字符的出现次数都足够小，可以用一个十进制数字表示，也就是最多九次。这样就无需考虑二进制计数与显示器上的 ASCII 表示之间复杂的转换例程；第 10 章会讨论这个问题，今天先不管它。

图 5.16 是解决问题的算法流程图，每一步都同时用英文和括号中的 LC-3 实现方式表示。

第一步照例是初始化全部变量，也就是为程序所用的四个寄存器 R0、R1、R2、R3 提供初值。R2 跟踪出现次数，在图中称为 **Count**，初始化为 0。R3 指向文件中下一个待检查的字符，因此称为**指针**：它保存下一个字符所在位置的地址。将它初始化为文件第一个字符的地址。R0 保存要统计的字符，从键盘输入后放入 R0。R1 则依次保存从文件取出的每个字符。

文件并不要求靠近程序，也不要求远离程序。例如，程序从 x3000 开始、文件从 x9000 开始，完全合理；此时初始化过程将 R3 设为 x9000。

下一步是统计输入字符的出现次数。依次处理文件中的每个字符，直到文件结束。处理一个字符需要循环的一次迭代。回想第 5.4.2 节，跟踪循环迭代有两种常用方法。这里使用哨兵法，以 EOT (End of Transmission, 传输结束) 的 ASCII 码 00000100 为哨兵。附录 E 给出了 ASCII 码表。

每次迭代首先将 R1 的内容与 EOT 的 ASCII 码比较。若相等，退出循环，进入最后一步，在屏幕上显示计数。若不相等，说明还有工作要做：将 R1 中当前检查的字符与 R0 中键盘输入的字符比较。若相同，R2 加一。无论是否相同，随后都取得下一个字符：R3 加一，将下一个字符装入 R1，再回到检查文件结束哨兵的测试。

到达文件末尾时，全部字符都已检查，计数以二进制形式保存在 R2。要显示计数，先转成 ASCII 码。由于假设计数小于 10，只需在其四位二进制表示前加上 0011。

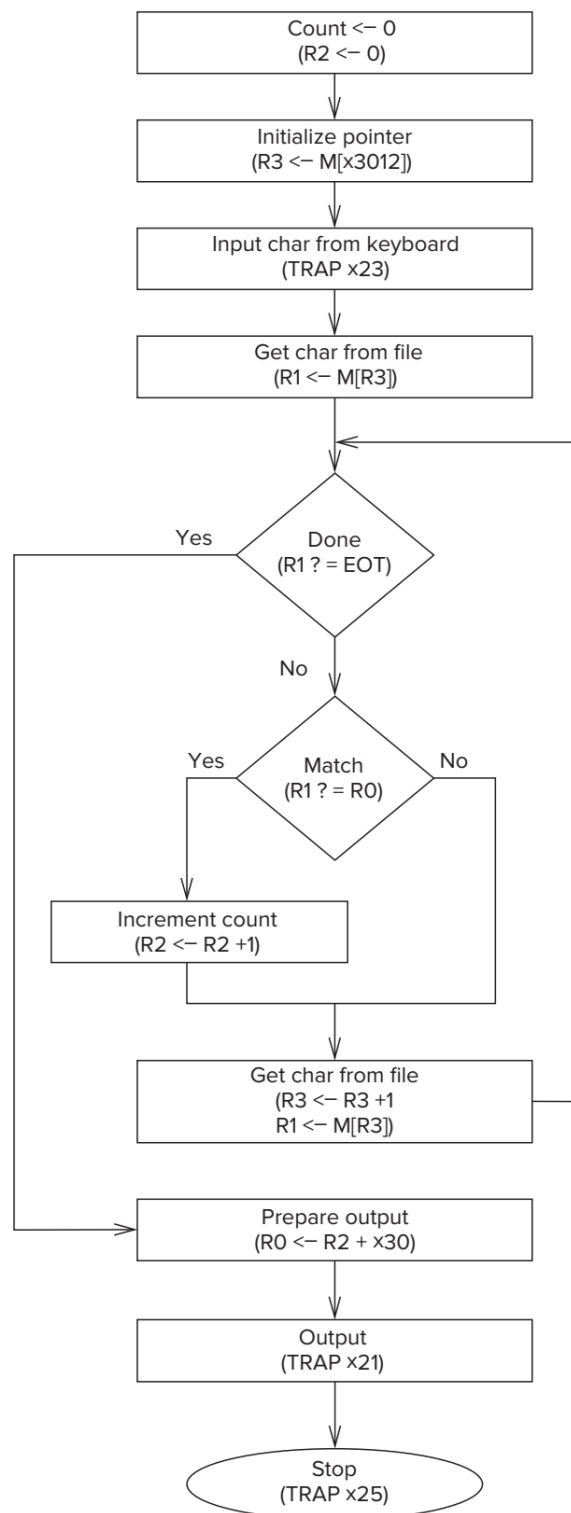


图 5.16 统计字符出现次数的算法

请查看图 E.2 中 0 至 9 各十进制数字的二进制值与 ASCII 码之间的关系。最后向显示器输出计数，程序结束。

图 5.17 是实现图 5.16 流程图的机器语言程序。

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 <- 0
x3001	0	0	1	0	0	1	1	0	0	0	0	1	0	0	0	0	R3 <- M[x3012]
x3002	1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1	TRAP x23
x3003	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 <- M[R3]
x3004	0	0	0	1	1	0	0	0	0	1	1	1	1	0	0	0	R4 <- R1-4
x3005	0	0	0	0	0	1	0	0	0	0	0	0	1	0	0	0	BRz x300E
x3006	1	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	R1 <- NOT R1
x3007	0	0	0	1	0	0	1	0	0	1	1	0	0	0	0	1	R1 <- R1 + 1
x3008	0	0	0	1	0	0	1	0	0	1	0	0	0	0	0	0	R1 <- R1 + R0
x3009	0	0	0	0	1	0	1	0	0	0	0	0	0	0	0	1	BRnp x300B
x300A	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	R2 <- R2 + 1
x300B	0	0	0	1	0	1	1	0	1	1	1	0	0	0	0	1	R3 <- R3 + 1
x300C	0	1	1	0	0	0	1	0	1	1	0	0	0	0	0	0	R1 <- M[R3]
x300D	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	0	BRnzp x3004
x300E	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	0	R0 <- M[x3013]
x300F	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	0	R0 <- R0 + R2
x3010	1	1	1	1	0	0	0	0	0	0	1	0	0	0	0	1	TRAP x21
x3011	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	TRAP x25
x3012	Starting address of file																
x3013	0	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0	ASCII TEMPLATE

图 5.17 实现图 5.16 算法的机器语言程序

先看初始化。x3000 把 R2 与 x0000 做 AND，将其清零。x3001 把文件起始地址装入 R3。再强调一次，文件可以在存储器的任何地方。开始从 x3000 执行前，必须已有某段指令把文件的第一个地址存入 x3012。x3002 的 TRAP 请求操作系统代为执行服务。八位陷阱向量 00100011，即 x23，指定把接下来键入字符的 ASCII 码装入 R0。表 A.2 列出可替用户程序执行的所有服务调用的陷阱向量。x3003 把 R3 指向的字符装入 R1。

接着开始检查字符。x3004 从 R1 减去 4，也就是 EOT 的 ASCII 码，将结果存入 R4。如果为零，说明已到文件末尾，应当输出计数。x3005 的条件分支转到 x300E，从那里开始输出。

如果 R4 非零，R1 就是需要检查的有效字符。x3006、x3007、x3008 三条指令共同计算 R0 - R1，判断两者内容是否相同。只有 R1、R0 的位模式相同，结果才全为零。若不同，x3009 的条件分支转到 x300B，跳过 x300A 中将计数器 R2 加一的指令。

x300B 将 R3 加一，使其指向文件下一个字符；x300C 把字符装入 R1；x300D 无条件返回 x3004，开始处理该字符。

最终检测到 EOT 后，从 x300E 开始输出计数。x300E 把 00110000 装入 R0，x300F 把计数加到 R0 上，将 R2 中的二进制计数转换为 R0 中的 ASCII 计数。x3010 通过

TRAP 请求操作系统向显示器输出 R0 的内容。服务结束、程序恢复执行后, x3011 再通过 TRAP 终止程序。

思考: 能否改进这个程序的执行效率? 提示: x3006、x3007 执行了多少次? 做什么小改动, 可以减少总共需要执行的指令数?

5.6 重访数据通路

结束第 5 章之前, 再看一次第 3 章首次遇到的数据通路图 (图 3.35)。其中许多结构已在本章图 5.4、5.5、5.6、5.7、5.8、5.9、5.11 中见过。图 5.18 再次给出完整数据通路。先注意两种箭头: 实心箭头表示被处理的信息, 空心箭头表示控制信号。控制信号来自标为 “Finite State Machine” 的方框。为减少图中不必要的杂乱, 图 5.18 省略了有限状态机到大多数控制信号的连线。

5.6.1 数据通路的基本部件

5.6.1.1 全局总线

图中最显眼的是两端带箭头的粗黑线, 表示数据通路的全局**总线**。LC-3 全局总线由 16 根导线及相关电子电路组成。通过建立必要的连接, 一个结构可以向另一个结构传送最多 16 位信息。总线上同一时刻只能传送一个值。每个向总线提供值的结构, 在进入总线的箭头后面都有一个三角形, 称为**三态器件**。它使控制逻辑能保证任一时刻恰好只有一个来源向总线提供信息。希望取得总线值的结构, 只需使自己的 LD.x (装入使能) 信号有效; 回想第 3.4.2 节对门控锁存器的讨论。并非所有计算机都只有一条全局总线, 单总线的利弊也留待今后学习。

5.6.1.2 存储器

保存指令和数据的存储器是计算机最重要的部分之一。访问时, 把目标位置的地址装入 MAR。执行装入操作时, 控制信号读取该位置的内容, 由存储器把结果送入 MDR。执行存储操作时, 先把待存储值装入 MDR, 再由控制信号使写使能 WE 有效, 将 MDR 的值存入 MAR 指定的位置。

5.6.1.3 ALU 和寄存器组

ALU 是处理部件, 有两个输入: 源 1 来自寄存器, 源 2 来自寄存器或指令提供的符号扩展立即数。R0 至 R7 可以提供两个值, 分别由三位寄存器编号 SR1 和 SR2 控制。SR1、SR2 是 LC-3 运算指令中的字段。第二个操作数使用寄存器还是符号扩展立即数, 由位 [5] 决定。注意向 ALU 提供源 2 的多路选择器, 它的选择线就是运算指令的位 [5]。

ALU 运算产生两类结果: (a) 存入一个寄存器的运算结果; (b) 三个一位条件码。ALU 可向总线提供 16 位, 再将它们写入三位寄存器编号 DR 指定的寄存器。总线上

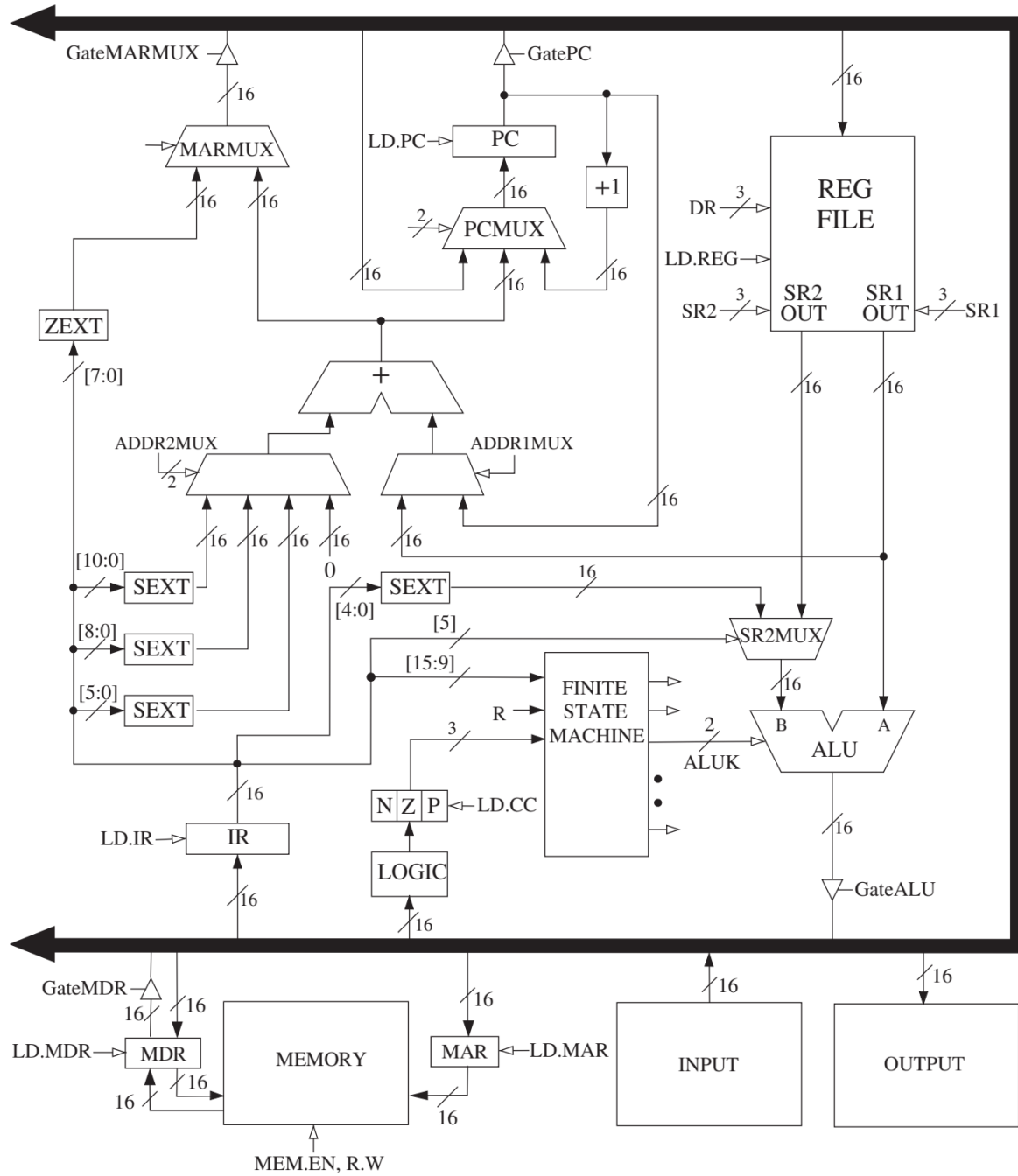


图 5.18 LC-3 的数据通路

的 16 位也送入另一段逻辑，判断该值是负、零还是正，并相应设置 N、Z、P。

5.6.1.4 PC 和 PCMUX

每个指令周期开始时，PC 通过全局总线向 MAR 提供待取指令的地址。PC 自身则由三选一的 PCMUX 提供输入。在取指令阶段，把 PC 加一后写回 PC，这对应 PCMUX 最右边的输入。

如果当前是控制指令，PCMUX 应使用哪个来源，取决于具体指令。若是条件分支且分支成立，就把加一后的 PC 与 PCOffset 相加的结果装入 PC，其中 PCOffset 是 IR[8:0] 符号扩展后的 16 位值。这次加法发生在专用加法器中，不是在 ALU 中；加法器输出是 PCMUX 的中间输入。第三个输入来自全局总线，第 9 章介绍其他控制指令后，它的用途就会清楚。

5.6.1.5 MARMUX

访问存储器需要向 MAR 提供地址。执行装入、存储或 TRAP 时，MARMUX 从两个来源中选择适当的地址。右输入来自一个加法结果：把自增后的 PC 或基址寄存器，与零或 IR 提供的字面量相加。究竟选择 PC 还是基址寄存器，以及使用哪个字面量，取决于操作码。ADDR1MUX 控制选择 PC 还是基址寄存器；ADDR2MUX 控制选择四个被加值中的哪一个。MARMUX 左输入提供零扩展后的 trapvector，调用服务时需要它，第 9 章将详细讨论。

5.6.2 LC-3 的具体指令周期

最后跟随一个指令周期中的数据流，完成对 LC-3 数据通路的考察。假设 PC 为 x3456，位置 x3456 的内容如下：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	1	0	0	1	1	0	1	0	0	0	0	1	0	0
LDR				R3			R2			4					

并假设 LC-3 刚执行完 x3455 中的一条 ADD。

5.6.2.1 取指令 (FETCH)

指令周期从取指令开始，用 PC 中的地址访问存储器。第一个周期，PC 的内容经全局总线装入 MAR，同时 PC 加一后写回。周期结束时 PC 为 x3457。下一个周期，如果存储器一个周期就能提供数据，读取存储器，将指令 0110011010000100 装入 MDR。再下一个周期，将 MDR 的内容装入 IR，完成取指令阶段。

5.6.2.2 译码 (DECODE)

下个周期对 IR 译码，控制逻辑产生正确的控制信号，即空心箭头所示信号，以控制其余处理过程。操作码 0110 表示 LDR，需要用 Base+offset 寻址确定待装入 R3

的数据地址。

5.6.2.3 计算地址 (EVALUATE ADDRESS)

下个周期，将基址寄存器 R2 的内容与 IR[5:0] 符号扩展后的值相加，结果经 MARMUX 送入 MAR。SR1 字段为 010，指定读取 R2 取得基址。ADDR1MUX 选择 SR1OUT，ADDR2MUX 选择从右数第二个来源。

5.6.2.4 取操作数 (OPERAND FETCH)

下个周期，将该地址中的值装入 MDR；如果存储器访问需要多个周期，这一步就持续多个周期。

5.6.2.5 执行 (EXECUTE)

LDR 不需要执行阶段，因此这一阶段耗时零个周期。

5.6.2.6 存储结果 (STORE RESULT)

最后一个周期，把 MDR 的内容送上全局总线，从总线装入 R3，同时送入条件码逻辑，设置 NZP。

习题

- 5.1** 对于 ADD、JMP、LEA 和 NOT，判断它们属于运算指令、数据传送指令还是控制指令，并列出各条指令可以使用的寻址方式。
- 5.2** 某个存储器的寻址粒度为 64 位，由此能知道 MAR 和 MDR 的大小吗？
- 5.3** 结束循环有两种常见方法。一种用计数器记录迭代次数，另一种使用一种称为_____的元素。该元素有什么与众不同的特征？
- 5.4** 假设存储器有 256 个位置，每个位置包含 16 位。
- 地址需要多少位？
 - 若采用 PC 相对寻址，希望允许相距 20 个位置的指令之间转移控制，分支指令的 PC 相对偏移量需要多少位？
 - 如果控制指令位于位置 3，地址 10 的 PC 相对偏移量是多少？假设控制转移指令的工作方式与 LC-3 相同。
- 5.5**
- 什么是寻址方式？
 - 列出指令操作数可能所在的三个地方。
 - 列出 LC-3 的五种寻址方式，分别说明操作数位于 (b) 中的哪个地方。
 - 第 5.1.2 节的 ADD 使用哪种寻址方式？
- 5.6** 回想第 2.6.7 节的机器忙闲示例。假设 BUSYNESS 位向量在 R2 中，指令 0101 011 010 1 00001 (AND R3, R2, #1) 可判断机器 0 是否忙。若结果为 0，机器 0 就忙。
- 写一条 LC-3 指令判断机器 2 是否忙。
 - 写一条指令判断机器 2、3 是否都忙。
 - 写一条指令判断是否没有任何机器忙。
 - 能用一条指令判断机器 6 是否忙吗？这里有什么问题？
- 5.7** LC-3 ADD 中能够以字面量（立即数）表示的最大正数是多少？
- 5.8** 如果把 LC-3 ADD 能指定的寄存器数量增加到 32，会有什么问题？请解释。
- 5.9** 我们希望有一条什么也不做的指令。许多 ISA 确实有专门的操作码，通常称为 NOP，即 NO OPERATION。它照样取指、译码、执行，只是执行阶段什么也不做！下面三条指令中，哪些可以作为 NOP，又让程序继续正确工作？
- 0001 001 001 1 00000
 - 0000 111 000000001
 - 0000 000 000000000

ADD 做了其他两条指令没有做的什么事？

5.10 以下 LC-3 指令 A、B 有何相同之处和不同之处？

A: 0000111101010101

B: 0100111101010101

5.11 希望用一条 LC-3 指令，从寄存器 1 中减去十进制数 20，并把结果存入寄存器 2。能做到吗？若能，写出指令；若不能，解释原因。

5.12 执行 ADD R2, R0, R1 后，发现 R0[15] 与 R1[15] 相同，却与 R2[15] 不同。已知 R0、R1 保存无符号整数，即 0 至 65,535 的非负整数。在什么条件下，可以相信 R2 中的结果？

5.13 a. 怎样用一条指令把 R2 的值传送到 R3？

b. LC-3 没有减法指令，怎样只用三条指令实现 $R1 \leftarrow R2 - R3$ ？

c. 怎样只用一条指令，且不改变任何寄存器的内容，就根据 R1 的值设置条件码？

d. 是否存在一段指令序列，使执行结束时 N=1、Z=1、P=0？请解释。

e. 写一条指令清零 R2。

5.14 LC-3 没有实现逻辑 OR 的操作码，但可用指令序列实现。下面四条指令将 R1 与 R2 的内容逐位 OR，结果放入 R3。填入缺失的两条指令。

(1): 1001 100 001 111111

(2): _____

(3): 0101 110 100 000 101

(4): _____

5.15 给出从 x3100 开始的下列程序停机后，R1、R2、R3、R4 的内容。

Address	Data
0011 0001 0000 0000	1110 001 000100000
0011 0001 0000 0001	0010 010 000100000
0011 0001 0000 0010	1010 011 000100000
0011 0001 0000 0011	0110 100 010 000001
0011 0001 0000 0100	1111 0000 0010 0101
:	:
:	:
0011 0001 0010 0010	0100 0101 0110 0110
0011 0001 0010 0011	0100 0101 0110 0111
:	:
:	:
0100 0101 0110 0111	1010 1011 1100 1101
0100 0101 0110 1000	1111 1110 1101 0011

- 5.16** 以下各情况下，使用 LC-3 的哪种寻址方式最合适？每问可能不止一个正确答案，请解释理由。
- 希望从相距不到 $\pm 2^8$ 个位置的地址装入一个值。
 - 希望从相距超过 2^8 个位置的地址装入一个值。
 - 希望装入一个位于连续地址中的数组。
- 5.17** 处理一条 LD 时，LC-3 会向存储器发出多少次读写请求？LDI 呢？LEA 呢？这里的处理包括指令周期的全部阶段。
- 5.18** PC 保存一条 LDR 的地址，处理它需要多少次存储器访问？再分别回答 STI 和 TRAP 的情况。
- 5.19** LC-3 的 IR 有 16 位，LD 用低九位 [8:0] 表示 PC 相对偏移量。如果修改 ISA，改用位 [6:0] 表示偏移量，LD 可装入数据的地址范围将变成什么？
- 5.20** 如果修改 LC-3 ISA，使 LD 只能从相对自增后的 PC 距离为 ± 32 个位置的范围装入数据，PC 相对偏移量需要多少位？
- 5.21** LC-3 ISA 最多支持多少个 TRAP 服务例程？请解释。
- 5.22** PC 为 x3010，各存储位置的值如下：

x3050:	x70A4
x70A2:	x70A3
x70A3:	xFFFF
x70A4:	x123B

随后执行以下三条指令，将一个值装入 R6。这个值是什么？

x3010	1110 0110 0011 1111
x3011	0110 1000 1100 0000
x3012	0110 1101 0000 0000

可以用一条指令替代这三条，它是什么？

- 5.23** 假设下列 LC-3 程序从 x30FF 开始装入存储器。执行结束时 R2 的值是多少？

x30FF	1110 0010 0000 0001
x3100	0110 0100 0100 0010
x3101	1111 0000 0010 0101
x3102	0001 0100 0100 0001
x3103	0001 0100 1000 0010

- 5.24** 位于 x3200 的 LDR 使用 R4 作基址寄存器，R4 当前为 x4011。这条指令可以从哪个最大地址装入数据？若把 LDR 的偏移量改为零扩展，而不是符号扩展，最大地址又是多少？采用新定义时，最小地址是多少？
- 5.25** 写一个 LC-3 程序，比较 R2、R3 中的两个数，把较大者放入 R1；若相等，R1 设为 0。
- 5.26** 考虑 LC-3 的后继机：新增十种操作码，寄存器从八个扩展到 16 个；存储器改为按字节寻址，总地址空间为 64K 字节；指令仍为 16 位。新旧指令都沿用原先 15 条指令的字段，但某些字段的大小可能要调整。
- 完成上述后继机的详细规范有困难吗？请解释。
 - PC 需要多少位才能寻址全部存储器？
 - 希望 TRAP 能调用 128 个不同的操作系统例程，并通过将陷阱向量左移五位形成每个例程的地址。这些服务例程至少需要多少存储空间？
 - 若新版把寄存器从八个减少为四个，操作码保持 16 种，ADD 能表示的最大立即数是多少？
- 5.27** 第 5.3.4 节的七条指令执行前，R2 为 xAAAA。执行过程中，R2 一共出现多少个不同值？分别是什么？
- 5.28** 实际上，我们并不一定需要 LDI (1010) 和 STI (1011)，其他指令序列也能产生相同效果。请把下列代码中的 STI 替换成实现相同功能所需的指令。

```

x3000    0010 0000 0000 0010
x3001    1011 0000 0000 0010
x3002    1111 0000 0010 0101
x3003    0000 0000 0100 1000
x3004    1111 0011 1111 1111

```

- 5.29** LC-3 ISA 中的 LDR DR, BaseR, offset 在译码后，通过以下操作（称为微指令）完成：

```

MAR ← BaseR + SEXT(Offset6) ; set up the memory
address
MDR ← Memory[MAR] ; read mem at BaseR + offset
DR ← MDR ; load DR

```

假设 LC-3 的架构设计者希望增加 MOVE DR, SR，把 SR 给出的地址中的存储器内容，复制到 DR 给出的地址中。

- MOVE 并非必需，因为可以用现有指令序列实现。哪段序列能实现（也称为模拟）MOVE R0, R1？
- 如果 ISA 增加 MOVE，译码后需要怎样的微指令序列，才能实现 MOVE DR, SR？

5.30 下表给出 LC-3 的部分存储器。如果条件分支转移到 x3100，关于 R1、R0 能知道什么？

Address	Data
0011 0001 0000 0000	1001 001 001 111111
0011 0001 0000 0001	0001 010 000 000 001
0011 0001 0000 0010	1001 010 010 111111
0011 0001 0000 0011	0000 010 111111100

5.31 下图给出执行 x1000 中的指令前后，八个寄存器的快照。填入 x1000 中指令的各位。

BEFORE		AFTER	
R0	x0000	R0	x0000
R1	x1111	R1	x1111
R2	x2222	R2	x2222
R3	x3333	R3	x3333
R4	x4444	R4	x4444
R5	x5555	R5	xFFF8
R6	x6666	R6	x6666
R7	x7777	R7	x7777

0x1000 :

0	0	0	1												
---	---	---	---	--	--	--	--	--	--	--	--	--	--	--	--

5.32 下列指令序列开始执行时，N=0、Z=0、P=1。执行结束时，它们的值分别是什么？

x3050	0000 0010 0000 0010
x3051	0101 0000 0010 0000
x3052	0000 1110 0000 0010
x3053	0101 0000 0010 0000
x3054	0001 0000 0011 1111

5.33 执行以下指令后，如果 R0 中的值为 5，能推断 R5 有什么性质？

x2FFF	0101 0000 0010 0000
x3000	0101 1111 1110 0000
x3001	0001 1101 1110 0001
x3002	0101 1001 0100 0110
x3003	0000 0100 0000 0001
x3004	0001 0000 0010 0001
x3005	0001 1101 1000 0110
x3006	0001 1111 1110 0001
x3007	0001 0011 1111 1000
x3008	0000 1001 1111 1001
x3009	0101 1111 1110 0000

5.34 利用图 5.18 的完整数据通路，指出实现图 5.4 中 NOT 指令的元件。

5.35 利用图 5.18，指出实现图 5.5 中 ADD 指令的元件。

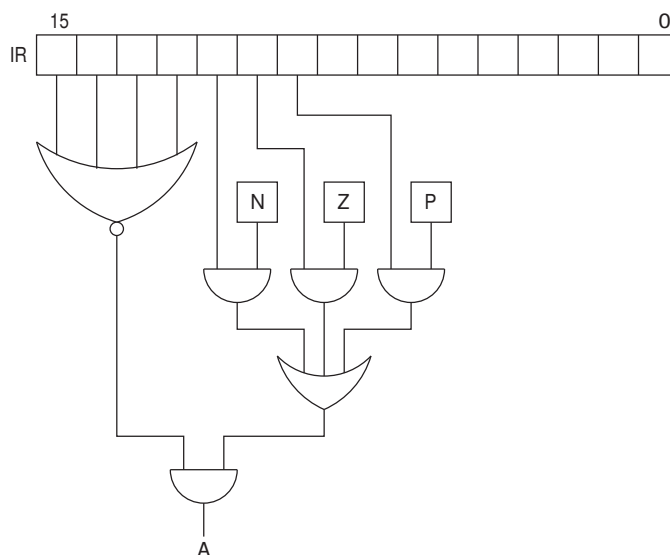
5.36 利用图 5.18，指出实现图 5.7 中 LD 指令的元件。

5.37 利用图 5.18，指出实现图 5.8 中 LDI 指令的元件。

5.38 利用图 5.18，指出实现图 5.9 中 LDR 指令的元件。

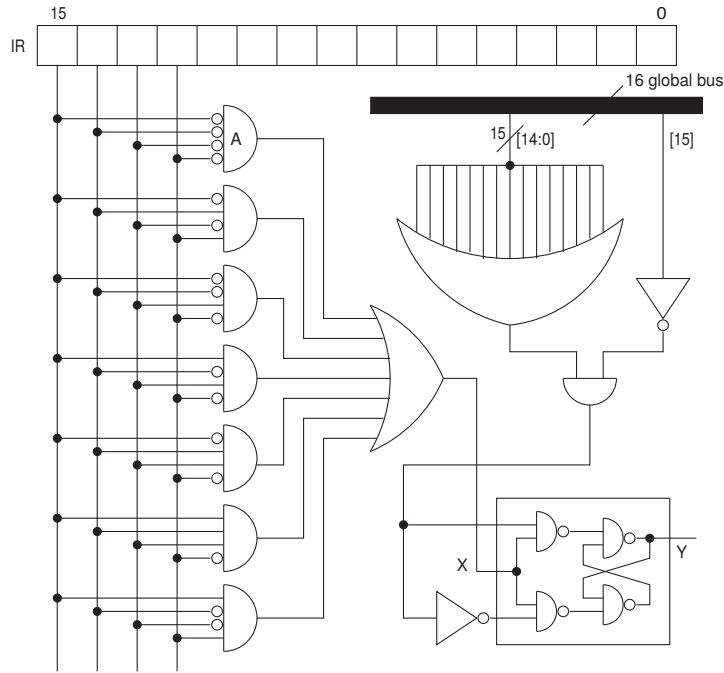
5.39 利用图 5.18，指出实现图 5.6 中 LEA 指令的元件。

5.40 下图是 LC-3 控制结构的一部分，标为 A 的信号有什么用途？



5.41 下图是 LC-3 体系结构的一部分实现。

- Y 提供什么信息？
- X 是控制门控 D 锁存器的信号。产生 X 的逻辑有错误吗？



5.42 LC-3 macho-company 决定用操作码 1101 实现一条新指令。请从以下候选中选出最有用的一条，并说明理由。

- MOVE R_i, R_j : 把 R_j 的内容复制到 R_i 。
- NAND R_i, R_j, R_k : R_i 为 R_j 、 R_k 逐位 NAND 的结果。
- SHFL $R_i, R_j, \#2$: 把 R_j 的内容左移两位，存入 R_i 。
- MUL R_i, R_j, R_k : R_i 为 R_j 、 R_k 中补码整数的乘积。

5.43 执行指令会改变计算机状态。假设执行前状态相同，分别执行下列指令 1、指令 2 后，LC-3 的状态有区别吗？请解释。

指令 1: 0001 000 000 1 00000, $R0 \leftarrow R0 + \#0$ 。

指令 2: 0000 111 000000000, 若 N、Z、P 任一个置位，转到 $PC' + \#0$ 。

5.44 程序希望从存储器装入一个值到 $R1$ ，并根据其值选择执行位置：正数从 x3040 开始，负数从 x3080 开始，零从 x3003 开始。第一条装入 $R1$ 的指令已在 x3000 给出。

- 写出 x3001、x3002 中的指令。

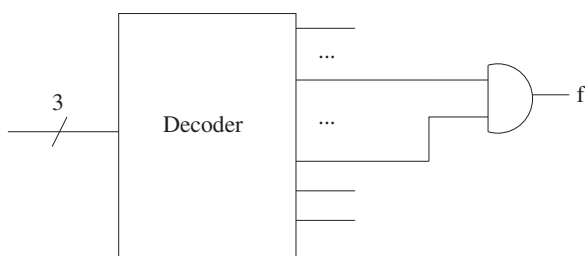
x3000	0010 001 011111111
x3001	
x3002	

- 下列程序段从 x3000 开始执行。停机后，寄存器 0 的内容是什么？

```

x3000: 0101 000 000 1 00000    ;register 0 <-- 0
x3001: 0001 000 000 1 00001    ;register 0 <-- register 0 + 1
x3002: 0000 001 111111110      ;branch p -2
x3003: 1111 0000 0010 0101     ;TRAP x25
  
```

c. 一个 3 线—8 线译码器的两个输出连接到 AND 门，如下图所示。



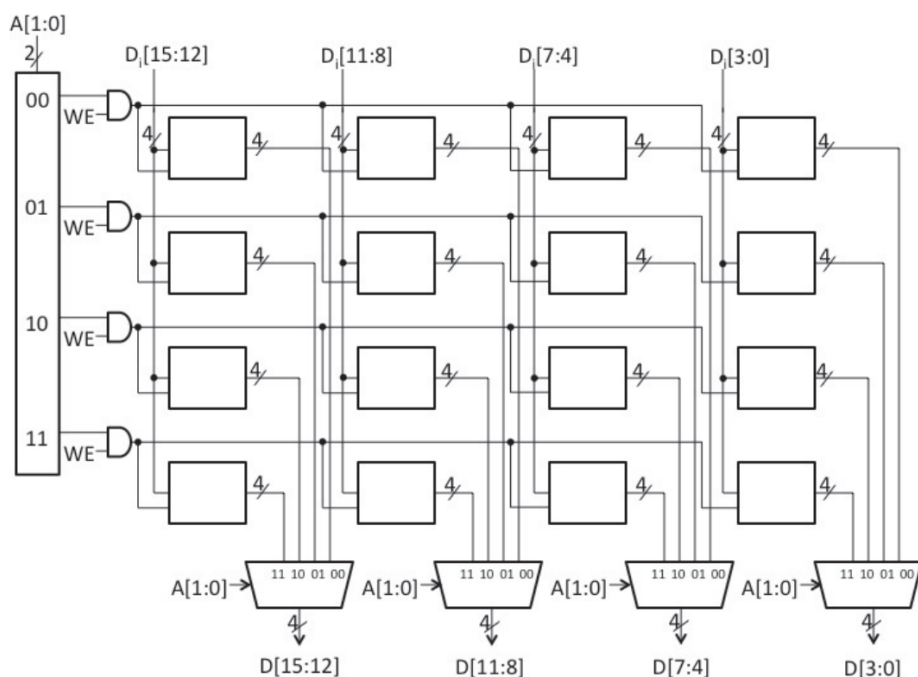
5.45 课堂上展示过处理 LC-3 程序指令所需的有限状态机的前几个状态。第一个状态完成两件事：

$$\text{MAR} \leftarrow \text{PC}, \quad \text{PC} \leftarrow \text{PC} + 1.$$

为什么微体系结构要把 PC 装入 MAR？为什么要使 PC 加一？

5.46 R0 保存一个大写英文字母的 ASCII 码。执行指令 0001000000000001 后，希望 R0 变成对应小写字母的 ASCII 码。执行前，其他寄存器中的值必须满足什么条件？

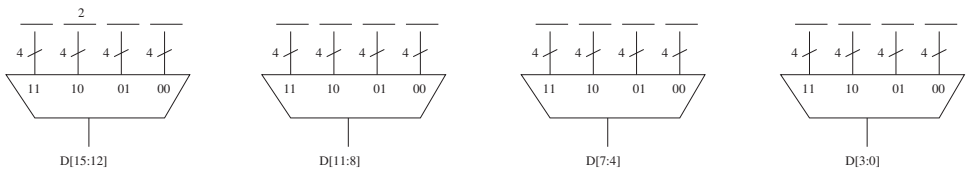
5.47 下图表示一个 $2^2 \times 16$ 位存储器。四个 mux 的输入源和输出均为四位，每个四位输入源都来自一个四位存储单元。



a. 不幸的是，一位学生接线时混淆了某些 mux 的输入。某个存储单元的四位本应接到正确的四位输入端，却全部接到了该 mux 的另一个四位输入端，结果一团糟。为了找出接线错误，进行了以下存储器访问：

Read/Write	MDR	MAR
Write	x134B	01
Write	xFCA2	10
Write	xBEEF	11
Write	x072A	00
Read	xF34F	10
Read	x1CAB	01
Read	x0E2A	00

注意：写操作之前装入 MDR，读操作则由访问结果装入 MDR。请找出接线错误，在下面的空白处填入相应地址，说明哪些存储单元接到了哪些 mux 输入。已经给出一个地址。



- b. 重新正确接好 mux，并把所有存储单元初始化为 xF，然后进行以下访问。部分信息未给出，请填空。

Read/Write	MDR	MAR
Write	x72 ____	0 __
Write	x8FAF	11
Read	x72A3	__0
Read	xFFFF	1__
Write	x732D	__1
Read	xFFFF	0__
Write	x__7____	0__
Read	x37A3	__1
Read	x____D	__1

全部访问完成后，在每个存储单元中填入所保存的十六进制数字。

Address	D[15:12]	D[11:8]	D[7:4]	D[3:0]
00				
01				
10				
11				

5.48 执行下面两条指令后，在什么条件下，接下来执行的指令位于 x3039？

```
x3030  0001 000 001 0 00 010
x3031  0000 011 000000111
```

5.49 希望判断某条 LDR 是否使用 R0 作为地址计算的基址寄存器。指令在存储器中，可以装入 R4。基址寄存器由指令的位 [8:6] 指定，因此可将 0000000111000000 装入 R5，再执行 AND R6, R5, R4。满足什么条件时，就能知道基址寄存器是 R0？

5.50 ★ 以下三条指令都将低九位符号扩展，再加入到自增后的 PC 上形成地址：条件分支、装入有效地址、LD。

The Conditional Branch	<table><tr><td>0000</td><td>111</td><td>xxxxxxxxxx</td></tr></table>	0000	111	xxxxxxxxxx
0000	111	xxxxxxxxxx		
The Load Effective Address	<table><tr><td>1110</td><td>111</td><td>xxxxxxxxxx</td></tr></table>	1110	111	xxxxxxxxxx
1110	111	xxxxxxxxxx		
The LD Instruction	<table><tr><td>0010</td><td>111</td><td>xxxxxxxxxx</td></tr></table>	0010	111	xxxxxxxxxx
0010	111	xxxxxxxxxx		

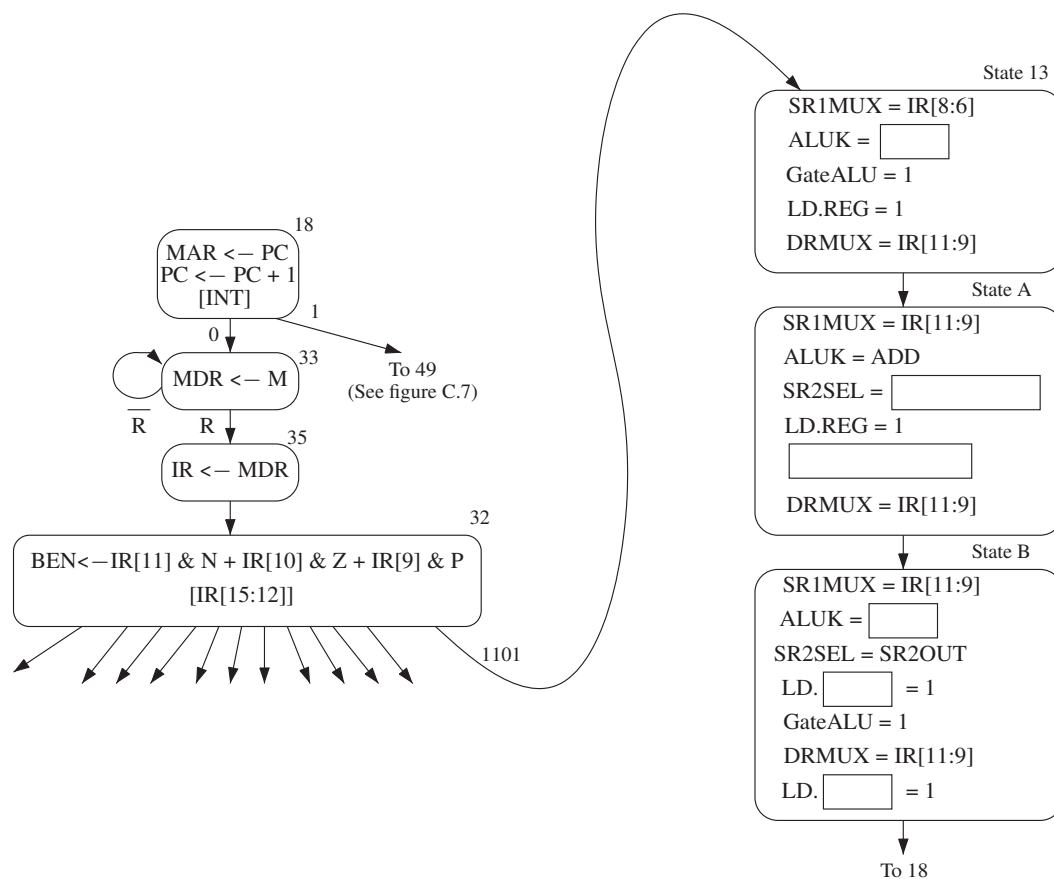
xxxxxxxxxx 表示要符号扩展的九位偏移量。对于每条指令，LC-3 微体系结构把这个加法结果放到哪里？

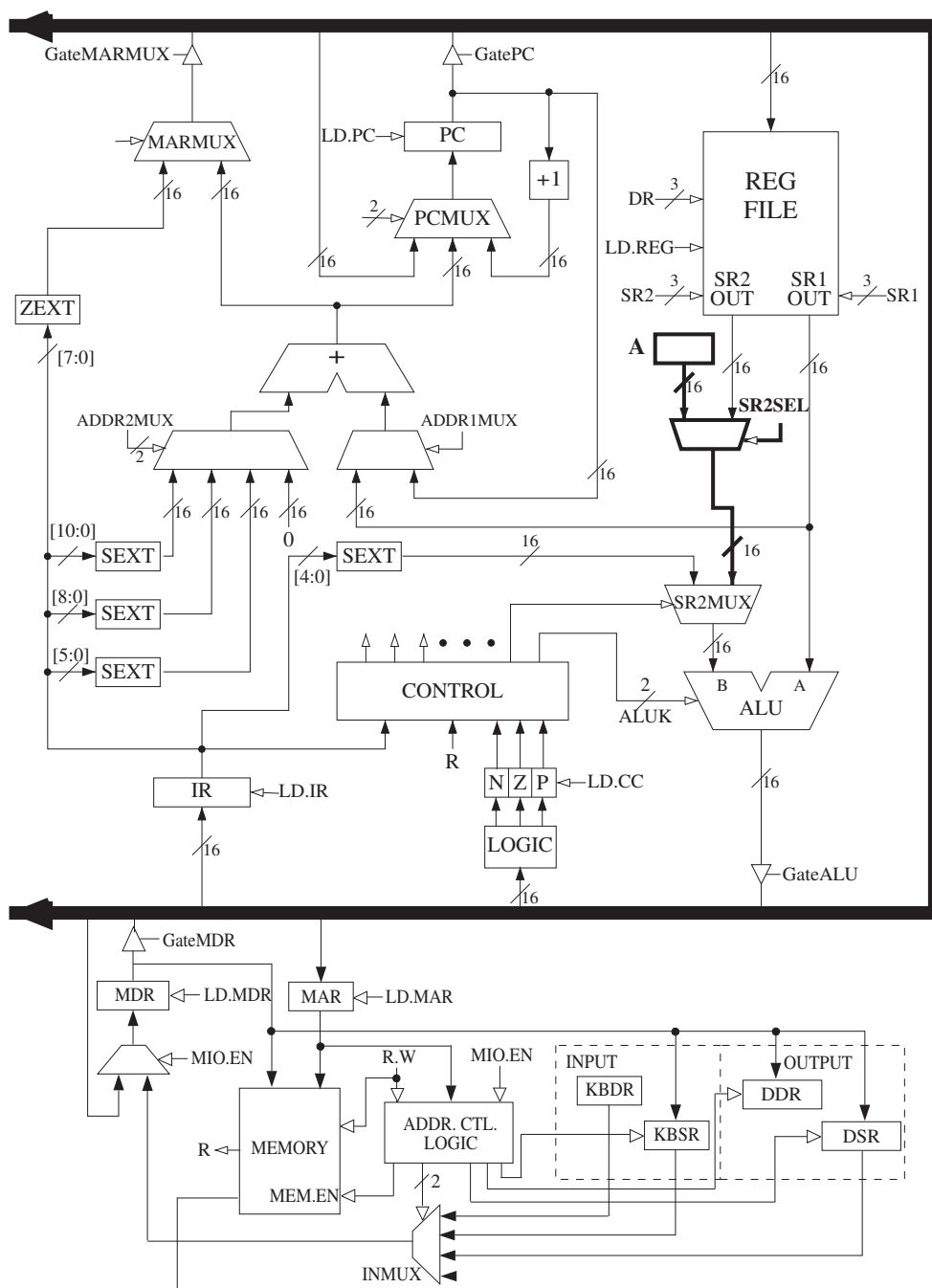
5.51 ★ 一位雄心勃勃的年轻工程师准备制造并销售 LC-3，却被告知：要成功，必须有 SUBTRACT 指令。他决定用空闲操作码 1101 按以下格式定义它：

15	12	11	9	8	6	5	3	2	0		
1101				DR		SR1		000		SR2	

指令含义为 $DR \leftarrow SR2 - SR1$ ，并设置条件码。假设 DR、SR1、SR2 是三个不同寄存器。工程师需要在状态机中增加三个状态，在数据通路中增加一个 mux 和寄存器 A。修改后的状态机与数据通路如下。新控制信号 SR2SEL 为一位，用于选择 SR2OUT 或 REGISTER A。

你的任务：在状态机空框中填入实现 SUBTRACT 所需的控制信号，并在数据通路中填入寄存器 A 的值。





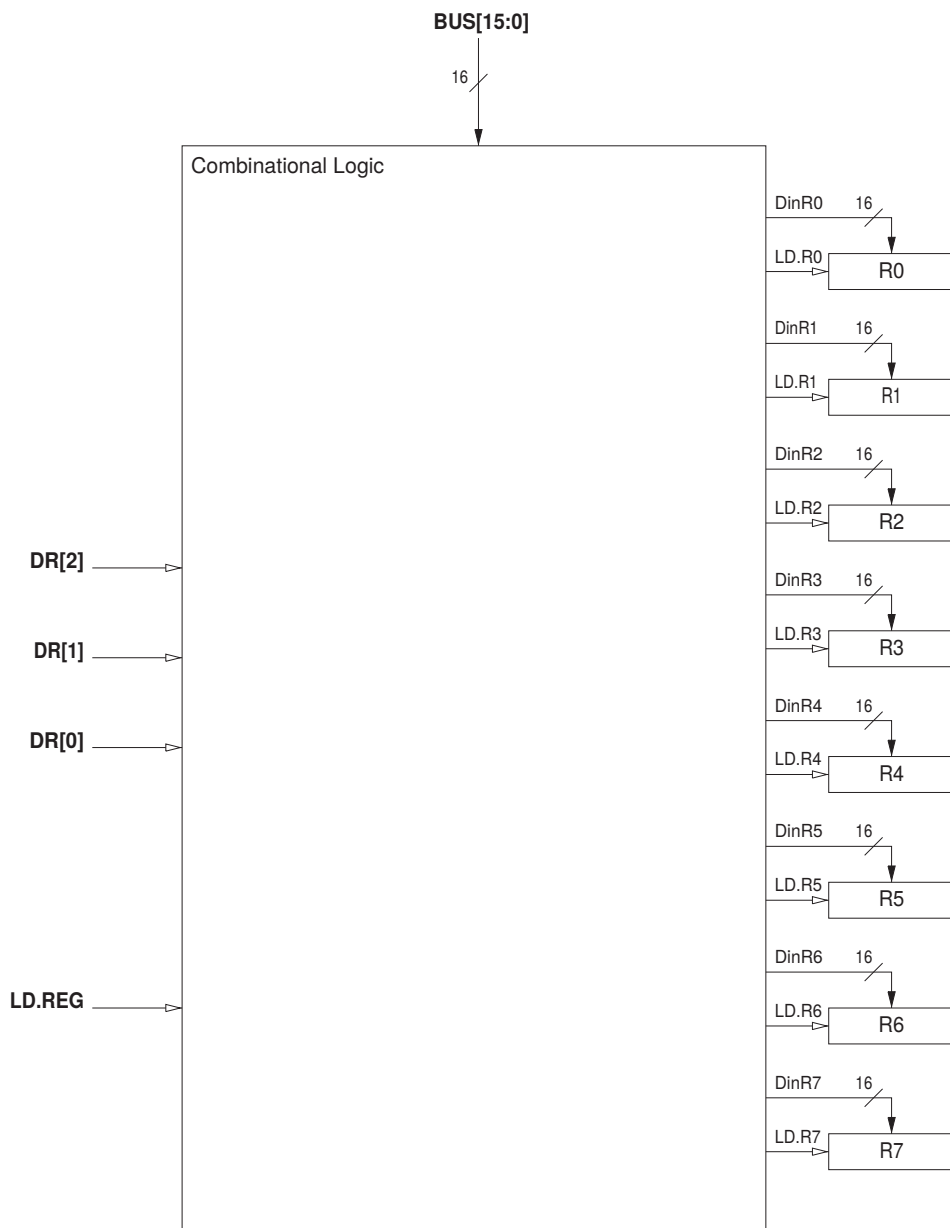
5.52 下列是 16 种操作码。圈出那些在指令周期某个时刻会写通用寄存器 R0 至 R7 的操作码。

ADD AND BR JMP JSR LD LEA LDI
LDR NOT RTI ST STI STR TRAP reserved

5.53 LC-3 的八个 GPR 构成寄存器组。写入一个寄存器时，控制单元必须提供 16 位数据 BUS[15:0]、目的寄存器编号 DR[2:0]，以及写使能 LD.REG。下图组合逻辑块的输入为 BUS[15:0]、DR[2:0]、LD.REG，输出为 DinR0[15:0]、DinR1[15:0]、...、DinR7[15:0] 和 LD.R0、LD.R1、...、LD.R7。

请添加必要的导线、逻辑门和标准逻辑块，完成该组合逻辑。若使用标准逻

辑块，无需画出内部每个门，但必须明确标识其类型，例如“16 选 1 mux”，并按功能标注有关输入输出。



5.54 所有指令都会在取指令阶段把存储器中的指令装入 MDR，再送往 IR。译码完成后，有些指令再次装入 MDR，使用图中 mux A 的源 0；另一些使用源 1。15 条指令中只有一条在译码后，会在不同时间分别使用源 0 和源 1 装入 MDR。它的操作码是什么？

5.55 ★ 一个 LC-3 程序从 x3000 开始执行。下表记录执行前，以及第 1、2、3、4、5、6 条指令执行后八个寄存器的快照。

Registers	Initial Value	After 1st Instruction	After 2nd Instruction	After 3rd Instruction	After 4th Instruction	After 5th Instruction	After 6th Instruction
R0	x4006	x4050	x4050	x4050	x4050	x4050	x4050
R1	x5009	x5009	x5009	x5009	x5009	x5009	x5009
R2	x4008	x4008	x4008	x4008	x4008	x4008	xC055
R3	x4002			x8005	x8005	x8005	x8005
R4	x4003	x4003	x4003	x4003			x4003
R5	x400D	x400D			x400D	x400D	x400D
R6	x400C	x400C	x400C	x400C	x400C	x400C	x400C
R7	x6001	x6001	x6001	x6001			x400E

执行期间还记录了 PC、MAR 和 MDR 的跟踪记录。PC 跟踪记录按执行顺序记录程序执行过的指令地址。

PC Trace
x400D
x400E

MAR Trace	MDR Trace
	xA009
x3025	
	x1703
	x4040
x400E	x1403

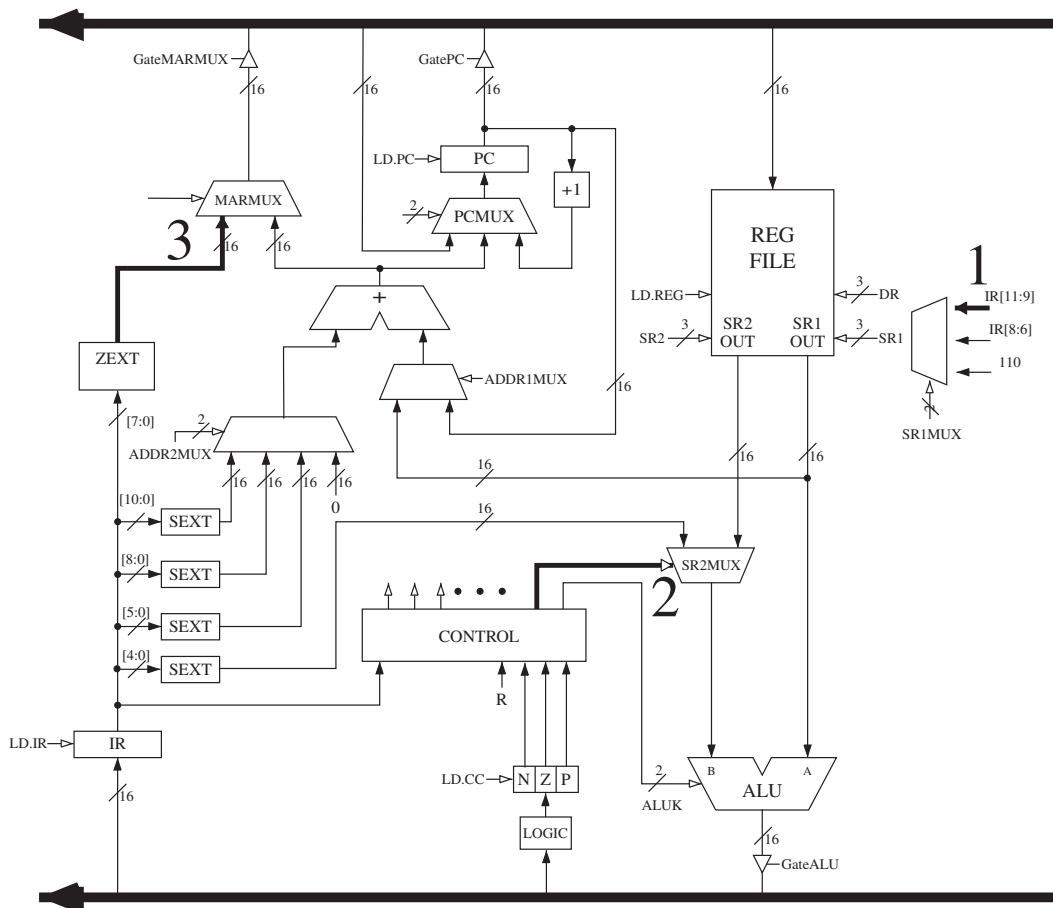
请填入上述三个表中缺失的项目。

- 5.56** 本题检查你对 NOT 指令周期的理解。需要给出处理 NOT 的每个时钟周期中若干控制信号的值。指令周期从下表所示状态 18 开始，请标识各状态，并按下列约定填写信号值。某状态中无关的信号填 X。可能不需要使用全部行。假设一次存储器访问只需一个时钟周期。

Cycle	State	LD.PC	LD.MAR	LD.MDR	LD.REG	LD.CC	GateALU	GatePC	ALUK	PCMUX
1	18									
2										
3										
4										
5										
6										
7										
8										
9										
10										

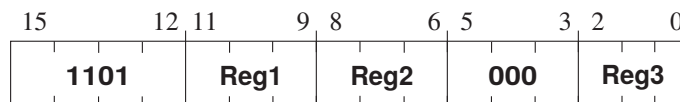
填写约定：LD.PC、LD.MAR、LD.MDR、LD.REG、LD.CC 的 0/1 分别表示不允许/允许装入；GateALU、GatePC 的 0/1 分别表示不传送/传送信号；ALUK 的 00、01、10、11 分别表示 ADD、AND、NOT、直接传递输入 A；PCMUX 的 00、01、10 分别选择 PC+1、BUS、加法器输出。

5.57 注意以下数据通路中的加粗信号线。

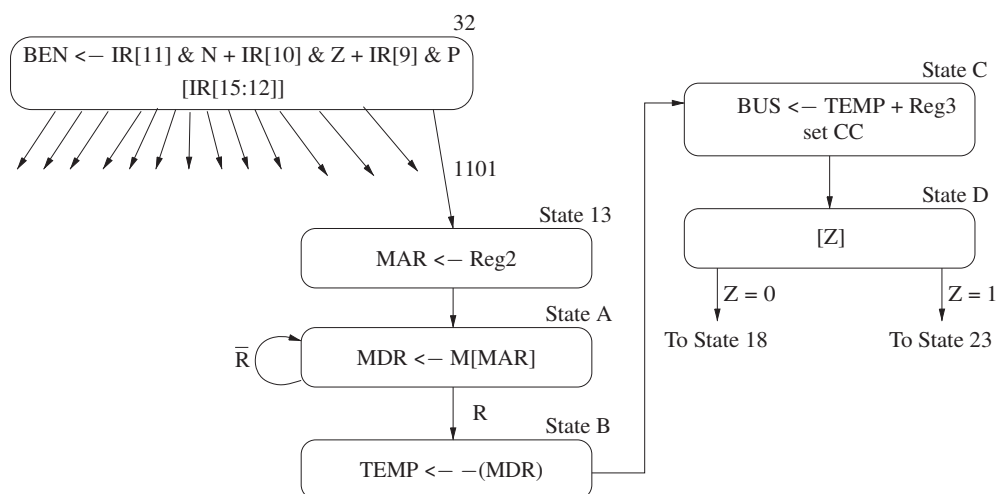
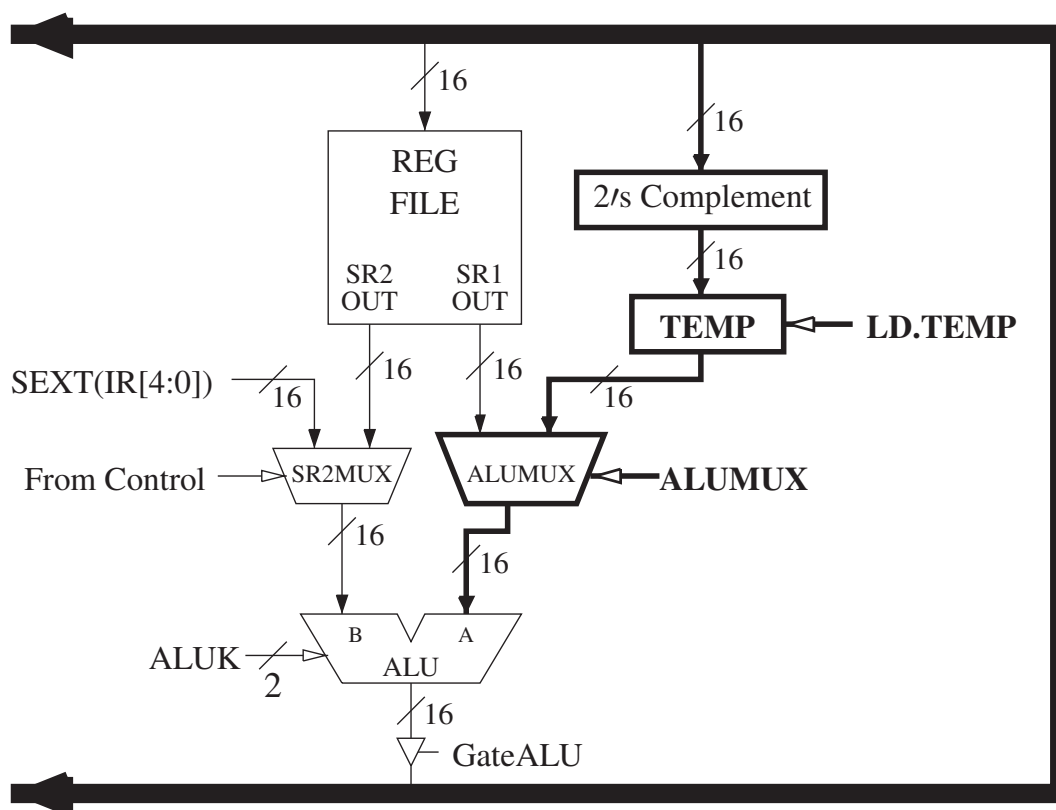


1. 哪些操作码使用 IR[11:9] 作为 SR1 输入？
2. 这个 mux 的控制信号来自哪里？请具体说明。
3. 哪些操作码使用 MARMUX 的这个输入？

5.58 用空闲操作码实现下列新指令：



为此，要在数据通路中增加图中加粗的部分，并新增五个状态来控制数据通路完成指令工作。



注意：状态 B 把 MDR 内容的相反数装入 TEMP。

a. 填写下表，给出每个状态所需的控制信号值。

	LD.PC	LD.MAR	LD.MDR	LD.CC	LD.TEMP	GatePC	GateMDR	GateALU	SR1MUX[1:0]	ALUMUX	ALUK[1:0]	MIO.EN	R.W
State 13													
State A													
State B													
State C													
State D													

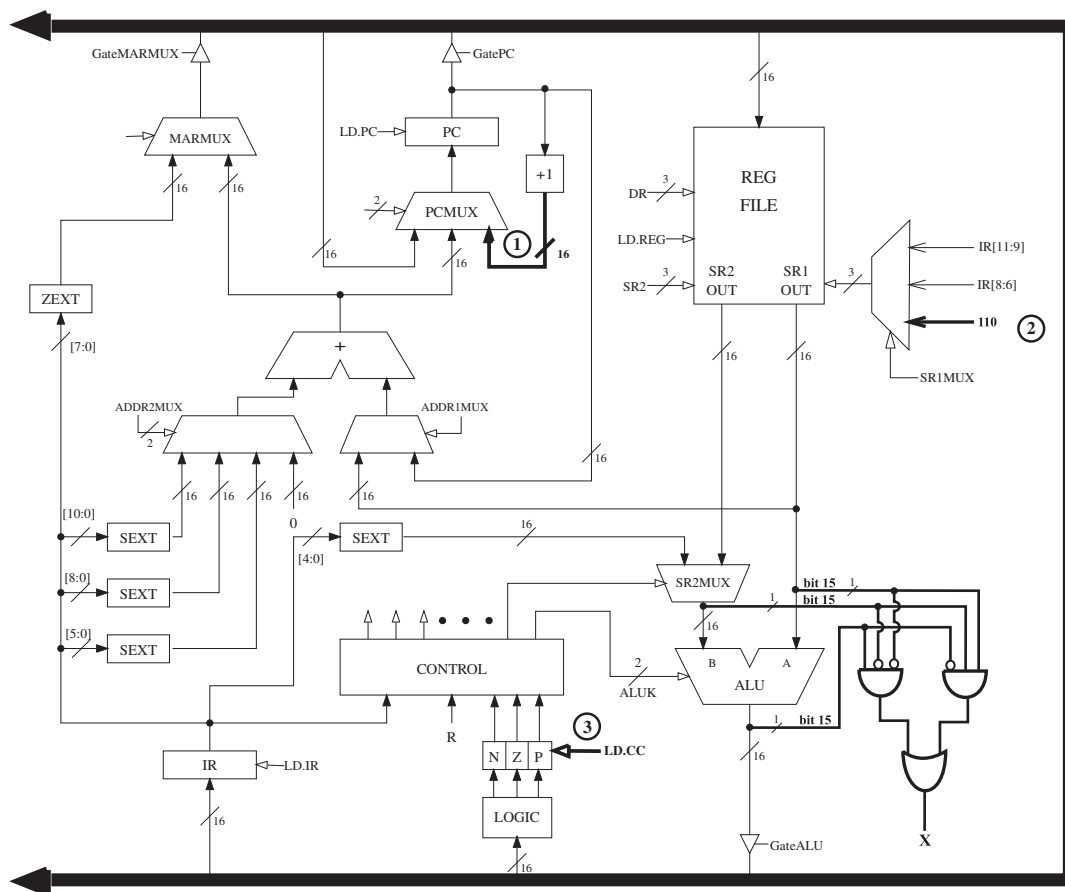
约定：各 LD 信号的 0/1 表示不允许/允许装入，GatePC、GateMDR、GateALU 的 0/1 表示不传送/传送；SR1MUX 的 00、01、10 分别选择 IR[11:9]、IR[8:6]、R6；ALUMUX 的 0/1 分别选择 SR1、TEMP；ALUK 的 00、01、10、11 分别表示 ADD、AND、NOT、直接传递输入 A；MIO.EN 的 0/1 表示禁止/允许存储器 I/O；R.W 的 0/1 分别表示读、写。

b. 新指令做什么？

5.59 假设每次存储器访问需要五个周期，LC-3 每条指令的取指令和译码共需要八个周期，但完整处理所需总周期数取决于指令的工作。仍假设每次存储器访问需要五个周期，且计算机一次从头到尾只处理一条指令，请给出每种指令需要的时钟周期数。

Instruction	Number of cycles
ADD	
AND	
LD	
LEA	
LDI	
NOT	
BRnzp	
TRAP	

5.60 下图数据通路中有几条线（信号）加粗，分别编号 1、2、3。并非每条指令都使用全部三条线；对于某些指令，如果删除某条线，就无法正确执行。



分别列出处理过程中使用线 1、使用线 2，以及要求线 3 上 LD.CC=1 的操作码。

注意图中新增加粗的逻辑，其输出为 X。当 ALUK 选择 ADD 时，X=1 表示什么？

5.61 ★ 某个 LC-3 程序执行时，监测了连续处理的四条指令的数据通路。下表列出总线被使用的时钟周期，以及其中部分周期的编号、总线值和状态机状态。第一条指令从时钟周期 T 开始，每次存储器访问需要五个周期。

请填写表中缺失项目，只需填写未标 x 的格子。五个时钟周期需要填写控制信号。虽然表中没有列出全部 LC-3 控制信号，但这五个周期所需的信号都已列出。

DRMUX 填“11.9”“R7”或“SP”; R.W 填“R”或“W”; PCMUX 填“PC+1”“BUS”或“ADDER”; 其他信号填写实际位值。若某周期不需要某控制信号, 填短横线 -。

Inst. #	Clock Cycle	Bus	State	Control Signals														
				Gate PC	Gate MDR	Gate ALU	Gate MARMUX	LD. PC	LD. MDR	LD. MAR	LD. CC	LD. Reg	DR MUX	MIO. EN	R.W	PC MUX		
Inst. 1	T + 0	x3010																
	T + 6			x	x	x	x	x	x	x	x	x	x		x	x	x	
	T + 8	x00AB		x	x	x	x	x	x	x	x	x	x		x	x	x	
	T + 9		28															
	T + 10		28															
	T + 11		28															
	T + 12	x	28	x	x	x	x	x	x	x	x	x	x		x	x	x	
	T + 13	x	28	x	x	x	x	x	x	x	x	x	x		x	x	x	
	T + 14			x	x	x	x	x	x	x	x	x	x		x	x	x	
Inst. 2	T + 15	x1510	18	x	x	x	x	x	x	x	x	x	x		x	x	x	
		x2219		x	x	x	x	x	x	x	x	x	x		x	x	x	
				x	x	x	x	x	x	x	x	x	x		x	x	x	
	T + 29	x8001																
Inst. 3	T + 30			x	x	x	x	x	x	x	x	x	x		x	x	x	
	T + 36	x0804		x	x	x	x	x	x	x	x	x	x		x	x	x	
Inst. 4			18	x	x	x	x	x	x	x	x	x	x		x	x	x	
	x	x1200	x	x	x	x	x	x	x	x	x	x	x		x	x	x	
	x	x0000	x	x	x	x	x	x	x	x	x	x	x		x	x	x	

第六章 程序设计

现在，我们已经可以编写程序，让计算机解决问题了。本章尝试完成两件事：首先，建立一套编写程序来解决问题的方法（6.1 节，问题求解）；其次，考虑到第一次编写时很可能无法把一切都做对，再建立一套修正程序的方法（6.2 节，调试）。

长期以来，人们习惯把程序中的错误称为 bug，把消除这些错误的过程称为**调试** (debugging)。复杂程序中可能引入错误的地方很多，因此，让程序正确工作所花的时间，也就是调试时间，通常远多于最初编写程序的时间。

6.1 问题求解

6.1.1 系统化分解

回想第 1 章，要让电子解决问题，就必须经过若干层转换，把用自然语言描述的问题变成电子能够处理的形式。对本书而言，这种自然语言是英语，当然，你们中的许多人可能更喜欢意大利语、汉语、印地语或其他语言。得到自然语言的问题描述之后，下一步就是把它转化为算法，也就是一个逐步执行的过程，并使其具有确定性（每一步都有精确的表述）、有效可计算性（每一步都能由计算机执行）和有穷性（过程能够终止）。

20 世纪 60 年代后期，**结构化程序设计** (structured programming) 的概念兴起。它让普通程序员处理复杂问题的能力大为提高：将复杂的问题描述系统地分解成越来越小、易于处理的单元，最终编写出正确执行的程序。这种方法也称为**系统化分解** (systematic decomposition)，因为它系统地把较大的任务拆成较小的任务。

我们将看到，系统化分解模型是设计程序来完成复杂任务的一种有用技术。

6.1.2 三种结构：顺序、条件和迭代

系统化分解，就是把一个任务，即一个工作单元（见图 6.1a），拆成若干较小的工作单元，使这些小单元合起来完成与原来大单元相同的任务。从一个庞大而复杂的任务出发，反复应用这一过程，最后就能得到很小的工作单元，也就容易为每个单元编写程序。这一过程也称为**逐步求精** (stepwise refinement)：每次只推进一步，把一个仍然过于复杂的任务细化成一组较简单的子任务。

基本思路是用一种结构替换较大的工作单元，使分解后的结构仍能正确完成原任务。基本上有三种这样的结构：顺序、条件和迭代。

顺序结构（图 6.1b）适用于可以分解成两个先后执行的子任务的情形。计算机先完整地执行第一个子任务，再完整地执行第二个；一旦开始第二个，就不再返回第一个。

条件结构（图 6.1c）适用于根据某个条件，在两个子任务中选择一个执行的情形，两者不会都执行。条件为真时执行其中一个，否则执行另一个。任一子任务都可以为空，即什么也不做。不论哪种情况，完成所选子任务之后，程序都继续向下执行，不会返回去重新测试条件。

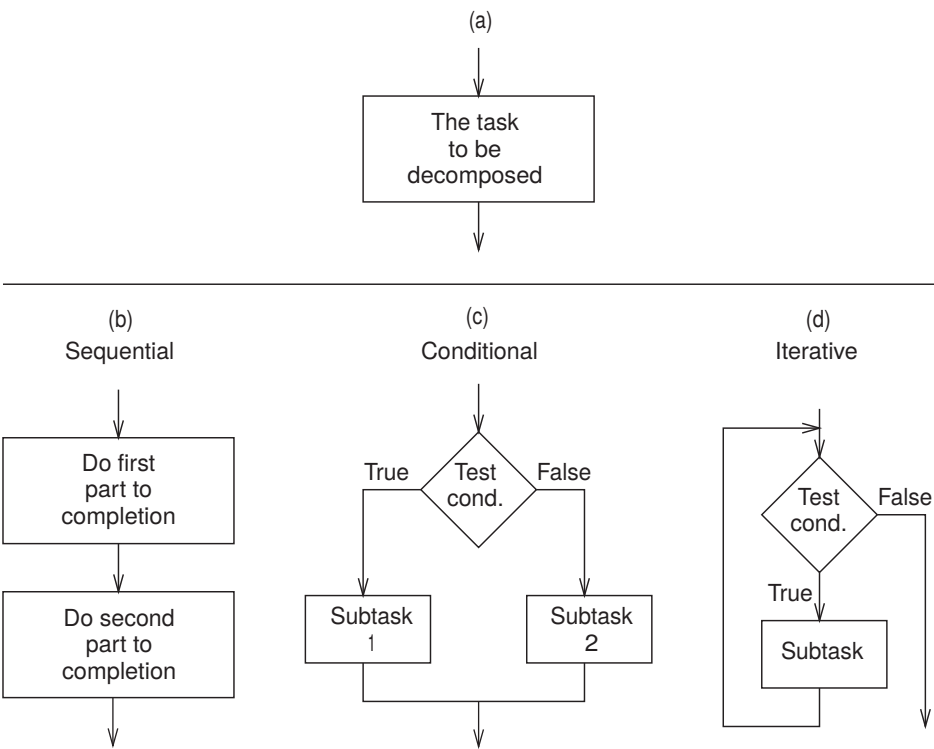


图 6.1 结构化程序设计的基本结构

迭代结构（图 6.1d）适用于在某个条件为真期间反复执行一个子任务的情形。条件为真就执行子任务；子任务完成后，再返回去测试条件。只要测试结果为真，程序就一遍又一遍地执行同一个子任务。第一次遇到条件不成立时，程序才继续向下执行。

注意图 6.1：无论图 6.1a 中的任务是什么，工作都从表示该任务的“方框”顶部的箭头进入，从底部的箭头离开。方框内发生什么并未说明。三种分解方式——图 6.1b、c、d——都恰好有一个入口和一个出口。因此，图 6.1a 形式的任何任务，都很容易用适当的分解结构替换。下面通过实例说明。

6.1.3 用 LC-3 控制指令实现三种结构

讨论实例之前，先看图 6.2：LC-3 控制指令如何引导程序计数器，实现三种分解结构。图 6.2b、c、d 分别对应图 6.1b、c、d。

字母 A、B、C、D 表示存储 LC-3 指令的存储器地址。例如，三种情况下，A 都是第一条要执行的 LC-3 指令的地址，因为它是待分解任务的起始地址（图 6.2a）。

图 6.2b 是顺序分解的控制流。这种情况不需要控制指令，因为 PC 从地址 B_1 自增到地址 $B_1 + 1$ ，程序继续执行，直到地址 D_1 ，不会返回第一个子任务。

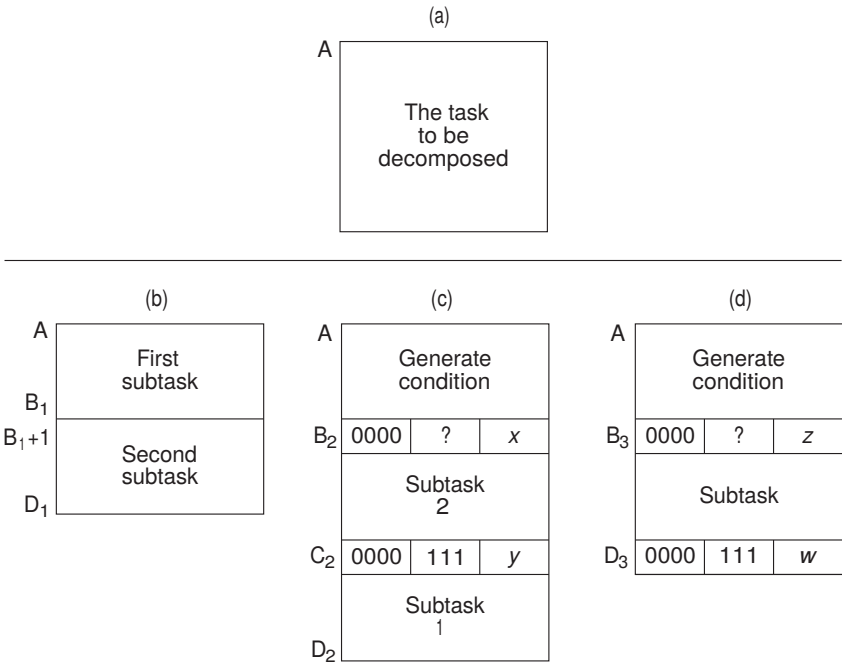


图 6.2 用 LC-3 控制指令实现结构化程序设计

图 6.2c 是条件分解的控制流。首先产生一个条件，设置某个条件码，再由地址 B_2 处的条件分支指令测试它。若条件为真，PC 被设为地址 $C_2 + 1$ ，执行子任务 1。注意， x 等于子任务 2 的指令数加 1。若条件为假，PC 在分支指令取指令阶段已经自增，因而接着取地址 $B_2 + 1$ 处的指令，执行子任务 2。子任务 2 之后，地址 C_2 处的一条无条件分支指令跳转到 $D_2 + 1$ 。注意， y 等于子任务 1 的指令数。

图 6.2d 是迭代分解的控制流。与条件结构一样，先产生条件、设置条件码，再执行条件分支指令。这里，地址 B_3 处指令的条件位被设置为在条件为假时跳转。条件为假，PC 就被设为 $D_3 + 1$ ； z 等于图中子任务的指令数加 1。反之，只要条件为真，PC 就自增到 $B_3 + 1$ ，执行子任务。其后，地址 D_3 处的无条件分支指令把 PC 设回 A，再次产生并测试条件。注意， w 等于图 6.2d 所示整个分解结构的指令总数。

现在，我们可以讨论实例了。

6.1.4 重新考察第 5 章的字符计数实例

回想 5.5 节的实例，问题是：“从键盘输入一个字符，统计这个字符在一个文件中出现的次数，并把计数值显示在显示器上。”

图 6.3 展示了如何将这一自然语言问题描述系统地分解，直到得到最终的 LC-3 实现。图 6.3a 是问题的简要表述。

求解问题时，最好先弄清楚到底要求什么，又有哪些可用的条件。本题要求从键盘取得待统计字符，检查文件中的所有字符，确定其中有多少与键盘输入的字符相同，最后输出结果。

因此，我们需要依次检查文件中的所有字符，将每个字符与键盘输入的字符比较，并用一个计数器在每次匹配时加 1。需要用寄存器保存以下信息：

1. 键盘输入的字符。
2. 扫描文件时当前所在的位置，即一个指针。
3. 文件中当前正在检查的字符。
4. 已统计的出现次数。

还必须知道何时到达文件末尾。

这个问题自然地按顺序结构分解成三部分，如图 6.3b：(A) 初始化，包括从键盘输入待“计数”的字符；(B) 确定该字符在文件中出现多少次；(C) 在显示器上显示计数值。

前面的几个实例已经说明了正确初始化的重要性。程序着手处理问题的核心之前，必须具有正确的初始值。这些值不会凭空出现在 GPR 中；它们来自每个算法最初的一组步骤：初始化变量。

本算法的初始化在第 5 章已说明：将计数器设为 0，将指针设为文件中第一个待检查字符的地址，从键盘取得输入字符，再从文件中取得第一个字符。这四步合起来就是图 6.3b 中的 A。

图 6.3c 将 B 分解为迭代结构：只要文件中还有待检查的字符，就继续循环。B1 表示每次迭代完成的工作：检查字符，若匹配就增加计数值，再为检查下一个字符做好准备。第 5 章介绍过控制循环次数的两种基本方法：哨兵法和计数器法。任意一个文件中有多少字符，我们通常事先不知道；而每个文件都以 EOT 字符结束，因此很自然地选择哨兵法：检查每个字符，判断它是文件中的普通字符还是 EOT。

图 6.3c 还进一步细化了初始化步骤，指定用 LC-3 的 R0、R1、R2、R3 分别保存键盘输入字符、当前正在测试的字符、计数器和指向待测试字符的指针。

图 6.3d 用顺序结构分别分解 B1 和 C。B1 先检查当前字符，若匹配则增加计数值 (B2)，再取下一个字符 (B3)。C 先将计数值从二进制补码整数转换成 ASCII 码，为显示作准备 (C1)，再真正输出字符 (C2)。

最后，图 6.3e 完成分解：用条件结构替换 B2，用顺序结构替换 B3，后者先增加指针，再装入下一个待扫描字符。

最后一步，也是通常最容易的一步，是为图 6.3e 的每个方框编写相应的 LC-3 代码。图 6.3e 与第 5 章的图 5.16 本质上相同，只是现在你知道它是怎么得来的了！

结束这个话题之前，还要指出：并不总能一开始就理解所有事情。遇到这种情况，不要立刻举手投降。实际上，大多数问题都是如此。可以先试着弄懂其中的一部分，再从这里扩展。问题就像谜题，起初可能看不透，但越用心钻研，就越能找到突破口。一旦理解了已知条件、要求以及如何着手，就可以回到起点（图 6.3a），重新开始系统化分解。

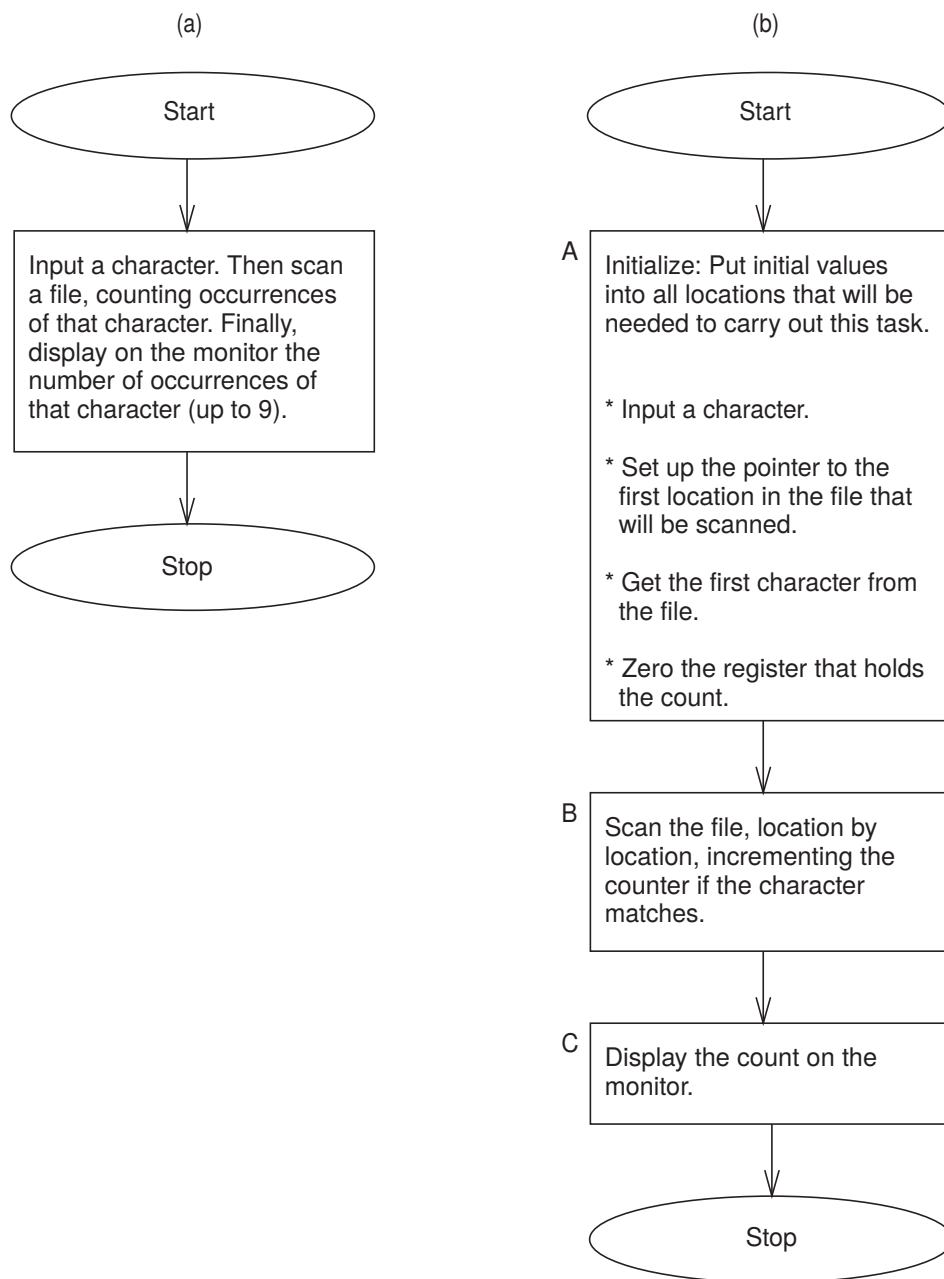


图 6.3 字符计数程序的逐步求精 (a、b)

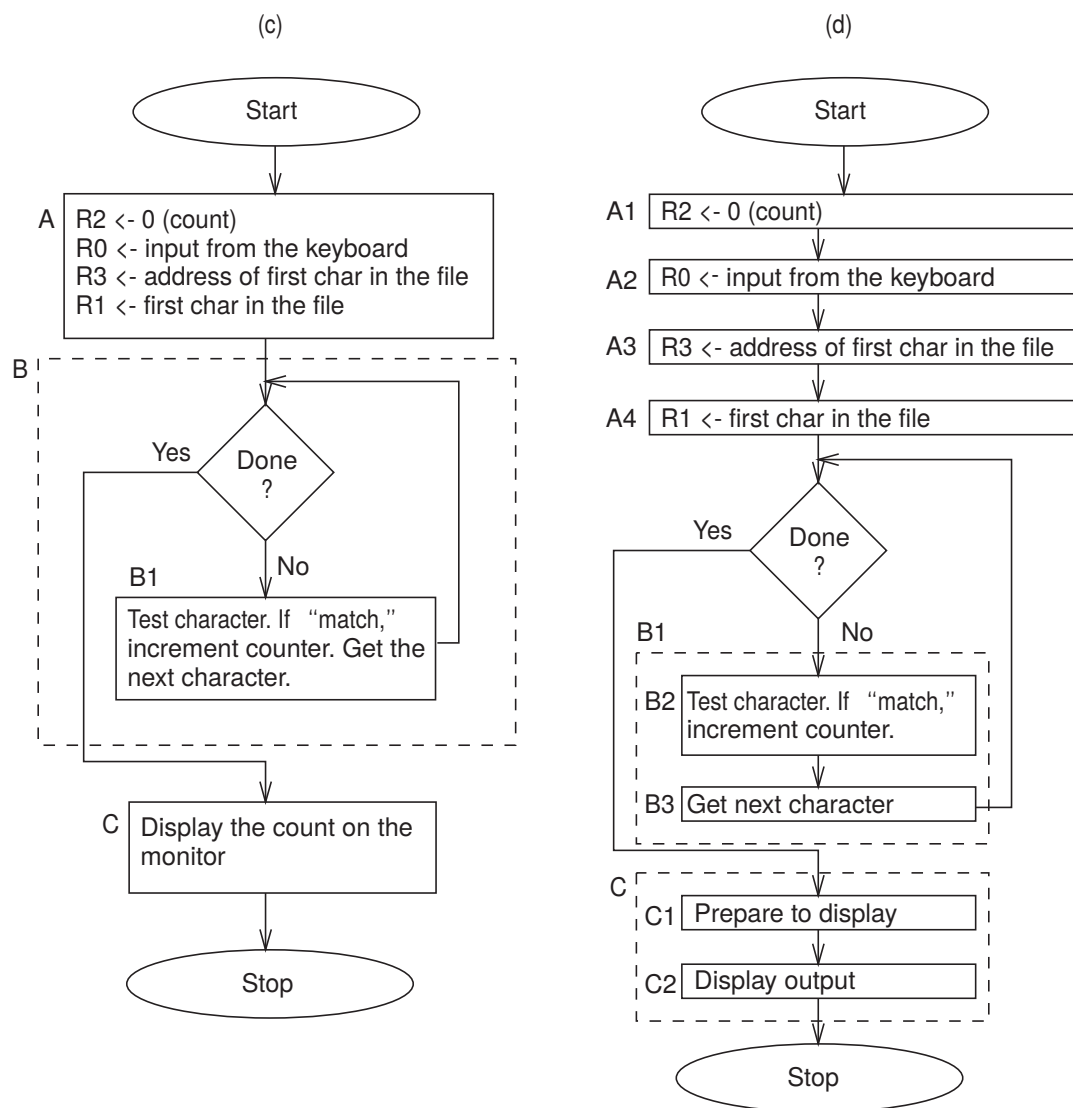


图 6.3 字符计数程序的逐步求精（续，c、d）

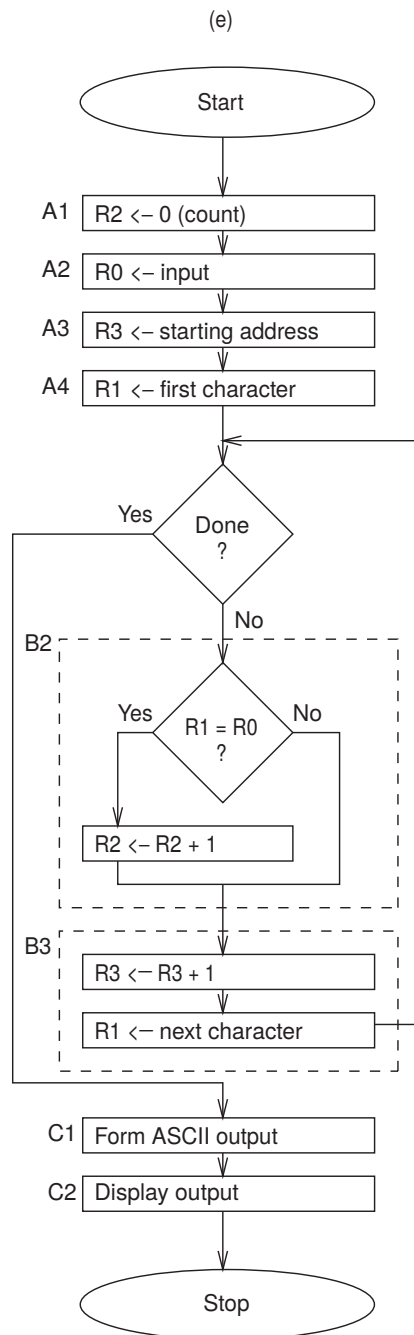


图 6.3 字符计数程序的逐步求精（续，e）

6.2 调试

调试程序很大程度上就是运用常识。举个简单的例子：你开车去一个从未去过的地方，途中某处转错了弯。现在怎么办？一种常见的“驾驶调试”方法是漫无目的地乱转，希望能找回路线。这样行不通，等你终于肯听旁边乘客的意见时，就掉头回到路线上的某个“已知”位置。然后对照地图——这对某些人很困难——沿着指示前进，不时根据窗外的地标判断自己在哪里，再与地图上应该到达的位置比较，直到抵达目的地。

调试有些类似。程序中的逻辑错误会让你“转错弯”。比较实际位置与预期位置，最简单的方法就是**跟踪**程序：记录已经执行的指令序列，以及每条指令产生的结果。检查指令序列可以发现程序控制流中的错误；比较每条指令实际完成的工作与它应当完成的工作，可以发现逻辑错误。只要程序执行时的行为偏离了预期，就说明存在错误。

一种有用的技术是把程序分成若干部分，通常称为**模块**，并检查每个模块执行结束时的计算结果。6.1 节的结构化程序设计方法实际上可以帮助你确定在程序执行到哪里时检查结果。这样，就能系统地缩小范围，最终把注意力集中到引起问题的一条或几条指令上。

6.2.1 调试操作

市面上有许多复杂的调试工具，今后你无疑会使用其中的许多种。例如，第 15 章将讨论如何使用 C 的源代码级调试器进行调试。

现在仍停留在机器体系结构这一层，看看几种很基本的交互式调试操作能做什么：对用 LC-3 ISA 编写的程序设置断点、单步执行，并检查程序状态。第 15 章还会再次讨论这里引入的断点、单步执行和状态检查，只不过对象换成 C 程序，而不是由 LC-3 ISA 的 0 和 1 组成的程序。

交互式调试时，用户坐在键盘和显示器前，向计算机发出命令。这里，就是通过 LC-3 模拟器的菜单操作模拟器。以下能力很重要：

1. 向存储位置和寄存器写入数值。
2. 执行程序中的指令序列。
3. 在需要时停止执行。
4. 在程序的任意位置检查存储器和寄存器中的内容。

这几种简单操作对调试大有帮助。

6.2.1.1 设置数值

为了单独测试程序的一部分，而不必顾虑前面的部分，通常可以预先向存储器和寄存器写入那些本应由前面部分产生的数值。例如，一个模块从键盘输入，后一个模

块处理这些输入。若想在第一个模块还未调试完成时测试第二个，只要知道键盘输入模块最终会在 R0 中留下一个 ASCII 码，就可以先把一个 ASCII 码写入 R0，再测试处理输入的模块。

6.2.1.2 执行序列

能够执行一段指令后停下来，检查这段指令计算出的结果，也很重要。通常有三种简单的机制：运行、步进和设置断点。

Run 命令让程序持续执行，直到遇到使它停止的条件，例如 HALT 指令或断点。

Step 命令让程序执行固定数量的指令后停止。交互式用户输入希望模拟器执行的指令数；若这个数是 1，计算机就执行一条指令后停止。这称为**单步执行**（single-stepping），使调试者能够检查每条指令各自产生的结果。

Set Breakpoint 命令让程序在指定的指令处停止。执行这条调试命令，就是向模拟器维护的一个列表加入一个地址。每条指令的取指令阶段，模拟器将 PC 与列表中的地址比较，若匹配就停止。因此，设置断点的效果是：程序继续执行，直到 PC 中出现被设为断点的地址。若想了解程序执行到某处已经计算出什么，可以在该地址设置断点并执行 Run。程序执行到那里就停下，用户便能检查此前的计算结果。不再需要在此处停止时，可用 Clear Breakpoint 命令清除断点。

6.2.1.3 显示数值

最后，模拟器停止执行后，需要能够检查执行结果。Display 命令允许用户检查任意存储位置或任意寄存器的内容。

6.2.2 使用交互式调试器

本章最后用四个实例说明交互式调试操作如何帮助我们查找程序错误。选择的四种错误是：（1）循环控制设置不当，循环次数错误；（2）混淆操作码为 0010 的装入指令与操作码为 1110 的有效地址装入指令，前者把存储位置的内容装入寄存器，后者把存储位置的地址装入寄存器；（3）忘记哪些指令设置条件码，导致分支指令测试了错误的条件；（4）未覆盖所有可能的输入情况。

6.2.2.1 例 1：没有乘法指令时进行乘法运算

先看一个见过的例子：计算机没有乘法指令时，如何将两个正整数相乘。LC-3 就是这样的计算机。这次假定两个待乘整数位于 R4 和 R5，执行结果，即它们的乘积，放入 R2。图 6.4 是我们为此编写的程序。

逐条检查可见，程序先清零 R2，再尝试将 R4 累加 R5 初始值那么多次来完成乘法。每加一次，R5 减 1。按预期，R5 为 0 时程序终止。

看起来程序应该能工作！然而，执行时发现 R4 初值为 10、R5 初值为 3，程序却给出 40。哪里出了问题？

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3200	0	1	0	1	0	1	0	0	1	0	1	0	0	0	0	0	R2 <- 0
x3201	0	0	0	1	0	1	0	0	1	0	0	0	0	1	0	0	R2 <- R2 + R4
x3202	0	0	0	1	1	0	1	1	0	1	1	1	1	1	1	1	R5 <- R5 - 1
x3203	0	0	0	0	0	1	1	1	1	1	1	1	1	0	1	1	BRzp x3201
x3204	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT

图 6.4 调试例 1：不用乘法指令实现乘法的 LC-3 程序

首先想到跟踪程序。程序假定 R4 和 R5 中是正整数，因此跟踪前先用设置数值的命令，将 10 写入 R4，将 3 写入 R5。

给每条指令加上准确描述其算法作用的注释也很有用。对一个包含 10,000 条指令的程序，这可能很烦琐，帮助也不大；但将错误定位到几条指令之后，往往很有帮助。快速扫一眼指令序列，与精确说出每条指令在做什么，差别很大。匆匆看过容易看错，精确表述通常不会。图 6.4 的每条指令旁都有这样的注释。

图 6.5a 给出可通过单步执行获得的跟踪记录。PC 列表示每条指令开始时 PC 的内容；R2、R4、R5 列也分别表示每条指令开始时这三个寄存器的值。

(a)

PC	R2	R4	R5
x3201	0	10	3
x3202	10	10	3
x3203	10	10	2
x3201	10	10	2
x3202	20	10	2
x3203	20	10	1
x3201	20	10	1
x3202	30	10	1
x3203	30	10	0
x3201	30	10	0
x3202	40	10	0
x3203	40	10	-1
x3204	40	10	-1
	40	10	-1

(b)

PC	R2	R4	R5
x3203	10	10	2
x3203	20	10	1
x3203	30	10	0
x3203	40	10	-1

图 6.5 调试例 1：(a) 乘法程序的跟踪记录；(b) 使用断点跟踪

一看记录便发现，循环体执行了四次，而不是三次。这提示分支指令的条件位可能设置错了。进而可发现，应该仅在 R5 为正时跳转，R5 为 0 时不应跳转。正是分支指令的位 [10] 为 1，使循环多执行了一次。

只需把 x3203 处的指令替换为下面这条指令，就可以修正程序：

```
0000 0011 1111 1101    BRp
```

使用断点还可以节省许多跟踪工作。在 x3203 设置断点，每次检查一轮循环的结果，而不必检查每条指令的结果。让程序在每轮循环后停止，通常就足以发现并修正

问题，免去逐条单步执行的烦琐。图 6.5b 就是以一轮循环为一步的跟踪记录：四轮而非三轮，立刻暴露了错误。

最后再说一点。开始跟踪前，我们把 R4 和 R5 初始化为 10 和 3。测试程序时，慎重选择初值很重要。本题只要求处理正整数，因此 10 和 3 大概合适。若另一个乘法程序要求处理所有整数，可以试试 -6 和 3、4 和 -12，以及 -5 和 -7。但这组测试遗漏了一个极重要的初值：0。要适用于“所有”整数，就必须也适用于 0。程序必须对所有合法源操作数都能正确工作；一种好的测试方法是使用不常见、程序员可能未考虑到的值。这类情况通常称为**边角情况**（corner cases），程序往往恰恰在这些值上出错。

6.2.2.2 例 2：一系列数求和

图 6.6 的程序应将从 x3100 开始的十个存储位置中的数相加，并将结果保存在 R1。从 x3100 开始的二十个存储位置的内容见图 6.7。

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	1	0	1	0	0	1	0	0	1	1	0	0	0	0	0	R1 <- 0
x3001	0	1	0	1	1	0	0	1	0	0	1	0	0	0	0	0	R4 <- 0
x3002	0	0	0	1	1	0	0	1	0	0	1	0	1	0	1	0	R4 <- R4 + 10
x3003	0	0	1	0	0	1	0	0	1	1	1	1	1	1	0	0	R2 <- M[x3100]
x3004	0	1	1	0	0	1	1	0	1	0	0	0	0	0	0	0	R3 <- M[R2]
x3005	0	0	0	1	0	1	0	0	1	0	1	0	0	0	0	1	R2 <- R2 + 1
x3006	0	0	0	1	0	0	1	0	0	1	0	0	0	0	1	1	R1 <- R1 + R3
x3007	0	0	0	1	1	0	0	1	0	0	1	1	1	1	1	1	R4 <- R4 - 1
x3008	0	0	0	0	0	0	1	1	1	1	1	1	1	0	1	1	BRp x3004
x3009	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT

图 6.6 调试例 2：将 10 个整数相加的 LC-3 程序

程序预期的工作过程如下。x3000 至 x3003 的指令初始化变量：x3000 将和 R1 初始化为 0，x3001 和 x3002 将循环控制计数器 R4 初始化为 #10。每轮减 1，直到 R4 为 0。x3003 将基址寄存器 R2 初始化为待加数的起始地址 x3100。

此后，每轮循环从存储器装入一个值到 R3（x3004），增加基址寄存器以准备下一轮（x3005），将 R3 加到保存累积和的 R1（x3006），递减计数器（x3007），再测试 P 位；若为真，就将 PC 设为 x3004，开始下一轮（x3008）。十轮之后，R4 为 0，P 位为 0，不发生跳转，程序在 x3009 终止。

看起来应该能工作。然而，执行后检查 R1，却得到 x0024，而非 x3100 至 x3109 中各数之和 x8135。哪里出了问题？

借助调试器跟踪程序，前四条指令的记录见图 6.8。注意，x3003 的指令执行后，R2 是 x3107，而非预期的 x3100。问题在于操作码 0010 把 M[x3100] 的内容，也就是 x3107，装入了 R2，而不是装入地址 x3100。结果，程序加的是从 M[x3107] 开始的十个数，而非从 M[x3100] 开始的十个数。

错误在于操作码选错了。应该使用 1110，它会将地址 x3100 装入 R2。将操作码 0010 改为 1110 后，程序就正确运行了。

Address	Contents
x3100	x3107
x3101	x2819
x3102	x0110
x3103	x0310
x3104	x0110
x3105	x1110
x3106	x11B1
x3107	x0019
x3108	x0007
x3109	x0004
x310A	x0000
x310B	x0000
x310C	x0000
x310D	x0000
x310E	x0000
x310F	x0000
x3110	x0000
x3111	x0000
x3112	x0000
x3113	x0000

图 6.7 调试例 2 中 x3100 至 x3113 的存储器内容

PC	R1	R2	R4
x3001	0	x	x
x3002	0	x	0
x3003	0	x	#10
x3004	0	x3107	#10

图 6.8 调试例 2：求和程序前四条指令的跟踪记录

6.2.2.3 例 3：一段存储位置中是否包含 5？

图 6.9 的程序检查从 x3100 开始的十个存储位置：只要其中任一个含有 5，就将 R0 设为 1；若都不含 5，就将 R0 设为 0。

程序预期的行为如下。前六条指令（x3000 至 x3005）分别将 R0 初始化为 1、R1 为 -5、R3 为 10。x3006 将 R4 初始化为第一个待检查位置的地址 x3100，x3007 将 x3100 的内容装入 R2。

x3008 和 x3009 通过给 R2 加上 -5 并在结果为 0 时跳转到 x300F，判断 R2 是否为 5。R0 初值为 1，因此这样终止时，R0 就报告已检查的位置中存在 5。

x300A 增加 R4，准备装入下一个值；x300B 减少 R3，表示剩余待检查值的数量；x300C 将下一个值装入 R2；若 R3 表示还有值未检查，x300D 就跳回 x3008，重复上述过程。若 R3 为 0，表示已全部检查，就在 x300E 将 R0 设为 0，然后在 x300F 终止。

用一组确实含有 5 的样本数据运行，程序却以 R0 为 0 结束，报告 x3100 至 x3109

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	R0 <- 0
x3001	0	0	0	1	0	0	0	0	0	0	1	0	0	0	0	1	R0 <- R0 + 1
x3002	0	1	0	1	0	0	1	0	0	1	1	0	0	0	0	0	R1 <- 0
x3003	0	0	0	1	0	0	1	0	0	1	1	1	1	0	1	1	R1 <- R1 - 5
x3004	0	1	0	1	0	1	1	0	1	1	1	0	0	0	0	0	R3 <- 0
x3005	0	0	0	1	0	1	1	0	1	1	1	0	1	0	1	0	R3 <- R3 + 10
x3006	0	0	1	0	1	0	0	0	0	0	0	0	1	0	0	1	R4 <- M[x3010]
x3007	0	1	1	0	0	1	0	1	0	0	0	0	0	0	0	0	R2 <- M[R4]
x3008	0	0	0	1	0	1	0	0	1	0	0	0	0	0	0	1	R2 <- R2 + R1
x3009	0	0	0	0	0	1	0	0	0	0	0	0	0	1	0	1	BRz x300F
x300A	0	0	0	1	1	0	0	1	0	0	1	0	0	0	0	1	R4 <- R4 + 1
x300B	0	0	0	1	0	1	1	0	1	1	1	1	1	1	1	1	R3 <- R3 - 1
x300C	0	1	1	0	0	1	0	1	0	0	0	0	0	0	0	0	R2 <- M[R4]
x300D	0	0	0	0	0	0	1	1	1	1	1	1	1	0	1	0	BRp x3008
x300E	0	1	0	1	0	0	0	0	0	0	1	0	0	0	0	0	R0 <- 0
x300F	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT
x3010	0	0	1	1	0	0	0	1	0	0	0	0	0	0	0	0	x3100

图 6.9 调试例 3：检测是否存在 5 的 LC-3 程序

中不存在 5。¹

哪里错了？在 x300D 设置断点，检查跟踪记录，结果见图 6.10。

PC	R1	R2	R3	R4
x300D	-5	7	9	3101
x300D	-5	32	8	3102
x300D	-5	0	7	3013

图 6.10 调试例 3：在 x300D 设置断点的跟踪记录

PC 第一次到达 x300D 时，已检查 x3100 的值，将 x3101 的内容 7 装入 R2；R3 表示还有九个值未检查。R4 是最近一次向 R2 装入数据所用的地址。

第二次到达时，R2 已装入 x3102 的内容 32，R3 表示还有八个值未检查。第三次，R2 已装入 x3103 的内容 0，R3 表示还有七个值未检查。装入 R2 的 0 导致 x300D 的分支不跳转，于是在 x300E 将 R0 设为 0，随后在 x300F 终止，还没检查到含有 5 的位置。

错误的原因是：分支指令紧跟在一条装入指令后面，而装入指令会根据装入的值设置条件码，覆盖 x300B 的循环控制指令根据剩余迭代次数设置的条件码。分支指令应当在还有存储位置未检查时跳转，所以它应紧跟循环控制指令，而不是同样会设置条件码的装入指令。

应把条件分支指令看成一对指令中的第二条：

指令 A 设置条件码
BR 指令 根据条件码跳转

¹原文此处写作 x310A；按题述十个存储位置及循环计数，应为 x3109。

第一条指令 A 设置条件码，第二条 BR 根据这些条件码决定是否跳转。绝不能在它们之间插入另一条设置条件码的指令，否则会覆盖 BR 所需的条件码。

x300D 根据装入 R2 的值，而非剩余待检查值的数量作判断，因此第三次执行时本该跳转却没有跳转。交换 x300B 和 x300C 的指令，让 x300D 紧跟循环控制指令，程序就能正确执行。

还要注意，前两次分支碰巧正确，因为 x300C 装入的都是正数。错误直到第三轮才表现出来。若每个错误第一次遇到就显现，当然很好，但事实往往并非如此。巧合确实会发生，也增加了调试的难度。

6.2.2.4 例 4：寻找一个字中的第一个 1

最后一个实例包含一种通常很难发现的错误。图 6.11 的程序检查某个存储位置，从左向右寻找第一个被置为 1 的位，把其位号存入 R1；若没有任何位为 1，就应把 -1 存入 R1。例如，若内容为 0010000110000000，结束时应有 R1 为 13；若内容为 0000000000000110，应有 R1 为 2。

Address	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0	
x3000	0	1	0	1	0	0	1	0	0	1	1	0	0	0	0	0	R1 <- 0
x3001	0	0	0	1	0	0	1	0	0	1	1	0	1	1	1	1	R1 <- R1 + 15
x3002	1	0	1	0	0	1	0	0	0	0	0	0	0	1	1	0	R2 <- M[M[x3009]]
x3003	0	0	0	0	1	0	0	0	0	0	0	0	0	1	0	0	BRn x3008
x3004	0	0	0	1	0	0	1	0	0	1	1	1	1	1	1	1	R1 <- R1 - 1
x3005	0	0	0	1	0	1	0	0	1	0	0	0	0	0	1	0	R2 <- R2 + R2
x3006	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	1	BRn x3008
x3007	0	0	0	0	1	1	1	1	1	1	1	1	1	1	0	0	BRnzp x3004
x3008	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1	HALT
x3009	0	0	1	1	0	1	0	0	0	0	0	0	0	0	0	0	x3400

图 6.11 调试例 4：寻找一个字中第一个 1 的 LC-3 程序

程序预期如下工作，而且通常确实如此。x3000 和 x3001 将 R1 初始化为最左一位的位号 15。x3002 将 x3400 中待检查的位模式装入 R2。x3400 离 x3000 太远，LD 够不到，因此使用间接装入指令，从 x3009 取得位模式所在的地址。

x3003 测试最高有效位 [15]。若为 1，就跳到 x3008 终止，R1 保持为 15；若为 0，就不跳转，继续执行 x3004。

循环体位于 x3004 至 x3007。首先，x3004 将 R1 减 1，得到右邻一位的位号。其次，x3005 将 R2 与自身相加，使内容左移一位，将右邻一位移到位 [15]。然后，x3006 的 BR 测试“新”的位 [15]：若为 1，就跳到 x3008 停机，此时 R1 是找到的最左侧 1 的实际位号；若为 0，x3007 就无条件跳回 x3004，开始下一轮。

这一过程持续到找到第一个 1。程序几乎总能正确工作。然而，用我们的数据执行时，程序却不能终止。哪里出了问题？

图 6.12 的记录显示，每次 PC 到达 x3007，R1 都比上一次小 1。原因如下：R1 递减、R2 左移后，被测试的位仍为 0，因此没有终止。R1 依次为 14、13、12、11、10、9、8、7、6、5、4、3、2、1、0、-1、-2、-3、-4，如此继续。

PC	R1
x3007	14
x3007	13
x3007	12
x3007	11
x3007	10
x3007	9
x3007	8
x3007	7
x3007	6
x3007	5
x3007	4
x3007	3
x3007	2
x3007	1
x3007	0
x3007	-1
x3007	-2
x3007	-3
x3007	-4

图 6.12 调试例 4：在 x3007 设置断点的跟踪记录

问题在于 x3400 的初值是 x0000。只要至少有一个 1，程序就正常；但全为 0 时，x3006 的条件分支永远不跳转，程序不断执行 x3007、x3004、x3005、x3006、x3007，再回到 x3004，没有办法退出这一序列。x3004 至 x3007 是一个循环；由于执行无法跳出，就称为**无限循环**（infinite loop）。程序永不终止，自然也得不到正确答案。

再次强调，这类错误往往最难发现，因为它属于前面说的边角情况。程序员假定至少有一位为 1；如果没有呢？程序在大多数情况下正确执行还不够，它必须始终正确执行，不因所要处理的数据而例外。

习题

- 6.1** 一个不是算法的过程，能否用结构化程序设计的三种基本结构构造出来？如果可以，请举例说明。
- 6.2** LC-3 没有减法指令。程序员若要将两个数相减，必须编写一段例程。请展示对两个整数相减这一过程的系统化分解。
- 6.3** 回想前面几章的机器忙闲状态实例。假定 x4000 中有一个 0 至 15 的整数，标识一台刚刚变忙的机器；x4001 中的值表示哪些机器忙、哪些机器空闲。编写 LC-3 机器语言程序，设置 x4001 中相应的位，表示 x4000 所标识的机器忙。例如，执行前 x4000 为 x0005、x4001 为 x3101，程序终止后 x4001 应为 x3121。²
- 6.4** 编写一段简短的 LC-3 程序，比较 R1 和 R2 中的两个数：若 $R1 = R2$ ，将 R0 设为 0；若 $R1 > R2$ ，设为 1；若 $R1 < R2$ ，设为 -1。
- 6.5** 下面两种乘法算法，哪一种更好？为什么？

$$88 \times 3 = 88 + 88 + 88 \quad \text{或} \quad 3 + 3 + 3 + 3 + \cdots + 3?$$

- 6.6** 利用习题 6.4 和 6.5 的答案，编写一个高效地将两个整数相乘、并把结果放入 R3 的程序。展示从问题描述到最终程序的完整系统化分解。
- 6.7** 下面的 LC-3 程序做什么？

x3001	1110	0000	0000	1100
x3002	1110	0010	0001	0000
x3003	0101	0100	1010	0000
x3004	0010	0100	0001	0011
x3005	0110	0110	0000	0000
x3006	0110	1000	0100	0000
x3007	0001	0110	1100	0100
x3008	0111	0110	0000	0000
x3009	0001	0000	0010	0001
x300A	0001	0010	0110	0001
x300B	0001	0100	1011	1111
x300C	0000	0011	1111	1000
x300D	1111	0000	0010	0101
x300E	0000	0000	0000	0101
x300F	0000	0000	0000	0100
x3010	0000	0000	0000	0011
x3011	0000	0000	0000	0110
x3012	0000	0000	0000	0010
x3013	0000	0000	0000	0100
x3014	0000	0000	0000	0111
x3015	0000	0000	0000	0110
x3016	0000	0000	0000	1000
x3017	0000	0000	0000	0111
x3018	0000	0000	0000	0101

²本题按给定示例以 1 表示忙，与前面采用 0 表示忙的约定不同；此处保留本题约定。

- 6.8** 为什么 6.1.4 节字符计数实例中必须初始化 R2? 换句话说, 若删去 $R2 \leftarrow 0$ 这一步, 程序可能怎样出错?
- 6.9** 使用迭代结构, 编写 LC-3 机器语言例程, 在屏幕上恰好显示 100 个 Z。
- 6.10** 使用条件结构, 编写 LC-3 机器语言例程, 判断 R2 中的数是否为奇数。
- 6.11** 编写 LC-3 机器语言例程, 将地址 A 至地址 B (含两端) 中存储的每个数加 1。假定这些位置已初始化为有意义的数; 地址 A 和 B 分别保存在 x3100 和 x3101 中。
- 6.12** a. 编写 LC-3 机器语言例程, 回显键盘上刚输入的字符。用户输入 R, 程序就立即在屏幕上输出 R。
 b. 扩展 a 的例程, 改为每次回显一行。例如, 用户输入:
 The quick brown fox jumps over the lazy dog.
 程序等待用户按下 Enter 键 (其 ASCII 码为 x0A), 然后输出同样的一行。
- 6.13** 将一个数与自身相加, 就能使其左移一位。例如, 二进制数 0011 与自身相加, 结果为 0110。右移一位却不那么容易。请设计 LC-3 机器码例程, 将 x3100 的内容右移一位。
- 6.14** 考虑下面的机器语言程序:

x3000	0101	0100	1010	0000
x3001	0001	0010	0111	1111
x3002	0001	0010	0111	1111
x3003	0001	0010	0111	1111
x3004	0000	1000	0000	0010
x3005	0001	0100	1010	0001
x3006	0000	1111	1111	1010
x3007	1111	0000	0010	0101

哪些 R1 初值会使 R2 的最终值为 3?

- 6.15** 下表给出 x3010 处的 LC-3 指令执行前后的存储器及寄存器内容。请确定 x3010 中的指令。注意: 所给信息足以唯一确定它。

	Before	After
R0:	x3208	x3208
R1:	x2d7c	x2d7c
R2:	xe373	xe373
R3:	x2053	x2053
R4:	x33ff	x33ff
R5:	x3f1f	x3f1f
R6:	xf4a2	xf4a2
R7:	x5220	x5220
...		
x3400:	x3001	x3001
x3401:	x7a00	x7a00
x3402:	x7a2b	x7a2b
x3403:	xa700	xa700
x3404:	xf011	xf011
x3405:	x2003	x2003
x3406:	x31ba	xe373
x3407:	xc100	xc100
x3408:	xefef	xefef
...		

6.16 一个 LC-3 程序位于 x3000 至 x3006，并从 x3000 开始执行。记录执行过程中装入 MAR 的所有值，可以得到一个以下列数值开头的序列，称为跟踪记录。

MAR Trace

x3000
x3005
x3001
x3002
x3006
x4001
x3003
x0021

下面给出了 x3000 至 x3006 中的部分位。请在每个空白处填入适当的 0 或 1。

x3000	0	0	1	0	0	0	0								
x3001	0	0	0	1	0	0	0	0	0	1	0	0	0	0	1
x3002	1	0	1	1	0	0	0								
x3003															
x3004	1	1	1	1	0	0	0	0	0	1	0	0	1	0	1
x3005	0	0	0	0	0	0	0	0	0	1	1	0	0	0	0
x3006															

- 6.17** 下表给出 x3210 处的 LC-3 指令执行前后的寄存器内容。请确定 x3210 中的指令。注意：所给信息足以唯一确定它。

	Before	After
R0:	xFF1D	xFF1D
R1:	x301C	x301C
R2:	x2F11	x2F11
R3:	x5321	x5321
R4:	x331F	x331F
R5:	x1F22	x1F22
R6:	x01FF	x01FF
R7:	x341F	x3211
PC:	x3210	x3220
N:	0	0
Z:	1	1
P:	0	0

- 6.18** LC-3 没有除法指令。程序员若需将两个数相除，必须编写例程。展示两个正整数相除的系统化分解，并编写从 x3000 开始的 LC-3 机器语言程序：以 x4000 中的数除以 x4001 中的数，把商存入 x5000，余数存入 x5001。
- 6.19** 常常需要将消息加密，防止他人窥视。消息可表示为存放在连续存储位置中的 ASCII 字符串，每个位置一个字符；每个位置的位 [15:8] 为 0，字符串紧接着的下一个位置为 x0000。

一位没上过本课程的学生写了下面的 LC-3 机器语言程序：将从 x4000 开始的消息中的每个字符加 4，将加密结果存放到从 x5000 开始的位置。例如，x4000 处的消息是 “Matt”，x5000 处的加密消息就是 “Qeyy”。然而代码中有四个错误。请找出并修正，使程序正确工作。

x3000	1110	0000	0000	1010
x3001	0010	0010	0000	1010
x3002	0110	0100	0000	0000
x3003	0000	0100	0000	0101
x3004	0001	0100	1010	0101
x3005	0111	0100	0100	0000
x3006	0001	0000	0010	0001
x3007	0001	0010	0110	0001
x3008	0000	1001	1111	1001
x3009	0110	0100	0100	0000
x300A	1111	0000	0010	0101
x300B	0100	0000	0000	0000
x300C	0101	0000	0000	0000

- 6.20** 重做习题 6.18，使程序适用于所有整数，而不仅仅是正整数。
- 6.21** 你受托设计音响的音量控制系统。用户通过 Volume Up 和 Volume Down 按钮控制音量：按前者增加 1，按后者减少 1。音量由 4 位无符号数表示，范围为

0 至 15。已经达到最大值 15 时再按 Volume Up, 应保持 15; 已经达到最小值 0 时再按 Volume Down, 应保持 0。存储位置 x3100 已直接连接到扬声器: 读取它的位 [3:0] 可以取得当前音量, 写入这些位则设置新的音量。

按下任一音量按钮后, 音响硬件将处理器 PC 重设为 x3000 并开始执行。若按的是 Volume Up, x3101 被设为 1; 若按的是 Volume Down, 则被设为 0。

下面是控制音量的程序, 其中缺了两条指令。请补全, 使程序按上述要求正确控制音量。

Address	Contents	Description
x3000	0010000011111111	$R0 \leftarrow M[x3100]$
x3001	0010001011111111	$R1 \leftarrow M[x3101]$
x3002	0000010000000100	Branch to x3007 if Z is set
x3003		
x3004	0000010000000101	Branch to x300A if Z is set
x3005	0001000000100001	$R0 \leftarrow R0 + x0001$
x3006	0000111000000011	Branch always to x300A
x3007	0001001000100000	$R1 \leftarrow R0 + x0000$
x3008	0000010000000001	Branch to x300A if Z is set
x3009		
x300A	0011000011110101	$M[x3100] \leftarrow R0$
x300B	1111000000100101	TRAP x25

- 6.22** [★] 一个仓库由具有 n 位数字密码的电子锁控制。锁面板有标为 0 至 9 的十个按钮。用户依次按 n 个按钮, 相应 ASCII 字符被装入从 x3150 开始的连续存储位置; 按完后, 下一个位置被写入空字符 x00。下列程序比较输入的密码与从 x3100 开始的 n 个存储位置中的密码, 决定是否开锁。应开锁时, 在 x3050 存入 1; 否则存入 0。程序缺少部分指令, 请补全。

x3000	0101 101 101 1 00000	; R5 ← x0000
x3001	0010 000 000001111	; R0 ← M[x3011]
x3002	0010 001 000001101	; R1 ← M[x3010]
x3003	0110 010 000 000000	; R2 ← M[R0]
x3004		
x3005	0110 011 001 000000	; R3 ← M[R1]
x3006	1001 011 011 111111	; NOT R3
x3007	0001 011 011 1 00001	; R3 ← R3 + 1
x3008		
x3009	0000 101 000000100	; Branch to x300E if N or P is set
x300A		
x300B		
x300C	0000 111 111110110	; Branch always to x3003
x300D		
x300E	0011 101 001000001	; Store R5 in x3050
x300F	1111 0000 0010 0101	; HALT
x3010	0011 0001 0000 0000	; x3100
x3011	0011 0001 0101 0000	; x3150

简单修改存储器中的内容，就可以省去 x3006 和 x3007 的指令。应作什么修改？

6.23 [*] 将 PC 装入 x3000，执行该地址的指令，随后再执行四条指令。下表给出这五条指令各自执行结束时若干寄存器的内容。请补全表格。

	PC	MAR	MDR	IR	R0	R1
Before execution starts	x3000	——	——	——	x0000	x0000
After the first finishes			xB333	x2005		
After the 2nd finishes				x0601		
After the 3rd finishes			x1---			x0001
After the 4th finishes			x1---		x6666	
After the 5th finishes				x0BFC		

现在重新从 PC = x3000 开始执行，先将 R0 和 R1 重新初始化为 0，并在 x3004 设置断点。按 RUN 十一次，每次程序都执行到断点为止。R0 和 R1 的最终值分别是多少？

6.24 一位学生正在调试程序。他的程序不能访问 x0000 至 x2FFF；原因将在本书后面讨论，术语是“特权存储器”，现在不必担心这个问题。

他在 x3050 设置断点，然后运行程序。停止后，他检查若干存储位置和寄存器，再执行一次单步。模拟器执行一条指令后停下，他再次检查，结果如下：

	Before	After
PC	x3050	x3051
R0	x2F5F	xFFFF
R1	x4200	x4200
R2	x0123	x0123
R3	x2323	x2323
R4	x0010	x0010
R5	x0000	x0000
R6	x1000	x1000
R7	x0522	x0522
M[x3050]	x6???	x6???
M[x4200]	x5555	x5555
M[x4201]	xFFFF	xFFFF

请补全 x3050 的内容。

0	1	1	0												
---	---	---	---	--	--	--	--	--	--	--	--	--	--	--	--

6.25 一位学生需要将 R2 的内容减去 R1，并把结果放入 R3。她没有写成：

```
NOT R3, R1
ADD R3, R3, #1
ADD R3, R3, R2
```

而是写成：

```
NOT R3, R1
.FILL x16E1
ADD R3, R3, R2
```

她汇编并尝试执行程序。这段减法能否正确执行？为什么？

6.26 [★] 某条 LC-3 指令从时钟周期 T 开始执行，共需 15 个周期。下表列出了处理该指令时用到总线的全部五个时钟周期，并给出周期、状态机状态、总线上的值，以及该周期有效的重要控制信号。

Cycle	State	Bus	Important Control Signals for This Cycle
T	18	x3010	LD.MAR = 1, LD.PC = 1, PCMux = PC + 1, GatePC = 1
T + 4			
T + 6		x3013	
T + 10		x4567	
T + 14		x0000	LD.REG = 1, LD.CC = 1, GateMDR = 1, DR = 001

a. 补全表格。

- b. 正在处理的是哪条指令?
- c. 该指令位于存储器的什么位置?
- d. 存储器完成一次读或写需要几个时钟周期?
- e. 所给信息足以确定三个存储位置的内容。它们的地址和内容分别是什么?

6.27 [★] 一个 LC-3 程序从 x3000 开始执行。下表是在程序执行前, 以及第 1、2、3、4、5、6 条指令各自执行后, 对全部八个寄存器取得的快照。

Registers	Initial Value	After 1st Instruction	After 2nd Instruction	After 3rd Instruction	After 4th Instruction	After 5th Instruction	After 6th Instruction
R0	x4006	x4050	x4050	x4050	x4050	x4050	x4050
R1	x5009	x5009	x5009	x5009	x5009	x5009	x5009
R2	x4008	x4008	x4008	x4008	x4008	x4008	xC055
R3	x4002			x8005	x8005	x8005	x8005
R4	x4003	x4003	x4003	x4003			x4003
R5	x400D	x400D			x400D	x400D	x400D
R6	x400C	x400C	x400C	x400C	x400C	x400C	x400C
R7	x6001	x6001	x6001	x6001			x400E

执行过程中还记录了 PC、MAR 和 MDR 的跟踪记录, 如下。注意, PC 跟踪记录按执行顺序记录程序执行过的指令地址。请补全这三张表中的空项。

PC Trace
x400D
x400E

MAR Trace	MDR Trace
	xA009
x3025	
	x1703
	x4040
x400E	x1403

第七章 汇编语言

到这里，你大概已经有点厌倦了 1 和 0，也厌倦了记住 0001 表示 ADD、1001 表示 NOT。要是能用有意义的符号名称来指代存储单元，而不用记住它的 16 位地址，岂不是很好？要是能用更容易理解的方式表示每条指令，而不用时刻记住指令的哪一位传递了哪项信息，岂不是也很好？这样的帮手就要来了。

本章介绍**汇编语言**（assembly language）：它不仅能做到上述这些，还能做更多事情。

7.1 汇编语言程序设计——向上一大步

回想第 1 章图 1.9 中各个转换层次。算法要转化为用某种机械语言描述的程序。这种机械语言可以像第 5 章那样，是某台计算机的机器语言。请记住，如果程序的每条指令都来自某台计算机的 ISA，那么这个程序就是用该计算机的机器语言编写的。

不过，机械语言也可以更便于人们使用。我们通常把机械语言分为高级语言和低级语言两类，其中高级语言要友好得多。C、C++、Java、Fortran、COBOL、Python 都是高级语言，除此之外还有一千多种。高级语言中的指令几乎——但并不完全——像英语等自然语言中的语句。高级语言通常与 ISA 无关。也就是说，学会了针对一种 ISA 用 C（或 Fortran、Python）编程，再针对另一种 ISA 使用同一种语言编程，只需跨出很小的一步。

高级语言程序在执行之前，必须先翻译成将在其上执行的计算机的 ISA 程序。高级语言中的一条语句往往对应多条 ISA 指令。第 11 章将介绍高级语言 C，第 12 至 19 章将说明各种 C 语句与其对应的 LC-3 代码之间的关系。不过，本章只在第 5 章讨论的 ISA 基础上向上迈一小步。

从机器的 ISA 向上迈一小步，就是该 ISA 的汇编语言。汇编语言是一种低级语言。谁也不会把低级语言中的一条指令误认为英语中的一句话。一条汇编语言指令通常指定一条 ISA 指令。高级语言通常与 ISA 无关，而低级语言则强烈依赖 ISA。事实上，一种 ISA 通常只有一种汇编语言。

汇编语言的目的是让编程比直接使用机器语言（也就是所用计算机的 ISA）更方便，同时仍让程序员能够精细地控制计算机执行的指令。例如，我们仍然可以控制计算机要执行的具体指令，却不必再记住 0001 和 1001 分别是什么操作码，也不必记住地址 0011111100001010 和 0011111100000101 分别存储什么。汇编语言允许用操作码的**助记符**，例如用 ADD 表示 0001、用 NOT 表示 1001；还允许用 SUM、PRODUCT 这样的有意义的符号名称来命名存储单元，代替它们的 16 位地址。这样就更容易分清哪个单元保存和、哪个单元保存积。这些名称称为**符号地址**（symbolic address）。

从第 11 章开始我们将看到，向 C 这样的高级语言再迈出更大的一步，编程会更

加方便；但与此同时，我们也会放弃一部分控制权，不能再精确指定究竟用哪些 ISA 指令来完成一条高级语言语句所要求的工作。

7.2 一个汇编语言程序

我们通过一个实例开始学习 LC-3 汇编语言。图 7.1 的程序用重复加法，将最初存储在 NUMBER 中的整数乘以 6。例如，如果这个整数是 123，就用 $123 + 123 + 123 + 123 + 123 + 123$ 计算乘积。你哪里见过这种做法？:-)

程序共 21 行。为了方便引用某一行，我们为各行加上了行号。这是一种常见做法，行号本身并不属于程序。十行以分号开头，表明这些行完全是写给人看的，稍后再讨论。七行（06、07、08、0C、0D、0E 和 10）指定汇编语言指令，它们将被翻译成 LC-3 机器语言指令，在程序运行时执行。其余四行（05、12、13 和 15）包含**伪操作**（pseudo-op），即程序员传给翻译程序、帮助它完成翻译的信息。这个翻译程序称为**汇编器**（assembler），这里是 LC-3 汇编器；翻译过程称为**汇编**（assembly）。

```

01 ;
02 ; Program to multiply an integer by the constant 6.
03 ; Before execution, an integer must be stored in NUMBER.
04 ;
05         .ORIG      x3050
06         LD         R1, SIX
07         LD         R2, NUMBER
08         AND        R3, R3, #0           ; Clear R3. It will
09                                           ; contain the product.
0A ; The inner loop
0B ;
0C AGAIN  ADD        R3, R3, R2
0D         ADD        R1, R1, #-1         ; R1 keeps track of
0E         BRp        AGAIN              ; the iterations
0F ;
10         HALT
11 ;
12 NUMBER .BLKW      1
13 SIX    .FILL      x0006
14 ;
15         .END

```

图 7.1 一个汇编语言程序

7.2.1 指令

LC-3 ISA 中的一条指令由 16 个 0 和 1 组成；汇编语言中的一条指令则由下面四部分组成：

Label Opcode Operands ; Comment

其中 Label（标号）和 Comment（注释）是可选的，稍后讨论。

7.2.1.1 操作码和操作数

Opcode（操作码）和 Operands（操作数）这两部分是必需的。一条汇编语言指令要对应 LC-3 ISA 中的一条指令，就必须有操作码（指令要做什么）以及适当数量的操作数（对什么做这件事）。这恰好就是第 5 章学习 LC-3 ISA 时遇到的内容。

操作码是对应 LC-3 指令操作码的符号名称。其想法在于，用 ADD、AND 或 LDR 这样的符号名称记住一种操作，比记住 0001、0101 或 0110 这样的 4 位数容易。图 5.3（以及图 A.2）列出了 LC-3 的 15 条指令的操作码。原书第 658 至 673 页（附录 A）给出了这 15 条指令的汇编语言表示。

操作数的数量取决于所执行的操作。例如，图 7.1 程序第 0C 行的 ADD 指令需要三个操作数：两个源操作数提供待相加的数，一个目的操作数指定结果存放的位置。三个操作数都必须在指令中明确指定。

```
AGAIN ADD R3, R3, R2
```

这里，待相加的操作数取自寄存器 2 和寄存器 3，结果放入寄存器 3。我们将寄存器 0 至 7 分别表示为 R0、R1、R2、...、R7，而不再写成 000、001、010、...、111。

图 7.1 第 07 行的 LD 指令需要两个操作数：从哪个存储单元读取值，以及指令执行完后用哪个目的寄存器保存这个值。LC-3 汇编语言用称为**标号**（label）的符号名称命名存储单元，使我们不必记住具体的 16 位地址。这里，要读取的存储单元的标号是 NUMBER，目的位置（也就是值要装入的位置）是寄存器 2。

```
LD R2, NUMBER
```

正如 5.1.6 节所述，操作数可以来自寄存器、存储器，也可以是指令中的字面量（即立即数）。寄存器操作数直接用寄存器名称表示，例如第 0C 行的 R2 和 R3；存储器操作数直接用存储单元的符号名称表示，例如第 07 行的 NUMBER 和第 06 行的 SIX；立即数操作数则直接写出实际的值，例如第 08 行的 0。

```
AND R3, R3, #0 ; Clear R3. It will contain the product.
```

字面量必须带有标明进制的符号：# 表示十进制，x 表示十六进制，b 表示二进制。有时并无歧义，例如 3F0A 显然是十六进制数，但仍要写成 x3F0A。有时则有歧义，例如 1000：x1000 表示十进制的 4096，b1000 表示十进制的 8，#1000 表示十进制的 1000。

7.2.1.2 标号

标号是用于标识程序中明确引用的存储单元的符号名称。在 LC-3 汇编语言中，一个标号由 1 至 20 个字母或数字组成，即每个字符都是大写或小写英文字母，或者十进制数字；第一个字符必须是字母。

不过，并非所有符合这些规则的字符序列都可以作为标号。你知道，计算机程序不能容忍歧义。因此，ADD、NOT、x1000、R4 以及其他在 LC-3 程序中具有特定

含义的字符串，都不能用作标号；否则，LC-3 汇编器在把汇编语言程序翻译成 LC-3 ISA 程序时就可能产生混淆。这些不允许用作标号的字符串通常称为**保留字**（reserved word）。

NOW、Under21、R2D2、R785 和 C3PO 都是合法的 LC-3 汇编语言标号。

前面说过，如果要在程序中明确引用一个存储单元，就为它指定标号，也就是符号名称。明确引用某个存储单元有两个原因：

1. 该单元是分支指令的目标，例如第 0C 行的 AGAIN。标号 AGAIN 标识的，是分支发生跳转后下一条要执行的指令的位置。
2. 该单元包含需要装入或存储的值，例如第 12 行的 NUMBER 和第 13 行的 SIX。

注意，第 0E 行的分支指令明确引用了第 0C 行标识的位置 AGAIN。

```
BRp AGAIN
```

如果 ADD R1, R1, #-1 的结果为正（因而 P 位被置位），程序就跳转到明确引用的位置 AGAIN，再进行一次迭代。

第 07 行的装入指令明确引用了 NUMBER，把该存储单元中的值装入 R2。

程序中没有被明确引用的位置，就不必设置标号。

7.2.1.3 注释

注释（comment）是专门写给人看的信息。它们不影响翻译过程，LC-3 汇编器实际上也不会处理它们。程序用分号标识注释：分号表示该行其余部分是注释，汇编器应当忽略。如果分号是该行第一个非空白字符，那么整行都会被忽略；如果分号位于指令的操作数之后，则只忽略注释部分。

注释的目的是让人更容易理解程序，帮助解释一条或一组指令中不那么直观的地方。第 08、09 行的注释“Clear R3; it will contain the product”告诉读者，第 08 行是在累加两个数的乘积之前初始化 R3。对于程序员来说，今天第 08 行的目的可能显而易见；但两年后又写了 30,000 条指令时，可能就记不起当初为什么写 AND R3, R3, #0 了。也可能两年后这位程序员已经离开公司，而公司为了产品更新需要修改程序。如果任务交给一个从没见过这段程序的人，注释会大大帮助他理解程序。

注释应当提供额外的解释，而不只是重复显而易见的事实。这有两个原因。首先，重复显而易见的内容是在浪费所有人的时间；其次，这些注释会让程序显得杂乱，反而掩盖真正重要的注释。例如，在第 0D 行写上“Decrement R1”就不是个好主意：它没有为这条指令提供任何额外解释，只会增加页面上的杂乱内容。

注释还有一个用途，就是使程序的视觉组织更易理解。也就是说，可以用注释把程序的不同部分分隔开来，提高可读性。共同计算一个结果的代码放在连续的几行，而与其他代码之间用空行隔开。例如，第 0C 至 0E 行共同构成程序核心的循环体，它们由第 0B 行和第 0F 行与其余代码隔开。这两行除了第一列的分号之外，什么也没有。

顺便说一句，恰当地使用空白也能使程序更易阅读：在一行中添加额外的空格，汇编器会忽略这些空格。例如，不论指令有没有标号，都让操作码从页面的同一列开始。

7.2.2 伪操作（汇编指示）

LC-3 汇编器接收一串字符作为输入，这串字符表示一个 LC-3 汇编语言程序；汇编器将其翻译成 LC-3 ISA 程序。伪操作帮助汇编器完成这项任务。

伪操作更正式的名称是**汇编指示**（*assembler directive*）。之所以称为“伪”操作，是因为正如其希腊语词根 *pseudes*（意为“假的”）所暗示的，它并不指程序执行期间将要完成的操作。它完全是汇编语言程序员传给汇编器、帮助汇编过程进行的信息。汇编器处理了这条信息后，就丢弃这条伪操作。LC-3 汇编语言包含五种对我们编程有用的伪操作：`.ORIG`、`.FILL`、`.BLKW`、`.STRINGZ` 和 `.END`。它们都以点号开头，很容易辨认。

7.2.2.1 .ORIG

`.ORIG` 告诉汇编器应将 LC-3 程序放在存储器的什么位置。第 05 行的 `.ORIG x3050` 表示将第一条 LC-3 ISA 指令放在地址 `x3050`。因此，`0010001000000111`（`LD R1, SIX` 翻译后的指令）被放入 `x3050`，其余翻译后的 LC-3 程序依次放入随后的连续存储单元。¹例如，若程序包含 `x100` 条 LC-3 指令，且 `.ORIG` 指定第一条指令放在 `x3050`，则其余 `xFF` 条指令放在 `x3051` 至 `x314F`。

7.2.2.2 .FILL

`.FILL` 告诉汇编器，为程序保留下一个存储单元，并用操作数的值将其初始化。这个值可以是一个数，也可以是一个标号。第 13 行将生成的 LC-3 程序中的第九个存储单元初始化为 `x0006`。

7.2.2.3 .BLKW

`.BLKW` 告诉汇编器，在程序中保留若干连续的存储单元，即一个字块（*BLock of Words*）。具体数量由这条伪操作的操作数指定。第 12 行要求汇编器保留一个存储单元，并将其标为 `NUMBER`。

当操作数的实际值尚未确定时，`.BLKW` 尤其有用。例子中，我们假设 `NUMBER` 中的数是 123。这个数是怎么放进去的？`.BLKW` 的一种常见用法，就是像这里一样先在程序中保留一个位置，再由另一段代码生成所需的数。例如，这个数可能来自键盘输入，编写程序时不可能知道。另一段代码会在执行图 7.1 的代码之前，把这个值存入 `NUMBER`。

¹译注：原文此处写作 `0010001000001100`，与图 7.1 不符。`SIX` 位于 `x3058`，自增后的 PC 为 `x3051`，偏移量为 7，因此机器码应为 `x2207`，即 `0010001000000111`。

7.2.2.4 .STRINGZ

.STRINGZ 告诉汇编器，初始化连续的 $n+1$ 个存储单元。其参数是双引号内的 n 个字符。前 n 个存储字用字符串中相应字符的 ASCII 码零扩展后的值初始化，最后一个字初始化为 0。包含 x0000 的最后一个字，为处理这串 ASCII 码提供了方便的哨兵。

例如，下面的代码片段：

```
.ORIG x3010
HELLO .STRINGZ "Hello, World!"
```

会使汇编器将地址 x3010 至 x301D 初始化为以下值：

x3010:	x0048	x3017:	x0057
x3011:	x0065	x3018:	x006F
x3012:	x006C	x3019:	x0072
x3013:	x006C	x301A:	x006C
x3014:	x006F	x301B:	x0064
x3015:	x002C	x301C:	x0021
x3016:	x0020	x301D:	x0000

7.2.2.5 .END

.END 告诉汇编器，已经到达程序末尾，后面的内容不必再看。也就是说，.END 之后的任何字符都不会由汇编器处理。注意：.END 不会在执行期间停止程序。事实上，执行时根本不存在 .END。它只是一个分隔标记，用来标明程序结束，是程序员告诉汇编器“汇编语言程序到此结束”的信息。

7.2.3 实例：再次考察 5.5 节的字符计数程序

现在可以来看一个完整实例了。再次考虑 5.5 节的问题：编写一个程序，从键盘输入一个字符，统计该字符在一个文件中出现的次数。和以前一样，我们先通过构建流程图来设计算法。回想 6.1 节，我们展示了如何系统地分解这个问题，得到图 5.16 的流程图。事实上，第 6 章这一过程的最后一步就是图 6.3e，它与图 5.16 基本相同。接着，根据流程图编写实际程序。不过，这一次我们可以享受不必操心 0 和 1 的便利，直接用 LC-3 汇编语言编写。程序见图 7.2。

关于这个程序，有几点要说明。程序有三次需要以服务调用的形式请求操作系统帮助，每次都使用 TRAP 指令。TRAP x23 从键盘输入一个字符并放入 R0（第 0D 行）；TRAP x21 将 R0 中 ASCII 码对应的字符显示在显示器上（第 28 行）；TRAP x25 使机器停机（第 29 行）。正如前面所说，TRAP 指令的具体执行方式留到第 9 章讨论。

十进制数字 0 至 9（二进制 0000 至 1001）的 ASCII 码是 x30 至 x39。只要给数字的二进制值加上 x30，就能转换为 ASCII 码。第 2D 行用标号 ASCII 标识包含

```

01 ;
02 ; Program to count occurrences of a character in a file.
03 ; Character to be input from the keyboard.
04 ; Result to be displayed on the monitor.
05 ; Program works only if no more than 9 occurrences are found.
06 ;
07 ;
08 ; Initialization
09 ;
0A .ORIG x3000
0B AND R2,R2,#0 ; R2 is counter, initialize to 0
0C LD R3,PTR ; R3 is pointer to characters
0D TRAP x23 ; R0 gets character input
0E LDR R1,R3,#0 ; R1 gets the next character
0F ;
10 ; Test character for end of file
11 ;
12 ;
13 TEST ADD R4,R1,#-4 ; Test for EOT
14 BRz OUTPUT ; If done, prepare the output
15 ;
16 ; Test character for match. If a match, increment count.
17 ;
18 NOT R1,R1
19 ADD R1,R1,#1 ; R1 <-- -R1
1A ADD R1,R1,R0 ; R1 <-- R0-R1. If R1=0, a match!
1B BRnp GETCHAR ; If no match, do not increment
1C ADD R2,R2,#1
1D ;
1E ; Get next character from the file
1F ;
20 GETCHAR ADD R3,R3,#1 ; Increment the pointer
21 LDR R1,R3,#0 ; R1 gets the next character to test
22 BRnzp TEST
23 ;
24 ; Output the count.
25 ;
26 OUTPUT LD R0,ASCII ; Load the ASCII template
27 ADD R0,R0,R2 ; Convert binary to ASCII
28 TRAP x21 ; ASCII code in R0 is displayed
29 TRAP x25 ; Halt machine
2A ;
2B ; Storage for pointer and ASCII template
2C ;
2D ASCII .FILL x0030
2E PTR .FILL x4000
2F .END

```

图 7.2 统计字符出现次数的汇编语言程序

x0030 的存储单元。第 26 行的 LD 指令利用这个标号把 x30 装入 R0，以便把 R2 中的二进制计数值转换为 ASCII 码。第 27 行的 ADD 指令完成转换，第 28 行的 TRAP x21 将对应字符输出到显示器。

要检查的文件从地址 x4000 开始（见第 2E 行）。通常，编写这段程序的人并不知道起始地址，因为我们希望程序能处理许多文件，而不只是从 x4000 开始的那个文件。为此，可以将第 2E 行改成 .BLKW 1，在执行图 7.2 的程序之前，由另一段知道目标文件起始地址的代码填写这个单元。7.4 节将讨论这种情况。

7.3 汇编过程

7.3.1 引言

LC-3 汇编语言程序在执行前，必须先翻译成机器语言程序，也就是每条指令都属于 LC-3 ISA 的程序。LC-3 汇编器的任务就是完成这种翻译。

如果已经有 LC-3 汇编器，就可以通过执行适当的命令，让它把汇编语言程序翻译成机器语言程序。在本书所述的、可通过网络获取的 LC-3 汇编器中，这个命令是 assemble，其参数需要包括汇编语言程序的文件名。例如，文件名为 solution1.asm 时，命令

```
assemble solution1.asm outfile
```

生成的 outfile 文件包含 LC-3 ISA 程序。应向任课教师确认正确的命令行用法，以便让所用的 LC-3 汇编器生成由 0 和 1 构成的 LC-3 ISA 文件。

7.3.2 两遍扫描

本节考察汇编器如何把汇编语言程序翻译成机器语言程序，输入采用图 7.2 的汇编语言程序。

你还记得，汇编语言程序中的指令与最终机器语言程序中的指令通常一一对应。我们可以尝试只扫描一遍程序就完成翻译。从图 7.2 顶部开始，汇编器丢弃第 01 至 09 行，因为它们只有注释。注释完全是写给人看的，与翻译过程无关。接下来是第 0A 行：这是一条伪操作，告诉汇编器机器语言程序从地址 x3000 开始。然后来到第 0B 行，很容易把它翻译成 LC-3 机器码。此时得到：

```
x3000: 0101010010100000
```

汇编器继续翻译下一条指令，即第 0C 行。不幸的是，它还不知道符号地址 PTR 的含义，无法翻译。汇编器到这里就卡住了，汇编过程失败。

为避免这种情况，汇编过程要从开头到 .END 完整地扫描整个汇编语言程序两遍。第一遍的目标是找出各个符号名称（或标号）对应的实际二进制地址。这组对应关系称为**符号表**（symbol table）。第一遍建立符号表，第二遍把各条汇编语言指令翻译成相应的机器语言指令。

这样，第二遍检查第 0C 行并准备翻译时，汇编器已经从第一遍得知 PTR 是存储单元 x3013 的符号地址，就可以轻松地把第 0C 行翻译成：

```
x3001: 0010011000010001
```

不知道 PTR 对应哪个 16 位地址的问题就不存在了。

7.3.3 第一遍：建立符号表

对我们来说，符号表就是符号名称与其 16 位存储器地址的对应关系。扫描一遍汇编语言程序，记录每条指令被分配到哪个存储单元，并把每个标号与其所在条目的存储器地址对应起来，就能得到这些关系。

回想一下，之所以设置标号，是因为要引用某个位置：它要么是分支指令的目标，要么包含必须装入或存储的数据。因此，只要没有编程错误，找出了所有标号，就找出了程序使用的所有符号地址。

上一段假定整个程序都位于一对 .ORIG 和 .END 伪操作之间。图 7.2 的程序确实如此。7.4 节将考虑由多个部分组成的程序，每一部分都有自己的 .ORIG 和 .END，并分别汇编。

第一遍开始时先丢弃第 01 至 09 行的注释，然后根据第 0A 行得知，第一条指令将被分配到地址 x3000。我们用位置计数器（location counter，LC）跟踪分配给每条指令的位置。LC 被初始化为 .ORIG 指定的地址，这里是 x3000。

汇编器依次检查各条指令，每遇到一条汇编语言指令，就将 LC 加 1。如果正在检查的指令带有标号，就为它建立一个符号表条目，以 LC 的当前值作为其地址。遇到 .END 时，第一遍结束。

第一条带标号的指令在第 13 行。它是程序的第五条指令，此时 LC 为 x3004，所以建立如下条目：

符号	地址
TEST	x3004

第二条带标号的指令在第 20 行，此时 LC 已增加到 x300B，于是建立以下条目：

符号	地址
GETCHAR	x300B

第一遍结束时，符号表包含以下条目：

符号	地址
TEST	x3004
GETCHAR	x300B
OUTPUT	x300E
ASCII	x3012
PTR	x3013

7.3.4 第二遍：生成机器语言程序

第二遍再次逐行扫描汇编语言程序，这次有了符号表的帮助。每到一行，就将其中的汇编语言指令翻译成 LC-3 机器语言指令。

汇编器又从顶部开始，丢弃仅含注释的第 01 至 09 行。第 0A 行的 .ORIG 用来把 LC 初始化为 $\times 3000$ 。随后，第 0B 行生成机器语言指令 0101010010100000，再来到第 0C 行。

这一次，汇编器知道 PTR 对应 $\times 3013$ ，可以完整地汇编第 0C 行。指令是 LD，操作码编码为 0010；目的寄存器 DR 是 R3，编码为 011。

LD 指令剩下要确定的部分只有 PCoffset。计算方法如下：汇编器知道 PTR 是地址 $\times 3013$ 的标号，而自增后的 PC 为 $LC + 1$ ，这里是 $\times 3002$ 。PTR ($\times 3013$) 必须等于自增后的 PC ($\times 3002$) 与符号扩展后的 PCoffset 之和，所以 PCoffset 必须为 $\times 0011$ 。将各部分合起来，汇编器把地址 $\times 3001$ 的内容设为 0010011000010001，并把 LC 增加到 $\times 3002$ 。

注意，要使用 LD 指令，装入源（这里是标号 PTR 对应的地址）相对于 LD 指令本身，不能超过向前 +256 或向后 -255 个存储单元。如果 PTR 的地址大于 $LC + 1 + 255$ ，或者小于 $LC + 1 - 256$ ，偏移量就无法放入指令的位 [8:0]。这种情况下会发生汇编错误，使汇编不能成功完成。幸好 PTR 离 LD 指令足够近，所以这条指令能正确汇编。

第二遍继续进行。每一步都按 LC 指示的位置放入翻译得到的 LC-3 指令，或者在遇到 .FILL 时放入所指定的值，并相应推进 LC。第二遍遇到 .END 时，汇编结束。

翻译得到的程序见图 7.3。²

即使一切顺利，这个过程也够枯燥的。幸好，你不必靠手工做这件事来谋生，LC-3 汇编器会代劳。而且，既然已经掌握 LC-3 汇编语言，就没有必要再用机器语言编程了。现在可以用 LC-3 汇编语言的符号形式编写程序，再调用汇编器生成能在 LC-3 计算机上执行的机器语言版本。

7.4 单个汇编语言程序之外

本章的目的是带你从计算机的 ISA 再向上迈一步，认识汇编语言。尽管它离 C 或 C++ 还有很大距离，但确实能为我们省去许多麻烦。我们还展示了一个基本的两遍扫描汇编器如何把汇编语言程序翻译成 LC-3 ISA 的机器语言。

复杂的汇编语言程序设计还有许多方面，远远超出了入门课程的范围。不过，我们教授汇编语言，是为了展示它本身的简单，而不是探讨复杂技巧。在结束本章之前，还有几个要点值得了解。

²译注：此图按约定保留原图，其中两行机器码有误。按图 7.2，第 19 行 ADD R1, R1, #1 位于 $\times 3007$ ，应编码为 0001001001100001 ($\times 1261$)；第 1A 行 ADD R1, R0, R1 位于 $\times 3008$ ，应编码为 0001001000000001 ($\times 1201$)。原图分别写成了 $\times 1240$ 和 $\times 927F$ 。

Address	Binary
	0011000000000000
x3000	0101010010100000
x3001	0010011000010001
x3002	1111000000100011
x3003	0110001011000000
x3004	0001100001111100
x3005	0000010000001000
x3006	1001001001111111
x3007	0001001001000000
x3008	1001001001111111
x3009	0000101000000001
x300A	0001010010100001
x300B	0001011011100001
x300C	0110001011000000
x300D	0000111111110110
x300E	0010000000000011
x300F	0001000000000010
x3010	1111000000100001
x3011	1111000000100101
x3012	0000000000110000
x3013	0100000000000000

图 7.3 图 7.2 汇编语言程序对应的机器语言程序

7.4.1 可执行映像

计算机开始执行程序时，所执行的实体称为**可执行映像**（executable image）。它由若干模块构成，这些模块往往由不同的程序员独立编写。每个模块分别翻译成一个**目标文件**（object file）。前面我们模仿 LC-3 汇编器，已经亲自经历了这种翻译过程。其他模块可能用 C 编写，由 C 编译器翻译。有些模块由用户编写，有些则以库例程的形式由操作系统提供。每个目标文件都由所用计算机 ISA 中的指令及其相关数据组成。最后一步，是将所有目标模块组合，即**链接**（link），成为一个可执行映像。程序执行期间，取指令、译码等指令周期阶段作用于可执行映像中的指令。

7.4.2 多个目标文件

用多个目标文件构成一个可执行映像非常常见。事实上，在现实中，大多数程序既会调用操作系统提供的库，也会调用其他程序员生成的模块，因此多个目标文件比单个目标文件常见得多。

字符计数程序就是一个例子。它统计一个字符在文件中出现的次数。典型应用很可能将程序作为一个模块，输入数据文件作为另一个模块。这样，图 7.2 第 2E 行中写作 x4000 的文件起始地址，在编写程序时就不知道。如果把第 2E 行改为：

```
PTR .FILL STARTofFILE
```

图 7.2 的程序将无法汇编，因为没有 STARTofFILE 的符号表条目。怎么办？

假如 LC-3 汇编语言包含伪操作 `.EXTERNAL`，我们就可以指出：STARTofFILE 是某个地址的符号名称，但在汇编图 7.2 的程序时还不知道该地址。可以用下面这一行来表达：

```
.EXTERNAL STARTofFILE
```

它会告诉 LC-3 汇编器，缺少标号 STARTofFILE 并不是程序错误；这个标号位于另一个将被独立翻译的模块中。在本例中，它就是字符计数程序要检查的文件中第一个字符所在位置的标号。

假如 LC-3 汇编语言有 `.EXTERNAL` 伪操作，而且我们已经将 STARTofFILE 声明为 `.EXTERNAL`，汇编器就能为它建立符号表条目，但不分配地址，而是将它标记为属于另一个模块。在所有模块组合起来的链接阶段，**链接器**（linker，即管理这一组合过程的程序）会利用另一个模块中 STARTofFILE 的符号表条目，完成对修改后的第 2E 行的翻译。

这样，`.EXTERNAL` 伪操作就使一个模块可以引用另一个模块中的符号位置，最终由链接器完成正确的地址解析。

习题

- 7.1** 一个汇编语言程序包含以下两条语句。汇编器将下面 LDI 指令翻译后的版本放在目标模块的地址 `x3025`。汇编完成后，`x3025` 中是什么？

```
PLACE .FILL x45A7
      LDI R3, PLACE
```

- 7.2** 一个 LC-3 汇编语言程序包含指令：

```
ASCII LD R1, ASCII
```

符号表中 ASCII 对应的地址为 `x4F08`。如果程序运行时执行了这条指令，那么刚执行完时 R1 中是什么？

- 7.3** 用字符串 AND 作为标号有什么问题？

- 7.4** 为下面的例程建立符号表条目，即汇编器将它翻译成机器码时生成的条目。

```

                .ORIG    x301C
                ST      R3, SAVE3
                ST      R2, SAVE2
                AND     R2, R2, #0
TEST           IN
                BRz     TEST
                ADD     R1, R0, #-10
                BRn     FINISH
                ADD     R1, R0, #-15
                NOT     R1, R1
                BRn     FINISH
                HALT
FINISH        ADD     R2, R2, #1
                HALT
SAVE3         .FILL    X0000
SAVE2         .FILL    X0000
                .END

```

7.5 a. 下面的程序做什么？

```

                .ORIG    x3000
                LD      R2, ZERO
                LD      R0, M0
                LD      R1, M1
LOOP          BRz     DONE
                ADD     R2, R2, R0
                ADD     R1, R1, -1
                BR      LOOP
DONE          ST      R2, RESULT
                HALT
RESULT        .FILL    x0000
ZERO          .FILL    x0000
M0            .FILL    x0004
M1            .FILL    x0803
                .END

```

b. 程序运行结束后，RESULT 中是什么值？

7.6 我们的汇编器崩溃了，需要你的帮助！为下面的程序建立符号表，并汇编标号 A、B 和 D 处的指令。

```

                .ORIG    x3000
                AND     R0, R0, #0
A             LD      R1, E
                AND     R2, R1, #1
                BRp     C
B             ADD     R1, R1, #-1
C             ADD     R0, R0, R1
                ADD     R1, R1, #-2
D             BRp     C
                ST      R0, F
                TRAP    x25
E             .BLKW    1
F             .BLKW    1
                .END

```

可以假定另一个模块会在本模块执行之前，向 E 存入一个正数。用不超过 15 个词说明这个程序做什么。

- 7.7** 编写一个 LC-3 汇编语言程序，统计 R0 中所存值包含多少个 1，并将结果存入 R1。例如，如果 R0 包含 0001001101110000，那么程序执行后，R1 中的结果应为 0000 0000 0000 0110。
- 7.8** 一位工程师正在调试自己编写的程序。她查看下面这段代码，决定在地址 0xA404 设置断点。她从 PC = 0xA400 开始，将所有寄存器初始化为 0，然后运行程序，直到遇到断点。

```
Code Segment:
...
0xA400 THIS1 LEA    R0, THIS1
0xA401 THIS2 LD     R1, THIS2
0xA402 THIS3 LDI    R2, THIS5
0xA403 THIS4 LDR    R3, R0, #2
0xA404 THIS5 .FILL  xA400
...
```

用十六进制写出遇到断点时寄存器组的内容。

- 7.9** .END 伪操作的用途是什么？它与 HALT 指令有什么不同？
- 7.10** 下面的程序片段有一个错误。找出错误，并说明如何修正。

```
ADD R3, R3, #30
ST R3, A
HALT
A .FILL #0
```

这个错误是在汇编代码时被发现，还是在 LC-3 上运行代码时被发现？

- 7.11** LC-3 汇编器必须能把用 ASCII 表示的常量转换成相应的二进制值。例如，x2A 转换为 00101010，#12 转换为 00001100。编写一个 LC-3 汇编语言程序，从键盘读取一个十进制或十六进制常量：前缀字符 # 表示十进制，x 表示十六进制；然后输出它的二进制表示。假定常量最多只需两位十进制数字或两位十六进制数字来表示。
- 7.12** 下面的 LC-3 程序做什么？

```

                                .ORIG  x3000
                                AND     R5, R5, #0
                                AND     R3, R3, #0
                                ADD     R3, R3, #8
                                LDI     R1, A
                                ADD     R2, R1, #0
AG      ADD     R2, R2, R2
                                ADD     R3, R3, #-1
                                BRnp    AG
                                LD      R4, B
                                AND     R1, R1, R4
                                NOT     R1, R1
                                ADD     R1, R1, #1
                                ADD     R2, R2, R1
                                BRnp    NO
                                ADD     R5, R5, #1
                                NO      HALT
                                B       .FILL  xFF00
                                A       .FILL  x4000
                                .END

```

7.13 下面的程序将存储单元 A、B 和 C 中的值相加，并把结果存入存储器。代码中有两个错误。分别说明是什么错误，以及是在汇编时还是运行时被发现。

```

Line No.
1          .ORIG  x3000
2          ONE   LD  R0, A
3          ADD R1, R1, R0
4          TWO   LD  R0, B
5          ADD R1, R1, R0
6          THREE LD  R0, C
7          ADD R1, R1, R0
8          ST  R1, SUM
9          TRAP x25
10         A     .FILL x0001
11         B     .FILL x0002
12         C     .FILL x0003
13         D     .FILL x0004
14         .END

```

7.14 a. 汇编下面的程序。

```

                                .ORIG  x3000
                                STI     R0, LABEL
                                OUT
                                HALT
                                LABEL   .STRINGZ "%"
                                .END

```

b. 程序员原本想让程序在显示器上输出一个%，然后停机。不幸的是，他混淆了各操作码的语义，即 LC-3 对各操作码究竟执行什么功能。只替换程序中的一个操作码，使程序按预期工作。

- c. 执行 a 中的原始程序时，出现了一些很奇怪的行为。一部分原因是程序中的错误，另一部分原因是程序开始执行时 R0 中的值为 x3000。说明出现了什么奇怪行为，以及为什么会这样。

7.15 下面是一个完成某种功能的 LC-3 程序。假定从地址 x4000 开始的连续存储单元中保存着一个整数序列，每个单元一个整数，以 x0000 结束。这个程序做什么？

```

                                .ORIG    x3000
                                LD        R0, NUMBERS
                                LD        R2, MASK
LOOP    LDR        R1, R0, #0
                                BRz      DONE
                                AND      R5, R1, R2
                                BRz      L1
                                BRnzp    NEXT
L1      ADD        R1, R1, R1
                                STR      R1, R0, #0
NEXT    ADD        R0, R0, #1
                                BRnzp    LOOP
DONE    HALT
NUMBERS .FILL     x4000
MASK    .FILL     x8000
.END

```

7.16 假定从地址 x4000 开始的连续存储单元中保存着一个非负整数序列，每个单元一个整数，每个整数的值都在 0 至 30,000（十进制）之间。序列以 -1，即 xFFFF，结束。

下面的程序做什么？

```

                                .ORIG    x3000
                                AND      R4, R4, #0
                                AND      R3, R3, #0
                                LD        R0, NUMBERS
LOOP    LDR        R1, R0, #0
                                NOT      R2, R1
                                BRz      DONE
                                AND      R2, R1, #1
                                BRz      L1
                                ADD      R4, R4, #1
                                BRnzp    NEXT
L1      ADD        R3, R3, #1
NEXT    ADD        R0, R0, #1
                                BRnzp    LOOP
DONE    TRAP      x25
NUMBERS .FILL     x4000
.END

```

7.17 假设你编写了两个独立的汇编语言模块，准备由链接器组合起来。每个模块都使用标号 AGAIN，而且两个模块都没有 .EXTERNAL AGAIN 伪操作。在两个模块中都使用 AGAIN 会有问题吗？为什么？

7.18 下面的 LC-3 程序比较两个长度相同的字符串。源字符串均采用 .STRINGZ 格式，第一个从地址 x4000 开始，第二个从地址 x4100 开始。如果两个字符串相同，程序结束时 R5 中的值为 1。在 (a)、(b) 和 (c) 处填入指令，完成程序。

```

                                .ORIG  x3000
                                LD      R1, FIRST
                                LD      R2, SECOND
                                AND      R0, R0, #0
LOOP
                                ----- (a)
                                LDR      R4, R2, #0
                                BRz      NEXT
                                ADD      R1, R1, #1
                                ADD      R2, R2, #1
                                ----- (b)
                                ----- (c)
                                ADD      R3, R3, R4
                                BRz      LOOP
                                AND      R5, R5, #0
                                BRnzp    DONE
NEXT
                                AND      R5, R5, #0
                                ADD      R5, R5, #1
DONE
                                TRAP     x25
FIRST
                                .FILL    x4000
SECOND
                                .FILL    x4100
                                .END

```

7.19 执行下面的 LC-3 程序时，标号 LOOP 所在地址处的指令会执行多少次？

```

                                .ORIG  x3005
                                LEA      R2, DATA
                                LDR      R4, R2, #0
LOOP
                                ADD      R4, R4, #-3
                                BRzp     LOOP
                                TRAP     x25
DATA
                                .FILL    x000B
                                .END

```

7.20 不同的程序员编写了 LC-3 汇编语言模块 (a) 和 (b)，都要将 x0015 存入地址 x4000。这两种做法有什么根本区别？

(a)

```

                                .ORIG  x5000
                                AND      R0, R0, #0
                                ADD      R0, R0, #15
                                ADD      R0, R0, #6
                                STI      R0, PTR
                                HALT
PTR
                                .FILL    x4000
                                .END

```

(b)

```

                                .ORIG  x4000
                                .FILL    x0015
                                .END

```

7.21 汇编下面的 LC-3 汇编语言程序。

```

        .ORIG    x3000
        AND      R0, R0, #0
        ADD      R2, R0, #10
        LD       R1, MASK
        LD       R3, PTR1
LOOP    LDR       R4, R3, #0
        AND      R4, R4, R1
        BRz      NEXT
        ADD      R0, R0, #1
NEXT    ADD      R3, R3, #1
        ADD      R2, R2, #-1
        BRp      LOOP
        STI      R0, PTR2
        HALT
MASK    .FILL    x8000
PTR1    .FILL    x4000
PTR2    .FILL    x5000
        .END

```

用不超过 20 个词说明程序做什么。

7.22 LC-3 汇编器必须能够把指令操作码的助记符映射为二进制操作码。例如，给定 ADD，必须生成二进制位模式 0001。编写一个 LC-3 汇编语言程序，提示用户输入一个 LC-3 汇编语言操作码，再显示相应的二进制操作码。如果输入的汇编语言操作码无效，就显示错误信息。

7.23 下面的 LC-3 程序判断一个字符串是否为回文。回文是正着读、反着读都相同的字符串，例如 racecar。假设字符串从地址 x4000 开始，以 .STRINGZ 格式存储。如果字符串是回文，程序结束时 R5 为 1，否则 R5 为 0。在 (a) 至 (e) 处填入指令，完成程序。


```

                .ORIG x3000
                LD     R0, PTR
                ADD    R1, R0, #0
AGAIN          LDR    R2, R1, #0
                BRz    CONT
                ADD    R1, R1, #1
                BRnzp  AGAIN
CONT           ----- (a)
LOOP          LDR    R3, R0, #0
                ----- (b)
                NOT    R4, R4
                ADD    R4, R4, #1
                ADD    R3, R3, R4
                BRnp   NO
                ----- (c)
                ----- (d)
                NOT    R2, R0
                ADD    R2, R2, #1
                ADD    R2, R1, R2
                BRnz   YES
                ----- (e)
YES            AND    R5, R5, #0
                ADD    R5, R5, #1
                BRnzp  DONE
NO             AND    R5, R5, #0
DONE          HALT
PTR           .FILL  x4000
                .END

```

7.24 我们希望下面的程序片段将 R3 左移 4 位，但代码中有一个错误。找出错误，并说明如何修正。

```

                .ORIG x3000
                AND    R2, R2, #0
                ADD    R2, R2, #4
LOOP          BRz    DONE
                ADD    R2, R2, #-1
                ADD    R3, R3, R3
                BR     LOOP
DONE          HALT
                .END

```

7.25 伪操作 .FILL xFF004 会做什么？为什么？

7.26 回想习题 7.6 的汇编语言程序，再考虑下面的程序。

```

                .ORIG    x3000
                AND      R0, R0, #0
D              LD        R1, A
                AND      R2, R1, #1
                BRp      B
E              ADD      R1, R1, #-1
B              ADD      R0, R0, R1
                ADD      R1, R1, #-2
F              BRp      B
                ST        R0, C
                TRAP     x25
A              .BLKW     1
C              .BLKW     1
                .END

```

汇编器分别将这两个汇编语言程序翻译成机器语言程序。对于得到的两个机器语言程序，你能得出什么结论？

7.27 考虑下面的 LC-3 汇编语言程序。

```

                .ORIG    x3000
                AND      R2, R2, #0
                AND      R6, R6, #0
                ADD      R2, R2, #1
TOP             ADD      R3, R2, #0
                ADD      R4, R1, #0
SEARCH         ADD      R3, R3, R3
                ADD      R4, R4, #-1
                BRp      SEARCH
                AND      R5, R3, R0
                BRz      NEXT
                ADD      R6, R6, R2
NEXT           ADD      R2, R2, R2
                BRzp     TOP
END            ST        R6, RESULT
                HALT
RESULT        .BLKW     1
                .END

```

用不超过 20 个词说明它做什么。请**简短而准确**。可以假定某些寄存器中已经存有与程序相关的数。

R0 的作用是什么？输入值在哪个范围内，程序才能实现你上面描述的功能？

R1 的作用是什么？输入值在哪个范围内，程序才能实现你上面描述的功能？

R6 的作用是什么？输入值在哪个范围内，程序才能实现你上面描述的功能？

7.28 [*] 考虑下面的程序。

```
.ORIG x3000
LD R0, A
LD R1, B
BRz DONE
----- (a)
----- (b)
BRnzp AGAIN
DONE ST R0, A
HALT
A .FILL x0--- (c)
B .FILL x0001
.END
```

程序只使用 R0 和 R1。(a) 和 (b) 两行缺少指令，(c) 行也需要补全。此外，程序中的一条指令应带有标号 AGAIN，但这个标号也缺失了。

程序执行结束后，A 的内容为 x1800。

在程序执行期间，我们在每个时钟周期检查计算机，并记录了某些周期的信息，得到下表。表格按收集信息的周期编号排列。注意，每次存储器访问需要五个时钟周期。

Cycle Number	State Number	Control Signals		
1	18	LD.MAR:	LD.REG:	GateMDR:
		LD.PC:	PCMUX:	GatePC:
	0	LD.MAR:	LD.REG:	BEN
		LD.PC:	LD.CC:	
		LD.REG: 1	DR: 000	GateMDR:
		GateALU:	GateMARMUX:	
57	1	LD.MAR:	ALUK:	GateALU:
		LD.REG:	DR:	GatePC:
77	22	ADDR1MUX:	ADDR2MUX:	
		LD.PC:	LD.MAR:	PCMUX:
101	15			

补上缺失的指令，并在适当的指令处添加标号 AGAIN，使程序完整；同时填入表格中缺少的信息。

给定 A 和 B 的值，这个程序做什么？

7.29 [*] 一个 LC-3 程序在 LC-3 模拟器中执行时遇到断点，模拟器停止。此时若干寄存器的内容如表格第一行所示。随后再次按下运行按钮，接下来执行的四条指令产生表中所列的值；这四条指令都不是 STI 或 LDI。每条指令对应两行：第一行是该指令取指令阶段结束后的内容，第二行是该指令执行完成后的内容。

有些值缺失，用字母 A、B、C、D、E、F、G、H、I 和 J 表示。

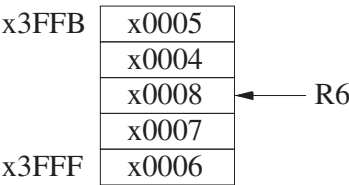
PC	MAR	MDR	IR	R0	R1	R2	R3	R4	R5	R6	R7
x1800	x7FFF	x2211	xBFFE	x31FF	x2233	x5177	x3211	x21FF	x5233	x3177	x2211
A	x1800	B	B	x31FF	x2233	x5177	x3211	x21FF	x5233	x3177	x2211
A	x1800	B	B	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
D	A	E	E	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
D	F	G	E	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
H	D	I	I	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
F	D	I	I	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
A	F	J	J	x31FF	x2233	x5177	x3211	x21FF	C	x3177	x2211
A	F	J	J	x31FF	x2233	x5177	x3211	x223A	C	x3177	x2211

你的任务是确定 A 至 J 的值。注意，某些值可能相同。

A	B	C	D	E

F	G	H	I	J
			x 'F'	

7.30 [★] 有时需要在存储器中实现一个栈，却无法提供足够多的空间来保证始终可以继续压栈。此外，有时允许丢失一些最早压入栈的值——这超出了 EE 306 的范围，如果你愿意，我们可以留到最后一课讨论。在这种情况下，一种合理的技术是采用下图所示的**循环栈**。本例中，栈占用 x3FFB 至 x3FFF 五个存储单元。最初栈为空，R6 = x4000。图中显示了依次压入 1、2、3、4、5、6、7、8 后的结果。



也就是说，1 写入 x3FFF，2 写入 x3FFE，依此类推。压入 6 时栈已满，于是将 R6 设为 x3FFF，将 6 写入 x3FFF，覆盖最早压入的 1。

如果现在弹出五个元素，就会得到 8、7、6、5、4，而且栈已经为空，尽管 R6 中是 x3FFD。为什么？因为 3、2、1 已经丢失。也就是说，虽然一共压入了八个值，但栈中真正可供弹出的值最多只有五个。我们用 R5 跟踪栈中实际值的数量。

调用例程知道 R5 和 R6，因此可以用 R5 检查下溢。此外，调用程序在调用 PUSH 前，会将要压入的值放在 R0 中。

你的任务是补全下面的汇编代码，实现循环栈的 PUSH 例程。在 (a)、(b)、(c) 和 (d) 每行各填入一条缺失的指令。

```

PUSH      ST R1, SAVER
          LD R1, NEGFULL
          ADD R1, R6, R1
          ----- (a)

          LD R6, BASE
SKIP      ADD R6, R6, #-1
          LD R1, MINUS5
          ADD R1, R5, R1
          BRz END
          ----- (b)

END        ----- (c)
          ----- (d)

          RET
NEGFULL   .FILL xC005      ; x-3FFB
MINUS5    .FILL xFFFB      ; #-5
BASE      .FILL x4000
SAVER     .BLKW #1

```

7.31 地址 x5000 至 x5FFF 中存放的是二进制补码整数。下面的程序做什么？

```

          .ORIG x3000
          LD R1, ARRAY
          LD R2, LENGTH
          AND R3, R3, #0
AGAIN     LDR R0, R1, #0
          AND R0, R0, #1
          BRz SKIP
          ADD R3, R3, #1
SKIP      ADD R1, R1, #1
          ADD R2, R2, #-1
          BRp AGAIN
          HALT
ARRAY     .FILL x5000
LENGTH   .FILL x1000
          .END

```

7.32 考虑下面这个有些不合常理的汇编语言程序。

```
line 1:          .ORIG x8003
line 2:          AND R1,R1,#0
line 3:          ADD R0,R1,#5
line 4:          ST  R1,B
line 5:          LD  R1,A
line 6:          BRz SKIP
line 7:          ST  R0,B
line 8:  SKIP    TRAP x25
line 9:  A       .BLKW #7
line 10: B       .FILL #5
line 11: BANNER .STRINGZ "We are done!"
line 12: C       .FILL x0
line 13:          .END
```

另一个独立模块会在程序执行前向 A 存入一个值。
建立符号表。
写出上面第 5 至 7 行的汇编结果。注意，第 8 行的指令已经替你汇编好了。

x8006																				
x8007																				
x8008																				
x8009	1	1	1	1	0	0	0	0	0	0	1	0	0	1	0	1				

注意，有两种不同的方式会使 B 中包含值 5：第 7 行的内容，或第 10 行的内容。解释由第 7 行使 B 中出现 5，与由第 10 行使 B 中出现 5，有什么区别。

7.33 [✱] 有一个缺少若干指令的程序，还有一张表，记录了程序执行过程中五个特定时钟周期的信息，其中也缺少一些内容。你的任务是把二者都补全！
填入程序中缺失的指令和表格中缺失的信息。时钟周期从 1 开始编号，即周期 1 是处理 LD R0, A 的第一个时钟周期。注意，我们没有说明一次存储器访问需要多少个时钟周期，但已有信息足以让你自己推断。还要注意，这里要求的是各个时钟周期**期间**寄存器的值。

```
.ORIG x3000
LD  R0, A
LD  R1, B
NOT R1, R1
ADD R1, R1, #1
AND R2, R2, #0
AGAIN ----- (a)
----- (b)
BRnzp AGAIN
DONE  ST  R2, C
HALT
A     .FILL #5
B     .FILL ----- (c)
C     .BLKW #1
.END
```

Cycle Number	State Number	Information					
		LD.REG: 1	DRMUX:	GateMDR:			
		LD.CC:	GateALU:	GatePC:			
16	35	LD.MDR:	MDR:	IR:			
		LD.IR:					
50		LD.REG: 1	MDR: x_4A_	DRMUX:			
		BUS: x0001		GateMDR:			
57	1	PC:	IR: x_040	GateALU:			
		BUS: x0003		GatePC:			
	23	ADDR1MUX:	ADDR2MUX:				
		LD.PC: 1	PC: x3008	PCMUX:	ADDER		

对于存储单元 A 和 B 中给定的具体操作数，执行结束时 C 中存储什么？

实际上，这段程序是一个学生写的，正如所料，他并没有完全写对。已经很接近了，但还差一点！本题的最后一个任务是检查代码，判断这个学生原本想做什么，指出他在哪里弄错了，以及你会如何修正。不必编写代码，简要说明修正方法即可。

这个学生原本想做什么？

他在哪里弄错了？

你会如何修正他的程序？

7.34 求两个值的中点常常很有用。本题假定 A 和 B 都是偶数，且 A 小于 B。例如，A = 2、B = 8 时，中点为 5。下面的程序通过不断将较小的数加 1、较大的数减 1，求两个偶数 A 和 B 的中点。可以假定程序开始执行前，A 和 B 已经装入了相应的值。

你的任务是填入缺失的指令。

```

                                .ORIG  x3000
                                LD      R0, A
                                LD      R1, B
X                                ----- (a)
                                ----- (b)
                                ADD     R2, R2, R1
                                ----- (c)
                                ADD     R1, R1, #-1
                                ----- (d)
                                BRnzp   X
                                DONE    ST      R1, C
                                TRAP     x25
A                                .BLKW   1
B                                .BLKW   1
C                                .BLKW   1
                                .END
```

7.35 [✱] 存储在地址 x3000 至 x3007 的程序，从 x3100 装入一个值，进行一些处理，再把结果存入 x3101。下面给出了不完整的程序描述。你的任务是补全它。

Address	Contents	Assembly code
x3000	0101 001 001 1 00000	AND R1, R1, #0
x3001	0010 000	LD R0, x3100
x3002	0000 110 000000011	BRnz x3006
x3003	0001	ADD
x3004		
x3005	0000 111	
x3006	0011 001	ST R1, x3101
x3007	1111 0000 0010 0101	HALT

为帮助你完成任务，我们在第一条指令执行之前，以及此后若干指令边界处，也就是执行了一定数量的指令之后，对部分机器状态进行了快照。下表给出了快照的一部分，你需要补全它。注意，程序在执行 17 条指令后进入 TRAP x25 服务例程，所以某些指令必然执行了不止一次。

下表中标为 xxxx 的条目不需要填写。对于表中未列出的指令，也不必考虑它们的快照。

Instruction #	PC	MAR	MDR	R0	R1
Initial	x3000	xxxx	xxxx	xxxx	xxxx
1	x3001	xxxx	xxxx	xxxx	
2	x3002				
3	x3003	xxxx	xxxx		
4	x3004		x1240		
5	x3005	xxxx	xxxx	x0002	
9	x3005	xxxx	xxxx	x0001	
13	x3005	xxxx	xxxx	x0000	
14	x3002	xxxx	xxxx		
15	x3006	xxxx	xxxx		
16	x3007				
17	xxxx	xxxx	xxxx		

7.36 取模运算 $A \bmod B$ 得到 A 除以 B 的余数。例如, $10 \bmod 5 = 0$, $12 \bmod 7 = 5$ 。下面的程序本应计算 $A \bmod B$, 其中 A 在 $x3100$, B 在 $x3101$, 结果应存入 $x3200$ 。但是程序员犯了一个严重错误, 所以程序不能正确工作。可以假定 A 和 B 都是正整数。

```

                                .ORIG x3000                ; Line 1
                                LD R3, L2                  ; 2
                                LDR R0, R3, #0              ; 3
                                LDR R1, R3, #1              ; 4
                                NOT R2, R1                 ; 5
                                ADD R2, R2, #1             ; 6
                                L1    ADD R0, R0, R2        ; 7
                                BRz L1                      ; 8
                                ADD R0, R0, R1             ; 9
                                ST R0, L3                  ; 10
                                HALT                       ; 11
                                L2    .FILL x3100           ; 12
                                L3    .FILL x3200           ; 13
                                .END                       ; 14

```

第 6 行的指令执行完后, $R0$ 、 $R1$ 和 $R2$ 分别是什么? 注意, 每个寄存器的正确答案都是以下值之一: A 、 $-A$ 、 B 、 $-B$ 、 0 、 1 、 -1 。

程序有一个 bug。它在哪一行? 正确的指令应该是什么?

7.37 [*] 在我们课堂上一直使用的数据通路处理一个 LC-3 程序的过程中, 计算机因地址 $x3000$ 处设置的断点而停止。此时某些寄存器和存储单元的内容如下:

```

R2 至 R7:      x0000
M[x3000]:      x1263
M[x3003]:      x0000

```

LC-3 重新启动，恰好执行四条指令。完成这些指令需要若干时钟周期，其中有 15 个时钟周期必须使用总线。下表按顺序列出这 15 个周期，以及各周期中送到 LC-3 总线上的值。

	BUS
1st:	x3000
2nd:	x1263
3rd:	x009A
4th:	x3001
5th:	xA000
6th:	
7th:	
8th:	
9th:	
10th:	
11th:	
12th:	
13th:	x3003
14th:	x1263
15th:	x009D

填入表格中缺少的条目。

执行的是哪四条指令？

这四条指令执行完后，R0 和 R1 中分别是什么？

第八章 数据结构

到目前为止，我们用计算机处理的每一项信息都只是一个值：一个整数、一个浮点数，或一个 ASCII 字符。现实世界中的许多信息，比单个简单的数复杂得多。公司的组织结构图、按字母顺序排列的项目列表，就是两个例子。我们把这些复杂的信息称为**抽象数据类型** (abstract data type)，或者更通俗地称为**数据结构** (data structure)。本章研究三种抽象数据类型：栈、队列和字符串。我们将编写程序，解决那些需要按信息自身结构来表示信息的问题。当然还有其他抽象数据类型，但要留到介绍 C 程序设计语言之后，在第 15 章讨论。

不过，在讨论栈、队列和字符串之前，先介绍一个对操作数据结构很有用的新概念：**子程序**，也称为函数。

8.1 子程序

在同一个程序中多次调用某个程序片段，而不必每次需要时都在源程序中重新写出它的细节，往往很有用。现实的软件开发中，也经常（或许通常？）是由一位软件工程师编写需要这些片段的程序，由另一位工程师编写这些片段。

也可能所需的程序片段已经由制造商或独立软件供应商提供。几乎总有这类片段的集合供用户程序员使用，使他们不必自己编写。这些集合称为**库** (library)。数学库就是一个例子，它包含计算平方根、正弦、反正切等函数的程序片段。事实上，正因为有数学库，用户程序员甚至不必知道如何编写相应的程序片段，就能让计算机算出这些函数的值！

因此，最好有一种能高效使用程序片段的方法。这些片段称为**子程序** (subroutine)，也称为**过程** (procedure)，或者用 C 的术语来说，称为**函数** (function)。

图 8.1 简单展示了程序的一部分，我们称其为 piece-of-code-A，其中有些片段需要在这部分中执行多次。第 9 章会详细研究图 8.1；现在先忽略其他内容，只看从符号地址 L1、L2、L3 和 L4 开始的四组三条指令。

```

01  START    ST      R1,SaveR1    ; Save registers needed
02          ST      R2,SaveR2    ; by this routine
03          ST      R3,SaveR3
04  ;
05          LD      R2,Newline
06  L1       LDI     R3,DSR
07          BRzp    L1            ; Loop until monitor is ready
08          STI     R2,DDR        ; Move cursor to new clean line
09  ;
0A          LEA     R1,Prompt     ; Starting address of prompt string
0B  Loop     LDR     R0,R1,#0     ; Write the input prompt
0C          BRz     Input        ; End of prompt string
0D  L2       LDI     R3,DSR
0E          BRzp    L2            ; Loop until monitor is ready
0F          STI     R0,DDR        ; Write next prompt character
10          ADD     R1,R1,#1     ; Increment prompt pointer
11          BRnzp   Loop         ; Get next prompt character
12  ;
13  Input    LDI     R3,KBSR
14          BRzp    Input        ; Poll until a character is typed
15          LDI     R0,KBDR      ; Load input character into R0
16  L3       LDI     R3,DSR
17          BRzp    L3            ; Loop until monitor is ready
18          STI     R0,DDR        ; Echo input character
19  ;
1A  L4       LDI     R3,DSR
1B          BRzp    L4            ; Loop until monitor is ready
1C          STI     R2,DDR        ; Move cursor to new clean line
1D          LD      R1,SaveR1    ; Restore registers
1E          LD      R2,SaveR2    ; to original values
1F          LD      R3,SaveR3
20          JMP     R7            ; Do the program's next task
21  ;
22  SaveR1   .BLKW   1            ; Memory for registers saved
23  SaveR2   .BLKW   1
24  SaveR3   .BLKW   1
25  DSR      .FILL   xFE04
26  DDR      .FILL   xFE06
27  KBSR     .FILL   xFE00
28  KBDR     .FILL   xFE02
29  Newline  .FILL   x000A        ; ASCII code for newline
2A  Prompt  .STRINGZ "Input a character>"

```

图 8.1 第 9 章将详细研究的一段指令序列 (piece-of-code-A)

这四个三指令序列分别完成以下操作：

```

label  LDI  R3,DSR
        BRzp label
        STI  Reg,DDR

```

四处所用的标号各不相同 (L1、L2、L3、L4)，但这不是问题，因为每处标号的唯一用途，都是让 BRzp 指令跳回 LDI 指令。

其中两个片段存储 R0 的内容，另外两个存储 R2 的内容，但这很容易处理，稍后就会看到。关键在于，除了 STI 指令使用哪个寄存器作为源这个小差别之外，四个片段做的事情完全相同，让程序员把代码写四遍是浪费。子程序的调用 / 返回机制使程序员只需写一遍代码。

8.1.1 调用 / 返回机制

调用 / 返回机制允许我们把这个三指令序列作为子程序在程序中只写一次，却执行多次。

图 8.2 给出了使用和不使用子程序时的指令执行流程。注意，不使用子程序时，

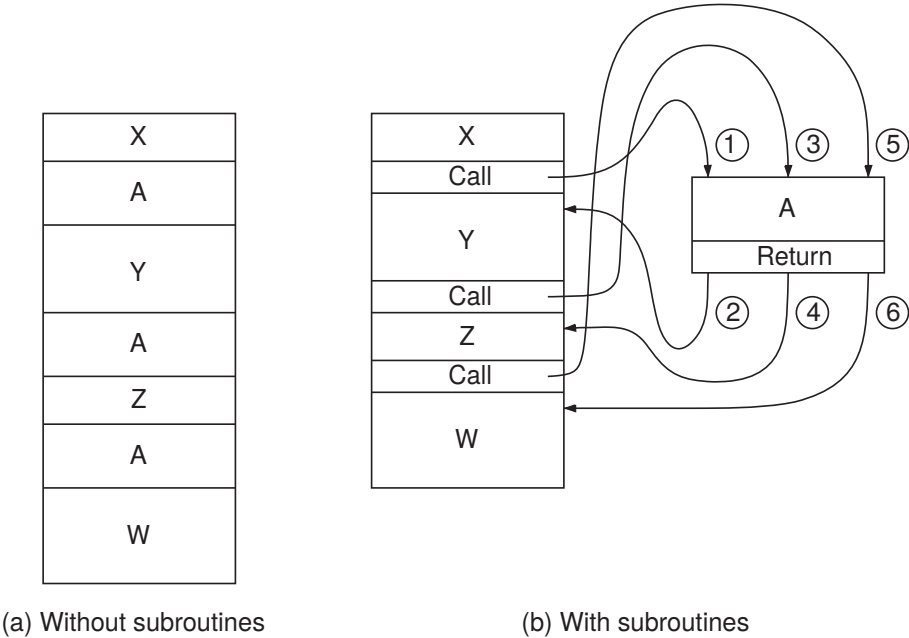


图 8.2 使用与不使用子程序时的指令执行流程

程序员必须分别在 X、Y 和 Z 之后提供同样的代码 A；使用子程序时，只需提供一次代码 A。程序员利用调用 / 返回机制，每次通过调用指令引导计算机执行代码 A；执行完后，再通过返回指令回到程序中正确的下一条指令。

包含调用指令的程序称为**调用者** (caller)，包含返回指令的子程序称为**被调用者** (callee)。

调用 / 返回机制由两条指令组成。第一条 JSR(R) 位于调用程序中，完成两件事：将子程序的起始地址装入 PC，并把紧接 JSR(R) 指令之后的地址装入 R7。这个地址就是子程序执行结束后应当回到的位置，我们将它称为**返回链接** (return linkage)。第二条 JMP R7 是子程序，即被调用程序中的最后一条指令。它将 R7 的内容装入 PC，也就是返回到 JSR 指令之后的位置，从而完成从调用者到被调用者、再回到调用者的控制流往返。

8.1.2 JSR(R)——调用子程序的指令

LC-3 定义了一条用于调用子程序的控制指令，操作码为 0100。它用两种寻址方式之一计算子程序的起始地址：PC 相对寻址或基址寄存器寻址。LC-3 汇编语言根据所用的寻址方式，为同一操作码提供两个不同的助记符：JSR 和 JSRR。

JSR(R) 指令完成两件事。和所有控制指令一样，它装入 PC，覆盖该指令在取指令阶段自增后写入 PC 的值。这里装入的是计算得到的子程序起始地址。第二件事是将返回地址保存到 R7。返回地址就是自增后的 PC，即调用程序中紧接在 JSR(R) 后面的指令地址。

JSR(R) 指令由三部分组成。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
Opcode					A	Address evaluation bits									

位 [15:12] 包含操作码 0100。位 [11] 指定寻址方式：1 表示 PC 相对寻址，0 表示基址寄存器寻址。位 [10:0] 包含用于获得子程序起始地址的信息。JSR 和 JSRR 的唯一区别，就是求子程序起始地址时所用的寻址方式。

JSR JSR 将指令中的 11 位偏移量（位 [10:0]）符号扩展为 16 位，再添加到自增后的 PC 上，得到子程序的目标地址。这与另一条控制指令 BR 的寻址方式几乎相同，只是 PCoffset 使用 11 位，而 BR 使用 9 位。

如果下面的 JSR 指令存储在 $\times 4200$ ，执行后会将 $\times 3E05$ （即 $\times FC04$ 加 $\times 4201$ ）装入 PC，并将 $\times 4201$ 装入 R7。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	1	1	0	0	0	0	0	0	0	1	0	0
JSR					A	PCoffset11									

JSRR 除寻址方式外，JSRR 与 JSR 完全相同。JSRR 获取子程序起始地址的方式与 JMP 相同：位 [8:6] 指定基址寄存器，其中保存着要装入 PC 的地址。

如果下面的 JSRR 指令存储在 $\times 420A$ ，且 R5 包含 $\times 3002$ ，则执行后 R7 为 $\times 420B$ ，PC 为 $\times 3002$ 。

思考：JSRR 提供了什么 JSR 所不具备的重要功能？

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
0	1	0	0	0	0	0	1	0	1	0	0	0	0	0	0
JSRR					A	BaseR									

8.1.3 保存和恢复寄存器

我们早已知道，每当指令把一个值装入寄存器，寄存器中原来的值就会丢失。因此，如果同时满足以下两点，就需要保存寄存器中的值：

- 后续某条指令会破坏这个值；
- 执行那条指令之后，还需要这个值。

在使用子程序时，这可能成为一个问题。

再看图 8.1 的代码。假设它是一个子程序，由某个调用程序中的 JSR START 调用；我们把调用程序称为 CALLER。

假定 CALLER 在执行 JSR START 之前，计算了一些值并装入 R1、R2 和 R3。从 START 开始的子程序中，第 05 行向 R2 装入一个值，第 06 行向 R3 装入一个值，第 0A 行向 R1 装入一个值。如果 CALLER 在子程序返回后还需要原来的那些值，会怎么样？糟糕了！子程序执行第 05、06、0A 行时已经破坏了这些值，所以在第 20 行 JMP R7 之后，CALLER 恢复执行时就找不到它们了。这当然不能接受。

我们在子程序的第一部分，也就是初始化阶段，防止这种情况发生。第 01、02、03 行把 R1、R2、R3 的内容分别存入 SaveR1、SaveR2 和 SaveR3。这三个存储单元由第 22、23、24 行专门保留。第 1D、1E、1F 行则在 JMP R7 之前，把保存的值装回 R1、R2、R3。也就是说，子程序使用这些寄存器之前，先保存调用程序放在其中的值；返回调用程序之前，再把这些值放回调用程序理应能找到它们的位置。

这种技术称为**被调用者保存**（callee save），因为保存和恢复寄存器的是子程序，即被调用者。由子程序保存寄存器是合理的，因为它知道完成自己的工作需要哪些寄存器。没有必要让编写调用程序的人也必须知道这些细节。

当然，也可以让调用程序在 JSR START 之前保存所有寄存器，这样子程序就不必操心保存工作。有些程序确实这样做，某些 ISA 的 JSR 指令甚至将这项工作作为指令执行的一部分。不过，如果希望消除不必要的保存和恢复，在这里可以让被调用者只保存自己需要的寄存器。

还要指出，JSR START 会把返回链接装入 R7，因此执行这条指令会破坏 R7 原有的内容。¹如果调用程序在调用 START 处的子程序之前已向 R7 存入一个值，而且返回后还需要它，就必须由调用程序保存和恢复 R7。为什么由调用程序做？因为它知道自己执行 JSR START 会破坏 R7。这称为**调用者保存**（caller save），因为保存和恢复寄存器值的是调用程序。

要点是：如果寄存器中存入别的内容之后，还要用到原来的值，就必须事先保存，并在以后使用之前恢复。保存的方法是将寄存器的值存入存储器，恢复的方法是将它重新装入寄存器。

保存 / 恢复问题既可以由调用程序在 JSR 发生之前处理，也可以由子程序处理。9.3 节将看到，另一类调用 / 被调用例程，即系统调用，也存在同样的问题。

由调用程序处理，称为调用者保存；由被调用程序处理，称为被调用者保存。应该由哪一方处理，取决于哪一方知道后续操作会破坏哪些寄存器。

被调用者知道完成自己的工作需要哪些寄存器，所以开始前用一串存储指令保存它们，结束后用一串装入指令恢复它们，并为这些寄存器值保留存储单元。

¹译注：原书本段多次写作 JMP START，但所述“将返回链接写入 R7”的行为属于 JSR。本段按上下文修正为 JSR START。

调用者知道自己控制的指令会破坏什么。它知道每次 JSR 都会破坏 R7 的内容，因此在执行 JSR 前保存 R7，并在子程序结束、调用程序恢复执行后恢复 R7。

8.1.4 库例程

本节开头提到，调用 / 返回机制有许多用途，其中之一就是让用户程序调用通常随计算机系统一起提供的库子程序。库为用户程序员提供便利。说它能提高生产效率并不过分，因为应用程序员不必了解或学习太多内部细节就能使用。例如，程序员往往知道什么是平方根（我们缩写为 SQRT），可能需要求某个 x 的 $\text{sqrt}(x)$ ，却完全不知道怎样写程序计算，也很可能不愿意为此专门学习。

一个简单的例子可以说明这一点：我们丢了钥匙，需要进入公寓。可以把梯子靠在墙上，顶端搭到敞开的窗户下沿，窗户离地 24 英尺。墙边有一个宽 10 英尺的花坛，所以梯子底端必须放在花坛之外。需要多长的梯子，才能靠墙爬进窗户？换一种不那么生动的说法，就是：直角三角形的两条直角边分别长 24 英尺和 10 英尺，斜边多长（见图 8.3）？高中时我们学过，毕达哥拉斯已经给出了答案：

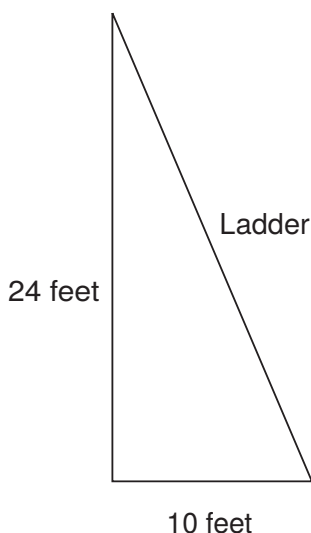


图 8.3 求斜边长度

$$c^2 = a^2 + b^2.$$

已知 a 和 b ，对 $a^2 + b^2$ 开平方就能求出 c 。求和不难，LC-3 的 ADD 指令就能完成；平方也不难，可以用连续加法实现乘法。可是，怎样求平方根？图 8.4 给出了求解程序的结构。SQRT 子程序还没有编写。如果没有数学库，程序员就得找本数学书（或请人帮忙），查阅 Newton-Raphson 方法，补写这个子程序。

有了数学库，问题基本就解决了。数学库提供包括 SQRT 在内的许多子程序，用户程序员可以继续对 Newton-Raphson 一无所知。不过，用户仍需要知道平方根库例程目标地址的标号、参数 x 应放在哪里，以及去哪里取得结果 $\text{SQRT}(x)$ 。这些只是简单的接口约定，可以从数学库的文档中查到。


```

01          ...
02          ...
03          LD      R0, SIDE1
04          BRz     S1
05          JSR     SQUARE
06      S1      ADD     R1, R0, #0
07          LD      R0, SIDE2
08          BRz     S2
09          JSR     SQUARE
0A      S2      ADD     R0, R0, R1
0B          JSR     SQRT
0C          ST      R0, HYPOT
0D          BRnzp   NEXT_TASK
0E      SQUARE  ADD     R2, R0, #0
0F          ADD     R3, R0, #0
10      AGAIN   ADD     R2, R2, #-1
11          BRz     DONE
12          ADD     R0, R0, R3
13          BRnzp   AGAIN
14      DONE    RET
15      SQRT    ...           ; R0 <-- SQRT(R0)
16            ...           ;
17            ...           ; How do we write this subroutine?
18            ...           ;
19            ...           ;
1A          RET
1B      SIDE1  .BLKW    1
1C      SIDE2  .BLKW    1
1D      HYPOT  .BLKW    1
1E          ...
1F          ...

```

图 8.4 计算直角三角形斜边的程序片段

如果库例程从 **SQRT** 开始，用 **R0** 传入参数，并从 **R0** 取得结果，图 8.4 就可以变成图 8.5。有两点需要注意。

```

01          ...
02          ...
03          .EXTERNAL SQRT
04          ...
05          ...
06          LD      R0, SIDE1
07          BRz     S1
08          JSR     SQUARE
09      S1      ADD     R1, R0, #0
0A          LD      R0, SIDE2
0B          BRz     S2
0C          JSR     SQUARE
0D      S2      ADD     R0, R0, R1 ; R0 contains argument x
0E          LD      R4, BASE ; BASE contains starting address of SQRT routine
0F          JSRR    R4
10          ST      R0, HYPOT
11          BRnzp   NEXT_TASK
12      SQUARE  ADD     R2, R0, #0
13          ADD     R3, R0, #0
14      AGAIN  ADD     R2, R2, #-1
15          BRz     DONE
16          ADD     R0, R0, R3
17          BRnzp   AGAIN
18      DONE   RET
19      BASE   .FILL   SQRT
1A      SIDE1 .BLKW   1
1B      SIDE2 .BLKW   1
1C      HYPOT .BLKW   1
1D          ...
1E          ...

```

图 8.5 图 8.4 的程序片段，改用库例程

第一，程序员不再需要操心怎样计算平方根，库例程会代劳。

第二，伪操作 **.EXTERNAL**。7.4.2 节已经讨论过，它告诉汇编器：汇编第 19 行 **.FILL** 所需的标号 **SQRT** 将由另一个程序片段，即另一个模块提供；生成可执行映像时，两个模块会被组合起来。可执行映像是真正执行的二进制模块，它在链接时生成。²

在链接时组合多个模块，生成可执行映像，是通常的做法。图 8.6 展示了这一过程。课程后半部分学习 C 语言时，你将看到具体实例。

²译注：这里沿用 7.4.2 节假设的 **.EXTERNAL** 扩展来说明库的链接过程，不表示所有 LC-3 汇编器都支持它。

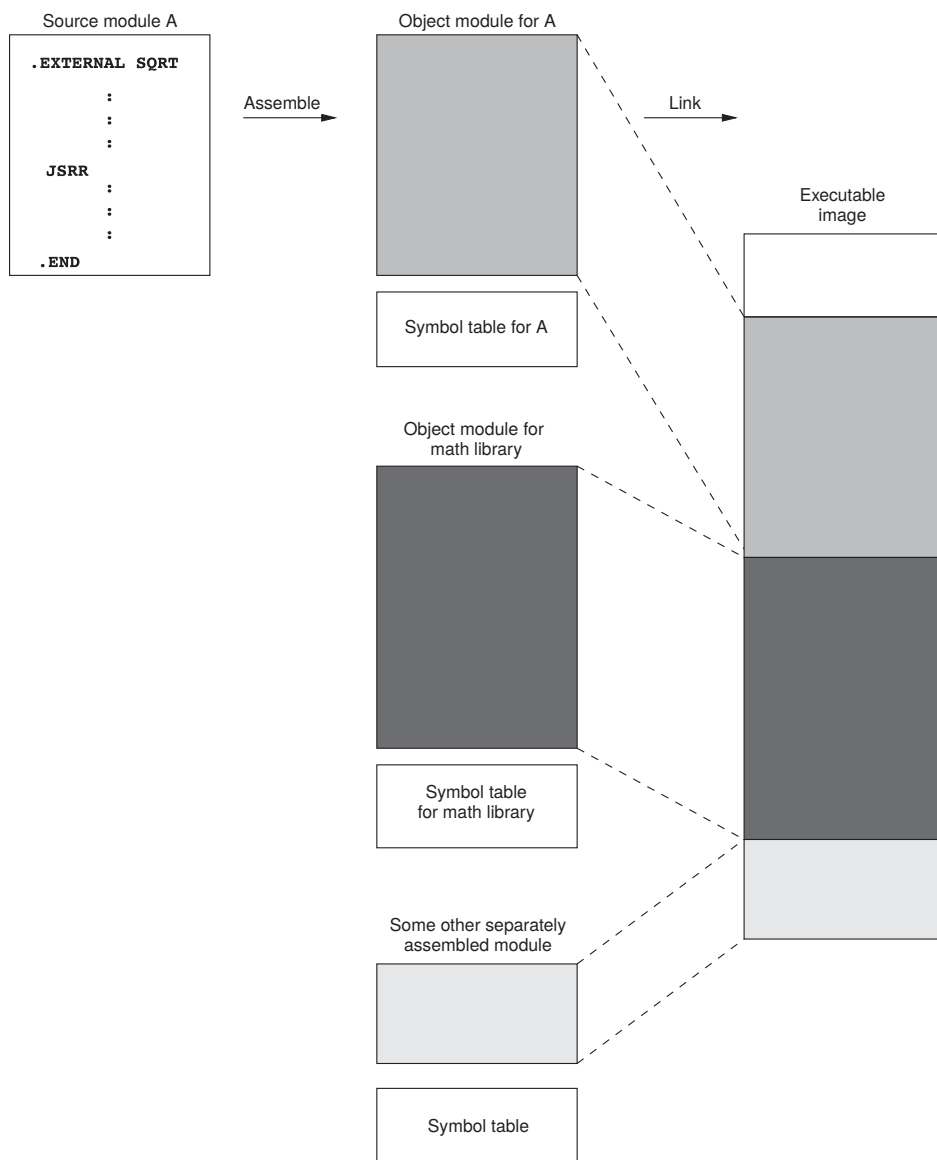


图 8.6 由多个文件构成的可执行映像

大多数应用软件都需要使用各种库中的例程。即便普通程序员能够编写这些例程，全部自行编写也非常低效。我们已经提到数学库；此外，还有用来生成漂亮图形的预处理例程，以及许多完成其他任务的例程。让程序员从头编写这些例程完全没有必要。只要求两件事要容易得多：(1) 有适当的文档，清楚说明库例程与调用程序之间的接口；(2) 源程序中使用 `.EXTERNAL` 等适当的伪操作。链接器就可以在链接时，将分别汇编的模块组合成可执行映像。

8.2 栈

现在可以开始研究数据结构了。首先，也是最重要的数据结构，是栈。

8.2.1 栈——一种抽象数据类型

今后与计算机打交道时，无论编写软件还是设计硬件，你都会一再遇到称为**栈** (stack) 的存储机制。栈有许多不同的实现方式，稍后就会介绍。但首先必须明白，栈的概念与具体实现无关，它规定的是如何访问。栈的定义性特征是：最后存入的元素，最先取出。正是这一点将栈与其他事物区别开来。简单说，就是**后进先出** (Last In, First Out, LIFO)。

用程序设计语言的术语说，栈是一种抽象数据类型。抽象数据类型是由作用于它的操作来定义的存储机制，而不是由某种具体实现方式定义的。本节将看到，用连续存储单元实现的栈。

8.2.2 两个实现实例

汽车驾驶员旁边扶手中的硬币盒，就是一个栈的例子。为了付过路费而取出的第一枚 25 美分硬币，是最后放入硬币堆的那枚。每放入一枚硬币，原先的硬币就被向下压入盒中。

图 8.7 展示了硬币盒的工作方式。最初盒子为空，如图 8.7a。第一次过路费是 75 美分，你给收费员 1 美元；他找回一枚 1995 年的 25 美分硬币，你把它放入盒中，成为图 8.7b 的样子。

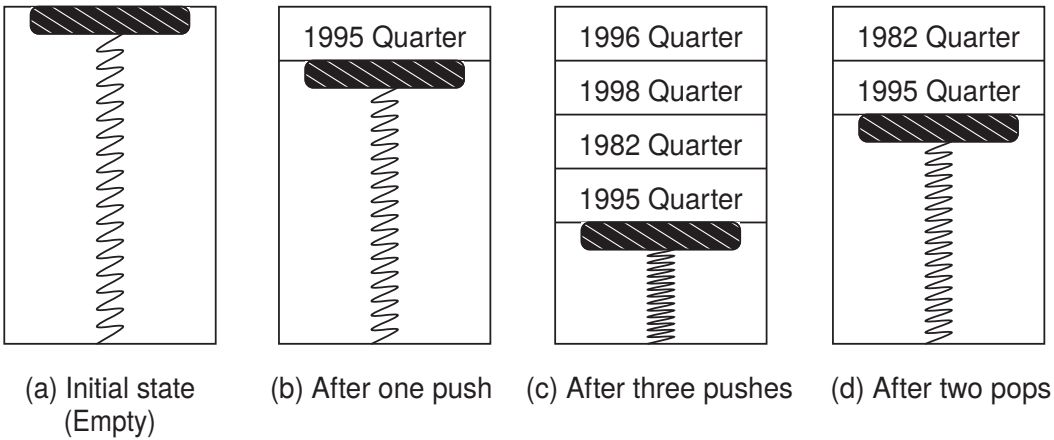


图 8.7 汽车扶手中的硬币盒——栈的一个例子

向栈插入元素和从栈删除元素，有专门的术语。插入一个元素称为将它**压栈** (push)，取出一个元素称为将它**弹栈** (pop)。

第二次过路费是 4.25 美元，你给收费员 5 美元；她找回 75 美分。你依次把三枚 25 美分硬币放入盒中：先是 1982 年的，再是 1998 年的，最后是 1996 年的。此时盒子如图 8.7c。第三次过路费是 50 美分，你取出，即弹出，最上面的两枚硬币：先取 1996 年的，再取 1998 年的。盒子随后如图 8.7d。

硬币盒之所以是栈，正是因为它满足 LIFO 要求：每次插入都在顶部，每次取出也都从顶部进行。最后插入的硬币最先取出，因此它是一个栈。

图 8.8 是栈的另一种实现，有时称为计算机硬件栈。它的行为类似于刚才的硬币

盒，由若干硬件寄存器组成，每个寄存器可以保存一个值。图中有五个寄存器。每向栈添加一个值，或从栈移除一个值，栈中已有的值都会移动。

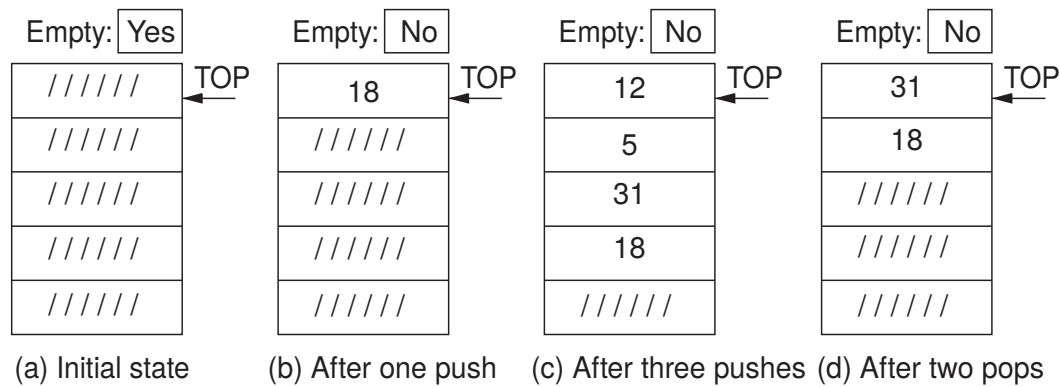


图 8.8 用硬件实现的栈——数据项会移动

图 8.8a 中，栈最初为空。访问总是经过标为 TOP 的第一个位置。压入 18 后，得到图 8.8b；再依次压入 31、5、12，得到图 8.8c；最后弹出两个值，得到图 8.8d。这种栈的一个显著特征是，与硬币盒一样，每当添加或移除一个值，其他已有的值都会移动。

8.2.3 在存储器中实现

计算机中最常见的栈实现如图 8.9 所示。它由一串存储单元和一个跟踪栈顶的机制组成，这个机制称为**栈指针**（stack pointer）。我们用 R6 保存栈顶地址，也就是说，在 LC-3 中，R6 是栈指针。

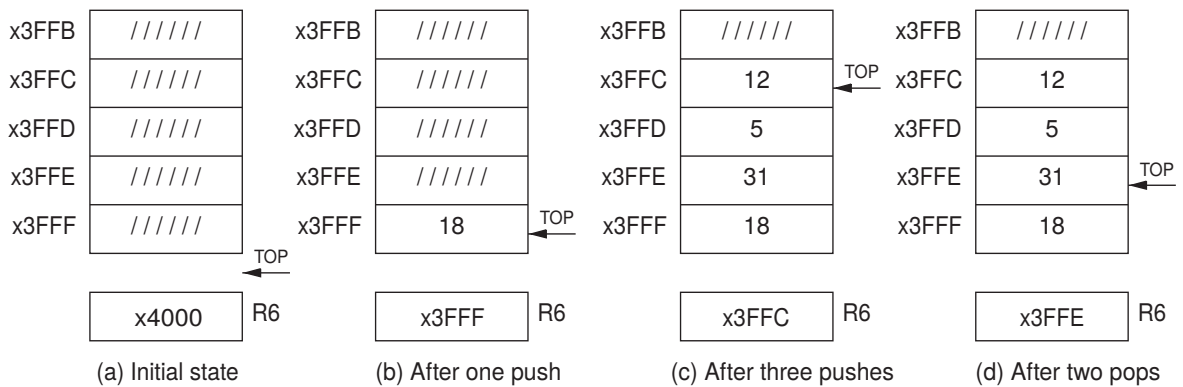


图 8.9 用存储器实现的栈——数据项不会移动

图 8.9 为栈提供了五个存储单元，即 x3FFF 至 x3FFB。任一时刻真正构成栈的，是从 x3FFF 到 R6 指定的位置，也就是栈顶，之间的连续单元。例如图 8.9c 中，栈包含 x3FFF、x3FFE、x3FFD 和 x3FFC 中的内容。

图 8.9a 是最初的空栈。由于没有元素，栈指针为 x4000，即为栈保留的区域之后的那个地址。展示压栈和弹栈的实际代码之后，你就会明白为什么这样设置。图 8.9b 是压入 18 后的栈，栈指针变为 x3FFF，即新的栈顶。

图 8.9c 是再依次压入 31、5、12 后的栈。注意，压入的值存储在地址不断减小的位置，我们说栈**朝地址零的方向增长**。最后，图 8.9d 是弹出栈顶两个元素之后的状态。

注意，被弹出的两个元素 5 和 12 仍在 $\times 3\text{FFD}$ 和 $\times 3\text{FFC}$ 中。但是，只要对这片存储器的每次访问都由栈机制控制，就不能再访问这两个已经弹出的值，稍后会看到原因。

还要注意，与上一节的硬币盒和硬件栈不同，用连续存储单元实现的栈在压入、弹出元素时，栈中已有的数据并不会发生物理移动。

压栈 执行以下两条指令，就能把一个值压入栈：

```
PUSH  ADD R6, R6, #-1
        STR R0, R6, #0
```

图 8.9a 中，R6 为 $\times 4000$ ，表示栈为空。要压入 18，先将栈指针 R6 减 1，使其指向准备存放新值的 $\times 3\text{FFF}$ 。实际压栈时，先将 18 装入 R0，再执行 `STR R0, R6, #0`，将 R0 的内容存入 $\times 3\text{FFF}$ 。

也就是说，要压入一个值，先把它装入 R0，再将原先指向栈顶的 R6 减 1，然后执行 `STR R0, R6, #0`，将 R0 存到 R6 指定的单元。

将 31、5、12 依次装入 R0，每次执行这两条指令，就能把它们压栈。图 8.9c 中，栈指针 R6 为 $\times 3\text{FFC}$ ，说明栈顶位于 $\times 3\text{FFC}$ ，最后压入的值是 12。

弹栈 弹栈时，先读取栈顶的值，再增加栈指针。下面两条指令将栈顶的值弹出并装入 R0：

```
POP    LDR R0, R6, #0
        ADD R6, R6, #1
```

栈指针 R6 加 1，表示原来的栈顶值已经弹出、不再属于栈，现在有了新的栈顶。

如果栈处于图 8.9c 的状态，执行两遍上述序列，就会先弹出 12，再弹出 5。如果弹出它们是为了使用，当然就必须在第二次调用 POP 之前，把 12 从 R0 转移到别处。

弹出 12 和 5 后，R6 为 $\times 3\text{FFE}$ ，表示它们不再位于栈中，栈顶现在是 31。图 8.9d 展示了这一操作序列后的状态。

12 和 5 仍分别存储在 $\times 3\text{FFC}$ 和 $\times 3\text{FFD}$ 中。³但栈要求我们用 PUSH 序列压入、用 POP 序列弹出，因此遵守规则就读不到这两个值。“这些规则”的正式名称是**栈协议**（stack protocol）。

下溢 如果现在尝试从栈中弹出三个值，会怎样？栈只剩两个值，所以会出问题。试图弹出没有事先压入的元素，会造成**下溢**（underflow）。本例可以通过比较栈指针和 $\times 4000$ 来检测：如果栈中已无元素可弹出，R6 就会等于 $\times 4000$ 。若 UNDERFLOW 是处理下溢的例程标号，则 POP 序列变为：

³译注：原书此处将两个地址的对应顺序写反；按图 8.9，12 位于 $\times 3\text{FFC}$ ，5 位于 $\times 3\text{FFD}$ 。

```
POP      LD      R1,EMPTY
         ADD     R2,R6,R1      ; Compare stack
         BRz     UNDERFLOW    ; pointer with x4000.
;
         LDR     R0,R6,#0
         ADD     R6,R6,#1
;
         RET
EMPTY    .FILL    xC000        ; EMPTY <-- negative of x4000
```

POP 不成功时，并不一定要立即跳转到 UNDERFLOW 例程。通常，让 POP 返回调用程序，并在寄存器中报告下溢信息，会更有用。我们用 R5 提供成功 / 失败信息。图 8.10 的流程图展示了如何扩展 POP，用 R5 报告结果。

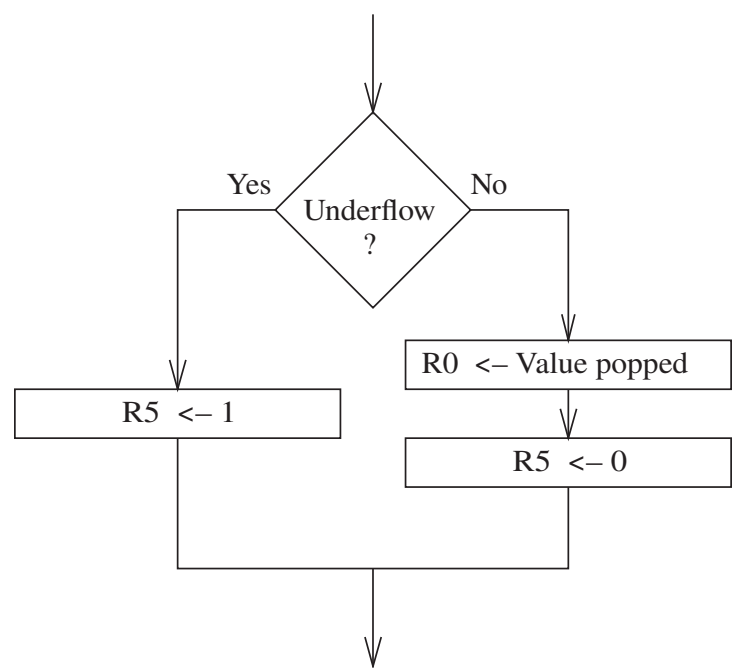


图 8.10 包含下溢检测的 POP 例程

POP 返回后，调用程序检查 R5，判断弹栈是否成功：R5 为 0 表示成功，1 表示失败。

由于 POP 用 R5 报告成功或失败，调用前 R5 中原有的内容会丢失。因此，如果以后还需要原值，就应由调用程序在执行 JSR 前保存 R5。回想 8.1 节，这是调用者保存的一个例子。

得到的 POP 例程如下：

```

POP          AND      R5, R5, #0
              LD       R1, EMPTY
              ADD      R2, R6, R1
              BRz      Failure
              LDR      R0, R6, #0
              ADD      R6, R6, #1
              RET
Failure      ADD      R5, R5, #1
              RET
EMPTY       .FILL     xC000          ; EMPTY <-- -x4000

```

溢出 如果可用空间已耗尽，还想压入一个值，会怎样？没有空间就不能继续存储，这会造成**溢出 (overflow)**。图 8.9 的例子中，可以通过比较栈指针和 x3FFB 来检测溢出。二者相等时，就没有位置可再压入一个值。若 **OVERFLOW** 是处理溢出的例程标号，则 **PUSH** 序列变为：

```

PUSH         LD       R1, MAX
              ADD      R2, R6, R1
              BRz      OVERFLOW
;
              ADD      R6, R6, #-1
              STR      R0, R6, #0
;
              RET
MAX          .FILL     xC005          ; MAX <-- negative of x3FFB

```

和 **POP** 返回成功 / 失败信息比立即跳到 **UNDERFLOW** 更有用一样，让 **PUSH** 采取相同方式也很有用。

根据压栈是否成功完成，我们在 **PUSH** 中增加指令，将 0（成功）或 1（失败）存入 **R5**。返回后，调用程序检查 **R5**，便能知道操作是否成功。

再次注意，**PUSH** 使用 **R5** 报告结果，因此又涉及调用者保存。如果以后还需要 **R5** 原来的值，调用程序必须在执行 **JSR** 之前保存它，因为 **PUSH** 会破坏它。

得到的 **PUSH** 例程如下：

```

PUSH         AND      R5, R5, #0
              LD       R1, MAX
              ADD      R2, R6, R1
              BRz      Failure
              ADD      R6, R6, #-1
              STR      R0, R6, #0
              RET
Failure      ADD      R5, R5, #1
              RET
MAX          .FILL     xC005          ; MAX <-- -x3FFB

```


8.2.4 完整实现

POP 和 PUSH 使我们能把 `x3FFF` 至 `x3FFB` 用作一个容量为五个元素的栈。要压入一个值，只需把它装入 `R0`，再执行 `JSR PUSH`；要把一个值从栈中弹出到 `R0`，只需执行 `JSR POP`。如果想改变栈的位置或大小，相应调整 `BASE` 和 `MAX` 即可。⁴

结束这个话题之前，还要处理 8.1.3 节讨论过的一个重要细节。PUSH 和 POP 会使用 `R1` 和 `R2`，而调用程序没有理由知道这一点。因此，子程序应在使用它们之前保存，并在返回之前恢复，这属于被调用者保存。

PUSH 和 POP 也会写入 `R5`。不过，正如前面指出的，调用程序知道子程序要用 `R5` 报告成功或失败，所以如果以后还需要 `R5` 的原值，就应由调用程序在 `JSR` 前保存它。这是 8.1.3 节所说的调用者保存。

PUSH 和 POP 操作的最终代码见图 8.11。

⁴译注：原文此处称 `BASE`；前面的代码以 `EMPTY` 标识空栈边界，以 `MAX` 标识容量边界；图 8.11 的最终版本则使用 `EMPTY` 和 `FULL`。移动栈或改变容量时应相应调整这些常量。

```

01 ;
02 ; Subroutines for carrying out the PUSH and POP functions. This
03 ; program works with a stack consisting of memory locations x3FFF
04 ; through x3FFB. R6 is the stack pointer.
05 ;
06 POP          AND      R5,R5,#0          ; R5 <-- success
07              ST       R1,Save1          ; Save registers that
08              ST       R2,Save2          ; are needed by POP
09              LD       R1,EMPTY          ; EMPTY contains -x4000
0B              ADD      R2,R6,R1          ; Compare stack pointer to x4000
0C              BRz      fail_exit         ; Branch if stack is empty
0D ;
0E              LDR      R0,R6,#0          ; The actual "pop"
0F              ADD      R6,R6,#1          ; Adjust stack pointer
10              BRnzp    success_exit
11 ;
12 PUSH         AND      R5,R5,#0          ; R5 <-- success
13              ST       R1,Save1          ; Save registers that
14              ST       R2,Save2          ; are needed by PUSH
15              LD       R1,FULL           ; FULL contains -x3FFB
16              ADD      R2,R6,R1          ; Compare stack pointer to x3FFB
17              BRz      fail_exit         ; Branch if stack is full
18 ;
19              ADD      R6,R6,#-1         ; Adjust stack pointer
1A              STR      R0,R6,#0          ; The actual "push"
1B success_exit LD       R2,Save2          ; Restore original
1C              LD       R1,Save1          ; register values
1D              RET
1E ;
1F fail_exit    LD       R2,Save2          ; Restore original
20              LD       R1,Save1          ; register values
21              ADD      R5,R5,#1          ; R5 <-- failure
22              RET
23 ;
24 EMPTY        .FILL    xC000             ; EMPTY contains -x4000
25 FULL         .FILL    xC005             ; FULL contains -x3FFB
26 Save1        .FILL    x0000
27 Save2        .FILL    x0000

```

图 8.11 栈协议

8.3 递归：运用得当时的强大技术

递归（recursion）是一种用函数自身来表述该函数的机制。有人把它比作“拽着自己的靴带把自己提起来”，因为乍看像魔法——当然，它并不是魔法。

使用得当时，递归的表达能力会为我们省去许多麻烦；随意使用时，它却会带来不必要的操作，延长执行时间，浪费能量。

这一机制十分重要，后面把抽象层次提升到高级语言编程之后，还会更详细地研究。但理解递归实现所需的一个关键概念，就是我们刚刚研究过的栈，因此现在不妨通过实例，说明何时适合使用递归，何时不适合。

我们将考察两个不明智的递归用法，再看一个能充分受益于递归表达能力的问题。

8.3.1 不好的例子之一：阶乘

说明递归最简单的例子，就是阶乘函数。下面的等式说明了一切：

$$n! = n \times (n-1)!$$

我们在用阶乘来表述阶乘本身！如何编写相应程序，稍后就会看到。

假设子程序 FACT (Factorial) 从 R0 得到正整数 n ，返回时将 $n!$ 放在 R0 中。0! 留作章末习题。

图 8.12 用图形表示这个递归子程序：FACT 是一个六边形，而六边形内部又有一个同样的六边形！之所以称为递归子程序，是因为 FACT 内部有一条 JSR FACT 指令。

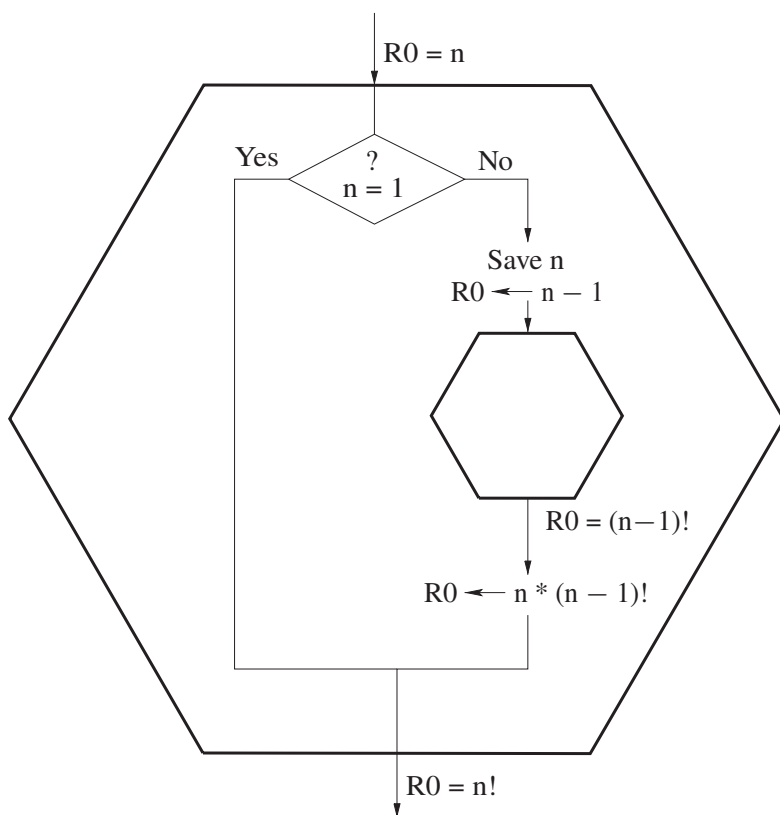


图 8.12 递归阶乘子程序的流程图

子程序首先检查 n 是否为 1。如果是，就已经完成，因为 $1! = 1$ 。必须强调，每个递归子程序都必须有这样的初始检测，判断是否还应执行递归调用。没有这个检测，子程序就会通过 JSR FACT 无穷无尽地调用自身！这显然不对。解决方法是在递归 JSR 指令之前进行检测。对 FACT 而言，如果 R0 为 1，就已经完成，因为 $1! = 1$ 。

如果 $n \neq 1$ ，则先保存 R1 中的值，再把 n 存入 R1，将 $n-1$ 装入 R0，执行 JSR FACT。FACT 返回时 R0 中为 $(n-1)!$ ，我们将它乘以 R1 中保存的 n ，得到 $n!$ 并放入 R0，然后将 R1 恢复成调用程序期望的值，执行 RET。

假如 LC-3 有 MUL 指令，FACT 的基本结构如下：

```

FACT    ST    R1, Save1 ; Callee save R1
        ADD   R1,R0,#-1 ; Test if R0=1
        BRZ   DONE     ; If R0=1, R0 also contains (1)!, so we are done
        ADD   R1,R0,#0  ; Save n in R1, to be used after we compute (n-1)!
        ADD   R0,R1, #-1 ; Set R0 to n-1, and then call FACT
B        JSR   FACT      ; On RET, R0 will contain (n-1)!
        MUL   R0,R0,R1   ; Multiply n times (n-1)!, yielding n! in R0
DONE     LD    R1, Save1 ; Callee restore R1
        RET
Save1    .BLKW 1

```

LC-3 实际上没有 MUL，因此乘法还需要另一次子程序调用。这里暂时忽略这个问题，以便专注于递归的本质。

不幸的是，刚写出的代码不能正确工作。为了说明原因，图 8.13 展示了我们希望得到的执行流程。主程序通过地址 A 处的 JSR 调用 FACT，开始执行标为 #1 的代码；在地址 B 处，FACT 又用 JSR FACT 调用自身，开始执行标为 #2 的代码，依此类推。

注意，主程序执行 JSR FACT 时，返回链接 A+1 保存在 R7 中。但在 #1 中，地址 B 处的 JSR FACT 把自己的返回链接 B+1 存入 R7，破坏了 A+1，于是再也无法回到主程序。糟糕！非常、非常糟糕！

解决方法是在执行 B 处的 JSR FACT 之前，将 A+1 压栈。后来返回 B+1 之后，再弹栈，将 A+1 装回 R7，然后执行 RET，返回主程序。

再看 ADD R1, R0, #0。它在 #1 中把 n 装入 R1，而在 #2 中把 n-1 装入 R1，覆盖了 #1 保存的 n。因此，控制流回到 #1、MUL R0, R0, R1 需要 n 时，它已经不存在了，又是一个严重问题！

这个问题也可以用栈解决。在把 n-1 装入 R0 之前，不把 n 移到 R1，而是将它压栈。等子程序返回、R0 中为 (n-1)!、需要 n 时，再将它弹出。

最后还要注意，子程序第一条指令将 R1 存入 Save1，RET 前的最后一条指令再把它恢复到 R1。这样，尽管子程序使用了 R1，从调用程序的角度看，调用前后 R1 的值仍相同。但由于子程序是递归的，在 B 处再次调用 FACT 时，R1 已不再是主程序中的原值，而是 ADD R1, R0, #0 最后存入的值。递归调用后的第一条指令又把这个值存入 Save1，覆盖主程序调用 FACT 时保存在那里的 R1 原值。

这个问题同样可以用栈解决：将 ST R1, Save1 替换为压栈，将 LD R1, Save1 替换为弹栈。

完成这些修改之后，如果 LC-3 有 MUL 操作码，递归子程序就能按预期工作。结果见图 8.14，几乎每条指令都有注释解释。

主程序以 $R0 = n$ 调用 FACT。#1 执行到 JSR FACT 时，R0 为 $n-1$ ，栈中有三个已压入的条目，如图 8.15a。#3 中的 JSR FACT 执行时，R0 为 $n-3$ ，栈中有九个条目，如图 8.15b。

此时你理应问：“ $n! = n \times (n-1)!$ 的表示如此优雅，为什么这是个不好的递归用法？”首先看看，为了压入和弹出这些栈元素，执行了多少条指令，浪费了多少时间。接着还应问：“有没有更好的方法来计算 $n!$ ？”

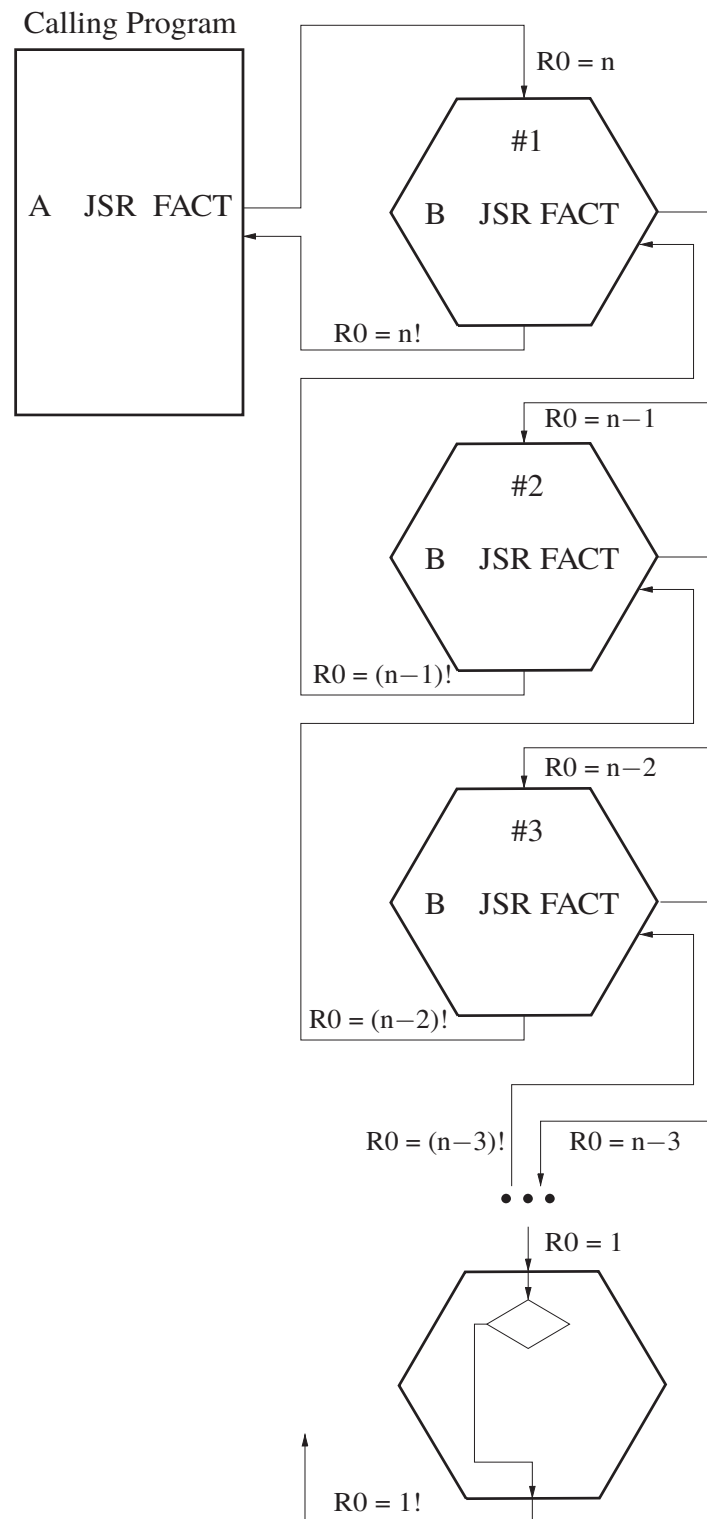


图 8.13 递归阶乘子程序的执行流程

```
FACT      ADD  R6,R6,#-1
          STR  R1,R6,#0    ; Push Caller's R1 on the stack, so we can use R1.
;
          ADD  R1,R0,#-1    ; If n=1, we are done since 1! = 1
          BRz  NO_RECURSE
;
          ADD  R6,R6,#-1
          STR  R7,R6,#0    ; Push return linkage onto stack
          ADD  R6,R6,#-1
          STR  R0,R6,#0    ; Push n on the stack
;
          ADD  R0,R0,#-1    ; Form n-1, argument of JSR
B         JSR  FACT
          LDR  R1,R6,#0    ; Pop n from the stack
          ADD  R6,R6,#1
          MUL  R0,R0,R1    ; form n*(n-1)!
;
          LDR  R7,R6,#0    ; Pop return linkage into R7
          ADD  R6,R6,#1
NO_RECURSE LDR  R1,R6,#0    ; Pop caller's R1 back into R1
          ADD  R6,R6,#1
          RET
```

图 8.14 递归子程序 FACT

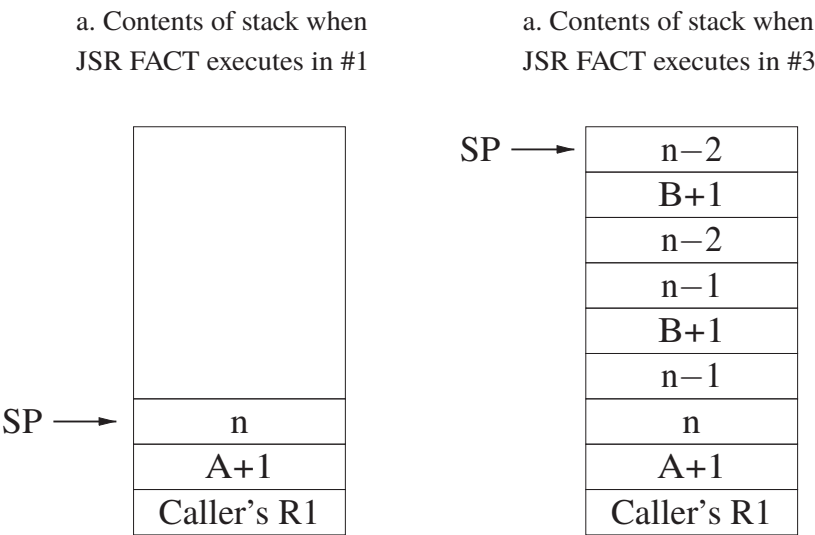


图 8.15 阶乘子程序执行过程中两个时刻的栈

考虑图 8.16 的另一种实现。

```

FACT      ST      R1, SAVE_R1
          ADD     R1, R0, #0
          ADD     R0, R0, #-1
          BRz     DONE
AGAIN     MUL     R1, R1, R0
          ADD     R0, R0, #-1 ; R0 gets next integer for MUL
          BRnp    AGAIN
DONE      ADD     R0, R1, #0 ; Move n! to R0
          LD      R1, SAVE_R1
          RET
SAVE_R1   .BLKW   1

```

图 8.16 以迭代方式实现 FACT，即不使用递归

8.3.2 Fibonacci：一个更糟的例子

递归的另一个不佳用法，是求 Fibonacci 数 $FIB(n)$ 。对所有非负整数，定义如下：

$$FIB(0) = 0, \quad FIB(1) = 1,$$

而当 $n > 1$ 时，

$$FIB(n) = FIB(n-1) + FIB(n-2).$$

表达式非常优雅，但执行时间却糟糕得很。

图 8.17 用图形表示递归子程序 FIB：整个子程序呈大写字母 F 的形状，而这个 F 内部又包含两个 F。

图 8.18 的递归子程序计算 $FIB(n)$ 。⁵

和所有递归子程序一样，首先需要检查**基本情况**。这里将 n 与 xFFFE 按位 AND，除了 $n = 0$ 和 $n = 1$ ，结果都不为零。如果 n 为 0 或 1，计算就基本完成：将 n 移入 R1，恢复 R2、R0 和 R7（实际上只有 R2 需要恢复），然后返回。

如果 n 既不是 0 也不是 1，就需要递归调用 FIB 两次，一次参数为 $n-1$ ，一次为 $n-2$ 。最后把 $FIB(n-1)$ 与 $FIB(n-2)$ 相加，将结果放入 R1，恢复 R2、R0 和 R7，返回。

注意， $FIB(n)$ 调用 $FIB(n-1)$ 和 $FIB(n-2)$ ； $FIB(n-1)$ 又要调用 $FIB(n-2)$ 和 $FIB(n-3)$ ； $FIB(n-2)$ 则调用 $FIB(n-3)$ 和 $FIB(n-4)$ 。这意味着 $FIB(n-2)$ 要计算两次， $FIB(n-3)$ 要计算三次。

思考：如果 $n = 10$ ，这个递归算法要重复计算多少次同一个 $FIB(5)$ ？

比较图 8.18 的递归算法和图 8.19 的非递归算法。后者执行得快得多！

⁵译注：原图开头的注释把递推式的第二项误写成 $FIB(n-1)$ ；应为 $FIB(n-2)$ ，与正文公式及实际指令一致。原图保留。

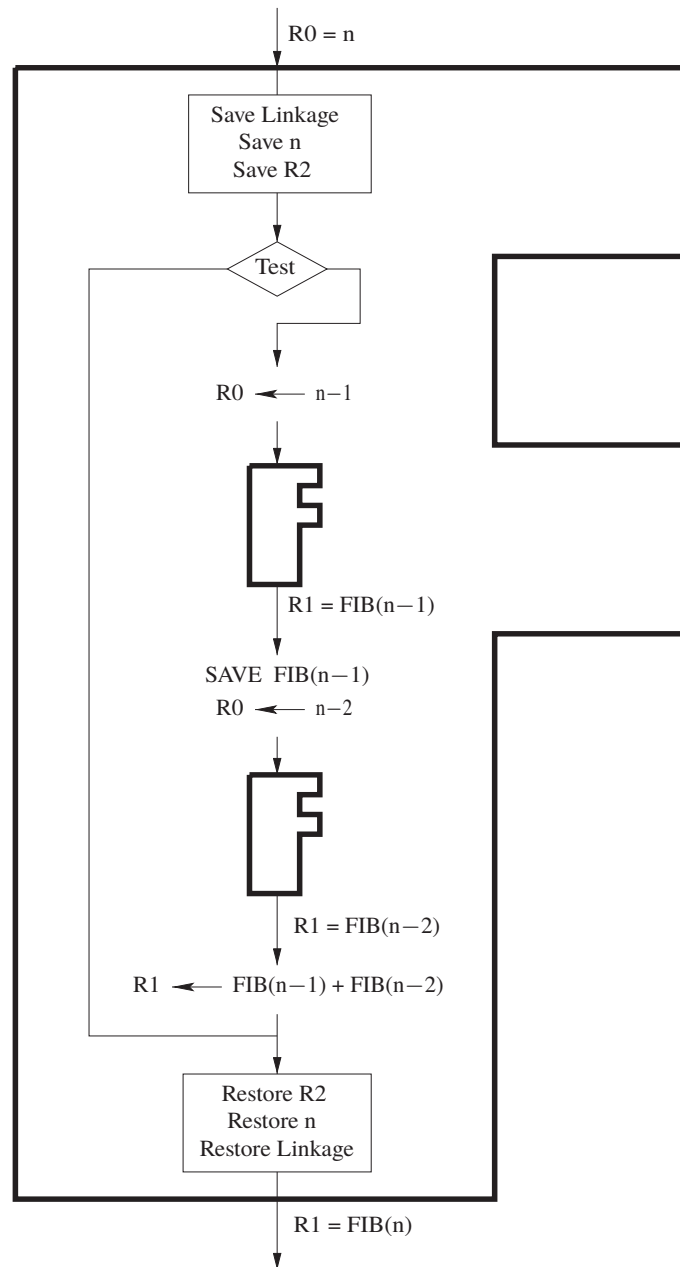


图 8.17 递归子程序 FIB 的图形表示


```

;FIB subroutine
; + FIB(0) = 0
; + FIB(1) = 1
; + FIB(n) = FIB(n-1) + FIB(n-1)
;
; Input is in R0
; Return answer in R1
;
FIB      ADD R6, R6, #-1
        STR R7, R6, #0 ; Push R7, the return linkage
        ADD R6, R6, #-1
        STR R0, R6, #0 ; Push R0, the value of n
        ADD R6, R6, #-1
        STR R2, R6, #0 ; Push R2, which is needed in the subroutine

; Check for base case
        AND R2, R0, #-2
        BRnp SKIP      ; Z=0 if R0=0,1
        ADD R1, R0, #0  ; R0 is the answer
        BRnzp DONE

; Not a base case, do the recursion
SKIP     ADD R0, R0, #-1
        JSR FIB         ; R1 = FIB(n-1)
        ADD R2, R1, #0  ; Move result before calling FIB again
        ADD R0, R0, #-1
        JSR FIB         ; R1 = FIB(n-2)
        ADD R1, R2, R1  ; R1 = FIB(n-1) + FIB(n-2)

; Restore registers and return
DONE     LDR R2, R6, #0
        ADD R6, R6, #1
        LDR R0, R6, #0
        ADD R6, R6, #1
        LDR R7, R6, #0
        ADD R6, R6, #1
        RET

```

图 8.18 Fibonacci 的递归实现

```

FIB      ST      R1, SaveR1
          ST      R2, SaveR2
          ST      R3, SaveR3
          ST      R4, SaveR4
          ST      R5, SaveR5
;
      NOT      R0, R0
      ADD      R0, R0, #1    ; R0 contains -n
      AND      R1, R1, #0    ; Suppose n=0
      ADD      R5, R1, R0    ; R5 = 0 -n
      BRz      DONE         ; if n=0, done almost
      AND      R3, R2, #0    ; if n>0, set up R3 = FIB(0) = 0
      ADD      R1, R3, #1    ; Suppose n=1
      ADD      R5, R1, R0    ; R5 = 1-n
      BRz      DONE         ; if n=1, done almost
      ADD      R4, R1, #0    ; if n>1, set up R4 = FIB(1) = 1
;
AGAIN    ADD      R1, R1, #1    ; We begin the iteration of FIB(i)
          ADD      R2, R3, #0    : R2= FIB(i-2)
          ADD      R3, R4, #0    : R3= FIB(i-1)
          ADD      R4, R2, R3    ; R4 = FIB(i)
          ADD      R5, R1, R0    ; is R1=n ?
          BRn      AGAIN
;
          ADD      R0, R4, #0    ; if n>1, R0=FIB(n)
          BRnzp    RESTORE
DONE     ADD      R0, R1, #0    ; if n=0,1, FIB(n)=n
RESTORE  LD       R1, SaveR1
          LD       R2, SaveR2
          LD       R3, SaveR3
          LD       R4, SaveR4
          LD       R5, SaveR5
          RET

```

图 8.19 Fibonacci 的迭代解法

8.3.3 迷宫：一个好的例子

不愿用递归计算阶乘或 Fibonacci 数，原因很简单：迭代算法已经足够容易理解，而且没有递归带来的巨大执行时间代价。不过，有时递归优美的表达方式，确实有助于处理复杂问题。下面的迷宫问题就是一例：给定一个迷宫及其中的起始位置，编写程序判断能否从该位置走出迷宫。

迷宫 迷宫可以是任意的 $n \times m$ 大小。例如，图 8.20 是一个 6×6 的迷宫。

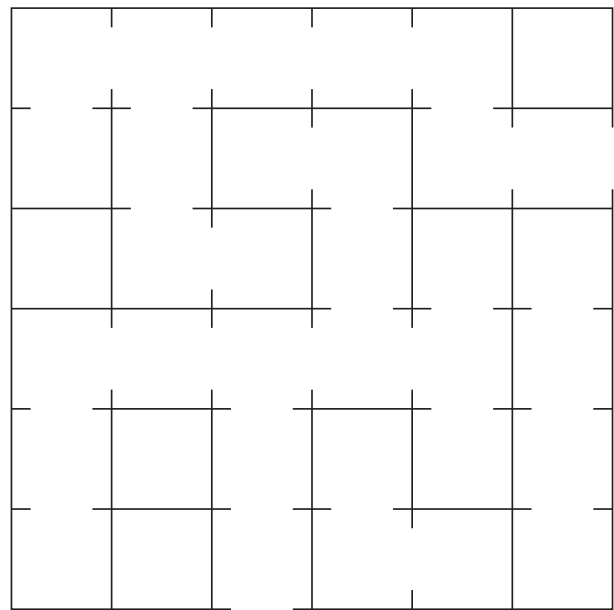


图 8.20 一个迷宫实例

36 个格子中的每一个，都可以用以下信息来描述：北、东、南、西四个方向是否有门，以及是否有通向外界的门。每个格子用一个存储字表示，如图 8.21。

Bit[4]=1 if there is a door to the outside world; Bit[4]=0 if no door.
Bit[3]=1 if there is a door to the cell to the north; Bit[3]=0 if no door.
Bit[2]=1 if there is a door to the cell to the east; Bit[2]=0 if no door.
Bit[1]=1 if there is a door to the cell to the south; Bit[1]=0 if no door.
Bit[0]=1 if there is a door to the cell to the west; Bit[0]=0 if no door.

图 8.21 迷宫中各格子的表示方式

这些字以行主序（row major order）存储：先存第一行，再存第二行，然后第三行，依此类推。图 8.22 给出了这个 6 × 6 迷宫的完整表示。

寻找迷宫出口的递归子程序 我们要设计算法，判断能否从给定起点走出迷宫。尝试过程中可能经过错综复杂的路径，记录所有细节似乎十分困难。递归让我们不必亲自跟踪这些路径！图 8.23 用图形表示递归子程序 FIND_EXIT，用于判断能否走出迷宫。整个子程序是一个八边形，内部又有四个八边形，表示对 FIND_EXIT 的递归调用。能走出去就返回 R1=1，不能则返回 R1=0。

算法如下工作：在每个格子中，先问它是否有通向外界出口；如果有，就令 R1 为 1 并返回。如果没有，就考虑是否应尝试北、东、南、西的相邻格子。显然，要尝试某个方向，就必须有通向那个方向的门。

此外，还必须防止进入无限循环。例如，沿着门先向北走一格，再向东、向南、向西各走一格，就回到了起点。为防止这种情况，我们在访问过的格子中留下一个“面包屑”，也就是访问标记，只对从未访问过的格子执行 JSR FIND_EXIT。

因此，算法是：

- a. 在当前格子中检查能否直接出去。若能，工作完成，令 R1=1 并返回。

```
00          .ORIG x5000
01 MAZE     .FILL x0006
02          .FILL x0007
03          .FILL x0005
04          .FILL x0005
05          .FILL x0003
06          .FILL x0000
07 ; second row: indices 6 to 11
08          .FILL x0008
09          .FILL x000A
0A          .FILL x0004
0B          .FILL x0003
0C          .FILL x000C
0D          .FILL x0015
0E ; third row: indices 12 to 17
0F          .FILL x0000
10          .FILL x000C
11          .FILL x0001
12          .FILL x000A
13          .FILL x0002
14          .FILL x0002
15 ; fourth row: indices 18 to 23
16          .FILL x0006
17          .FILL x0005
18          .FILL x0007
19          .FILL x000D
1A          .FILL x000B
1B          .FILL x000A
1C ; fifth row: indices 24 to 29
1D          .FILL x000A
1E          .FILL x0000
1F          .FILL x000A
20          .FILL x0002
21          .FILL x0008
22          .FILL x000A
23 ; sixth row: indices 30 to 35
24          .FILL x0008
25          .FILL x0000
26          .FILL x001A
27          .FILL x000C
28          .FILL x0001
29          .FILL x0008
2A          .END
```

图 8.22 图 8.20 迷宫的完整表示

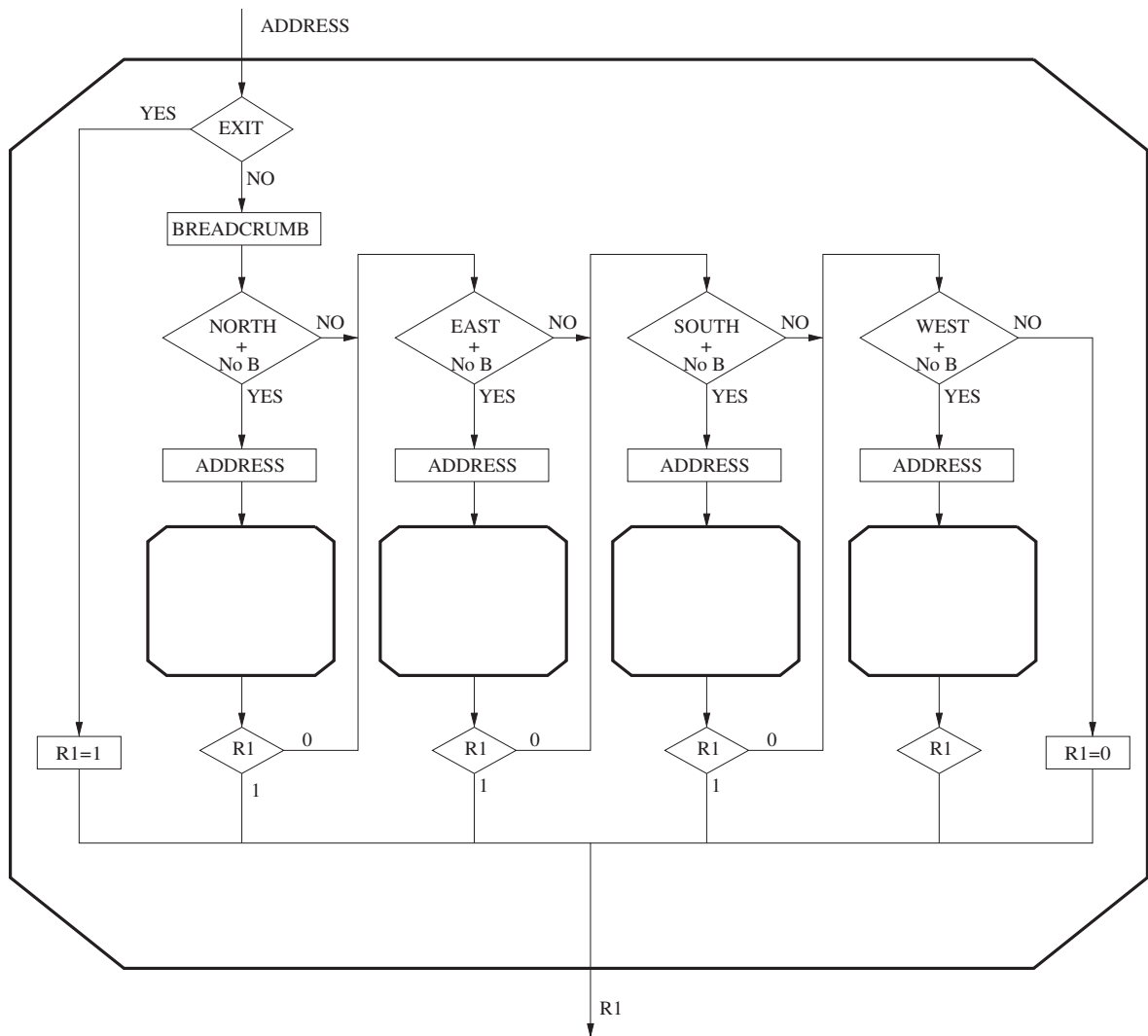


图 8.23 寻找迷宫出口的递归子程序的图形表示

- b. 否则，在当前格子留下访问标记，将表示该格子的字的位 [15] 设为 1。
- c. 检查两个问题：北面是否有门？北面的格子是否从未访问过？若都满足，将地址改成北面格子的地址，然后执行 JSR FIND_EXIT。北面格子的地址等于当前地址减 6。为什么是 6？因为格子按行主序存储，而迷宫有 6 列。
- d. 如果任一条件不满足，或者向北尝试失败，就检查东面是否有门，以及东面格子是否从未访问。若都满足，将当前地址加 1，得到东面格子的地址，再执行 JSR FIND_EXIT。
- e. 如果向东仍走不出去，就同样尝试南面；再不成功，就尝试西面。
- f. 如果西面没有通向未访问格子的门，或者虽然有门、也未访问，但尝试仍失败，就结束：无法从起始位置走出迷宫。令 R1=0 并返回。

图 8.24 给出了根据起始地址判断能否走出迷宫的递归算法。

```

; Recursive subroutine that determines if there is a path from current cell
; to the outside world.
; input: R0, current cell address
; output: R1, YES (1) or NO (0)
        .ORIG x4000

01 FIND_EXIT    ; save modified registers into the stack.
02              ADD R6, R6, #-1
03              STR R2, R6, #0    ; R2 holds the cell data of the caller
04              ADD R6, R6, #-1
05              STR R3, R6, #0    ; R3 holds the cell address of the caller
06              ADD R6, R6, #-1
07              STR R7, R6, #0    ; R7 holds the PC of the caller
08
09              ; Move cell address to R3, since we need to use R0
0A              ; as the input to recursive subroutine calls.
0B              ADD R3, R0, #0
0C
0D              ; If the exit is in this cell, return YES
0E              LDR R2, R0, #0    ; R2 now holds the current cell data
0F              LD  R7, EXIT_MASK
10              AND R7, R2, R7
11              BRnp DONE_YES
12
13              ; Put breadcrumb in the current cell.
14              LD  R7, BREADCRUMB
15              ADD R2, R2, R7
16              STR R2, R0, #0
17
18              ; check the north cell for a path to exit
19 CHECK_NORTH  LD  R7, NORTH_MASK
1A              AND R7, R2, R7
1B              BRz CHECK_EAST    ; If north is blocked, check east
1C              LDR R7, R3, #-6
1D              BRn CHECK_EAST    ; If a breadcrumb in the north cell, check east
1E              ADD R0, R3, #-6
1F              JSR FIND_EXIT    ; Recursively check the north cell
20              ADD R1, R1, #0
21              BRp DONE_YES     ; If a path from north cell found, return YES
22
23              ; check the north cell for a path to exit
24 CHECK_EAST  LD  R7, EAST_MASK
25              AND R7, R2, R7
26              BRz CHECK_SOUTH   ; If the way to east is blocked, check south
27              LDR R7, R3, #1
28              BRn CHECK_SOUTH   ; If a breadcrumb in the east cell, check south
29              ADD R0, R3, #1
2A              JSR FIND_EXIT    ; Recursively check the east cell
2B              ADD R1, R1, #0
2C              BRp DONE_YES     ; If a path from east cell found, return YES
2D

```

图 8.24 判断迷宫是否存在出口的递归子程序

```

2E          ; check the south cell for a path to exit
2F CHECK_SOUTH LD R7, SOUTH_MASK
30          AND R7, R2, R7
31          BRz CHECK_WEST      ; If the way to south is blocked, check west
32          LDR R7, R3, #6
33          BRn CHECK_WEST      ; If a breadcrumb in the south cell, check west
34          ADD R0, R3, #6
35          JSR FIND_EXIT        ; Recursively check the south cell
36          ADD R1, R1, #0
37          BRp DONE_YES        ; If a path from south cell found, return YES
38
39          ; check the west cell for a path to exit
3A CHECK_WEST LD R7, WEST_MASK
3B          AND R7, R2, R7
3C          BRz DONE_NO         ; If the way to west is blocked, return NO
3D          LDR R7, R3, #-1
3E          BRn DONE_NO         ; If a breadcrumb in the west cell, return NO
3F          ADD R0, R3, #-1
40          JSR FIND_EXIT        ; Recursively check the west cell
41          ADD R1, R1, #0
42          BRp DONE_YES        ; If a path from west cell found, return YES
43
44 DONE_NO   AND R1, R1, #0
45          BR RESTORE
46
47 DONE_YES  AND R1, R1, #0
48          ADD R1, R1, #1
49
4A RESTORE  ADD R0, R3, #0 ; restore R0 from R3
4B          ; restore the rest of the modified registers from the stack.
4C          LDR R7, R6, #0
4D          ADD R6, R6, #1
4E          LDR R3, R6, #0
4F          ADD R6, R6, #1
50          LDR R2, R6, #0
51          ADD R6, R6, #1
52          RET
53
54 BREADCRUMB .FILL x8000
55 EXIT_MASK  .FILL x0010
56 NORTH_MASK .FILL x0008
57 EAST_MASK  .FILL x0004
58 SOUTH_MASK .FILL x0002
59 WEST_MASK  .FILL x0001
5A          .END

```

图 8.24 判断迷宫是否存在出口的递归子程序（续）

8.4 队列

接下来的数据结构是**队列** (queue)。回想栈的定义性特征 LIFO：最后压入的元素，最先弹出。抽象数据类型队列的定义性特征则是**先进先出** (First In, First Out, FIFO)。队列就像秩序良好的超市或售票窗口前的队伍，先排队的人先得到服务。在数据结构中，这意味着要跟踪存储结构的两端：**FRONT** 指针用于服务，即从队首移除元素；**REAR** 指针用于进入，即从队尾插入元素。

图 8.25 给出为队列分配的六个连续存储单元，地址从 x8000 到 x8005。我们约定 **FRONT** 指向队首元素之前的那个位置，**REAR** 指向最近加入的元素所在位置。用 R3 作为 **FRONT** 指针，R4 作为 **REAR** 指针。

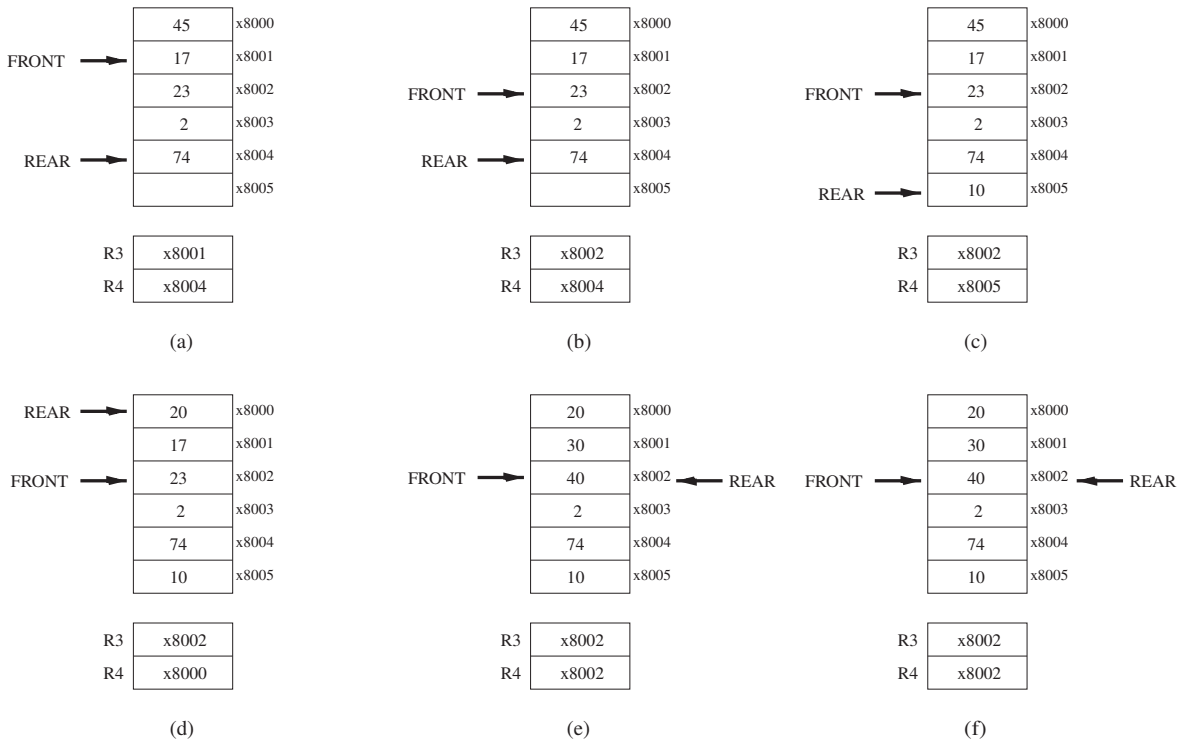


图 8.25 分配在地址 x8000 至 x8005 的队列

图 8.25a 的队列已经插入过五个值。**FRONT** 为 x8001，说明 x8000 中的 45 和 x8001 中的 17 已被移除，现在的队首元素是 x8002 中的 23。

注意，虽然 45 和 17 已经移除，但仍留在原来的存储单元中。这和前面讨论的栈一样，是装入指令的性质：通过装入指令移除一个值，并不会擦除存储单元，只会把内容复制到目的寄存器。不过，**FRONT** 已为 x8001，只要 x8000 至 x8005 按队列的 FIFO 规则访问，就无法再从 x8000 和 x8001 装入这两个值。

8.4.1 基本操作：从队首移除，向队尾插入

由于 **FRONT** 指向队首元素之前的位置，移除时先将 **FRONT** 加 1，再装入新地址中的值。本例中，下一个被移除的是队首 x8002 中的 23。下面的代码将它移除，得到图 8.25b：

```
ADD R3, R3, #1
LDR R0, R3, #0
```

由于 REAR 为 x8004，最后进入队列的值是 74。图 8.25b 中剩余的值是 2 和 74。要再向队尾插入一个元素，例如 10，先让 R0 包含这个值，再执行：

```
ADD R4, R4, #1
STR R0, R4, #0
```

结果如图 8.25c。

8.4.2 回绕

乍看之下，似乎已经不能再插入元素了，其实不然！从队列移除一个值后，原来的位置就可以重新存放其他元素。我们通过让可用存储区域**回绕**（wrap-around）来利用这些位置。例如，要插入 20 时，x8000 已经空闲，因为原来的 45 已被移除，所以可以把 20 存入 x8000，如图 8.25d。

实现回绕，需要让移除和插入算法检查 FRONT、REAR 是否为 x8005。若准备插入而 REAR 为 x8005，就知道已经到达可用存储区域末尾，必须查看 x8000 是否可用。若准备移除，也要先检查 FRONT 是否为 x8005；如果是，队首就在 x8000。

因此，移除和插入代码都必须包含回绕检测。移除代码变为：

```
LD    R2, LAST
ADD   R2, R3, R2
BRnp  SKIP_1
LD    R3, FIRST
BR    SKIP_2
SKIP_1 ADD   R3, R3, #1
SKIP_2 LDR   R0, R3, #0 ; R0 gets the front of the queue
RET
LAST   .FILL x7FFB      ; LAST contains the negative of 8005
FIRST  .FILL x8000
```

插入代码类似。如果 REAR 为 x8005，在向队尾插入元素之前，需要先把 R4 设为 x8000。代码如下：

```
LD    R2, LAST
ADD   R2, R4, R2
BRnp  SKIP_1
LD    R4, FIRST
BR    SKIP_2
SKIP_1 ADD   R4, R4, #1
SKIP_2 STR   R0, R4, #0 ; R0 gets the front of the queue
RET
LAST   .FILL 7FFB      ; LAST contains the negative of 8005
FIRST  .FILL x8000
```

8.4.3 队列能存储多少个元素？

再看图 8.25d，队列中有四个值：2、74、10 和 20。如果向队尾插入 30 和 40，就得到图 8.25e。R3 和 R4 都是 x8002，队列已满。现在开始从队首移除元素。先移除队首的 2，R3 变为 x8003；再移除剩余五个元素，就得到图 8.25f。注意，e 和 f 中 FRONT 与 REAR 的值完全相同，但 e 是满队列，f 却是空队列！这显然不能接受。

解决方法是：分配 n 个元素的空间，只允许队列存储 $n - 1$ 个元素。如果插入第 n 个元素会使 FRONT 等于 REAR，就禁止这次插入；已有 $n - 1$ 个元素时，就宣布队列已满。

再看图 8.25d，从队首到队尾依次是 2、74、10、20，x8001 和 x8002 两个位置为空。可以把 30 插入 x8001，得到图 8.26a。这是第五个元素。由于队列只分配了六个字，我们现在就宣布队列已满，不允许再插入第六个元素。接着持续移除，直到队列为空，如图 8.26b。现在，满队列和空队列不会再混淆，因为队列为空时才有 FRONT=REAR。

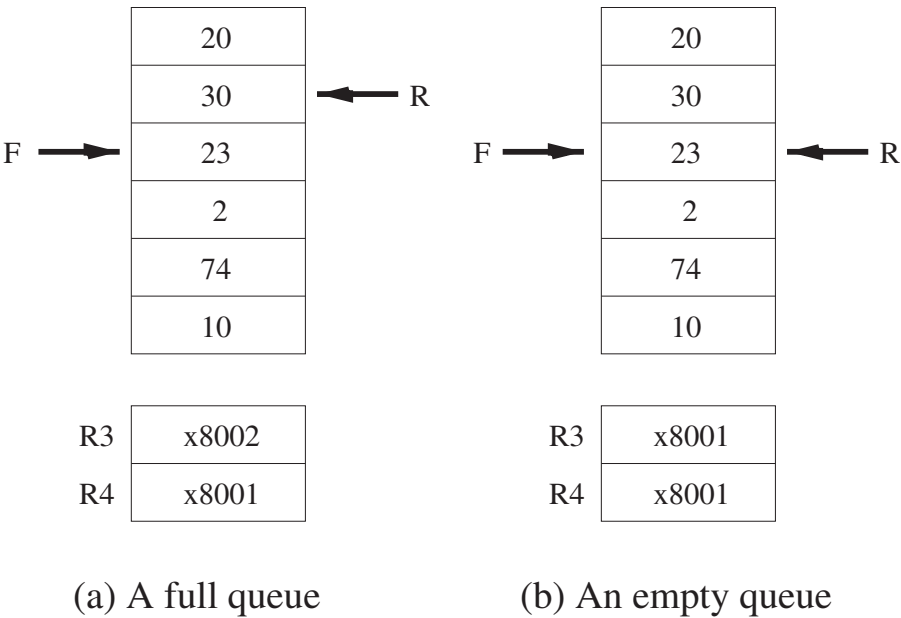


图 8.26 满队列与空队列

8.4.4 下溢和溢出检测

和栈一样，队列有元素时才能移除，未滿时才能插入。空队列上移除元素会下溢，满队列上插入元素会溢出。如果使用子程序管理队列，这两种情况都需要向调用程序报告成功或失败。和栈一样，用 R5 报告。

下溢检测很直接。由图 8.26 可知，FRONT=REAR 时队列为空，因此检测代码是：

```
        AND    R5,R5,#0    ; Initialize R5 to 0
        NOT    R2,R3
        ADD    R2,R2,#1    ; R2 contains negative of R3
        ADD    R2,R2,R4
        BRz    UNDERFLOW
        ; code to remove the front of the queue and return success.
UNDERFLOW ADD    R5,R5,#1
        RET
```

也就是说，先检查 R3 是否等于 R4。若相等，则跳到 UNDERFLOW，将 R5 设为失败，恢复 R1，然后返回；否则，继续执行移除队首元素的代码。

溢出检测类似。向队尾插入时，先推进 REAR；若这使 FRONT=REAR，就说明队列已经有 $n-1$ 个元素，不能继续插入。将 REAR 退回原值，把 R5 设为 1，然后返回。

8.4.5 完整实现

最后，用一个完整子程序总结队列操作：它支持从队首移除、向队尾插入；指针到达最后一个位置时回绕；返回时报告成功 (R5=0)，或者因下溢、溢出而失败 (R5=1)。

具体采用图 8.25 的队列，分配地址 x8000 至 x8005，其中 FIRST 为 x8000，LAST 为 x8005。

插入时，先确认队列未满。为此，先推进 REAR (R4)，再检测 REAR=FRONT。如果原来的 REAR 为 x8005，就通过将它设为 x8000 来推进，即进行回绕。队列已满时，必须将 REAR 恢复原值，并报告失败 (R5=1) 后返回；未满时，将 R0 中待插入的值存入 REAR 所指单元，报告成功 (R5=0) 后返回。

移除时，先检测 REAR=FRONT，确认队列非空。若相等，队列为空，报告失败并返回。否则，检查 FRONT 是否为 x8005；是就设为 x8000，不是就加 1。然后从 FRONT 指定的单元将值装入 R0，报告成功并返回。

完整子程序见图 8.27。

```

00 ;Input: R0 for item to be inserted, R3 is FRONT, R4 is REAR
01 ;Output: R0 for item to be removed
02
03 INSERT      ST      R1,SaveR1      ; Save register we need
04             AND      R5,R5,#0      ; Set R5 to success code
05             ; Initialization complete
06             LD      R1,NEG_LAST
07             ADD      R1,R1,R4      ; R1 = REAR MINUS x8005
08             BRnp     SKIP1         ; SKIP WRAP AROUND
09             LD      R4,FIRST       ; WRAP AROUND, R4=x8000
0A             BR      SKIP2
0B SKIP1       ADD      R4,R4,#1      ; NO WRAP AROUND, R4=R4+1
0C SKIP2       NOT      R1,R4
0D             ADD      R1,R1,#1      ; R1= NEG REAR
0E             ADD      R1,R1,R3      ; R1= FRONT-REAR
0F             BRz      FULL
10             STR      R0,R4,#0      ; DO THE INSERT
11             BR      DONE
12 FULL        LD      R1,NEG_FIRST
13             ADD      R1,R1,R4      ; R1 = REAR MINUS x8000
14             BRnp     SKIP3
15             LD      R4,LAST        ; UNDO WRAP AROUND, REAR=x8005
16             BR      SKIP4
17 SKIP3       ADD      R4,R4,#-1     ; NO WRAP AROUND, R4=R4-1
18 SKIP4       ADD      R5,R5,#1      ; R5=FAILURE
19             BR      DONE
1A
1B REMOVE      ST      R1,SaveR1      ; Save register we need
1C             AND      R5,R5,#0      ; Set R5 to success code
1D             ; Initialization complete
1E             NOT      R1,R4
1F             ADD      R1,R1,#1      ; R1= NEG REAR
20             ADD      R1,R1,R3      ; R1= FRONT-REAR
21             BRz      EMPTY
22             LD      R1, NEG_LAST
23             ADD      R1,R1,R3      ; R1= FRONT MINUS x8005
24             BRnp     SKIP5
25             LD      R3, FIRST      ; R3=x8000
26             BR      SKIP6
27 SKIP5       ADD      R3,R3,#1      ; R3=R3+1
28 SKIP6       LDR      R0,R3,#0      ; DO THE REMOVE
29             BR      DONE
2A EMPTY       ADD      R5,R5.#1      ; R5=FAILURE
2B DONE        LD      R1,SaveR1      ; Restore register
2C             RET
2D FIRST       .FILL x8000
2E NEG_FIRST   .FILL x8000
2F LAST        .FILL x8005
30 NEG_LAST    .FILL x7FFB
31 SaveR1      .BLKW 1

```

图 8.27 完整的队列子程序

8.5 字符串

最后一种数据结构：字符串！

本章最后研究**字符串** (character string)：由键盘字符（字母、数字及其他符号）组

成的序列，被组织成一维 ASCII 码数组，通常表示姓名、地址或其他字母数字字符串。图 8.28 表示已故的斯坦福著名教授 Bill Linvill 的姓名，占用从 x5000 开始的 13 个连续存储字。每个字母的 ASCII 码单独占一个字。

x5000	x0042
	x0069
	x006C
	x006C
	x0020
	x004C
	x0069
	x006E
	x0076
	x0069
	x006C
	x006C
	x0000

图 8.28 表示姓名 Bill Linvill 的字符串

ASCII 码占一个字节，因此每个单元的高位补上 x00。例如，大写 B 的 ASCII 码是 x42，所以 x5000 包含 x0042。姓名中的 11 个字母各占一个字，名和姓之间的空格 ASCII 码 x20 占一个字，最后还需要空字符 x0000 表示字符串结束，共 13 个单元。不同字符串长度不同，但这不是问题：分配所需数量的存储字，再以 x0000 标记结束即可。

字符串的一种常见用途，是标识与某个人相关的一组信息。图 8.29 给出了公司某位员工的信息，通常称为**人事记录**（personnel record）。

这条记录从 x4000 开始，占六个连续存储字：

1. 第一个字包含姓氏字符串的起始地址。x4000 中的指针为 x6000；从这里开始的六字字符串包含 Jones 的 ASCII 码，最后以空字符结束。
2. 第二个字位于 x4001，包含名字字符串的指针。本例名字是 Mary，字符串从 x4508 开始。
3. 第三个字位于 x4002，包含指向九位社会保障号码的指针 xCA9B。该号码用来唯一标识在美国工作的个人。
4. 第四个字位于 x4003，保存薪资，以千美元为单位。

x4000	x6000	x6000	x004A	x4508	x004D	xCA9B	x0030	x8E25	x0045
x4001	x4508	x6001	x006F	x4509	x0061	xCA9C	x0031	x8E26	x006E
x4002	xCA9B	x6002	x006E	x450A	x0072	xCA9D	x0032	x8E27	x0067
x4003	\$84,000	x6003	x0065	x450B	x0079	xCA9E	x0036	x8E28	x0069
x4004	4	x6004	x0073	x450C	x0000	xCA9F	x0035	x8E29	x006E
x4005	x8E25	x6005	x0000			xCAA0	x0034	x8E2A	x0065
						xCAA1	x0036	x8E2B	x0065
						xCAA2	x0032	x8E2C	x0072
						xCAA3	x0031	x8E2D	x0000

图 8.29 Mary Jones 的人事记录

5. 第五个字保存她在公司工作的年数。
6. 第六个字包含指针 x8E25，指向职位字符串，本例为 Engineer。

也就是说，这位员工名为 Mary Jones，社会保障号码为 012654621，职位是 Engineer，在公司工作了四年，年薪 84,000 美元。

可以编写程序，检查员工记录并查找各种人事信息。例如，要知道某位员工的薪资，可以逐条检查记录。程序调用子程序，将某条记录的社会保障号码字符串与待查找者的号码逐字符比较。如果所有字符都匹配，子程序返回成功代码（R5=0），程序继续读取记录第四个字中的薪资；如果未全部匹配，则返回失败代码（R5=1），程序再用另一位员工社会保障号码的起始地址调用子程序。

图 8.30 的子程序比较两个字符串，判断它们是否完全相同。

另一个例子：用字符串表示“整数” 还可以用字符串表示任意长的整数。例如图 8.31 是表示整数 79,245 的字符串。

图 8.32 的子程序检查这类字符串，确认其中所有 ASCII 码确实都表示十进制数字。如果所有条目都是十进制数字的 ASCII 码，即在 x30 至 x39 之间，子程序返回成功（R5=0）；否则返回失败（R5=1）。

```
STRCMP      ST      R0, SaveR0
            ST      R1, SaveR1
            ST      R2, SaveR2
            ST      R3, SaveR3
;
            AND     R5, R5, #0    ; R5 <-- Match
;
NEXTCHAR    LDR     R2, R0, #0    ; R2 contains character from 1st string
            LDR     R3, R1, #0    ; R3 contains character from 2nd string
            BRnp    COMPARE       ; String is not done, continue comparing
            ADD     R2, R2, #0
            BRz     DONE          ; If both strings done, match found
COMPARE     NOT     R2, R2
            ADD     R2, R2, #1    ; R2 contains negative of character
            ADD     R2, R2, R3    ; Compare the 2 characters
            BRnp    FAIL         ; Not equal, no match
            ADD     R0, R0, #1
            ADD     R1, R1, #1
            BRnzp   NEXTCHAR     ; Move on to next pair of characters
;
FAIL        ADD     R5, R5, #1    ; R5 <-- No match
;
DONE        LD      R0, SaveR0
            LD      R1, SaveR1
            LD      R2, SaveR2
            LD      R3, SaveR3
            RET
;
SaveR0      .BLKW   1
SaveR1      .BLKW   1
SaveR2      .BLKW   1
SaveR3      .BLKW   1
```

图 8.30 比较两个字符串的子程序

x0037
x0039
x0032
x0034
x0035
x0000

图 8.31 表示整数 79,245 的字符串，每位十进制数字对应一个 ASCII 码


```

; Input: R0 contains the starting address of the character string
; Output: R5=0, success; R5=1, failure.
;
TEST_INTEGER    ST    R1,SaveR1    ; Save registers needed by subroutine
                ST    R2,SaveR2
                ST    R3,SaveR3
                ST    R4,SaveR4
;
                AND    R5,R5,#0    ; Initialize success code to R5=0, success
                LD     R2,ASCII_0   ; R2=xFFD0, the negative of ASCII code x30
                LD     R3,ASCII_9   ; R3=xFFC7, the negative of ASCII code x39
;
NEXT_CHAR       LDR    R1,R0,#0    ; Load next character
                BRz    SUCCESS
                ADD    R4,R1,R2
                BRn    BAD          ; R1 is less than x30, not a decimal digit
                ADD    R4,R1,R3
                BRp    BAD          ; R1 is greater than x39, not a decimal digit
                ADD    R0,R0,#1    ; Character good! Prepare for next character
                BR     NEXT_CHAR
;
                BAD    ADD    R5,R5,#1    ; R5 contains failure code
SUCCESS         LD     R4,SaveR4    ; Restore registers
                LD     R3,SaveR3
                LD     R2,SaveR2
                LD     R1,SaveR1
                RET
ASCII_0         .FILL xFFD0
ASCII_9         .FILL xFFC7
SaveR1         .BLKW 1
SaveR2         .BLKW 1
SaveR3         .BLKW 1
SaveR4         .BLKW 1

```

图 8.32 判断字符串是否表示整数的子程序

习题

- 8.1 栈的定义性特征是什么？
- 8.2 相比图 8.8 的模型，用图 8.9 的模型实现栈有什么优点？
- 8.3 为 LC-3 ISA 扩展了以下压栈和弹栈指令：Push Rn 将寄存器 n 中的值压栈；Pop Rn 从栈中取出一个值，装入 Rn。下图给出了执行六次栈操作前后，LC-3 八个寄存器的快照。确定 (a) 至 (d)。

BEFORE			AFTER		
R0	x0000	PUSH R4	R0	x1111	
R1	x1111	PUSH (a)	R1	x1111	
R2	x2222	POP (b)	R2	x3333	
R3	x3333	PUSH (c)	R3	x3333	
R4	x4444	POP R2	R4	x4444	
R5	x5555	POP (d)	R5	x5555	
R6	x6666		R6	x6666	
R7	x7777		R7	x4444	

- 8.4 编写函数，实现另一个栈操作 `peek`。它返回栈顶元素的值，但不将该元素从栈中移除。`peek` 还应检查下溢错误。为什么不必检查溢出错误？
- 8.5 如果按图 8.8 的模型实现栈，怎样检测下溢和溢出？重写 `PUSH` 和 `POP`，使其模拟图 8.8 那样的栈，即每次操作时数据项都会移动。
- 8.6 重写 `PUSH` 和 `POP`，使其所操作的栈中每个元素占两个存储单元。
- 8.7 重写 `PUSH` 和 `POP`，使其支持任意大小的栈元素。
- 8.8 对一个栈依次执行以下操作：

PUSH A, PUSH B, POP, PUSH C, PUSH D, POP,
PUSH E, POP, POP, PUSH F

- a. 执行 `PUSH F` 后，栈中有什么？
- b. 哪个时刻栈中的元素最多？不移除先前操作留下的元素，继续执行：

PUSH G, PUSH H, PUSH I, PUSH J, POP,
PUSH K, POP, POP, POP, PUSH L, POP, POP,
PUSH M

- c. 现在栈中有什么？

- 8.9 栈的输入流，是按压入顺序排列的所有压栈元素。习题 8.8 的输入流是 ABCDEFGHIJKLM。输出流则是按弹出顺序排列的所有弹栈元素。
- a. 习题 8.8 的输出流是什么？提示：BDE...
- b. 输入流为 ZYXWVUTSR 时，构造一个压栈和弹栈操作序列，使输出流为 YXVUWZSRT。
- c. 输入流为 ZYXW 时，能产生多少种不同的输出流？

8.10 [*] 地图上相邻城市采用不同颜色，会更容易辨认城市边界。例如，在得克萨斯州的地图上，不应把 Austin 和 Pflugerville 涂成同一种颜色，否则会模糊二者的边界。

下面的递归子程序 EXAMINE 检查表示地图的数据结构，判断是否存在颜色相同的相邻城市。每个节点包含该城市的颜色，以及与它相邻的城市的地址。

如果不存在同色相邻城市, EXAMINE 返回 $R1=0$; 如果至少存在一对, 则返回 $R1=1$ 。主程序在执行 JSR EXAMINE 前, 将表示其中一座城市的节点地址放入 $R0$ 。

```

        .ORIG x4000
EXAMINE ADD R6, R6, #-1
        STR R0, R6, #0
        ADD R6, R6, #-1
        STR R2, R6, #0
        ADD R6, R6, #-1
        STR R3, R6, #0
        ADD R6, R6, #-1
        STR R7, R6, #0

        AND R1, R1, #0 ; Initialize output R1 to 0
        LDR R7, R0, #0
        BRn RESTORE    ; Skip this node if it has already been visited

        LD  R7, BREADCRUMB
        STR R7, R0, #0 ; Mark this node as visited
        LDR R2, R0, #1 ; R2 = color of current node
        ADD R3, R0, #2

AGAIN   LDR R0, R3, #0 ; R0 = neighbor node address
        BRz RESTORE
        LDR R7, R0, #1
        NOT R7, R7     ; <-- Breakpoint here
        ADD R7, R7, #1
        ADD R7, R2, R7 ; Compare current color to neighbor's color
        BRz BAD
        JSR EXAMINE    ; Recursively examine the coloring of next neighbor
        ADD R1, R1, #0
        BRp RESTORE    ; If neighbor returns R1=1, this node should return R1=1
        ADD R3, R3, #1
        BR  AGAIN      ; Try next neighbor

BAD     ADD R1, R1, #1
RESTORE LDR R7, R6, #0
        ADD R6, R6, #1
        LDR R3, R6, #0
        ADD R6, R6, #1
        LDR R2, R6, #0
        ADD R6, R6, #1
        LDR R0, R6, #0
        ADD R6, R6, #1
        RET

BREADCRUMB .FILL x8000
        .END

```

你的任务是构造表示某张地图的数据结构。执行 JSR EXAMINE 前, $R0$ 被设为 $x6100$, 即某个节点的地址, 并在 $x4012$ 设置断点。下表给出 EXAMINE 运行时每次遇到断点所收集的相关信息。

PC	R0	R2	R7
x4012	x6200	x0042	x0052
x4012	x6100	x0052	x0042
x4012	x6300	x0052	x0047
x4012	x6200	x0047	x0052
x4012	x6400	x0047	x0052
x4012	x6100	x0052	x0042
x4012	x6300	x0052	x0047
x4012	x6500	x0052	x0047
x4012	x6100	x0047	x0042
x4012	x6200	x0047	x0052
x4012	x6400	x0047	x0052
x4012	x6500	x0052	x0047
x4012	x6400	x0042	x0052
x4012	x6500	x0042	x0047

根据这些断点信息，构造相应地图的数据结构。注意，要求的是递归子程序执行之后的数据结构。

x6100		x6200		x6300	
x6101	x0042	x6201	x0052	x6301	
x6102	x6200	x6202		x6302	
x6103		x6203		x6303	
x6104		x6204		x6304	
x6105		x6205		x6305	
x6106		x6206		x6306	
x6400		x6500			
x6401		x6501			
x6402		x6502			
x6403		x6503			
x6404		x6504			
x6405		x6505			
x6406		x6506			

8.11 下面的程序需要汇编后存入 LC-3 存储器。存放汇编后的程序需要多少个 LC-3 存储单元？

```

                                .ORIG x4000
                                AND  R0,R0,#0
                                ADD  R1,R0,#0
                                ADD  R0,R0,#4
                                LD    R2,B
A                               LDR  R3,R2,#0
                                ADD  R1,R1,R3
                                ADD  R2,R2,#1
                                ADD  R0,R0,#-1
                                BRnp A
                                JSR  SHIFTR
                                ADD  R1,R4,#0
                                JSR  SHIFTR
                                ST   R4,C
                                TRAP x25
B                               .BLKW 1
C                               .BLKW 1
                                .END

```

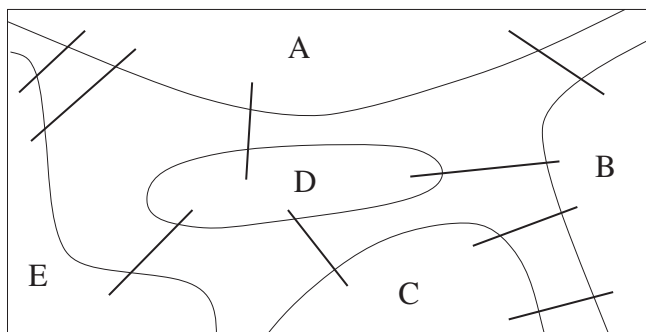
汇编后的程序需要多少个存储单元？

标号 C 对应的地址是什么？

程序执行之前，必须通过某种外部方式为 B 装入一个值。可以假定这在程序开始前已经完成。还可以假定，从 SHIFTR 开始的子程序可供本程序使用：它将 R1 中的值右移一位，把结果存入 R4。

程序执行后，C 中是什么？

- 8.12** [☆] 纽约、斯德哥尔摩、柯尼斯堡等许多城市，由通过桥梁相连的若干区域组成。下图是 FiveParts 城的地图，该城由 A、B、C、D、E 五个区域组成，区域之间有图示的九座桥。



下面的程序提示用户输入两个区域，再将从第一个区域通往第二个区域的桥梁数量存入 x4500。你的任务是设计 FiveParts 城的数据结构，供该程序统计两个区域之间的桥梁数量。

```

                                .ORIG x3000
                                LEA R0, FROM
                                TRAP x22
                                TRAP x20          ; Inputs a char without banner
                                NOT R1, R0
                                ADD R1, R1, #1
                                LEA R0, TO
                                TRAP x22
                                TRAP x20
                                NOT R0, R0
                                ADD R0, R0, #1
                                AND R5, R5, #0
                                LDI R2, HEAD
SEARCH                          BRz DONE
                                LDR R3, R2, #0
                                ADD R7, R1, R3
                                BRz FOUND_FROM
                                LDR R2, R2, #1
                                BRnzp SEARCH
FOUND_FROM                     ADD R2, R2, #2
NEXT_BRIDGE                    LDR R3, R2, #0
                                BRz DONE
                                LDR R4, R3, #0
                                ADD R7, R0, R4
                                BRnp SKIP
                                ADD R5, R5, #1    ; Increment Counter
SKIP                           ADD R2, R2, #1
                                BRnzp NEXT_BRIDGE
DONE                           STI R5, ANSWER
                                HALT
HEAD                           .FILL x3050
ANSWER                         .FILL x4500
FROM                           .STRINGZ "FROM: "
TO                             .STRINGZ "TO: "
                                .END

```

给出表示这一数据结构所需存储单元的内容。图中已经给出数据结构的 **HEAD** 指针，以及五个存储单元和它们的内容；还在每个单元之后提供了足够多的连续存储单元供你使用。按需要使用其中的单元即可。

x4000	x0041
x4001	
x4002	
x4003	
x4004	
x4005	
x4006	

x3050	
-------	--

xA243	x0042
xA244	
xA245	
xA246	
xA247	
xA248	
xA249	

x4100	x0043
x4101	
x4102	
x4103	
x4104	
x4105	
x4106	

xBBBB	x0044
xBBBC	
xBBBD	
xBBBE	
xBBBF	
xBBC0	
xBBC1	

x3100	x0045
x3101	
x3102	
x3103	
x3104	
x3105	
x3106	

8.13 我们计算 $n!$ 的代码适用于所有正整数 n 。按照正文中的约定，本题要求你扩展 FACT 的迭代解法，使其也能处理 $0!$ 。

8.14 如你所知，LC-3 的 ADD 指令将两个 16 位二进制补码整数相加。如果要将两个 32 位二进制补码整数相加，可以使用下面的程序。程序需要调用子程序 X，它将 R1 与 R2 相加产生的进位存入 R0。

补全程序和子程序 X 中用空框标出的部分。每个空框对应一条指令，或一条指令的操作数。注意，一个 32 位操作数需要两个 16 位存储单元。若操作数为 Y，则 Y[15:0] 存在地址 A，Y[31:16] 存在地址 A + 1。

```

.ORIG    x3000
    LEA R3, NUM1
    LEA R4, NUM2
    LEA R5, RESULT
    LDR R1, R3, #0
    LDR R2, R4, #0
    ADD R0, R1, R2
    STR R0, R5, #0
    ----- (a)
    LDR ----- (b)
    LDR ----- (c)
    ADD R0, R1, R2
    ----- (d)
    TRAP x25

X      ST R4, SAVER4
    AND R0, R0, #0
    AND R4, R1, R2
    BRn ----- (e)
    ADD R1, R1, #0
    BRn ----- (f)
    ADD ----- (g)
    BRn ADDING
    BRnzp EXIT
ADDING ADD R4, R1, R2
    BRn EXIT
LABEL  ADD R0, R0, #1
EXIT   LD R4, SAVER4
    RET

NUM1   .BLKW 2
NUM2   .BLKW 2
RESULT .BLKW 2
SAVER4 .BLKW 1

.END

```

8.15 [★] 程序遇到断点并停止。操作员不改变计算机的任何状态，立即按下运行按钮恢复执行。

下表给出恢复执行后，LC-3 前九次存储器访问时 MAR 和 MDR 的内容。填

入缺少的条目。

	MAR	MDR
1st:		x5020
2nd:		xF0F0
3rd:		
4th:	x2000	x020A
5th:		x040A
6th:		x61FE
7th:		
8th:		xC1C0
9th:	x4002	xF025

第九章 I/O

到目前为止，我们完全忽略了输入和输出的细节：计算机究竟怎样从键盘取得信息（输入），又怎样把信息送到显示器（输出）。我们一直依靠 TRAP 指令完成这些任务，例如用 TRAP x23 输入，用 TRAP x21 输出。TRAP 指令让我们通过陷阱向量告诉操作系统需要做什么，然后相信它会替我们完成。

TRAP 指令更一般的名称是**系统调用**（system call），因为它请求操作系统替我们做事，而我们可以完全不知道具体怎样实现。现在可以研究 LC-3 中输入和输出的实际工作方式、用户程序执行 TRAP 发起系统调用时发生的事情，以及操作系统怎样控制这一切。

我们将从实现输入和输出所需的实际硬件结构开始。不过，在此之前，先简要介绍操作系统，以及几个此前并不重要、但对理解操作系统工作十分重要的基本概念。

你可能熟悉 Microsoft 的各种 Windows、Apple 的 MacOS 和 Linux。这些都是操作系统。它们有共同的目标：优化计算机系统全部资源的使用，同时确保任何软件都不能破坏自己无权干涉的程序或数据。要更好地理解这一工作，需要理解特权、优先级和存储器地址空间的布局，也就是各个存储区域及其用途。

9.1 特权、优先级与存储器地址空间

9.1.1 特权与优先级

与计算机处理有关的两个截然不同的概念是特权和优先级。我们常说，这两个概念是正交的。

9.1.1.1 特权

特权关乎做某件事的权利，例如执行某条指令，或者访问某个存储单元。并非所有程序都有权执行全部指令。假如一台计算机由许多用户共享，而 ISA 中有 HALT 指令，我们就不希望任意程序都能执行 HALT，令计算机停机，否则其他用户恐怕会非常不满。同样，有些存储单元只能由操作系统访问。我们不希望任意程序都能干扰操作系统的数据结构或代码，因为这很可能让整个系统崩溃。

为防止这两种情况，我们把每个程序划分为有特权或无特权。我们常用**管理特权**（supervisor privilege）表示有特权，说程序在**管理模式**（Supervisor mode）下执行；无特权则称为在**用户模式**（User mode）下执行。管理模式中的程序可以执行所有指令、访问全部存储器，用户模式中的程序则不能。如果用户模式程序试图执行要求管理模式的指令，或访问要求管理模式的存储单元，计算机就会阻止它。

9.1.1.2 优先级

优先级 (priority) 关乎程序执行的紧迫程度。每个程序都被赋予一个优先级，表明它相对于其他程序的紧迫程度，使更紧迫的程序可以中断较不紧迫的程序。例如，普通用户程序可被赋予优先级 0，键盘可被赋予优先级 4，而计算机与墙上插座等能源的连接状态可被赋予优先级 6。

这样，如果键盘前的人希望执行一个把数据输入计算机的程序，普通用户程序就会被中断。而如果有人拔掉电源线，使计算机即将失去能源，这个输入程序又会被中断。遇到这种情况，我们希望计算机执行操作系统中专门处理这一事件的程序。

9.1.1.3 两个正交的概念

说特权和优先级正交，意思是它们彼此无关。人们有时不容易接受这一点：消防车因为必须迅速赶到火场，就拥有不受交通信号灯约束的特权。日常生活中，紧迫的事情常常会带来某种特权，但计算机系统并非如此。

例如，一个用户程序可能关联某项物理实验，需要在特定时刻中断计算机，记录实验正在产生的信息。如果它不能在那个时刻抢占正在运行的程序，实验数据就可能丢失。它是用户程序，没有管理特权；但它更紧迫，因此有更高的优先级。

再看另一个例子：系统管理员希望运行诊断程序，作为日常预防性维护的一部分。诊断程序需要访问全部存储单元、执行全部指令，因此需要管理特权。但它并不紧迫。相比那些必须在特定时刻使用计算机的程序，诊断是在凌晨一点还是两点进行并不重要。它需要特权，却不需要高优先级。

最后举一个例子，说明在人类活动中也可以有优先级而没有特权。我们的朋友 Bob 在纽约一座摩天大楼的地下室工作。他正要去男洗手间，经理却让他立刻把一条消息送给 88 层的副总裁，再带回答复。于是 Bob 暂时不去洗手间，乘电梯来到 88 层。副总裁让他等了很久，他开始担心自己会憋不住。终于拿到答复后，他按电梯按钮，准备返回地下室，因为急着上洗手间，已经很难受了。等电梯时，另一位副总裁走来，打开管理层专用男洗手间的门，走了进去。Bob 虽然难受，却不能进去。他当然足够紧迫，却没有相应的特权！

9.1.1.4 处理器状态寄存器 (PSR)

计算机上执行的每个程序都关联两个非常重要的寄存器。一个是你熟悉的程序计数器 PC，另一个是图 9.1 所示的**处理器状态寄存器** (Processor Status Register, PSR)，其中包含赋予该程序的特权和优先级。

位 [15] 指定特权：PSR[15]=0 表示管理特权，PSR[15]=1 表示无特权。位 [10:8] 指定程序的**优先级** (priority level, PL)。最高为 7，即 PL7；最低为 PL0。

PSR 还包含条件码的当前值，如图 9.1。9.4 节将解释为什么必须把条件码纳入 PSR。

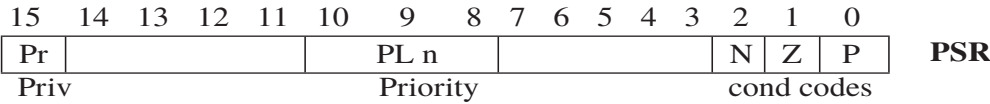


图 9.1 处理器状态寄存器（PSR）

9.1.2 存储器的组织

图 9.2 给出了 LC-3 存储器的布局。LC-3 的地址空间是 16 位，因此地址从 x0000

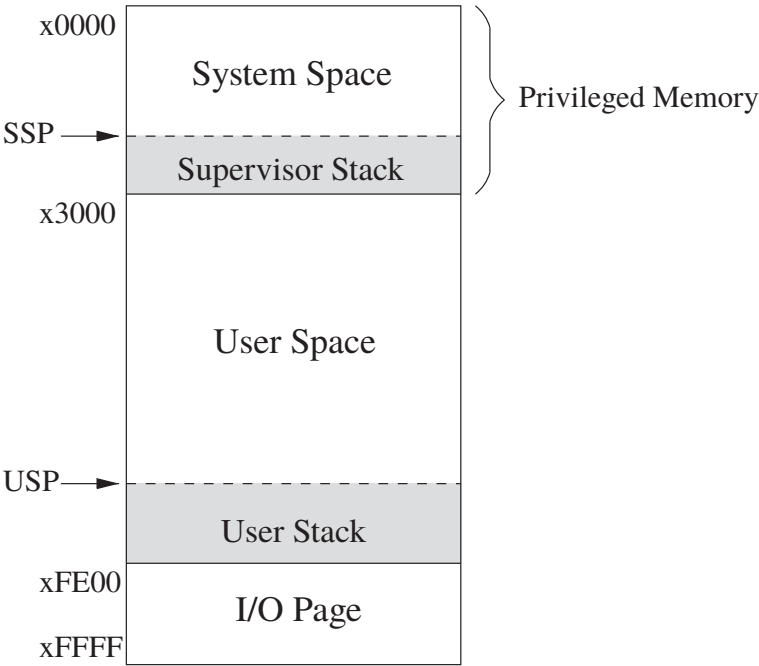


图 9.2 存储器的各个区域

到 xFFFF。x0000 至 x2FFF 是特权存储单元，包含操作系统的各种数据结构和代码，访问它们需要管理特权。这一区域称为**系统空间**（system space）。

x3000 至 xFDFF 是非特权存储单元，访问它们不需要管理特权。所有用户程序和数据都使用这一区域，通常称为**用户空间**（user space）。

xFE00 至 xFFFF 根本不对应存储单元；最后一个实际存储单元的地址是 xFDFF。这些地址用于标识参与输入、输出的寄存器，以及与处理器有关的一些特殊寄存器。例如，PSR 被赋予地址 xFFFC，处理器的**主控制寄存器**（Master Control Register, MCR）被赋予地址 xFFFE。从存储器地址空间中分配地址的好处将在 9.2 节讨论。xFE00 至 xFFFF 通常称为**I/O 页**，因为大多数地址用于标识参与输入或输出的寄存器。访问这些寄存器需要管理特权。

图 9.2 还显示了两个栈：系统空间中的**管理栈**（supervisor stack），以及用户空间中的**用户栈**（user stack）。管理栈由操作系统控制，访问时需要管理特权；用户栈由用户程序控制，访问时不需要特权。

两个栈各有一个栈指针，分别称为**管理栈指针**（Supervisor Stack Pointer, SSP）和**用户栈指针**（User Stack Pointer, USP），用于指示栈顶。程序在任一时刻只能处于管

理模式或用户模式之一，因此任一时刻也只有一个栈处于活动状态。通常用 R6 作为活动栈的栈指针 SP。另外提供 Saved SSP 和 Saved USP 两个寄存器，用于保存未使用的栈指针。例如，从管理模式切换到用户模式时，将 SP 存入 Saved SSP，再从 Saved USP 装入 SP。

9.2 输入 / 输出

输入、输出设备，无论是键盘、显示器、磁盘，还是商场里的自助服务终端，都通过适应自身需要的寄存器处理输入或输出数据。即使最简单的 I/O 设备，通常也至少需要两个寄存器：一个保存设备与计算机之间传送的数据，另一个指示设备状态，例如设备是否可用，还是仍忙于处理上一项 I/O 任务。

9.2.1 I/O 的一些基本特性

全部 I/O 活动都由 ISA 中的指令控制。ISA 是否需要专门的 I/O 指令？I/O 设备与计算机的速度是否相同？如果不同，怎样协调速度差异？计算机与 I/O 设备之间的信息传送，是由计算机中的程序发起，还是由设备发起？这些问题的答案构成了 I/O 的一些基本特性。

9.2.1.1 存储器映射 I/O 与专用 I/O 指令

与输入、输出设备寄存器交互的指令，必须指明具体是哪个寄存器。过去采用过两种方案：一些计算机使用专门的输入、输出指令；大多数计算机则倾向于使用读写存储器时所用的数据传送指令。

年代久远的 PDP-8 就采用专用 I/O 指令。它由 Digital Equipment Corporation 于 1965 年推出，距原书写作时已超过 50 年。PDP-8 的指令长 12 位，其中操作码占 3 位。操作码为 110 时表示 I/O 指令，余下 9 位指定 I/O 设备寄存器及要执行的操作。

大多数计算机设计者不愿为输入、输出再规定一套指令，而是使用在存储器与通用寄存器之间装入、存储数据时的同一组指令。例如，源地址是输入设备寄存器地址的装入指令 LD、LDI 或 LDR，就是输入指令；目的地址是输出设备寄存器地址的存储指令 ST、STI 或 STR，就是输出指令。

既然使用相同的数据传送指令，输入和输出设备寄存器就必须像存储单元一样得到唯一标识。因此，为每个设备寄存器分配 ISA 存储器地址空间中的一个地址。这些地址分配给 I/O 设备寄存器，而非存储单元，所以称为**存储器映射 I/O**（memory-mapped I/O）。

最初的 PDP-11 ISA 有 16 位地址空间，所有位 [15:13]=111 的地址都分配给 I/O 设备寄存器。因此， 2^{16} 个地址中只有 57,344 个对应存储单元，余下 2^{13} 个是存储器映射 I/O 地址。

LC-3 采用存储器映射 I/O。如 9.1.2 节所述，x0000 至 xFDFF 对应实际存储单元，xFE00 至 xFFFF 留给输入、输出设备寄存器。表 A.3 列出了已经分配的 LC-3

设备寄存器映射地址。随着新颖的应用出现，未来 LC-3 微处理器的使用和销售也许还会要求分配更多设备寄存器地址！

9.2.1.2 异步与同步

大多数 I/O 的速度远低于处理器速度。打字员每按下下一个键，就向输入设备寄存器装入一个 ASCII 码。计算机每执行一次操作数地址为该设备寄存器映射地址的装入指令，就能读取一次其内容。

许多微处理器的时钟频率远高于 2 GHz。即使仅为 2 GHz，一个时钟周期也只有 0.5 纳秒。假设处理器一次执行一条指令，读取输入设备寄存器并保存内容的指令需要 10 个周期，那么每隔 5 纳秒就能读一次。可惜，人打字的速度远远不足以让处理器一直忙于读字符。思考：要按处理器能够接收的最高速率提供字符，一个人需要打多快？

可以设计硬件，按较低的固定速率接收字符，以缓解这种速度差异。例如，每隔两亿个周期接收一个字符。原文称，若平均每词五个字母，这要求每分钟输入 100 个单词，是可以做到的。¹但这还要求打字员与计算机时钟严格同步，而同一个人的打字速度也会随时变化，因此不能接受。

这说明什么？I/O 设备通常既与微处理器速度相差很大，也不与其保持同一步调。后一个特性称为**异步**（asynchronous）。处理器与 I/O 设备的交互大多是异步的。要在异步环境中控制处理，就需要某种协议或**握手**（handshaking）机制。键盘需要一个单比特状态标志，表示是否有人输入了字符；显示器也需要一个单比特状态标志，表示上一个字符是否已显示完毕，能否再接收下一个字符。

这些标志是最简单的同步方式。一个称为**就绪位**（ready bit）的标志，就足以协调每分钟输入 100 词的打字员与每秒可接收两亿个字符的处理器。每输入一个字符，就绪位置 1；计算机每读取一个字符，就清除就绪位。读字符之前检查该位，就能知道上一个字符是否已经读过。该位为 0，表示自上次读取后还没有新字符，因此不应再次读取；该位为 1，只能是因为又输入了一个字符，因此应该再读一次。

一个就绪位就提供了足够的握手信息，保证打字员与微处理器之间的异步信息传送能够准确进行。

假如打字员能以恒定速度打字，硬件也在精确间隔接收字符，例如每两亿周期接收一个，就不需要就绪位了。计算机做完两亿周期的其他工作，就知道恰好又有一个字符，可以直接读取。在这个假想情形中，打字员与处理器保持同一步调，不需要额外同步机制。我们称二者**同步**（synchronous）运行，也就是输入活动是同步的。

9.2.1.3 中断驱动与轮询

处理器在计算，打字员在打字，二者是各自工作的独立实体，但又必须交互，把输入的数据送进计算机。中断驱动与轮询的区别，在于由谁控制这种交互。

¹译注：按上文 2 GHz 时钟计算，两亿周期为 0.1 秒，即每秒 10 个字符；若每词严格按五个字母计，为每分钟 120 词。若再计入词间空格，约为每分钟 100 词。

处理器是否一直做自己的事，直到键盘通知它：“有一个键被按下了！ASCII 码在输入设备寄存器里，请来读取。”这称为**中断驱动 I/O**（interrupt-driven I/O），由键盘控制交互。另一种情况是处理器控制交互，不断询问就绪位是否已置 1；一旦检测到置 1，就知道该读取设备寄存器了。这称为**轮询**（polling），因为处理器不断查询就绪位，询问是否有人按键。

9.2.2.2 节介绍轮询，9.4 节介绍中断驱动 I/O。

9.2.2 从键盘输入

9.2.2.1 基本输入寄存器：KBDR 与 KBSR

从键盘输入字符，需要保存字符的数据寄存器，以及通知处理器输入已经发生的同步机制。同步机制包含在键盘的状态寄存器中。

这两个寄存器分别是键盘数据寄存器 KBDR 和键盘状态寄存器 KBSR，均被赋予存储器地址空间中的地址。表 A.3 中，KBDR 的地址为 xFE02，KBSR 的地址为 xFE00。虽然字符只需 8 位，同步机制只需 1 位，但像 LC-3 存储字一样为每个寄存

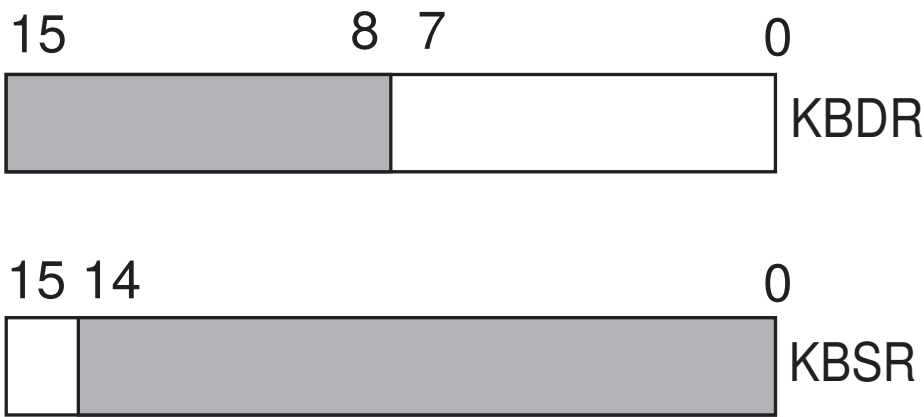


图 9.3 键盘设备寄存器

器安排 16 位更方便。KBDR[7:0] 保存数据，[15:8] 为 x00。KBSR[15] 包含同步机制，也就是就绪位。图 9.3 显示了这两个寄存器。

9.2.2.2 基本输入服务例程

KBSR[15] 协调慢速键盘与快速处理器。按下一个键时，对应 ASCII 码装入 KBDR[7:0]，键盘电路自动把 KBSR[15] 置 1。LC-3 读取 KBDR 时，键盘电路自动清除 KBSR[15]，允许输入下一个字符。KBSR[15]=1 表示上一个字符尚未读取，此时键盘被禁用，必须先读取上一字符，才能接受下一个按键。

采用轮询时，程序反复检查 KBSR[15]，直到发现置 1，再把 KBDR 中的 ASCII 码装入某个 LC-3 寄存器。只在就绪位为 1 时读取，避免重复读取同一个字符；在上一字符被读走之前禁用键盘，又避免漏读。因此，每次接受的按键都恰好被读取一次。

下面的输入例程把键盘输入的 ASCII 码装入 R0，然后继续执行程序 NEXT_TASK。

```
01      START  LDI      R1, A           ; Test for
02                      BRzp     START       ; character input
03                      LDI      R0, B
04                      BRnzp    NEXT_TASK   ; Go to the next task
05  A      .FILL    xFE00               ; Address of KBSR
06  B      .FILL    xFE02               ; Address of KBDR
```

只要 KBSR[15]=0，就说明自上次读取数据寄存器后还没有新按键。第 01、02 行构成检查 KBSR[15] 的循环。注意 LDI：它把 KBSR 映射地址 xFE00 中的内容装入 R1。如果就绪位为 0，BRzp 就跳到 START，再循环一次。有人按键时，对应 ASCII 码装入 KBDR，就绪位置 1，分支不再跳转，转而执行第 03 行。这一次 LDI 从 KBDR 的映射地址 xFE02 装入 R0。输入例程完成后，无条件跳转到 NEXT_TASK。

9.2.2.3 存储器映射输入的实现

图 9.4 显示实现存储器映射输入所需的附加数据通路。第 5 章已经介绍了装入指令执行阶段的数据通路，基本分为三步：

- 1. 将待读存储单元的地址装入 MAR。
- 2. 读取存储器，把该单元的内容装入 MDR。
- 3. 将 MDR 的内容装入目的寄存器 DR。

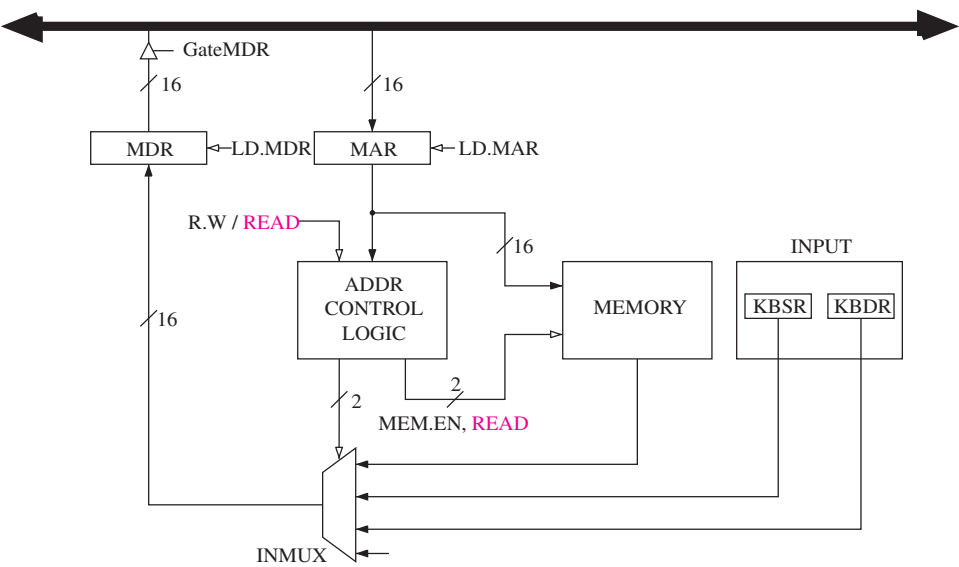


图 9.4 存储器映射输入

存储器映射输入也执行同样的步骤，只是 MAR 中装入设备寄存器地址。地址控制逻辑不再使能存储器读取，而是选择相应设备寄存器，为 MDR 提供输入。

9.2.3 向显示器输出

9.2.3.1 基本输出寄存器：DDR 与 DSR

输出与输入十分相似，DDR、DSR 分别承担 KBDR、KBSR 的角色。DDR 是显示数据寄存器，驱动显示器显示；DSR 是显示状态寄存器。在 LC-3 中，DDR 地址为 xFE06，DSR 地址为 xFE04。与输入相同，虽然输出字符只需 8 位、同步机制只需 1

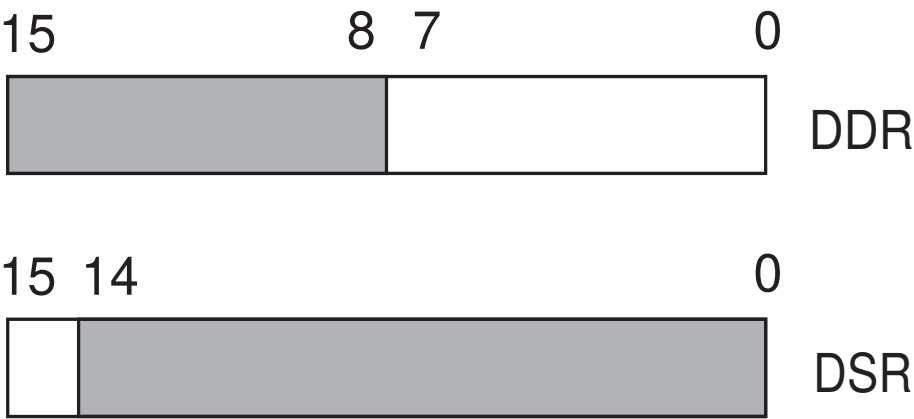


图 9.5 显示器设备寄存器

位，为每个输出设备寄存器安排 16 位仍更方便。DDR[7:0] 保存数据，[15:8] 为 x00；DSR[15] 是就绪位。图 9.5 显示了这两个寄存器。

9.2.3.2 基本输出服务例程

DSR[15] 协调快速处理器与慢速显示器。LC-3 把 ASCII 码送入 DDR[7:0] 时，显示器电路开始处理该字符，并自动清除 DSR[15]。字符显示完成后，电路自动把 DSR[15] 置 1，通知处理器可以向 DDR 传送下一个 ASCII 码。只要 DSR[15] 为 0，显示器仍在处理上一字符，暂时不能接收处理器的其他输出。

采用轮询时，程序反复检查 DSR[15]，发现置 1 后，才将所需字符的 ASCII 码存入 DDR[7:0]，启动显示。下面的例程在显示器上输出 R0 中的 ASCII 字符：

```
01      START      LDI      R1, A           ; Test to see if
02                      BRzp    START       ; output register is ready
03                      STI      R0, B
04                      BRnzp   NEXT_TASK
05      A           .FILL   xFE04           ; Address of DSR
06      B           .FILL   xFE06           ; Address of DDR
```

与 9.2.2.2 节一样，第 01、02 行反复轮询就绪位，检查显示器是否处理完上一个字符。LDI 间接访问 DSR 的映射地址 xFE04。只要 DSR[15] 为 0，BRzp 就跳回 START，再循环一次。显示器完成后自动把该位置 1，使分支不跳转，执行第 03 行。

这里 STI 把 R0 存到 DDR 的映射地址 xFE06。写 DDR 的同时也清除 DSR[15], 暂时禁止后续输出。显示器电路接手, 把字符写到屏幕上。输出例程已经完成, 因此第 04 行无条件跳转到 NEXT_TASK。

9.2.3.3 存储器映射输出的实现

图 9.6 显示存储器映射输出所需的附加数据通路。与前面的输入一样, 在已有存储器访问机制上增加设备寄存器支持, 并不会带来多少额外复杂性。

第 5 章介绍的存储指令执行过程为:

- 1. 将待写存储单元的地址装入 MAR。
- 2. 将待写数据装入 MDR。
- 3. 写入存储器, 将 MDR 的内容存入指定存储单元。

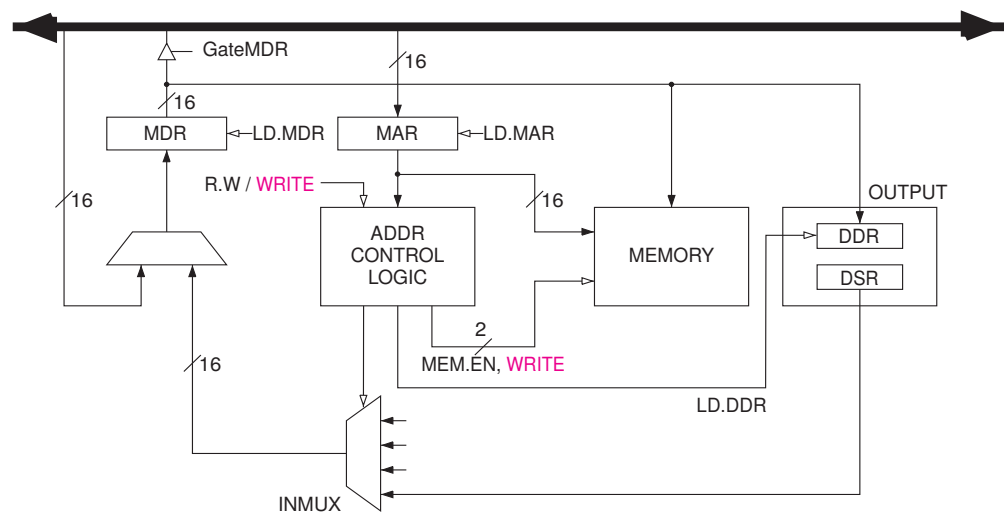


图 9.6 存储器映射输出

存储器映射输出也执行这些步骤, 只是 MAR 装入设备寄存器地址, 地址控制逻辑不使能存储器写入, 而是置位 DDR 的装入使能信号。

存储器映射输出也必须能够读取输出设备寄存器。9.2.3.2 节说明, 在装入 DDR 之前, 就绪位必须为 1, 表示上一个字符已显示完毕。第 01、02 行的 LDI 和 BRzp 完成这一检查: LDI 读取 DSR, BRzp 检查位 [15]。MAR 装入 DSR 地址 xFE04 时, 地址控制逻辑选择 DSR 作为 MDR 的输入, 再把它装入 R1, 并设置条件码。

9.2.3.4 例：键盘回显

打字时, 我们希望看清自己究竟输入了什么。只要组合刚才的两个例程, 就能实现回显 (echo), 不需要复杂的额外电路: 键盘输入的字符会显示在屏幕上。

```

01      START      LDI      R1, KBSR      ; Test for character input
02                      BRzp     START
03                      LDI      R0, KBDR
04      ECHO      LDI      R1, DSR      ; Test output register ready
05                      BRzp     ECHO
06                      STI      R0, DDR
07                      BRnzp    NEXT_TASK
08      KBSR      .FILL    xFE00      ; Address of KBSR
09      KBDR      .FILL    xFE02      ; Address of KBDR
0A      DSR       .FILL    xFE04      ; Address of DSR
0B      DDR       .FILL    xFE06      ; Address of DDR

```

9.2.4 更完善的输入例程

9.2.2.2 节的输入例程属于正在运行的程序，程序大概需要键盘字符输入。但坐在键盘前的人怎样知道什么时候该打字？他可能怀疑程序是否真的在运行，或者计算机是否忙于别的事情。

为通知用户程序正在等待键盘输入，计算机通常在显示器上打印一条消息，称为**提示信息** (prompt)。操作系统显示的%、C:，或者编辑器显示的:，都是提示信息的例子。

图 9.7 的程序仍采用轮询取得键盘输入，但增加了提示信息，通知用户何时按键。下面逐段分析。第 13–19 行以及第 25–28 行你已经熟悉，它们对应 9.2.3.4 节的键盘输入与显示器回显代码。

例程需要使用的寄存器应保存、恢复，这一点也已经熟悉。第 01–03 行保存 R1、R2、R3，第 1D–1F 行恢复，第 22–24 行为这些寄存器值分配存储单元。

剩下的第 05–08、0A–11、1A–1C、29 和 2A 行用于提醒用户开始输入。

第 05–08 行向显示器写 ASCII 码 x0A，即换行符。大多数 ASCII 码对应屏幕上可见的字符，但 x0A 等少数码是**控制字符** (control character)，会引发某种动作。x0A 使光标移到下一行的最左端，所以称为 Newline。当然，写入之前仍须检查 DSR[15]。如果该位为 0，显示器正忙，第 06、07 行继续循环；如果为 1，第 07 行不跳转，第 08 行把 x0A 写入 DDR。

第 0A–11 行将提示信息 Input a character> 写到屏幕。第 2A 行的.STRINGZ 指定该字符串，分配 19 个存储单元：18 个单元各保存一个字符的 ASCII 码，最后一个保存结束哨兵 x0000。

第 0C 行反复检查是否到达字符串末尾，即是否检测到 x0000。如果没有，则等待 DDR 可用，由第 0F 行把下一个提示字符写入 DDR。检测到 x0000 时，说明整条提示信息已输出，程序跳到第 13 行开始真正的键盘输入。

用户输入字符并完成回显之后（第 13–19 行），程序再输出一行换行（第 1A–1C 行），然后继续 NEXT_TASK。

```

01  START    ST      R1,SaveR1    ; Save registers needed
02          ST      R2,SaveR2    ; by this routine
03          ST      R3,SaveR3
04  ;
05          LD      R2,Newline
06  L1       LDI     R3,DSR
07          BRzp    L1            ; Loop until monitor is ready
08          STI     R2,DDR        ; Move cursor to new clean line
09  ;
0A          LEA     R1,Prompt     ; Starting address of prompt string
0B  Loop     LDR     R0,R1,#0     ; Write the input prompt
0C          BRz     Input        ; End of prompt string
0D  L2       LDI     R3,DSR
0E          BRzp    L2            ; Loop until monitor is ready
0F          STI     R0,DDR        ; Write next prompt character
10          ADD     R1,R1,#1     ; Increment prompt pointer
11          BRnzp   Loop         ; Get next prompt character
12  ;
13  Input     LDI     R3,KBSR
14          BRzp    Input        ; Poll until a character is typed
15          LDI     R0,KBDR      ; Load input character into R0
16  L3       LDI     R3,DSR
17          BRzp    L3            ; Loop until monitor is ready
18          STI     R0,DDR        ; Echo input character
19  ;
1A  L4       LDI     R3,DSR
1B          BRzp    L4            ; Loop until monitor is ready
1C          STI     R2,DDR        ; Move cursor to new clean line
1D          LD      R1,SaveR1    ; Restore registers
1E          LD      R2,SaveR2    ; to original values
1F          LD      R3,SaveR3
20          BRnzp   NEXT_TASK    ; Do the program's next task
21  ;
22  SaveR1    .BLKW   1           ; Memory for registers saved
23  SaveR2    .BLKW   1
24  SaveR3    .BLKW   1
25  DSR       .FILL   xFE04
26  DDR       .FILL   xFE06
27  KBSR      .FILL   xFE00
28  KBDR      .FILL   xFE02
29  Newline   .FILL   x000A      ; ASCII code for newline
2A  Prompt    .STRINGZ "'Input a character>'"

```

图 9.7 更完善的输入例程

9.2.5 再看存储器映射 I/O 的实现

图 9.4 和图 9.6 分别给出了输入、输出数据通路的部分实现。此外，为支持中断驱动 I/O，两个状态寄存器不仅必须可读，还必须可写。图 9.8（附录 C 的图 C.3 也给出

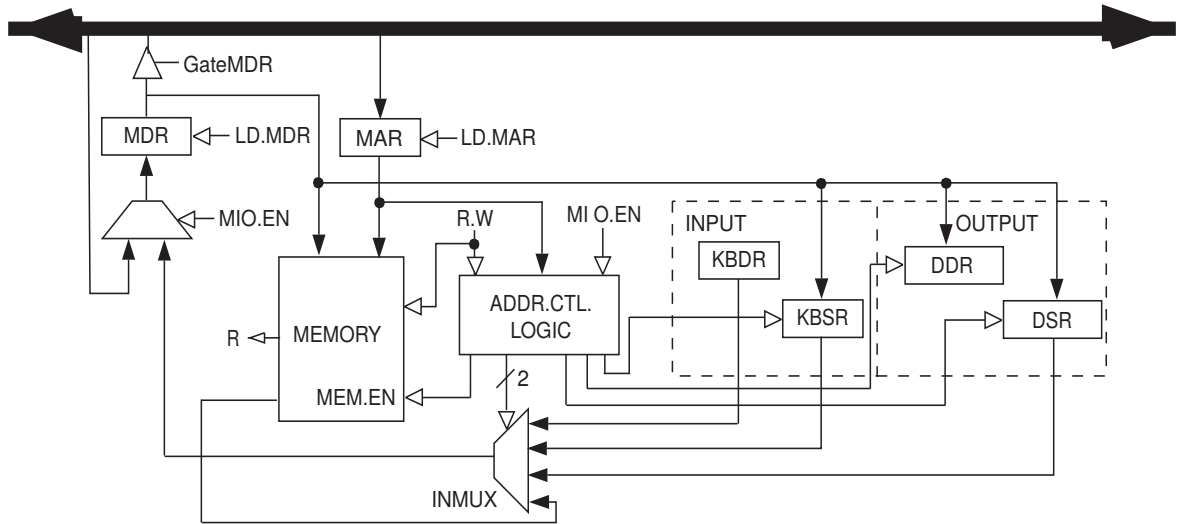


图 9.8 存储器映射 I/O 的相关数据通路实现

了同一图）包含支持前述设备寄存器全部功能所需的数据通路。地址控制逻辑块控制输入、输出操作。注意它有三个输入：MIO.EN 表示本周期是否发生存储器或 I/O 数据传送；MAR 包含存储单元地址或设备寄存器的映射地址；R.W 表示装入还是存储。

根据这三个输入，地址控制逻辑可以不做任何事（MIO.EN=0），也可以产生控制信号，指挥 MDR 与存储器、或 MDR 与某个 I/O 寄存器之间传送数据。

若 R.W 指定装入，数据由存储器或 I/O 设备送往 MDR。地址控制逻辑为 INMUX 提供选择信号，根据 MAR 选择相应设备寄存器或存储器；如果 MAR 对应实际存储单元，还要使能存储器。

若 R.W 指定存储，则把 MDR 的内容写入存储器或某个设备寄存器。地址控制逻辑相应地使能存储器写入，或者置位 MAR 所指定设备寄存器的装入使能信号。

9.3 操作系统服务例程：LC-3 陷阱例程

9.3.1 引言

回看图 9.7。要成功取得键盘输入，程序员必须知道：

- 1. 显示器和键盘的数据寄存器：前者用于显示提示，后者用于取得输入字符。
- 2. 显示器和键盘的状态寄存器：前者用于判断何时可显示提示的下一个字符，后者用于判断是否有人按键。
- 3. 键盘输入相对于程序执行是异步的。

这些已经超出大多数应用程序员的知识范围。现实中，如果应用程序员（也称用户程序员）必须在这一层次理解 I/O，那么 I/O 应用会少得多，从业程序员也会少得多。

让用户程序直接访问 KBDR、KBSR 还有另一个问题：设备寄存器由许多程序共享。允许用户程序员直接访问硬件寄存器，如果他弄错了，就可能扰乱其他用户程序。因此，通常不宜让用户程序员访问这些寄存器。这也正是硬件寄存器地址被纳入特权地址空间、只能由具有管理特权的程序访问的原因。

更简单、更安全的办法，是通过 TRAP 指令请求具有管理特权的操作系统完成 I/O。

第 4 章首次介绍 TRAP，用它请求操作系统停机；第 5 章又用它请求操作系统执行 I/O。TRAP 的重要好处是，用户程序员不必了解本章前面那些繁杂细节，同时保护用户程序免受其他不熟练程序员的影响。

图 9.9 中，用户程序执行到 $\times 4000$ 时需要 I/O，于是用 TRAP 请求操作系统代为执行。操作系统接管计算机，处理 TRAP 指定的请求，然后在 $\times 4001$ 将控制权交回用户程序。这样的请求称为系统调用，也称**服务调用**（service call）。

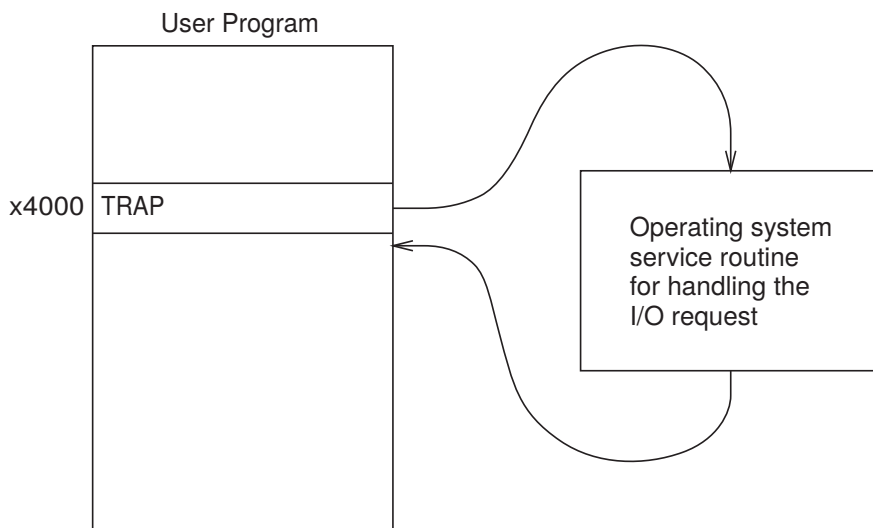


图 9.9 用 TRAP 指令调用操作系统服务例程

9.3.2 陷阱机制

陷阱机制包含以下要素：

1. 操作系统替用户程序执行的一组服务例程。它们属于操作系统，可以从系统空间中的任意地址开始。LC-3 最多支持指定 256 个服务例程；附录 A 的表 A.2 列出当前完整清单。
2. 保存这 256 个例程起始地址的表，位于 $\times 0000$ 至 $\times 00FF$ 。不同公司有不同名称，如 System Control Block 或**陷阱向量表**（Trap Vector Table）。图 9.10 标出了若干具体地址： $\times 0021$ 保存字符输出例程起始地址 $\times 0420$ ； $\times 0023$ 保存键盘输入例程地址 $\times 04A0$ ； $\times 0025$ 保存停机例程地址 $\times 0520$ 。

3. TRAP 指令。用户程序希望操作系统执行指定服务例程，并在完成后返回时，就使用该指令，正如我们从第 4 章起一直所做的那样。
4. 返回用户程序的链接。服务例程必须有将控制权交回用户程序的机制。

x0000	⋮
⋮	⋮
x0020	x03E0
x0021	x0420
x0022	x0460
x0023	x04A0
x0024	x04E0
x0025	x0520
⋮	⋮
x00FF	⋮

图 9.10 陷阱向量表

9.3.3 TRAP 指令

TRAP 通过两件事启动服务例程：根据陷阱向量把 PC 改成相应例程的起始地址；提供返回执行 TRAP 的程序的办法。这个“返回办法”称为链接。

TRAP 指令的操作码为 1111，位 [7:0] 是标识所请求服务的陷阱向量，位 [11:8] 必须为 0。下面的陷阱向量是 x23。

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	1	1	1	0	0	0	0	0	0	1	0	0	0	1	1
TRAP								trap vector							

TRAP 指令周期的执行阶段完成三件事：

1. 将 PSR 和 PC 依次压入管理栈。PC 已在取指令阶段自增，因此保存的是 TRAP 后一条指令的地址，返回时就能从那里继续。请求服务的程序既可能处于管理模式，也可能处于用户模式。如果处于用户模式，R6 指向用户栈，因此压栈之前，必须先把 R6 存入 Saved USP，再把 Saved SSP 装入 R6。

- 2. 把 PSR[15] 设为 0，因为服务例程需要管理特权。PSR[10:8] 不变，因为陷阱例程的优先级与请求它的程序相同。
- 3. 把 8 位陷阱向量零扩展成 16 位地址，访问陷阱向量表。向量 x23 形成地址 x0023，其中存有服务例程起始地址 x04A0。将 x04A0 装入 PC，完成指令周期。

PC 现在为 x04A0，因此从这个地址继续处理。这里是操作系统键盘字符输入例程的起点。我们说，陷阱向量“指向”陷阱例程起始地址；TRAP x23 就这样启动键盘输入服务。

9.3.4 RTI 指令：将控制权交回调用程序

现在只需说明，服务例程执行完后怎样返回调用程序。这由 RTI，即 Return from Trap or Interrupt 指令完成：

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0

RTI

RTI 的操作码是 1000，没有操作数。它把管理栈顶两个值依次弹出到 PC 和 PSR。恢复的 PC 包含 TRAP 后一条指令的地址，因此能返回正确位置。

PSR 弹出后，还要检查 PSR[15]，判断执行 TRAP 时原程序处于用户模式还是管理模式。如果原来是用户模式，就必须调整栈指针，使活动栈重新成为用户栈：把当前 R6 存入 Saved SSP，再把 Saved USP 装入 R6。

9.3.5 陷阱服务过程小结

图 9.11 用 LC-3 的 TRAP、RTI 实现图 9.9 的例子。控制流由需要键盘输入的用户程序 A，转到代为完成任务的操作系统服务例程 B，再回到使用输入字符的用户程序 C。计算机不断执行取指令、译码等指令周期，顺序处理指令，直到当前指令在执行阶段修改 PC，改变控制流。下一次取指令就来自新的地址。

用户程序中的 TRAP x23 正是这样：把 PSR 和自增后的 PC 压入管理栈，将 x0023 的内容 x04A0 装入 PC。图 9.11 的虚线表示根据向量 x23 查陷阱向量表、取得例程起始地址的过程。

下一指令周期从 x04A0 取出服务例程第一条指令。例程完成时执行 RTI，从管理栈恢复 PC、PSR，即恢复执行 TRAP 时保存的值。PC 入栈前已经自增，因此用户程序从 TRAP 后一条指令恢复执行。

例 9.1 下面的程序说明 TRAP 的用法，也可以逗一逗普通的四岁孩子！

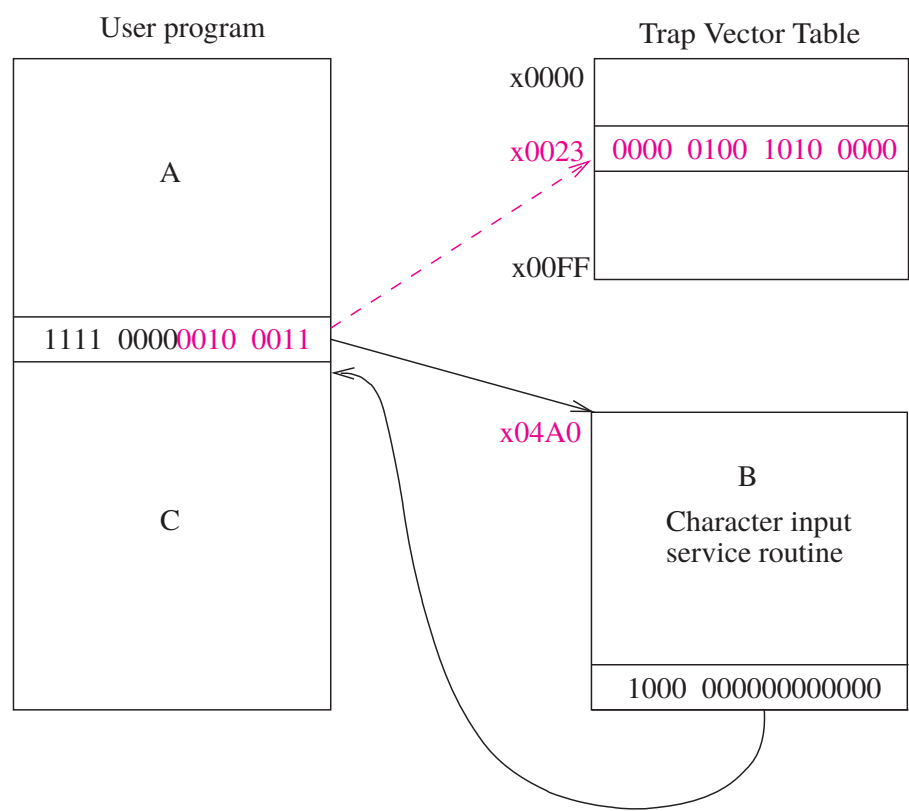


图 9.11 从用户程序转入操作系统服务例程，再返回用户程序的控制流

```
01          .ORIG x3000
02          LD      R2,TERM    ; Load -7
03          LD      R3,ASCII   ; Load ASCII difference
04  AGAIN    TRAP    x23       ; Request keyboard input
05          ADD     R1,R2,R0   ; Test for terminating
06          BRz     EXIT      ; character
07          ADD     R0,R0,R3   ; Change to lowercase
08          TRAP    x21       ; Output to the monitor
09          BRnzp   AGAIN     ; ... and do it again!
0A  TERM     .FILL   xFFC9    ; FFC9 is negative of ASCII 7
0B  ASCII    .FILL   x0020
0C  EXIT     TRAP    x25      ; Halt
0D          .END
```

程序先把常量 `xFFC9`、`x0020` 分别装入 `R2`、`R3`。`xFFC9` 是字符 7 的 ASCII 码的负值，用于判断小朋友是否还想继续玩。`x0020` 是同一字母小写与大写 ASCII 码的差的零扩展值。例如，A、a 的 ASCII 码分别为 `x41`、`x61`，Z、z 分别为 `x5A`、`x7A`。

然后执行 `TRAP x23`，调用键盘输入服务。返回应用程序第 05 行时，`R0` 包含输入字符的 ASCII 码。`ADD`、`BRz` 检查是否为终止字符 7；若不是，就将大小写之差 `x0020` 加到输入 ASCII 码上，把结果存入 `R0`。随后调用显示器输出陷阱服务，显示该字母的小写形式。返回第 09 行后，无条件跳回 `AGAIN`，再次请求输入。

程序正确运行的前提是，用户只输入大写字母和数字 7。如果输入 \$ 呢？更好的

程序应检查输入是否为 26 个大写字母之一或字符 7，否则采取纠正措施。

思考：增加对无效输入的检查，编写程序，将任何输入的大写字母转换成小写输出，而一旦输入非大写字母就终止。参见习题 9.20。

9.3.6 处理 I/O 的陷阱例程

利用上述机制，稍加修改图 9.7，就得到图 9.12 的输入服务例程。需要两处修改：添加适当的 .ORIG、.END，其中 .ORIG 指定例程起始地址，即陷阱向量表 x0023 中保存的地址；将图 9.7 第 20 行的 BR NEXT_TASK 改为 RTI。因为例程由 TRAP x23 调用，已经不再是用户程序自身的一部分，所以用 RTI 返回。

9.2.3.2 节的输出例程可作类似修改，结果如图 9.13。这样，输入、输出例程就能通过相应陷阱向量，简单而安全地由 TRAP 调用。TRAP x23 完成后，R0 包含键盘字符的 ASCII 码；输出时，调用程序先把待显示字符的 ASCII 码装入 R0，再执行 TRAP x21。

```

01      ; Service Routine for Keyboard Input
02      ;
03      .ORIG    x04A0
04  START  ST      R1,SaveR1      ; Save the values in the registers
05          ST      R2,SaveR2      ; that are used so that they
06          ST      R3,SaveR3      ; can be restored before RET
07      ;
08          LD      R2,Newline
09  L1      LDI      R3,DSR        ; Check DSR -- is it free?
0A          BRzp    L1
0B          STI      R2,DDR        ; Move cursor to new clean line
0C      ;
0D          LEA      R1,Prompt     ; Prompt is starting address
0E          ; of prompt string
0F  Loop   LDR      R0,R1,#0       ; Get next prompt character
10          BRz      Input         ; Check for end of prompt string
11  L2      LDI      R3,DSR
12          BRzp    L2
13          STI      R0,DDR        ; Write next character of
14          ; prompt string
15          ADD      R1,R1,#1       ; Increment prompt pointer
16          BRnzp   Loop
17      ;
18  Input   LDI      R3,KBSR        ; Has a character been typed?
19          BRzp    Input
1A          LDI      R0,KBDR        ; Load it into R0
1B  L3      LDI      R3,DSR
1C          BRzp    L3
1D          STI      R0,DDR        ; Echo input character
1E          ; to the monitor
1F      ;
20  L4      LDI      R3,DSR
21          BRzp    L4
22          STI      R2,DDR        ; Move cursor to new clean line
23          LD      R1,SaveR1      ; Service routine done, restore
24          LD      R2,SaveR2      ; original values in registers.
25          LD      R3,SaveR3
26          RTI                    ; Return from Trap
27      ;
28  SaveR1  .BLKW    1
29  SaveR2  .BLKW    1
2A  SaveR3  .BLKW    1
2B  DSR     .FILL    xFE04
2C  DDR     .FILL    xFE06
2D  KBSR    .FILL    xFE00
2E  KBDR    .FILL    xFE02
2F  Newline .FILL    x000A        ; ASCII code for newline
30  Prompt  .STRINGZ "Input a character>"
31          .END

```

图 9.12 字符输入服务例程

```

01          .ORIG    x0420          ; System call starting address
02          ST      R1, SaveR1      ; R1 will be used to poll the DSR
03                                     ; hardware
04          ; Write the character
05  TryWrite  LDI     R1, DSR        ; Get status
06                                     ; Bit 15 on says display is ready
07  WriteIt   STI     R0, DDR        ; Write character
08
09          ; return from trap
0A  Return   LD      R1, SaveR1      ; Restore registers
0B                                     ; Return from trap
0C  DSR      .FILL   xFE04          ; Address of display status register
0D  DDR      .FILL   xFE06          ; Address of display data register
0E  SaveR1   .BLKW   1
0F          .END

```

图 9.13 字符输出服务例程

9.3.7 使计算机停机的陷阱例程

回想 4.5 节，RUN 锁存器与晶体振荡器信号相与，产生控制计算机工作的时钟。如果清除这个单比特锁存器，AND 门输出就变为 0，时钟停止。

早年的许多 ISA 有 HALT 指令。但它极少执行，为其专门分配操作码似乎浪费。许多现代计算机通过陷阱例程清除 RUN。在 LC-3 中，RUN 是主控制寄存器 MCR 的位 [15]，MCR 映射到 xFFFE。图 9.14 是停止处理器时钟的陷阱服务例程。

```

01          .ORIG    x0520          ; Where this routine resides
02          ST      R1, SaveR1      ; R1: a temp for MC register
03          ST      R0, SaveR0      ; R0 is used as working space
04
05          ; print message that machine is halting
06
07          LD      R0, ASCIINewLine
08          TRAP    x21
09          LEA     R0, Message
10          TRAP    x22
11          LD      R0, ASCIINewLine
12          TRAP    x21
13
14          ;
15          ; clear bit 15 at xFFFE to stop the machine
16          ;
17
18          LDI     R1, MCR          ; Load MC register into R1
19          LD      R0, MASK         ; R0 = x7FFF
20          AND     R0, R1, R0       ; Mask to clear the top bit
21          STI     R0, MCR          ; Store R0 into MC register

```

图 9.14 LC-3 的 HALT 服务例程

```

14 ;
15 ; return from HALT routine.
16 ; (how can this routine return if the machine is halted above?)
17 ;
18             LD      R1, SaveR1 ; Restore registers
19             LD      R0, SaveR0
1A             RTI
1B ;
1C ; Some constants
1D ;
1E ASCIINewLine .FILL  x000A
1F SaveR0       .BLKW  1
20 SaveR1       .BLKW  1
21 Message      .STRINGZ "Halting the machine."
22 MCR          .FILL  xFFFE      ; Address of MCR
23 MASK        .FILL  x7FFF      ; Mask to clear the top bit
24             .END

```

图 9.14 LC-3 的 HALT 服务例程 (续)

首先, 第 02、03 行保存例程需要使用的 R1、R0。然后, 第 07–0C 行在显示器上显示 Halting the machine。最后, 第 10–13 行将 MCR 与 0111111111111111 按位 AND, 清除 RUN, 即 MCR[15], 同时保持 MCR[14:0] 不变。

思考: 用哪条指令或哪个陷阱服务例程可以重新启动时钟? 提示: 这是个有圈套的问题! :-)

9.3.8 字符输入陷阱例程: 最后再看一次

再看图 9.12, 尤其是 L1、L2、L3、L4 四处出现的三指令序列:

```

label  LDI  R3, DSR
        BRzp label
        STI  Reg, DDR

```

能否通过 JSR / RET 机制, 把这四处相同序列替换成一个子程序? 答案是: 可以, 差不多。

图 9.15 的“改进版”键盘输入例程在第 04、0A、10、13 行使用 JSR WriteChar, 并在第 1A–1D 行提供四指令子程序:

```

WriteChar LDI  R3, DSR
          BRzp WriteChar
          STI  R2, DDR
          RET

```

```

01      .ORIG    x04A0
02  START      JSR      SaveReg
03              LD       R2,Newline
04              JSR      WriteChar
05              LEA      R1,PROMPT
06      ;
07      ;
08  Loop        LDR      R2,R1,#0      ; Get next prompt char
09              BRz      Input
10              JSR      WriteChar
11              ADD      R1,R1,#1
12              BRnzp    Loop
13      ;
14  Input        JSR      ReadChar
15              ADD      R2,R0,#0      ; Move char to R2 for writing
16              JSR      WriteChar    ; Echo to monitor
17      ;
18              LD       R2, Newline
19              JSR      WriteChar
20              JSR      RestoreReg
21              RTI          ; RTI terminates the trap routine
22      ;
23  Newline      .FILL    x000A
24  PROMPT      .STRINGZ  "Input a character>"
25      ;
26  WriteChar    LDI      R3,DSR
27              BRzp     WriteChar
28              STI      R2,DDR
29              RET          ; JMP R7 terminates subroutine
30      DSR      .FILL    xFE04
31      DDR      .FILL    xFE06
32      ;
33  ReadChar     LDI      R3,KBSR
34              BRzp     ReadChar
35              LDI      R0,KBDR
36              RET
37      KBSR     .FILL    xFE00
38      KBDR     .FILL    xFE02
39      ;
40  SaveReg      ST       R1,SaveR1
41              ST       R2,SaveR2
42              ST       R3,SaveR3
43              ST       R4,SaveR4
44              ST       R5,SaveR5
45              ST       R6,SaveR6
46              RET
47      ;
48  RestoreReg   LD       R1,SaveR1
49              LD       R2,SaveR2
50              LD       R3,SaveR3
51              LD       R4,SaveR4
52              LD       R5,SaveR5
53              LD       R6,SaveR6
54              RET
55  SaveR1       .FILL    x0000
56  SaveR2       .FILL    x0000
57  SaveR3       .FILL    x0000
58  SaveR4       .FILL    x0000
59  SaveR5       .FILL    x0000
60  SaveR6       .FILL    x0000
61      .END

```

图 9.15 LC-3 字符输入陷阱服务例程：最终版本

注意，子程序必须以 **RET**（即 **JMP R7**）结束。为什么说“差不多”？原先 **L2**、**L3** 处的 **STI** 把 **R0** 而非 **R2** 送往 **DDR**。解决很简单：图 9.15 第 08 行用 **LDR R2, R1, #0** 替代 **LDR R0, R1, #0**，把提示字符串的每个字符装入 **R2**，**WriteChar** 再将其送到 **DDR**。

第 0F 行插入 **ADD R2, R0, #0**，把 **R0** 中的键盘输入复制到 **R2**，再由 **WriteChar** 输出。**R0** 仍保留输入字符，后面也没有指令重新装入 **R0**，因此返回用户程序时输入字符仍在 **R0** 中。

第 12 行插入 **LD R2, Newline**，把换行符装入 **R2**，再由 **WriteChar** 输出。

图 9.15 就是 **LC-3** 实际提供的键盘输入陷阱服务例程。

9.3.9 PUTS：向显示器输出字符串

离开图 9.15 之前，再看第 08–0C 行。它把字符序列 *Input a character* 写到显示器上。字符序列通常称为字符的串，或字符串。图 9.14 也包含这个片段，用于输出 *Halting the machine*。

用户程序经常需要输出字符串，因此 **LC-3** 操作系统专门提供了一项陷阱服务。用户程序只须把字符串起始地址放入 **R0**，再执行 **TRAP x22**。在 **LC-3** 汇编语言中，该 **TRAP** 的名称是 **PUTS**。

PUTS（即 **TRAP x22**）把控制权交给操作系统，执行图 9.16 的例程。注意，**PUTS** 基本就是图 9.15 第 08–0C 行代码，只作了少量调整。

```

01      ; This service routine writes a NULL-terminated string to the console.
02      ; It services the PUTS service call (TRAP x22).
03      ; Inputs: R0 is a pointer to the string to print.
04      ;
05      .ORIG    x0460
06      ST      R0, SaveR0      ; Save registers that
07      ST      R1, SaveR1      ; are needed by this
08      ST      R3, SaveR3      ; trap service routine
09      ;
0A      ; Loop through each character in the array
0B      ;
0C      Loop    LDR      R1, R0, #0      ; Retrieve the character(s)
0D      BRz     Return          ; If it is 0, done
0E      L2      LDI      R3, DSR
0F      BRzp    L2
10      STI     R1, DDR          ; Write the character
11      ADD     R0, R0, #1        ; Increment pointer
12      BRnzp   Loop            ; Do it all over again
13      ;
14      ; Return from the request for service call
15      Return   LD       R3, SaveR3
16      LD      R1, SaveR1
17      LD      R0, SaveR0
18      RTI
19      ;
1A      ; Register locations
1B      DSR     .FILL    xFE04
1C      DDR     .FILL    xFE06
1D      SaveR0  .FILL    x0000
1E      SaveR1  .FILL    x0000
1F      SaveR3  .FILL    x0000
20      .END

```

图 9.16 LC-3 的 PUTS 服务例程

9.4 中断与中断驱动 I/O

9.2.1.3 节指出，处理器与 I/O 设备的交互可以由处理器控制，即轮询；也可以由设备控制，即中断驱动。9.2.2、9.2.3、9.2.4 节的例子都是轮询：处理器反复检查状态寄存器的就绪位，直到其为 1，才执行输入、输出操作。现在研究由 I/O 设备控制交互的情况。

9.4.1 什么是中断驱动 I/O？

中断驱动 I/O 的核心是，一个可能与当前程序毫无关系的 I/O 设备能够：让当前程序停下来；让处理器执行满足设备需要的程序；再让原程序继续执行，仿佛什么也没有发生。图 9.17 显示这三个阶段。从程序 A 的角度看，完成的工作和计算结果都与中断从未发生时相同，就好像执行流程一直是：

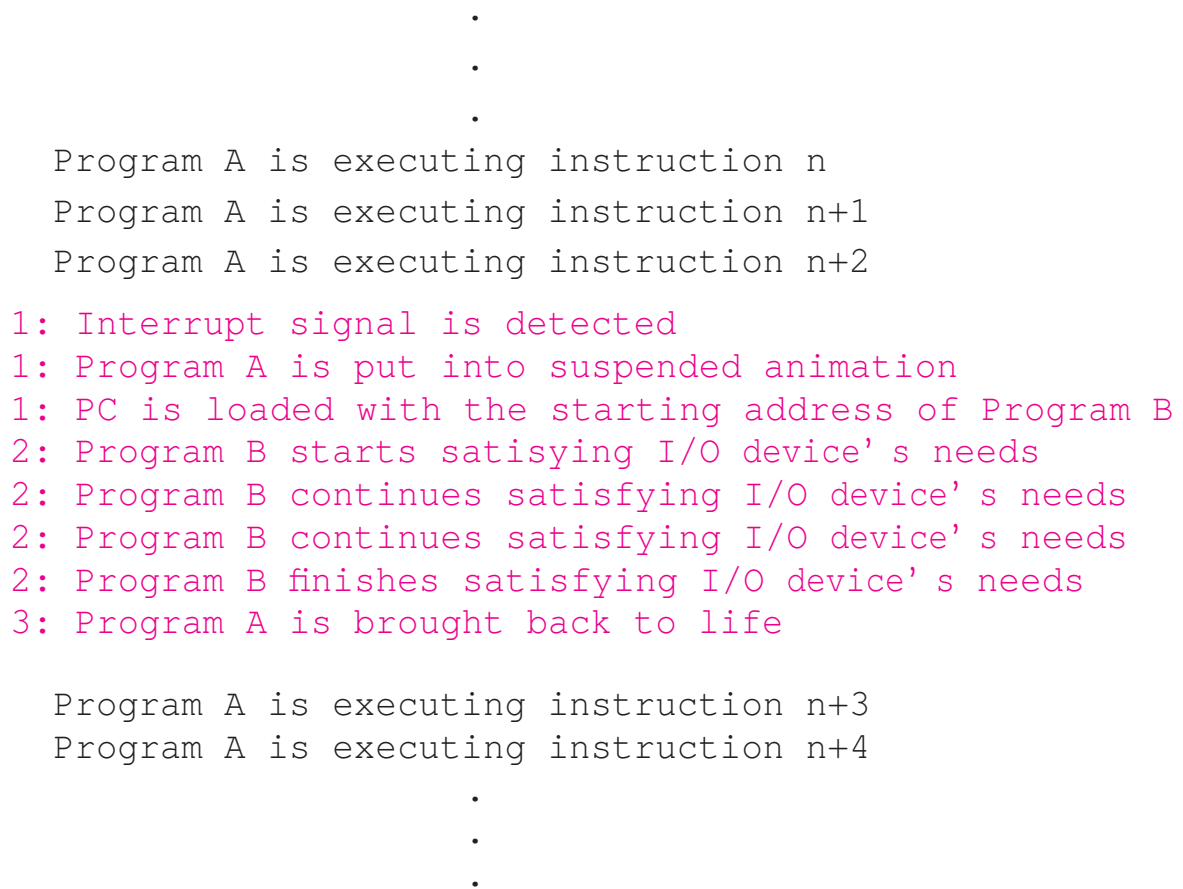


图 9.17 中断驱动 I/O 的指令执行流程

```

      .
      .
      .
Program A is executing instruction n
Program A is executing instruction n+1
Program A is executing instruction n+2
Program A is executing instruction n+3
Program A is executing instruction n+4
      .
      .
      .

```

9.4.2 为什么需要中断驱动 I/O ?

显然，轮询让处理器浪费大量时间，不断重复 LDI、BR，直到就绪位置 1。采用中断驱动 I/O，就不必反复检查、分支。处理器可以利用时间做有用的工作，例如运行其他程序，直到收到设备需要服务的通知。

例 9.2 假设要编写程序，接收键盘输入的 100 个字符，处理其中的信息。输入速度为每分钟 80 词，相当于每 0.125 秒输入一个字符。处理一组 100 字符需要 12.49999 秒，程序要连续处理 1000 组。完成任务需要多久？为什么选择 12.49999？当然是为了让数字好看些。:-)

若像 9.2.2 节一样轮询，每次都要浪费时间等待“下一个”字符。取得一组 100 字符需要 $100 \times 0.125 = 12.5$ 秒。

采用中断驱动 I/O，等待期间不必反复执行 LDI、BR，可以处理前一组字符，只在新字符到达时，用极短时间响应中断并读取。假设每次读取执行 10 条指令，每条平均耗时 0.00000001 秒，那么每个字符需要 0.0000001 秒，一组只占用处理器 0.00001 秒。因此，每个 12.50000 秒的输入间隔中，处理器有 12.49999 秒可用于有用的工作，例如处理上一组字符。

采用轮询，每组总耗时为 $12.5 + 12.49999 = 24.99999$ 秒。²采用中断驱动 I/O，第一组之后，每组的约需 12.5 秒：读取占 0.00001 秒，处理占 12.49999 秒。对于 1000 组，约是 7 小时与 $3\frac{1}{2}$ 小时的差别。

9.4.3 过程的两个部分

中断驱动 I/O 包括：让 I/O 设备能够中断处理器的机制，以及处理该中断请求的机制。

9.4.4 第一部分：使中断发生

I/O 设备要真正中断当前程序，必须同时满足：

1. 设备需要服务。

²译注：原书此处合计写为 24.9999，少了一位 9；按所列加数修正。

- 2. 设备有权请求该服务。
- 3. 设备请求比处理器当前的工作更紧迫。

三者同时满足，处理器才会暂停当前程序，处理中断。

9.4.4.1 设备的中断信号

设备要产生中断请求，必须需要服务，并有权提出请求。

设备必须需要服务。轮询时已经讨论过，这由 KBSR 或 DSR 的就绪位表示。键盘有人按键，就需要服务；显示器处理完上一字符、需要下一个输出字符，也需要服务。两种情况都以相应就绪位置 1 表示。

设备必须有权请求服务。这由中断使能位（interrupt enable，IE）决定。处理器，通常是操作系统，可以设置或清除它，决定是否允许设备请求服务。多数设备把 IE 放在状态寄存器中。图 9.18 中，KBSR、DSR 的 IE 都是位 [14]。设备中断请求信号是 IE 与就绪位的逻辑 AND。若位 [14] 为 0，无论就绪位是否为 1，设备都不能中断处理

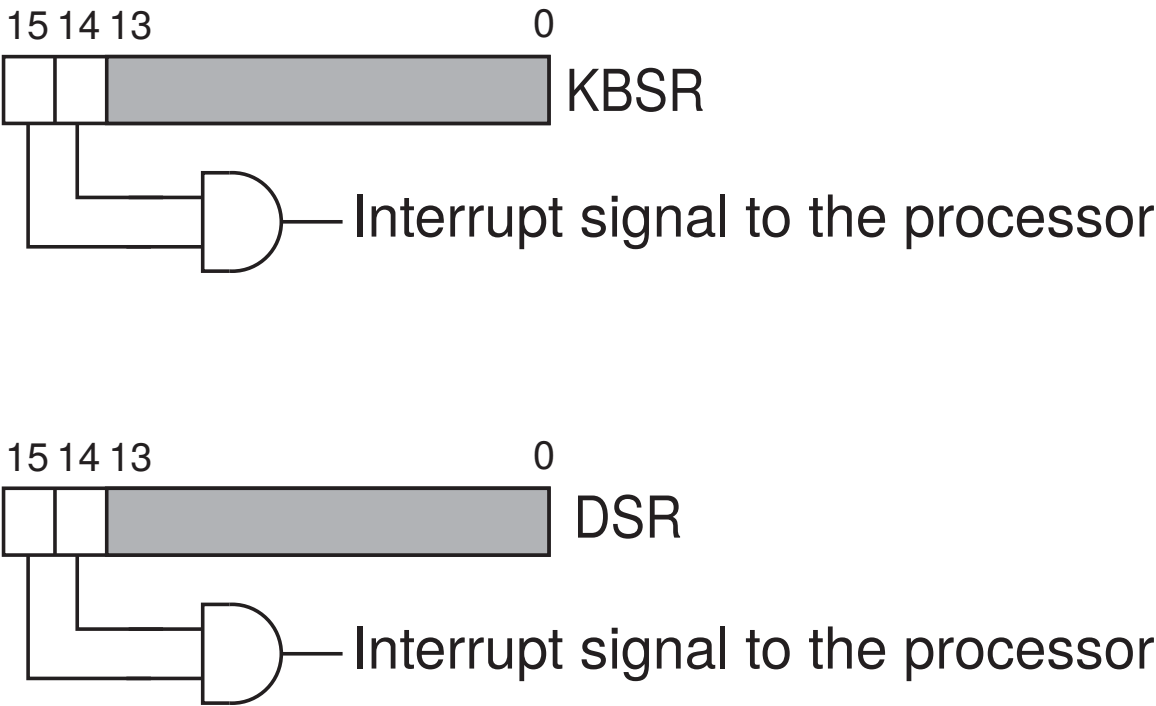


图 9.18 中断使能位及其用途

器，因为它没有被授予请求中断的权利。这时程序必须轮询设备是否就绪。

若位 [14] 为 1，就启用了中断驱动 I/O。有人按键，或显示器处理完上一字符，就使位 [15] 置 1。设备既需要服务又有权请求，AND 门输出置位，产生设备中断请求。

9.4.4.2 请求的紧迫程度

第三个条件是，请求比当前程序更紧迫。9.1.1.2 节已说明，程序以指定优先级运行。设备优先级必须高于当前程序，才可以中断它。同一时刻可能有许多设备请求中断，成功取得处理器的设备必须具有所有竞争请求中最高的优先级。

几乎所有计算机都规定一组程序优先级。LC-3 有 PL0–PL7 八级，数值越高，程序越紧迫。程序的 PL 通常与请求执行该程序的事件的 PL 相同。如果更高 PL 的请求需要计算机，低 PL 程序就暂停，等待高 PL 程序满足更紧迫的请求。

例如，工资计算程序可以在夜间以 PL0 运行，有整晚时间，不太紧迫；纠正核电站电流突增的程序则可能以 PL6 运行。为了避免事故，我们当然愿意让工资计算等一等。

因此，设备请求必须比被中断程序优先级高。通常，我们不会允许教授查看电子邮件时的键盘中断，打断核电站的纠正程序。

9.4.4.3 INT 信号

要让处理器暂停当前程序并服务中断请求，必须置位 INT。图 9.19 显示其产生方式。不同设备具有不同 PL；位 [14]、[15] 都为 1 的设备发出请求。请求进入**优先编码器**（priority encoder），这个组合逻辑结构选择所有有效请求中优先级最高者。如果它的 PL 高于当前程序 PL，就置位 INT。

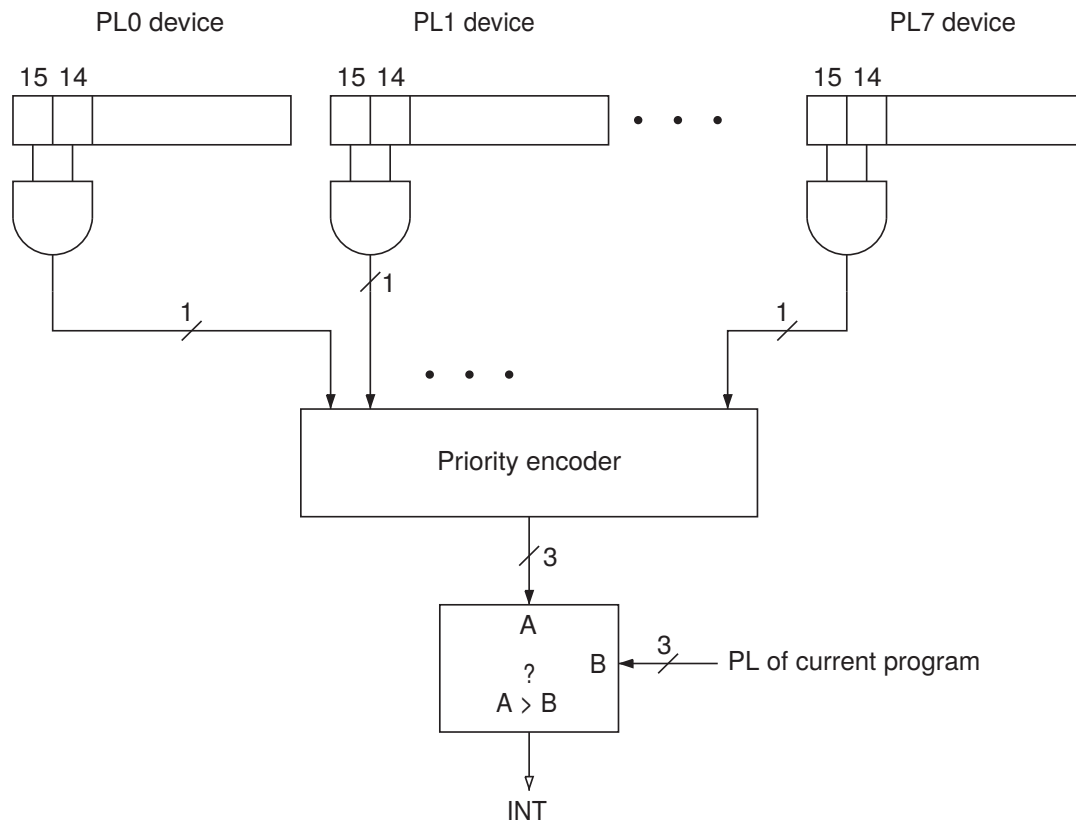


图 9.19 INT 信号的产生

9.4.4.4 检查 INT

最后需要检查 INT，决定是否暂停并处理中断。回想第 4 章，指令周期不断经历取指令、译码、地址计算、取操作数、执行、存储结果等阶段。每条指令改变计算机状态，到该指令周期结束时，变化才全部完成。也就是说，在返回下一条指令的取指令阶段之前的最后一个周期，计算机处于当前指令完整执行后的状态。

中断可以随时发生，相对于控制计算机的同步有限状态机是异步的。例如，中断信号可能在取操作数阶段到达。若立刻停下来，就必须记录当前指令哪些部分已执行、哪些尚未完成。更合理的办法是只在**指令边界**检查中断：当前指令已经完成，下一条尚未开始。这样不必处理“执行了一半”的指令，计算机状态就是上一条完整指令产生的状态。

因此，在每条指令周期的最后一个状态增加 INT 检查。下一状态不再必定是下一条指令的取指令状态：INT 未置位，就照常取下一条指令；INT 已置位，就进入第二部分——处理中断请求——的第一个状态。

9.4.5 第二部分：处理中断请求

如图 9.17，处理中断分为三个阶段：启动中断，对应标为 1 的三行；服务中断，对应标为 2 的四行；中断返回，对应标为 3 的一行。下面分别说明。

9.4.5.1 启动中断

INT 已置位，处理器不再返回下一条指令的取指令状态，而是开始启动中断。它必须保存被中断程序的状态，以便服务完成后从原处继续；还必须装入高优先级中断程序的状态，开始满足其请求。

保存被中断程序的状态。程序状态是其全部资源内容的快照，包括程序涉及的存储单元、全部通用寄存器，以及 PC、PSR。

回想图 9.1，PSR[15] 表示管理模式或用户模式；PSR[10:8] 表示 PL0–PL7；PSR[2:0] 保存条件码，依次为 N、Z、P。

启动中断的第一步，是保存足以恢复程序的状态。LC-3 保存 PC、PSR。PC 指明恢复后应执行哪条指令；条件码可能被恢复后的 BR 使用；优先级决定哪些程序能够中断恢复后的程序；特权则决定它可以访问哪些处理器资源。

许多计算机还会保存通用寄存器，但这里不这样做。我们假定服务例程在使用任何通用寄存器之前都会自行保存，并在返回前恢复。LC-3 自动保存的状态只有 PC、PSR。

与 TRAP 相同，这些值保存在管理栈上。如果被中断程序处于用户模式，先把 USP 存入 Saved USP，再从 Saved SSP 把 SSP 装入 R6。随后将被中断程序的 PSR、PC 依次压入管理栈，在服务例程执行期间妥善保管。

装入中断服务例程的状态。原程序状态保存后，第二步是装入服务例程的 PC、PSR。中断服务例程与陷阱服务例程相似，是用于服务请求的程序片段，通常存放在

系统空间。

大多数处理器使用**向量中断** (vectored interrupt)。这与 TRAP 中的陷阱向量类似, 不过此处的 8 位向量由请求中断的设备提供。设备同时向处理器发送中断请求、优先级和 8 位**中断向量** (interrupt vector)。最高优先级请求的向量被送给处理器, 称为 INTV。

处理器接受中断后, 把 INTV 扩展成 16 位地址, 查**中断向量表** (Interrupt Vector Table)。陷阱向量表位于 $x0000-x00FF$; 中断向量表位于 $x0100-x01FF$, 每项保存一个中断服务例程起始地址。处理器把 INTV 对应表项的内容装入 PC。

例如, LC-3 键盘可以在每次按键时中断处理器; 键盘中断向量标识的表项中保存键盘中断服务例程的起始地址。

PSR 的装入方式如下: 服务例程尚未执行任何指令, PSR[2:0] 还没有有意义的条件码, 任意初始化为 010; 例程以管理模式运行, 因此 PSR[15]=0; PSR[10:8] 设为该中断请求的优先级。

启动阶段至此完成, 可以执行中断服务例程。

9.4.5.2 服务中断

PC 已包含中断服务例程的起始地址, 因此例程开始执行, 满足 I/O 设备的需求。

9.4.5.3 中断返回

每个中断服务例程的最后一条指令都是 RTI, 即陷阱或中断返回。到达 RTI 时, 设备的需求已处理完毕。

与 9.3.4 节的陷阱返回相同, RTI (操作码 1000) 从管理栈依次弹出 PC、PSR, 恢复到处理器中。条件码恢复成中断发生时的值, 以便后续 BR 使用; PSR[15]、PSR[10:8] 恢复原程序的特权和优先级。如果原程序无特权, 还要保存 SSP, 并将 USP 装入 R6。PC 则恢复为若未发生中断、本来将执行的下一条指令地址。

所有状态恢复后, 程序就能像什么也没发生一样继续执行。

9.4.6 一个例子

最后用一个例子结束中断驱动 I/O 的讨论。程序 A 正在运行, 优先级更高的设备 B 请求服务; 服务 B 的过程中, 更紧迫的设备 C 又请求服务。图 9.20 是相应执行流程。

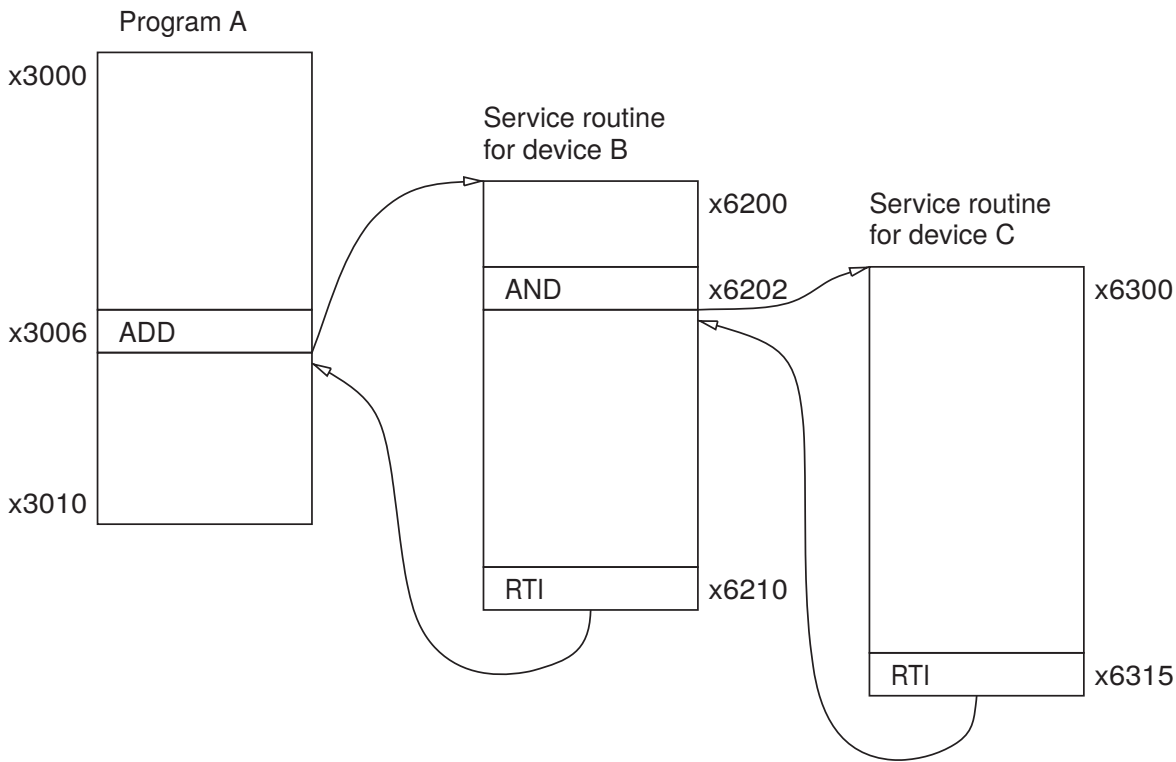


图 9.20 中断驱动 I/O 的执行流程

A 位于 x3000–x3010。执行 x3006 的 ADD 期间, B 发出中断请求和向量 xF1, 使 INT 置位。B 的服务例程位于 x6200–x6210, 末尾为 RTI。执行 x6202 的 AND 期间, C 发出请求和向量 xF2; 它的优先级更高, INT 再次置位。C 的例程位于 x6300–x6315, 末尾同样为 RTI。³

图 9.21 显示执行过程中管理栈和 PC 的若干快照。

³译注: 本例原图将两个中断服务例程放在用户地址区域, 以这些地址演示控制流和管理栈变化; 管理模式能够访问这些地址。前文关于例程通常位于系统空间的说明不改变本例给定地址。

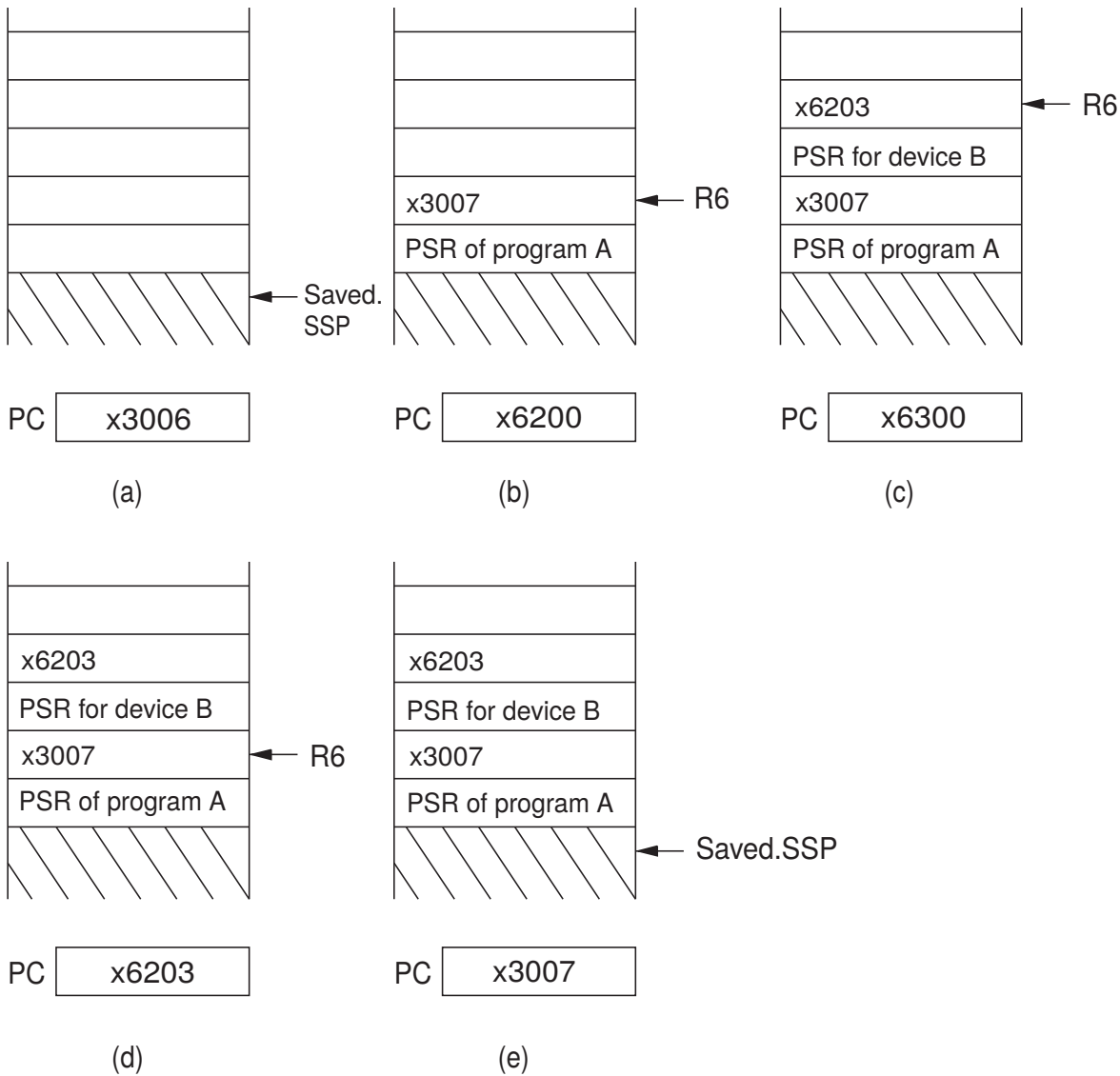


图 9.21 中断驱动 I/O 过程中管理栈与 PC 的快照

图 9.21a 是 A 尚未取出 x3006 指令时的状态。此时管理栈指针标为 Saved SSP，而不是 R6，因为尚未中断，R6 指向未画出的用户栈。

x3006 执行结束时检测到 B 导致的 INT。为使用管理栈，先把 R6 存入 Saved USP，再把 Saved SSP 装入 R6。⁴把 A 的 PSR（包含 ADD 产生的条件码）压栈，再把下一条指令地址 x3007 压栈。B 的向量扩展为 x01F1，从该处读出 x6200 装入 PC，得到图 9.21b。

B 执行到 x6202 结束时，检测到 C 的更高优先级中断。将 B 的 PSR（含该 AND 产生的条件码）和地址 x6203 压栈。C 的向量扩展为 x01F2，该表项内容 x6300 装入 PC，得到图 9.21c。

假定 C 执行到 x6315 的 RTI。管理栈弹出两个值，恢复 PC=x6203 及 B 的 PSR，包括 x6202 的 AND 产生的条件码，得到图 9.21d。

⁴译注：原文这里写为 Saved.UPC，应为 Saved USP。

B 从 x6203 继续, 最终执行 x6210 的 RTI。再次弹出两个值, 恢复 PC=x3007 及 A 的 PSR, 包括 x3006 的 ADD 产生的条件码。A 处于用户模式, 因此把 R6 存入 Saved SSP, 再从 Saved USP 恢复 R6, 得到图 9.21e。

A 从 x3007 继续执行。

9.4.7 不只是 I/O 设备

前面围绕优先级高于当前程序 I/O 设备讨论中断。但并不是所有中断都与 I/O 设备有关。任何独立于当前程序、优先级更高的事件, 都可以提供中断请求、INTV 和优先级, 申请中断计算机。如果它是优先级最高的有效请求, 就采用前述相同机制。

例如, 定时器中断用于标记一个时间单位已经过去; 机器检查中断通知计算机系统某个部分工作异常; 电源故障中断通知计算机, 例如有人拔掉了电源线。这些内容只能留待后续课程讨论。

9.5 了解中断之后, 再看轮询

9.5.1 问题

回想轮询: 不断检查相应状态寄存器的就绪位, 未置位就跳回去继续检查。例如, 向显示器输出字符串时, 通过轮询确认当前字符已显示完, 再发送下一字符。

我们想当然地认为, LDI (装入 DSR 就绪位)、BRzp (检查它, 就绪时不跳转)、STI (向 DDR 写下一字符) 这三条指令作为一个原子单元 (atomic unit) 工作。但如果同时允许中断呢? 假设在这段序列中、恰好在 STI 之前发生中断: LDI 原先发现 DDR 就绪, BRzp 不跳转; 可是中断服务结束后, DDR 可能已经不再就绪。计算机虽然执行 STI, 这次写入却不会生效。

一个简单但有些刻意的例子可以说明问题。某个 for 循环执行十次, 每次轮询显示器是否就绪, 再输出指定字符, 预期共输出十次。同时允许键盘中断, 键盘服务例程会回显输入字符。

循环体执行 LDI, 发现就绪, BRzp 不跳转; 但 STI 尚未执行时, 有人按键。键盘中断发生, 服务例程将输入字符写入 DDR 回显, 然后返回。

原循环体恢复, 仍“知道”显示器就绪, 于是执行 STI。可显示器还没有处理完键盘例程写入的字符! 由于 DDR 不就绪, 这次写入没有生效, 最终只输出九个指定字符, 而不是十个。

如果输出的是编码信息, 少一个字符导致无法解码, 问题就更严重。

简单的解决办法是在整个轮询期间禁用全部中断。但如果轮询持续很久, 中断也会被禁用很久。在关注高优先级事件从发生到得到服务所需时间的环境中, 这是不能接受的。

9.5.2 解决办法

图 9.22 给出更好的方案。

```

01          .ORIG x0420
02      ADD    R6,R6,#-1
03      STR    R1,R6,#0
04      ADD    R6,R6,#-1
05      STR    R2,R6,#0
06      ADD    R6,R6,#-1
07      STR    R3,R6,#0      ; Save R1,R2,R3 on the stack
08 ;
09      LDI    R1, PSR
0A      LD     R2,INTMASK
0B      AND    R2,R1,R2      ; R1=original PSR, R2=PSR with interrupts disabled
0C
0D POLL    STI    R1,PSR      ; enable interrupts (if they were enabled to begin)
0E      STI    R2,PSR      ; disable interrupts
0F      LDI    R3,DSR
10      BRzpb POLL          ; Poll the DSR
11      STI    R0,DDR      ; Store the character into the DDR
12      STI    R1,PSR      ; Restore original PSR
13
14      LDR    R3,R6,#0
15      ADD    R6,R6,#1
16      LDR    R2,R6,#0
17      ADD    R6,R6,#1
18      LDR    R1,R6,#0
19      ADD    R6,R6,#1      ; Restore R3,R2,and R1 from the stack
1A
1B      RTI
1C
1D INTMASK .FILL    xBFFF
1E PSR     .FILL    xFFFC
1F DSR     .FILL    xFE04
20 DDR     .FILL    xFE06
21
22      .END

```

图 9.22 既轮询，又允许中断

我们希望第 0F–11 行不可被中断。第 09 行先把 PSR 装入 R1，第 0A 行装入掩码，第 0B 行将它与原 PSR 按位 AND，在 R2 中形成禁用中断后的 PSR 值。⁵本节使用 PSR[14] 作为与当前程序相关的全局中断使能位。PSR 映射到 xFFFC。

第 0D 行把 R1 存入 PSR，启用中断；紧接着第 0E 行把 R2 存入 PSR，又禁用中断。在中断禁用期间执行 LDI、BRzpb；若设备就绪，再执行 STI，即第 0F、10、11 行。若设备未就绪，BRzpb 跳回第 0D 行，重新允许中断。⁶

这样，每次禁用中断只持续执行这段短指令序列所需的时间，随后重新允许中断。中断只须等待这一短序列，而不必等整个轮询过程结束。

⁵译注：原书只提到第 0A 行；按图 9.22，第 0A 行装入 INTMASK，第 0B 行才计算禁用中断后的值。

⁶译注：原书本段把第三条指令误写为 LDI，下一段又把 STI 所在行误写为 0D；按图 9.22 应为 STI、第 11 行。PSR[14] 的全局控制与 KBSR、DSR[14] 的设备中断使能是不同层次的控制。

习题

- 9.1** a. 什么是设备寄存器?
b. 什么是设备数据寄存器?
c. 什么是设备状态寄存器?
- 9.2** 为什么同步 I/O 不需要就绪位?
- 9.3** 原题提到,打字员很难以 300 MHz 处理器能够接收的最高速率,即每 33 纳秒一个字符,提供键盘输入。假设平均每词包括词间空格共六个字符,打字员每分钟必须输入多少词,才会超过处理器处理输入的能力?⁷
- 9.4** 以下交互通常是同步还是异步? 分别在什么条件下同步,什么条件下异步?
a. 遥控器与电视机。
b. 邮递员通过信箱与你交互。
c. 鼠标与你的 PC。
- 9.5** KBSR 的位 [15] 有什么作用?
- 9.6** 如果程序读取 KBDR 前不检查 KBSR 就绪位,会出现什么问题?
- 9.7** 以下哪些组合描述了 9.2.2.2 节的系统?
a. 存储器映射、中断驱动。
b. 存储器映射、轮询。
c. 专用 I/O 操作码、中断驱动。
- 9.8** 编写程序,检查存储单元 $\times 4000$ 的初值是否为有效 ASCII 码。若有效,输出该字符;若无效,不输出任何内容。
- 9.9** 如果键盘硬件写入 KBDR 前不检查 KBSR,可能出现什么问题?
- 9.10** 如果显示硬件写入 DDR 前不检查 DSR,可能出现什么问题?
- 9.11** 中断驱动 I/O 与轮询,哪一种效率更高? 解释原因。
- 9.12** Adam H. 设计了一个不需要键盘状态寄存器的 LC-3 变体,改用可读写的键盘数据与状态寄存器 KBDSR,其中包含与 KBDR 相同的数据。需要输入的程序等待 KBDSR 出现非零值,将其作为最后一次按键的 ASCII 码读出,再写入零,表示已读取该字符。修改 9.2.2 节的基本输入服务,实现 Adam 的方案。⁸

⁷译注: 原题引用 9.2.1.3, 沿用 300 MHz、33 纳秒的例子; 本版正文在 9.2.1.2 使用 2 GHz、5 纳秒。题设数值保留。

⁸译注: 原题写为 8.2.2, 按内容应指本章 9.2.2。

- 9.13** 几名计算机工程学生把修改 LC-3 作为毕业设计，设计 LC-4 时，为节省设备寄存器，将 KBSR、DSR 合并为 I/O 状态寄存器 IOSR。IOSR[15] 是键盘就绪位，IOSR[14] 是显示器就绪位。这对需要执行 I/O 的程序有什么影响？这是一个糟糕的设计决定吗？
- 9.14** 某条 LC-3 装入指令指定地址 xFE02。怎样知道应从 KBDR 装入，还是从地址为 xFE02 的存储单元装入？
- 9.15** 相比每次需要 I/O 都自己编写例程，通过陷阱例程执行 I/O 有哪些优点？
- 9.16**
- LC-3 可以实现多少个陷阱服务例程？为什么？
 - 为什么必须用返回指令从陷阱例程返回，而不能改用无条件 BR？⁹
 - 处理一条 TRAP 指令需要访问存储器多少次？假设 TRAP 已在 IR 中。
- 9.17** 参照图 9.14 的 HALT 服务例程。
- 停机后，由什么重新启动时钟？提示：清除 MCR[15] 后，HALT 服务例程怎样才能返回？
 - 哪条指令真正使机器停机？
 - 机器重新启动后，执行的第一条指令是什么？
 - HALT 例程的返回指令将返回哪里？¹⁰
- 9.18** 考虑下面的 LC-3 汇编程序。

```

                .ORIG    x3000
L1              LEA      R1, L1
                AND      R2, R2, x0
                ADD      R2, R2, x2
                LD       R3, P1
L2              LDR      R0, R1, xC
                OUT
                ADD      R3, R3, #-1
                BRZ      GLUE
                ADD      R1, R1, R2
                BR       L2
GLUE            HALT
P1              .FILL    xB
                .STRINGZ "HBoeoakteSmtHaotren!s"
                .END

```

- 汇编并装入后，x3005 中存储什么二进制位模式？
- 执行 x3005 中的指令后，下一条执行的指令位于哪个存储器地址？

⁹译注：原题此处写为 RET，但本版 9.3.4 明确规定陷阱服务使用 RTI 返回；这里按本版机制理解“返回指令”。

¹⁰译注：原题写为 RET，图 9.14 实际使用 RTI。

- c. 执行 x3006 中的指令之前，上一条执行的指令位于哪个存储器地址？
- d. 程序输出什么？

9.19 下面的 LC-3 程序汇编后执行，没有汇编期或运行期错误。假设开始前所有寄存器均初始化为 0，程序输出什么？

```

                .ORIG      x3000
                LEA        R0, LABEL
                STR        R1, R0, #3
                TRAP       x22
                TRAP       x25
LABEL           .STRINGZ  "FUNKY"
LABEL2          .STRINGZ  "HELLO WORLD"
                .END

```

9.20 例 9.1 假设用户只输入大写字母和数字 7。如果输入 \$ 呢？更好的程序应检查输入是否确实为 26 个大写字母之一，否则采取措施。扩展例 9.1，增加无效数据检查：输入大写字母时输出相应小写字母；输入任何非大写字母时终止。

9.21 假设执行下面的程序之前，另一模块已把一个大于 2、小于 32,768 的整数存入 A。

```

                .ORIG      x3000
                AND        R4, R4, #0
                LD         R0, A
                NOT        R5, R0
                ADD        R5, R5, #2
                ADD        R1, R4, #2
                ;
REMOD          JSR        MOD
                BRz        STORE0
                ;
                ADD        R7, R1, R5
                BRz        STORE1
                ADD        R1, R1, #1
                BR         REMOD
                ;
STORE1         ADD        R4, R4, #1
STORE0         ST         R4, RESULT
                TRAP       x25
                ;
MOD            ADD        R2, R0, #0
                NOT        R3, R1
                ADD        R3, R3, #1
DEC            ADD        R2, R2, R3
                BRp        DEC
                RET
                ;
A              .BLKW      1
RESULT         .BLKW      1
                .END

```

用不超过 20 个词概括该程序的功能。

9.22 回想机器忙闲状态的例子。假设表示哪些机器忙、哪些空闲的位模式存于 $x4001$ ，编写子程序完成：

- a. 检查是否没有机器忙；若是，返回 1。
- b. 检查是否全部机器都忙；若是，返回 1。
- c. 返回忙的机器数量。
- d. 返回空闲机器数量。
- e. R5 传入机器编号，检查该机器是否忙；若忙，返回 1。
- f. 返回某台空闲机器的编号。

9.23 陷阱例程起始地址保存在 TRAP 指令指定的地址中。为什么不把例程的第一条指令直接放在那里？假设每个例程最多需要 16 条指令，修改 LC-3 TRAP 的语义，使陷阱向量直接给出服务例程起始地址。

9.24 下面是交给 LC-3 汇编器的程序片段。它从控制台读入若干行文本到缓冲区，查找指定字符，输出出现次数。输入以 EOT 结束，保证不超过 1000 个字符。读完文本后，再读取要统计的字符。

真正执行统计的 COUNT 由另一人编写，位于 $x3500$ 。调用时要求 R5 为缓冲区地址、R6 为待统计字符的地址；缓冲区以 NULL 标记结束；返回时 R6 为计数值。

把二进制计数转换为 ASCII 数字并显示的 OUTPUT 也由另一人编写，位于 $x3600$ ，要求 R6 为待输出数值。输入及调用代码如下：

```

        .ORIG    x3000
        LEA      R1, BUFFER
G_TEXT  TRAP      x20          ; Get input text
        ADD      R2, R0, x-4
        BRz      G_CHAR
        STR      R0, R1, #0
        ADD      R1, R1, #1
        BRnzp    G_TEXT
G_CHAR  STR      R2, R1, #0    ; x0000 terminates buffer
        TRAP      x20          ; Get character to count
        ST       R0, S_CHAR
        LEA      R5, BUFFER
        LEA      R6, S_CHAR
        LD       R4, CADDR
        JSRR     R4           ; Count character
        LD       R4, OADDR
        JSRR     R4           ; Convert R6 and display
        TRAP      x25
CADDR   .FILL    x3500        ; Address of COUNT
OADDR   .FILL    x3600        ; Address of OUTPUT
BUFFER  .BLKW    1001
S_CHAR  .FILL    x0000
        .END

```

这段代码有一个问题。是什么？怎样修复？问题不是没有给出 COUNT、OUTPUT 的代码。

9.25 考虑下面的 LC-3 汇编程序。

```

                                .ORIG    x3000
                                LEA      R0, DATA
                                AND      R1, R1, #0
                                ADD      R1, R1, #9
LOOP1      ADD      R2, R0, #0
                                ADD      R3, R1, #0
LOOP2      JSR      SUB1
                                ADD      R4, R4, #0
                                BRzp     LABEL
                                JSR      SUB2
LABEL      ADD      R2, R2, #1
                                ADD      R3, R3, #-1
                                BRP      LOOP2
                                ADD      R1, R1, #-1
                                BRp      LOOP1
                                HALT
DATA      .BLKW    10  x0000
SUB1      LDR      R5, R2, #0
                                NOT      R5, R5
                                ADD      R5, R5, #1
                                LDR      R6, R2, #1
                                ADD      R4, R5, R6
                                RET
SUB2      LDR      R4, R2, #0
                                LDR      R5, R2, #1
                                STR      R4, R2, #1
                                STR      R5, R2, #0
                                RET
                                .END

```

假设执行前 **DATA** 各单元已填入初值，执行结束后 **DATA** 中的值与初值有什么关系？

9.26 下面的程序本应在屏幕上打印数字 5，但不能正常工作。为什么？请用不超过十个词回答。


```

                                .ORIG    x3000
                                JSR      A
                                OUT
                                BRnzp   DONE
A                                AND      R0, R0, #0
                                ADD      R0, R0, #5
                                JSR      B
                                RET
                                DONE
                                HALT
ASCII                           .FILL    x0030
B                               LD        R1, ASCII
                                ADD      R0, R0, R1
                                RET
                                .END

```

9.27 图 9.14 的服务例程通过清除 MCR[15]，即 RUN 锁存器，使机器停机。停机后，第 19–1C 行的指令有什么作用？¹¹

9.28 假设新增一个从 x4000 开始的服务例程，读取字符并回显，x0072 中包含 x4000。例程如下：

```

                                .ORIG    x4000
                                ST R7,  SaveR7
                                GETC
                                OUT
                                LD R7,  SaveR7
                                RET
SaveR7  .FILL    x0000

```

- a. 哪条指令可以调用它？
- b. 这个服务例程能够正常工作吗？解释原因。

9.29 下面的代码序列 a、b 分别汇编。其中有两个错误，会在汇编期或链接期被发现。指出错误，解释原因，并说明在哪个阶段发现。

¹¹译注：原题称第 14 行清除 RUN；本版图 9.14 中执行此操作的是第 13 行 STI；题设的第 19–1C 行也沿用旧行号，本版恢复寄存器并返回的代码在第 18–1A 行。

```
a.          .ORIG x3500
            SQR T  ADD  R0, R0, #0
                ; code to perform square
                ; root function and
                ; return the result in R0
            RET
            .END
b.          .EXTERNAL SQR T
            .ORIG  x3000
            LD      R0, VALUE
            JSR     SQR T
            ST      R0, DEST
            HALT
VALUE       .FILL   x30000
DEST        .FILL   x0025
            .END
```

9.30 下面的程序尚未写完。它询问用户姓名，把 `Hello, name` 作为字符串存入 `HELLO` 指定的地址，再输出该句。按 **Enter** 表示姓名输入完成，ASCII 码为 `x0A`；姓名不超过 25 个字符。若用户输入 **Onur** 后回车，输出应恰为：

```
Please enter your name: Onur
Hello, Onur
```

在 (a)–(d) 填入指令，补全程序。

```

                                .ORIG x3000
                                LEA    R1, HELLO
AGAIN    LDR    R2, R1, #0
                                BRz    NEXT
                                ADD    R1, R1, #1
                                BR      AGAIN
NEXT     LEA    R0, PROMPT
                                TRAP    x22                ; PUTS
                                ----- (a)
AGAIN2   TRAP    x20                ; GETC
                                TRAP    x21                ; OUT
                                ADD    R2, R0, R3
                                BRz    CONT
                                ----- (b)
                                ----- (c)
                                BR      AGAIN2
CONT     AND    R2, R2, #0
                                ----- (d)
                                LEA    R0, HELLO
                                TRAP    x22                ; PUTS
                                TRAP    x25                ; HALT
NEGENTER .FILL xFFF6                ; -x0A
PROMPT   .STRINGZ "Please enter your name: "
HELLO    .STRINGZ "Hello, "
                                .BLKW #25
                                .END

```

9.31 补全下面的程序，使其在显示器上打印 ABCFGH。在 (a)–(d) 填入指令。

```

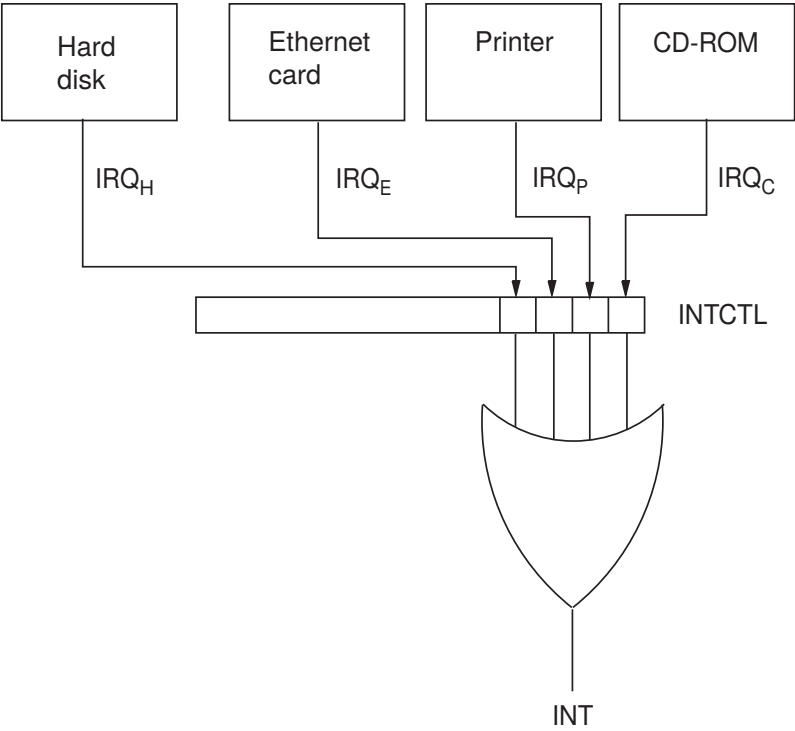
                                .ORIG x3000
                                LEA    R1, TESTOUT
BACK_1    LDR    R0, R1, #0
                                BRz    NEXT_1
                                TRAP   x21
                                ----- (a)
                                BRnzp  BACK_1
                                ;
NEXT_1    LEA    R1, TESTOUT
BACK_2    LDR    R0, R1, #0
                                BRz    NEXT_2
                                JSR    SUB_1
                                ADD    R1, R1, #1
                                BRnzp  BACK_2
                                ;
NEXT_2    ----- (b)
                                ;
SUB_1     ----- (c)

K         LDI    R2, DSR
                                ----- (d)

                                STI    R0, DDR
                                RET
DSR       .FILL  xFE04
DDR       .FILL  xFE06
TESTOUT   .STRINGZ "ABC"
                                .END

```

9.32 一家本地公司要制造真实的 LC-3，并接入四个中断驱动 I/O 设备，以便联网。设备请求服务时置位 **IRQ**，在映射到 **xFF00** 的中断控制寄存器 **INTCTL** 中设置一位，进而置位 LC-3 数据通路中的 **INT**。中断服务例程判断请求来源，调用该设备的子程序。若多个设备同时请求，只调用最高优先级设备的子程序；子程序执行期间清除 **INTCTL** 中对应位。



各设备子程序入口标号为 **HARDDISK**、**ETHERNET**、**PRINTER**、**CDROM**。例如，最高优先级请求来自打印机时，执行 `JSR PRINTER`。

按以下优先顺序补全中断服务代码的 (a)–(k)。本题数值越小优先级越高：1. 硬盘；2. 以太网卡；3. 打印机；4. **CD-ROM**。

```

DEV0      LDI    R1, INTCTL
          LD     R2, ----- (a)
          AND    R2, R2, R1
          BRnz   DEV1
          JSR    ----- (b)
          ----- (c)
          ;
DEV1      LD     R2, ----- (d)
          AND    R2, R2, R1
          BRnz   DEV2
          JSR    ----- (e)
          ----- (f)
          ;
DEV2      LD     R2, ----- (g)
          AND    R2, R2, R1
          BRnz   DEV3
          JSR    ----- (h)
          ----- (i)
          ;
DEV3      JSR    ----- (j)
          ;
END       ----- (k)

INTCTL    .FILL   xFF00
MASK8     .FILL   x0008
MASK4     .FILL   x0004
MASK2     .FILL   x0002
MASK1     .FILL   x0001
```

9.33 中断驱动 I/O:

- a. 下面的 LC-3 程序做什么？

```

                                .ORIG      x3000
                                LD          R3, A
                                STI        R3, KBSR
AGAIN    LD          R0, B
                                TRAP      x21
                                BRnzp     AGAIN
A        .FILL      x4000
B        .FILL      x0032
KBSR     .FILL      xFE00
                                .END

```

- b. 有人按键时，程序被中断，执行下面的键盘中断服务例程。它做什么？

```

                                .ORIG      x1000
                                LDI        R0, KBDR
                                TRAP      x21
                                TRAP      x21
                                RTI
KBDR     .FILL      xFE02
                                .END

```

注意：RTI 使计算机返回被中断程序，将 PC 恢复为中断发生时本来将要取出的指令地址。¹²

- c. 若 a 中的程序开始运行后有人按下一个键，屏幕上会显示什么？
 d. c 中输入的数字会在屏幕上出现多少次？为什么正确回答是“不能确定”？

9.34 下面的 LC-3 程序做什么？

¹²译注：原题说 RTI 将在第 10 章学习，属旧版交叉引用；本版已在 9.3、9.4 节解释。

```

                                .ORIG    x3000
                                LD        R0, ASCII
                                LD        R1, NEG
AGAIN    LDI        R2, DSR
                                BRzp     AGAIN
                                STI        R0, DDR
                                ADD        R0, R0, #1
                                ADD        R2, R0, R1
                                BRnp     AGAIN
                                HALT
ASCII    .FILL      x0041
NEG      .FILL      xFFB6      ; -x004A
DSR      .FILL      xFE04
DDR      .FILL      xFE06
                                .END

```

- 9.35** 启动中断服务时，N、Z、P 条件码会被压栈。用一个简单例子说明，如果不保存条件码，怎样产生错误结果。
- 9.36** 在 9.4.6 节的例子中，x01F1、x01F2 的内容分别是什么？它们属于一个更大的结构，该结构叫什么？提示：参见表 A.3。
- 9.37** 扩展 9.4.6 节的例子：C 执行 x6310 的指令期间，更紧迫的设备 D 请求中断。假设 D 的向量为 xF3，服务例程位于 x6400–x6412。给出执行流程中各个相关时刻的栈和 PC 内容。
- 9.38** 如果习题 9.37 的 D 优先级低于 C、但高于 B，重新分析该题。
- 9.39** 编写键盘中断处理程序：缓冲区位于 x4000–x40FE，接收下一字符并存入下一个“空闲”单元。x40FF 保存下一个可用单元地址。如果缓冲区已满，即 x40FE 已存入字符，就显示 Character cannot be accepted; input buffer full.
- 9.40** 修改习题 9.39：缓冲区改为 x4000–x40FC。x40FF 仍保存下一空闲单元地址，x40FE 保存最早字符的地址，x40FD 保存字符数量。其他程序可以移除字符。修改处理程序，使 x40FC 填满后，下一个写入位置回到 x4000，前提是其原字符已被移除。缓冲区满时，仍显示 Character cannot be accepted; input buffer full.
- 9.41** 将习题 9.40 的处理程序与移除缓冲区字符的程序一起使用。什么问题可能使添加字符的中断处理程序与移除字符的程序不能正确协作？
- 9.42** 假设键盘中断向量为 x34，服务例程从 x1000 开始。能据此推断哪个存储

单元包含什么内容?

9.43 两名学生编写的中断服务例程完成完全相同的工作,但第一人误用 **RET** 结束,第二人正确使用 **RTI**。第一人的错误导致三个问题,描述其中任意两个。

9.44 [★] 每个 ASCII 码占 8 位,因此 LC-3 一个存储字可以保存两个 ASCII 码。用户输入 $2n$ 个字符后按 **Enter**,子程序 **PACK** 把相应 ASCII 码存入从 **A** 开始的 n 个连续单元,每个单元两个。假设用户绝不输入奇数个字符。补全程序中的空白。

如果用户输入 **Please help!** 后按 **Enter**,程序做什么?

```

                .ORIG x7020
PACK            ST R7, SAVER7
                ST R6, SAVER6
                ST R4, SAVER4
                ST R3, SAVER3
                LEA R6, A          ; R6 is the pointer
                AND R4, R4, #0
                ADD R4, R4, #8     ; R4 is our counter
                AND R3, R3, #0
                LEA R0, PROMPT
                TRAP x22

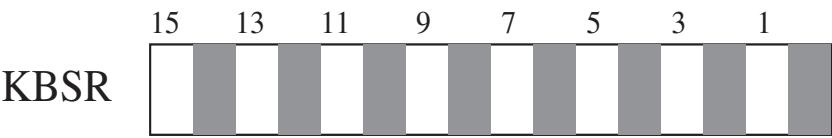
POLL            ----- (a)
                BRzpp POLL
                ----- (b)
                LD R0, NEG_LF
                ADD R0, R7, R0
                ----- (c)
                ADD R4, R4, #0
                BRz NOSHIFT
SHIFT          ADD R7, R7, R7
                ADD R4, R4, #-1
                BRp SHIFT
                ADD R3, R7, #0
                BRnzpp POLL
NOSHIFT        ADD R3, R3, R7
                ----- (d)
                ADD R6, R6, #1
                ADD R4, R4, #8
                BRnzpp POLL
DONE           LD R7, SAVER7
                LD R6, SAVER6
                LD R4, SAVER4
                LD R3, SAVER3
                LEA R0, A          ; Returns a pointer to the characters
                RET

KBSR           .FILL xFE00
KBDR           .FILL xFE02
NEG_LF         .FILL xFFF6
PROMPT         .STRINGZ "Please enter a string: "
A              .BLKW #5
SAVER7         .BLKW #1
SAVER6         .BLKW #1
SAVER4         .BLKW #1
SAVER3         .BLKW #1
                .END

```

9.45 [★] 现在要支持八个输入键盘,需要 **KBSR** 中的八个就绪位及八个独立

KBDR。将 KBSR 的八个奇数编号位用作就绪位，其他八位设为 0，如图。



各设备的映射地址和就绪位如下：

地址	寄存器	就绪位
xFE04	KBSR	—
xFE06	KBDR1	KBSR[1]
xFE08	KBDR2	KBSR[3]
xFE0A	KBDR3	KBSR[5]
xFE0C	KBDR4	KBSR[7]
xFE0E	KBDR5	KBSR[9]
xFE10	KBDR6	KBSR[11]
xFE12	KBDR7	KBSR[13]
xFE14	KBDR8	KBSR[15]

编写程序轮询键盘，将最高优先级键盘输入的 ASCII 码装入 R0。如果键盘 1 已有输入，读取 KBDR1；否则检查键盘 2，依此类推。KB1 最高，KB8 最低。填入缺失指令，使下面的程序完成任务。

```
.ORIG X3000
LD      R0, KBDR1
POLL    LDI      R1, KBSR
        BRz      POLL
        AND      R2, R2, #0
        ADD      R2, R2, #2
AGAIN    ----- (a)
        BRnp     FOUND
        ADD      R0, R0, #2
        ----- (b)
        ----- (c)
        BRnp     AGAIN
        HALT
FOUND    ----- (d)
        HALT
KBSR    .FILL    xFE04
KBDR1   .FILL    xFE06
.END
```

9.46 [★] 下面的程序用 JSR PUSH 压栈、JSR POP 弹栈。

```

                                .ORIG X3000
                                LEA  R6, STACK\_BASE

X                                TRAP  x20                ;GETC
                                TRAP  x21                ;OUT
                                ADD   R1, R0, x-0A        ;x0A is ASCII code for line feed,
                                                ;x-0A is the negative of x0A
                                BRz   Y
                                JSR   PUSH
                                BRnzp X

Y                                LEA   R2, STACK\_BASE
                                NOT   R2, R2
                                ADD   R2, R2, #1
                                ADD   R3, R2, R6
                                BRz   DONE
                                JSR   POP
                                TRAP  x21                ;OUT
                                BRnzp Y

DONE                            TRAP  x25                ;HALT
STACK                          .BLKW  5
STACK\_BASE                    .FILL  x0FFF

PUSH                           ADD   R6, R6, #-1
                                STR   R0, R6, #0
                                RET

POP                             LDR   R0, R6, #0
                                ADD   R6, R6, #1
                                RET

                                .END

```

用户依次按 a、b、c、Enter，屏幕上会出现什么？如果依次按 a、b、c、d、e、f、g、h、Enter，又会发生什么？

9.47 新增一个由 TRAP x9A 调用的服务例程。它等待输入小写字母，再回显相应大写字母。假设用户只输入小写字母。代码如下：

```

                                .ORIG      x2055
                                ----- (a)
                                ST R1, SaveR1
                                ST R0, SaveR0
                                TRAP x20
                                LD R1, A
                                ----- (b)
                                TRAP x21
                                ----- (c)
                                LD R1, SaveR1
                                LD R0, SaveR0
                                JMP R7
SaveR1 .BLKW 1
SaveR0 .BLKW 1
A      .FILL ----- (d)
_____ .BLKW          ; (e) a missing label

                                .END
```

为了让 TRAP x9A 调用它，哪个地址的存储单元必须包含什么值？¹³补全汇编程序中的三条指令、一个标号及 .FILL 的操作数。

9.48 某程序员编写、汇编并执行以下程序，开始时 PC=x3000。

```

                                .ORIG x3000

                                LEA R0, Message
                                TRAP x01
                                TRAP x22    ; What is the output here?
                                TRAP x25

                                Message .STRINGZ "Cat in the hat."

                                .END
```

除已用过的表项外，陷阱向量表还包含：

Memory Address	Memory Contents
x0000	x0100
x0001	x0102
x0002	x0107
x0003	x010A

假设存储器中还预先装入了以下附加陷阱服务例程：

¹³译注：本题原代码以 **JMP R7** 返回，沿用 **TRAP** 将返回地址写入 **R7** 的旧机制，与本版正文的管理栈 / **RTI** 机制不同。原题代码及空白保留。

```

                                .ORIG x0100

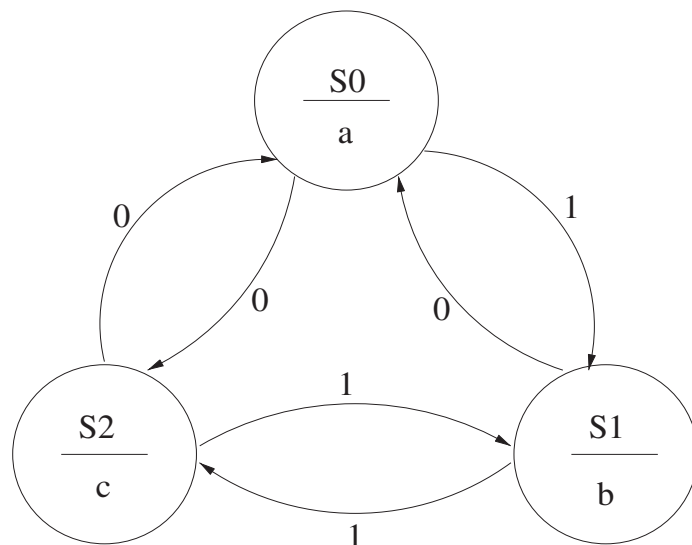
                                LD R7, SaveR7
                                RET
                                ST R7, SaveR7
                                TRAP x02
                                AND R1, R1, #0
                                STR R1, R0, #3
                                RET
                                AND R1, R1, #0
                                STR R1, R0, #5
                                TRAP x00
                                RET
SaveR7    .BLKW #1

                                .END

```

译注：本题原陷阱服务例程以 *RET* 返回，沿用旧版的 *R7* 返回机制；本版正文则使用管理栈和 *RTI*。题设代码原样保留。程序执行的结果是什么？

9.49 [×] 下面的状态机接收输入序列，产生输出序列，初态为 *S0*。



例如，输入 100 产生输出 **bac**。我们编写了模拟该状态机的程序，从键盘请求输入，在屏幕显示输出。上述例子会显示：

```

INPUT (either 0 or 1): 1
OUTPUT: b
INPUT (either 0 or 1): 0
OUTPUT: a
INPUT (either 0 or 1): 0
OUTPUT: c

```

每个空白填入一行 LC-3 汇编代码，补全程序。可以假设用户能正确输入 0 或 1，不需要检查无效输入。

```

                                .ORIG      x3000
                                LEA         R6, S0
Loop ----- (a)
                                TRAP        x22
                                TRAP        x20          ; inputs a character
                                TRAP        x21

                                LD          R1, NEGASCII
                                ADD         R0, R0, R1
                                ----- (b)
                                LDR         R6, R6, #0
                                LD          R0, NEWLINE
                                TRAP        x21
                                LEA         R0, OUTPUT
                                TRAP        x22
                                ----- (c)
                                TRAP        x21
                                LD          R0, NEWLINE
                                TRAP        x21
                                BRnzp      LOOP

S0  .FILL      S2
    .FILL      S1
    .FILL      x0061

S1  .FILL      S0
    .FILL      S2
    .FILL      x0062

S2  .FILL      _____ (d)
    .FILL      _____ (e)
    .FILL      _____ (f)

NEGASCII .FILL      xFFD0          ; the value -48
OUTPUT   .STRINGZ   "OUTPUT:"
INPUT    .STRINGZ   "INPUT (either 0 or 1):"
NEWLINE  .FILL      x000A

.END

```

9.50 [*] 此前唯一的输出设备是显示器，其寄存器地址为 xFE04、xFE06。现在增加一盏灯，只需一个映射到 xFE08 的设备寄存器。写 1 开灯，写 0 关灯。

一位 Aggie 编写程序，用键盘中断控制灯：按 0 关灯，按 1 使灯反复闪烁。下面是主程序和键盘中断服务例程。

The User Program:

```

                                .ORIG x3000
0                                LEA R7, LOOP
1      LOOP                    LDI R0, ENABLE
2                                LD R1, NEG_OFF
3                                ADD R0, R0, R1 ; check if switch is on
4                                BRnp BLINK
                                ;
5                                AND R0, R0, #0
6                                STI R0, LIGHT ; turn light off
7                                RET
                                ;
8      BLINK                    ST R7, SAVE_R7 ; save linkage
9                                LDI R0, LIGHT
A                                ADD R0, R0, #1
B                                AND R0, R0, #1 ; toggle LIGHT between 0 and 1
C                                STI R0, LIGHT
D                                JSR DELAY ; 1 second delay
E                                LD R7, SAVE_R7
F                                RET ; <-- Breakpoint here
                                ;
LIGHT .FILL xFE08
ENABLE .FILL x4000
NEG_OFF .FILL x-30
SAVE_R7 .BLKW #1
                                .END

```

The Keyboard Interrupt Routine:

```

                                .ORIG x1500
0                                ADD R6, R6, #-1 ; <-- Breakpoint here
1                                STR R0, R6, #0 ; save R0 on stack
2                                ADD R6, R6, #-1
3                                STR R7, R6, #0 ; save R7 on stack
                                ;
4                                TRAP x20
5                                STI R0, ENABLE2
                                ;
6                                RTI ; <-- Breakpoint here
7      ENABLE2 .FILL x4000
                                .END

```

DELAY 子程序让开、关灯动作相隔一秒，使肉眼看得清闪烁；它不修改任何寄存器。可惜，这位 Aggie 的程序又出了错，于是他开始调试。

他在 x1500、x1506、x300F 设置断点，初始化 PC=x3000、键盘 IE=1、x0180 内容为 x1500。按 Run 后，程序第一次在 PC=x1500 停止；随后又按三次 Run，每次都在断点停下。整个运行期间，他恰好按了一次键。

下表记录每次断点时各寄存器与存储单元的内容。假设启动中断时先压入 PSR，再压入 PC。补全该表。

	Initial	Breakpoint 1	Breakpoint 2	Breakpoint 3	Breakpoint 4
PC	x3000	x1500	x1506	x1506	x300F
R0	x1234		x0030		
R6	x3000				
R7	x1234				
M[x2FFC]	x0000				
M[x2FFD]	x0000				
M[x2FFE]	x0000	x300D			
M[x2FFF]	x0000	x8001			
M[x4000]	x0031				
M[xFE00]	x4000				

9.51 [★] 下面的用户程序以优先级 0 运行，已汇编并装入存储器。

```

                                .ORIG x8000
                                LD R0, Z
AGAIN    ADD R0, R0, #-1
                                BRnp AGAIN
                                LD R0, W
                                BRp L1
                                LD R0, X
                                TRAP x21
                                BRnzp DONE
L1        LEA R0, Y
                                TRAP x22
DONE      HALT

X          .FILL x34
Y          .STRINGZ "OOOOPS!"
Z          .FILL x100
W          .BLKW #1
                                .END
```

执行前，另一程序向 W 装入一个值，并在 DONE 设置断点。启动后，在断点停机前发生多次中断，相应服务例程都已执行。最终在断点停机，检查下图所示存储器，以及 R6（题目称其为管理栈指针）。

	Memory
x2FF8	x0601
x2FF9	x0601
x2FFA	x0500
x2FFB	x0504
x2FFC	x0204
x2FFD	x0201
x2FFE	x8004
x2FFF	x8002
x3000	x8010
x3001	x8012

R6x3000

用户程序在显示器上输出什么？你怎样判断？

9.52 [★] 补全检测回文的程序。回文从左到右、从右到左读取都相同，例如 racecar、112282211。本程序输入只包含小写字母，不包含空格或大写字母。

程序同时使用栈和队列。**PUSH**、**POP** 分别压栈、弹栈；**ENQUEUE** 向队尾加入元素，**DEQUEUE** 从队首移除元素。访问这些结构的子程序如下：

```

PUSH      .ORIG x3050
          ADD R6, R6, #-1
          STR R0, R6, #0
          RET
POP        LDR R0, R6, #0
          ADD R6, R6, #1
          RET
STACK     .BLKW #20
          .END

          .ORIG x3080
ENQUEUE   ADD R5, R5, #1
          STR R0, R5, #0
          RET
DEQUEUE   LDR R0, R4, #0
          ADD R4, R4, #1
          RET
QUEUE     .BLKW #20
          .END
```

程序分两个阶段。第一阶段逐个接收键盘字符，以 **Enter**（换行）结束。每个字符的 **ASCII** 码压栈，其负值则从队尾入队。向队尾插入元素称为入队。

第二阶段分别从栈和队列逐个移除元素，检查字符。若是回文，向 **RESULT** 存入 1；否则存入 0。假设用户输入不超过 20 个字符。补全以下主程序：


```

                                .ORIG X3000
                                LEA    R4, QUEUE
                                LEA    R5, QUEUE
                                ADD     R5, R5, #-1
                                LEA    R6, ENQUEUE           ; Initialize SP
                                LD      R1, ENTER
                                AND     R3, R3, #0
                                ----- (a)
PHASE1  TRAP x22
                                TRAP x20
                                ----- (b)
                                BRz PHASE2
                                JSR PUSH
                                ----- (c)
                                ----- (d)
                                JSR ENQUEUE
                                ADD R3, R3, #1
                                BRnzp PHASE1
;
PHASE2  JSR POP
                                ----- (e)
                                JSR DEQUEUE
                                ADD R1, R0, R1
                                BRnp FALSE
                                ----- (f)
                                ----- (g)
                                BRnzp PHASE2
;
TRUE    AND R0, R0, #0
        ADD R0, R0, #1
        ST R0, RESULT
        HALT
FALSE   AND R0, R0, #0
        ST R0, RESULT
        HALT
RESULT  .BLKW #1
ENTER   .FILL x-0A
PROMPT  .STRINGZ "Enter an input string: "
        .END
```

9.53 [★] 键盘中断已经熟悉，现在引入 INTA、INTB 两种中断，所需硬件已添加。INTA 优先级 2、向量 x50；INTB 优先级 4、向量 x60。优先级由 PSR[10:8] 指定。完整 PSR 说明如下：

	15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
PSR:	Pr	0	0	0	0	Priority	0	0	0	0	0	0	N	Z	P	

其中 PSR[15] 为 0 表示管理模式、为 1 表示用户模式；PSR[14:11] 为 0000；PSR[10:8] 为优先级 0-7；PSR[7:3] 为 00000；PSR[2:0] 为 N、Z、P 条件码。用户程序及两个中断服务例程如下。用户程序在第 1 周期开始，以优先级 0 运行。

User program:INTA service routine:INTB service routine:

.ORIG x3000
AND R0,R0,#0
ADD R0,R0,#5
LD R1,COUNT
NOT R0,R0
ADD R0,R0,#1
AGAIN ADD R2,R0,R1
BRz DONE
ADD R1,R1,#-1
BRnzp AGAIN
DONE TRAP x25
COUNT .FILL x000F
.END

VAL

.ORIG x1000
AND R5,R4,#0
ADD R5,R5,#-1
LD R3,VAL
ADD R3,R3,R5
ST R3,VAL
RTI
.BLKW 1
.END

VAL2

.ORIG x2000
LDI R4,VAL2
NOT R4,R4
ADD R4,R4,#1
STI R4,VAL2
RTI
.FILL xFE08
.END

假设两种中断均已使能。¹⁴用户模式启动中断需要 22 个周期，即从检查中断到服务例程开始；管理模式需要 21 个周期。原题假定你已从“问题 1”知道各指令的周期数。¹⁵

为支持 INTA、INTB，中断向量表需有哪些表项？填写地址和内容。

Memory address	Content

假设 INTA 在第 30 周期请求服务，INTB 在第 68 周期请求服务，各例程在哪个周期开始执行？下表给出用户程序开始前 x2FFA–x3002 的内容，填写第 100 周期这些单元和栈指针的内容。

¹⁴译注：本题给定 PSR[14:11]=0000，沿用未使用 PSR[14] 的布局，与 9.5 节引入的全局中断使能位不同。本题保留原布局和“两种中断均已使能”的题设。

¹⁵译注：原书这里引用的“问题 1”与本章习题 9.1 不对应，且未在本题另列各指令周期数。保留给定时序条件，不擅自补入假设。

	Initial	At the end of cycle 100
x2FFA	x0001	
x2FFB	x0010	
x2FFC	x0100	
x2FFD	x1000	
x2FFE	x1100	
x2FFF	x1110	
x3000	x5020	
x3001	x1025	
x3002	x2207	
Stack Pointer	x3000	

9.54 [*] 一个双人游戏要求双方在每次轮到自己时迅速思考。每人拥有总的思考时间，两只钟显示各自剩余时间。谁正在思考，谁的钟倒计时；时间耗尽，另一方获胜。走完一步就按按钮，停止自己的钟，启动对方的钟。

下面的程序实现这一机制。主程序在玩家思考时，每秒递减对应计数器；计数器到零，就打印获胜消息。按钮触发中断，中断服务例程采取相应动作，使主程序返回后开始递减另一计数器。

按钮中断向量为 x35，优先级 2。假设操作系统已设置设备中断使能位，主程序在用户模式以优先级 1 运行。要在按按钮时执行服务例程，哪个存储单元必须包含什么值？

若在第 16 行指令执行期间按按钮，哪两个十六进制值会被压栈？补全用户程序中的指令。程序还有一个只在不适当时机发生中断才会出现的错误：给出一条指令的行号，使得在该指令执行期间按按钮会产生意外行为。

```

; Interrupt Service Routine
    .ORIG x1550
    NOT    R0, R0
    RTI
    .END

; User Program
    .ORIG x3000
    AND    R0, R0, #0           ; Line 1
    LD     R1, TIME             ; Line 2
    LD     R2, TIME             ; Line 3

NEXT  ----- (a)
      ----- (b)
      BRn   P2_DEC              ; Line 6
      ADD   R1, R1, #-1         ; Line 7
      ----- (c)
      LEA   R0, P2WINS          ; Line 9
      BRnzp END                 ; Line 10
P2_DEC ADD   R2, R2, #-1         ; Line 11
      ----- (d)
      LEA   R0, P1WINS          ; Line 13
END    PUTS                      ; Line 14
      HALT                      ; Line 15
COUNT LD    R3, SECOND         ; Line 16
LOOP   ADD   R3, R3, #-1         ; Line 17
      BRp   LOOP                ; Line 18
      ----- (e)
TIME   .FILL   #300
SECOND .FILL   #25000           ; 1 second
P1WINS .STRINGZ "Player 1 Wins."
P2WINS .STRINGZ "Player 2 Wins."
    .END

```

9.55 [★] 一个程序正以管理模式运行，PSR[15]=0。在 x2000 设置断点后，操作员立即按 Run。随后 MAR、MDR 的值依次是什么？

MAR	MDR
	x8000
	x1050
	x0004
	xBCAE
x2800	x2C04
x1052	x3C4D
	x2C0A

第十章 计算器

在告别 LC-3 汇编语言、把抽象层次提高到 C 语言之前，我们不妨退后一步，用一个综合实例把已经学过的许多知识串起来。这个例子既要展示前面讨论的诸多概念怎样运用，也要示范注释充分、表达清楚的代码是什么样的，而其复杂程度远非一两页代码所能容纳。

我们的例子是一个模拟计算器行为的程序，用户可以用它对二进制补码整数进行加、减、乘运算。用户通过键盘输入数值，计算结果则显示在显示器上。计算器模拟程序由一个主程序和十一个独立的子程序组成。建议你在进入第 11 章、学习高级语言程序设计之前，认真研究这个例子。

理解计算器模拟程序的工作方式，还需要两个此前没有讨论过的主题：(1) 整数在 ASCII 字符串与二进制补码表示之间的转换；(2) 使用栈进行算术运算，也就是多数计算器采用的方法。

为什么需要两种数据类型，还要在二者之间转换？因为输入 / 输出需要一种数据类型，算术运算需要另一种。通过键盘输入、在显示器上显示的数值，用 ASCII 码表示；进行算术运算时，则使用二进制补码整数。

我们需要把用户输入的数值从 ASCII 码转换成二进制补码整数；为了显示计算结果，又必须把结果从二进制补码整数转换成 ASCII 码。10.1 节讨论数据类型转换。

计算器进行算术运算的机制，与多数台式机和笔记本电脑很不相同。后两者的 ISA 通常与 LC-3 类似：算术指令从通用寄存器取得源操作数，再把运算结果存入通用寄存器。我们的计算器模拟模型与多数计算器一样，用栈来承担这一角色：从栈中弹出源操作数，把运算结果压回栈中。¹10.2 节讨论如何以栈代替通用寄存器进行算术运算。

最后，10.3 节完整讨论计算器模拟程序，并给出使它工作所需的全部子程序。

10.1 数据类型转换

距离上次讨论数据类型，已经过了很久。我们接触过多种数据类型：地址运算使用无符号整数，整数算术使用二进制补码整数，逻辑运算使用 16 位二进制位串，科学计算使用浮点数，与输入、输出设备交互则使用 ASCII 码。

每条指令的源操作数都必须具有该指令要求的数据类型，这一点很重要。例如，ALU 执行这里的 ADD 时，需要二进制补码整数操作数；如果提供浮点操作数，得到的就会是毫无意义的结果。

高级语言程序中，常常可以看到 $A = R + I$ 这样的语句，其中 R 是浮点数，I 是二进制补码整数，二者采用不同的数据类型。

¹译注：这里指所模拟计算器的操作数组织方式。实现模拟程序时，仍会使用 LC-3 的通用寄存器。

如果由浮点加法器执行运算，I 就不能直接使用。必须先把 I 从原来的二进制补码整数类型，转换成执行运算的功能单元要求的浮点类型。使用某种高级语言时，编译器通常会生成执行这种转换的代码，程序员甚至不必专门考虑它。

早在第 5 章的“字符计数问题”中，我们就已经需要数据类型转换了。程序从键盘输入字符，扫描文件并统计该字符出现的次数，最后把计数显示在显示器上。回想一下：显示最终计数之前，必须把二进制补码整数转换成 ASCII 码。为什么？因为计数时对二进制补码整数做算术运算，显示时却需要 ASCII 表示。你还记得，我们把程序限制为只处理总计数不超过 9 的文件。因此，只须给二进制补码整数加上 $\times 30$ ，就能得到相应 ASCII 码。例如，用一个字节表示 6，其二进制补码是 00000110，即 $\times 06$ ；字符 6 的 ASCII 码则是 00110110，即 $\times 36$ 。

把计数限制为一位十进制数，实在是很大的限制。不过，那时是第 5 章，现在已经是第 10 章了！如果数值有多位十进制数字，仅仅加 $\times 30$ 就不行。例如，通过键盘输入两位十进制数 25，先输入 ASCII 码 $\times 32$ ，再输入 $\times 35$ ，得到位串 0011001000110101。要对这个整数进行算术运算，必须先把它转换成 25 的二进制补码表示 0000000000011001。显示算术运算结果也存在相似问题：必须先把二进制补码整数结果转换成 ASCII 字符串。

本节将编写例程，把多位十进制整数从 ASCII 字符串转换成二进制补码，也把二进制补码转换成 ASCII 字符串。

10.1.1 例子：一个有问题的程序—— $2 + 3 = e$

在研究实际转换例程之前，先看一个简单、具体的例子，说明它们为什么重要。图 10.1 展示了忽视数据类型会带来什么麻烦。

假设要从键盘输入两个一位整数，把它们相加，再将结果显示在显示器上。乍看之下，我们可能写出图 10.1 中这个简单的程序。结果会怎样？

```

01  TRAP    x23          ; Input from the keyboard.
02  ADD     R1,R0,#0      ; Make room for another input.
03  TRAP    x23          ; Input another character.
04  ADD     R0,R1,R0      ; Add the two inputs.
05  TRAP    x21          ; Display result on the monitor.
06  TRAP    x25          ; Halt.
```

图 10.1 没有注意数据类型的加法

假设先输入 2，再输入 3。程序结束之前，显示器上会出现什么？输入 2 后，装入 R0 的是字符 2 的 ASCII 码 $\times 0032$ 。当然，随后先把它移到 R1，再输入 3，于是 R0 装入字符 3 的 ASCII 码 $\times 0033$ 。ADD 把 $\times 0032$ 和 $\times 0033$ 两个位串相加，得到 $\times 0065$ 。显示这个值时，它又被当作 ASCII 码解释。 $\times 0065$ 是小写 e 的 ASCII 码，因此显示器上出现的是小写 e。

为什么没有得到 5——至少按我们上次计算， $2 + 3$ 的正确结果仍是 5？因为：(a) 相加之前，没有把两个输入字符从 ASCII 转换成二进制补码整数；(b) 显示之前，没

有把运算结果转换回 ASCII。

练习 修改图 10.1，使它把两个一位正整数相加，正确输出一位正整数和。假设两个输入数字之和确实只有一位。

10.1.2 输入数据：从 ASCII 到二进制

图 10.2 是三位十进制整数 295 的 ASCII 表示，它作为 ASCII 字符串，存放在从 ASCIIBUFF 开始的三个连续 LC-3 存储单元中。R1 保存这个正整数的十进制位数。我们的 ASCII 到二进制转换子程序把整数限制在 0 到 999 的范围内。

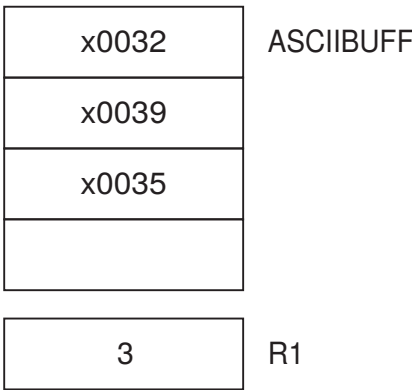


图 10.2 295 的 ASCII 表示存放在连续存储单元中

ASCIIBUFF 是所分配的四个连续存储单元中第一个单元的地址。这片存储区有两个用途：(a) 保存从键盘输入的十进制数字的 ASCII 码；(b) 保存算术运算结果对应的 ASCII 码，准备将结果写到显示器上。

你可能会问：每个十进制数字的 ASCII 码只需要一个字节，图 10.2 为什么使用整个 16 位字？实际上，通常确实只用一个字节保存一个 ASCII 码。这里为简化算法，才让每个 ASCII 字符独占一个存储字。

既然输入限制为至多三位十进制正整数，为什么还要为 ASCIIBUFF 分配四个字？三个不就够了吗？对输入来说确实够了。但你会在 10.1.3 节看到，为输出作准备时，还需要一个字保存结果的正负号，因为算术运算结果可能为负。

图 10.3 给出一个子程序的流程图，将图 10.2 中保存的整数 ASCII 表示转换成二进制整数。

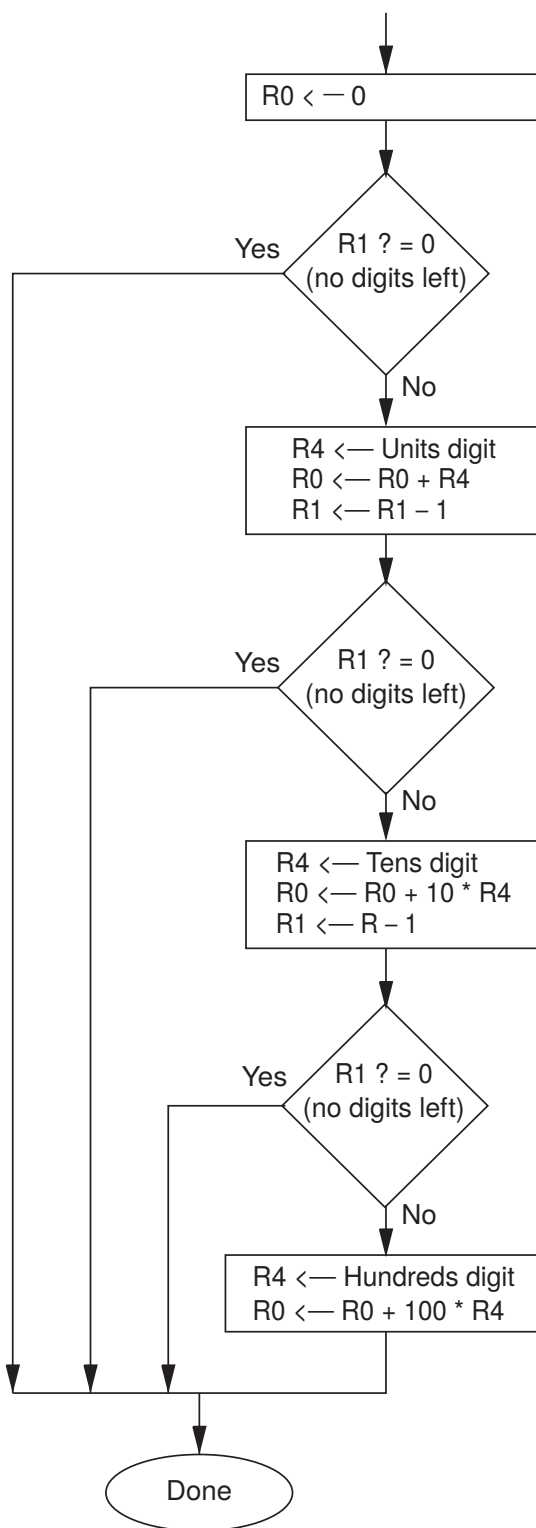


图 10.3 ASCII 到二进制转换子程序的流程图

子程序逐位处理：只保留每个 ASCII 码的低四位，将其转换成相应数字的二进制码，再用这四位作为索引，访问包含十个二进制值的表。因为至多转换三位十进制数，只需要十位表和百位表。每张表中的每一项，都对应十个数字中的一个。例如，百位表中索引为 6 的表项，其值为 #600，二进制表示为 0000001001011000。把该值加到 R0 中。R0 用于累加各位对整个数值的贡献，结果也通过 R0 返回。

问题 如果要转换四位十进制整数, 是否还需要千位表? 有没有办法把更长的十进制字符串转换成二进制, 而不必再使用千位表、万位表等?

练习 (有挑战性) 假设十进制数可以任意长。设计一种不使用任何表的转换算法, 避免为千位建立十项表、再为万位建立另一张十项表, 依此类推。参见习题 10.4。

图 10.4 给出了实现这一子程序的 LC-3 代码。

```

01 ;
02 ; This subroutine takes an ASCII string of up to three decimal digits and
03 ; converts it into a binary number. R0 is used to collect the result.
04 ; R1 keeps track of how many digits are left to process. ASCIIIBUFF
05 ; contains the most significant digit in the ASCII string.
06 ;
07 ASCIItoBinary ST R1,AtoB_Save1
08               ST R2,AtoB_Save2
09               ST R3,AtoB_Save3
0A               ST R4,AtoB_Save4
0B               AND R0,R0,#0 ; R0 will be used for our result.
0C               ADD R1,R1,#0 ; Test number of digits.
0D               BRz AtoB_Done ; There are no digits, result is 0.
0E ;
0F               LD R2,AtoB_ASCIIIBUFF ; R2 points to ASCIIIBUFF
10               ADD R2,R2,R1
11               ADD R2,R2,#-1 ; R2 now points to "ones" digit.
12 ;
13               LDR R4,R2,#0 ; R4 <-- "ones" digit
14               AND R4,R4,x000F ; Strip off the ASCII template.
15               ADD R0,R0,R4 ; Add ones contribution.
16 ;
17               ADD R1,R1,#-1
18               BRz AtoB_Done ; The original number had one digit.
19               ADD R2,R2,#-1 ; R2 now points to "tens" digit.
1A ;
1B               LDR R4,R2,#0 ; R4 <-- "tens" digit
1C               AND R4,R4,x000F ; Strip off ASCII template.
1D               LEA R3,LookUp10 ; LookUp10 is BASE of tens values.
1E               ADD R3,R3,R4 ; R3 points to the right tens value.
1F               LDR R4,R3,#0
20               ADD R0,R0,R4 ; Add tens contribution to total.
21 ;
22               ADD R1,R1,#-1
23               BRz AtoB_Done ; The original number had two digits.
24               ADD R2,R2,#-1 ; R2 now points to "hundreds" digit.
25 ;
26               LDR R4,R2,#0 ; R4 <-- "hundreds" digit
27               AND R4,R4,x000F ; Strip off ASCII template.
28               LEA R3,LookUp100 ; LookUp100 is hundreds BASE.
29               ADD R3,R3,R4 ; R3 points to hundreds value.
2A               LDR R4,R3,#0
2B               ADD R0,R0,R4 ; Add hundreds contribution to total.
2C ;
2D AtoB_Done LD R1,AtoB_Save1
2E           LD R2,AtoB_Save2
2F           LD R3,AtoB_Save3

```

图 10.4 ASCII 到二进制转换子程序

```

30          LD      R4, AtoB_Save4
31          RET
32      ;
33      AtoB_ASCIIIBUFF .FILL  ASCIIIBUFF
34      AtoB_Save1      .BLKW  #1
35      AtoB_Save2      .BLKW  #1
36      AtoB_Save3      .BLKW  #1
37      AtoB_Save4      .BLKW  #1
38      LookUp10        .FILL  #0
39                    .FILL  #10
3A                    .FILL  #20
3B                    .FILL  #30
3C                    .FILL  #40
3D                    .FILL  #50
3E                    .FILL  #60
3F                    .FILL  #70
40                    .FILL  #80
41                    .FILL  #90
42      ;
43      LookUp100       .FILL  #0
44                    .FILL  #100
45                    .FILL  #200
46                    .FILL  #300
47                    .FILL  #400
48                    .FILL  #500
49                    .FILL  #600
4A                    .FILL  #700
4B                    .FILL  #800
4C                    .FILL  #900

```

图 10.4 ASCII 到二进制转换子程序（续）

关于第 10 章的子程序，需要说明两点。它们都是 10.3 节完整介绍的计算器模拟程序的一部分。第一，不能单独汇编；第二，正因如此，必须确保任何标号都不重复使用。

为什么图 10.4 不能单独汇编？第 33 行的 `.FILL` 指定值为 `ASCIIIBUFF`，²但这个子程序中没有标为 `ASCIIIBUFF` 的存储单元，因此单独汇编会失败。我们本可以采用第 7 章简要讨论的 `.EXTERNAL`，让各子程序分别汇编；但这里选择把整个计算器模拟程序及十一个子程序作为一个整体汇编。正如你所预料的，图 10.15 第 42 行定义了 `ASCIIIBUFF`。³

第二，主程序和各子程序既然作为一个整体汇编，就必须避免不同子程序重复使

²译注：原文写为第 36 行；按图 10.4 续图，引用 `ASCIIIBUFF` 的是第 33 行。

³译注：原文写为第 43 行；图中第 42 行分配四个字，第 43 行是随后用于结束字符串的零值哨兵。

用同一标号。注意，图 10.4 中的大多数标号以 `At0B_` 开头，其他子程序也采用类似的命名方式。

10.1.3 显示结果：从二进制到 ASCII

要在显示器上显示计算结果，必须先把二进制补码整数结果转换成 ASCII 字符串。图 10.5 的子程序把 `R0` 中的二进制补码整数，转换成存放在 `ASCIIBUFF` 起始的四个连续存储单元中的 ASCII 字符串。`R0` 初值限制在 -999 到 $+999$ 。子程序执行完成后，`ASCIIBUFF` 保存原数值的正负号（+ 或 -），后面三个单元保存其绝对值的三位十进制数字所对应的 ASCII 码。子程序的工作方式如下：首先确定待显示结果的符号，把 + 或 - 的 ASCII 码存入 `ASCIIBUFF`，并把 `R0` 中的结果替换成其绝对值。然后不断从 `R0` 减去 `#100`，直到结果变负，据此确定百位数字。随后用同样的方法确定十位数字，最后剩下的就是个位数字。

练习 无论待转换整数的正负和大小，这个子程序总是生成四个字符。设计一种算法，去掉不必要的字符，也就是前导零和前导 +。参见习题 10.6。

```

01 ; This subroutine converts a 2's complement integer within the range
02 ; -999 to +999 (located in R0) into an ASCII character string consisting
03 ; of a sign digit, followed by three decimal digits, and stores the
04 ; character string into the four memory locations starting at ASCIIIBUFF
05 ; (see Figure 10.4).
06 ;
07 BinarytoASCII ST R0,BtoA_Save0
08               ST R1,BtoA_Save1
09               ST R2,BtoA_Save2
0A               ST R3,BtoA_Save3
0B               LD R1,BtoA_ASCIIIBUFF ; R1 keeps track of output string.
0C               ADD R0,R0,#0          ; R0 contains the binary value.
0D               BRn NegSign           ;
0E               LD R2,ASCIIplus       ; First store the ASCII plus sign.
0F               STR R2,R1,#0
10               BRnzp Begin100
11 NegSign       LD R2,ASCIIminus      ; First store ASCII minus sign.
12               STR R2,R1,#0
13               NOT R0,R0              ; Convert the number to absolute
14               ADD R0,R0,#1           ; value; it is easier to work with.
15 ;
16 Begin100      LD R2,ASCIIoffset      ; Prepare for "hundreds" digit.
17 ;
18               LD R3,Neg100           ; Determine the hundreds digit.
19 Loop100       ADD R0,R0,R3
1A               BRn End100
1B               ADD R2,R2,#1
1C               BRnzp Loop100
1D ;
1E End100       STR R2,R1,#1           ; Store ASCII code for hundreds digit.
1F               LD R3,Pos100
20               ADD R0,R0,R3           ; Correct R0 for one-too-many subtracts.
21 ;
22               LD R2,ASCIIoffset      ; Prepare for "tens" digit.
23 ;
24 Loop10        ADD R0,R0,#-10         ; Determine the tens digit.
25               BRn End10
26               ADD R2,R2,#1
27               BRnzp Loop10
28 ;
29 End10         STR R2,R1,#2           ; Store ASCII code for tens digit.
2A               ADD R0,R0,#10          ; Correct R0 for one-too-many subtracts.
2B Begin1       LD R2,ASCIIoffset      ; Prepare for "ones" digit.
2C               ADD R2,R2,R0
2D               STR R2,R1,#3
2E               LD R0,BtoA_Save0
2F               LD R1,BtoA_Save1
30               LD R2,BtoA_Save2
31               LD R3,BtoA_Save3
32               RET
33 ;
34 ASCIIplus     .FILL x002B
35 ASCIIminus    .FILL x002D
36 ASCIIoffset   .FILL x0030
37 Neg100       .FILL #-100
38 Pos100       .FILL #100
39 BtoA_Save0    .BLKW #1
3A BtoA_Save1    .BLKW #1
3B BtoA_Save2    .BLKW #1
3C BtoA_Save3    .BLKW #1
3D BtoA_ASCIIIBUFF .FILL ASCIIIBUFF

```

图 10.5 二进制到 ASCII 转换子程序

10.2 使用栈进行算术运算

10.2.1 把栈用作临时存储

你已经知道，LC-3 的 ADD 指令从寄存器取得两个源操作数，相加后把结果存入八个通用寄存器之一。保存结果的寄存器称为目的寄存器。R0 到 R7 这八个通用寄存器构成临时存储，使 ADD 这样的运算指令访问源操作数和目的位置，比访问存储器快得多。由于三个位置都显式指定，例如：

```
ADD R0, R1, R2
```

我们把 LC-3 称为**三地址机器** (three-address machine)。多数台式机和笔记本电脑，要么是类似 LC-3 的三地址机器，要么是**二地址机器** (two-address machine)，例如许多台式机和笔记本采用的 x86 ISA。二地址机器显式指定两个位置，下面是一条 x86 ADD 指令：

```
ADD EAX, EBX
```

EAX 和 EBX 是原文所讨论的 x86 八个通用寄存器中的两个。这里 EAX 既是一个源操作数的位置，又是目的操作数的位置。因此，二地址机器会用运算结果覆盖一个源寄存器。

还有一些 ISA，完全不用通用寄存器保存源操作数或运算指令的结果。其中最常见的是**栈机器** (stack machine)，因为它们把栈用作临时存储。多数计算器，包括将在 10.3 节模拟的计算器，都采用栈，而不是一组通用寄存器。

源操作数通过弹出栈顶两个元素取得，结果，也就是目的操作数，随后压回栈中。计算机总是在栈上弹出、压入操作数，因此不必显式指定地址。栈机器也就有时被称为**零地址机器** (zero-address machine)。指令只须写成：

```
ADD
```

计算机就知道到哪里寻找操作数。对计算器来说，这很方便：用户按下 + 键，就能执行加法。注意，这里实际完成弹栈、压栈和相加的底层动作，并不是 ISA 中分别提供给程序员的指令，而由硬件控制信号实现。⁴控制信号属于微体系结构，类似第 4、5 章介绍的装入使能信号和多路选择器选择信号。就像 LC-3 的 LD、ST 指令与 PCMUX、LD.MDR 控制信号之间的关系一样，程序员只要求计算机执行 ADD，其余工作由微体系结构完成。

10.2.2 一个例子

假设要计算 $(A + B) \cdot (C + D)$ ，A 中存放 25，B 中存放 17，C 中存放 3，D 中存放 2，并把结果存入 E。如果 LC-3 有乘法指令——我们大概会把它叫作 MUL——就可以编写如下程序：

⁴译注：原文在“不是 ISA 的一部分”之后写成“因此程序员可以使用它们”，与上下文矛盾。这里按后文对微体系结构控制信号的解释译出。

```
LD  R0, A
LD  R1, B
ADD R0, R0, R1
LD  R2, C
LD  R3, D
ADD R2, R2, R3
MUL R0, R0, R2
ST  R0, E
```

使用计算器时，则执行下面八个操作：

序号	操作	操作数
(1)	push	25
(2)	push	17
(3)	add	
(4)	push	3
(5)	push	2
(6)	add	
(7)	multiply	
(8)	pop	E

最后弹出的 210 就是计算结果。图 10.6 显示了每个操作完成后的栈快照。本例把 x3FFB 到 x3FFF 分配给栈，栈指针初值为 x4000，表示起初栈中没有元素。

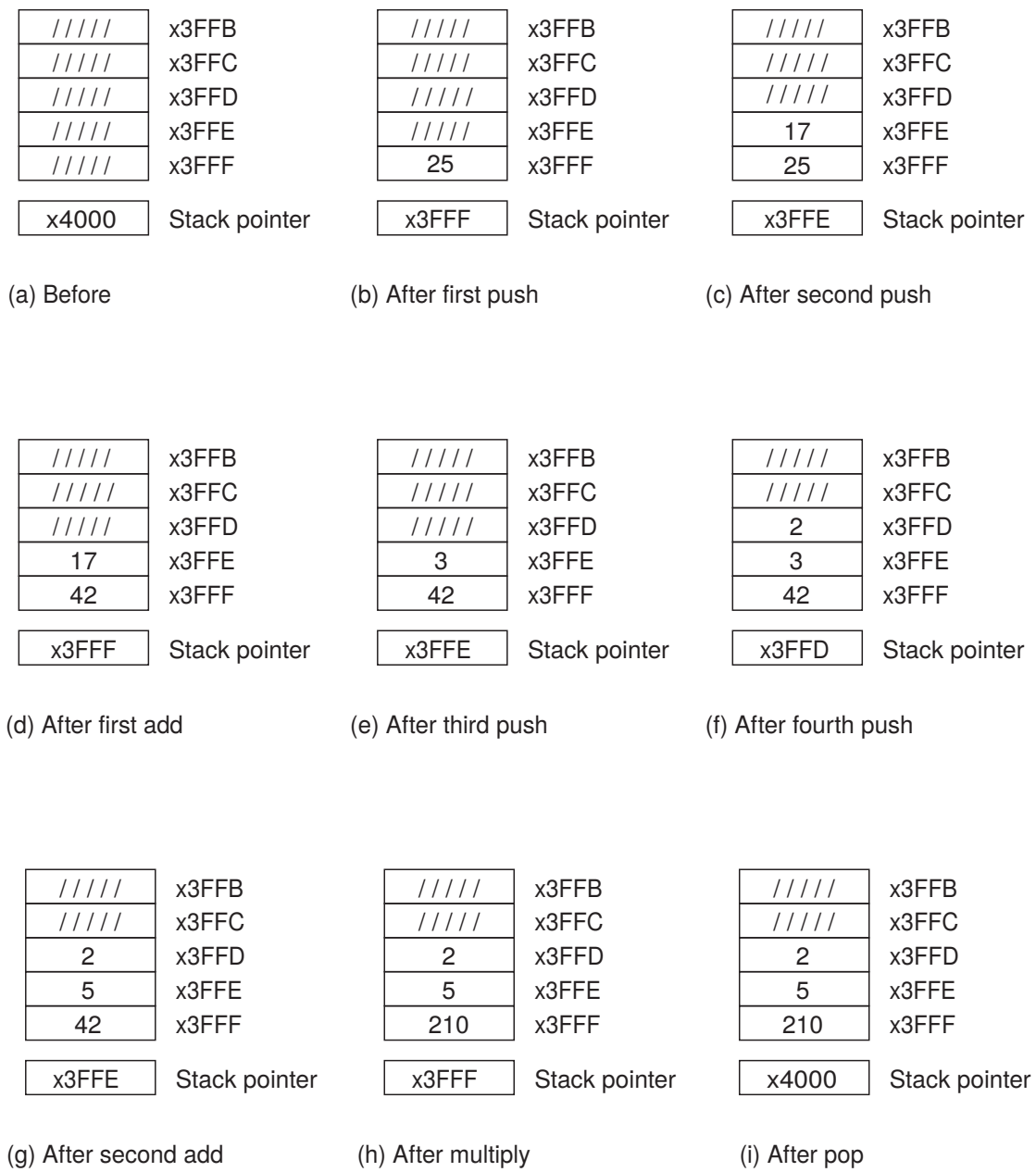


图 10.6 计算 (25 + 17) · (3 + 2) 时栈的使用情况

10.3 节将编写一个程序，使带有键盘和显示器的 LC-3 表现得像这样的计算器。LC-3 执行该程序时，我们说它在**模拟**计算器。为此，程序需要执行各种算术运算的子程序。

10.2.3 OpAdd、OpMult 和 OpNeg

10.3 节的计算器模拟程序需要三个子程序，才能完成加、减、乘运算：

- 1. OpAdd：从栈中弹出两个值，相加后把结果压回栈中。
- 2. OpMult：从栈中弹出两个值，相乘后把结果压回栈中。

3. OpNeg: 弹出栈顶值, 形成其二进制补码相反数, 再压回栈中。这样, 计算 A 减 B 时, 就可以先形成 $-B$, 再把它加到 A 上。

OpAdd 子程序 图 10.7 是其流程图。基本做法是尝试从栈中弹出两个值; 如果成功, 就把它相加。若结果在允许范围内, 即 -999 到 $+999$ 之间, 就将结果压回栈中。

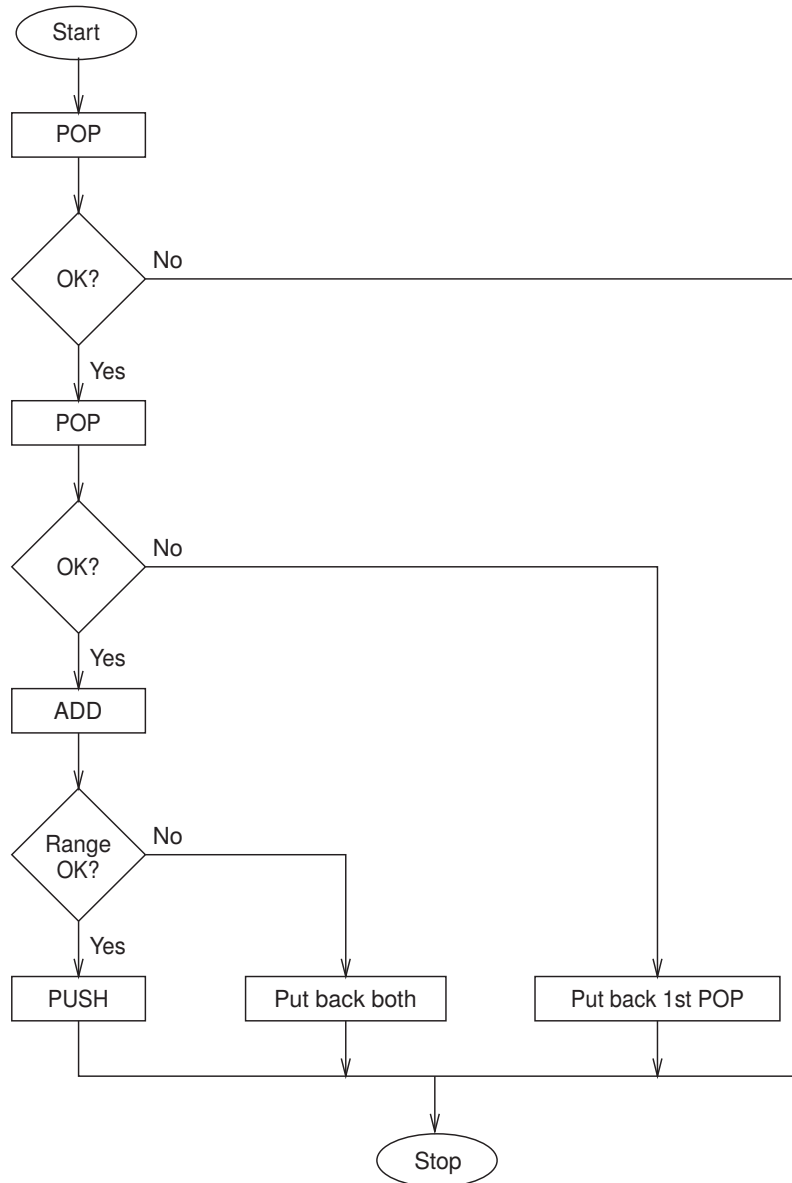


图 10.7 OpAdd 算法流程图

两种情况会阻止 OpAdd 成功完成: 栈中不足两个源操作数, 或者运算结果超出范围。无论哪种情况, 都把栈恢复成调用 OpAdd 之前的状态。如果第一次弹栈失败, POP 本身不会改变栈, 因此无须恢复。如果第二次弹栈失败, 就把栈指针减 1, 相当于把第一次弹出的值放回栈顶。如果结果超出允许范围, 则把栈指针减两次, 将两个值都放回栈顶。

图 10.8 给出 OpAdd 子程序。

```

01      ;
02      ;      Subroutine to pop the top two elements from the stack,
03      ;      add them, and push the sum onto the stack.  R6 is
04      ;      the stack pointer.
05      ;
06  OpAdd          ST      R0,OpAdd_Save0
07                  ST      R1,OpAdd_Save1
08                  ST      R5,OpAdd_Save5
09                  ST      R7,OpAdd_Save7
0A                  JSR      POP                ; Get first source operand.
0B                  ADD      R5,R5,#0          ; Test if POP was successful.
0C                  BRp      OpAdd_Exit        ; Branch if not successful.
0D                  ADD      R1,R0,#0          ; Make room for second operand.
0E                  JSR      POP                ; Get second source operand.
0F                  ADD      R5,R5,#0          ; Test if POP was successful.
10                  BRp      OpAdd_Restore1    ; Not successful, put back first.
11                  ADD      R0,R0,R1          ; THE Add.
12                  JSR      RangeCheck        ; Check size of result.
13                  ADD      R5,R5,#0          ; Check R5 for success/failure.
14                  BRp      OpAdd_Restore2    ; Out of range, restore both.
15                  JSR      PUSH              ; Push sum on the stack.
16                  BRnzp    OpAdd_Exit        ; On to the next task...
17  OpAdd_Restore2 ADD      R6,R6,#-1          ; Decrement stack pointer.
18  OpAdd_Restore1 ADD      R6,R6,#-1          ; Decrement stack pointer.
19  OpAdd_Exit      LD      R0,OpAdd_Save0
1A                  LD      R1,OpAdd_Save1
1B                  LD      R5,OpAdd_Save5
1C                  LD      R7,OpAdd_Save7
1D                  RET
1E  OpAdd_Save0     .BLKW   #1
1F  OpAdd_Save1     .BLKW   #1
20  OpAdd_Save5     .BLKW   #1
21  OpAdd_Save7     .BLKW   #1

```

图 10.8 OpAdd 子程序

注意，它会调用 **RangeCheck** 子程序。这个简单的检查用于确保运算结果处在可以接受、可以存入一个栈单元的范围之内。这里把数值限制为 -999 到 $+999$ 之间的整数，这将便于我们在 10.3 节设计自己的计算器。图 10.9 给出 **RangeCheck** 流程图，图 10.10 给出相应 LC-3 程序。

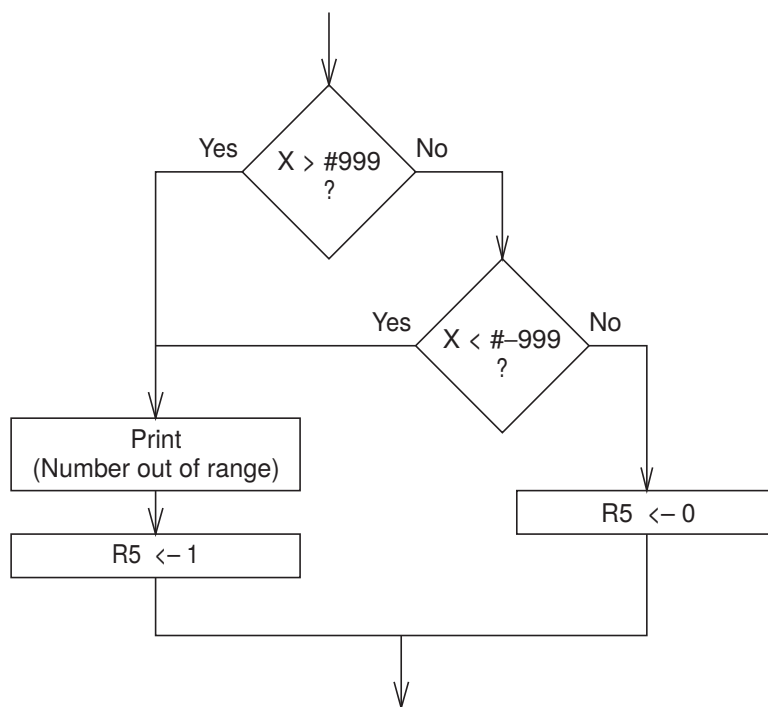


图 10.9 RangeCheck 算法流程图

```

01 ;
02 ; Subroutine to check that a value is
03 ; between -999 and +999.
04 ;
05 RangeCheck LD R5, Neg999
06 ADD R5, R0, R5 ; Recall that R0 contains the
07 BRp BadRange ; result being checked.
08 LD R5, Pos999
09 ADD R5, R0, R5
0A BRn BadRange
0B AND R5, R5, #0 ; R5 <-- success
0C RET
0D BadRange ST R0, RangeCheck_Save0
0E LEA R0, RangeErrorMsg
0F TRAP x22 ; Output character string
10 AND R5, R5, #0 ;
11 ADD R5, R5, #1 ; R5 <-- failure
12 LD R0, RangeCheck_Save0
13 RET
14 Neg999 .FILL #-999
15 Pos999 .FILL #999
16 RangeErrorMsg .FILL x000A
17 .STRINGZ "Error: Number is out of range."
18 RangeCheck_Save0 .BLKW #1
  
```

图 10.10 RangeCheck 子程序

OpMult 子程序 图 10.11 给出流程图，图 10.12 给出实现它的 LC-3 程序。与 OpAdd 相似，它尝试弹出两个值；成功后，将它们相乘。由于 LC-3 没有乘法指令，这里仍像前面一样，通过连续相加实现乘法。图 10.12 第 1D–1F 行是实际乘法的核心。⁵结

⁵译注：原文写为第 17–19 行，按图中 MulLoop 循环应为第 1D–1F 行。

果处在允许范围内时，就将它压回栈中。

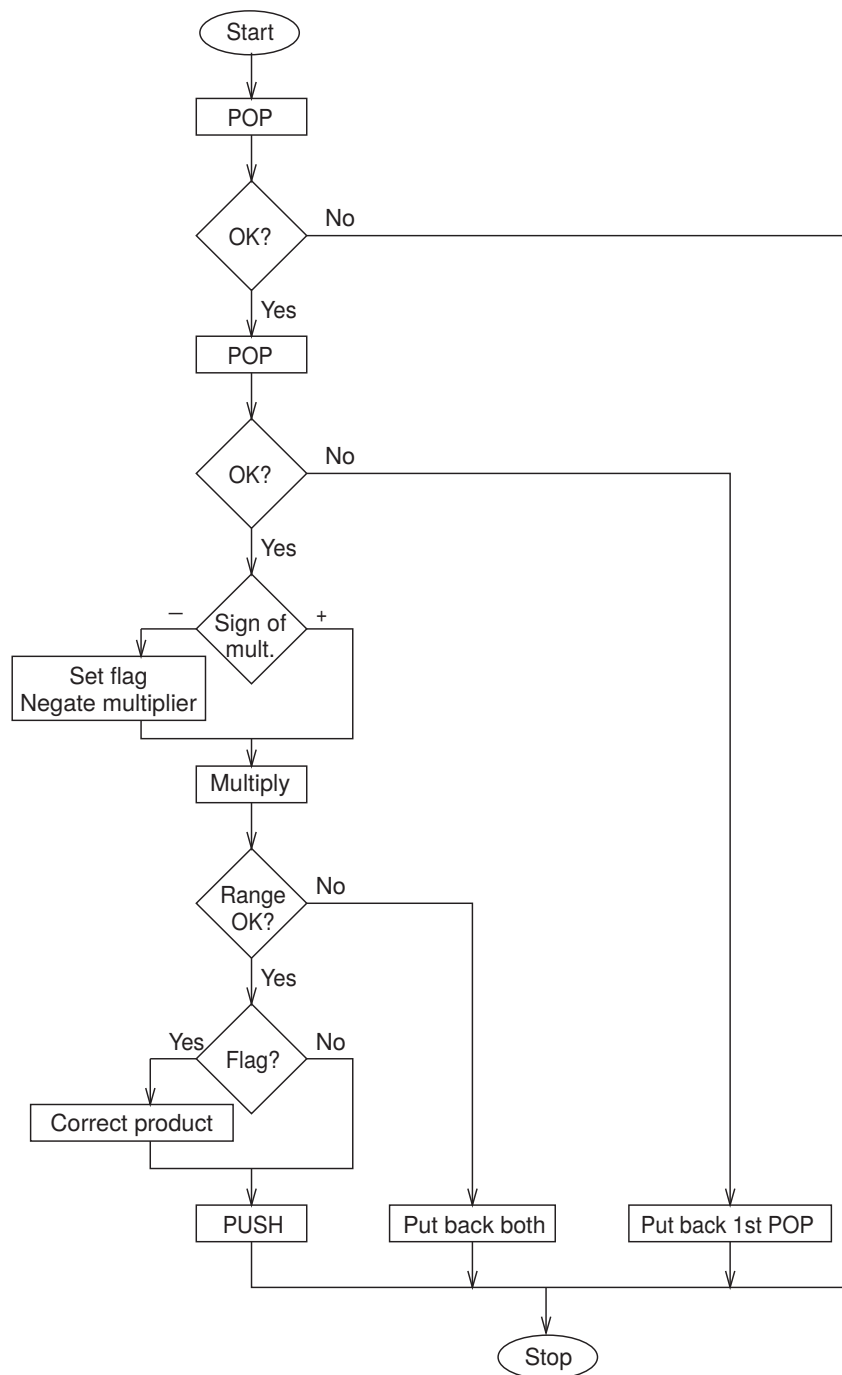


图 10.11 OpMult 子程序流程图

```

01 ;
02 ;   Two values are popped from the stack, multiplied, and if
03 ;   their product is within the acceptable range, the result
04 ;   is pushed onto the stack.  R6 is the stack pointer.
05 ;
06 OpMult          ST      R0,OpMult_Save0
07                ST      R1,OpMult_Save1
08                ST      R2,OpMult_Save2
09                ST      R3,OpMult_Save3
0A                ST      R5,OpMult_Save5
0B                ST      R7,OpMult_Save7
0C                AND     R3,R3,#0          ; R3 holds sign of multiplier.
0D                JSR     POP                ; Get first source from stack.
0E                ADD     R5,R5,#0          ; Test for successful POP.
0F                BRp     OpMult_Exit       ; Failure
10                ADD     R1,R0,#0          ; Make room for next POP.
11                JSR     POP                ; Get second source operand.
12                ADD     R5,R5,#0          ; Test for successful POP.
13                BRp     OpMult_Restore1    ; Failure; restore first POP.
14                ADD     R2,R0,#0          ; Moves multiplier, tests sign.
15                BRzp    PosMultiplier
16                ADD     R3,R3,#1          ; Sets FLAG: Multiplier is neg.
17                NOT     R2,R2
18                ADD     R2,R2,#1          ; R2 contains -(multiplier).
19 PosMultiplier   AND     R0,R0,#0          ; Clear product register.
1A                ADD     R2,R2,#0
1B                BRz     PushMult          ; Multiplier = 0, Done.
1C ;
1D MultLoop        ADD     R0,R0,R1          ; THE actual "multiply"
1E                ADD     R2,R2,#-1         ; Iteration Control
1F                BRp     MultLoop
20 ;
21                JSR     RangeCheck
22                ADD     R5,R5,#0          ; R5 contains success/failure.
23                BRp     OpMult_Restore2
24 ;
25                ADD     R3,R3,#0          ; Test for negative multiplier.
26                BRz     PushMult
27                NOT     R0,R0              ; Adjust for
28                ADD     R0,R0,#1          ; sign of result.
29 PushMult         JSR     PUSH             ; Push product on the stack.
2A                BRnzp   OpMult_Exit
2B OpMult_Restore2  ADD     R6,R6,#-1        ; Adjust stack pointer.
2C OpMult_Restore1  ADD     R6,R6,#-1        ; Adjust stack pointer.
2D OpMult_Exit     LD      R0,OpMult_Save0
2E                LD      R1,OpMult_Save1
2F                LD      R2,OpMult_Save2
30                LD      R3,OpMult_Save3
31                LD      R5,OpMult_Save5
32                LD      R7,OpMult_Save7
33                RET
34 OpMult_Save0     .BLKW  #1
35 OpMult_Save1     .BLKW  #1
36 OpMult_Save2     .BLKW  #1
37 OpMult_Save3     .BLKW  #1
38 OpMult_Save5     .BLKW  #1
39 OpMult_Save7     .BLKW  #1

```

图 10.12 OpMult 子程序

如果第二次弹栈失败，就将栈指针减 1，把第一次弹出的值放回栈顶。如果结果超出允许范围，仍会通过 R5 中的 1 表示失败；此时把栈指针减两次，把两个值都放回栈顶。

OpNeg 子程序 要用栈顶两个元素进行减法，先把栈顶元素替换为其相反数，再执行 OpAdd。也就是说，如果栈顶为 A，下一元素为 B，先对栈顶取负，再做加法，就能

把 $B - A$ 压入栈中。图 10.13 给出了对栈顶元素取负的 OpNeg 子程序。

```

01    ; Subroutine to pop the top of the stack, form its negative,
02    ; and push the result onto the stack.
03    ;
04    OpNeg      ST      R0,OpNeg_Save0
05              ST      R5,OpNeg_Save5
06              ST      R7,OpNeg_Save7
07              JSR     POP      ; Get the source operand.
08              ADD     R5,R5,#0  ; Test for successful pop
09              BRp     OpNeg_Exit ; Branch if failure.
0A              NOT     R0,R0
0B              ADD     R0,R0,#1  ; Form the negative of source.
0C              JSR     PUSH     ; Push result onto the stack.
0D    OpNeg_Exit LD      R0,OpNeg_Save0
0E              LD      R5,OpNeg_Save5
0F              LD      R7,OpNeg_Save7
10              RET
11    OpNeg_Save0 .BLKW  #1
12    OpNeg_Save5 .BLKW  #1
13    OpNeg_Save7 .BLKW  #1

```

图 10.13 OpNeg 子程序

10.3 计算器

10.3.1 功能

现在可以给出计算器的全部代码了。正如前面所说，按原书写作时的标准，这个计算器并不复杂。它允许用户输入至多三位十进制正整数，执行基本的加、减、乘运算，并显示十进制结果；结果也限制为至多三位数字。

我们通过键盘告诉计算器要做什么。可以输入至多三位的正整数，算术操作符 + (ADD)、* (MUL)、- (取负)，以及三个额外命令：D 把计算结果显示在显示器上，C 清除已输入的全部数值，X 关闭计算器。

算法的工作方式如下：用键盘输入命令和十进制值，用显示器输出结果，按 10.2 节的方法用栈保存源操作数和算术运算结果。数值的幅度限制为三位十进制数字，即计算结果在 -999 到 +999 之间，包含端点；负值由运算产生。

图 10.14 概括了计算器模拟算法。开始时先初始化，包括把栈指针 R6 设置为指示空栈。然后向键盘前的用户显示提示 Enter a Command。

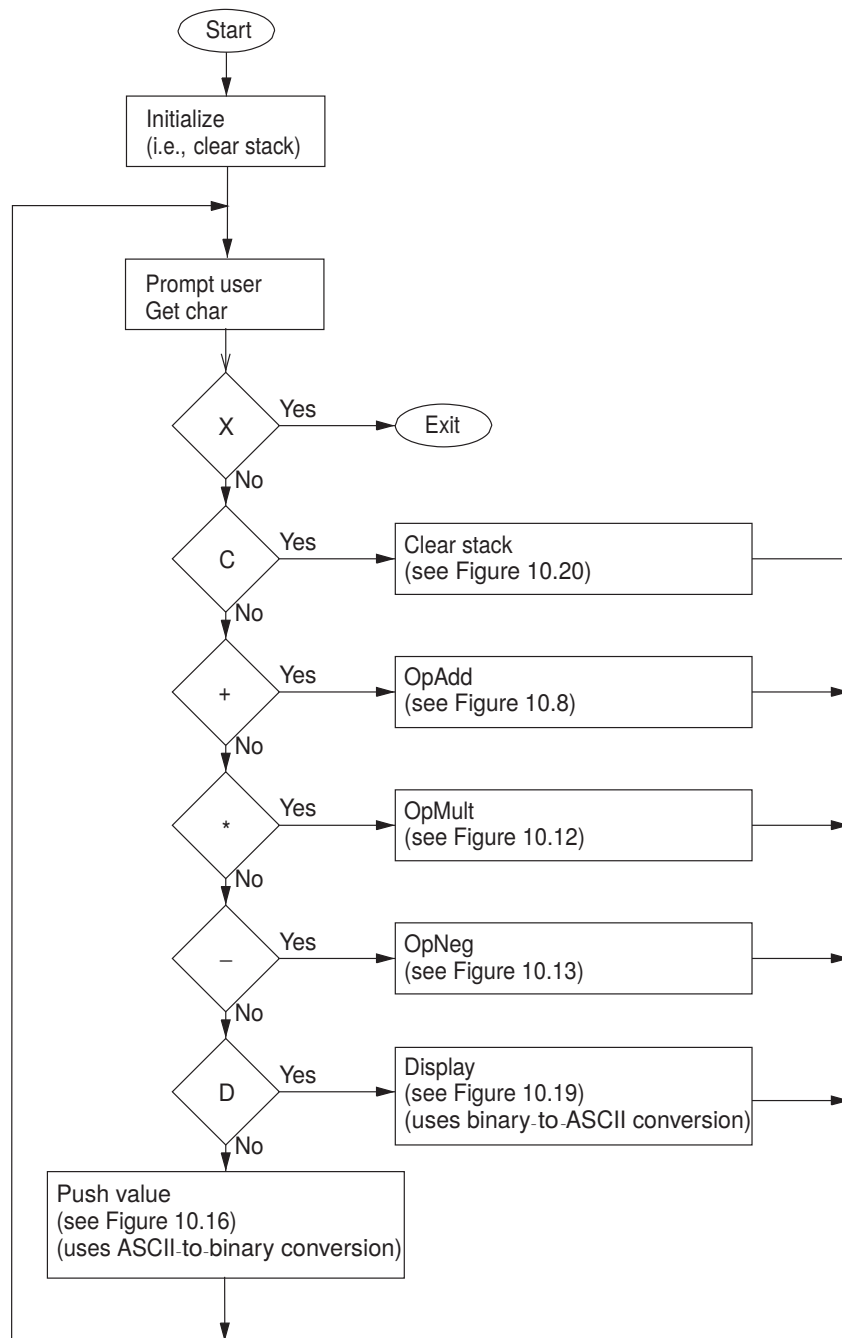


图 10.14 计算器概览

用户可以使用以下命令：

命令	功能
X	退出模拟程序。
D	显示栈顶的值。
C	清空栈中的全部值。
+	弹出栈顶两个元素 A、B，压入 $A + B$ 。
*	弹出栈顶两个元素 A、B，压入 $A \cdot B$ 。
-	弹出栈顶元素 A，压入 $-A$ 。
Enter 或 LF	将通过键盘输入的数值压入栈顶。

要输入数值，用户先键入至多三位十进制数字，再按 Enter 或换行键 LF（Line Feed）。

程序回显输入，并逐一检查字符，识别用户命令。根据命令调用相应子程序，完成指定工作。子程序返回后，再次提示输入命令。如此继续，直到用户按下 X，表示不再使用计算器。

例如，要计算

$$(51 - 49) \cdot (172 + 205) - (17 \cdot 2)$$

并在显示器上显示结果 720，可以依次按下以下键。下面分行只为排版，LF 表示一次换行键输入：

```
5, 1, LF, 4, 9, LF, -, +,  
1, 7, 2, LF, 2, 0, 5, LF, +, *,  
1, 7, LF, 2, LF, *, -, +, D.
```

10.3.2 代码

计算器模拟程序共包含十二个例程：图 10.15 是主算法，另外十一个子程序为它提供支持。注意，三个全局标号 StackMax、StackBase 和 ASCIIBUFF 都属于图 10.15 的主程序，向引用这些位置的子程序提供所需的符号表项。主程序为栈分配了十个单元，并在第 05 行把 R6 初始化为指示空栈。


```

01 ;
02 ; The Calculator, Main Algorithm
03 ;
04         LEA      R6,StackBase ; Initialize the Stack Pointer.
05         ADD      R6,R6,#1     ; R6 = StackBase + 1 --> empty stack
06
07 NewCommand LEA      R0,PromptMsg
08         PUTS
09         GETC
0A         OUT
0B ;
0C ; Check the command
0D ;
0E TestX     LD      R1,NegX     ; Check for X.
0F         ADD      R1,R1,R0
10         BRnp     TestC
11         HALT
12 ;
13 TestC     LD      R1,NegC     ; Check for C.
14         ADD      R1,R1,R0
15         BRnp     TestAdd
16         JSR      OpClear     ; See Figure 10.20
17         BRnzp    NewCommand
18 ;
19 TestAdd   LD      R1,NegPlus   ; Check for +
1A         ADD      R1,R1,R0
1B         BRnp     TestMult
1C         JSR      OpAdd      ; See Figure 10.8
1D         BRnzp    NewCommand
1E ;
1F TestMult  LD      R1,NegMult   ; Check for *
20         ADD      R1,R1,R0
21         BRnp     TestMinus
22         JSR      OpMult     ; See Figure 10.12
23         BRnzp    NewCommand
24 ;
25 TestMinus LD      R1,NegMinus  ; Check for -
26         ADD      R1,R1,R0
27         BRnp     TestD
28         JSR      OpNeg      ; See Figure 10.13
29         BRnzp    NewCommand
2A ;
2B TestD    LD      R1,NegD      ; Check for D
2C         ADD      R1,R1,R0
2D         BRnp     EnterNumber
2E         JSR      OpDisplay   ; See Figure 10.19
2F         BRnzp    NewCommand
30 ;
31 ; Then we must be entering an integer
32 ;
33 EnterNumber JSR      PushValue ; See Figure 10.16
34         BRnzp    NewCommand
35 ;
36 PromptMsg .FILL     x000A
37         .STRINGZ "Enter a command:"
38 NegX      .FILL     xFFA8
39 NegC      .FILL     xFFBD
3A NegPlus  .FILL     xFFD5
3B NegMinus .FILL     xFFD3
3C NegMult  .FILL     xFFD6
3D NegD     .FILL     xFFBC
3E
3F ; Globals
40 StackMax .BLKW     #9
41 StackBase .BLKW    #1
42 ASCIIIBUFF .BLKW   #4
43         .FILL     x0000 ; ASCIIIBUFF sentinel

```

图 10.15 计算器主算法

图 10.16 接收以 LF 结尾的 ASCII 字符串，检查它是否由不超过三个十进制数字组成；如果符合要求，就把它转换成二进制数，再压到栈顶。ASCII 到二进制的转换由图 10.4 的例程完成。

图 10.19 弹出栈顶元素，将其转换成 ASCII 字符串，并显示出来。二进制到 ASCII 的转换由图 10.5 完成。图 10.8 (OpAdd)、10.12 (OpMult)、10.13 (OpNeg) 提供基于

栈的基本算术算法。图 10.17、10.18 分别给出基本 POP、PUSH 例程。最后，图 10.20 用于清空栈。

```

01 ; This subroutine takes a sequence of not more than three decimal digits
02 ; typed by the user, converts its ASCII string to a binary value using the
03 ; ASCIItoBinary subroutine, and pushes the binary value onto the stack.
04 ; Anything else typed results in an error message.
05 ;
06 PushValue      ST      R0,PushValue_Save0
07                ST      R1,PushValue_Save1
08                ST      R2,PushValue_Save2
09                ST      R7,PushValue_Save7
0A                LD      R1,PushValue_ASCIIIBUFF ; R1 points to string being
0B                LD      R2,MaxDigits           ; generated.
0C ;
0D ValueLoop     ADD      R3,R0,x-0A      ; Test for line feed, x0A
0E                BRz     GoodInput
0F                ADD     R2,R2,#0
10                BRz     TooLargeInput
11                LD      R3,NEGASCIIO
12                ADD     R3,R0,R3
13                BRn     NotInteger
14                LD      R3,NEGASCIIO9
15                ADD     R3,R0,R3
16                BRp     NotInteger
17                ADD     R2,R2,#-1      ; Still room for more digits.
18                STR     R0,R1,#0      ; Store last character read.
19                ADD     R1,R1,#1
1A                GETC
1B                OUT
1C                BRnzp   ValueLoop
1D ;
1E GoodInput     LD      R2,PushValue_ASCIIIBUFF
1F                NOT     R2,R2
20                ADD     R2,R2,#1
21                ADD     R1,R1,R2      ; R1 now contains no. of char.
22                BRz     NoDigit
23                JSR     ASCIItoBinary
24                JSR     PUSH
25                BRnzp   PushValue_Done

```

图 10.16 计算器的 PushValue 例程

```

26  NoDigit          LEA      R0,NoDigitMsg
27                  PUTS
28                  BRnzp    PushValue_Done
29  NotInteger       GETC          ; Spin until carriage return.
2A                  OUT
2B                  ADD      R3,R0,x-0A    ; Test for line feed, x0A
2C                  BRnp     NotInteger
2D                  LEA      R0,NotIntegerMsg
2E                  PUTS
2F                  BRnzp    PushValue_Done
30  TooLargeInput   GETC          ; Spin until carriage return.
31                  OUT
32                  ADD      R3,R0,x-0A    ; Test for line feed, x0A
33                  BRnp     TooLargeInput
34                  LEA      R0,TooManyDigits
35                  PUTS
36  PushValue_Done  LD        R0,PushValue_Save0
37                  LD        R1,PushValue_Save1
38                  LD        R2,PushValue_Save2
39                  LD        R7,PushValue_Save7
3A                  RET
3B  TooManyDigits   .FILL    x000A
3C                  .STRINGZ "Too many digits"
3D  NoDigitMsg      .FILL    x000A
3E                  .STRINGZ "No number entered"
3F  NotIntegerMsg   .FILL    x000A
40                  .STRINGZ "Not an integer"
41  MaxDigits       .FILL    x0003
42  NegASCII0       .FILL    x-30
43  NegASCII9       .FILL    x-39
44  PushValue_ASCIIIBUFF .FILL  ASCIIIBUFF
45  PushValue_Save0 .BLKW    #1
46  PushValue_Save1 .BLKW    #1
47  PushValue_Save2 .BLKW    #1
48  PushValue_Save7 .BLKW    #1

```

图 10.16 计算器的 PushValue 例程 (续)

```

01 ; This subroutine POPs a value from the stack and puts it in
02 ; R0 before returning to the calling program. R5 is used to
03 ; report success (R5 = 0) or failure (R5 = 1) of the POP operation.
04 POP          LD      R0,POP_StackBase
05              NOT     R0,R0          ; R0 = -(addr. of StackBase + 1)
06              ADD     R0,R0,R6      ; R6 = StackPointer
07              BRz     Underflow
08              LDR     R0,R6,#0      ; The actual POP
09              ADD     R6,R6,#1      ; Adjust StackPointer
0A              AND     R5,R5,#0      ; R5 <-- success
0B              RET
0C Underflow    LEA     R0,UnderflowMsg
0D              PUTS                    ; Print error message.
0E              AND     R5,R5,#0
0F              ADD     R5,R5,#1      ; R5 <-- failure
10              RET
11 UnderflowMsg .FILL   x000A
12              .STRINGZ "Error: Too Few Values on the Stack."
13 POP_StackBase .FILL   StackBase

```

图 10.17 计算器的 POP 例程

```

01 ; This subroutine PUSHes on the stack the value stored in R0.
02 ; R5 is used to report success (R5 = 0) or failure (R5 = 1) of
03 ; the PUSH operation.
04 PUSH         ST      R1,PUSH_Save1 ; R1 is needed by this routine.
05              LD      R1,PUSH_StackMax
06              NOT     R1,R1
07              ADD     R1,R1,#1      ; R1 = - addr. of StackMax
08              ADD     R1,R1,R6      ; R6 = StackPointer
09              BRz     Overflow
0A              ADD     R6,R6,#-1     ; Adjust StackPointer for PUSH.
0B              STR     R0,R6,#0      ; The actual PUSH
0C              LD      R1,PUSH_Save1 ; Restore R1.
0D              AND     R5,R5,#0      ; R5 <-- success
0E              RET
0F Overflow    LEA     R0,OverflowMsg
10              PUTS
11              LD      R1,PUSH_Save1 ; Restore R1.
12              AND     R5,R5,#0
13              ADD     R5,R5,#1      ; R5 <-- failure
14              RET
15 PUSH_Save1   .BLKW   #1
16 OverflowMsg  .FILL   x000A
17              .STRINGZ "Error: Stack is Full."
18 PUSH_StackMax .FILL   StackMax

```

图 10.18 计算器的 PUSH 例程

```

01 ; This subroutine calls BinarytoASCII to convert the 2's complement
02 ; number on the top of the stack into an ASCII character string, and
03 ; then calls PUTS to display that number on the screen.
04 OpDisplay      ST      R0,OpDisplay_Save0
05               ST      R5,OpDisplay_Save5
06               ST      R7,OpDisplay_Save7
07               JSR      POP          ; R0 gets the value to be displayed.
08               ADD      R5,R5,#0
09               BRp      OpDisplay_DONE ; POP failed, nothing on the stack.
0A               JSR      BinarytoASCII
0B               LD      R0,NewlineChar
0C               OUT
0D               LD      R0,OpDisplay_ASCIIIBUFF
0E               PUTS
0F               ADD      R6,R6,#-1      ; Push displayed number back on stack.
10 OpDisplay_DONE LD      R0,OpDisplay_Save0
11               LD      R5,OpDisplay_Save5
12               LD      R7,OpDisplay_Save7
13               RET
14 NewlineChar    .FILL    x000A
15 OpDisplay_ASCIIIBUFF .FILL ASCIIIBUFF
16 OpDisplay_Save0 .BLKW    #1
17 OpDisplay_Save5 .BLKW    #1
18 OpDisplay_Save7 .BLKW    #1

```

图 10.19 计算器的显示例程

```

01 ;
02 ; This routine clears the stack by resetting the stack pointer (R6).
03 ;
04 OpClear        LD      R6,OpClear_StackBase ; Initialize the Stack Pointer.
05               ADD      R6,R6,#1          ; R6 = StackBase + 1 --> empty stack
06               RET
07 OpClear_StackBase .FILL StackBase

```

图 10.20 OpClear 例程

习题

- 10.1** 用自己的话描述 OpMult 算法中 Multiply 步骤的工作方式。执行这个步骤需要执行多少条指令？用乘数值 n 表示答案。注意，同一条指令若执行五次，就计入五次。编写一个程序片段，使乘数小于 25 时能用更少的指令完成 Multiply 步骤。需要多少条？⁶
- 10.2** 修改图 10.1，使它把两个一位正整数相加，并输出一位正整数和。假设两个输入数字之和确实只有一位。
- 10.3** 修改图 10.1，假设输入数值是一位十六进制正数，并且两个输入数值之和也确实只有一位十六进制数字。
- 10.4** 图 10.4 给出了把 ASCII 字符串转换成二进制值的算法。假设十进制数可以任意长。设计一种完全不使用表的转换算法，避免为千位建立十项表、再为万位建立另一张十项表，依此类推。

⁶译注：原题引用图 10.14；本版 OpMult 流程图和代码分别见图 10.11、10.12。

10.5 图 10.4 把以 ASCII 数字表示的十进制数转换成二进制。扩展这段代码，使它还能把以 ASCII 表示的十六进制数转换成二进制。如果数值前有一个 x，后续至多三个 ASCII 数字表示十六进制数；否则表示十进制数。

10.6 无论整数的符号和大小，图 10.5 的算法总是产生四个字符。设计一种算法，去掉通常表示中不必要的字符，也就是不保存前导 0 和前导 +。

10.7 下面的 LC-3 程序做什么？

```

                .ORIG    x3000
                LEA      R6, STACKBASE
                LEA      R0, PROMPT
                TRAP      x22                ; PUTS
                AND      R1, R1, #0
LOOP            TRAP      x20                ; IN
                TRAP      x21
                ADD      R3, R0, #-10        ; Check for newline
                BRz      INPUTDONE
                JSR      PUSH
                ADD      R1, R1, #1
                BRnzp    LOOP
INPUTDONE      ADD      R1, R1, #0
                BRz      DONE
LOOP2          JSR      POP
                TRAP      x21
                ADD      R1, R1, #-1
                BRp      LOOP2
DONE           TRAP      x25                ; HALT

PUSH           ADD      R6, R6, #-2
                STR      R0, R6, #0
                RET

POP            LDR      R0, R6, #0
                ADD      R6, R6, #2
                RET

PROMPT         .STRINGZ  "Please enter a sentence: "
STACKSPAC     .BLKW    #50
STACKBASE     .FILL    #0
                .END

```

10.8 [★] 计算器程序假定，如果用户没有输入 X、C、+、-、*、D 之一，就一定是在压入一个数值，因此执行 BRnzp PushValue。修改程序，使它更健壮：若用户输入的不是数字，就把 X 的 ASCII 码装入 R0，再跳到 Test；若输入的是数字，则跳到 PushValue。⁷

10.9 [★] 针对习题 10.8 的计算器程序，修改 PushValue，检查输入的所有字符确实都是数字，进一步提高健壮性。

⁷译注：题设的 BRnzp PushValue 沿用旧代码表述；本版图 10.15 的 EnterNumber 处实际为 JSR PushValue。保留原题要求，修改时需注意调用与返回链接。

第十一章 C/C++ 程序设计入门

11.1 我们的目标

祝贺你，也欢迎你进入本书的后半部分！我们刚刚介绍了现代计算设备的基本底层结构。不论是智能手机还是智能汽车，处理数字数据的底层机制都大同小异。打牢了这些基础，我们就可以进一步探索高级编程语言中的程序设计了。

在本书的后半部分，我们将以 C 和 C++ 为背景，讨论高级程序设计概念。每走一步，每遇到一个新的高级概念，我们都能把它与数字系统的较低层次联系起来。我们的观点是：任何东西都不应该停留在抽象或神秘的层面。如果能把新概念拆解为底层各层执行的操作，我们就能成为更熟练的开发者和工程师，更善于构建新的硬件和软件系统。

先快速回顾一下前半部分。前十章介绍了 LC-3：一种简单的计算机体系结构，却具备更复杂的实际系统的所有重要特征。LC-3 背后的一个基本思想，也是所有现代数字系统背后的基本思想，是把简单元件系统地互连起来，构成能力更复杂的部件。MOS 晶体管连接起来构成逻辑门，逻辑门构成存储器和数据通路部件，再把这些部件互连起来，就构成了 LC-3。通过系统地连接简单部件来构成更复杂的系统，是贯穿计算领域的重要概念；它不仅用于硬件设计，也用于软件设计。正是这一构建原则，使我们能够造出从整体上看复杂得令人难以置信的数字计算系统。

介绍完 LC-3 的硬件后，我们用它本身的机器语言，也就是 1 和 0，为它编程。尝过机器语言编程既容易出错又不自然的滋味后，我们很快转向了更便于人使用的 LC-3 汇编语言。我们介绍了如何系统地把编程问题分解成易于用 LC-3 编码的部分，也考察了底层 TRAP 子程序怎样代替程序员完成输入、输出等常见任务。系统化分解和子程序都是软件中广泛使用的设计方法。在本书结束之前，我们还会不断看到这些概念的实例。

本书后半部分的主要目标，是介绍高级程序设计的基本构造，包括变量、控制结构、函数、数组、指针、递归和简单的数据结构，并教给你一套解决编程问题的良好方法。实现这些目标的主要工具，就是 C 和 C++。

我们的目标并不是全面介绍 C 或 C++。这两种语言都十分庞大，尤其是 C++；经过几十年的演进，它们能够支持大规模软件的构建。人们已用它们写出了数十亿行代码。一些使用最广泛的应用、云服务和设备，都是用 C 或 C++ 构建的。这两种语言包含许多特性，使开发团队能够开发稳定、可维护、可扩展的软件；但初次接触这些语言时，并不必学习其中的许多特性。

我们的目标是先探索 C 的核心要素，再探索 C++ 的核心要素。这些要素是进入这两种语言的起点，能够让我们写出有趣而富有挑战性的程序：代码约一百行，放在一个文件里。

总要有个起点。因此，本章将从低级汇编语言编程转向用高级语言 C 编程。C 和 C++ 有一段引人入胜的历史。我们将了解它们如何诞生、为何诞生，以及为什么在问世近 50 年之后仍如此流行。随后，我们会直接考察一个简单的示例程序，开始学习 C。现在就开始吧！

11.2 跨越层次之间的距离

几乎总是软件让数字系统发挥出具体的作用。硬件提供通用的基础，软件则赋予它具体能力。即使是智能音箱或 Bluetooth 耳机这类我们未必会想到由软件驱动的设备，内部也嵌入了大量软件来实现各种功能。是软件让设备运转起来。

实现这些复杂功能，需要规模更大、结构更复杂的软件。一个典型的智能手机应用可能有数十万行代码，一个 Web 浏览器可能有数百万行，而驱动一辆现代汽车（非自动驾驶汽车）的代码可能达到数亿行。随着一代代设备的发展，驱动它们的软件也越来越复杂。底层硬件变得更快，人们对新功能的需求不断增长，因此可以预期，所需的软件量还会继续增加。请看图 11.1，它直观地展示了不同应用的软件系统规模。

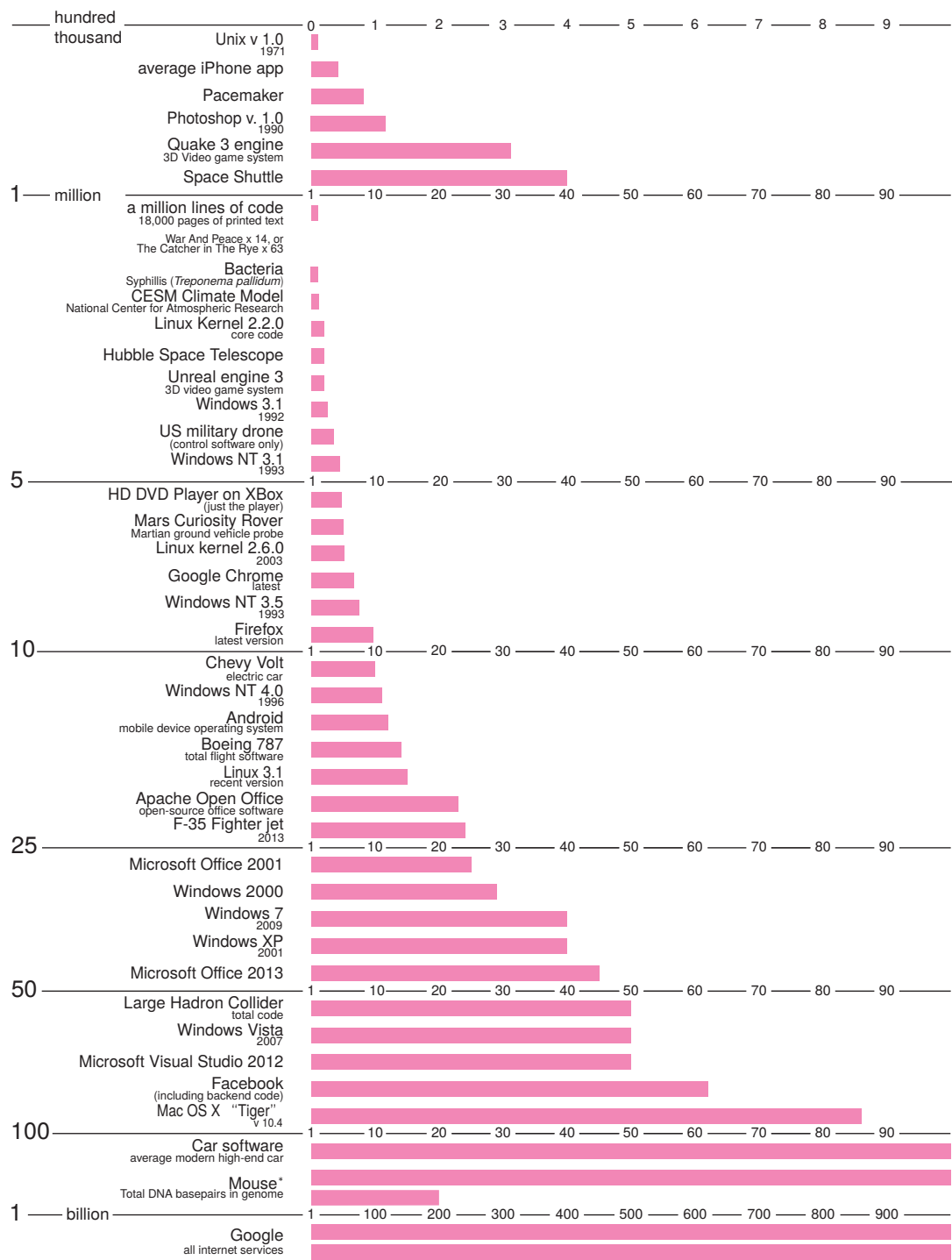


图 11.1 不同计算应用的估计代码行数。有些应用需要大量代码；观察 Windows 操作系统等具体项目，还可以看出其复杂度随新版本增长。Codebases: Millions of Line of Code，构思与设计：David McCandless；研究：Pearl Doughty-White 和 Miriam Quick。Informationisbeautiful.net，2015 年 9 月 24 日。David McCandless@informationisbeautiful.net。经许可使用。

C、C++ 等编程语言诞生的首要目的，就是提高程序员的生产率。最常用的语言使程序员团队能够更快地开发出正确工作、便于维护的代码。如果没有有效的编程语言及相应的开发环境，我们创造这些出色设备的能力就会大打折扣。建造一座 100 层的摩天大楼或一架 Boeing 787 飞机，是需要专门技术和工具的工程壮举；开发和维护

一个需要一千万行代码的 Web 服务，同样如此。

显然，LC-3 汇编语言不会成为开发大多数实际软件的首选语言；x86 或 ARM 汇编语言也不会。x86 和 ARM 是当今最普及的两种指令集体系结构，几乎所有 PC、服务器和移动设备都由它们驱动。不过，无论开发语言是 C、C++、Java 还是 Python，代码最终都会自动转换为底层硬件的指令集所能执行的操作，不论硬件采用 ARM、x86 还是 LC-3。因此，这里的基本过程是：编程语言让人能够用更自然的方式表达代码，同时又能将其自动、无歧义地转换成机器可执行的形式。理想情况下，我们或许希望直接用人类语言来表达，但我们知道，那会有多少歧义，又有多容易出错。因此，我们选择 C、C++ 这样更精确、面向机器的语言。

从第 5、6 章的 LC-3 机器语言转向第 7 章的 LC-3 汇编语言时，你一定注意到了汇编语言怎样简化 LC-3 编程，并体会到了其中的好处。1 和 0 变成了助记符，存储器地址变成了符号标号。指令和地址都采用了更适合人，而非更适合机器的形式。在转换层次中，汇编器弥合了算法层与 ISA 层之间的一部分距离，见图 1.9。¹我们希望语言层能够弥合更多距离。高级语言正是这样做的：它们让编程更容易。下面看看它们能提供哪些帮助。

- **高级语言帮助管理参与计算的值。**用机器语言编程时，如果要记录循环的迭代次数，就必须留出一个存储器单元或寄存器存放计数值。更新计数器时，我们必须记住上次把它存在哪里，再将它加载出来，进行运算，然后存回原处。汇编语言让这一过程简单一些，因为可以给计数器所在的存储器单元起一个有意义的标号。在 C 这样的高级语言中，程序员只需给这个值起一个有意义的符号名称；语言的实现就会负责分配存储空间，并在代码引用它时完成适当的数据传送。大多数程序都包含许多值，因此，能够方便地处理这些值，对提高程序员的生产率至关重要。
- **高级语言提供了便于人表达计算的方式。**对大多数人而言，描述现实世界中事物之间的相互作用，比描述数字世界中整数、字符和浮点数等对象之间的相互作用更自然。高级语言面向人的表达习惯，因此能让程序员表达更多含义。在高级语言中，复杂任务可以用更少的代码表达，代码本身也更像人类语言。例如，用 C 计算三角形面积时，可以写成：

```
area = 0.5 * base * height;
```

再看一个例子：我们经常需要检测某个条件，条件为真时做一件事，为假时做另一件事。在高级语言中，这样的常见任务可以用类似英语的形式简单表达。例如，如果希望条件 `isItCloudy` 为真时执行 `get(Umbrella)`，否则执行 `get(Sunglasses)`，就可以在 C 中使用如下控制结构：

```
if (isItCloudy)
    get(Umbrella);
else
    get(Sunglasses);
```

¹译注：原文此处引用图 1.6；本版“转换层次”实际为图 1.9，已据图题更正。

这种表达能力让我们用比汇编语言更少的代码完成更多工作，从而显著提高编程效率。

- **高级语言提供了对底层硬件的抽象。**高级语言提供统一的编程接口，使其独立于底层硬件。这带来两个不同的优点。首先，代码具有可移植性（portability）。C、C++ 或 Java 代码可以方便、高效地面向多种设备，而不依赖某一种硬件。直接用汇编语言编写的代码会利用特定硬件系统的细节，因此不易在其他硬件上运行。不难理解可移植性为什么重要：为 Android 编写了应用的开发者，会希望尽快将应用移植到 iOS，以利用 Apple 设备的庞大用户群。

第二，我们可以使用硬件本身不直接支持的操作。例如，LC-3 中没有一条能完成整数乘法的指令，汇编语言程序员必须编写一小段代码来实现乘法。高级语言支持的操作集合通常比 ISA 支持的操作集合更大。每当程序员使用这样的操作时，语言的实现会生成完成它所需的代码。程序员知道这些高级操作会得到正确执行，因而可以专注于实际的编程任务，而不必处理底层实现。

- **高级语言提高了可维护性。**常见控制结构用简单、类似英语的语句表达，程序本身就更容易阅读，也更容易由他人修改和修复。阅读高级语言程序时，可以辨认出循环和选择结构，理解代码所花的力气比阅读汇编程序少。即使只离开几个小时，你大概也经历过重新熟悉自己写的 LC-3 汇编代码的过程；那可不是什么乐事！程序员还经常需要调试别人的代码，或在其基础上继续开发。如果语言的组织形式本来就便于人理解，读懂用这种语言编写的代码也会简单得多。
- **许多高级语言提供了防范程序错误的措施。**语言要求程序员遵守更严格的规则，因此能够在程序转换时或执行时进行检查。如果违反某些规则或条件，错误信息就会把程序员引向代码中可能存在错误的位置。这样，语言可以帮助程序员更快地让程序正确工作。

11.3 高级语言程序的转换

正如 LC-3 汇编程序必须转换为机器语言，更准确地说，必须经过汇编一样，所有用高级语言编写的程序也必须经过转换。毕竟，底层硬件只能执行机器代码。如何转换，取决于具体高级语言的设计。一种技术称为**解释**（interpretation）。采用这种方式时，称为**解释器**（interpreter）的转换程序读入高级语言程序，并执行程序员指定的操作。高级语言程序并不直接执行，而是由解释器程序来执行。另一种技术称为**编译**（compilation），相应的转换程序称为**编译器**（compiler）。编译过程将整个高级语言程序转换成机器语言，或非常接近机器语言的形式。可执行映像是编译过程的一种输出，可以直接在硬件上执行。请记住，解释器和编译器本身也都是运行在某种设备上的软件。

11.3.1 解释

如果你运行过 Python 代码，就运行过解释器。在解释方式下，高级语言程序是给解释器程序的一组“命令”。解释器读入这些命令，并按语言的定义执行它们。高级语言程序并不直接由硬件执行；实际上，它只是解释器的输入数据。解释器是一台虚拟机，在隔离的沙箱中执行程序。²

许多解释器逐段转换高级语言程序，每次处理一行、一条命令或一个子程序。例如，解释器可能读入高级语言程序的一行，然后直接在底层硬件上完成该行指定的操作。如果这一行说的是“求 B 的平方根，并将其存入 C”，解释器就会发出计算机 ISA 中适当的指令序列来完成开平方运算。处理完当前行后，解释器移到下一行并执行它。这一过程不断继续，直到整个高级语言程序结束。

11.3.2 编译

采用编译方式时，高级语言程序被转换成可以直接由硬件执行的机器代码。为了有效地完成这一任务，编译器必须先把源程序作为较大的整体进行分析，通常是整个源文件，然后再生成转换后的版本。程序只需编译一次，就可以执行多次。C、C++ 及其变体等许多编程语言通常采用编译方式。编译器处理包含高级语言程序的一个或多个文件，并生成可执行映像。编译器并不执行程序，虽然某些复杂的编译器也会执行程序以便更好地优化性能；它的任务只是将程序从高级语言转换为计算机本身的机器语言。

11.3.3 优点与缺点

两种转换技术各有优缺点。采用解释方式时，开发和调试程序通常更容易。解释器往往允许每次执行程序的一部分，例如一行，使程序员能够检查中间结果，并随时修改代码。因此，解释方式下的调试通常较容易。解释执行的代码也更容易在不同计算系统之间移植。不过，由于中间还有解释器来实际完成工作，程序执行所需的时间会更长。采用编译方式时，程序员可以得到执行更快、存储器使用效率更高的代码。因为编译能生成效率更高的代码，大多数实际投入使用的软件倾向于用编译型语言编写。

11.4 C/C++ 编程语言

11.4.1 C 和 C++ 的起源

先来了解一点历史。C 语言由 Bell Laboratories 的 Dennis Ritchie 于 20 世纪 70 年代初开发。Ritchie 当时是 Bell Labs 的 Unix 操作系统开发团队成员。团队需要一种既有强大能力、又足够精简的语言，使简单的编译器也能生成高效代码；C 就是为满足这一需求而诞生的。Ritchie 及其团队所用的计算机存储器容量很小，因此，编译器生

²译注：这里描述的是解释器提供的抽象执行环境；采用解释执行本身并不保证具备安全沙箱的隔离能力。

成的代码必须很小，编译器本身也必须很小！正是这种务实的取向，使 C 流行起来，并成为当时最受欢迎的实用编程语言之一。

在计算行业，一样东西问世 50 年后仍然有用，实在难得。当然，本书的作者们是例外。C 语言依然充满活力，仍位居软件开发最常用的语言之列。

1979 年，也就是 C 问世后不久，同样在 Bell Labs 工作的计算机科学家 Bjarne Stroustrup，在 Ritchie 及其同事身边工作时，开始为 C 引入一系列改进。这些改进帮助程序员更好地组织 C 代码，尤其是大型、复杂程序的代码。Stroustrup 将类（class）的概念引入 C，由此发展出了 C++。C++ 使用 C 的基本类型、语法和构造，因此同样高效；它还提供类和模板（template）等附加特性，而这些特性对构建大型编码项目的基本组成部分十分重要。C 让我们能够创建数十万行代码的计算应用，C++ 则让我们能够创建数亿行代码的应用。

11.4.2 我们将怎样学习 C 和 C++

由于 C 和 C++ 广泛使用，而且贴近底层硬件，它们非常适合本书自底向上的探索方式。第 11 至第 19 章的大部分篇幅将用于 C，第 20 章则介绍 C++ 的一些核心概念。C++ 以 C 为基础，因此，前面介绍的 C 语言内容会有助于我们理解 C++。

C 和 C++ 都经过长期发展和演进，能够支持大规模编程。本书不会涵盖它们的所有方面，而是介绍共同的核心子集，使你能够独立地在一个源文件中编写数百行规模的程序。我们的目标，是让你打下数字系统、硬件和软件方面的基础。在后续的数据结构和软件工程课程中，这些基础将有助于你完成更复杂的软件项目。

本书中 C 语言的所有示例和具体细节，都以称为 ANSI C 或 C18 的标准版本为依据。与许多编程语言一样，C 多年来出现过若干变体。美国国家标准学会（ANSI）批准一种“无歧义且与机器无关的 C 语言定义”，以使语言标准化，促进 C 代码在不同系统和编译器之间的可移植性。原书写作时最新采用的 ANSI C 版本来自 2018 年，因此称为 C18。同样，本书中的 C++ 示例使用 ISO C++，也常称为 Standard C++。

介绍许多新的 C 和 C++ 概念时，我们都会配上由一个假想的 LC-3 C 编译器生成的 LC-3 代码。有时还会描述这些代码执行时实际发生的事情。请记住，你所用的计算机很可能并非基于 LC-3，而是基于 x86、ARM 等实际使用的 ISA。例如，如果你使用 Windows PC，编译器很可能生成 x86 代码，而非 LC-3 代码。本书的许多示例是完整程序，可以直接编译和执行。为了说明得更清楚，另一些示例并非完整程序，必须补全后才能编译。为加以区分，我们把这些不完整的代码示例称为**代码片段**（code segment）。

11.4.3 编译过程

C 和 C++ 都是编译型语言。C 或 C++ 编译器遵循从源程序转换为可执行映像的典型流程。**可执行映像**是程序的机器语言表示，已经可以加载进存储器并执行；可以回顾 7.4.1 节的有关介绍。编译机制涉及若干不同的组成部分，主要是预处理器、编译器本身和链接器。图 11.2 展示了 C 的整体编译过程。下面简要看看各个主要组成部分。

11.4.3.1 预处理器

顾名思义，**预处理器**（preprocessor）会在将源程序交给编译器之前，先进行“预处理”。它扫描源文件，也就是包含实际 C 程序的文件，查找并执行**预处理指令**（preprocessor directive）。这些指令类似于 LC-3 汇编语言中的伪操作，要求预处理器以规定的方式转换源文件。例如，可以要求预处理器把 `DAYS_THIS_MONTH` 替换成 30，或者在当前行插入 `stdio.h` 文件的内容。后续章节会讨论这两种操作为什么有用。所有预处理指令都以井号 # 开始。实用的 C 和 C++ 程序都会以某种方式借助预处理器。

11.4.3.2 编译器

预处理器转换了输入源文件后，程序就可以交给编译器了。编译器把预处理后的程序转换成**目标模块**。回顾 7.4.2 节，目标模块是整个程序中某一部分的机器代码。编译主要分为两个阶段：**分析**（analysis）和**综合**（synthesis）。分析阶段把源程序分解，也就是解析（parse）为组成部分；综合阶段则生成程序的机器代码版本。分析阶段负责读入、解析原始程序，并建立其内部表示。综合阶段生成机器代码；如果有相应要求，还会尝试优化代码，使其在目标计算机上执行得更快、更高效。这两个阶段通常又划分为若干子阶段，分别完成语法分析、寄存器分配、指令调度等具体任务。有些编译器会生成汇编代码，再使用汇编器完成到机器代码的转换。

编译器转换程序时使用的重要内部记录机制之一，是**符号表**。编译器用它跟踪程序员在程序中使用的所有符号名称。编译器的符号表与 LC-3 汇编器维护的符号表非常相似，后者见 7.3.3 节。第 12 章将更详细地考察编译器的符号表。³

³译注：原文写作“第 8 章”；有关编译器符号表的后续介绍实际在 12.5.1 节，已更正。

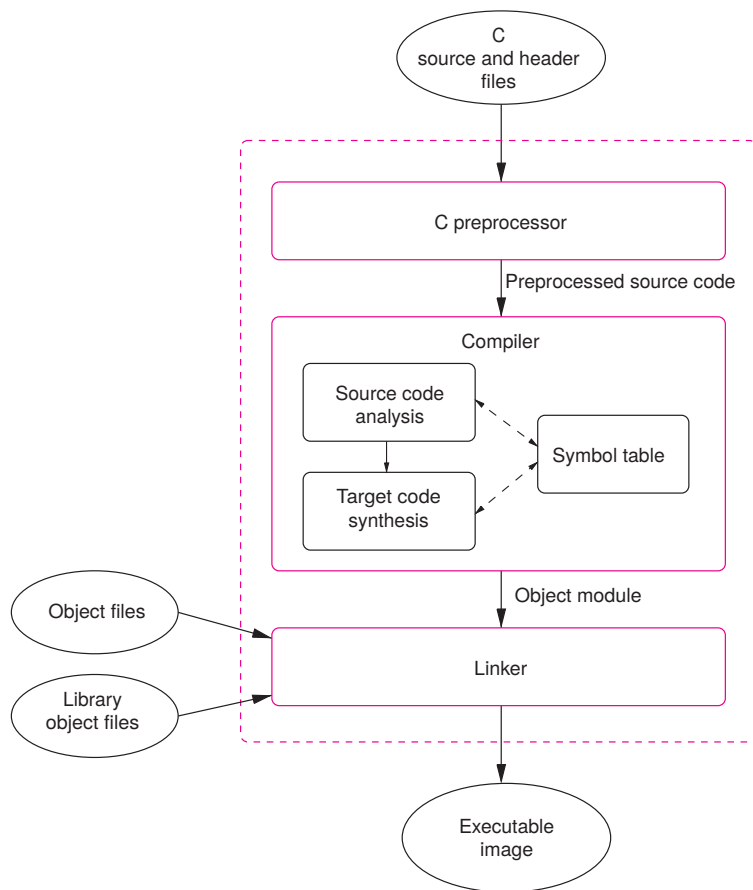


图 11.2 虚线框表示整体编译过程，包括预处理器、编译器和链接器。尽管编译器只是其中一部分，整个过程仍称为编译。输入是 C 源文件、头文件及各种目标文件，输出是可执行映像。

11.4.3.3 链接器

编译器把源文件转换成目标代码后，**链接器**接手。它的任务是把所有目标模块链接起来，构成程序的可执行映像。可执行映像是能够加载进存储器、由底层硬件执行的程序版本。例如，点击 Web 浏览器图标或启动手机上的应用，就是在指示操作系统把应用的可执行映像加载到存储器中，并开始执行。

C 程序经常借助**库例程**。库例程完成 I/O 等常见而有用的任务，由操作系统、编译器等系统软件的开发者提供，供大家使用。如果程序使用了某个库例程，链接器就会找到该例程对应的目标代码，将其链接到最终的可执行映像中。把库目标代码链接进来的过程，对你来说应该并不陌生：我们已在 8.1.4 节结合 LC-3 介绍过这一过程。⁴库文件通常存放在特定位置，具体位置取决于计算机系统。

11.4.4 软件开发环境

对于我们的学习任务，一种简单有效的开发流程是：先用 Notepad、TextEdit 或 vim 等文本编辑器编写 C 或 C++ 源代码，再通过命令行界面调用 C 或 C++ 编译器，

⁴译注：原文误写为 8.4.1 节；“库例程”实际位于 8.1.4 节。

执行完整的编译过程。生成的可执行程序也可以直接从命令行界面运行。对于本书中开发的简短程序，这样的流程已经能够满足很多需要。

对于大多数实际的软件开发任务，**集成开发环境**（integrated development environment, IDE）往往是更好的选择。IDE 把若干常用的软件开发工具组合到同一个框架中，包括用来创建、编辑源代码的文本编辑器、编译器，以及让调试代码更方便的调试器；第 15 章将介绍调试器。由于这些工具结合得更紧密，程序员使用 IDE 就能在整个开发过程中更迅速地编辑、编译和调试程序。

11.5 一个简单的 C 示例

现在就从一个简单的 C 示例入手。图 11.3 给出了源代码。这个程序提示用户输入一个数，然后从该数开始倒数到 0。

你可以从本书网站下载这一示例，用自己选择的开发环境编译它；为避免编译问题，请使用 ANSI C。现在不必急于弄懂每一行代码的作用。我们会先介绍这份源代码的总体组织方式，再涉及一些重要细节，帮助你开始编写自己的代码。这里重点关注四个方面：main 函数、编程风格、预处理指令以及输入输出的使用。


```
1  //
2  //
3  // Countdown, our first C program
4  //
5  // Description: This program prompts the user to type in
6  // a positive number and counts down from that number to 0,
7  // displaying each number along the way.
8  //
9  //
10
11 // The next two lines are preprocessor directives
12 #include <stdio.h>
13 #define STOP 0
14
15 // Function : main
16 // Description : prompt for input, then countdown
17 int main(void)
18 {
19     // Variable declarations
20     int counter;    // Holds intermediate count values
21     int startPoint; // Starting point for count down
22
23     // Prompt the user for input
24     printf("==== Countdown Program ====\\n");
25     printf("Enter a positive integer: ");
26     scanf("%d", &startPoint);
27
28     // Count down from the input number to 0
29     for (counter = startPoint; counter >= STOP; counter--)
30         printf("%d\\n", counter);
31 }
```

图 11.3 我们的第一个 C 程序。它提示用户输入一个正数，然后倒数到 0。

11.5.1 main 函数

main 函数从包含 `int main(void)` 的第 17 行开始，到第 31 行的右花括号结束。这些源代码构成了名为 main 的函数的**函数定义**。在 LC-3 汇编程序设计中称为子程序的东西，在 C 中称为**函数**。函数是 C 的重要组成部分，我们将用整个第 14 章来介绍它。在 C 中，main 函数有一个特殊作用：它是程序开始执行的地方。每个完整的 C 程序都需要一个 main 函数。注意，在 ANSI C 中，main 必须声明为返回整数值，因此函数名 main 前面有关键字 `int`。至于 `(void)`，后面再详细说明。

在这个示例中，main 函数的代码，即第 18 行和第 31 行两个花括号之间的代码，可以分成两部分。第一部分是函数的**变量声明**，位于第 20、21 行。这里创建了两个变量，分别名为 `counter` 和 `startPoint`，供 main 函数使用。变量是所有高级编程语言都提供的一种有用而基本的特性。借助变量，我们可以用符号名称指代程序中的值，而不必用存放这些值的存储器单元或寄存器来引用它们。

第二部分包含函数的**语句** (statement)，对应第 24 至第 31 行。这些语句表达了函数执行时要完成的动作。C 程序从 main 的第一条语句开始执行，逐条推进，直到 main 的最后一条语句完成。

在本例中，第一组语句，也就是第 24 至第 26 行，显示提示信息，要求用户输入一个整数。用户输入后，程序进入最后一条语句，即 for 循环。这是一种迭代结构，我们将在第 13 章讨论。循环从用户输入的数开始向下计数，直到 0。例如，用户输入 5 时，程序的输出如下：

```
==== Countdown Program ====  
Enter a positive integer: 5  
5  
4  
3  
2  
1  
0
```

请注意，本例中的许多源代码行都以分号；结尾。在 C 中，分号用于结束声明和语句，使编译器能够无歧义地将程序解析为各个组成部分。

11.5.2 格式、注释与风格

C 是一种**自由格式** (free-format) 语言。也就是说，程序中词与词、行与行之间用于分隔的空白，包括空格、制表符和换行，通常不改变程序的含义。只要遵守 C 的语法规则，程序员就可以按自己认为合适的方式组织程序。程序员利用这种自由安排代码格式，使代码更易阅读。请注意图 11.3 中 for 循环的缩进：它让反复执行的语句更容易辨认。还要注意，main 函数中不同区域的代码之间使用了空行。空行不是必需的，而是为了在视觉上分隔代码。共同完成一个较大任务的语句，通常会被组织成一个在视觉上易于识别的单元。

本书的 C 示例采用一种常见的传统缩进风格。风格并不唯一，程序员有时也借此表达自己的偏好。你可以确定自己的风格，但要记住，格式的目的是帮助传达程序的含义。

C 中的注释与 LC-3 汇编语言中的注释不同。C 的行注释以 // 开始，一直延续到该行末尾。请注意，本例既有整行注释，也有放在代码行末尾的注释。C++ 中的注释也可以以 // 开始并延续到行末；这是两种语言众多相似之处中的一个。

每一种编程语言都提供了让程序员编写注释的方式。注释使程序员能够用人类语言描述代码的作用。恰当地编写注释，是软件开发过程的重要组成部分。好的注释提高代码的可读性，使不熟悉代码的人能够更快地理解它。编程任务经常需要团队协作，因此代码常常在程序员之间共享或借用。要想在团队中有效工作，或写出值得分享的代码，就应当尽早养成良好的注释习惯。

良好注释风格的一个方面，是在每个源文件的开头说明文件中代码的用途、最后修改日期和修改者。此外，每个函数都应有一段简短说明，介绍它完成的任务以及输

入和输出，参见示例中的 main 函数。代码内部通常也会穿插注释，说明不同部分的意图。不过，注释过多也有害：它会让代码显得杂乱，反而更难阅读。尤其要避免那些只重复代码中显而易见的信息、没有提供任何新信息的注释。

11.5.3 C 预处理器

11.4.3 节简要提到过预处理器。回顾一下，它在把原始源程序交给编译器之前，先对其进行转换。我们的简单示例包含两种常用的预处理指令：`#define` 和 `#include`。本书的 C 和 C++ 示例只依赖这两种指令。`#define` 简单而强大，它要求预处理器将与 X 匹配的文本替换为 Y；也就是说，用 Y 替换 X。⁵在本例中，`#define` 将 `STOP` 替换为 `0`。因此，下面这行源代码：

```
for (counter = startPoint; counter >= STOP; counter--)
```

在内部，也就是预处理器与编译器之间，被转换为：

```
for (counter = startPoint; counter >= 0; counter--)
```

这样做为什么有用？`#define` 指令经常用于在程序中定义固定的值。例如：

```
#define NUMBER_OF_STUDENTS 25
#define MAX_LENGTH 80
#define LENGTH_OF_GAME 300
#define PRICE_OF_FUEL 1.49
#define COLOR_OF_EYES brown
```

这样，我们就可以用符号名称 `PRICE_OF_FUEL` 来指代燃料价格。价格变化时，只需修改它的定义，预处理器就会替我们完成整个代码中的实际替换。这很方便：即使程序中大量使用燃料价格，也只需修改源代码的一行，就能改变各处所用的价格。最后一个例子与前几个略有不同：这里是将一串字符 `COLOR_OF_EYES` 替换为另一串字符 `brown`。C 和 C++ 的常见编程风格，是将被替换的名称写成大写形式。

另一种将要遇到的指令是 `#include`。它指示预处理器在指令出现的位置，将另一个源文件插入代码。实质上，这条指令本身被另一个文件的内容替换。现在，这条命令的用途也许还不完全明显；随着我们深入学习 C 和 C++，你会了解到，可以用**头文件**（header file）保存供多个源文件使用的 `#define` 定义和声明。

在 C 程序中，我们经常会遇到 `#include <stdio.h>`。进行常规输入输出的 C 程序需要包含 C 输入输出库的头文件 `stdio.h`，其中定义了与库中 I/O 函数有关的信息。预处理指令 `#include <stdio.h>` 用来在开始编译前插入这一头文件。

`#include` 指令有两种形式：

```
#include <stdio.h>
#include "program.h"
```

⁵译注：这里用文本替换作入门说明。实际宏展开按预处理记号进行，不会任意替换标识符内部的子串或字符串字面量中的文字。

第一种用尖括号 `< >` 括住文件名，告诉预处理器到预先定义的系统目录中查找头文件。该目录通常由系统配置确定，其中包含 `stdio.h` 等许多常用的系统头文件。我们还经常需要包含为当前程序自行创建的头文件。第二种形式用双引号括住文件名，告诉预处理器可以在 C 源文件所在目录，或编译器已知的其他目录中查找头文件。

注意，这些预处理指令都不以分号结尾。`#define` 和 `#include` 是预处理指令，而不是 C 语句，因此不需要用分号结束。

11.5.4 输入与输出

本章最后介绍如何在 C 程序中进行输入输出。在 C 中，I/O 通过一组 I/O 函数完成。现在先从较高层次介绍这些函数；等第 18 章具备足够的背景知识后，再深入底层细节。实用程序都会进行某种形式的 I/O，因此，熟悉 C 的 I/O 能力是重要的第一步。

C 的 I/O 函数类似于 LC-3 系统软件提供的 IN 和 OUT TRAP 例程。图 11.3 的示例中有三行使用 C 库函数 `printf` 进行输出，见第 24、25、30 行。`printf` 的含义是格式化打印（print formatted）。它向标准输出设备输出，通常就是显示设备。它需要一个格式字符串（format string），其中提供两类内容：（1）要输出的文字；（2）如何在文字中输出程序中的值的格式说明。例如，语句：

```
printf("43 is a prime number.");
```

会向输出设备输出以下文字：

```
43 is a prime number.
```

除了文字，输出代码中生成的值也很有用。格式字符串中的格式说明指出了我们希望怎样输出这些值。来看几个例子：

```
printf("%d is a prime number.", 43);
```

这个例子的格式字符串含有格式说明 `%d`。它将格式字符串后面列出的值作为十进制数嵌入输出，取代 `%d` 的位置。得到的输出为：

```
43 is a prime number.
```

下面几个例子展示了同一 `printf` 的其他变化形式：

```
printf("43 plus 59 in decimal is %d.", 43 + 59);  
printf("43 plus 59 in hexadecimal is %x.", 43 + 59);  
printf("43 plus 59 as a character is %c.", 43 + 59);
```

第一条语句将 `43 + 59` 的结果按十进制数输出，因此嵌入文字中的值为 102。第二条的格式说明 `%x` 将 66 嵌入文字，因为十进制 102 等于 x66。同样，第三条中的 `%c` 将该值解释为 ASCII 字符并显示出来，此处就是小写字母 f。这条语句的输出为：

```
43 plus 59 as a character is f.
```

关键在于，这三条语句在格式字符串之后传给 `printf` 的二进制位模式是相同的。`printf` 在第一个例子中将位模式 `0110 0110`，即十进制 102，解释为十进制数；在第二个例子中解释为十六进制数；在第三个例子中解释为 ASCII 字符。C 的输出函数 `printf` 根据提供的格式说明，将位模式转换成适当的 ASCII 字符序列。所有格式说明都以百分号 % 开始。

最后一个例子展示了 `printf` 一种常见而有用的用法：

```
printf("The wind speed is %d km/hr.", windSpeed);
```

这里将程序运行过程中产生的值，也就是变量 `windSpeed` 的值，作为十进制数输出。显示什么，取决于执行这行代码时 `windSpeed` 的值。因此，如果执行这条 `printf` 语句时它等于 2，就会得到：

```
The wind speed is 2 km/hr.
```

如果希望输出中出现换行，就必须在格式字符串的相应位置明确写出。换行符、制表符等特殊字符需要使用以反斜杠 \ 开始的特殊序列。例如，要输出换行符，从而换到新的一行，就使用 `\n`。考虑以下语句：

```
printf("%d is a prime number.\n", 43);  
printf("43 plus 59 in decimal is %d.\n", 43 + 59);  
printf("The wind speed is %d km/hr.\n", windSpeed);
```

注意，每个格式字符串最后都会输出换行符 `\n`，因此后续的 `printf` 都会从新的一行开始。这三条语句的输出如下：⁶

```
43 is a prime number.  
43 plus 59 in decimal is 102.  
The wind speed is 2 km/hr.
```

图 11.3 的示例程序中，`printf` 出现了三次。前两次只显示文字，不显示值，因此没有格式说明。第三次则输出变量 `counter` 的值。一般而言，一次 `printf` 可以显示多个值。格式说明，如 `%d`，的个数必须与格式字符串后面要输出的值的个数相匹配。

问题：如果把示例程序中的第三个 `printf` 替换为下面这条语句，会发生什么？表达式 `startPoint - counter` 计算的是 `startPoint` 的值减去 `counter` 的值。

```
printf("%d %d\n", counter, startPoint - counter);
```

⁶译注：原文此处写作“五条语句”，实际列出了三条，已更正。

现在来看看输入函数 `scanf`。它从标准输入设备读入数据，通常就是键盘。它需要一个格式字符串，类似于 `printf` 所需的格式字符串，以及用于存放键盘输入值的变量列表。`scanf` 从键盘读入输入，按照格式字符串中的转换字符进行转换，再将转换后的值赋给所列出的变量。

在图 11.3 中，我们用 `scanf` 的格式说明 `%d` 读入一个十进制数。回顾 LC-3 键盘输入的讨论：经键盘接收到的值采用 ASCII 编码。格式说明 `%d` 告诉 `scanf`，预期输入的是一串数字的 ASCII 按键，也就是数字 0 至 9。这一序列被解释为十进制数，并转换为整数。得到的二进制位模式存入名为 `startPoint` 的变量中。`scanf` 自动替我们完成了类型转换，在这里是从 ASCII 转换为整数！`%d` 只是 `scanf` 可用的多种格式说明之一；其他格式说明可以读入单个字符、浮点数、以十六进制表示的整数等。

关于 `scanf`，有一点值得注意：在这里，被 `scanf` 修改的变量，例如 `startPoint`，前面必须加上 `&` 符号。第 16 章讨论指针时，我们将说明为什么需要这样做。

下面再给出几个 `scanf` 的例子：

```
// Reads in a character and stores it in nextChar
scanf("%c", &nextChar);

// Reads in a floating point number into radius
scanf("%f", &radius);

// Reads two decimal numbers into length and width
scanf("%d %d", &length, &width);
```

11.6 小结

本章介绍了高级编程语言的一些主要特点，尤其是 C 和 C++ 的特点，并初步接触了 C 语言。主要包括：

- **高级编程语言。**高级语言针对数字系统直接操作的位和存储器，提供便于人使用的抽象，以简化编程过程。计算机只能执行机器代码，因此高级语言程序必须通过编译或解释，转换成硬件能够执行的形式。
- **C 和 C++ 编程语言。**C 和 C++ 具有贴近底层硬件的特点，非常适合自底向上地学习计算；它们也是当今最流行的编程语言中的两种。C/C++ 的编译过程涉及预处理器、编译器和链接器。
- **我们的第一个 C 程序。**我们用一个非常简单的程序说明了 C 程序的若干基本特征。注释、缩进和风格有助于向阅读代码的人传达程序的含义。许多 C 程序使用 `#define` 和 `#include` 预处理指令。C 程序从 `main` 函数开始执行，该函

数本身由变量声明和语句组成。最后，C 中的 I/O 可以借助库函数 `printf` 和 `scanf` 来完成。

习题

- 11.1 描述你用低级语言编程时遇到的一些问题或不便。
- 11.2 高级语言怎样帮助减少低级语言编程中的繁琐工作？
- 11.3 用高级语言编程有哪些缺点？
- 11.4 比较解释器执行程序与编译后的二进制程序执行这两个过程的异同。解释对性能有什么影响？
- 11.5 如果一种语言的源代码能在不同计算系统上运行，例如在具有不同 ISA 的系统上运行，就说它具有可移植性。为什么解释型语言比编译型语言更容易移植？
- 11.6 命令行界面是一种解释器。为什么它不能是编译器？
- 11.7 LC-3 模拟器是编译器还是解释器？
- 11.8 编译相对于解释的另一个优点，是编译器能够更彻底地优化代码。生成机器代码时，编译器能够考察整个程序，因此可以分析程序试图完成的任务，减少计算量。

下面的算法根据键盘输入的值完成一些很直接的算术运算，并输出一个结果。

- 1. 从键盘获取 W 。
- 2. $X \leftarrow W + W$
- 3. $Y \leftarrow X + X$
- 4. $Z \leftarrow Y + Y$
- 5. 将 Z 输出到屏幕。
 - a. 解释器会逐条执行程序语句，总共执行五条语句。如果底层 ISA 能完成所有算术运算，即加、减、乘、除，那么执行这个程序至少需要多少个操作？说明这些操作分别是什么。
 - b. 编译器会在生成机器代码之前分析整个程序，并可能优化代码。如果底层 ISA 能完成所有算术运算，即加、减、乘、除，那么执行这个程序至少需要多少个操作？说明这些操作分别是什么。
- 11.9 本题参照图 11.2。
 - a. 描述 C 预处理器的输入。
 - b. 描述 C 编译器的输入。
 - c. 描述链接器的输入。

11.10 如果将图 11.3 程序的倒数第二行，从 `printf("%d\n", counter);` 分别改为以下语句，会发生什么？

- a. `printf("%c\n", counter + 'A');`
- b. `printf("%d\n%d\n", counter, startPoint + counter);`
- c. `printf("%x\n", counter);`

11.11 `scanf` 从键盘读入字符，`printf` 将它输出。下面两条语句完成什么任务？

```
scanf("%c", &nextChar);  
printf("%d\n", nextChar);
```

11.12 某个程序中出现了以下几行 C 代码。每条 `printf` 语句的输出是什么？

```
#define LETTER 'l'  
#define ZERO 0  
#define NUMBER 123  
printf("%c", 'a');  
printf("x%x", 12288);  
printf("$%.%c%d n", NUMBER, LETTER, ZERO);
```

11.13 描述一个程序：它从键盘读入一个十进制数，并输出其十六进制等价表示。目前还不要求你写出能够工作的 C 代码。

第十二章 变量与运算符

12.1 引言

本章介绍高级语言程序设计中的两个基本概念：变量和运算符。变量保存程序要处理的值，运算符则是语言中用于操作这些值的构造。两者结合起来，使程序员更容易表达程序要完成的工作。

下面这条 C 语句同时涉及变量和运算符。加法运算符 + 将变量 score 原来的值加上 3，再用赋值运算符 = 将新值赋回 score。如果执行前 score 等于 7，执行后就等于 10。

```
score = score + 3;
```

本章首先仔细考察 C 的变量。C 变量很直观，最基本的四类是整数、字符、浮点数和布尔值；布尔变量取值为 0 或 1。随后，我们介绍 C 丰富的运算符，并用大量代码示例说明它们的操作。我们的学习方式有一个独特之处：能把这些高级概念与底层联系起来。因此，本章第三部分将考察编译器如何转换包含变量和运算符的 C 代码。最后，我们还会编写一些涉及变量与运算符的简单 C 代码。

12.2 变量

值是程序对其执行操作的任何数据项。例如，循环的迭代计数值、用户输入的值，或者一系列数相加时得到的部分和，都是值。

跟踪这些值会耗费程序员大量精力，高级语言让管理它们的过程变得更容易。程序员可以用符号名称引用值，而不必直接使用存放它的存储位置。要操作一个值，只需引用其符号名称，实际存储位置则被抽象起来。编译器会生成完整的数据传送操作，从值所在的存储器位置访问它。程序员因此可以专注于程序逻辑，而不必关心值当前存在哪里。在高级语言中，这些具有符号名称的值称为**变量**。

变量是最基本的一种存储器对象。随着 C 和 C++ 的学习深入，我们会进一步解释对象的概念。目前，可以将对象看作存储器中一个有名称、具有某些特征和行为的数据单位。从 C 到 C++，再到 Java、C# 等语言的演进，扩展了我们在代码中创建和管理更一般对象的能力。不过，现在先专注于最简单的存储器对象：变量。

为了正确跟踪程序中的变量，C 或 C++ 编译器需要知道每个变量的若干特征，而这些信息最终由程序员提供。编译器需要知道变量的符号名称、将要保存的信息类型，以及代码中哪些地方可以访问它。在 C 和 C++ 中，这些信息通过变量的**声明**提供。

例如，下面声明了一个名为 echo、保存整数值的变量：

```
int echo;
```

根据这条声明，编译器为 `echo` 保留足以保存一个整数的存储空间。在后续 C 代码中引用 `echo` 时，编译器就生成适当的机器代码来访问它。有时，编译器可以优化程序，将 `echo` 放在寄存器中，从而不需要存储器单元；这一点以后再谈。

12.2.1 四种基本数据类型

有一个概念我们已反复以不同形式见过：某个位模式的含义，取决于赋予它的数据类型。例如，二进制位模式 `0110 0110` 可以表示小写字母 `f`，也可以表示十进制数 `102`，取决于我们将它视为 ASCII 数据还是二进制补码整数。变量声明告诉编译器变量的类型。编译器利用类型信息，为变量分配适当数量的存储空间。类型还决定了对变量的操作在机器层面如何实现。例如，两个整数变量相加，在 LC-3 中可以用一条 `ADD` 完成；如果两个变量是浮点类型，LC-3 编译器就必须生成一串指令，因为 LC-3 没有单条浮点加法指令。

C 支持整数、字符、浮点数和布尔值这四种基本数据类型。可以分别用类型说明符 `int`、`char`、`float`（或 `double`）以及 `_Bool`（或 `bool`）创建这些类型的变量。

int 下面声明了一个名为 `numberOfSeconds` 的整数变量。编译器会为它分配一个整数所需的存储空间；在本书的 LC-3 实现中，就是一个存储器单元。

```
int numberOfSeconds;
```

整数变量在程序中十分常见，这并不奇怪。它们能方便地表示我们希望程序处理的、来自现实世界的离散数据。例如，以秒表示时间时，整数变量可能很合适。在跟踪鲸鱼迁徙的应用中，可以用整数表示在 California 海岸外观察到的灰鲸群规模。整数还可用于程序控制，例如作为计数器控制循环的迭代计数器。

`int` 的内部表示和取值范围取决于底层硬件 ISA 及其实现。在 LC-3 中，`int` 是 16 位二进制补码整数，能表示 $-32\,768$ 至 $+32\,767$ 。在基于 x86 的系统中，`int` 很可能是 32 位二进制补码数，能表示 $-2\,147\,483\,648$ 至 $+2\,147\,483\,647$ 。原书将多数情形下的 `int` 概括为与底层 ISA 字长相同的二进制补码整数。¹

char 类型说明符 `char` 声明一个用其数据值表示字符的变量。下面第一条声明创建 `lock`，第二条创建 `key`。第二条稍有不同：它还包含**初始化器**（`initializer`）。在 C 中，可以在变量声明中直接设置初值。这里，`key` 的初值是大写字母 `Q` 的 ASCII 码。还要注意 `Q` 两侧的单引号：在 C 中，按字符字面量解释的字符用单引号括起来。`lock` 的初值又是什么呢？本章稍后会仔细讨论。

```
char lock;  
char key = 'Q';
```

¹译注：`int` 的宽度由 C 实现决定，不必等于处理器字长；例如，64 位系统中的 `int` 也常为 32 位。这里的 16 位约定针对本书假想的 LC-3 C 编译器。

虽然 8 位足以保存一个 ASCII 字符，但为了简化本书示例，LC-3 上的所有 char 变量都占 16 位。也就是说，char 和 int 一样，各占一个 LC-3 存储器单元。

float / double float 声明单精度浮点数，double 声明双精度浮点数。它们的表示类似于 2.7.1 节介绍的形式。

浮点数使我们能够方便地处理有小数部分的数，以及很大或很小的数。回顾 2.7.1 节：在最底层，浮点数是一个位模式，其中一位表示符号，一部分位表示尾数，其余位表示指数。下面是三个 double 变量的例子：

```
double costPerLiter;  
double electronsPerSecond;  
double averageTemp;
```

与 int 和 char 一样，也可以选择声明中初始化浮点变量。不过，浮点值比整数和字符稍复杂。浮点字面量包含小数点或指数，也可以同时包含两者，如下所示。指数用字母 e 或 E 标记，可以为正，也可以为负；它表示 e 或 E 前面那部分数值应乘以 10 的多少次幂。注意，指数本身必须是整数。

```
double twoPointOne = 2.1;      // This is 2.1  
double twoHundredTen = 2.1E2;  // This is 210.0  
double twoHundred = 2E2;      // This is 200.0  
double twoTenths = 2E-1;      // This is 0.2  
double minusTwoTenths = -2E-1; // This is -0.2  
double extTemp = -0.2;        // This is -0.2
```

浮点数的精度取决于表示中分配给小数部分的位数。在 C 中，具体取决于编译器和 ISA，double 为小数部分分配的位数可以多于 float，但不会少于它。double 的大小也取决于 ISA 和编译器。通常，按 IEEE 754 浮点标准，double 长 64 位，float 长 32 位。

_Bool / bool _Bool 是后来加入 C 语言的类型，现已纳入本书采用的 ANSI 标准。它的变量取值为 0 或 1，表示假或真。C++ 原生支持布尔类型，C 后来也加入了支持，为代码中经常出现的一类值提供了表示方式。

C 的布尔类型说明符是 _Bool。更方便的写法是使用 bool，但在本书采用的 C18 中，需要包含头文件 stdbool.h。这一头文件还将符号 true 定义为 1，将 false 定义为 0。

```
_Bool flag = 1; // Initialized to 1 or true  
bool test = false; // Initialized to false, which is symbolically  
// defined to 0. Requires stdbool.h
```

12.2.2 选择标识符

大多数高级语言对程序员选择的变量名，或者更一般地说，**标识符** (identifier)，都有较灵活的规则。C 允许用字母、数字和下划线 _ 构成标识符，但开头只能是字母

或下划线。大小写不同，因此 Capital 与 capital 是不同名称。ANSI C 不限制标识符的书写长度。标识符还不能是**关键字**，也就是语言中已有保留含义的词，例如 int。

下面是常见 C 命名约定中的几条建议。以下划线开头的名称，例如 `_index`，通常只在特殊的库代码中使用。变量名几乎不会全部大写；全大写约定留给用 `#define` 创建的符号值。程序员喜欢在视觉上分隔由多个词构成的变量名。本书通过大写后续单词的首字母来分隔，例如 `wordsPerSecond`；另一些程序员则偏好下划线，例如 `words_per_second`。

为变量起有意义的名字，是写出良好代码的重要一环。变量名应反映其值的某种特征，让程序员容易想起这个值的用途。例如，表示物体速度的值可以命名为 `metersPerSecond`，而不只是 `x`。

12.2.3 作用域：局部与全局

我们已经提到，变量声明帮助编译器管理变量的存储。在 C 中，声明向编译器传递三类信息：标识符、类型和**作用域**（scope）。前两项直接写在声明中，第三项则由编译器根据声明在代码中的位置推断。

变量的作用域，是程序中可以访问该变量的区域。好消息是，就这里讨论的变量而言，C 中只有两种基本情况：变量对整个程序是全局的，或者只属于某个特定代码块，是局部的、私有的。²

12.2.3.1 局部变量

在 C 中，所有变量都必须先声明后使用。如果变量声明在一个块内，那么从声明处直到块末尾都可见。**块**（block）是程序中以左花括号 { 开始、以右花括号 } 结束的一部分。这样的变量是其声明所在块的局部变量。

下面的简单 C 程序从键盘读入一个数，再将其显示到屏幕上。整数变量 `echo` 声明在包含 `main` 函数代码的块中，只对 `main` 可见。如果程序还包含其他函数，它们就不能访问这个变量。ANSI C 允许局部变量在其作用域内、使用之前的许多位置声明。本书示例采用一种简单而更有条理的风格：在使用变量的块的开头声明它们。

```
#include <stdio.h>
int main(void)
{
    int echo;
    scanf("%d", &echo);
    printf("%d\n", echo);
}
```

有时，在同一函数的不同块中声明两个同名变量很有用。例如，一个程序有几个不同

²译注：作用域描述名称在代码中的可见范围，不等同于对象在运行期间存在多久的“存储期”。原文用“存活”作直观说明，应注意区分。

循环时，为各自的计数器都使用 `count` 这一名称可能很方便。只要同名的不同变量在不同块中声明，C 就允许这样做。下一部分讨论的图 12.1 给出了例子。

12.2.3.2 全局变量

局部变量只能在声明它的块内访问，而全局变量可以在整个程序中访问。³全局变量在程序运行的整个期间保留其存储空间和值。

```
1 #include <stdio.h>
2
3 int globalVar = 2;          // This variable is a global variable
4
5 int main(void)
6 {
7     int localVar = 3;       // This variable is local to main
8
9     printf("globalVar = %d, localVar = %d\n", globalVar, localVar);
10
11     // Creating a new sub-block within main
12     {
13         int localVar = 4;   // This local to the sub-block within main
14
15         printf("globalVar = %d, localVar = %d\n", globalVar, localVar);
16     }
17
18     printf("globalVar = %d, localVar = %d\n", globalVar, localVar);
19 }
```

图 12.1 演示嵌套作用域的 C 程序

全局变量很有诱惑力！它们经常能轻松解决编程时遇到的各种难题。不过，即使适量使用全局变量，也可能给大型项目带来问题。因为代码中任何地方都可能修改它们，它们会为程序错误打开方便之门，给维护带来挑战，使扩展和修改更加困难。因此，实际项目的编码规则往往限制全局变量的使用。本书也会尽量少用全局变量。

来看一个同时包含全局变量和局部变量的完整例子。图 12.1 中，`globalVar` 是可以从代码各处直接访问的全局变量。程序还包含两个同名、但作用域不同的局部变量：第 7 行声明的 `localVar` 属于 `main`；第 13 行声明的另一个 `localVar` 属于第 12 行左花括号开始、到第 16 行结束的子块。编译、运行这段代码，会得到以下输出：

```
globalVar = 2, localVar = 3
globalVar = 2, localVar = 4
globalVar = 2, localVar = 3
```

³原注：这略有简化。C 也允许全局变量只对某个源文件可见，而不是整个程序；这一细节与这里的讨论无关。

12.2.3.3 变量的初始化

没有初始化器的变量，其初值是什么？在 C 中，默认情况下，局部变量的初值是不确定的。也就是说，除非明确初始化，否则其中可能是无意义的“垃圾值”。全局变量则默认初始化为 0。在声明中明确初始化局部变量，是常见的编码习惯。⁴

12.2.4 更多例子

以下代码给出了本章讨论的四种基本类型的例子。有些声明有初始化器，有些没有。请注意浮点字面量和字符字面量在 C 中的写法。

```
double width;
double pType = 9.44;
double mass = 6.34E2;
double verySmallAmount = 9.1094E-31;
double veryLargeAmount = 7.334553E102;
int average = 12;
int windChillIndex = -21;
int unknownValue;
int mysteryAmount;
bool flag = false;
char car = 'A'; // single quotes specify a single ASCII character
char number = '4'; // single quotes specify a single ASCII
                  ↪ character
```

12.3 运算符

与许多高级语言一样，C 提供了丰富的运算符，让程序员能够操作变量。有些完成算术运算，有些完成逻辑运算，还有些比较两个值。这些运算符使用我们熟悉的符号，使计算的表达更自然、方便、紧凑，比将操作写成一串汇编指令更便于人理解。

给定 C 代码后，编译器的任务是将其转换为底层硬件能执行的机器代码。我们假想的 LC-3 C 编译器必须把程序包含的各种操作转换为 LC-3 指令。LC-3 直接支持的运算很少，主要是 AND、ADD、NOT，这显然不是轻松的任务。为说明这一点，来看一条将两个整数相乘的 C 语句。x、y、z 都是整数变量，将 x 和 y 相乘，再把结果赋给 z：

```
z = x * y;
```

LC-3 没有将两个值相乘的单条指令，因此编译器必须生成一串代码，完成两个可能为负的整数的乘法。一种方法是反复把 x 加到累计结果中，总共加 y 次，类似于第 10 章计算器中的做法。

⁴译注：这里的局部变量指本章示例中的自动存储期变量。具有静态存储期的局部变量也会默认零初始化。不要把未初始化的值当作一个可安全使用的随机数。

图 12.2 给出了编译器生成的 LC-3 代码。假设 R5 保存变量 x 所在的存储器地址，前一个位置，即 $R5-1$ ，分配给 y ，再前一个位置分配给 z 。这种分配经过专门安排，遵循 12.5.2 节将介绍的方法。

```

AND R0, R0, #0      ; R0 <= 0
LDR R1, R5, #0      ; load value of x
LDR R2, R5, #-1     ; load value of y
BRz DONE           ; if y is zero, we're done
BRp LOOP           ; if y is positive, start mult

                        ; y is negative
NOT R1, R1
ADD R1, R1, #1      ; R1 <= -x

NOT R2, R2
ADD R2, R2, #1      ; R2 <= -y (-y is positive)

LOOP ADD R0, R0, R1   ; Multiply loop
    ADD R2, R2, #-1   ; The result is in R2
    BRp LOOP

DONE STR R0, R5, #-2 ; z = x * y;

```

图 12.2 实现 C 乘法的 LC-3 代码

译注：图中循环旁的英文注释“*The result is in R2*”应为 $R0$ ；乘积累计在 $R0$ 中，最后也是将 $R0$ 存入变量 z 。原程序图保留原貌。

12.3.1 表达式与语句

继续之前，先介绍一点 C 语法。变量、字面值可以通过运算符组合成 C 表达式 (expression)，例如前面的乘法表达式 $x * y$ 。表达式可以组成语句，例如 $z = x * y$ ；。C 语句类似于英语中的完整句子：句子表达完整的思想或动作，C 语句则表达数字硬件要完成的一个完整工作单位。这里的简单语句以分号；结束，就像标点符号结束一个句子一样。程序执行从 main 的第一条语句开始，按程序规定的顺序逐条推进。

把一个或多个简单语句和声明放在花括号 { } 中，就构成复合语句 (compound statement)，也称为块。从语法上说，可以在需要一条语句的位置使用复合语句。下面给出几个例子：

```
z = x * y; // This statement accomplishes some work
{ // This is a compound statement
    a = b + c;
    i = p * r * t;
}
k = k + 1; // This is another simple statement
```

12.3.2 赋值运算符

我们已经见过 C 的**赋值运算符**，它的符号是等号 =。这个运算符先求出右侧表达式的值，再将它赋给左侧指定的对象，例如变量。在下面的语句中，变量 a 被设置为表达式 b + c 的值：

```
a = b + c;
```

虽然数学中的等号和 C 的赋值符号相同，但含义不同。在数学中，写等号是在断言左右两侧的表达式相等；在 C 中，= 让编译器生成代码，改变左侧的值，使其等于右侧的值。换句话说，是把右侧的值赋给左侧。

看看 LC-3 C 编译器怎样为包含赋值运算符的语句生成代码。下面这条 C 语句使整数变量 x 增加 4：

```
x = x + 4;
```

对应的 LC-3 代码很直接。假设 R5 保存 x 的地址：

```
LDR R0, R5, #0 ; Get the value of x
ADD R0, R0, #4 ; calculate x + 4
STR R0, R5, #0 ; x = x + 4;
```

在 C 中，每个表达式都会求得一个具有特定类型的值。前例中，x + 4 的结果为整数，因为整数 4 与另一个整数，即变量 x，相加；随后将结果赋给整数变量。如果构造混合类型表达式，例如 x + 4.3，会怎样？对于这样的例子，C 会把整数转换为浮点数。如果表达式同时涉及整数和字符类型，字符值会提升为整数类型。一般来说，混合运算会进行相应的类型转换，通常由较小的类型转换为较大的类型。

如果把一种类型的表达式赋给另一种类型的变量，例如下面这样，又会怎样？

```
x = x + 4.3;
```

在 C 中，变量的类型是固定的，不会改变，因此表达式的结果要转换为变量的类型。这里，浮点表达式 x + 4.3 被转换为整数。C 将浮点数转换为整数时，丢弃小数部分，也就是向零截断：4.3 转换为 4，5.9 转换为 5。

12.3.3 算术运算符

算术运算符很容易理解。许多操作和符号，从小学学习算术时我们就已经熟悉。例如，+ 表示加法，- 表示减法，* 表示乘法，/ 表示除法。乘法采用 *，而非平常手

写的符号，是为了避免与字母 x 混淆。与手算一样，表达式中的运算有一定的优先关系：乘除优先于加减。这称为**优先级**（precedence），12.3.4 节将详细讨论。下面是用算术运算符和赋值运算符构成的几条 C 语句：

```
distance = rate * time;
netIncome = income - taxesPaid;
fuelEconomy = milesTraveled / fuelConsumed;
area = 3.14159 * radius * radius;
y = a*x*x + b*x + c;
```

C 还提供一个可能不如加减乘除熟悉的算术运算符：整数**余数运算符** `%`。来看两个整数相除时的情况。C 的整数除法会丢弃小数部分，只保留整数部分。因此， $11 / 4$ 的值为 2。`%` 可计算整数余数，例如 $11 \% 4$ 的值为 3。换句话说， $(11 / 4) * 4 + (11 \% 4)$ 等于 11。以下例子中的变量均为整数：

```
quotient = x / y; // if x = 7 and y = 2, quotient = 3
remainder = x % y; // if x = 7 and y = 2, remainder = 1
```

表 12.1 列出算术运算及其符号。乘法、除法和取余的优先级高于加法、减法。

表 12.1 C 中的算术运算符

运算符	运算	用法示例
*	乘法	$x * y$
/	除法	x / y
%	整数余数	$x \% y$
+	加法	$x + y$
-	减法	$x - y$

12.3.4 求值次序

执行下面语句后， x 中存放什么值？如果没有相关规则，结果就会有歧义。

```
x = 2 + 3 * 4;
```

译注：本节主要讲表达式的分组规则。优先级和结合性决定表达式如何解析，并不普遍规定各个操作数在运行时的实际求值先后；括号也不自动建立副作用之间的先后关系。

12.3.4.1 优先级

与手算一样，表达式中的运算具有优先关系。例如，乘除的优先级高于加减。C 的算术运算符遵循小学算术中相同的规则。前面的语句会把 14 赋给 x ，因为乘法优先于加法，表达式按 $2 + (3 * 4)$ 分组。

12.3.4.2 结合性

那么，优先级相同的运算符怎么办？例如：

```
x = 2 + 3 - 4 + 5;
```

如果采用不同的分组方式， $2 + 3 - 4 + 5$ 可以得到 6，也可以得到 -4。C 中加法与减法的优先级相同，因此还需要规则来确定分组。对于同级运算符，这一规则称为**结合性**（associativity）。加法和减法都从左向右结合，因此这个表达式等同于 $((2 + 3) - 4) + 5$ 。

表 12.5 汇总了 C 运算符的优先级和结合性。我们不建议死记这张表，除非你很喜欢向朋友背诵 C 的冷知识。重要的是知道这些规则存在，并大致理解其中的逻辑。需要知道特定运算符之间的关系时，随时查表即可。此外，还有一个可靠的帮手：括号。

12.3.4.3 括号

括号明确指定表达式的分组，从而可以覆盖默认的优先级规则。与算术一样，可以从最内层括号开始理解表达式。希望把某个子表达式作为整体时，就用括号将其括起来。假设 a、b、c、d 均为 4，那么语句：

```
x = a * b + c * d / 4;
```

可以等价地写成：⁵

```
x = (a * b) + ((c * d) / 4);
```

两条语句都会使 x 等于 20。理解表达式时，先处理括号内的子表达式，再逐层向外，未加括号的部分则遵循优先级规则。如果四个变量仍均为 4，下面的表达式结果又是多少？

```
x = a * (b + c) * d / 4;
```

括号还可以提高可读性。大多数读代码的人不会背下全部 C 优先级规则。因此，表达式较长或较复杂时，即使不加括号也能正确运行，使用括号通常仍是更好的风格。

12.3.5 位运算符

现在继续讨论 C 的运算符。**位运算符**（bitwise operator）操作一个值中的各个位，逐位执行 AND、OR、NOT、XOR 等逻辑操作。例如，& 的作用类似于 LC-3 的 AND 指令，对两个输入操作数逐位与。| 执行逐位或，~ 执行逐位非；后者只需要一个操作数，因此是一元运算符。^ 执行逐位异或。⁶下面给出对 16 位值操作的例子。在 C 中，前缀 0x 表示十六进制值。

⁵译注：原书第一式的除数写作 2，而第二式写作 4，却称两式等价、结果均为 20。这里按第二式和所述结果，将第一式的除数更正为 4；若保留除数 2，第一式的结果应为 24。

⁶译注：原文说明 XOR 时误用了 OR 的符号 |，此处按后面的示例与运算符表更正为 ^。

```
0x1234 | 0x5678 // equals 0x567C
0x1234 & 0x5678 // equals 0x1230
0x1234 ^ 0x5678 // equals 0x444C
~0x1234 // equals 0xEDCB
1234 & 5678 // equals 1026
```

C 的位运算符还包括两个移位运算符：左移 << 和右移 >>。两者都是二元运算符，需要两个操作数：第一个是要移动的值，第二个是移动的位数。左移时，空出的位填 0；本书所说明的算术右移则进行符号扩展。表达式的值是移位结果，两个原操作数本身并不改变。下面仍以 16 位整数为例：

```
0x1234 << 3      // equals 0x91A0
0x1234 >> 2      // equals 0x048D
1234 << 3        // equals 9872
1234 >> 2        // equals 308
0x1234 << 5      // equals 0x4680 (result is 16 bits)
0xFEDC >> 3      // equals 0xFFDB (from sign-extension)
```

译注：这些例子按原书的 16 位截断与算术右移模型展示位模式，不能直接推广到任意 C 实现。无符号数右移补 0；在 C18 中，负的有符号数右移由实现定义，有符号左移也有取值限制。尤其在 16 位 int 实现中，字面量 0xFEDC 为 unsigned int，直接右移应补 0；原例的 0xFFDB 是将同一位模式视为负的有符号数并采用算术右移时的结果。

下面是用位运算符组成的 C 语句。位运算的操作数必须是整数类型，不能是浮点类型。这里 f、g、h 都是整数：

```
h = f & g;      // if f = 7, g = 8, h will equal 0
h = f | g;      // if f = 7, g = 8, h will equal 15
h = f << 1;     // if f = 7, g = 8, h will equal 14
h = g << f;     // if f = 7, g = 8, h will equal 1024
h = ~f | ~g;    // if f = 7, g = 8, h will equal -1
// because h is a signed integer
```

表 12.2 列出位运算及其符号。优先级由高到低为：NOT、同级的左右移位、AND、XOR、OR。二元位运算符从左向右结合，一元 NOT 从右向左结合。⁷

表 12.2 C 中的位运算符

运算符	运算	用法示例
~	逐位 NOT	~x
&	逐位 AND	x & y
	逐位 OR	x y
^	逐位 XOR	x ^ y
<<	左移	x << y
>>	右移	x >> y

⁷译注：原文称所有位运算符均从左向右结合，并称表 12.2 按优先级排列；这两点均不准确。表中顺序保留原书编排，完整优先关系见表 12.5。

12.3.6 关系运算符

C 提供若干运算符，检测两个值之间的关系。第 13 章将看到，它们可用于构成改变语句执行流程的条件结构。相等运算符 `==` 就是一个例子：两个值相等时，表达式的值为 1；不相等时为 0。

```
q = (312 == 83); // q will equal 0
z = (x == y); // z will equal 1 if x equals y
```

第二个例子中，赋值运算符 `=` 右侧的表达式 `x == y` 根据两者是否相等，得到 1 或 0。

与相等运算符相反，`!=` 在两个操作数不等时得到 1。其他关系运算符检测大于、小于等关系，如下所示。f、g、h 都是整数，f 为 7，g 为 8。

```
h = f == g; // Equal To operator. h will equal 0
h = f > g; // Greater Than operator. h will equal 0
h = f != g; // Not Equal To operator. h will equal 1
h = f <= g; // Less Than Or Equal To. h will equal 1
```

下一个例子预告了后面的内容。关系运算符特别适合通过检测变量来改变程序流程。下面只有在 `tankLevel` 为 0 时才输出消息。它使用 C 的 `if` 语句，第 13 章将详细讨论。我们在 LC-3 编程时已见过选择结构，因此这个概念并不陌生。

```
if (tankLevel == 0)
    printf("Warning: Tank Empty!!\n");
```

表 12.3 列出关系运算符及各自的简单示例。前四个的优先级高于后两个，两组都从左向右结合。

表 12.3 C 中的关系运算符

运算符	运算	用法示例
>	大于	<code>x > y</code>
>=	大于或等于	<code>x >= y</code>
<	小于	<code>x < y</code>
<=	小于或等于	<code>x <= y</code>
==	等于	<code>x == y</code>
!=	不等于	<code>x != y</code>

12.3.7 逻辑运算符

乍看之下，C 的逻辑运算符与某些位运算符十分相似，初学者有时会混淆。与位运算符不同，逻辑运算符根据条件的真假，只产生整数值 1 或 0。这与 `bool` 用 1 表示真、0 表示假的方式一致。

C 支持三种逻辑运算符：&&、|| 和!。&& 执行逻辑 AND；两个操作数都非零时得到整数 1，否则得到 0。因此，3 && 4 为 1，3 && 0 为 0。

|| 是逻辑 OR。x || y 在 x 或 y 至少一个非零时得到 1。例如，3 || 4 为 1，3 || 0 也为 1。

逻辑非! 得到操作数相反的逻辑状态：只有 x 等于 0 时，!x 才为 1，否则为 0。这些运算符可用于构造程序中的逻辑条件。例如，组合关系运算和逻辑运算，可以判断变量是否处于某个范围。判断 x 是否在 10 到 20 之间，包含端点，可写成：

```
(10 <= x) && (x <= 20)
```

判断字符 c 是否为英文字母，可写成：

```
(( 'a' <= c) && (c <= 'z')) || (( 'A' <= c) && (c <= 'Z'))
```

下面列出一些逻辑运算的例子，并穿插前面的位运算示例，以突出两者的区别。变量仍均为整数，f 为 7，g 为 8。

```
h = f & g;      // bitwise operator: h will equal 0
h = f && g;     // logical operator: h will equal 1
h = f | g;     // bitwise operator: h will equal 15
h = f || g;    // logical operator: h will equal 1
h = ~f | ~g;   // bitwise operator: h will equal -1
h = !f && !g;  // logical operator: h will equal 0
h = 29 || -52; // logical operator: h will equal 1
```

表 12.4 列出 C 的逻辑运算符及符号。逻辑 NOT 优先级最高，其次是 AND，再其次是 OR。完整列表见表 12.5。

表 12.4 C 中的逻辑运算符

运算符	运算	用法示例
!	逻辑 NOT	!x
&&	逻辑 AND	x && y
	逻辑 OR	x y

12.3.8 自增与自减运算符

变量的递增和递减十分常见，因此 C 的设计者专门提供了相应运算符。++ 将变量增加到下一个值，-- 则使其递减。例如，x++ 将整数变量 x 增加 1，x-- 将其减少 1。请记住，它们会改变变量本身的值：x++ 类似于 x = x + 1。这一特性如此鲜明，以至于 C 的改进版本取名 C++，就是借用了这个语言特性。

++ 和 -- 可以放在变量的任一侧：放在前面称为前置自增或前置自减，放在后面称为后置自增或后置自减。++x 与 x++ 有一个微妙的区别。看看下面的语句序列：

```
x = 4;
y = x++;
```

执行后，x 为 5，y 为 4。也就是说，后置形式的表达式值取自自增之前：x++ 得到旧值 4，这个值赋给 y，而 x 本身增加为 5。

下面用前置自增作对照：

```
x = 4;
y = ++x;
```

++x 得到变量自增之后的值，因此 x 和 y 最后都为 5。

不过，C 和 C++ 的一种常见编码风格，是尽量避免写出严重依赖前置、后置差别的代码。阅读和继续开发代码的人可能不容易立即看出这种用法的区别，所以应尽量避免。多数程序员倾向于在代码中一致地采用一种形式。

表 12.5 C 中运算符的优先级与结合性

优先级	结合性	运算符
1 (最高)	从左向右	() (函数调用)、[] (数组下标)、. (结构体成员)、-> (通过结构体指针访问成员)、++ -- (后置)
2	从右向左	++ -- (前置)、* (间接访问)、& (取地址)、+ - (一元)、~ (逐位 NOT)、! (逻辑 NOT)、sizeof、(type) (类型转换)
3	从左向右	* (乘法)、/ (除法)、% (整数余数)
4	从左向右	+ (加法)、- (减法)
5	从左向右	<< (左移)、>> (右移)
6	从左向右	<、>、<=、>= (关系比较)
7	从左向右	== (相等)、!= (不等)
8	从左向右	& (逐位 AND)
9	从左向右	^ (逐位 XOR)
10	从左向右	(逐位 OR)
11	从左向右	&& (逻辑 AND)
12	从左向右	(逻辑 OR)
13	从右向左	? : (条件表达式)
14	从右向左	=、+=、-=、*= 等赋值运算符
15 (最低)	从左向右	, (逗号运算符, 译注补列)

译注：原表分为 17 组，把后置自增、自减与其他后缀运算符分开，又将前置、一元和类型转换分列。这里按通常的 C 优先级表合并为相应层级，并补列原表遗漏的逗号运算符。原表把后置形式写成从右向左结合，把条件运算符写成 & : 且从左向右结合，还把 XOR 符号写成 &&、将 % 解释为整数除法；以上均已修正。表格是对语法规则的简表，优先级不等于运行时操作数的求值先后。

12.3.9 包含多个运算符的表达式

目前的示例大多只有一两个运算符，实际使用的表达式有时会更多。可以把多种运算符和操作数组合成复杂表达式，例如：

```
y = x & z + 3 || 9 - w % 6;
```

要理解这条语句，需要考察运算符的优先关系。表 12.5 包含了 C 运算符，其中一些尚未介绍、将在后续章节讨论。按照优先级规则，上式等价于：

```
y = (x & (z + 3)) || (9 - (w % 6));
```

下面是更实用的多运算符表达式。如果 age 在 18 到 25 之间，表达式为 1，否则为 0。虽然不加括号也能得到所述结果，但括号让代码更易阅读。

```
(18 <= age) && (age <= 25)
```

12.4 用运算符解决问题

现在用已学内容解决一个编程问题。我们要编写一个简单的网络计算程序：已知要传输的字节数和网络传输速率，以字节每秒为单位，计算传输所需时间。特别之处在于，时间必须以小时、分钟、秒的形式显示。

我们采用第 6 章介绍的分解技术：从方法的高层描述开始，用顺序、选择和迭代结构不断细化，直到可以方便地写成 C 代码。如有需要，可以回顾第 6 章。这称为自顶向下分解，因为我们从算法的高层描述出发，把大步骤拆成小步骤，最终得到易于转换为 C 代码的形式。

分解之前，先考虑如何表示程序要操作的数据。目前可选的基本 C 类型有整数、字符、浮点数和布尔值。既然进行的是算术计算，最自然的候选就是浮点数和整数。最终要算的是小时数、分钟数和秒数，因此不需要时间的小数部分。将总时间显示为“10.1 小时、12.7 分钟、9.3 秒”并不合适，更希望看到“10 小时、18 分钟、51 秒”。因此，这次时间计算选择整数更好。确实存在取整问题，但假设本次计算可以忽略它。

选好数据表示后，就可以用逐步求精将问题分解为 C 代码。图 12.3 展示了这一过程。Step 1 是问题的初始描述，分为三阶段：获取输入、计算结果、输出结果。第一阶段询问用户要传输多少字节，以及网络每秒能传输多少字节；第二阶段完成所需计算；第三阶段输出结果。

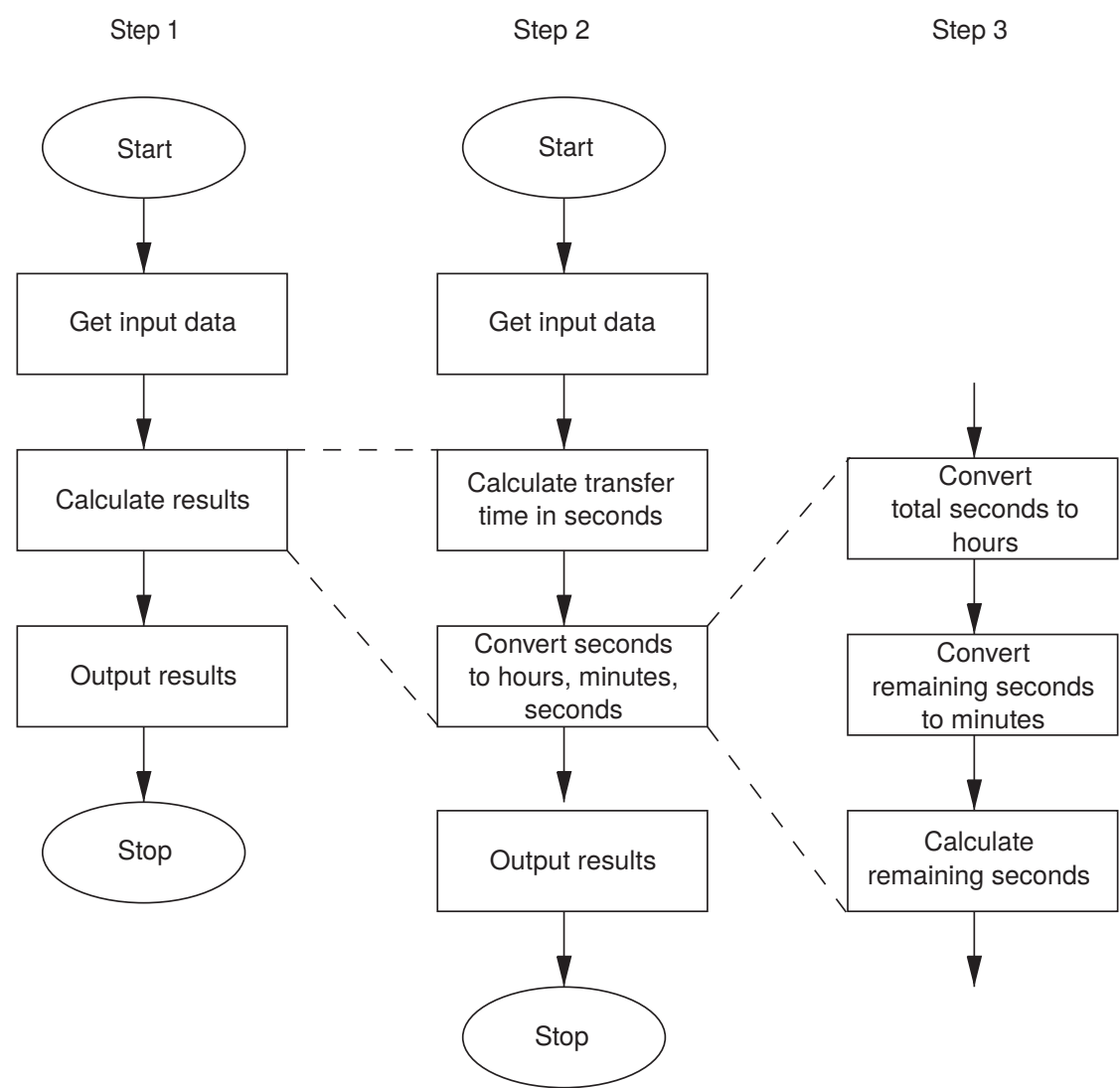


图 12.3 对简单网络传输时间计算问题进行逐步求精

Step 1 还不够具体，不能直接转换为 C 代码，因此在 Step 2 再细化一次。计算阶段可以分为两个子阶段：先算出以秒为单位的总时间，这由输入数据很容易得到；再把总秒数转换为小时、分钟和秒。Step 2 仍不足以完全映射为 C，所以在 Step 3 再次细化。Step 2 的大部分阶段已经够简单，只有时间转换还需要分解。Step 3 将其分成三个子阶段：先根据总秒数计算小时数，再用剩余秒数计算分钟数，最后求出扣除整分钟之后剩余的秒数。从 Step 3 到 C 代码，就是很直接的转换了。

图 12.4 给出了完整 C 程序。


```
1  #include <stdio.h>
2  int main(void)
3  {
4      int amount;          // The number of bytes to be transferred
5      int rate;            // The average network transfer rate
6      int time;            // The time, in seconds, for the transfer
7      int hours;          // The number of hours for the transfer
8      int minutes;        // The number of mins for the transfer
9      int seconds;        // The number of secs for the transfer
10
11     // Get input: number of bytes and network transfer rate
12     printf("How many bytes of data to be transferred? ");
13     scanf("%d", &amount);
14     printf("What is the transfer rate (in bytes/sec)? ");
15     scanf("%d", &rate);
16
17     // Calculate total time in seconds
18     time = amount / rate;
19
20     // Convert time into hours, minutes, seconds
21     hours = time / 3600;          // 3600 seconds in an hour
22     minutes = (time % 3600) / 60; // 60 seconds in a minute
23     seconds = ((time % 3600) % 60); // remainder is seconds
24
25     // Output results
26     printf("Time : %dh %dm %ds\n", hours, minutes, seconds);
27 }
```

图 12.4 进行简单网络传输时间计算的 C 程序

12.5 把各个概念联系起来

至此，我们已介绍 C 的基本类型，以及本书将使用的大多数运算符。变量是需要存储空间的存储器对象，运算符表达对这些对象所做的计算。C 编译器必须为对象，在这里具体就是变量，正确分配空间，并生成与 C 语句相应的 LC-3 指令序列。要在各种情况下正确完成转换，就必须系统地管理对象及其位置。编译器用符号表跟踪代码中声明的对象，并借助存储器映射有条理地分配它们。下面仔细看看。

12.5.1 符号表

第 7 章介绍过汇编器如何用符号表系统地跟踪汇编程序中的标号。与 LC-3 汇编器一样，C 编译器也用符号表跟踪程序中的变量。每读到一条变量声明，编译器就在符号表中为所声明的变量建立一条记录。记录包含足够的信息，以便编译器管理存储

分配，并在程序使用变量时生成正确的机器代码序列。现在，假设每条记录包含：(1) 标识符；(2) 类型；(3) 所分配的存储位置；(4) 变量的作用域。对这里的讨论而言，作用域可以用变量在代码中可见的行号范围描述。

图 12.5 给出了图 12.4 网络传输计算程序中各变量的符号表项。程序有六条变量声明，因此编译器最终建立六条记录，每个变量一条。注意，编译器用偏移量记录变量的存储位置，而且大多数偏移量为负。偏移量表示变量在所分配存储区域中的相对位置。这是重要概念，下面进一步解释。

Identifier	Type	Location (as an offset)	Scope	Other info...
amount	int	0	main	...
hours	int	-3	main	...
minutes	int	-4	main	...
rate	int	-1	main	...
seconds	int	-5	main	...
time	int	-2	main	...

图 12.5 编译图 12.4 代码时编译器建立的符号表

12.5.2 为变量分配空间

C 中声明的变量主要在两个区域分配存储空间：**全局数据区**（global data section）和**运行时栈**（run-time stack）。⁸全局变量放在全局数据区，局部变量放在运行时栈。符号表中的偏移量字段，让编译器能精确定位这两个区域中的变量。它表示变量相对于区域基址隔了多少个存储器位置。例如，全局变量 earth 的偏移量为 4，全局数据区从 0x5000 开始，则 earth 存放在 0x5004。如果 R4 保存全局数据区的起始地址，可以用下面的 LC-3 指令将它加载到 R3：

```
LDR R3, R4, #4
```

如果 earth 是局部变量，例如属于 main，它就位于运行时栈中。栈上的分配单位称为**活动记录**（activation record），也称**栈帧**（stack frame）。函数开始执行时，它的栈帧被加入运行时栈，具体过程将在第 14 章说明。栈帧是一片连续的存储器位置，其中包含该函数的全部局部变量。

某个函数执行时，R5 保存其局部变量区域中编号最高的地址，R5 称为**帧指针**（frame pointer）。例如，图 12.6 展示了图 12.4 中 main 的栈帧。按图中从低地址到

⁸原注：本书示例为所有变量分配存储器位置。但实际编译器会进行优化，尝试把变量放在寄存器中。访问寄存器比访问存储器快，因此将频繁访问的值放入寄存器可使程序运行得更快。

高地址的顺序看，变量排列与声明顺序相反：amount 最先声明，因此最靠近帧指针 R5。⁹

引用某个局部变量时，编译器依据其符号表项生成访问代码。表项中的偏移量指出变量在栈中的位置。例如，访问 seconds 时，可生成：

```
LDR R0, R5, #-5
```

先预告后面的内容：每次在 C 中调用函数，即 C 中的子程序时，函数的栈帧被压入运行时栈，也就是在栈顶分配。R5 随之调整，指向活动记录中约定的基准位置。因此，函数中访问局部变量的代码可以继续使用符号表中记录的偏移量。函数完成、即将把控制权交回调用者时，它的活动记录被弹出，R5 调整为指向调用者的活动记录。在整个过程中，R6 始终保存运行时栈栈顶的地址，称为**栈指针**（stack pointer）。第 14 章将更详细地介绍这一过程。

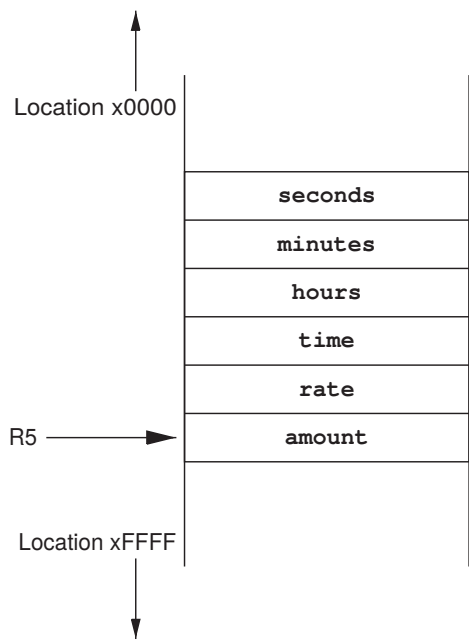


图 12.6 图 12.4 中 main 函数的栈帧示例。该函数有六个局部变量。R5 是帧指针，指向第一个局部变量。

译注：原图题称“五个局部变量”，但程序及图中实际列出六个，已更正图题。

图 12.7 展示了程序运行时 LC-3 存储器的组织。程序本身占据一个区域，图中标为 Program text；运行时栈和全局数据区也各占一区。此外，还有为动态分配数据保留的区域，称为**堆**（heap），将在第 19 章讨论。运行过程中，栈和堆都可能改变大小。例如，一个函数调用另一个函数时，会向栈中再压入一个活动记录，因此运行时栈增长，具体来说是向 x0000 的方向增长。堆则相反，向 0xFFFF 的方向增长。由于栈向 x0000 增长，活动记录看起来是“倒着”的：第一个局部变量在 R5 指向的位置，下一个在 R5-1，再下一个在 R5-2，而不是 R5、R5+1、R5+2。

⁹译注：本章只展示栈帧的局部变量部分。完整活动记录还可包含返回地址、参数等；不能将 R5 理解为整个完整栈帧中最高地址的通用定义。

执行过程中，PC 指向程序代码区的某个位置，R4 指向全局数据区的起始位置，R5 指向运行时栈内部，R6 指向栈顶。图 12.7 中标为 System space 的某些区域保留给操作系统，用于 TRAP 例程、向量表、I/O 寄存器和引导代码等。

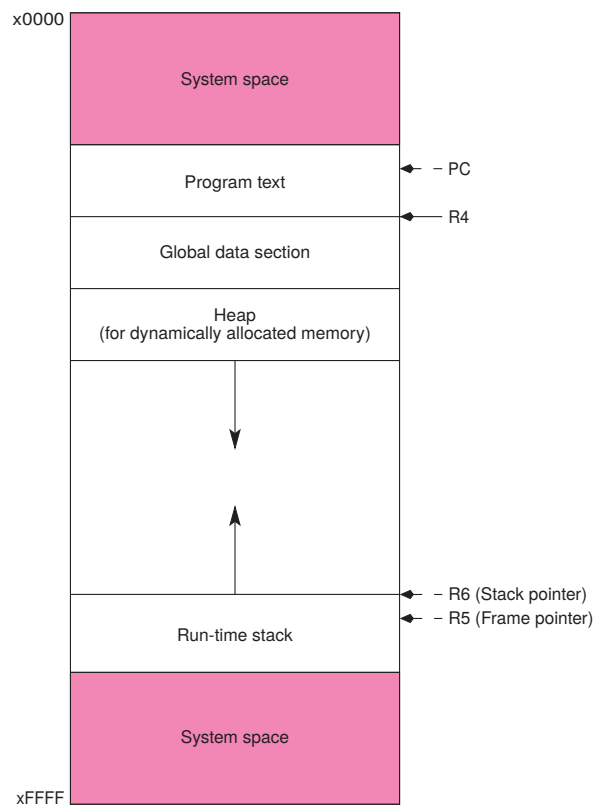


图 12.7 LC-3 存储器映射，展示程序执行期间使用的各个区域

12.5.3 一个综合实例

现在结合一段 C 代码及其 LC-3 译码，综合考察本章各概念之间的联系。图 12.8 给出一个完整 C 程序，对整数变量做一些简单操作，再输出结果。它包含一个全局变量 inGlobal 和三个属于 main 的局部变量：inLocal、outLocalA、outLocalB。

程序先为 inLocal 和 inGlobal 设置初值，其中 inLocal 在声明中初始化。随后，利用这两个输入变量进行两次计算，将结果分别赋给 outLocalA 和 outLocalB。计算完成后，用库函数 printf 输出两个结果。由于使用了 printf，必须包含标准 I/O 库头文件 stdio.h。

```
1  #include <stdio.h>
2
3  int inGlobal;    // inGlobal is a global variable.
4                  // It is declared outside of all blocks
5
6  int main(void)
7  {
8      int inLocal = 5;    // inLocal, outLocalA, outLocalB are all
9      int outLocalA;      // local to main
10     int outLocalB;
11
12     // Initialize
13     inGlobal = 3;
14
15     // Perform calculations
16     outLocalA = inLocal & ~inGlobal;
17     outLocalB = (inLocal + inGlobal) - (inLocal - inGlobal);
18
19     // Print results
20     printf("outLocalA = %d, outLocalB = %d\n", outLocalA, outLocalB);
21 }
```

图 12.8 执行简单运算的 C 程序。输出为 “outLocalA = 4, outLocalB = 6”。

分析代码时，LC-3 C 编译器将全局变量 inGlobal 分配到全局数据区中第一个可用位置，即偏移量 0。分析 main 时，将 inLocal 分配到栈帧内偏移量 0，outLocalA 分配到 -1，outLocalB 分配到 -2。¹⁰相应的符号表快照和 main 活动记录见图 12.9。编译器生成的汇编代码见图 12.10，执行从标号 main 开始。

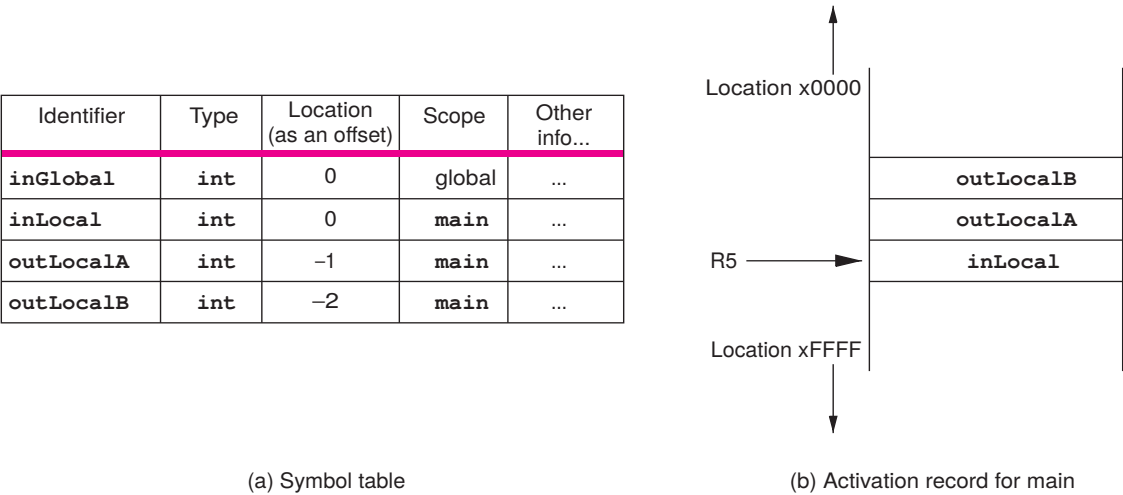


图 12.9 LC-3 C 编译器编译图 12.8 时的符号表，以及 main 的栈帧格式。(a) 符号表；(b) main 的活动记录

¹⁰译注：原文将此处第一个局部变量误写为 inLocalA，已按代码和符号表更正为 inLocal。

```

1 main:
2 :
3 :
4 <startup code>
5 :
6 :
7 AND R0, R0, #0
8 ADD R0, R0, #5      ; inLocal is at offset 0
9 STR R0, R5, #0      ; inLocal = 5;
10
11 AND R0, R0, #0
12 ADD R0, R0, #3      ; inGlobal is at offset 0, in globals
13 STR R0, R4, #0      ; inGlobal = 3;
14
15 LDR R0, R5, #0      ; get value of inLocal
16 LDR R1, R4, #0      ; get value of inGlobal
17 NOT R1, R1          ; ~inGlobal
18 AND R2, R0, R1      ; calculate inLocal & ~inGlobal
19 STR R2, R5, #-1     ; outLocalA = inLocal & ~inGlobal;
20                      ; outLocalA is at offset -1
21
22 LDR R0, R5, #0      ; get value of inLocal
23 LDR R1, R4, #0      ; get value of inGlobal
24 ADD R0, R0, R1      ; calculate inLocal + inGlobal
25
26 LDR R2, R5, #0      ; get value of inLocal
27 LDR R3, R4, #0      ; get value of inGlobal
28 NOT R3
29 ADD R3, R3, #1      ; calculate -inGlobal
30
31 ADD R2, R2, R3      ; calculate inLocal - inGlobal
32 NOT R2
33 ADD R2, R2, #1      ; calculate -(inLocal - inGlobal)
34
35 ADD R0, R0, R2      ; (inLocal + inGlobal) - (inLocal - inGlobal)
36 STR R0, R5, #-2     ; outLocalB = ...
37                      ; outLocalB is at offset -2
38 :
39 :
40 <code for calling the function printf>
41 :
42 :

```

图 12.10 图 12.8 中 C 程序对应的 LC-3 代码

译注：原图第 28、32 行分别写作 *NOT R3*、*NOT R2*，省略了源寄存器。按 *LC-3* 的双操作数语法，应分别写为 *NOT R3, R3* 和 *NOT R2, R2*。原图保留作者的省略写法。

12.6 补充主题

本章最后一个主要部分补充一些与变量、运算符有关的主题。其中有些是前面概念的进一步展开，有些是 C 的其他特性。这些内容能让常用 C 特性的介绍更完整，但不是理解后续章节的必要前提。如果你希望更全面地了解 C 的变量和运算符，请继续阅读。

12.6.1 基本类型的变体

C 允许程序员指定基本数值类型的较大或较小变体。修饰符 `long` 和 `short` 可用于 `int`，表达扩展或缩短默认大小的意图。例如，某个实现中的 `long int` 可以具有普通 `int` 两倍的位数，从而表示更大的整数范围。`long` 也可用于 `double`，在系统支持时得到范围或精度更大的浮点类型。

`short` 可用来创建比默认整数类型更小的变量。数据不需要默认类型的完整范围时，这有助于节省存储空间。声明方式如下：

```
long double particlesInUniverse;
long int worldPopulation;
short int ageOfStudent;
```

基本 C 类型的大小与底层 ISA 所支持的类型密切相关，因此实际位数取决于编译器和目标系统。即使声明为 `long int`，在某些实现上也可能与普通 `int` 一样大。¹¹

`int` 的另一种有用变体是无符号整数，可用 `unsigned` 修饰符声明。它用全部位表示非负整数，也就是正数和零。例如，本书 LC-3 实现中的 16 位无符号整数取值为 0 至 65,535。处理本来就不会出现负值的现实对象时，无符号整数可能是合适的选择。

```
unsigned int numberOfDays;
unsigned int populationSize;
```

下面再列出几种基本算术类型的变体：

```
long int ounces;
short int gallons;
long double veryVeryLargeNumber = 4.12936E361;
unsigned int sizeOfClass = 900;
float tonsOfGrain = 2.998E8;
```

译注：原例的 `4.12936E361` 没有 `L` 后缀，因此字面量本身是 `double`；它超出了常见 IEEE 754 双精度的范围。若实现支持足够大的 `long double`，通常应写为 `4.12936E361L`。本例保留原代码，不能据此假设每个编译器都能接受该常量。

¹¹译注：原文把长短类型的支持与 ISA 的原生支持直接联系起来。更准确地说，C 实现必须满足标准规定的最小范围和相对大小要求，也可以用多条指令实现硬件不直接支持的宽度；`long int` 不保证恰为 `int` 的两倍。

12.6.2 字面量、常量与符号值

在 C 中，可以在类型说明符前加上 `const` 限定符，将所声明的对象限定为常量。它们是只读的，程序运行期间不能通过这些对象修改其值。例如，计算给定半径圆的面积和周长时，可以创建名为 `pi` 的浮点常量，并初始化为 3.14159。图 12.11 给出了这样的程序。

```
1  #include <stdio.h>
2
3  #define RADIUS 15.0                // This value is in centimeters
4
5  int main(void)
6  {
7      const double pi = 3.14159;
8      double area;
9      double circumference;
10
11     // Calculations
12     area = pi * RADIUS * RADIUS;    // area = pi*r^2
13
14     circumference = 2 * pi * RADIUS; // circumference = 2*pi*r
15
16     printf("Area of circle with radius %f cm is %f cm^2\n", RADIUS, area);
17     printf("Circumference of the circle is %f cm\n", circumference);
18 }
```

图 12.11 计算半径为 15 cm 的圆的面积和周长的 C 程序

这个例子有助于区分 C 代码中常见的三类常量值。第一类是**字面常量**，即直接写在源代码中的无名值；例子中的 2 和 3.14159 就属于这一类。整数前加 0x 可表示十六进制，如 0x1DB。ASCII 字符字面量用单引号括起来，例如 'R' 表示字符 R 的 ASCII 值。浮点字面量可以采用 12.2.1 节介绍的指数形式。

第二类是 `pi` 这样的值，通过带 `const` 限定符的声明定义。第三类则由 `#define` 预处理指令创建，例如符号值 `RADIUS`。这三种方式都可以表达程序一次运行期间不变的值。

`const` 声明的常量与 `#define` 定义的符号值之间，差别也许显得有些微妙。在这里的简单用途下，选择往往体现编程风格。我们习惯把永远不变的值用声明的常量表示，例如 `pi`；光速等物理常量，或一周的天数，也通常这样表示。¹²

一次运行中保持不变、但可能因用户不同或运行配置不同而变化的值，则可用 `#define` 表示，可视为程序的参数。例如，可以修改图 12.11 中的 `RADIUS`，重新编译，再次执行。总的来说，用 `const` 或 `#define` 为常量命名，通常比在代码各处直接写字面值更好，因为名称能传达更多含义。

¹²译注：两者在语言层面并不等价。`const` 对象具有类型和作用域，`#define` 执行预处理宏替换；在需要整型常量表达式等场合，其适用性也可能不同。

12.6.3 其他 C 运算符

C 还提供一些不太寻常的运算符，其中某些已融入常见的 C 编程风格。它们在现有 C 和 C++ 代码中经常出现，值得了解。大多数是已学运算符的组合，可以简化常见计算的表达。

12.6.3.1 赋值运算符

C 允许把算术或位运算符与赋值运算符组合，构成“运算并赋值”的操作。例如，要将变量 x 增加 29，可用简写 +=：

```
x += 29;
```

它等价于：

```
x = x + 29;
```

更多例子如下：

```
h += g; // Equivalent to h = h + g;
h %= f; // Equivalent to h = h % f;
h <<= 3; // Equivalent to h = h << 3;
```

表 12.6 列出 C 提供的一些特殊运算符。

表 12.6 C 中的特殊运算符

运算符	运算	用法示例
+=	相加并赋值	x += y
-=	相减并赋值	x -= y
*=	相乘并赋值	x *= y
/=	相除并赋值	x /= y
%=, &=, =, ^= 等	其他运算并赋值	x &= y

12.6.3.2 条件表达式

条件表达式让我们用一个简单表达式完成简单的选择。它使用问号和冒号？：。例如：

```
x = a ? b : c;
```

这里，根据 a 的逻辑值，x 得到 b 或 c 的值。如果 a 非零，就得到 b；否则得到 c。

因此，要将 valueA、valueB 中较大的值赋给 maxValue，可以写成：

```
maxValue = (valueA > valueB) ? valueA : valueB;
```

12.7 小结

本章的三个关键概念如下。

- **C 中的变量。**C 支持四种基本类型的变量：整数 `int`、字符 `char`、浮点数 `float` 和 `double`，以及布尔值 `_Bool` 或 `bool`。与其他高级语言一样，C 允许程序员用符号名称引用变量。变量可以在某个块，例如函数中局部声明，也可以作为全局变量供各块访问。
- **C 中的运算符。**可以按功能将运算符分成赋值、算术、位操作、逻辑和关系检测等类别。变量和运算符组成表达式，按优先级和结合性规则确定分组并求值。表达式进一步组成语句，表达程序要完成的工作。
- **将 C 变量和运算符转换为 LC-3 代码。**编译器用符号表跟踪变量声明，并在函数的栈帧内分配局部变量。执行函数时，其栈帧，也称活动记录，被压入运行时栈。程序中的全局变量则分配在全局数据区。

习题

12.1 为下面代码生成编译器符号表。假设每个变量都占一个存储器单元。

```
{
    double ff;
    char cc;
    int ii;
    char dd;
    bool zz;
}
```

12.2 程序中出现以下变量声明：

```
int r;
```

- 如果 `r` 是局部变量，它的初值是什么？
- 如果 `r` 是全局变量，它的初值是什么？

12.3 如果下面两个变量都用 32 位存储，它们各自的取值范围是什么？

```
int plusOrMinus;
unsigned int positive;
```

12.4 求出以下浮点字面量的值，用通常的十进制记法表示。

- `111E-11`
- `-0.00021E4`
- `101.101E0`

12.5 写出以下局部变量声明经 LC-3 C 编译器编译后得到的 LC-3 代码。

```
char c = 'a';
int x = 3;
int y;
int z = 10;
```

12.6 指出下面代码中每条 printf 输出的值。语句按 A、B、C、D 的顺序执行。

```
int t; // This variable is global
{
    int t = 2;
    printf("%d\n", t);    // A
    {
        printf("%d\n", t);    // B
        t = 3;
    }
    printf("%d\n", t);    // C
}
{
    printf("%d\n", t);    // D
}
```

12.7 整数 a 和 b 的值分别为 6 和 9。下面各表达式的值是什么？如果 a 或 b 改变，还要给出其新值。

- a. a | b
- b. a || b
- c. a & b
- d. a && b
- e. !(a + b)
- f. a % b
- g. b / a
- h. a = b
- i. a = b = 5
- j. ++a + b--
- k. a = (++b < 3) ? a : b
- l. a <= b

12.8 对字符变量 letter，分别写出完成以下检测的 C 表达式。

- a. 检测它是否为任意英文字母或数字字符。
- b. 检测它是否为英文字母和数字字符以外的字符。

12.9

- a. 下面语句完成什么任务？letter 是字符变量。

```
letter = ((letter >= 'a' && letter <= 'z') ? '!' :
↪ letter);
```

b. 修改 a 项中的语句, 使其将小写字母转换为大写字母。

12.10 编写程序, 从键盘读入一个整数; 如果它能被 3 整除, 就显示 1, 否则显示 0。

12.11 解释以下 C 语句之间的区别。

a. `j = i++;`

b. `j = ++i;`

c. `j = i + 1;`

d. `i += 1;`

e. `j = i += 1;`

f. 哪些语句修改 i? 哪些修改 j? 如果初始时 `i = 1`、`j = 0`, 每条语句分别单独执行后, i 和 j 各是多少?

12.12 假设变量 a 和 b 都在局部声明为 `long int`。

a. 将表达式 `a + b` 转换为 LC-3 代码, 假设一个整数占两个字节。a 分配在所属函数活动记录的偏移量 0 处, b 在偏移量 -1 处。

b. 再次转换该表达式, 这次假设一个整数占四个字节, a 在偏移量 0 处, b 在偏移量 -2 处。

译注: 题干先声明 `long int`, 分问却简称 `int`。这里按分问明确给出的存储宽度和偏移量理解, 不额外假设 `long int` 必为普通 `int` 的两倍。

12.13 如果初始时 `a = 1`、`b = 1`、`c = 3`、`result = 999`, 执行下面语句后, 各变量的值是多少?

```
result = b + 1 | c + a;
```

12.14 回顾第 2 章的机器忙闲示例。假设整数变量 `machineBusy` 跟踪全部 16 台机器的忙闲状态。某位为 0 表示对应机器忙, 为 1 表示空闲。

a. 写一条 C 语句, 将机器 5 标为忙。

b. 写一条 C 语句, 将机器 10 标为空闲。

c. 写一条 C 语句, 将机器 n 标为忙; 整数变量 n 给出变忙的机器编号。

d. 写一个 C 表达式, 检测机器 3 是否空闲。空闲时得到 1, 忙时得到 0。

e. 写一个 C 表达式, 求空闲机器的数量。例如, `machineBusy` 中的位模式为 1011 0010 1110 1001 时, 表达式应得到 9。

12.15 分号在 C 中有什么作用?

12.16 假设正在设计一种新的编程语言, 其中有运算符 @、#、\$ 和 U。在下面各组约束下, 表达式 `w @ x # y $ z U a` 应如何分组求值?

a. 优先级由高到低为 @、#、\$、U。用括号表示次序。

b. 优先级由高到低为 #、U、@、\$。

- c. 优先级相同，从左向右结合。
- d. 优先级相同，从右向左结合。

12.17 注意，赋值运算符在原书所列 C 运算符中优先级最低。假设我们设计了一种新语言 Q，除赋值运算符的优先级最高以外，其余都与 C 相同。

- a. 下面这条 Q 语句产生什么结果？也就是说，执行后 x 的值是什么？

$x = x + 1;$

- b. 怎样修改这条 Q 语句，使其作用与原语句在 C 中的作用相同？

12.18 修改第 11 章图 11.3 的示例程序：提示用户输入一个字符，然后按照 ASCII 表中的次序，从该字符向前一直输出到字符！。

12.19 编写 C 程序，计算购物交易的销售税。提示用户输入购买金额和税率，输出销售税额，以及包含税款的购买总金额。

12.20 假设程序中有两个整数变量 x 和 y ，值分别为 3 和 4。编写 C 语句交换它们的值，使执行后 x 为 4、 y 为 3。

- a. 先使用一个临时变量完成交换。
- b. 再改写为不使用临时变量的版本。

第十三章 控制结构

13.1 引言

第 6 章介绍了自顶向下的问题求解方法：利用顺序结构、条件结构和迭代结构这三种程序设计结构，将一个问题系统地细化为更小、更具体的子任务。

第 12 章运用这一方法，推导出一个计算网络传输时间的简单 C 程序。把那个问题细化为程序时，只需要使用顺序结构。为了将更复杂的问题转化为 C 代码，我们还需要在 C 中构造条件结构和迭代结构。本章介绍这两种结构在 C 中的形式。

首先介绍 C 的条件结构。if 和 if-else 语句允许我们有条件地执行一条语句。随后介绍 C 的迭代结构：for、while 和 do-while 语句，它们都可以表达循环。对于其中的多种结构，我们还会给出假想的 LC-3 C 编译器生成的相应 LC-3 代码，帮助理解这些结构在较低层次上的行为。C 还提供 switch、break 和 continue 等控制结构，便于表达某些特定的控制任务；第 13.5 节讨论这些结构。本章还将运用自顶向下的问题求解方法，解决一些需要使用控制结构的较复杂编程问题。

13.2 条件结构

条件结构使程序员能够根据某个条件选择要执行的动作。这是一种很常见的程序设计惯用法，每一种实用的程序设计语言都提供了方便的表达方式。C 提供两种基本的条件结构：if 和 if-else。

13.2.1 if 语句

if 语句很简单：条件为真时，就执行一个动作。动作是一条 C 语句；条件是一个 C 表达式，只有它的求值结果非零，即逻辑上为真时，才执行这个动作。来看一个例子。

```
if (x <= 10)
    y = x * x + 5;
```

只有当表达式 $x \leq 10$ 的值非零、即逻辑上为真时，才会执行 $y = x * x + 5$ 。回忆一下小于或等于运算符 $\leq$ ：关系成立时，其值为 1；否则为 0。条件后面的语句也可以是**复合语句**，或称**块**，即以左花括号开始、以右花括号结束的一系列语句。复合语句将一条或多条简单语句组合成一个整体，这个整体本身也可以作为一条语句使用。将复合语句与 if 配合，就能根据一个条件决定是否执行多条语句。例如，下面的代码在 x 小于或等于 10 时，同时修改 y 和 z 。

```
if (x <= 10) {  
    y = x * x + 5;  
    z = (2 * y) / 3;  
}
```

与 C 的所有语句一样，if 语句的书写格式很灵活。上例中的换行和缩进是一种常用风格，能让阅读者迅速看出条件成立时执行哪部分代码。但要记住：缩进不会影响程序行为，哪怕我们希望它能起作用。下面的代码采用了类似的缩进，却具有不同的行为。第二条语句 $z = (2 * y) / 3;$ 不属于 if 所控制的语句，无论条件是否成立，它都会执行。

```
if (x <= 10)  
    y = x * x + 5;  
    z = (2 * y) / 3;
```

图 13.1 给出了 if 语句的控制流，对应于下面的代码。

```
if (condition)  
    action;
```

从语法上看，条件必须用圆括号括起来，使编译器能够明确区分条件和 if 语句的其余部分。动作必须是一条简单语句或复合语句。

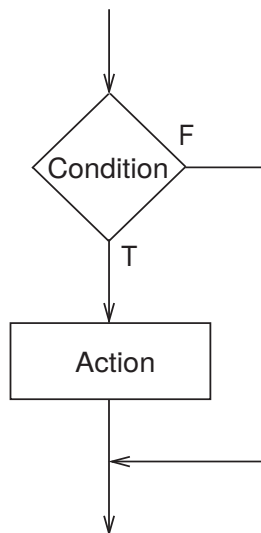


图 13.1 C 的 if 语句的图示

下面是更多 if 语句示例，展示这种判定结构可以用于哪些程序设计情境。

```

if (temperature <= 0)
    printf("At or below freezing point.\n");

if ('a' <= key && key <= 'z')
    numLowerCase++;

if (current > currentLimit)
    blownFuse = 1;

if (loadMAR & clock)
    registerMAR = bus;

if (month==4 || month==6 || month==9 || month==11)
    printf("The month has 30 days\n");

if (x = 2) // This condition is always true. Why?
    y = 5; // The variable y will always be 5

```

最后一个例子展示了 C 编程中常见的错误：条件中使用了赋值运算符 =，而不是相等运算符 ==，因而把 x 改成了 2。这个条件始终为真：包含赋值运算符的表达式，其求值结果就是被赋的值，这里为 2。因为条件始终非零，y 总会被赋为 5，而 x 总会被赋为 2。

下面的代码乍看相似，却是对前一段代码的“修复”。

```

if (x == 2)
    y = 5;

```

来看看为这段代码生成的 LC-3 代码。假设 x 和 y 都是局部声明的整数，那么 R5 指向变量 x，R5+1 指向 y。

```

LDR R0, R5, #0      ; load x into R0
ADD R0, R0, #-2     ; subtract 2 from x
BRnp NOT_TRUE       ; If condition is not true,
                    ; then skip the assignment

AND R0, R0, #0      ; R0 <- 0
ADD R0, R0, #5      ; R0 <- 5
STR R0, R5, #-1     ; y = 5;
NOT_TRUE            ; the rest of the program
:
:

```

注意，对 LC-3 C 编译器来说，最直接的做法是生成代码，测试原条件的反面，即 x 不等于 2，再根据测试结果进行分支。

if 语句本身也是一条语句，所以可以在一条 if 语句中嵌套另一条 if 语句，如下所示。第一个 if 后面只有一条语句，即内层的 if，因此外层不需要花括号。


```
if (x == 3)
    if (y != 6) {
        z = z + 1;
        w = w + 2;
    }
```

只有 x 等于 3 时，才会执行内层 `if`。还有一种更简单的写法：用一个 `if` 就能表达同样的行为。

```
if ((x == 3) && (y != 6)) {
    z = z + 1;
    w = w + 2;
}
```

13.2.2 if-else 语句

如果希望条件为真时执行一组动作、同一条件为假时执行另一组动作，可以使用下面这一组 `if` 语句。

```
if (temperature <= 0)
    printf("At or below freezing point.\n");
if (temperature > 0)
    printf("Above freezing.\n");
```

根据变量 `temperature` 是小于或等于零，还是大于零，这段代码会打印其中一条消息。这种条件执行方式在程序设计中非常有用，C 通过 `if-else` 语句直接支持它。下面的代码与前一段等价。紧跟在 `else` 关键字之后的语句，只在条件为假时执行。

```
if (temperature <= 0)
    printf("At or below freezing point.\n");
else
    printf("Above freezing.\n");
```

图 13.2 给出了 `if-else` 的流程图，对应代码如下。

```
if (condition)
    action_if;
else
    action_else;
```

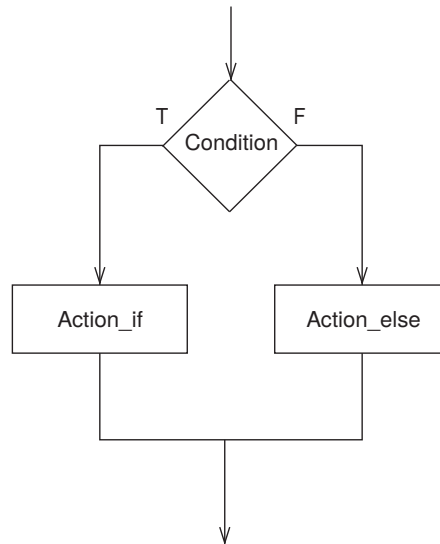


图 13.2 C 的 if-else 语句的图示

action_if 和 action_else 都可以是复合语句，因此可以包含多条语句，如下例所示。

```
if (x) {  
    y++;  
    z--;  
}  
else {  
    y--;  
    z++;  
}
```

如果 x 非零，if 的条件为真， y 自增、 z 自减；否则， y 自减、 z 自增。图 13.3 给出了 LC-3 C 编译器生成的代码。这里的 x 、 y 和 z 都是局部声明的整数。

```

1      LDR R0, R5, #0      ; load the value of x
2      BRz ELSE           ; if x equals 0, perform else part
3
4      LDR R0, R5, #-1     ; load y into R0
5      ADD R0, R0, #1
6      STR R0, R5, #-1     ; y++;
7
8      LDR R0, R5, #-2     ; load z into R0
9      ADD R0, R0, #-1
10     STR R0, R5, #-2     ; z--;
11     BR DONE
12
13 ELSE: LDR R0, R5, #-1   ; load y into R0
14     ADD R0, R0, #-1
15     STR R0, R5, #-1     ; y--;
16
17     LDR R0, R5, #-2     ; load z into R0
18     ADD R0, R0, #1
19     STR R0, R5, #-2     ; z++;
20 DONE:
21     :
```

图 13.3 为一条 if-else 语句生成的 LC-3 代码

我们可以把多个条件结构连接起来，构成更长的条件测试序列。图 13.4 只使用 if 和 if-else，没有使用其他控制结构，就构造出了较复杂的判定结构。这个程序要求用户输入月份编号，然后输出该月的天数。

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int month;
6
7      printf("Enter the number of the month: ");
8      scanf("%d", &month);
9
10     if (month==4 || month==6 || month==9 || month==11)
11         printf("The month has 30 days\n");
12     else if (month==1 || month==3 || month==5 ||
13             month==7 || month==8 || month==10 || month==12)
14         printf("The month has 31 days\n");
15     else if (month==2)
16         printf("The month has either 28 days or 29 days\n");
17     else
18         printf("Don't know that month\n");
19 }
```

图 13.4 确定指定月份有多少天的程序

这里需要明确 C/C++ 中 else 与 if 的配对规则：一个 **else** 与离它最近、且尚

未配对的 if 配对。下面的例子说明了这条规则为什么重要。

```
if (x != 10)
    if (y > 3)
        z = z / 2;
    else
        z = z * 2;
```

没有这条规则，就无法确定 else 应当与外层还是内层 if 配对。根据规则，这里的 else 与内层 if 配对，因为内层更近，而且尚未与其他 else 配对。代码等价于：

```
if (x != 10) {
    if (y > 3)
        z = z / 2;
    else
        z = z * 2;
}
```

正如圆括号可以改变表达式的求值次序，花括号可以确定语句如何组合。如果想让 else 与外层 if 配对，可以写成：

```
if (x != 10) {
    if (y > 3)
        z = z / 2;
}
else
    z = z * 2;
```

最后用图 13.5 的简单例子结束对 if 和 if-else 的讨论。这个例子展示了条件结构的一种常见用途。程序从键盘读入两个数并进行除法。由于除以 0 没有定义，如果用户输入的除数为 0，程序就显示一条消息，说明无法生成结果。if-else 很适合检查这种错误情况。注意，正常情况写在前面，错误情况写在后面。虽然两种顺序都可以实现，但把常见的正常情况写在前面，能向代码阅读者传达一个直观提示：错误情况是不常见的。

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int dividend;
6      int divisor;
7      int result;
8
9      printf("Enter the dividend: ");
10     scanf("%d", &dividend);
11
12     printf("Enter the divisor: ");
13     scanf("%d", &divisor);
14
15     if (divisor != 0) {
16         result = dividend / divisor;
17         printf("The result of the division is %d\n", result);
18     }
19     else
20         printf("A divisor of zero is not allowed\n");
21 }
```

图 13.5 包含错误检查代码的程序

13.3 迭代结构

计算机的强大能力，很大程度上来自通过迭代进行计算的能力。实用程序都会以某种形式进行迭代。与其他程序设计语言一样，C 和 C++ 直接提供了表达迭代计算的结构。在 C/C++ 中，有三种迭代结构：while、for 和 do-while，它们彼此之间略有不同。下面逐一考察。

13.3.1 while 语句

先介绍 C 中最简单的迭代语句。只要条件为真，while 循环就重复执行一条语句。每次迭代前都要检查条件。如果条件的值在逻辑上为真，即非零，就再执行一次语句。下例中，只要变量 x 小于 10，循环就继续迭代。程序的输出和代码如下。

```
0 1 2 3 4 5 6 7 8 9
```

```
#include <stdio.h>
int main(void)
{
    int x = 0;
    while (x < 10) {
        printf("%d ", x);
        x = x + 1;
    }
}
```

while 语句包含两个部分。测试条件 `test` 是一个表达式，用于确定是否继续执行循环，每次执行 `loop_body` 前都要测试它。`loop_body` 是一条语句，表示循环内要完成的工作；与其他语句一样，它也可以是一条复合语句。

```
while (test)
    loop_body;
```

图 13.6 使用系统化分解的记法表示控制流。实现这个结构需要两个分支：测试后通过一个条件分支退出循环；执行 `loop_body` 后，通过一个无条件分支回到 `test`，确定是否开始下一次迭代。图 13.7 给出了编译器为前面从 0 数到 9 的 `while` 示例生成的 LC-3 代码。

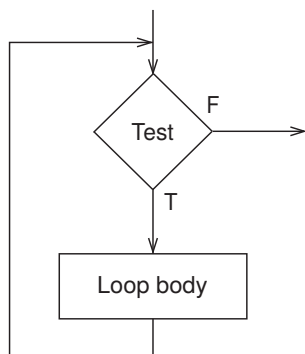


图 13.6 C 的 `while` 语句的图示

```

1          AND R0, R0, #0    ;   clear out R0
2          STR R0, R5, #0    ;   x = 0;
3
4          ; while (x < 10)
5  LOOP    LDR R0, R5, #0    ;   perform the test
6          ADD R0, R0, #-10
7          BRpz DONE        ;   x is not less than 10
8
9          ; loop body
10         :
11         <code for calling the function printf>
12         :
13         LDR R0, R5, #0    ;   R0 <- x
14         ADD R0, R0, #1    ;   x + 1
15         STR R0, R5, #0    ;   x = x + 1;
16         BR LOOP          ;   end of iteration
17  DONE    :
18

```

图 13.7 为一个数到 9 的 while 循环生成的 LC-3 代码

while 适合编写需要测试**哨兵条件**（sentinel condition）的循环。也就是说，我们事先不知道迭代次数，但希望一直循环，直到某个事件，即哨兵事件出现。例如，第 5 章和第 7 章的字符计数程序在检测到哨兵字符 EOT，即 ASCII 码为 4 的字符时终止循环。如果用 C 而不是 LC-3 汇编语言编写那个程序，就可以使用 while 循环。图 13.8 的程序使用 while 语句测试哨兵条件。¹

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      char echo = 'A'; // Initialize char variable echo
6
7      while (echo != '\n') {
8          scanf("%c", &echo);
9          printf("%c", echo);
10     }
11 }

```

图 13.8 另一个包含简单 while 循环的程序

最后指出一种与 while 循环有关的常见错误。下面的程序永远不会终止，因为

¹这个程序的行为可能与你预想的略有不同。你也许以为它会像 LC-3 版本一样，在用户每输入一个字符时立即打印该字符。但由于 C 处理键盘 I/O 的方式，用户按下 Enter 键之前，程序不会取得输入。第 18 章讨论 I/O 的底层问题时，将解释其原因。

循环体没有改变循环条件，条件会一直保持为真。这种循环称为**无限循环**，通常是程序员无意中引入的错误，当然也有例外。

```
#include <stdio.h>
int main()
{
    int x = 0;
    while (x < 10)
        printf("%d ", x);
}
```

13.3.2 for 语句

正如 while 循环很适合哨兵控制循环 (sentinel-controlled loop)，C 的 for 循环很适合计数器控制循环 (counter-controlled loop)。实际上，可以把 for 循环看作 while 循环的一种特殊形式；当事先知道迭代次数时，它用起来尤其方便。

最直接的 for 用法，是将一条语句重复执行指定次数。例如：

```
#include <stdio.h>
int main(void)
{
    int x;
    for (x = 0; x < 10; x++)
        printf("%d ", x);
}
```

这个循环恰好执行十次，输出如下。

```
0 1 2 3 4 5 6 7 8 9
```

乍看之下，C 的 for 语法可能有些费解。它由四个部分组成：

```
for (init; test; update)
    loop_body;
```

圆括号中的 init、test 和 update 控制循环行为，彼此之间必须用分号分隔。最后一个部分 loop_body 指定每次迭代实际执行的计算。

下面逐一考察。init 是一个表达式，也可以是声明，在第一次迭代之前执行，通常用于声明和初始化变量，为循环作准备。test 是一个表达式，在每次迭代前求值，确定是否再执行一次迭代。如果其值为零，for 终止，控制流转移到紧跟在 for 后面的语句；如果非零，就再执行一次 loop_body。因此，前面示例中的 $x < 10$ 使循环在 x 小于 10 时持续重复。update 是每次迭代结束时求值的表达式，用来更新状态，为下一次迭代作准备。前面的例子在更新步骤中将 x 加 1。loop_body 定义每次迭代要完成的工作，其作用与 while 中的循环体相同，也可以是一条复合语句。

前面提到，init 也可以是声明。如果采用声明，所声明变量的作用域就是这条 for 语句本身。这样可以很方便地把迭代变量限制在循环内部。下面是常见的写法，在循环中直接声明迭代变量。这样做有明显好处：迭代变量不会散落在代码的其他地方，而是在循环的几行代码中完成声明和使用。这个版本与前面的版本有所不同，区

别是什么？提示：不是输出不同。

```
#include <stdio.h>
int main(void)
{
    for (int x = 0; x < 10; x++)
        printf("%d ", x);
}
```

图 13.9 给出了 for 的流程图。四个方框分别对应它的四个组成部分。一个条件分支根据 test 的结果，决定是退出循环还是继续迭代。每次迭代结束、update 求值之后，通过一个无条件分支回到测试处。

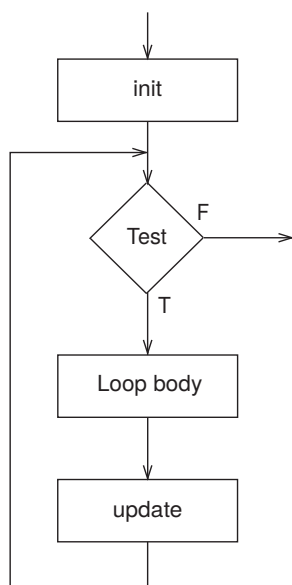


图 13.9 C 的 for 语句的图示

虽然 for 的语法十分灵活，但你遇到或编写的大多数 for 循环都属于计数器控制循环，即执行一定次数的循环。下面是一些常见例子，包括计数器控制循环和在循环中局部声明变量的写法。

```
// --- What does the loop output? ---
for (int x = 0; x <= 10; x++)
    printf("%d ", x);

// --- What does this one output? ---
char letter = 'a';
for (char c = 0; c < 26; c++)
    printf("%c ", letter + c);

// --- What does this loop do? ---
int numberOfOnes = 0;
for (int bitNum = 0; bitNum < 16; bitNum++) {
    if (inputValue & (1 << bitNum))
        numberOfOnes++;
}
```

来看看一个简单 `for` 循环对应的 LC-3 代码。这个程序计算 0 到 9 之间所有整数之和。

```
#include <stdio.h>
int main(void)
{
    int x;
    int sum = 0;
    for (x = 0; x < 10; x++)
        sum = sum + x;
}
```

编译器生成的 LC-3 代码如图 13.10 所示。

```

1          AND R0, R0, #0    ; clear out R0
2          STR R0, R5, #-1   ; sum = 0;
3
4          ; init
5          AND R0, R0, #0    ; clear out R0
6          STR R0, R5, #0    ; init (x = 0)
7
8          ; test
9  LOOP    LDR R0, R5, #0    ; perform the test
10         ADD R0, R0, #-10
11         BRpz DONE        ; x is not less than 10
12
13         ; loop body
14         LDR R0, R5, #0    ; get x
15         LDR R1, R5, #-1   ; get sum
16         ADD R1, R1, R0    ; sum + x
17         STR R0, R5, #-1   ; sum = sum + x;
18
19         ; reinit
20         LDR R0, R5, #0    ; get x
21         ADD R0, R0, #1
22         STR R0, R5, #0    ; x++
23         BR LOOP
24
25  DONE
26         :
```

图 13.10 为一条 `for` 语句生成的 LC-3 代码

译注：原图第 16 行将和保存在 *R1* 中，因此第 17 行应为 `STR R1, R5, #-1`；原图误写为存储 *R0*。原图按原样保留。

下面的代码包含一个编写 `for` 循环时很容易犯的错误。

```
sum = 0;
for (x = 0; x < 10; x++);
    sum = sum + x;
printf("sum = %d\n", sum);
printf("x = %d\n", x);
```

第一个 printf 输出什么？答案是 `sum = 10`。为什么？第二个输出 `x = 10`，又是为什么？仔细观察，也许你会发现一个放错位置的分号。

可以用 while 构造 for 循环，反过来也可以。编程时，两者在一定程度上可以互换。起初，你可能不清楚什么场合应该用哪一种。记住这条一般原则：while 适合涉及哨兵条件的循环，而 for 适合事先知道迭代次数的情形。

13.3.2.1 嵌套循环

图 13.11 展示了一个循环体中又包含 for 循环的 for。这种结构称为**嵌套循环** (nested loop)，因为内层循环嵌套在外层循环之中。本例打印 0 到 9 的乘法表。内层循环每次迭代打印一个乘积；外层循环每次迭代时，内层循环执行十次。因此，外层循环每次迭代会打印完整的一行。注意第 9 行 printf 的格式字符串中有一个特殊字符序列 `\t`，它输出一个制表符 (tab character)，使乘法表的列对齐，令输出更整齐。

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      // Outer Loop
6      for (int multiplicand = 0; multiplicand < 10; multiplicand++) {
7          // Inner Loop
8          for (int multiplier = 0; multiplier < 10; multiplier++)
9              printf("%d\t", multiplier * multiplicand);
10         printf("\n");
11     }
12 }
```

图 13.11 打印乘法表的程序

图 13.12 的例子稍复杂一些。内层循环的迭代次数取决于外层循环所确定的 outer 值。内层循环依次执行一次、两次、三次，依此类推。²本书后面会遇到许多采用类似迭代结构的有趣算法。

²译注：原文写成“先零次，再一次、两次”，但图 13.12 中 outer 从 1 开始，inner 从 0 开始且测试 `inner < outer`，所以首次进入外层循环时，内层实际执行一次。这里按代码更正。

```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int sum = 0;    // Initial the result variable
6      int input;      // Holds user input
7
8      // Get input
9      printf("Input an integer: ");
10     scanf("%d", &input);
11
12     // Perform calculation
13     for (int outer = 1; outer <= input; outer++) {
14         for (int inner = 0; inner < outer; inner++) {
15             sum += inner;
16         }
17     }
18
19     // Output result
20     printf("The result is %d\n", sum);
21 }
```

图 13.12 包含嵌套 for 循环的程序

13.3.3 do-while 语句

在 while 循环中，总是在迭代前检查条件。因此，如果条件一开始就为假，循环可能一次也不执行。C 还提供了一个略有不同的形式 do-while，它总会执行至少一次迭代。do-while 在完成第一次迭代后才计算条件。下面的例子说明其行为。

```
x = 0;
do {
    printf("%d \n", x);
    x = x + 1;
} while (x < 10);
```

这里的条件 $x < 10$ 在每次迭代末尾求值，所以循环体至少执行一次。只有条件值非零时，才进行下一次迭代。代码的输出如下，每个数占一行。

0
1
2
3
4
5
6
7
8
9

译注：原文将这些数排在同一行，但代码的格式字符串含有 `\n`，这里按代码的实际换行方式排出。

在语法上，`do-while` 与 `while` 一样，也由两个部分组成。

```
do
    loop_body;
while (test);
```

`loop_body` 是一条简单语句或复合语句，描述循环要执行的计算。`test` 是决定是否再执行一次迭代的表达式。图 13.13 给出了控制流。注意，它相对于 `while` 的流程只有一点变化：循环体与测试的位置互换了。条件分支回到循环体的开头，开始下一次迭代。

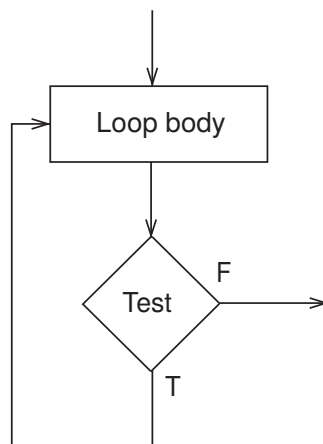


图 13.13 C 的 `do-while` 语句的图示

现在看来，三种 C 迭代结构的区别或许很细微。但随着你熟悉它们、积累使用经验，就会更容易选择适合当前情境的结构。它们在很大程度上可以互换，不过从程序风格看，有时某一种更合适。你所选择的循环类型，往往能向代码阅读者传达这个循环的意图。

13.4 使用控制结构求解问题

掌握了这些新的控制结构，我们就可以尝试解决更复杂的编程问题。本节将运用自顶向下的问题求解方法，解决三个需要使用 C 控制结构的问题。³

有效地解决编程问题，需要理解所使用系统提供的基本构件。程序设计的艺术，往往就在于巧妙而富有创造性地运用这些基本构件。目前我们已经掌握 C 的基本类型变量、运算符、两种判定结构和三种迭代结构。⁴这些工具已经相当有力，足以完成许多工作。

13.4.1 问题 1：求 π 的近似值

第一个编程问题，是利用下面的级数展开求 π 的值：

$$\pi = 4 - \frac{4}{3} + \frac{4}{5} - \frac{4}{7} + \cdots = \sum_{n=1}^{\infty} (-1)^{n-1} \frac{4}{2n-1}.$$

译注：原文通项为 $(-1)^{n-1}4/(2n+1)$ ，与前面列出的各项不一致。这里按从第 1 项开始计数的约定，将分母更正为 $2n-1$ ，并明确写出无穷和；图 13.17 的程序也使用这一分母。

我们将编写 C 程序，根据用户指定的项数计算这个级数。例如，用户输入 3，程序就计算 $4 - 4/3 + 4/5$ 。这是一个无穷级数，计算的项数越多，得到的 π 近似值就越精确。不过它收敛得不快，所以需要计算很多项才能获得较好的近似。

与第 12 章的问题求解示例一样，首先执行第 0 步：选择计算中数据的表示方式。由于级数涉及分数，直接参与级数计算的变量都采用 double 浮点类型。

接下来运用逐步求精，将一个粗略描述的算法分解为 C 程序。大致说来，程序先完成初始化，再提示用户输入要计算的项数，接着计算指定项数的级数，最后打印结果。这样，我们就用一组顺序结构描述了问题。图 13.14 展示初步分解。其中大多数顺序结构都很直接，很容易转换为 C 代码。

³译注：原文此处写“四个问题”，但第 13.4 节实际列出三个。第四个计算器示例位于第 13.5.3 节。这里按本节内容更正。

⁴译注：原文此处及本章小结沿用了“三种基本类型”的说法；第 12.2.1 节已包括布尔类型，共介绍四种。此处统一称为“基本类型”。原文的“三种控制结构”根据上下文文明确为“三种迭代结构”。

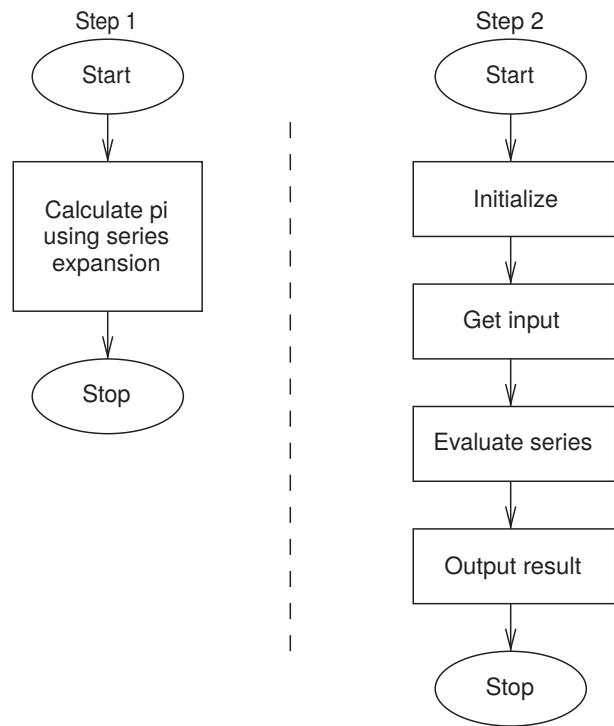


图 13.14 按给定项数计算 π 级数展开的程序的初步分解

不过，其中有一个部分还需要进一步细化，即 *Evaluate series*（计算级数）子任务。对于这个子任务，基本思路是逐项迭代，直到恰好计算了用户指定的项数。因为确切知道迭代次数，所以应使用计数器控制的迭代结构。图 13.15 展示进一步的分解：维护一个记录当前迭代次数的计数器，只要它小于用户指定的上限，就再计算一项。

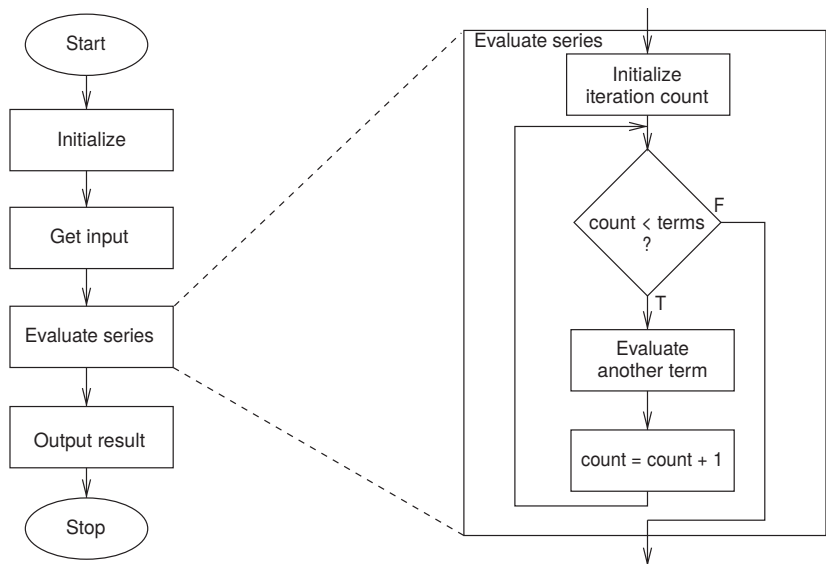


图 13.15 将 *Evaluate series* 子任务细化为执行指定次数的迭代结构；循环内计算 π 级数展开中的各项。

注意，细化后的子任务形状很像 `for` 循环的流程图。剩下唯一需要认真处理的子任务是 *Evaluate another term*（再计算一项）。级数的偶数项都要减去，奇数项都要

加上。因此，需要确定当前项的序号是奇数还是偶数，再将它计入当前近似值。这要使用图 13.16 所示的判定结构。逐步求精所得的完整代码见图 13.17。

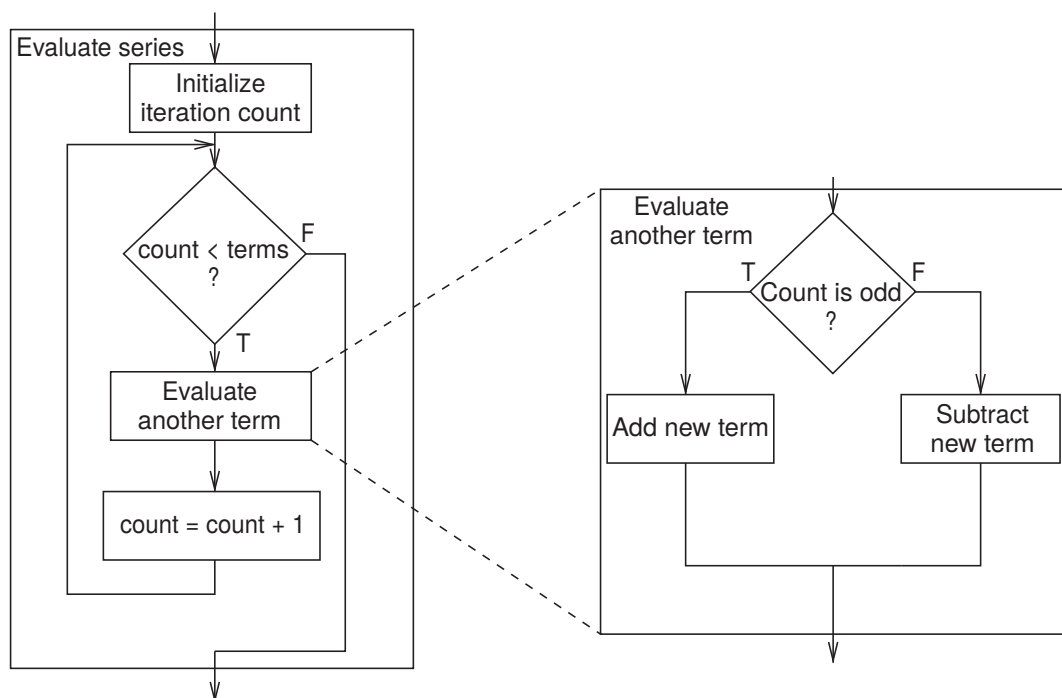


图 13.16 根据当前项的序号是奇数还是偶数，将它计入结果。

```

1  #include <stdio.h>
2
3  int main(void)
4  {
5      int numOfTerms;    // Number of terms to evaluate
6      double pi = 0;     // approximation of pi
7
8      printf("Number of terms (must be 1 or larger) : ");
9      scanf("%d", &numOfTerms);
10
11     for (int count = 1; count <= numOfTerms; count++) {
12         if (count % 2)
13             pi = pi + (4.0 / (2.0 * count - 1)); // Odd term
14         else
15             pi = pi - (4.0 / (2.0 * count - 1)); // Even term
16     }
17
18     printf("The approximate value of pi is %f\n", pi);
19 }

```

图 13.17 计算 π 的程序

译注：图 13.15、13.16 采用 $count < terms$ ，而图 13.17 的代码从 $count = 1$ 开始并使用 $count \leq num\ of\ Terms$ 。流程图中的计数上限和初始化约定应配套使用；不能将图中的严格小于条件直接替换到这段代码中，否则会少计算一项。

13.4.2 问题 2：找出小于 100 的素数

下一个问题是找出所有小于 100 的素数。一个大于 1 的整数，如果只有 1 和它本身能够将它整除，就是素数。

与之前的例子一样，第 0 步是为问题中的各种变量选择合适的数据表示。素数是整数的性质，因此主要计算采用整数类型最合适。然后先把问题描述成一个任务，这是第 1 步。第 2 步将它细化为两个顺序执行的子任务：*Initialize* (初始化) 和 *Calculation* (进行计算)。主要的编程工作集中在后者。其基本思路是：遍历 2 到 100 的每一个整数，判断它是否为素数，只打印其中的素数。由于确切知道迭代范围，计数器控制循环非常适合。

可以将 *Calculation* 进一步细化为图 13.18 中的小任务。注意，流程图已呈现出 `for` 循环的形状，问题开始逐渐落实为 C 代码。不过，*CalcPrime* 子任务仍需细化，以判断当前数是否为素数。这里利用这样一个事实：2 到 100 之间的任何合数，至少有一个不等于它本身、且位于 2 到 10 之间的因子。

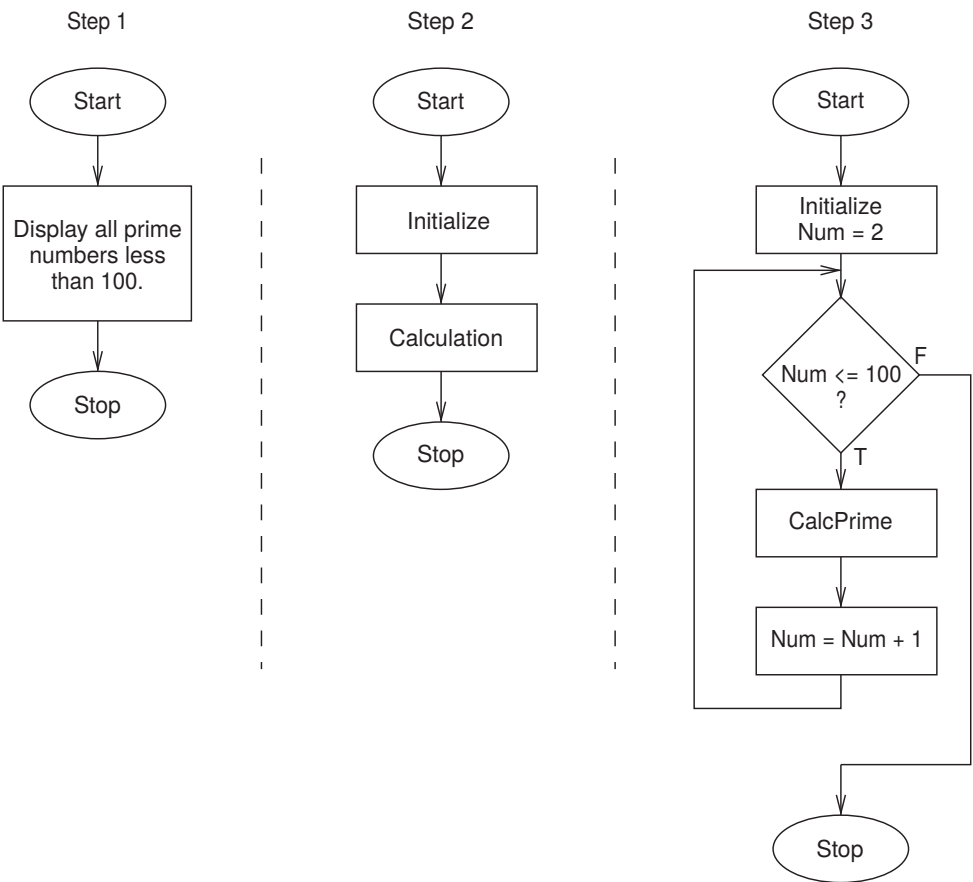


图 13.18 分解求小于 100 的素数的问题。前三步构造一个遍历 2 到 100 的循环。

按照图 13.19 细化 *CalcPrime*：检查当前数能否被 2 到 10 之间的整数整除，并注意排除用它本身作除数的情况。如果除它本身外，2 到 10 之间没有整数能整除它，那么它就是素数。

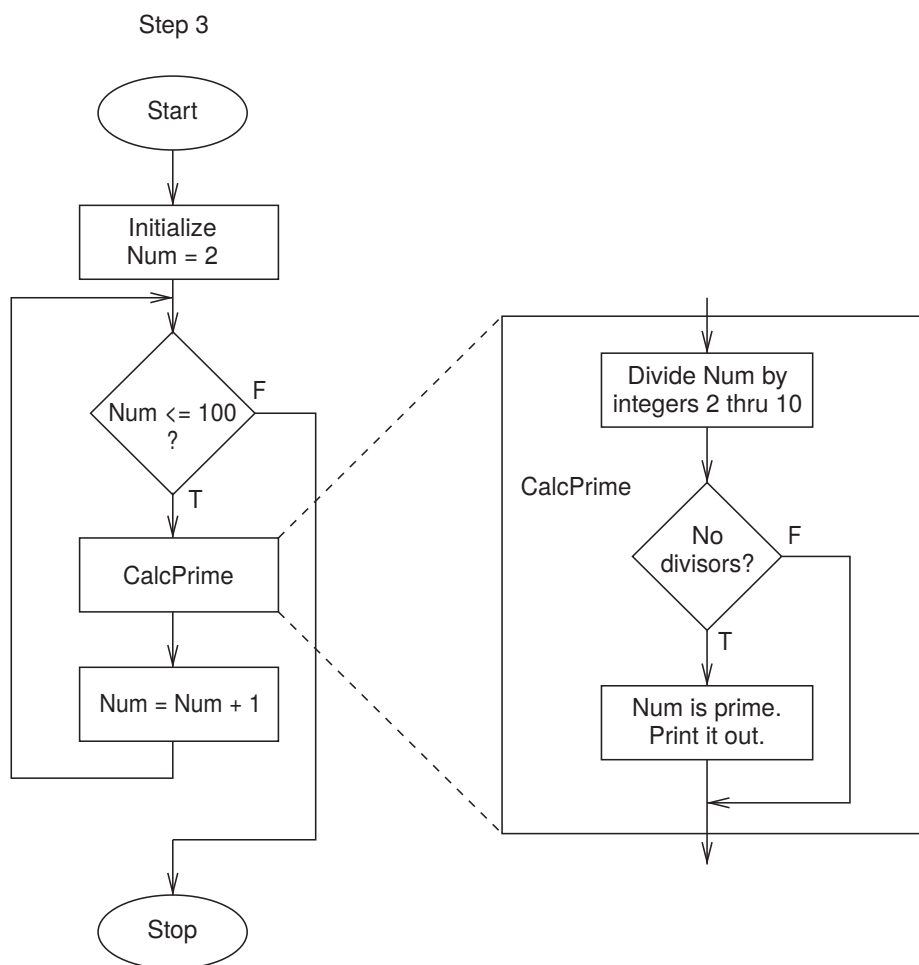


图 13.19 分解 *CalcPrime* 子任务

最后要细化 *Divide number by integers 2 through 10*（用 2 到 10 的整数除当前数）这个子任务。它需要用 2 到 10 之间的每个整数去除当前数，判断是否存在能够整除的数。简单的实现方式是再用一个计数器控制循环，遍历 2 到 10 的所有整数。图 13.20 展示了这种迭代分解。

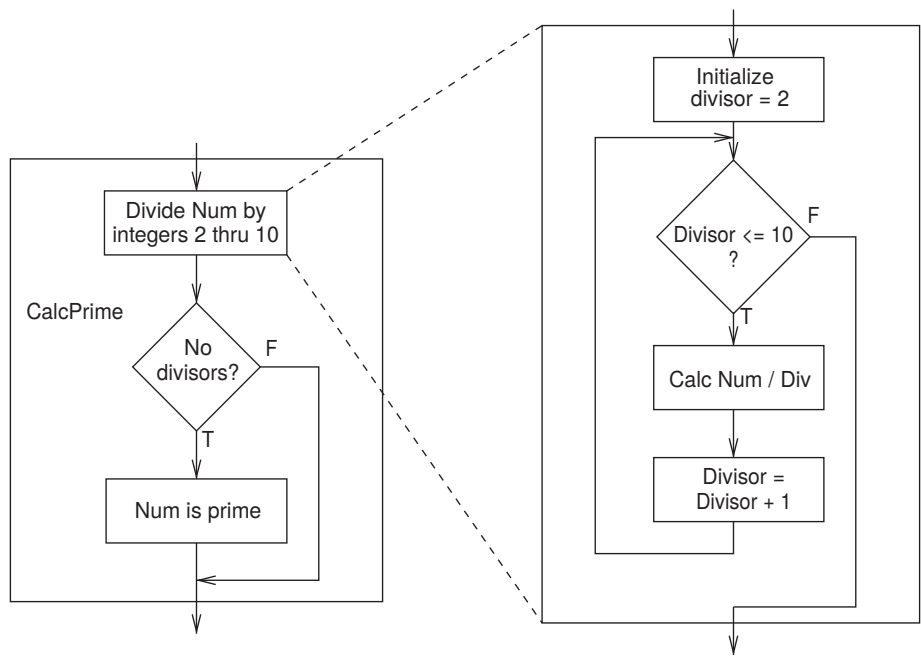


图 13.20 分解用 2 到 10 的整数除当前数的子任务

现在得到的程序表示已经很容易转化为 C 代码，完整程序见图 13.21。程序中有两个 for 循环，一个嵌套在另一个之中。外层循环遍历 2 到 100 的整数，对应分解 *Calculation* 时构造的循环；内层循环判断外层生成的数是否存在其他因子，对应分解除法子任务时构造的循环。

```
1  #include <stdio.h>
2  #include <stdbool.h>
3
4  int main(void)
5  {
6      bool prime = true;
7
8      // Start at 2 and go until 0
9      for (int num = 2; num <= 100; num++) {
10         prime = true;    // Assume the number is prime
11
12         // Test if the candidate number is a prime
13         for (int divisor = 2; divisor <= 10; divisor++)
14             if ((num % divisor) == 0) && num != divisor)
15                 prime = false;
16
17         if (prime)
18             printf("The number %d is prime\n", num);
19     }
20 }
```

图 13.21 找出 2 到 100 之间所有素数的程序

译注：代码遍历到 100，标题则说“小于 100”；由于 100 不是素数，两者的输出相同。原图第 8 行注释中的 *go until 0* 应理解为遍历到 100，图内文字保留原样。

13.4.3 问题 3：分析电子邮件地址

本节最后一个问题，是分析从键盘输入的电子邮件地址，判断它的格式是否有效。本例采用一个简单的有效性定义：地址是一串字符，其中必须含有 at 符号 @ 和句点 .，而且 @ 必须出现在句点之前。这只是本例的简化规则。

仍然先为问题所涉及的数据选择表示方式。这里处理用户输入的文本，适合使用表示 ASCII 字符的 `char`。实际上，输入文本更适合表示成字符数组，即字符串；但我们尚未介绍数组，要到第 16 章才会讨论，所以这里只使用一个 `char` 变量。

接下来逐步求精，整个过程见图 13.22。第 1 步先勾勒程序的大致流程，包含两个任务：*Process input*（处理输入）和 *Output results*（输出结果）。输出结果很直接，只需报告输入字符序列是否有效。

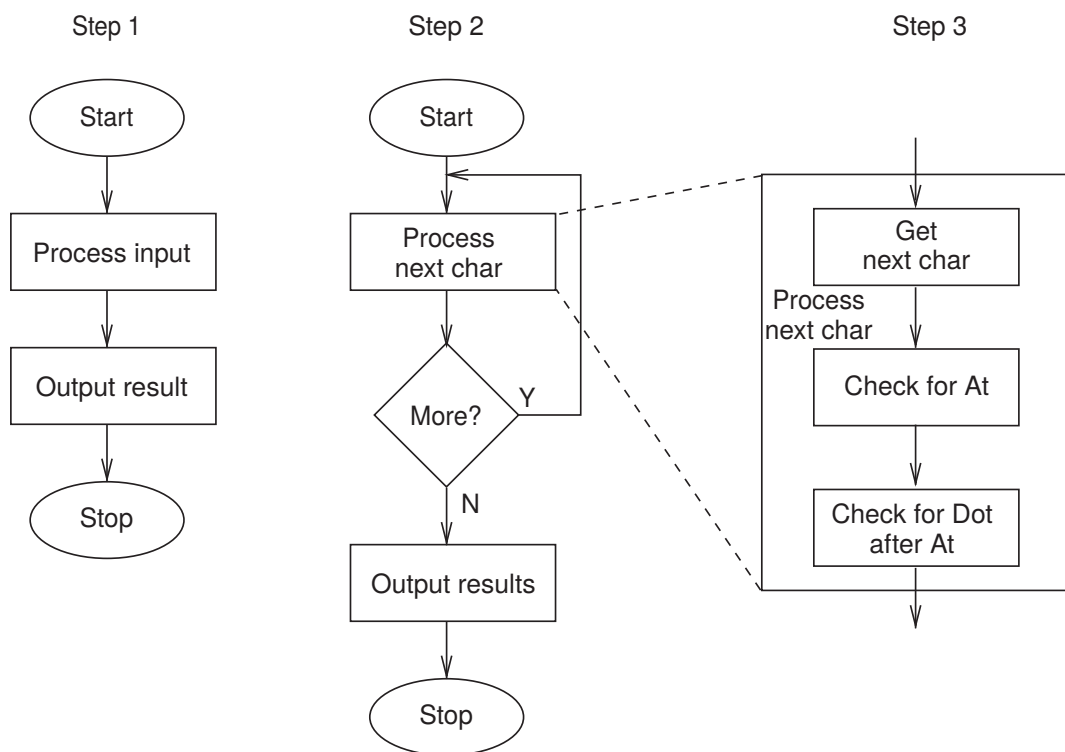


图 13.22 对电子邮件地址分析程序进行逐步求精

处理输入则需要进一步细化。分解这个任务时，第 2 步要考虑数据表示的选择：只有一个 `char` 变量，就必须逐个读取并处理输入字符。一直处理到地址末尾，意味着需要哨兵控制循环。这里把空格或换行符 `\n` 视为地址结束的标志，遇到它们就终止循环。

第 3 步细化循环内部的处理。需要检查地址中的每个字符，记录是否按正确顺序遇到了 `@` 和句点。我们用两个变量分别记录这两个符号是否出现过。循环终止、准备显示结果时，检查这两个变量，就能选择适当的输出消息。

现在距离 C 代码已经很近了。注意，循环结构很像 `do-while` 的流程图，所以选择它来实现迭代。完整代码见图 13.23。

```
1  #include <stdio.h>
2  #include <stdbool.h>
3
4  int main(void)
5  {
6      char nextChar;           // Next character in e-mail address
7      bool gotAt = false;      // Indicates if At @ was found
8      bool gotDot = false;     // Indicates if Dot . was found
9
10     printf("Enter your e-mail address: ");
11
12     do {
13         scanf("%c", &nextChar);
14
15         if (nextChar=='@')
16             gotAt = true;
17
18         if (nextChar=='.' && gotAt)
19             gotDot = true;
20     }
21     while (nextChar != ' ' && nextChar != '\n');
22
23     if (gotAt && gotDot)
24         printf("Your e-mail address appears to be valid.\n");
25     else
26         printf("Your e-mail address is not valid!\n");
27 }
```

图 13.23 对电子邮件地址有效性进行简单检查的 C 程序

13.5 其他 C 控制结构

最后介绍 `switch`、`break` 和 `continue`，以完成对 C 控制结构的讨论。它们提供了一些专门的程序控制方式，在特定编程情境中很方便。这里主要是为了完整性而介绍它们，本书后续的示例不再使用这三种结构。

13.5.1 `switch` 语句

有时，我们需要根据某个变量的值，在 n 个不同动作中选择一个。例如，下面的代码检查字符变量 `keyPress` 等于哪一个字符值，再执行相应的动作。注释中的 A、B、C、D 代表有待填入的实际语句。

```
// keypress is a char variable
if (keyPress == 'a')
    // Do statement A
else if (keyPress == 'b')
    // Do statement B
else if (keyPress == 'x')
    // Do statement C
else if (keyPress == 'y')
    // Do statement D
```

根据 keyPress 的值，标为 A、B、C、D 的语句中会有一条执行，也可能全都不执行。keyPress 等于字符 a 时执行 A，等于 b 时执行 B，依此类推。如果它不是 a、b、x、y 中的任何一个，则不执行这些语句。如果候选条件很多，找到匹配项可能要经过许多次测试。为简化这种程序设计惯用法，C 提供了 switch 语句。下面的代码与前例行为相同，但使用 switch 代替级联的 if-else。

```
switch (keyPress) {
    case 'a':
        // Do statement A
        break;

    case 'b':
        // Do statement B
        break;

    case 'x':
        // Do statement C
        break;

    case 'y':
        // Do statement D
        break;
}
```

注意，switch 中有多行以关键字 case 开头，后面跟一个标签值和冒号：。如果某个 case 标签的值与 keyPress 相同，就从相应位置开始执行。

下面逐步考察 switch。关键字后面是用于作出选择的表达式，它必须具有整数类型，例如 int 或 char。如果某个 case 标签与表达式的值匹配，控制流就转移到该标签之后的语句或块。每个 case 后可以跟零条或多条语句，类似复合语句，但不需要用额外的花括号界定。究竟从哪里开始执行，由哪个 case 与 switch 表达式匹配决定。同一条 switch 中，各个 case 标签必须互不相同，不能重复。而且标签值必须是常量表达式，不能依赖程序运行过程中会改变的值。

前例中，每个 case 都以 break 结束。break 将控制流转移到整个 switch 之后，从而退出这一结构。break 是可选的；如果省略，执行就会从当前 case 继续落入下一个，这种写法相对少见。还可以加入可选的 default 分支。当表达式不匹配任何 case 常量时，选择该分支。如果既没有匹配项，也没有 default，就不执行其中的任何分支。

关于代码风格还有一点：最后一个 case 即使没有 break，执行到结尾也会自然退出 switch。不过，仍然写上 break 是一个好习惯。如果以后在末尾再添加一个 case，就不必记得回头给原来的最后一个分支补上 break。

13.5.2 break 和 continue 语句

第 13.5.1 节展示了 break 与 switch 配合的例子。break 和 continue 有时也与迭代结构一起使用。break 使编译器生成立即退出循环或 switch 的代码。在循环体中使用它，会跳转到循环语句之后，终止循环，不再进行后续迭代。continue 则只结束当前这次迭代；是否执行下一次，还要看循环条件。

这两条语句都可以出现在循环体中，作用于包围它们的相应结构：continue 针对最近的外层循环，break 针对最近的外层循环或 switch。⁵从本质上说，它们使编译器生成无条件分支，从循环体内部的某个位置离开整个循环或当前迭代。下面两段 C 代码分别使用 break 和 continue。

```
// This code segment produces the output: 0 1 2 3 4
for (i = 0; i < 10; i++) {
    if (i == 5)
        break;
    printf("%d ", i);
}

// This code produces the output: 0 1 2 3 4 6 7 8 9
for (i = 0; i < 10; i++) {
    if (i == 5)
        continue;
    printf("%d ", i);
}
```

13.5.3 示例：简单计算器

图 13.24 的程序与第 10 章的计算器示例功能类似。它提示用户输入三个项目：第一个整数操作数、要执行的运算，以及第二个整数操作数。程序对两个输入值执行指定运算，并显示结果。它使用 switch，根据用户选择的运算符决定要进行的计算。

⁵译注：原文笼统地说两者都作用于直接包围它们的迭代结构；当循环内还嵌套有 switch 时，break 会先退出这个 switch，所以这里明确其作用范围。对于 for，continue 转到更新表达式，再测试条件；对于 while 和 do-while，则转到条件测试。


```
1  #include <stdio.h>
2
3  int main(void)
4  {
5      int operand1, operand2; // Input values
6      int result = 0;         // Result of the operation
7      char operation;         // operation to perform
8
9      // Get the input values
10     printf("Enter first operand: ");
11     scanf("%d", &operand1);
12     printf("Enter operation to perform (+, -, *, /): ");
13     scanf("\n%c", &operation);
14     printf("Enter second operand: ");
15     scanf("%d", &operand2);
16
17     // Perform the calculation
18     switch(operation) {
19     case '+':
20         result = operand1 + operand2;
21         break;
22
23     case '-':
24         result = operand1 - operand2;
25         break;
26
27     case '*':
28         result = operand1 * operand2;
29         break;
30
31     case '/':
32         if (operand2 != 0) // Error-checking code.
33             result = operand1 / operand2;
34         else
35             printf("Divide by 0 error!\n");
36         break;
37
38     default:
39         printf("Invalid operation!\n");
40         break;
41     }
42
43     printf("The answer is %d\n", result);
44 }
```

图 13.24 用 C 编写的计算器程序

译注：原程序在除数为零或运算符无效时打印错误消息，随后仍会执行末尾的结果输出语句，打印初值为 0 的 *result*。这里保留原代码，错误分支的行为也按原样理解。

13.6 小结

本章的基本目标，是通过学习 C 支持的各种控制结构，扩充我们求解问题时可以使用的基本构件。主要内容如下。

- **C 的判定结构。**介绍了 if 和 if-else 两种基本判定语句。它们根据指定表达式的真假，有条件地执行语句。
- **C 的迭代结构。**C 提供 while、for 和 do-while 三种迭代语句，依据迭代准则或条件重复执行语句。while 和 do-while 尤其适合表达哨兵控制循环，for 则很适合表达计数器控制循环。
- **使用控制结构求解问题。**已有的工具包括 C 的基本类型、变量、运算符，以及使用 printf 和 scanf 的 I/O；现在又加入了控制结构。本章通过若干问题求解示例，练习了如何运用这些结构。

习题

13.1 写出 LC-3 编译器在编译图 13.24 的计算器程序时建立的符号表。

13.2 a. 下面的代码经过预处理器处理后会变成什么样？

```
#define VERO -2
if (VERO)
    printf("True!");
else
    printf("False!");
```

b. 运行这段代码会产生什么输出？

c. 如果改成下面的代码，行为是否发生变化？如果有，是什么变化？

```
#define VERO -2
if (VERO)
    printf("True!");
else if (!VERO)
    printf("False!");
```

13.3 if-else 语句可以代替 C 的条件运算符，见第 12.6.3 节。用 if-else 改写下面的语句，不使用条件运算符。

```
x = a ? b : c;
```

13.4 分别说明 x 等于 0 和等于 1 时，下列语句的行为。假设所有变量都是整数类型。

```
a. if (x = 0)
    printf("x equals 0\n");
else
    printf("x does not equal 0\n");
```

- ```
b. if (x == 0)
 printf("x equals 0\n");
else
 printf("x does not equal 0\n");
```
- ```
c. if (x == 0)
    printf("A\n");
else if (x != 1)
    printf("B\n");
else if (x < 1)
    printf("C\n");
else if (x)
    printf("D\n");
```
- ```
d. switch (x) {
 case 0:
 y = 3;
 case 1:
 y = 4;
 break;
 default:
 y = 5;
 break;
}
```
- e. 对于 d 小题，如果  $x$  既不等于 0，也不等于 1，会发生什么？

**13.5** 给出本书的 LC-3 C 编译器在编译习题 13.4 d 中的 switch 时生成的 LC-3 代码。

**13.6** 图 13.12 包含一个带有嵌套 for 循环的 C 程序。

- 用数学形式表示这个程序所计算的级数。
- 编写程序，计算下列函数：

$$f(n) = f(n-1) + f(n-2),$$

初始条件为

$$f(0) = 1, \quad f(1) = 1.$$

**13.7** 下面的 if-else 能否转换为 switch? 如果可以, 给出转换结果; 如果不可以, 说明原因。

```
if (x == 0)
 y = 3;
else if (x == 1)
 y = 4;
else if (x == 2)
 y = 5;
else if (x == y)
 y = 6;
else
 y = 7;
```

**13.8** 在下列各结构中, 名为 loopBody 的语句至少会执行多少次?

- a. while (condition)  
    loopBody;
- b. do  
    loopBody;  
    while (condition);
- c. for (init; condition; update)  
    loopBody;
- d. while (condition1)  
    for (init; condition2; reinit)  
        loopBody;
- e. do  
    do  
        loopBody;  
    while (condition1);  
    while (condition2);

**13.9** 下列各段代码的输出是什么?

a. a = 2;  
   while (a > 0)  
      a--;  
   printf("%d", a);

b. a = 2;  
   do {  
      a--;  
   } while (a > 0);  
   printf("%d", a);

译注: 原文 do-while 末尾遗漏分号, 这里补上以符合题目所问的运行输出。

```
c. b = 0;
 for (a = 3; a < 10; a += 2)
 b = b + 1;
 printf("%d %d", a, b);
```

**13.10** 将图 13.4 的程序改写为使用 `switch`，而不使用 `if-else` 的程序。

**13.11** 修改图 13.23 的电子邮件地址检查程序，进一步要求有效地址满足：@ 之前至少有一个字母，@ 与句点之间至少有一个字母，句点之后也至少有一个字母。

**13.12** 以下各小题中，`x` 是一个初值为 4 的整数。

a. 下面的代码产生什么输出？

```
if (7 > x > 2)
 printf("True.");
else
 printf("False.");
```

b. 为什么下面的代码不会形成无限循环？

```
while (x > 0)
 x++;
```

译注：原题的问题隐含了固定宽度整数加法回绕的机器层假设。对于 *C* 的有符号 *int*，自增溢出属于未定义行为，不能据此保证循环会终止。题目原文与代码均保留，讨论时应区分本书的 *LC-3* 机器模型与 *C* 语言语义。

c. 下面的代码执行后，`x` 的值是多少？

```
for (x = 4; x < 4; x--) {
 if (x < 2)
 break;
 else if (x == 2)
 continue;
 x = -1;
}
```

**13.13** 修改下面的程序，用 do-while 循环代替 for 循环。

```
int main(void)
{
 int sum;

 for (int i = 0; i <= 100; i++) {
 if (i % 4 == 0)
 sum = sum + 2;
 else if (i % 4 == 1)
 sum = sum - 6;
 else if (i % 4 == 2)
 sum = sum * 3;
 else if (i % 4 == 3)
 sum = sum / 2;
 }
 printf("%d\n", sum);
}
```

译注：原程序没有初始化局部变量 *sum*，也未列出声明 *printf* 所需的头文件。这里保留原题；若要作为独立程序运行，需要补充 `#include <stdio.h>` 并为 *sum* 指定初值，不能假定局部变量自动为 0。

**13.14** 编写 C 程序，读入一个整数 *k*，然后输出如下图案：第一行一个 1，第二行两个 2，第三行三个 3，依此类推，最后一行输出 *k* 个 *k*。

例如，输入为 5 时，输出应为：

```
1
2 2
3 3 3
4 4 4 4
5 5 5 5 5
```

**13.15** a. 将下面的 while 循环改写为 for 循环。

```
while (condition)
 loopBody;
```

## b. 将下面的 for 循环改写为 while 循环。

```
for (init; condition; update)
 loopBody;
```

**13.16** 下面代码的输出是什么？

```
int w = 12;
int sum = 0;
for (int r = 1; r <= w; r++)
 for (int s = r; s <= w; s++)
 sum = sum + s;
printf("sum = %d\n", sum);
```

**13.17** 下面的代码完成了一个很具体的任务，请描述它的输出。

```
int i;
scanf("%d", &i);
for (int j = 0; j < 16; j++) {
 if (i & (1 << j))
 count++;
}
printf("%d\n", count);
```

译注：原题没有给出 *count* 的声明和初值。分析预期功能时，需要补充它是初值为 0 的整数这一前提。此外，若 *int* 为 16 位， $1 \ll 15$  涉及有符号左移的可表示范围问题；按位操作的可移植写法应使用适当宽度的无符号类型。这里保留原题代码。

**13.18** 给出下列各段代码的输出。

- a. 

```
int x = 20;
int y = 10;
while ((x > 10) && (y & 15)) {
 y = y + 1;
 x = x - 1;
 printf("*");
}
```
- b. 

```
for (int x = 10; x ; x = x - 1)
 printf("*");
```
- c. 

```
for (int x = 0; x < 10; x = x + 1)
 if (x % 2)
 printf("*");
```
- d. 

```
int x = 0;
while (x < 10) {
 for (int i = 0; i < x; i = x + 1)
 printf("*");
 x = x + 1;
}
```

**13.19** 为什么 C 不允许像 for 循环那样，在 while 循环中声明变量？

```
while (int x < 10) {
 // Loop body
 x = x + 1;
}
```

# 第十四章 函数

## 14.1 引言

函数是当今大规模代码开发的基本构件。软件开发者借助函数，可以在语言直接支持的结构之外，增加新的基本操作。函数如此必要而重要，以至于我们在硬件的指令集体系结构层面直接为它提供支持。

函数为什么如此重要？函数、过程、子程序、方法，虽然各有差别，却都体现了同一种思想：支持抽象。也就是说，它们增强了我们将一个部件“做什么”与“怎样做”分开的能力。一旦构造好一个部件并理解了它的构造，就可以把它当作基本构件使用，而不必反复考虑实现细节。没有抽象，我们构造复杂而精巧的数字系统的能力就会受到严重限制。

函数对我们并不陌生。早在用 LC-3 汇编语言编写子程序时，我们就已经在使用函数的某种形式。LC-3 汇编子程序和 C 函数虽然语法不同，背后的概念却大体相同。C 语言以函数为中心组织程序：一个 C 程序本质上是一组函数，每条语句都属于某一个、且只属于一个函数。C 程序从 main 开始执行，并在它结束时结束。main 可以调用其他函数，其他函数又可以继续调用更多函数；控制最终回到 main，而 main 结束意味着程序结束。<sup>1</sup>C++ 还把函数的思想推进了一步，允许程序员将函数与特定类型关联起来，构造对象。那是第 20 章的主题。

本章详细考察 C 函数。首先编写几个小程序，熟悉函数的 C 语法；随后研究函数的实现，考察使函数正常工作所需的底层操作；最后运用问题求解方法，解决一些适合使用函数的编程问题。

## 14.2 C 中的函数

先看一个使用函数的简单 C 程序。图 14.1 通过名为 PrintBanner 的函数打印横幅。程序从 main 开始执行，然后调用 PrintBanner，后者向输出设备打印一行由字符 = 组成的文本。PrintBanner 是最简单的函数形式：完成任务不需要调用者提供输入，也不向调用者提供输出数据，这里不把屏幕上打印的横幅算作返回给调用者的数据。换言之，main 没有向 PrintBanner 传递实参，PrintBanner 也没有向 main 返回值。正式地说，main 是**调用者**，PrintBanner 是**被调用者**。

---

<sup>1</sup>译注：这里描述的是本书采用的宿主环境中正常进入 main 并从中返回的执行路径。程序也可以通过其他终止机制结束；并非所有 C 执行环境都要求以 main 为入口。



```
1 #include <stdio.h>
2
3 void PrintBanner(); // Function declaration
4
5 int main(void)
6 {
7 PrintBanner(); // Function call
8 printf("A simple C program.\n");
9 PrintBanner();
10 }
11
12 void PrintBanner() // Function definition
13 {
14 printf("=====\n");
15 }
```

图 14.1 使用函数打印横幅消息的 C 程序

### 14.2.1 带有形参的函数

PrintBanner 与 main 不需要交换信息，所以接口很简单。但一般来说，我们希望能在调用者和被调用者之间传递信息。下面的例子说明 C 如何做到这一点。图 14.2 中的 Factorial 根据一个输入实参进行计算。

Factorial 将 1 到  $n$  的所有整数相乘，其中  $n$  是调用者，本例中为 main，提供的值。它所完成的计算可以表示为

$$\text{factorial}(n) = n! = 1 \times 2 \times 3 \times \cdots \times n.$$

```
1 #include <stdio.h>
2
3 int Factorial(int n); // Function Declaration
4
5 int main(void) // Definition for main
6 {
7 int number; // Number from user
8 int answer; // Answer of factorial
9
10 printf("Input a number: "); // Call to printf
11
12 scanf("%d", &number); // Call to scanf
13
14 answer = Factorial(number); // Call to factorial
15
16 printf("The factorial of %d is %d\n", number, answer);
17 }
18
19 int Factorial(int n) // Function Definition
20 {
21 int result = 1; // Initialized result
22
23 for (int i = 1; i <= n; i++) // Calculate factorial
24 result = result * i;
25
26 return result; // Return to caller
27 }
```

图 14.2 计算阶乘的 C 程序

图 14.2 中计算得到的值存放在 `result` 中，并通过 `return` 语句返回给调用者。我们说，`Factorial` 要求调用者提供一个整数实参，并向调用者返回一个整数值。本例的第 14 行把 `Factorial` 的返回值赋给调用者中的变量 `answer`。

仔细观察语法，图中有四行值得关注：第 3 行是 `Factorial` 的声明；第 19 行开始是它的定义；第 14 行是函数调用，用来启动函数；第 26 行从 `Factorial` 返回调用者。

#### 14.2.1.1 声明

第 3 行的函数声明看起来有些孤零零。它有什么用？与变量声明告诉编译器变量的属性类似，函数声明告诉编译器函数的有关属性。带有参数类型信息的函数声明也称为**函数原型**（`function prototype`），其中包含函数名、返回值类型，以及期望接收的输入参数及其类型的列表。函数声明以分号结束。

声明中的第一项是返回值类型，可以是 `int`、`char`、`double` 等 C 数据类

型。它描述函数产生的单个输出值的类型。并非所有函数都返回值，例如前面的 `PrintBanner` 就不返回值。这类函数应将返回类型声明为 `void`，告诉编译器它不返回任何值。

下一项是函数名，可以是任何合法的 C 标识符。程序员通常会仔细选择名称，使它反映函数完成的操作。例如，`Factorial` 就很合适，因为其运算的数学名称正是 `factorial`，即阶乘。采用便于区分函数名和变量名的命名约定，也是一种良好风格。本书示例通常将函数名的首字母大写，如 `Factorial`。

声明还描述函数所需输入参数的类型和顺序，即函数期望从调用者接收哪些类型的值，以及这些值按什么顺序提供。声明中可以选择写出每个形参的名称，我们也经常这样做。例如，`Factorial` 接收一个整数输入，并在内部用 `n` 引用它。有些函数不需要输入，如 `PrintBanner`，所以原图使用了空参数列表。<sup>2</sup>

#### 14.2.1.2 调用

示例第 14 行调用 `Factorial`。在它开始执行前，`main` 必须先传递一个整数值。调用者中传给被调用者的这些值称为**实参** (`argument`)。实参可以是任何合法表达式，但类型应与被调用者的要求匹配。实参写在紧跟被调用函数名的圆括号内。本例中，`main` 将 `number` 的值作为实参传递，然后把 `Factorial` 的返回值赋给整数变量 `answer`。

#### 14.2.1.3 定义

第 19 行开始是 `Factorial` 的函数定义。定义的第一行与声明相同，只是没有末尾的分号。函数名后面圆括号中的部分，是**形式参数列表** (`formal parameter list`)，也简称形参列表。它由一组变量声明组成，每个变量都用调用者提供的相应实参初始化。本例在第 14 行调用 `Factorial` 时，形参 `n` 被初始化为 `main` 中 `number` 的值。程序中每次调用该函数时，实际参数的类型和顺序都应形式参数列表匹配。

形参列表后面的花括号中是**函数体** (`function body`)，其中的声明和语句定义了函数要完成的计算。花括号内声明的变量是该函数的局部变量，只能在其相应作用域内访问和修改。在 C 中，调用者的实参通过**传值** (`pass by value`) 交给被调用者。也就是说，`Factorial` 不能修改 `number` 这个变量，因为传给它的只是 `number` 的值。

#### 14.2.1.4 返回值

第 26 行将控制从 `Factorial` 交回调用者 `main`。由于 `Factorial` 要返回一个值，`return` 后面必须跟一个表达式，其类型应与声明的返回类型匹配。`return result;` 将 `result` 中计算好的值传回调用者。

一般来说，要返回值的函数应当通过 `return` 明确给出返回值。不返回值的函数，即返回类型为 `void` 的函数，可以省略 `return`；执行完最后一条语句后，控制

---

<sup>2</sup>译注：在本书采用的 C99/C11 语境中，声明 `void PrintBanner();` 的空括号表示未说明参数信息，不能作为“无参数”的函数原型；明确声明无参数应写成 `void PrintBanner(void);`。原图保留原样。

便返回调用者，也可以用 `return;` 提前返回。

那么 `main` 呢？它的返回类型是 `int`，但前面示例中没有 `return`。可以在 `main` 末尾显式写出 `return 0;`，表示正常结束。在 C99 及之后的规则中，到达 `main` 末尾的右花括号与返回 0 等价，因此本书为使示例紧凑，省略了显式返回。<sup>3</sup>本例中 `main` 的形式参数列表写为 `void`。实际上，也可以向 `main` 传入参数，这在许多场合很有用；不过本书大多数示例不使用这些参数，所以写成 `void`。

总结一下这些语法成分。函数声明告诉编译器函数的名称、期望接收的参数数量和类型，以及返回值类型。函数定义给出实际源代码，其形式参数列表说明形参名称及顺序。函数通过函数调用启动，调用中的圆括号列出输入值，即实参。各个实参的值赋给相应形参：第一个对应第一个，第二个对应第二个，依此类推。返回值是函数的输出，由被调用函数传回调用者。

### 14.2.2 示例：圆环面积

图 14.3 进一步演示函数语法。这个 C 程序计算从一个圆中去掉较小圆后的面积，也就是具有指定内外半径的圆环面积。程序通过一个函数，计算给定半径的圆的面积。`AreaOfCircle` 接收一个 `double` 实参，并向调用者返回一个 `double` 值。

---

<sup>3</sup>译注：原文声称非 `void` 函数没有显式返回值时，会把“最后一条语句的值”返回给调用者，这是错误的。`main` 的隐式返回 0 是特殊规则；其他非 `void` 函数若到达末尾而没有返回值，调用者又使用该次调用的值，则行为未定义。这里更正原说明。

```
1 #include <stdio.h>
2
3 // Function declarations
4 double AreaOfCircle(double radius);
5
6 int main(void)
7 {
8 double outer; // Inner radius
9 double inner; // Outer radius
10 double areaOfRing; // Area of ring
11
12 printf("Enter inner radius: ");
13 scanf("%lf", &inner);
14
15 printf("Enter outer radius: ");
16 scanf("%lf", &outer);
17
18 areaOfRing = AreaOfCircle(outer) - AreaOfCircle(inner);
19 printf("The area of the ring is %f\n", areaOfRing);
20 }
21
22 // Calculate area of circle given a radius
23 double AreaOfCircle(double radius)
24 {
25 double pi = 3.14159265; // This can be optionally declared as a const
26
27 return pi * radius * radius;
28 }
```

图 14.3 计算圆环面积的 C 程序

译注：原图第 8、9 行的 *Inner radius* 与 *Outer radius* 注释互换了；*outer* 是外半径，*inner* 是内半径。程序的输入和面积计算使用正确，图内原注释保留。

这里要再次强调：AreaOfCircle 执行时，可以“看见”并修改自己的局部变量 *pi* 和形参 *radius*，却不能直接修改 *main* 中的变量。这个例子的函数用法与前面的略有不同：*main* 多次调用 AreaOfCircle。它提供了一个有用的基本计算操作，将其封装为函数十分合适。

在更大规模的真实程序中，一个函数可能被几百甚至几千个地方调用。把 AreaOfCircle 这样的基本操作组织成函数，有可能减少源代码量，有利于维护，也使程序结构更清晰。与直接把面积公式写在原处相比，AreaOfCircle 更直观地表达了代码的意图。

## 14.3 C 函数的实现

我们已经遇到以及以后将遇到的每一种 C 语言特性，都必须转换为机器能够执行的形式，才能真正发挥作用。C 函数是 LC-3 机器层子程序在高级语言中的对应物，其实现机制与汇编层子程序相似。本节详细考察整个过程。

一次函数调用的执行有四个基本阶段：第一，将调用者的实参值传给被调用者；第二，将控制转移给被调用者；第三，被调用者执行任务；第四，将控制连同返回值一起交回调用者。C 要求函数独立于调用者，也就是说，一个函数应能够从任何函数中被调用。这样，函数就成为可以组合、可以在程序各处使用的基本构件。本节研究 LC-3 如何完成这些工作。

### 14.3.1 运行时栈

在汇编层面，函数就是用 JSR 调用的一串指令，RET 将控制交回调用者。这部分很简单。为了完成整个转换框架，还必须处理一些棘手的问题：实参怎样传递，返回值怎样送回，以及局部变量怎样分配空间。解决这些问题要用到数字系统体系结构的一项基础机制：**运行时栈**。

函数被调用时，需要一种办法将它“激活”。具体说，函数开始执行时，必须在存储器中为局部变量分配空间。可能的解决办法有很多，这里考察两种。

**方案 1：**编译器为每个函数的局部变量系统地指定固定存储位置。例如，为函数 A 指定从地址 X 开始的一块存储区，为 B 指定从 Y 开始的一块存储区，等等，当然这些区域不能重叠。这看似最直接，却有局限：如果函数 A 调用自己，会发生什么？这称为递归，我们在第 8.3 节已经见过，第 17 章还将专门讨论 C 的递归。如果 A 调用自己，新调用的 A 就会覆盖调用者 A 的局部变量，程序行为将不符合预期。C 允许递归函数，因此方案 1 不可行。它也浪费存储器：即使某个函数并未执行，仍要为其保留空间，而且可能占用很多空间。

**方案 2：**不静态地把局部变量安排在固定位置，而是在函数开始执行时才分配空间，如何？函数返回调用者时，回收这块空间，以后再分配给其他函数。如果函数调用自身，新的一次调用就获得独立的空间，与其他仍处于活动状态的调用相区别。这样就解决了方案 1 的问题。

怎样实现这种分配？可以利用函数调用模式容易用栈跟踪这一事实。每个函数都有一个存储布局模板，用来保存局部变量、一些管理信息，以及形参变量。后面将进一步讨论形参和管理信息。这个模板称为**栈帧**或**活动记录**。每次调用函数时，都要为它的栈帧分配存储空间。函数调用天然遵循类似栈的模式，因此空间的分配和释放也对应于栈的压入和弹出。

图 14.4 用一个例子说明这一点。程序包含 main、Watt 和 Volt 三个函数。每个函数具体做什么并不重要，所以省略了一些细节，只留下足以显示调用关系的代码。main 先调用 Watt，Watt 再调用 Volt；控制最终回到 main，然后它又调用 Volt。

```
1 int main(void)
2 {
3 int a;
4 int b;
5
6 :
7 b = Watt(a); // main calls Watt first
8 b = Volt(a, b); // then calls Volt
9 }
10
11 int Watt(int a)
12 {
13 int w;
14
15 :
16 w = Volt(w, 10); // Watt calls Volt
17
18 return w;
19 }
20
21 int Volt(int q; int r)
22 {
23 int k;
24 int m;
25
26 :
27 return k;
28 }
```

图 14.4 展示函数调用具有栈式特征的代码示例

译注：原图 `int Volt(int q; int r)` 中的分号应为逗号，即 `int Volt(int q, int r)`。这是一段省略了声明及部分执行语句的示意代码，原图保留。

图 14.5 显示了这一过程。每块阴影区域代表某次函数调用的活动记录，即栈帧。随着函数被调用并返回，运行时栈不断增长和收缩。记住，压栈时栈顶会向编号更小的存储器地址移动，也就是向低地址方向“增长”。

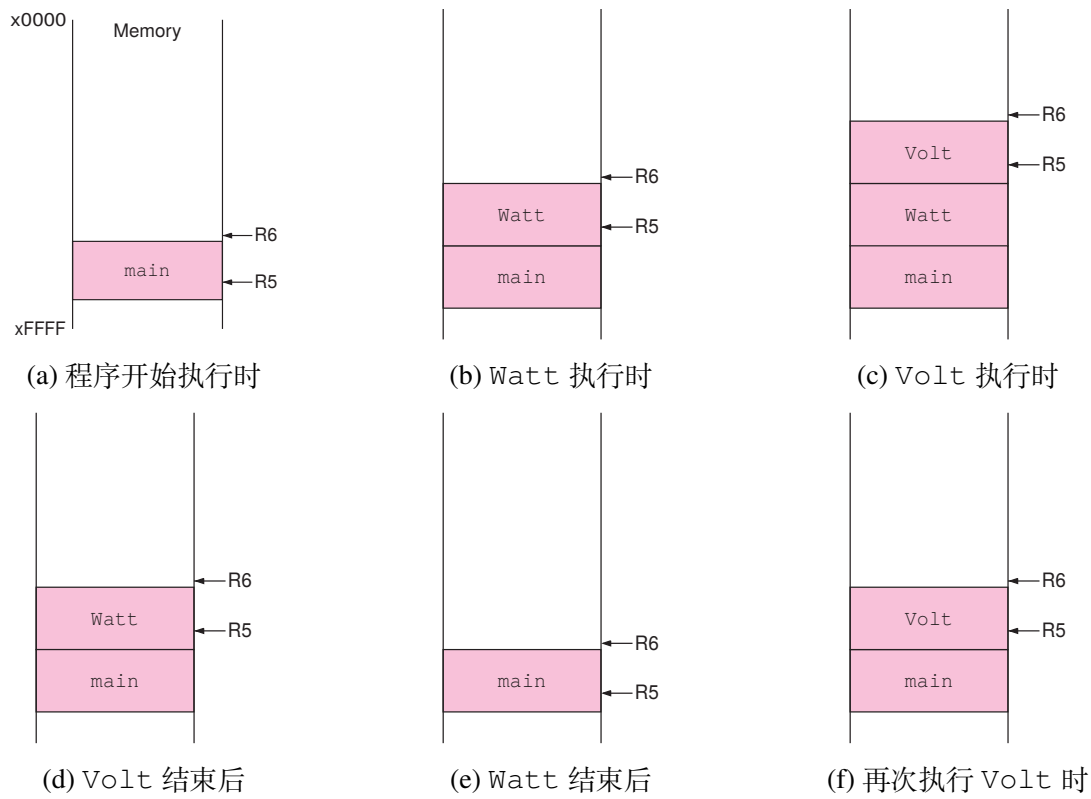


图 14.5 图 14.4 程序执行过程中，运行时栈在不同时刻的状态

图 14.5(a) 是程序刚开始执行时的运行时栈。由于 C 程序从 main 开始，首先为 main 分配栈帧。图 14.5(b) 是 main 调用 Watt 后的状态。注意，栈帧按栈的方式分配：每次调用函数，就把它栈帧压栈；每次函数返回，就把栈帧弹出。图中的 (c) 至 (f) 展示其余几个执行时刻的栈状态。

最终要把这个过程映射为 LC-3 汇编代码，所以需要方便地访问栈帧中的数据，并管理栈帧的压入和弹出。为此，我们使用 R5 和 R6。R5 指向栈顶活动栈帧内部的某个位置，即当前函数局部变量区的基址，称为**帧指针**。R6 始终指向栈顶，称为**栈指针**。这两个寄存器对于运行时栈乃至整个 C 函数机制的实现都很关键。

14.3.2 让各部分协同工作

现在已经很清楚，一次函数调用在机器层面涉及许多工作：传递实参、压入和弹出栈帧、在函数之间转移控制。虽然步骤不少，实现却相当直接。下面详细研究 LC-3 上的 C 运行时栈协议。

将协议分成四步。第 1 步，调用者把被调用者所需的实参复制到运行时栈，并转移控制。第 2 步，被调用者在运行时栈中为局部变量和其他信息分配空间，建立自己的栈帧。第 3 步，被调用者执行任务。第 4 步，准备返回时，被调用者拆除自己的栈帧，将返回值和控制交还调用者；剩余的返回值槽与实参由调用者清理。

通过下面这行代码的 LC-3 实现，逐步考察每个步骤。它是图 14.4 的第 16 行。

```
w = Volt(w, 10);
```



### 14.3.2.1 调用

这条语句用两个实参调用 Volt，然后将返回值赋给局部整数变量 w。编译器为这次调用生成的 LC-3 代码完成两项工作：

1. 将两个实参值直接压到运行时栈顶，传给 Volt。R6 是栈指针，保存当前栈顶数据所在单元的地址。压入一个数据项时，先将 R6 减 1，再以 R6 为基址存入数据。本书的栈协议按照实参在调用中出现的顺序，从右向左压栈。所以 Watt 先压入最右边的 10，再压入 w 的值。
2. 通过 JSR 将控制转移给 Volt。

译注：这里的从右向左是本书 LC-3 调用协议规定的压栈顺序，不应推断为 C 语言规定了实参表达式的求值顺序。

下面的 LC-3 代码实现从 Watt 调用 Volt。第 1-4 行将实参 10 压栈，第 6-8 行将 w 的值压栈。w 本身是 Watt 的局部变量，必须从它的栈帧中加载，稍后继续讨论。

```
1 AND R0, R0, #0 ; R0 <- 0
2 ADD R0, R0, #10 ; R0 <- 10
3 ADD R6, R6, #-1 ;
4 STR R0, R6, #0 ; Push 10 onto stack
5
6 LDR R0, R5, #0 ; Load w
7 ADD R6, R6, #-1 ;
8 STR R0, R6, #0 ; Push w
9 JSR Volt
```

图 14.6 显示这些指令对运行时栈的修改。实参紧接在调用者 Watt 的栈帧上方压入。被调用者 Volt 的栈帧，也将在调用者栈帧的上方建立。

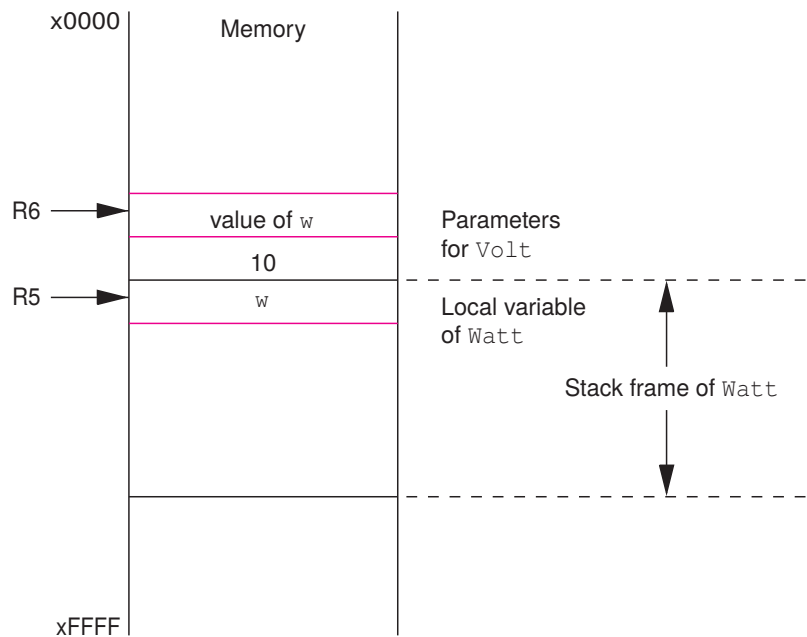


图 14.6 Watt 将传给 Volt 的实参压栈后的运行时栈

14.3.2.2 启动被调用函数

Watt 中 JSR 之后实际执行的下一条指令，是被调用函数 Volt 的第一条指令。在执行 Volt 的 C 代码所描述的工作之前，需要先准备运行环境：正确分配栈帧，设置栈指针 R6 和帧指针 R5。

此时，栈顶已经有 Volt 的两个实参。Volt 的序言代码 (preamble code) 要继续准备栈，使其中包含返回值的位置、返回 Watt 的地址、Watt 的帧指针，以及 Volt 的所有局部变量。分为四项操作：

- 1. 为被调用者的返回值保留空间。返回值位于形参区紧上方；准备返回时，再将结果写入这个位置。
- 2. 将 R7 中的返回地址压栈。JSR 发起子程序调用时，R7 保存返回地址。将它保存起来，就能保证被调用者即使继续调用其他函数，也能够回到原调用者。
- 3. 将 R5 中调用者的帧指针压栈。保存 R5，才能在函数调用完成后恢复调用者的栈帧访问环境，使它继续执行。如果返回地址或帧指针遭到破坏，就难以正确恢复调用者，所以必须把两者都备份到存储器中。
- 4. 为被调用者的局部变量分配栈空间，调整 R5，使其指向局部变量区基址；调整 R6，使其指向栈顶。

下面是 Volt 对应的序言代码，位于它的其他语句所对应的机器代码之前。

```
1 Volt: ADD R6, R6, #-1 ; Allocate spot for the return value
2
3 ADD R6, R6, #-1 ;
4 STR R7, R6, #0 ; Push R7 (Return address)
5
6 ADD R6, R6, #-1 ;
7 STR R5, R6, #0 ; Push R5 (Caller's frame pointer)
8
9 ADD R5, R6, #-1 ; Set frame pointer for Volt
10 ADD R6, R6, #-2 ; Allocate memory for Volt's local variables
```

图 14.7 汇总了到目前为止，函数调用代码对存储器所作的修改。Watt 和 Volt 两个栈帧的布局清楚可见。注意，Volt 栈帧的一些内容由 Watt 写入：形参槽使用 Watt 的实参值初始化。Watt 把局部变量 w 的值作为第一个实参，把 10 作为第二个实参。从右向左压栈，所以 w 的值在 10 上方。Volt 启动后，通过形参 q 和 r 引用这两个值。

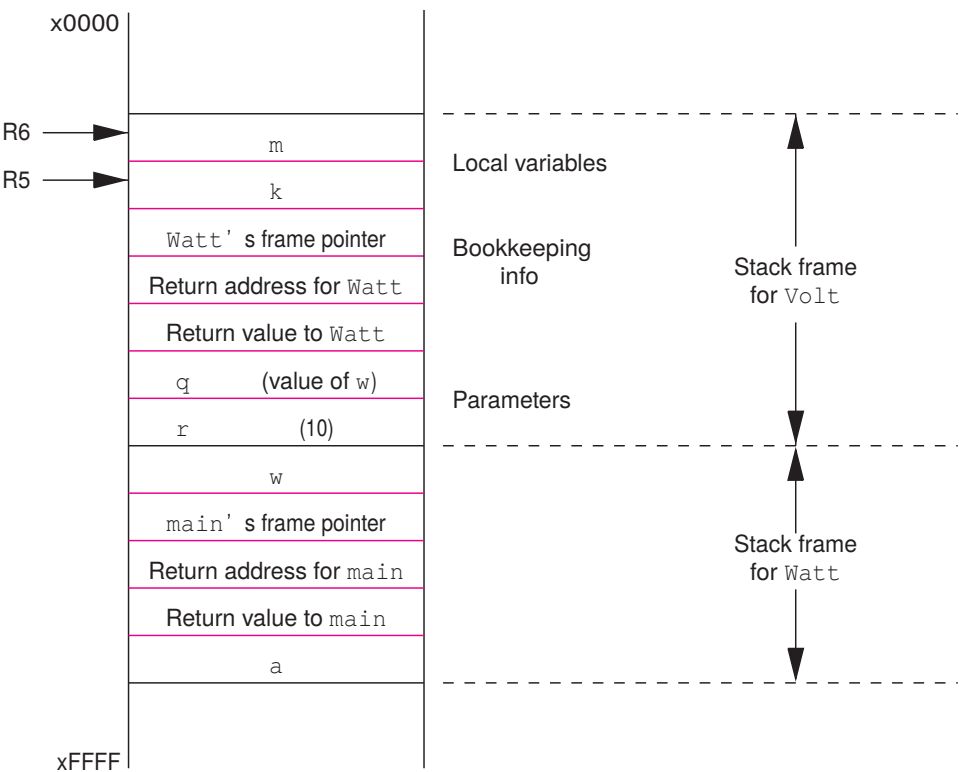


图 14.7 Volt 的栈帧压栈后的运行时栈

问题：Volt 的局部变量初值是什么？回忆第 12 章，这类没有初始化器的自动局部变量未被初始化。<sup>4</sup>

注意，各栈帧具有相同的基本结构：局部变量区、管理信息区、返回值槽，以及形参区。管理信息包括调用者的返回地址和帧指针。

<sup>4</sup>译注：原文此处引用第 11 章，实际关于局部变量初值的讨论在第 12 章，这里更正。

### 14.3.2.3 结束被调用函数

被调用函数完成工作后，准备返回调用者。这主要涉及拆除栈帧，并恢复运行时栈的状态。需要以下四项操作：

1. 如果有返回值，将其写入当前活动栈帧的返回值槽。
2. 弹出被调用者的局部变量。既可以逐个弹出，也可以直接将 R6 设为紧接帧指针 R5 后面的地址，即 R5+1。
3. 恢复调用者的帧指针和返回地址。
4. 通过 RET 将控制交回调用者。

Volt 对应的 LC-3 指令如下。记住，栈帧虽然已被弹出，原来的值仍留在存储器中，直到被明确覆盖。

```

1 LDR R0, R5, #0 ; Load local variable k
2 STR R0, R5, #3 ; Write it in return value slot, which will always
3 ; be at location R5 + 3
4
5 ADD R6, R5, #1 ; Pop local variables
6
7 LDR R5, R6, #0 ; Pop the frame pointer
8 ADD R6, R6, #1 ;
9
10 LDR R7, R6, #0 ; Pop the return address
11 ADD R6, R6, #1 ;
12 RET

```

### 14.3.2.4 返回调用函数

被调用者执行 RET 后，控制回到调用者。有时函数声明为 void，没有返回值；有时调用者会忽略返回值。在前面的例子中，返回值要赋给 Watt 中的 w。这里还有两项操作：

1. 从栈中取出返回值，如果存在，并弹出对应槽位。
2. 将实参弹栈。

JSR 之后的代码如下。开始执行返回后的代码时，R6 指向栈顶的返回值槽，R5 则已经恢复为 Watt 的帧指针。

```

1 JSR Volt
2 LDR R0, R6, #0 ; Load the return value
3 STR R0, R5, #0 ; w = Volt(w, 10);
4 ADD R6, R6, #1 ; Pop return value
5
6 ADD R6, R6, #2 ; Pop arguments

```

执行完这些代码，整次调用就结束了，调用者可以继续正常执行。注意，被调用者在返回前恢复了调用者的环境。在调用者看来，除了栈上出现返回值以外，它所需的调用环境已恢复；随后再完成栈清理。

#### 14.3.2.5 调用者保存与被调用者保存

结束函数实现的讨论前，还需要说明一个此前暂未处理的问题。函数执行期间，R0–R3 可能保存正在进行的计算所需的临时值。R4–R7 已有专门用途：R4 指向全局数据区，R5 是帧指针，R6 是栈指针，R7 保存返回地址。根据前面的调用约定，调用函数时，R4–R7 要么不变，要么按预先规定的方式变化。那么 R0、R1、R2、R3 怎么办？数字系统通常采用以下两种策略之一。

1. **调用者保存**。调用者将需要保存的寄存器压入自己的栈帧，返回后再弹栈恢复。这就是调用者保存约定，第 8.1.1 节也讨论过。
2. **被调用者保存**。被调用者在活动记录的管理信息区中增加四个槽位，保存这些寄存器。启动时，除保存 R5 和 R7 外，也保存 R0–R3；返回前恢复它们。这称为被调用者保存约定。

译注：增加寄存器保存槽位后，必须相应安排栈帧偏移。下文图 14.8 展示的是此前的基本调用框架，并未加入 R0–R3 的保存与恢复代码。

#### 14.3.3 将各部分整合起来

图 14.8 汇集了 Watt 中的函数调用，以及 Volt 的开头和结尾代码，展示整体结构。与前面逐段讲解的版本相比，这里做了一点优化：把返回后为弹出返回值和实参而进行的 R6 调整合并为一条指令。<sup>5</sup>

<sup>5</sup>译注：原文笼统说合并了与返回值压入、弹出有关的栈指针操作；图中实际体现的合并是调用者的 ADD R6, R6, #3，一次弹出返回值和两个实参。

```

1 Watt:
2 ...
3 AND R0, R0, #0 ; R0 <- 0
4 ADD R0, R0, #10 ; R0 <- 10
5 ADD R6, R6, #-1 ;
6 STR R0, R6, #0 ; Push 10 onto stack
7
8 LDR R0, R5, #0 ; Load w
9 ADD R6, R6, #-1 ;
10 STR R0, R6, #0 ; Push w
11 JSR Volt
12
13 LDR R0, R6, #0 ; Load the return value
14 STR R0, R5, #0 ; w = Volt(w, 10);
15 ADD R6, R6, #3 ; Pop return value and arguments
16 ...
17
18 Volt: ADD R6, R6, #-1 ; Allocate spot for the return value
19
20 ADD R6, R6, #-1 ;
21 STR R7, R6, #0 ; Push R7 (Return address)
22
23 ADD R6, R6, #-1 ;
24 STR R5, R6, #0 ; Push R5 (Caller's frame pointer)
25
26 ADD R5, R6, #-1 ; Set frame pointer for Volt
27 ADD R6, R6, #-2 ; Allocate memory for Volt's local variables
28
29 ... ; Volt performs its work
30
31 LDR R0, R5, #0 ; Load local variable k
32 STR R0, R5, #3 ; Write it in return value slot, which will always
33 ; be at location R5 + 3
34
35 ADD R6, R5, #1 ; Pop local variables
36
37 LDR R5, R6, #0 ; Pop the frame pointer
38 ADD R6, R6, #1 ;
39
40 LDR R7, R6, #0 ; Pop the return address
41 ADD R6, R6, #1 ;
42 RET

```

图 14.8 C 函数调用与返回所对应的 LC-3 代码

总结一下本书的 LC-3 C 函数调用协议。调用者把各个实参值压栈，再通过 JSR 跳转到被调用者。被调用者为返回值分配空间，保存调用者的帧指针和返回地址，然后为局部变量分配空间。完成任务后，将返回值写入预留位置，弹出局部变量并恢复调用者的帧指针和返回地址，再返回调用者。<sup>6</sup>最后，调用者弹出返回值和实参，继续执行。

<sup>6</sup>译注：原文此处写“恢复帧指针和返回值”，根据前述协议和图 14.8，应为“帧指针和返回地址”，这里更正。

你也许会想，仅仅调用一个函数，为什么要经过这么多步骤？这些代码真的都必要吗，调用约定能不能更简单？真实调用约定的一项重要特征是：在一般情形下，任何函数都应能够调用其他函数。为此，调用者除了被调用者的接口，即返回值类型以及实参类型之外，不需要知道其内部细节；被调用者也应独立于调用它的函数而编写。正是这种通用性，使调用协议需要这些步骤。这些开销是让函数成为大规模软件基本构件所需的一部分代价。

## 14.4 使用函数求解问题

要让函数真正为我们所用，就需要把它纳入编程问题的求解方法。本节通过两个例子，展示两种略有不同的函数使用方式。

从概念上说，在自顶向下地将问题设计成算法时，函数是很合适的划分单位。分解问题的过程中，算法要完成的任务中会自然出现一些“部件”，它们就是函数的候选。第一个例子把文本中的小写字母转换为大写字母，展示如何从自顶向下设计中自然地识别出这样的函数部件。

函数也适合封装程序中多个位置都需要的基本操作。从某种意义上说，这相当于扩充语言的操作集合，使其适应手头的问题。第二个例子求勾股数组，我们将构造一个计算  $x^2$  的基本函数来辅助计算。

### 14.4.1 问题 1：大小写转换

下面开发一个从键盘读取输入并回显到屏幕的程序，类似于图 13.8，但增加一点变化：在回显前，将小写字母转换为大写字母。

以图 13.8 的回显程序为起点。原程序用 `while` 循环从键盘读取一个字符，再将它打印到输出设备。现在添加一个部件，检查字符是否为小写字母；如果是，就转成大写。可以把相应代码直接写入 `while` 循环，但这个部件的功能相对独立，因此选择将它写成函数。

每次从键盘读入字符之后、显示字符之前，调用这个转换函数。它接收一个字符实参：如果字符已经是大写字母，或者不是字母，就返回原字符；如果是小写字母，就返回相应大写字母。图 14.9 给出了程序流程。阴影部分是原回显程序的流程图，我们在其中增加了完成转换的函数部件。

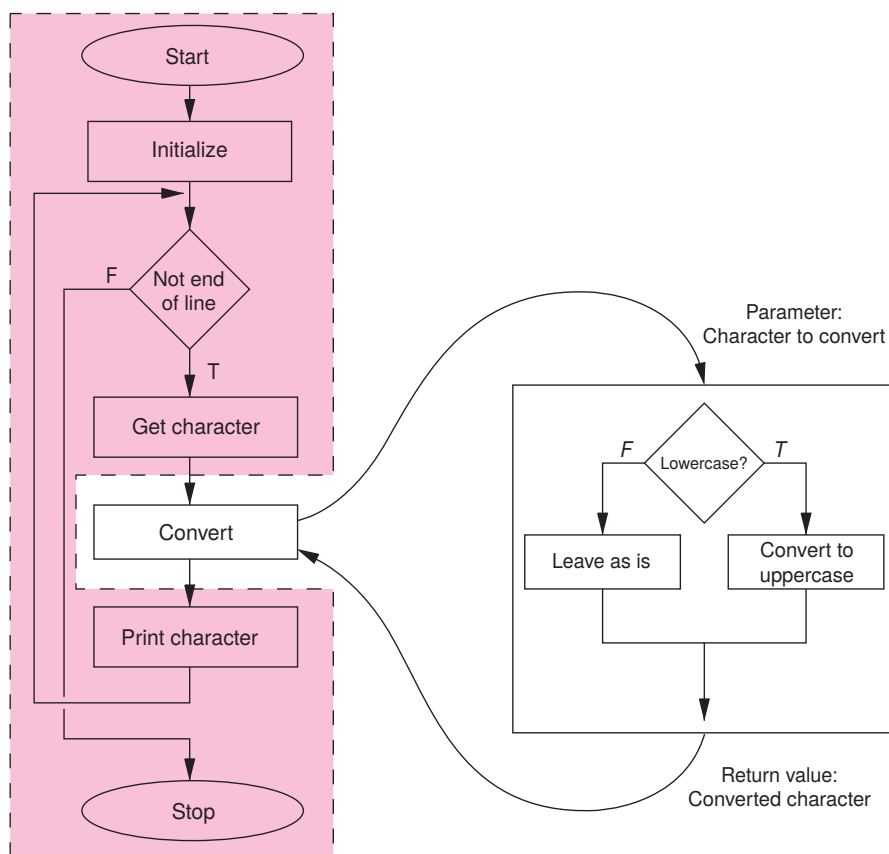


图 14.9 将输入字符转换为大写的程序分解为较小子任务

完整的 C 代码见图 14.10。程序从键盘接收输入，转换字母大小写，再打印结果；输入字符为换行符时，程序终止。转换由 `ToUpper` 完成。注意，函数体用字符字面量完成实际转换。单引号中的字符，如 `'A'`，在本书使用的 ASCII 编码中表示该字符的 ASCII 值。因此，`'a' - 'A'` 就是小写 `a` 的 ASCII 值减去大写 `A` 的 ASCII 值。



```
1 #include <stdio.h>
2
3 char ToUpper(char inchar); // Function Declaration
4
5 // Prompt for a line of text, Read one character,
6 // convert to uppercase, print it out, then get another
7 int main(void)
8 {
9 char echo = 'A'; // Initialize input character
10 char upcase; // Converted character
11
12 printf("Type input: ");
13 while (echo != '\n') {
14 scanf("%c", &echo);
15 upcase = ToUpper(echo);
16 printf("%c", upcase);
17 }
18 }
19
20 // If the parameter is lowercase, return
21 // its uppercase ASCII value
22 char ToUpper(char inchar)
23 {
24 char outchar;
25
26 if ('a' <= inchar && inchar <= 'z')
27 outchar = inchar - ('a' - 'A');
28 else
29 outchar = inchar;
30
31 return outchar;
32 }
```

图 14.10 使用函数将小写字母转换为大写字母的程序

#### 14.4.2 问题 2: 勾股数组

现在尝试找出给定输入上限以内的所有**勾股数组** (Pythagorean triple)。它是一组三个正整数  $a$ 、 $b$ 、 $c$ ，满足

$$c^2 = a^2 + b^2.$$

换言之,  $a$  和  $b$  是直角三角形的两条直角边,  $c$  是斜边。例如, 3、4、5 就是一组勾股数。本题要计算所有三个分量都不超过用户给定上限的勾股数组。<sup>7</sup>

这个问题采用**穷举法** (brute force)。如果用户给出的上限是  $\text{max}$ , 就检查上限以内三个整数的所有组合, 判断它们是否满足勾股关系。为检查所有组合, 要让  $\text{sideA}$ 、 $\text{sideB}$ 、 $\text{sideC}$  分别从 1 变化到上限。这意味着使用计数器控制循环: 用一个 `for` 改变  $\text{sideC}$ , 内部再嵌套改变  $\text{sideB}$  的循环, 最内层再嵌套改变  $\text{sideA}$  的循环。三层循环的核心检查三个数是否满足条件; 如果满足, 就打印它们。

检查勾股关系时, 要求下列表达式的值:

```
(sideC * sideC == (sideA * sideA + sideB * sideB))
```

平方是这个问题中多处需要的基本操作, 所以将它封装为函数 `Squared`, 返回整数形参的平方。这样, 前面的表达式可以改写为:

```
(Squared(sideC) == Squared(sideA) + Squared(sideB))
```

这种写法更清楚地表达了计算意图。完整程序见图 14.11。

```
1 #include <stdio.h>
2
3 int Squared(int x);
4
5 int main(void)
6 {
7 int maxC;
8
9 printf("Enter the maximum length of hypotenuse: ");
10 scanf("%d", &maxC);
11
12 for (int sideC = 1; sideC <= maxC; sideC++) {
13 for (int sideB = 1; sideB <= maxC; sideB++) {
14 for (int sideA = 1; sideA <= maxC; sideA++) {
15 if (Squared(sideC) == Squared(sideA) + Squared(sideB))
16 printf("%d %d %d\n", sideA, sideB, sideC);
17 }
18 }
19 }
20 }
21
22 // Calculate the square of a number
23 int Squared(int x)
24 {
25 return x * x;
26 }
```

图 14.11 计算勾股数组的 C 程序

<sup>7</sup>译注: 原文多次写“小于”上限, 而图 14.11 的三层循环均使用 `<= maxC`, 包含上限。这里按程序统一为“不超过”。

当然，还有比检查所有组合更好的方法。你能修改代码，让它执行得更高效吗？这里使用穷举法，是因为它适合展示函数的用法。

## 14.5 小结

本章介绍了 C 中的函数。函数这类子程序的思想，从程序设计语言的早期就已存在。函数使我们能够构造适用于某项任务或多种任务的新基本构件，相当于用新的基本函数扩充语言原有的操作和结构。本章的主要概念如下。

- **C 函数的语法。**使用函数前，要向编译器提供函数声明，通常放在代码前部，说明函数名、返回值类型，以及输入参数的类型和顺序。函数定义包含实际代码。执行函数调用时启动该函数，调用中列出的实参就是要传给函数的值。
- **C 函数的底层实现。**实现的部分复杂性来自函数之间可以相互调用，甚至可以从其他目标文件中的函数调用。为此，我们采用通用调用约定。为支持函数调用自身，这个约定以运行时栈为基础：调用者将实参值压栈并调用函数，这些实参成为被调用者栈帧中的形参；被调用者执行任务，拆除栈帧的相应部分，为调用者留下返回值。
- **编程中使用函数。**也可以不把程序进一步拆分为函数，将代码都堆在 `main` 中，但那样的代码难以阅读、维护和扩展，也更容易出错。函数支持抽象：可以为特定任务编写函数，完成调试和测试，再在程序中所有需要的地方使用它。

## 习题

**14.1** `main` 函数有什么特殊意义？为什么程序必须包含这个函数？

**14.2** 参考栈帧结构，回答下列问题。

- a. 帧指针的作用是什么？
- b. 返回地址的作用是什么？
- c. 返回值的作用是什么？

**14.3** 参考 C 函数语法，回答下列问题。

- a. 什么是函数声明？它有什么作用？
- b. 什么是函数原型？
- c. 什么是函数定义？
- d. 什么是实参？
- e. 什么是形参？

**14.4** 对于下列各项，指出由调用者还是被调用者执行。

- a. 将实参写入栈帧。
- b. 写入返回值。
- c. 写入帧指针。
- d. 修改 R5，使其指向被调用者栈帧内部。

**14.5** 下面程序的输出是什么？请解释。

```
void MyFunc(int z);

int main(void)
{
 int z = 2;
 MyFunc(z);
 MyFunc(z);
}

void MyFunc(int z)
{
 printf("%d ", z);
 z++;
}
```

译注：原题未列出 *stdio.h*；作为独立程序编译时，应补充相应头文件以声明 *printf*。

**14.6** 下面程序的输出是什么？

```
#include <stdio.h>
int Multiply(int d, int b);

int d = 3;
int main()
{
 int a = 1, b = 2, c;
 int e = 4;

 c = Multiply(a, b);
 printf("%d %d %d %d %d\n", a, b, c, d, e);
}

int Multiply(int d, int b)
{
 int a = 2;
 b = 3;
 return (a * b);
}
```

**14.7** 下面是名为 Bump 的 C 函数。

```
int Bump(int x)
{
 int a;
 a = x + 1;
 return a;
}
```

- a. 画出 Bump 的栈帧。
- b. 在栈帧每个槽位中写入下列一项，说明其中保存的内容：
  - (1) 局部变量。
  - (2) 实参。
  - (3) 指令地址。
  - (4) 数据地址。
  - (5) 其他。
- c. Bump 栈帧中的一些槽位由调用它的函数写入，另一些由 Bump 自己写入。指出由 Bump 写入的槽位。

**14.8** 下面代码的输出是什么？解释 Swap 为什么具有这种行为。

```
void Swap(int x, int y);
int main(void)
{
 int x = 1;
 int y = 2;

 Swap(x, y);
 printf("x = %d y = %d\n", x, y);
}

void Swap(int y, int x)
{
 int temp;

 temp = x;
 x = y;
 y = temp;
}
```

译注：原题 `int temp` 后缺少分号，这里补上。作为独立程序编译还应包含 `stdio.h`。声明中的形参名为 `x, y`，定义中的形参名为 `y, x`，二者均按原文保留。

**14.9** 函数实参是在执行调用它的 JSR 之前，还是之后放入栈中？为什么？

**14.10** 一个含有 `food` 函数的 C 程序已被编译为 LC-3 汇编。该函数的部分代码如下。

```
food:
 ADD R6, R6, #-2 ;
 STR R7, R6, #0 ;
 ADD R6, R6, #-1 ;
 STR R5, R6, #0 ;
 ADD R5, R6, #-1 ;
 ADD R6, R6, #-4 ;
 ...
```

- a. 这个函数有多少个局部变量？
- b. 假设它有两个整数形参 `x` 和 `y`，生成计算表达式 `x + y` 的代码。

**14.11** 下面给出了包含 `Unit` 函数的 C 程序。

```
int Init(int x);
int Unit(int x);
int main()
{
 int a = 1;
 int b = 2;

 a = Init(a);
 b = Unit(b);
 printf("a = %d b = %d\n", a, b);
}

int Init(int x)
{
 int y = 2;
 return y + x;
}

int Unit(int x)
{
 int z;
 return z + x;
}
```

- a. 程序的输出是什么?
- b. `Unit` 开始执行时, 什么决定了局部变量 `z` 的值?

译注: 原题在 `Unit` 中读取了未初始化的自动局部变量 `z`, 不能据 C 语法规则确定输出。按本章具体的栈布局, 可讨论栈空间复用留下的位模式, 但不能把这种机器层现象视为语言保证。原题保留; 独立编译时还需补充 `stdio.h`。

- 14.12** 修改图 14.10 的示例, 使它还能够将各字符转换为小写。新程序应同时打印每个输入字符的小写形式和大写形式。
- 14.13** 编写函数, 将一个整数用四进制打印出来, 只使用数字 0、1、2、3。利用这个函数编写程序, 从键盘读取两个整数, 在屏幕上以四进制显示两个数及其和。
- 14.14** 编写函数: 如果第一个整数形参能被第二个整除, 就返回 1。利用此函数编写程序, 找出能被所有小于 10 的正整数整除的最小正整数。<sup>8</sup>

---

<sup>8</sup>译注: 原题未限定正整数; 若把 0 也作为除数, 问题就没有意义。这里明确通常的正整数范围, 不改变求最小公倍数的题意。

- 14.15** 下面的 C 程序被编译为 LC-3 机器语言，并在执行前装入从 x3000 开始的位置。不计调用 I/O 库例程的 JSR，目标代码中共有三条 JSR，分别调用 f、g 和 h。假设这三条指令的地址分别为 x3102、x3301 和 x3304，用户输入 4、5、6。画出程序即将从 f 返回时的运行时栈，尽可能填出各存储位置的内容。假设运行时栈底为 xEFFF。

```
#include <stdio.h>
int f(int x, int y, int z);
int g(int arg);
int h(int arg1, int arg2);

int main(void)
{
 int a, b, c;

 printf("Type three numbers: ");
 scanf("%d %d %d", &a, &b, &c);
 printf("%d", f(a, b, c));
}

int f(int x, int y, int z)
{
 int x1;

 x1 = g(x);
 return h(y, z) * x1;
}

int g(int arg)
{
 return arg * arg;
}

int h(int arg1, int arg2)
{
 return arg1 / arg2;
}
```

- 14.16** 再次考虑前几章的机器忙闲状态问题。用一个位模式表示 16 台机器的状态：某位为 0 表示对应机器忙，为 1 表示对应机器空闲。

- 编写函数，根据给定忙闲位模式统计忙碌机器数。输入为位模式，可以用整数变量表示；输出为表示忙碌机器数的整数。
- 编写函数，接收两个忙闲位模式，确定哪些机器的状态发生变化，即由忙变为空闲，或由空闲变为忙。输出另一个位模式，每个发生变化的机器所对应的位为 1。
- 编写程序，从键盘读取由十个忙闲位模式组成的序列，计算平均忙碌机



器数，以及相邻两个位模式之间平均有多少台机器改变状态。用户输入全 1 位模式，即所有机器空闲，表示输入结束。使用 a、b 小题编写的函数完成程序。

译注：原题同时规定“十个”输入和“全 1 结束”，但没有说明两者的优先关系，以及结束标志是否参与统计。这里保留原要求，作答时应明确采用的输入与统计约定。

- 14.17**    a. 编写 C 函数，模拟 4 选 1 多路选择器的行为。参见图 3.13。  
            b. 编写 C 函数，模拟 LC-3 ALU 的行为。
- 14.18** 电话数字键盘上标有 2、3、4、...、9 的键还对应着字母。例如，2 对应 A、B、C。编写程序，把一个七位电话号码映射为它能够表示的所有字符序列。使用一个函数完成数字与字符之间的映射。数字 1 和 0 不对应任何字符。

**14.19** 下面的 C 程序同时使用了全局变量和不同作用域中的局部变量。输出是什么？

```
#include <stdio.h>

int t = 1; // Global variable

int sub1(int fluff);

int main (void)
{
 int t = 2;
 int z;
 z = t;
 z = z + 1;
 printf("A: The variable z equals %d\n", z);
 {
 z = t;
 t = 3;
 {
 int t = 4;
 z = t;
 z = z + 1;
 printf("B: The variable z equals %d\n", z);
 }
 z = sub1(z);
 z = z + 1;
 printf("C: The variable z equals %d\n", z);
 }
 z = t;
 z = z + 1;
 printf("D: The variable z equals %d\n", z);
}

int sub1(int fluff)
{
 int i;
 i = t;
 return (fluff + i);
}
```

# 第十五章 测试与调试

## 15.1 引言

1999 年 12 月，NASA 的任务控制人员在火星极地着陆器接近火星表面时与它失去了联系。这艘探测器的任务是研究这颗红色星球的南极地区。此后，通信始终未能恢复，NASA 随后宣布，探测器很可能在着陆过程中坠毁。调查人员评估后认为，最可能的原因是控制软件存在错误：探测器还在距地面 40 米的空中，发动机就提前关闭了，而不是等到真正着陆之后才关闭。据推测，软件把着陆架展开时产生的振动误判为接触地面的信号，从而触发了推进器关闭。

就在几个月前，NASA 的另一项火星任务——火星气候轨道器——也因另一种软件错误在进入轨道时失败。控制轨道器推进器的一个地面软件子系统使用英制单位进行计算，而航天器其他系统要求使用公制单位，结果使轨道器飞上了灾难性的轨迹。这个臭名昭著的错误最终导致一艘价值 6.55 亿美元的探测器损失。<sup>1</sup>

把探测器送入太空所涉及的物理问题极其复杂，控制航天器的软件系统也同样复杂。软件与任何机械或电气子系统一样，都是整个系统不可缺少的部分；而且由于软件“看不见”，要保证它正确往往更困难。与推进系统或着陆系统不同，软件难以同样直观地观察或定量测量。软件在数字世界中发挥着至关重要的作用，这是显而易见的。甚至打开水龙头这样看起来与数字技术无关的事情，也需要执行数十万行代码，跟踪水的输送以及流入水杯的过程。

既然软件如此重要，就必须让它按照规定的需求正确运行。写出能够正常工作的代码并不是自然而然的事。程序不会因为被编写出来就自动正确，尤其是 C 或 C++ 程序。我们必须主动开展测试和调试，并把它们作为开发过程不可分割的一部分。

根据经验，有阅历的程序员用于调试的时间常常超过总工作时间的一半，多于真正编写代码的时间。优秀的程序员通常也精通调试。理解测试和调试的概念，与理解编程的基本思想同样重要。本章介绍测试和调试的基本观念，帮助你掌握有效编程所必需的技能。

**测试** (testing) 是暴露错误的过程，**调试** 则是修复错误的过程。测试一段代码，就是尽可能用多种输入条件考验它，促使隐藏的错误显现出来。例如，测试第 14 章的 `ToUpper` 函数时，可以把每个可能的 ASCII 值都作为实参传入，再观察输出，判断它是否符合规格要求。这个函数将作为实参传入的字母转换为大写形式。如果某个输入产生了错误的输出，我们就发现了一个错误。在开发阶段发现错误，总比毫无防备的用户在使用时偶然撞上它要好。NASA 的软件工程师若能在地球表面发现火星极地着陆器的错误，显然好过在火星地面上方 40 米处才遇到它。

程序员可以利用有关程序及其执行过程的信息，运用常识推断哪里出了问题。正

---

<sup>1</sup>译注：金额按原书保留，本译稿未对该历史金额作独立核验。

如侦探通过检查证据破案，程序员也必须审视现有线索，追查错误的根源。如果能够有条理地收集程序的信息，例如执行期间关键变量的值，调试就会容易得多。

本章介绍几种查找和修复程序错误的方法。首先说明程序中可能出现的几大类错误，然后介绍尽快发现这些错误的测试方法，接着讨论定位和修复错误的调试技术，最后给出一些防御式编程方法，以减少你编写的代码中的错误。

## 15.2 错误的类型

要更好地理解如何查找、修复乃至避免代码中的错误，先建立一个分类体系会很有帮助。程序员引入代码的错误大致分为四类。**语法错误**最容易处理，因为编译器能够发现它们。在把源代码翻译为机器代码时，编译器会报告这些错误，而且常常能准确指出出错的行。**语义错误**则往往难以修复：程序在语法上正确，行为却并不完全符合我们的意图。这里所说的语法错误和语义错误通常源于书写失误，也就是写下了并非自己本意的内容。**算法错误**不是这样的偶然疏忽，而是解决问题的方法本身出了错。它们难以发现，也常常难以修复，因此会给整个软件开发过程带来很高的代价。**规格错误**则源于程序需求定义不清，或实现需求的个人、团队对它们产生了误解。

从整个数字系统来看，错误可能出现在任何地方，并不限于软件。硬件也会有错误，传感器可能不可靠，物理系统也可能失效，这些都会表现为设备不能正常工作。任何一个子系统还可能因为不利的工作条件而短暂失效：设备只出错一瞬间，随后又恢复正常。保证整个系统健壮且没有错误十分重要；作为系统设计者，我们需要一套良好的测试和调试方法。现在先把注意力放在软件上。你会发现，这里形成的许多思想也适用于硬件测试和调试。

本章还不讨论安全漏洞。许多数字设备保存和处理个人及私密信息，设计要求它们保障这些信息的安全，尤其要防范恶意攻击者。为满足这些要求，硬件和软件都必须经过精心设计和测试，建立抵御攻击的基本安全防线。尽管如此，安全漏洞仍然存在，也曾被利用。其中一些，例如 Heartbleed、Meltdown、Spectre 等，广为人知并造成了昂贵的损失。本章关注一般的软件错误，重点是测试与调试的基础。系统安全是值得在后续深入学习的重要主题。

### 15.2.1 语法错误

在 C、C++ 以及其他编程语言中，**语法错误**（syntactic error，也称 syntax error 或 parse error）是由编译器或其他翻译程序检测的错误。请求编译器翻译不符合语言语法规则的代码时，就会出现这类错误。例如，图 15.1 的代码有一个语法错误，编译器会在编译时指出它：变量 `i` 的声明后缺少分号。<sup>2</sup>对正在学习新语言的人来说，语法错误会占相当大的比例。好消息是，它们通常容易发现，因为编译器会检测；修复起来也比较简单。编译图 15.1 时，编译器会报告语法错误，并指出无法继续正确分析的位置。有时报告比较晦涩，因为编译器也会被错误扰乱，无法精确指出根源。

<sup>2</sup>译注：原书正文写成变量 `j`，但图 15.1 第 5 行缺少分号的是 `i` 的声明，这里更正。

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5 int i
6 int j;
7
8 for (i = 0; i <= 10; i++) {
9 j = i * 7;
10 printf("%d x 7 = %d\n", i, j);
11 }
12 }
```

图 15.1 这个程序含有一个语法错误。

### 15.2.2 语义错误

**语义错误** (semantic error) 与语法错误有相似之处：写代码时，大脑和手指没有完全协调一致，都会导致这类失误。但语义错误不涉及错误的语法，所以程序仍然能够完成翻译并执行。直到分析输出时，我们才发现程序没有按预期工作。

图 15.2 给出了与图 15.1 基本相同的程序。语法错误已经修复，却引入了一个简单的语义错误。这个程序本来应该打印 7 的乘法表，而只要运行一次就会发现问题：它只打印了乘法表中的一项。

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5 int i;
6 int j;
7
8 for (i = 0; i <= 10; i++)
9 j = i * 7;
10 printf("%d x 7 = %d\n", i, j);
11 }
```

图 15.2 一个含有语义错误的程序

凭借已经掌握的 C 知识，你应该能够推断出程序为什么运行不正确。为什么输出只有下面这一行？

```
11 x 7 = 70
```

这个例子展示了**控制流错误** (control-flow error): 程序的控制流, 也就是语句执行的顺序, 与我们的本意不同。

图 15.3 的代码也含有语义错误。它本来要计算从 1 到键盘输入的数为止所有整数的和, 也就是  $1 + 2 + 3 + \dots + n$ , 却有一个常见而隐蔽的语义错误。运行后你会发现, 输出不一定是预期的结果。问题出在 AllSum 的变量 result: 它没有初始化。这是一种**初始化错误** (initialization error)。

```
1 #include <stdio.h>
2
3 int AllSum(int n);
4
5 int main(void)
6 {
7 int in; // Input value
8 int sum; // Value of 1+2+3+...+ n
9
10 printf("Input a number: ");
11 scanf("%d", &in);
12
13 sum = AllSum(in);
14 printf("The AllSum of %d is %d\n", in, sum);
15 }
16
17
18 int AllSum(int n)
19 {
20 int result; // Result to be returned
21 int i; // Iteration count
22
23 for (i = 1; i <= n; i++) // This calculates sum
24 result = result + i;
25
26 return result; // Return to caller
27 }
```

图 15.3 一个含有局部变量相关错误的程序

有些错误会在执行期间因程序进行了非法操作而被发现。具有存储保护机制的数字系统会划出受保护的存储区域, 只有相应特权级的代码才能访问, 并非所有程序都有权进入。例如, 系统存储区域只能由具有系统级特权的代码访问。这样就能保护系统, 防止有错误的应用程序或恶意代码访问不该访问的存储位置。程序进行这类非法访问时, 操作系统可以终止它的执行。要构造一个可能引起此类问题的例子, 可以把 AllSum 示例中的 scanf 改为:

```
scanf("%d", in);
```

字符 & 是 C 的一个特殊运算符，第 16 章将介绍它。这里漏掉它，就把本应传入的存储地址换成了整数值，可能导致非法的存储写入和运行时错误。后续章节会更详细地分析这个例子及其原因。<sup>3</sup>

### 15.2.3 算法错误

**算法错误** (algorithmic error) 来自代码设计本身的缺陷：程序完全按照设计运行，但设计就是错的。这类错误可能隐藏很深，运行许多次才暴露出来；即使已经发现并定位，也可能很难修复。不过，在写任何代码之前，先在设计阶段认真规划，通常能减少乃至避免许多这类错误。

算法错误的范围很广，从简单、孤立的问题到牵涉大片功能的复杂问题都有。先看一种初学者和有经验的程序员都常犯的错误：**差一错误** (off-by-one error)。它表示代码中的某个量差了 1，或类似的一个小数目。这个量可能是循环迭代次数、为数据对象分配的存储空间大小，或其他数值。图 15.4 中迭代次数就差了 1：输入 10 时，程序不会执行十次，而只执行九次。这个错误显然容易修复。编程时，我们很容易算错循环终止条件、存储对象大小，或程序逻辑中的其他类似数值；而这些计算常常只差 1 或一个很小的数。

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5 int iterations; // Input value
6
7 printf("How many iterations? ");
8 scanf("%d", &iterations);
9
10 for (int i = 1; i < iterations; i++)
11 printf("Executing iteration %d\n" , i);
12 }
```

图 15.4 一个含有差一错误的程序

图 15.5 给出了另一个更深一些、涉及程序逻辑的算法错误。它输入一个年份，判断该年是否为闰年。乍看之下，代码似乎正确：闰年确实每四年出现一次。然而，公历中整百年份要跳过，除非它又是 400 的倍数。例如，2000 年是闰年，2100、2200、2300 年却不是。代码对绝大多数年份有效，唯独会在这些特殊情况下出错。我们把这种错误归为算法错误。

<sup>3</sup>译注：原书把漏写 & 的后果说成必然触发保护错误。实际上，%d 所要求的实参类型是 int \*，传入 int 属于未定义行为，不能保证一定由操作系统捕获。并非所有数字系统都有这里描述的存储保护机制。

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5 int year;
6
7 printf("Input a year (i.e., 1942): ");
8 scanf("%d", &year);
9
10 if ((year % 4) == 0)
11 printf("This year is a leap year\n");
12 else
13 printf("This year is not a leap year\n");
14 }
```

图 15.5 这个判断闰年的程序含有算法错误。

#### 15.2.4 规格错误

最为人知的软件错误之一，可能就是 2000 年问题，也叫 Y2K 错误。早期许多计算机的存储容量很小，程序员不得不节省每个数据项占用的空间。保存日期时，许多软件只用足够表示年份末两位的位数。例如，1975 年只保存 75，隐含假定世纪为 1900 年代。采用这种简化后，2000 年与 1900 年便无法区分，实际上也无法与 1800 或 2100 年区分。随着 1999 年 12 月 31 日之后即将进入 2000 年，这成了一个全球性挑战。许多最初为存储受限系统编写的软件仍在使用，必须修改它们，才能正确跨越世纪。因此，在 2000 年 1 月 1 日到来之前，人们投入了大量资金和精力排查 Y2K 问题。

我们把这类错误称为**规格错误**（specification error）。软件按照规格运行，但规格本身存在缺陷。当年的软件架构设计者认为，只编码年份末两位可以接受，因为他们没有预料到软件会一直使用到 1999 年底。

如果规格没有正确预见软件的工作需求，就可能产生规格错误，Y2K 就是一个例子。规格沟通不清也会造成错误。通常，软件的工作要求会写入规格文档；如果文档表述不清或写错，就可能导致软件出错。即使需求经过了周密考虑、文档也写得很好，实现团队仍可能因理解有误而引入规格错误。规格是面向人的概念，形成和落实规格的各个环节都可能受到人为错误的影响。

### 15.3 测试

有经验的程序员常说：任何没有测试过的代码行，都可能藏着错误。良好的测试方法是编写优质软件的关键。什么是测试？测试就是让软件经历人为构造的试验：提供设计好的输入，观察软件的行为，争取在它交到实际用户手中之前发现错误。现实中的软件在发布前可能会经历数百万次这样的试验。



在理想世界里，我们可以检查程序在每一种可能输入下的运行情况。然而，对大多数程序来说，遍历所有输入组合是不可能的。例如，测试一个寻找整数  $A$  到  $B$  之间素数的程序，如果  $A$  和  $B$  都是 32 位输入，就会有  $(2^{32})^2$  种输入组合。即使每秒能够运行一百万次试验，完成全部测试仍需大约五十万年。显然，逐一测试所有输入组合不可行。

那么应该选择哪些输入？可以随机挑选，希望其中一些模式能暴露错误。这是一种有用的策略，但通常不足以让我们高度确信代码符合规格。现代软件开发以更系统的方式组织测试。其中，**黑盒测试**用于检查代码是否符合规格；**白盒测试**则针对程序实现的各个方面，以便让许多代码行，最好是每一行，都得到测试。

### 15.3.1 黑盒测试

**黑盒测试** (black-box testing) 检查程序是否满足输入和输出规格，而不考虑内部实现。我们把待测软件当作一个看不到内部的黑盒。因此，黑盒测试关心程序做了什么，而不是它怎样完成这些工作。例如，对图 15.3 的 AllSum 程序进行黑盒测试，可以运行程序、键入一个数，再把输出与手算结果比较。如果两者不同，要么程序有错误，要么你的算术不够可靠。可以继续尝试不同输入，直到对程序能够正常工作有合理的信心。

对于更大的程序，测试过程需要自动化，以便在单位时间内完成更多测试。也就是说，自动运行原程序、提供一些随机输入、检查输出是否满足规格，然后重复。这样显然比逐次人工操作能够完成更多试验。

要让黑盒测试自动化，就需要一种方法，能够针对给定的测试输入，自动判断输出是否符合规格。我们常常需要开发一个**校验程序** (checker program)，完成与待测程序相似的计算，但独立编写。由于开发相互独立，两个程序不太可能在同一处出现同一种错误，因此可以通过相互比较来发现问题。编写校验程序的黑盒测试人员往往不被允许查看待测黑盒内部的代码，以确保校验程序确实是独立的实现。

### 15.3.2 白盒测试

对较大的软件系统，除了黑盒测试，还需要其他手段来提高测试效果。仅靠黑盒测试，无法知道哪些代码行已经执行过、哪些没有；根据前面那句经验之谈，没有得到测试的代码仍应被视为可能有错误。

开发者用**白盒测试** (white-box testing) 补充黑盒测试。白盒测试把软件内部的部件分离出来，检查它们是否符合各自的设计要求。也就是说，测试是在了解软件内部构造的基础上进行的。例如，知道程序由若干函数组成，并分别测试每个函数，即使把单个函数当作黑盒，也是一种面向整个程序的白盒测试过程。同样的方法也可用于函数中的循环和其他子结构。

怎样构造白盒测试？许多白盒测试需要修改代码本身。例如，为了检查某个函数是否正常，可以添加代码，以不同输入多调用几次该函数，再比较实际输出和预期结果。也可以加入额外的 printf，观察内部变量是否具有预期值。等代码完成、准备发布时，可以禁用这些输出语句，使它们不参与编译，或直接移除。

另一种常见的白盒策略是在程序的关键位置加入错误检测代码，检查是否出现表明程序工作异常的状态。如果发现异常，就采取某种纠正措施，例如关闭应用的一部分功能，或终止、重新启动应用。这些检测用于确认程序执行期间某些条件始终成立，因此通常称为**断言**（assertion）。例如，可以检查函数返回值是否落在预期范围内；一旦超出范围，就触发相应措施。

下面的例子检查 CalculateArea 是否产生合理的值。假设函数的输入是一个由两个高度和两个宽度表示的几何对象。即使不了解函数的其他细节，也可以检查计算得到的面积是否非负：除非哪里出了问题，否则面积不应为负。

```
area = CalculateArea(height1, height2, width1, width2);
if (area < 0)
 printf("Error in function CalculateArea!\n");
```

4

实际测试方法通常要求结合黑盒测试和白盒测试。只有白盒测试不能覆盖软件的完整功能：即使所有白盒测试都通过，而且每行代码都单独执行过，仍可能有部分工作规格未被检验。同样，仅靠黑盒测试也不能保证每一行代码都执行过。把两者结合起来，才能更有力地探索现实软件庞大的运行空间。

## 15.4 调试

一旦发现错误，就需要定位、修复，并验证修复结果。这就是调试。调试需要推理能力，而且常常需要集中精力推理。我们可能只看到了错误症状，例如不正确的输出，或知道一组能够触发错误的输入；有时大致知道错误在哪个区域，但很多时候并不知道。必须从这些有限的信息出发，一步步推断出错误的确切位置，才能着手处理。

有效调试的关键，是快速收集能够帮助识别和解决错误的信息。这就像侦探在现场取证，或熟练的医生通过一系列检查诊断疾病。程序员有多种收集调试信息的方法，从简单直接的临时手段，到借助软件调试工具的系统方法，都有用途。

### 15.4.1 临时调试方法

最简单、最自然的方法是直接阅读源代码。有时故障的表现会提示错误可能位于哪个区域。检查这一部分代码，寻找初始化错误、差一错误及其他简单的语义错误，是有效的第一步。如果范围不大，而且你很熟悉代码，这种方法就很适合。

还可以在代码中插入语句，在执行时打印额外信息，作为阅读代码的补充。输出关键变量的值，能帮助我们快速了解程序逻辑哪里出了问题。添加跟踪控制流的 printf，可以观察哪些语句正在执行以及执行顺序。例如，想迅速判断计数控制循环的迭代次数是否正确，可以在循环体中放一条 printf。对于简单程序，这些临时方法方便而合理。但大型或复杂程序中的错误可能盘根错节，需要更强有力的手段。

<sup>4</sup>译注：原书此处把调用写成 CaculateArea，并把第二个高度写成 height2 的错拼 heighth2；这里统一为正文所指的 CalculateArea 与 height2。这个例子使用条件判断演示断言思想，并未调用 C 标准库的 assert 宏。

### 15.4.2 源代码级调试器

临时方法需要额外的工作、代码和重新编译才能收集信息，会占用程序员不少时间。简化调试过程可以显著提高效率。**源代码级调试器**（source-level debugger）正是为此而设：它把原本需要临时手段完成的工作变得容易。这类软件开发工具允许我们在程序执行的不同位置检查变量、存储对象、寄存器和内存。

源代码级调试器可以让代码一次执行一条 C 语句，并沿途检查程序状态，包括变量、寄存器、内存和运行时栈等。它与前面使用的 LC-3 调试器相似，只是依据高级语言源代码工作，而不是依据 LC-3 汇编代码。

要使用源代码级调试器，编译时必须在可执行映像中附加足够的信息。例如，调试器需要借助编译阶段提供的信息，把机器指令对应到高级语言源代码中的语句；它还需要变量名称及其存储位置的信息，也就是符号表相关信息。这样，程序员才能用源代码中的名字查看变量的值。

如今，许多程序员使用集成开发环境（IDE）编写代码。IDE 把代码仓库、编辑器、编译系统等常用开发工具整合进同一个用户界面。原书列举的例子包括 Windows 上的 Visual Studio、面向 Apple 设备的 Xcode、面向 Android 设备的 Android Studio，以及用于跨平台开发的 Eclipse。这些 IDE 都提供源代码级调试器。虽然具体操作依 IDE 而异，但几乎所有调试器都支持一组核心操作。本节逐一介绍它们。这些命令大致分为两类：一类控制程序执行，另一类检查执行期间变量和内存中的值。

#### 15.4.2.1 断点

**断点**允许我们指定程序执行期间暂时停止的位置，以检查或修改程序状态。它有助于观察错误附近区域的执行情况。例如，可以在源代码某一行或某个函数处设置断点。当执行到达该位置时，程序暂停，我们就能检查它在这一时刻的状态。添加断点的具体方式取决于 IDE 的界面，通常是在目标代码行上点击。

有时，我们只希望在某个条件成立时才在该行停止。这种**条件断点**（conditional breakpoint）适合定位可疑的特定情形。例如，怀疑 PerformCalculation 在实参为 16 时工作不正确，可以在下面的代码中设置仅当 x 等于 16 时才停止的断点：

```
for (x = 0; x < 100; x++)
 PerformCalculation(x);
```

也可以使用**监视点**（watchpoint），在程序中使某个条件成立的位置停止。例如，监视变量 LastItem，让程序在它变为 4 时暂停。调试器就会在导致 LastItem 变为 4 的语句处停下。与绑定某一代码行的断点不同，监视点关注被监视的数据，不限于某一行。<sup>5</sup>

---

<sup>5</sup>译注：具体调试器中的监视点可能由数据写入、读取或值变化触发；若只希望在值等于 4 时停止，通常还需设置相应条件。原书在此描述的是这种带条件的数据监视。

### 15.4.2.2 单步执行

调试器到达断点或监视点后，会暂停程序，等待下一条命令。此时可以检查变量等程序状态，也可以继续执行。常见的做法是从断点开始一次向前执行一条语句，称为**单步执行**。LC-3 调试器有一次执行一条 LC-3 指令的命令；类似地，C 的源代码级调试器可以一次推进一条 C 语句。单步命令执行当前源代码行，然后再次暂停。

单步执行非常适合把错误缩小到某一行。可以先在可疑区域附近设断点，再一边单步前进，一边检查变量值。另一个常见用途是验证控制流是否符合预期：逐步执行循环，检查迭代次数；或者逐步经过 if-else，检查条件是否写对。

单步执行还有一些变体，例如跳过函数内部的执行细节，或直接运行到循环的最后一次迭代。当断点与错误之间夹着一些暂时没有理由怀疑的代码时，这些操作可以帮助我们快速越过它们。

### 15.4.2.3 检查数值

调试的技巧在于收集足够的信息，从而根据逻辑推断出错误来源。调试大型程序时，源代码级调试器是收集信息的重要工具。程序停在断点时，可以检查与可疑错误相关的变量。一般来说，我们能够查看断点处的各种执行状态，包括变量值、内存、栈，甚至寄存器。

具体查看方式取决于 IDE 界面。有些 IDE 允许把鼠标停在源代码窗口的变量名上，通过弹出窗口显示当前值。建议你熟悉所用 IDE 的源代码级调试器。本章末尾安排了几道习题，帮助你积累使用这一实用工具的经验。

## 15.5 为正确性而编程

懂得测试和调试，是成为合格程序员的前提。优秀的程序员还懂得从一开始就避开许多容易引发错误的情形。不良的编程方法会催生错误，尤其是在多人协作的项目中。了解一些防御式编程技术，可以缩短让代码正常运行所需的时间。与错误的斗争，在写下第一行代码之前就已经开始。下面给出三种一般方法，争取在问题形成之前就把它们消除。

### 15.5.1 明确规格

许多错误源于不清楚或不完整的程序规格。有时规格没有覆盖所有可能的工作情形，把一些情况留给程序员自行解释。例如，回想第 14 章的阶乘示例：图 14.2 根据键盘输入计算阶乘。它的规格可能只是“编写一个程序，从键盘读入一个整数，计算它的阶乘”。这样的规格并不完整。如果输入负数怎么办？输入 0 呢？输入的数太大、导致溢出时又怎么办？必须明确这些情况下程序应该采取的行为，否则它就可能不符合预期。

一种做法是把允许的输入限定为正整数，并要求结果能够由返回类型表示；输入不符合要求时，程序报告错误。例如，若使用 32 位有符号整数，应要求  $n! \leq 2^{31} - 1$ ，

也就是正整数输入不超过 12。图 15.6 展示了给 Factorial 加入输入范围检查的做法，越界时返回 -1。不过，原图使用的上限 31 有误，不能真正防止阶乘溢出，使用时应按整数类型的范围修正。<sup>6</sup>

```
1 int Factorial(int n) // Function Definition
2 {
3 int result = 1; // Initialized result
4
5 if ((n < 1) || (n > 31)) // Bad parameter
6 return -1;
7
8 for (int i = 1; i <= n; i++) // Calculate factorial
9 result = result * i;
10
11 return result; // Return to caller
12 }
```

图 15.6 加入输入参数范围检查的阶乘函数；原图上限 31 的问题见正文译注。

### 15.5.2 模块化设计

函数能够扩展编程语言的功能。通过函数，我们可以添加适合特定任务的新操作和新构件，从而以模块化的方式组织程序。

完成一个函数后，可以把它单独拿出来测试，作为整个程序白盒测试的一部分，确定它是否按预期工作。通常，一个函数只完成整个程序中的一项较小任务，因此比整个程序更容易测试。先分别测试、调试每个函数，再将它们集成起来，就更容易让整个程序正常运行。

用经过测试、能够正常工作的简单部件组成程序，这种**模块化设计**（modular design）是系统设计的基本思想。后续章节将介绍库的概念。库是一组预先测试过、可供程序员编写代码时使用的部件。由于模块化设计的这些好处，现代编程实践广泛依赖库。类似的思想不仅用于软件，也用于电路、硬件以及计算系统的其他层次。

### 15.5.3 防御式编程

有经验的程序员都有防止错误混入代码的办法。他们在组织代码时，就设法通过设计排除可能影响程序的问题。这就是**防御式编程**（defensive programming）。下面列出几条通用方法，建议在编程中采用，以减少问题。

- **为代码写注释。**写注释会促使你思考已经写下的代码。代码文档不仅告诉别人

---

<sup>6</sup>译注：原书由  $n! > 2^{31}$  推出  $n$  的上限为 31，这是错误的。对 32 位有符号整数， $12! = 479001600$  可表示，而  $13! = 6227020800$  已超出范围。C 并不保证 int 总是 32 位；若为 16 位有符号整数，最大可接受的正整数输入为 7。此外，数学上  $0! = 1$ ，第 14 章的实现也能正确返回这个值。是否拒绝 0 是接口规格的选择，不应把 0 本身说成错误输入。原图代码按约定保留。

程序如何工作，也让自己重新审视代码。这个过程中，可能会发现遗漏的特殊情况或工作条件，而它们本来会在之后导致程序失效。

- **采用一致的编码风格。**例如，让左右花括号对齐，有助于发现与漏写花括号有关的简单语义错误。变量命名也要一致，名字应当传达变量所保存数值的实际含义。
- **避免没有依据的假设。**编程时很容易作出一些看似无害的假设，最后却导致程序出错。例如，编写函数时假定输入参数总在某个范围内。如果规格没有保证这一点，就已经引入了出错的可能。尽量让代码不依赖这种假设；至少也应通过断言或适当的检查，在假设不成立时给出指示。
- **避免使用全局变量。**虽然一些有经验的程序员大量使用全局变量，许多软件工程师仍主张尽可能避免。全局变量可能使某些任务更容易完成，却常常使代码更难理解和扩展，也使发现错误后的分析更加困难。
- **借助编译器。**大多数良好的编译器都有选项，可以认真检查可疑代码，例如未初始化变量，或把赋值运算符 = 误当成相等运算符 == 等常见误用。虽然这些检查并不完备，却有助于发现常见的编程失误。

这里介绍的方法针对的是已经讨论过的编程概念。后续章节介绍新概念时，也会说明如何在使用它们的程序中采用防御式编程。

## 15.6 小结

本章介绍了查找和修复代码错误的方法。数字系统越来越依赖软件，而现代软件往往非常复杂。为避免错误频繁地使个人设备无法使用，或偶尔导致太空探测器坠毁，软件必须严格满足其规格。本章的关键概念如下。

- **测试。**发现代码中的错误并不容易，尤其是程序规模很大时。软件工程师通过系统化测试查找错误。黑盒测试验证程序行为是否符合规格；白盒测试针对程序结构，帮助确保各代码行得到一定程度的测试。
- **调试。**调试要求我们利用现有信息推断错误根源。临时方法能提供一些额外信息，而源代码级调试器是大多数调试任务的重要工具。它使程序员能够控制程序执行，并查看执行期间的各种数值和状态。
- **为正确性而编程。**有经验的程序员在写下第一行代码前就努力避免错误。程序规格常常就是问题来源；提前澄清遗漏和含糊之处，有助于减少实现后的错误。模块化设计用经过测试的简单函数组成较大的程序，从而降低测试难度。遵循防御式编程风格，也能减少容易产生错误的情形。

## 习题

**15.1** 下面每个程序都有一个错误，使它无法按要求运行。请尽量少作修改，修正这些程序。它们都应该只输出一个数：从 1 到 10（含 1 和 10）的所有整数之和。

```
a. #include <stdio.h>
 int main(void)
 {
 int i = 1;
 int sum = 0;

 while (i < 11) {
 sum = sum + i;
 ++i;
 printf("%d\n", sum);
 }
 }

b. #include <stdio.h>
 int main(void)
 {
 int sum = 0;

 for (int i = 0; i >= 10; ++i)
 sum = sum + i;
 printf("%d\n", sum);
 }

c. #include <stdio.h>
 int main(void)
 {
 int i = 0;
 int sum = 0;

 while (i <= 11)
 sum = sum + i++;
 printf("%d\n", sum);
 }
```

```
d. #include <stdio.h>
 int main(void)
 {
 int sum = 0;
 for (int i = 0; i <= 10;)
 sum = sum + ++i;
 printf("%d\n", sum);
 }
```

**15.2** 下面的程序片段含有语法错误，因此无法编译。假定所有变量均已正确声明。修正错误，使这些片段不再引起编译错误。

```
a. i = 0;
 j = 0;
 while (i < 5)
 {
 j = j + 1;
 i = j >> 1
 }

b. if (cont == 0)
 a = 2;
 b = 3;
else
 a = -2;
 b = -3;

c. #define LIMIT 5;
 if (LIMIT)
 printf("True");
 else
 printf("False");
```



- 15.3** 下面的 C 代码用于求用户从键盘输入的一组正整数中的最小值。用户输入 -1 表示这组数据结束。读入并处理完所有数之后，程序输出最小值。然而，代码中有一个错误。请找出错误，并提出修复方法；必要时可使用源代码级调试器帮助查找。

```
#include <stdio.h>
int main(void)
{
 int smallestNumber = 0;
 int nextInput;
 // Get the first input number
 scanf("%d", &nextInput);

 // Keep reading inputs until user enters -1
 while (nextInput != -1) {
 if (nextInput < smallestNumber)
 smallestNumber = nextInput;
 scanf("%d", &nextInput);
 }
 printf("The smallest number is %d\n", smallestNumber);
}
```

- 15.4** 下面的程序从键盘读入一行字符，只回显其中的字母。例如，输入 Let's meet at 6:00pm. 时，输出应为 Letsmeetatpm。然而，程序中存在一个错误。你能找出并修正它吗？

```
#include <stdio.h>
int main(void)
{
 char echo = '0';
 while (echo != '\n') {
 scanf("%c", &echo);
 if ((echo > 'a' || echo < 'z') &&
 (echo > 'A' || echo < 'Z'))
 printf("%c", echo);
 }
}
```

**15.5** 使用源代码级调试器监视下面代码的执行。

```
#include <stdio.h>
int IsDivisibleBy(int dividend, int divisor);

int main(void)
{
 int f; // The number of integer factors of a number
 for (int i = 2; i < 1000; i++) {
 f = 0;
 for (int j = 2; j < i; j++) {
 if (IsDivisibleBy(i, j))
 f++;
 }
 printf("The number %d has %d factors\n", i, f);
 }
}

int IsDivisibleBy(int dividend, int divisor)
{
 if (dividend % divisor == 0)
 return 1;
 else
 return 0;
}
```

- a. 在 `IsDivisibleBy` 函数开始处设置断点，检查前十次调用的形参值。它们分别是什么？
- b. 当 `i` 等于 660，且内层 `for` 循环结束时，`f` 的值是多少？
- c. 能否把这个程序写得更高效？提示：观察 `IsDivisibleBy` 返回 1 时的实参值。

**15.6** 使用源代码级调试器，确定函数 `Mystery` 在实参取哪些值时返回 0。

```
#include <stdio.h>
int Mystery(int a, int b, int c);

int main(void)
{
 int sum = 0; // running sum of Mystery
 for (int i = 100; i > 0; i--) {
 for (int j = 1; j < i; j++) {
 for (int k = j; k < 100; k++)
 sum = sum + Mystery(i, j, k);
 }
 }

 int Mystery(int a, int b, int c)
 {
 int out;

 out = 3*a*a + 7*a - 5*b*b + 4*b + 5*c;
 return out;
 }
}
```

译注：原程序的累计和超出 32 位有符号整数的范围，会产生有符号溢出。为可靠地执行完整调试过程，可把累计变量 `sum` 改为足够宽的整数类型，例如 `long long`；此处保留原题代码。

**15.7** 下面的程序为一家只有一架飞机的小航空公司管理机票预订。这架飞机有 SEATS 个乘客座位。程序处理航空公司网站发来的订座请求。命令 R 表示请求订座：如果还有座位，就批准预订；没有座位则拒绝。随后，已预订的乘客可以用命令 P 购买机票。因此，每个 P 之前都必须有相应的 R，但并非每个 R 最终都会转化为购票。输入命令 X 时，程序结束。下面的程序包含严重的设计错误。请找出这些错误，提出并实现正确的解决方案。

```
#include <stdio.h>
#define SEATS 10

int main(void)
{
 int seatsAvailable = SEATS;
 char request = '0';
 while (request != 'X') {
 scanf("%c", &request);
 if (request == 'R') {
 if (seatsAvailable)
 printf("Reservation Approved!\n");
 else
 printf("Sorry, flight fully booked.\n");
 }
 if (request == 'P') {
 seatsAvailable--;
 printf("Ticket purchased!\n");
 }
 }
 printf("Done! %d seats not sold\n", seatsAvailable);
}
```

# 第十六章 指针与数组

## 16.1 引言

本章介绍两种简单而强大的数据对象：指针和数组。严格来说，这是再次介绍，因为编写 LC-3 程序时，我们已经接触过它们。现在，我们在高级编程语言的背景下正式讨论这些概念。指针就是某个存储对象，例如变量，的地址。借助指针，可以间接访问对象，从而获得一些十分有用的能力。例如，可以编写函数，修改调用者提供地址的对象；还可以构造复杂的数据组织形式，让它们像运行时栈一样，在程序执行期间增长或缩小。

数组是一组类型相同、在内存中顺序排列的数据对象。本书前半部分的一些 LC-3 示例把字符文件表示为内存中连续排列的字符序列，这种组织形式正式的名称就是数组。访问数组中的某一项时，通过**下标**指定所需元素。例如，`a[4]` 访问名为 `a` 的数组的第五个元素；之所以是第五个，是因为元素编号从 0 开始。数组能方便地处理向量、矩阵、列表和字符串等成组数据，而这些数据又自然地对应现实世界中的许多对象，因此数组非常有用。

## 16.2 指针

我们从一个经典例子开始讨论指针的用途。图 16.1 的 C 程序中，`Swap` 试图交换两个实参所对应变量的值。`main` 用值为 3 的 `valueA` 和值为 4 的 `valueB` 调用它。我们希望 `Swap` 返回后，这两个变量的值已经交换。但编译运行程序后，会发现 `main` 中的变量并没有改变。<sup>1</sup>

---

<sup>1</sup>译注：原书此处误写为 `Swap` 中的变量不变；实际上，该函数内的形参已经交换，不变的是调用者 `main` 的变量。

```
1 #include <stdio.h>
2
3 // Swap has no return value thus is declared as void
4 void Swap(int firstVal, int secondVal);
5
6 int main(void)
7 {
8 int valueA = 3;
9 int valueB = 4;
10
11 printf("Before Swap ");
12 printf("valueA = %d and valueB = %d\n", valueA, valueB);
13
14 Swap(valueA, valueB);
15
16 printf("After Swap ");
17 printf("valueA = %d and valueB = %d\n", valueA, valueB);
18 }
19
20 void Swap(int firstVal, int secondVal)
21 {
22 int tempVal; // Holds firstVal when swapping
23
24 tempVal = firstVal;
25 firstVal = secondVal;
26 secondVal = tempVal;
27 }
```

图 16.1 函数 Swap 试图交换两个参数的值。

为什么会这样？考察 Swap 执行期间的运行时栈。图 16.2 表示函数即将完成时的状态：第 26 行刚执行完，但控制尚未返回 main。<sup>2</sup>Swap 修改的是自己栈帧内的形参 firstVal 和 secondVal。函数结束、栈帧弹出后，这些修改过的形参值就不再使用。从 main 的角度看，两个值根本没有交换，程序仍然有错。

<sup>2</sup>译注：原书正文写成第 25 行，但图 16.1 中完成最后一次赋值的是第 26 行，这里按图示状态更正。

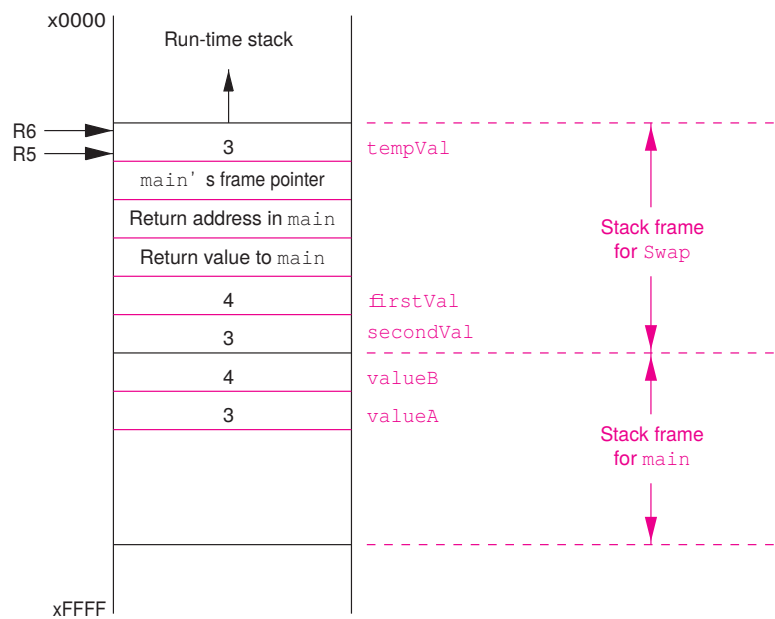


图 16.2 函数 Swap 即将返回 main 时的运行时栈快照

C 总是以**传值**方式把实参从调用者传给被调用者。每个实参都作为表达式求值，得到的值成为被调用函数形参变量的初值。在本书的 LC-3 实现中，这些值通过运行时栈传递。要修改调用者的变量，Swap 必须能够访问调用者栈帧中保存这些变量的存储位置，也就是需要 main 中 valueA 与 valueB 的地址。接下来将看到，指针及其相关运算符可以实现这一点。

16.2.1 声明指针变量

整数变量保存一个按整数解释的位模式；类似地，指针变量保存一个按存储对象地址解释的位模式。如果指针保存了某变量的地址，就说它“指向”该变量。指针类型还包含它所指对象的类型信息。例如，整数指针指向整数对象。在 C 中，声明指针变量的语法如下：

```
int *ptr;
```

这里声明了名为 ptr 的变量，它指向整数。星号 \* 表明后面的标识符声明为指针变量。C 程序员常把 ptr 的类型口头读作 “int star”。类似地，还可以声明：

```
char *cp;
double *dp;
```

cp 指向字符，dp 指向双精度浮点数。指针变量的初始化规则与其他变量相同：未显式初始化的自动局部指针不会自动得到有效初值。刚开始，使用 \* 声明指针的方式可能显得有些奇怪；了解指针运算符后，这种语法的道理就会更清楚。

16.2.2 指针运算符

C 提供两个与指针操作密切相关的运算符：取地址运算符 `&` 和间接访问运算符 `*`。

16.2.2.1 取地址运算符 `&`

**取地址运算符**（address operator）`&` 产生操作数的存储地址。操作数可以是变量等可取地址的存储对象。在下面的代码中，`ptr` 将指向整数变量 `object`。第二条赋值语句右侧的表达式产生 `object` 的地址。

```
int object;
int *ptr;
object = 4;
ptr = &object;
```

看看对应的 LC-3 代码。假定这两个变量都是分配在运行时栈上的局部变量。回忆本书约定，帧指针 `R5` 指向第一个声明的标量局部变量，本例中就是 `object`。

```
AND R0, R0, #0 ; Clear R0
ADD R0, R0, #4 ; R0 = 4
STR R0, R5, #0 ; object = 4;
ADD R0, R5, #0 ; Generate memory address of object
STR R0, R5, #-1 ; Ptr = &object;
```

图 16.3 给出了 `ptr = &object;` 执行后的栈帧。为使例子具体些，我们任意选用了 `xEFF0` 附近的一组地址。`R5` 当前指向 `xEFF2`。`object` 保存整数 4，`ptr` 则保存 `object` 的地址。

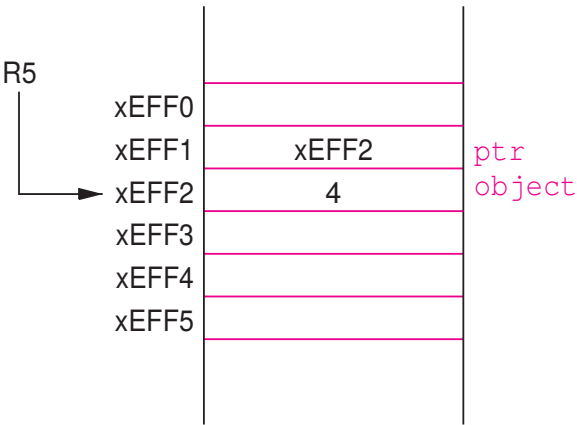


图 16.3 执行 `ptr = &object;` 后，包含 `object` 与 `ptr` 的运行时栈帧

16.2.2.2 间接访问运算符 `*`

第二个运算符称为**间接访问运算符**（indirection operator），也叫**解引用运算符**（dereference operator），符号是 `*`，通常读作“star”。它允许我们间接操作存储对象。在上例中，`*ptr` 表示指针 `ptr` 所指的對象，也就是变量 `object`。在这里，两者可



用于访问同一个对象。给前面的代码增加一条语句：

```
int object;
int *ptr;
```

```
object = 4;
ptr = &object;
*ptr = *ptr + 1;
```

`*ptr = *ptr + 1;` 与 `object = object + 1;` 完成相同的操作。`*ptr` 在赋值两侧的作用不同：在右侧，需要读取该位置的值，此处为 4；在左侧，则指定要修改的存储位置，此处就是 `object` 所在的位置。最后一条语句对应的 LC-3 代码如下：

```
LDR R0, R5, #-1 ; R0 contains the value of ptr
LDR R1, R0, #0 ; R1 <- *ptr
ADD R1, R1, #1 ; *ptr + 1
STR R1, R0, #0 ; *ptr = *ptr + 1;
```

这与直接写 `object = object + 1;` 所生成的代码不同。为处理右侧解引用，编译器生成两条 **LDR**：先取出 `ptr` 保存的地址，再取出该地址中的值。左侧解引用最终对应 `STR R1, R0, #0`。如果改成 `object = *ptr + 1;`，最后的存储指令就会是 `STR R1, R5, #0`。

### 16.2.3 用指针传递引用

借助取地址和间接访问运算符，可以修复图 16.1 中未能交换调用者变量的 `Swap`。图 16.4 使用了修改后的函数 `NewSwap`。

```
1 #include <stdio.h>
2
3 void NewSwap(int *firstVal, int *secondVal);
4
5 int main(void)
6 {
7 int valueA = 3;
8 int valueB = 4;
9
10 printf("Before Swap ");
11 printf("valueA = %d and valueB = %d\n", valueA, valueB);
12
13 NewSwap(&valueA, &valueB);
14
15 printf("After Swap ");
16 printf("valueA = %d and valueB = %d\n", valueA, valueB);
17 }
18
19 void NewSwap(int *firstVal, int *secondVal)
20 {
21 int tempVal; // Holds firstVal when swapping */
22
23 tempVal = *firstVal;
24 *firstVal = *secondVal;
25 *secondVal = tempVal;
26 }
```

图 16.4 函数 NewSwap 交换两个实参所指对象的值。

首先，NewSwap 的形参不再是整数，而是指向整数的指针，即 `int *`。它们保存两个待交换变量的地址。函数体中使用 `*` 取得这些地址所指对象的值。

从 `main` 调用时，必须提供待交换变量的地址，而不是像原版本那样提供它们的值。取地址运算符 `&` 正好完成这一工作。图 16.5 给出 NewSwap 执行过程中的栈快照，三个子图分别对应第 23、24、25 行执行后的状态。

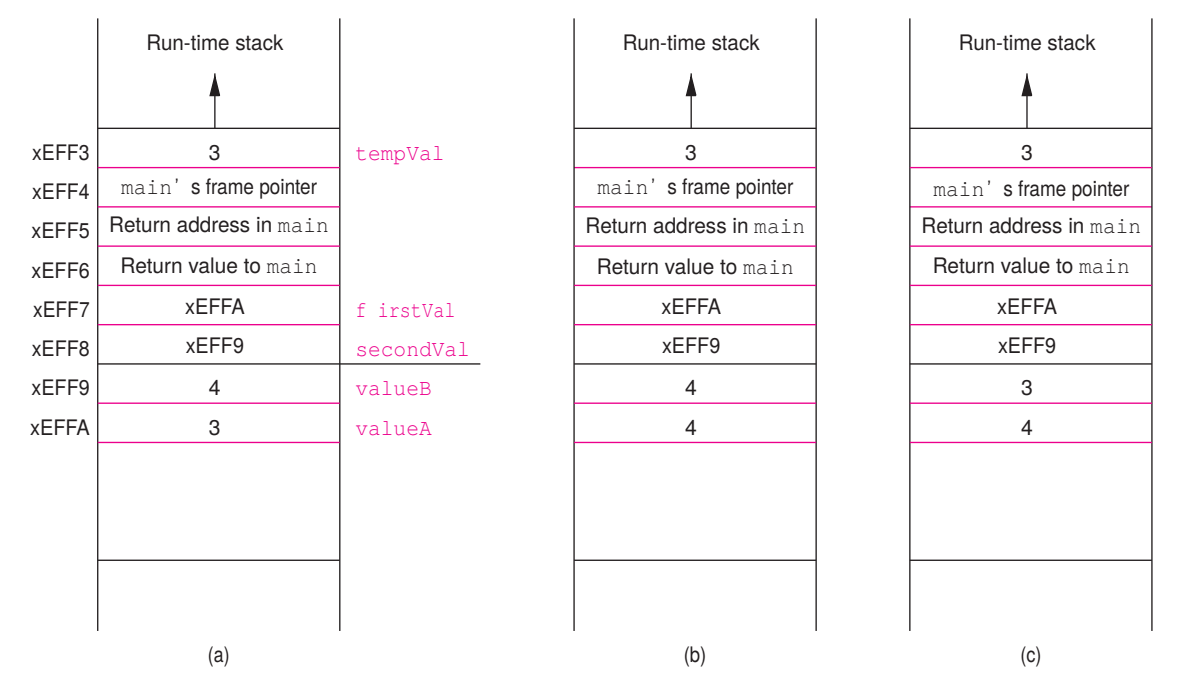


图 16.5 NewSwap 执行过程中，第 (a) 23 行、(b) 24 行、(c) 25 行执行后的运行时栈快照

C 的基本规则仍然是传值：调用表达式中的每个实参求值后，其值传给被调用者。不过，我们用 & 把对象的地址作为值传入，就实现了通过引用访问原对象的效果。被调用函数再使用 \*，即可读取或修改调用者中的对象。被取地址的对象必须是能够取地址的变量或其他对象。<sup>3</sup>

16.2.4 空指针

有时，让指针不指向任何对象很方便，这样就能把它与指向实际对象的指针区分开。这种指针称为**空指针**（null pointer）。在 C 中可以这样写：

```
int *ptr;
ptr = NULL;
```

这里把 NULL 赋给 ptr。NULL 是标准头文件提供的宏，可用作空指针常量。在本书采用的简单地址模型中，可以把它直观地理解为“不指向任何对象”的特殊值；第 19 章的动态数据结构会经常用到它。<sup>4</sup>

16.2.5 理解这些语法

再看第 11 章引入的 scanf 写法：

```
scanf("%d", &input);
```

<sup>3</sup>译注：原书称这种用法为 call by reference。这里的“传递引用”描述的是传入地址后访问原对象的效果；C 的参数传递机制依然是传值，传递的是指针值的副本，并不是 C++ 的引用形参。

<sup>4</sup>译注：原书把 NULL 直接等同于数值 0、再把空指针等同于内存地址 0。C 允许用整数常量表达式 0 构造空指针，但不保证空指针的底层位模式全为 0，也不应把它当作可访问的地址 0。使用 NULL 应包含定义它的标准头文件，例如 stddef.h。

`scanf` 要把键盘输入的十进制整数写入变量 `input`，因此需要的是它的地址，而不是原来的值，所以必须使用 `&`。如果漏掉取地址运算符，就会传入错误类型的实参，可能引起运行时错误。你能解释原因吗？为什么在这里不提供变量地址，`scanf` 就无法正确更新它？<sup>5</sup>

在结束指针的入门讨论前，再来理解声明语法：

```
type *ptr;
```

`type` 可以是 `int`、`char`、`float`、`double` 等预定义类型，也可以是程序员定义的类型；`ptr` 是合法标识符。这样声明意味着，对 `ptr` 使用解引用运算符后，`*ptr` 所表示的对象具有 `type` 类型。

取地址和间接访问运算符也遵循 C 的优先级与结合性规则。它们与其他运算符的规则列在表 12.5 中。注意，这两个一元运算符都有很高的优先级。

### 16.2.6 一个使用指针的问题

假设要编写 C 程序，根据整数被除数和除数计算商与余数，也就是 `dividend / divisor` 与 `dividend % divisor`。程序结构简单，只需顺序结构，不需要迭代。特别的要求是：商和余数都要由同一个 C 函数计算。

只产生一个结果很容易，可以通过返回值交给调用者。例如，只求商的函数可以仅包含 `return dividend / divisor;`。要向调用者提供多个结果，就可以通过指针传入结果对象的地址。

图 16.6 的 `IntDivide` 正是这样做的。它有四个形参：两个整数、两个整数指针。函数用第一个整数 `x` 除以第二个整数 `y`，把整数商写入 `quoPtr` 所指位置，把整数余数写入 `remPtr` 所指位置。这样就可以向调用者“返回”多个输出值。

---

<sup>5</sup>译注：与第 15 章相同，实参类型不匹配导致未定义行为，并不保证每次都报错终止。

```
1 include <stdio.h>
2
3 int IntDivide(int x, int y, int *quoPtr, int *remPtr);
4
5 int main(void)
6 {
7 int dividend; // The number to be divided
8 int divisor; // The number to divide by
9 int quotient; // Integer result of division
10 int remainder; // Integer remainder of division
11 int error; // Did something go wrong?
12
13 printf("Input dividend: ");
14 scanf("%d", ÷nd);
15 printf("Input divisor: ");
16 scanf("%d", &divisor);
17
18 error = IntDivide(dividend, divisor, "ient, &remainder);
19
20 if (!error) // !error indicates no error
21 printf("Answer: %d remainder %d\n", quotient, remainder);
22 else
23 printf("IntDivide failed.\n");
24 }
25
26 int IntDivide(int x, int y, int *quoPtr, int *remPtr)
27 {
28 if (y != 0) {
29 *quoPtr = x / y; // Modify *quoPtr
30 *remPtr = x % y; // Modify *remPtr
31 return 0;
32 }
33 else
34 return -1;
35 }
```

图 16.6 函数 IntDivide 计算整数商和余数；除数为 0 时返回 -1。

IntDivide 同时用真正的返回值表示状态：除数为 0 时返回 -1，表示发生错误；否则返回 0，表示计算成功。main 检查这个状态，再决定是否使用 quotient 与 remainder。用返回值报告错误状态是一种常见约定，复杂函数尤其如此。<sup>6</sup>

<sup>6</sup>译注：图 16.6 原书第 1 行的 include 前缺少 #，原图保留，独立编译应补上。示例还默认输出指针有效、输入转换成功，且商可表示；例如最小有符号整数除以 -1 并未被这项除零检查覆盖。

## 16.3 数组

考虑一个记录计算机工程课程中 50 名学生期末考试成绩的程序。最方便的方法，是声明一个名为 `examScore` 的存储对象，在其中保存 50 个不同的整数值。通过相对于起始位置的下标，可以访问某个学生的成绩。例如，`examScore[32]` 是第 33 名学生的成绩，`examScore[19]` 是第 20 名学生的成绩，而第一名学生的成绩在 `examScore[0]` 中。

这里的 `examScore` 就是整数数组。数组是在内存中连续排列、通过一个名称访问的一组同类数据项。所有元素的类型相同，例如都为 `int` 或都为 `char`，并通过数组名及下标来访问。

当程序处理的数据自然地表现为连续的一组数值时，数组尤其有用。课程成绩等许多现实数据都属于这种情况。从计算的角度看，下标本身也可以通过运算产生，这使数组能够支持复杂的计算模式。例如，要把键盘输入的 100 个数按升序排列，用数组存放它们就是自然的选择。若只使用彼此独立的简单变量，程序几乎难以编写。借助迭代结构计算下标，则能使排序代码紧凑而富于表达力。

### 16.3.1 声明和使用数组

先看如何在 C 中声明数组。与其他声明一样，数组也有类型，决定其中各元素的性质。下面声明一个包含十个整数的数组：

```
int grid[10];
```

关键字 `int` 指定整数类型，`grid` 是名称，方括号表示数组，10 指定元素个数。这十个整数在内存中连续存放。图 16.7 展示了分配方式：第一个元素 `grid[0]` 位于最低地址，最后一个元素 `grid[9]` 位于最高地址。如果 `grid` 是局部数组，在本书实现中，它的空间就分配在运行时栈上。

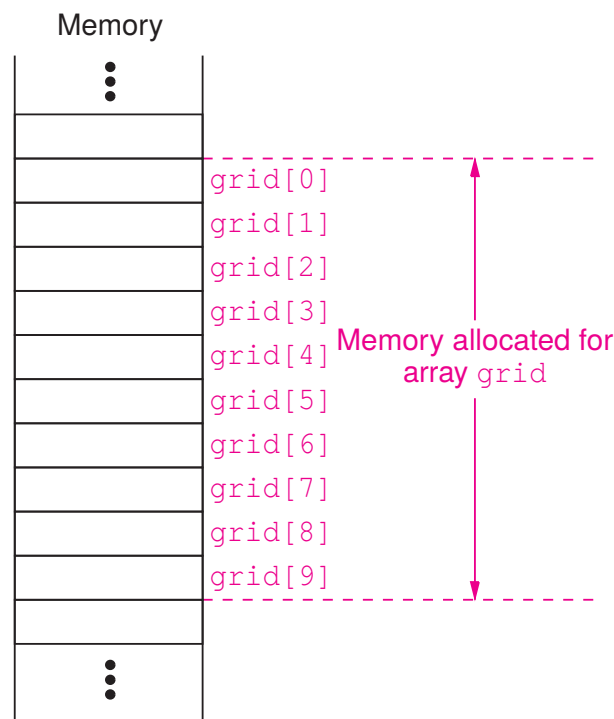


图 16.7 数组 grid 在内存中的分配

要访问某一元素，在方括号内给出下标即可。例如：

```
grid[6] = grid[3] + 1;
```

这条语句读取第四个元素的值，加 1 后存入第七个元素。记住，元素编号从 0 开始。假定 grid 是栈帧中唯一的局部对象，按本书的布局约定，R5 指向数组最后一个元素 grid[9]。对应 LC-3 代码如下：

```
ADD R0, R5, #-9 ; Put the base address of grid into R0
LDR R1, R0, #3 ; R1 <-- grid[3]
ADD R1, R1, #1 ; R1 <-- grid[3] + 1
STR R1, R0, #6 ; grid[6] = grid[3] + 1;
```

第一条指令计算数组的**基地址**，也就是 grid[0] 的地址，并存入 R0。在这里，一个整数占一个 LC-3 存储单元，因此把下标加到基地址上就得到相应元素的地址。<sup>7</sup>

数组的威力在于下标可以是 C 的整型表达式。例如，下面通过变量 x 访问数组：

```
grid[x+1] = grid[x] + 2;
```

<sup>7</sup>译注：一般情况下，字节地址的偏移等于下标乘以元素大小。C 的指针算术会自动按所指元素的大小调整，不是无条件每加 1 就增加一个字节。

假定 `x` 紧挨着 `grid` 分配在其低地址一侧，也就是栈顶方向，对应代码为：

```
LDR R0, R5, #-10 ; Load the value of x
ADD R1, R5, #-9 ; Put the base address of grid into R1
ADD R1, R0, R1 ; Calculate address of grid[x]
LDR R2, R1, #0 ; R2 <-- grid[x]
ADD R2, R2, #2 ; R2 <-- grid[x] + 2
LDR R0, R5, #-10 ; Load the value of x
ADD R0, R0, #1 ; R0 <-- x + 1
ADD R1, R5, #-9 ; Put the base address of grid into R1
ADD R1, R0, R1 ; Calculate address of grid[x+1]
STR R2, R1, #0 ; grid[x+1] = grid[x] + 2;
```

### 16.3.2 使用数组的例子

先看一个简单程序：将两个数组对应的元素相加，形成结果数组。两个输入数组分别记录同一门课中每名学生的考试成绩。要得到各学生的总分，就计算  $Total[i] = Exam1[i] + Exam2[i]$ 。图 16.8 读入两个包含十个整数的数组，把对应元素相加后存入第三个数组，再打印结果。



```
1 #include <stdio.h>
2 #define NUM_STUDENTS 10
3
4 int main(void)
5 {
6 int Exam1[NUM_STUDENTS];
7 int Exam2[NUM_STUDENTS];
8 int Total[NUM_STUDENTS];
9
10 // Input Exam 1 scores
11 for (int i = 0; i < NUM_STUDENTS; i++) {
12 printf("Input Exam 1 score for student %d : ", i);
13 scanf("%d", &Exam1[i]);
14 }
15 printf("\n");
16
17 // Input Exam 2 scores
18 for (int i = 0; i < NUM_STUDENTS; i++) {
19 printf("Input Exam 2 score for student %d : ", i);
20 scanf("%d", &Exam2[i]);
21 }
22 printf("\n");
23
24 // Calculate Total Points
25 for (int i = 0; i < NUM_STUDENTS; i++) {
26 Total[i] = Exam1[i] + Exam2[i];
27 }
28
29 // Output the Total Points
30 for (int i = 0; i < NUM_STUDENTS; i++) {
31 printf("Total for Student %d = %d\n", i, Total[i]);
32 }
33 }
```

图 16.8 计算两个十元素数组之和的 C 程序

注意，预处理宏 NUM\_STUDENTS 表示输入集合的大小。这是宏的一种常见用途；宏通常定义在源文件开头或头文件中。若学生人数变化，需要扩大数组，只要修改这一处宏定义，再重新编译即可。若不用宏，数组大小改变时就得修改代码中的多个位置，不但难以找全，漏改一处也很可能导致错误。

再看稍复杂的例子。图 16.9 从键盘读入 MAX\_NUMS 个十进制整数，计算每个输入值在整个序列中出现的次数，并将各值及其次数打印出来。

```
1 #include <stdio.h>
2 #define MAX_NUMS 10
3
4 int main(void)
5 {
6
7 int repIndex; // Loop variable for rep loop
8 int numbers[MAX_NUMS]; // Original input numbers
9 int repeats[MAX_NUMS]; // Number of repeats
10
11 // Get input
12 printf("Enter %d numbers.\n", MAX_NUMS);
13 for (int index = 0; index < MAX_NUMS; index++) {
14 printf("Input number %d : ", index);
15 scanf("%d", &numbers[index]);
16 }
17
18 // Scan through entire array, counting number of
19 // repeats per element within the original array
20 for (int index = 0; index < MAX_NUMS; index++) {
21 repeats[index] = 0;
22 for (repIndex = 0; repIndex < MAX_NUMS; repIndex++) {
23 if (numbers[repIndex] == numbers[index])
24 repeats[index]++;
25 }
26 }
27
28 // Print the results
29 for (int index = 0; index < MAX_NUMS; index++)
30 printf("Original number %d. Number of repeats %d\n",
31 numbers[index], repeats[index]);
32 }
```

图 16.9 统计数组中各数值出现次数的 C 程序

程序使用两个数组 `numbers` 和 `repeats`，都含有 `MAX_NUMS` 个整数。前者保存输入序列，后者保存对应输入值在序列中出现的次数。例如，若 `numbers[3]` 为 115，而输入中一共出现四个 115，那么 `repeats[3]` 就为 4。

程序有三个主要循环，分别从第 13、20、29 行开始。第一个读入数据，最后一个打印结果。中间的循环包含一个嵌套循环，用于统计次数。外层循环让 `index` 从 0 变化到 `MAX_NUMS - 1`，逐一选择 `numbers` 中的元素。内层循环也遍历整个数组，统计有多少元素等于外层所选的 `numbers[index]`。每当下面的表达式成立，

便在第 24 行将相应的 `repeats` 元素加 1：<sup>8</sup>

```
(numbers[repIndex] == numbers[index])
```

### 16.3.3 数组作为参数

在函数之间传递数组，能让我们编写对整组数据操作的函数。例如，要计算整数数组的均值、中位数，就需要把整个数组复制给函数，或向函数提供对原数组的访问方式。数组很大时，从一个栈帧把所有元素复制到另一个栈帧会耗费大量时间。C 的常见做法是传递首元素指针，从而直接访问原数组。

图 16.10 的 `Average` 接收整数数组作为参数。`main` 调用时，使用数组名 `numbers` 作实参，没有写通常的方括号。在这个表达式中，数组名转换为首元素地址，即 `&numbers[0]`，类型为 `int *`。因此，在本书的 LC-3 实现中，压栈的是这个地址，而不是整个数组。被调用者的形参 `inputValues` 接收到地址后，便可以通过普通数组下标写法访问原数组。

---

<sup>8</sup>译注：原书把循环范围及最后一个元素写成包含 `MAX_NUMS`，与图中严格小于的条件不符。合法下标为 0 至 `MAX_NUMS - 1`，这里已更正；正文中的 `number` 也统一为实际数组名 `numbers`。

```
1 #include <stdio.h>
2 #define MAX_NUMS 10
3
4 int Average(int input_values[]);
5
6 int main(void)
7 {
8 int mean; // Average of numbers
9 int numbers[MAX_NUMS]; // Original input numbers
10
11
12 // Get input
13 printf("Enter %d numbers.\n", MAX_NUMS);
14 for (int index = 0; index < MAX_NUMS; index++) {
15 printf("Input number %d : ", index);
16 scanf("%d", &numbers[index]);
17 }
18
19 mean = Average(numbers);
20
21 printf("The average of these numbers is %d\n", mean);
22 }
23
24 int Average(int inputValues[])
25 {
26 int sum = 0;
27
28 for (int index = 0; index < MAX_NUMS; index++) {
29 sum = sum + inputValues[index];
30 }
31
32 return (sum / MAX_NUMS);
33 }
```

图 16.10 把数组作为函数参数的例子

图 16.11 展示 Average 执行返回语句之前的运行时栈。

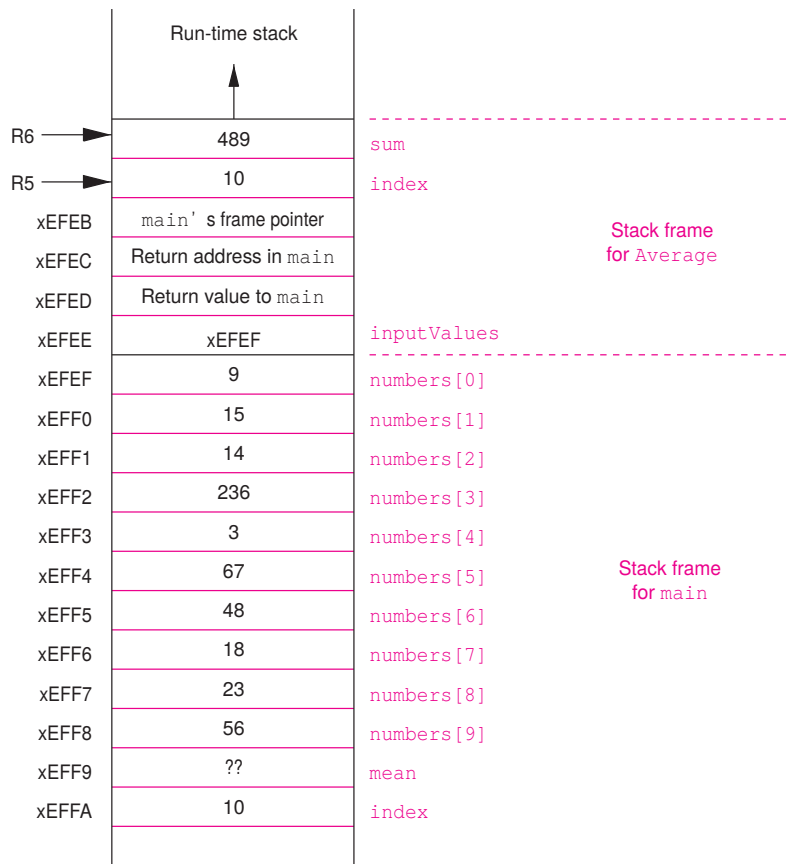


图 16.11 Average 执行 return 之前的运行时栈

注意 Average 声明中的数组形参写法。这里的空方括号使形参按“指向指定元素类型的指针”处理，本例就是整数指针。函数可以通过它修改原数组，因此修改会对调用者可见。<sup>9</sup>如果只想按值传递一个元素，应怎样写？如果要传递这个元素的地址，又该怎样写？

16.3.4 C 中的字符串

字符串是表示文本的字符序列。在 C 中，字符串通常保存在字符数组中，各元素依次保存一个字符。例如：

```
char word[10];
```

这个数组有十个字符位置，可以容纳 elephant 或 giraffe，却装不下完整的 hippopotamus。更长的字符串需要更大的数组。

giraffe 本身有七个字符，剩下三个位置。如果不作特殊处理，就可能无法判断字符串在哪里结束。为消除歧义，C 使用 ASCII 值为 0 的空字符标记字符串结尾。它充当哨兵，指出文本到此结束，因此这种字符串也称为以空字符结尾的字符串（null-

<sup>9</sup>译注：原书把数组名直接说成指针，并称 C 以引用方式传数组。更准确地说，数组是数组对象；在这里作实参时转换为首元素指针，而函数形参中的数组声明调整为指针声明。该指针仍按值传递。图 16.10 声明中的 input\_values 与定义中的 inputValues 名称不同，这是合法的，按原图保留。

terminated string)。在 C 代码中，用 '\0' 表示这个字符。延续上例，可以写成：

```
char word[10];
word[0] = 'g';
word[1] = 'i';
word[2] = 'r';
word[3] = 'a';
word[4] = 'f';
word[5] = 'f';
word[6] = 'e';
word[7] = '\0';
printf("%s", word);
```

这里逐个设置元素，形成字符串 giraffe。结束标记本身也占一个数组元素。因此，十元素数组最多保存九个文本字符；rhinoceros 有十个字母，无法同时容纳它和结束标记。<sup>10</sup>

例子还使用了新的 printf 格式说明 %s，从对应实参所指的字符开始打印，直到遇到结束标记，结束标记本身不输出。C 也允许在声明时初始化字符串，例如：

```
char word[10] = "Hello";
printf("%s", word);
```

注意两点：字符串字面量用双引号，单个字符如 'A' 用单引号；在这个数组容量足够的例子中，初始化会自动加入结尾的空字符。原书这里改用了 Hello，所以它演示相同的初始化方法，但与前面 giraffe 的文本内容不同。

#### 16.3.4.1 字符串示例

图 16.12 展示一个常见而有用的基本操作：计算字符串长度。数组大小不代表当前字符串的实际长度，只能据此确定能够安全容纳的最大长度。要求实际长度，就必须查看字符本身。算法很简单：从第一个字符开始计数，直到遇到空字符。StringLength 完成这一计算。

---

<sup>10</sup>译注：原书本节开头说十元素数组可容纳十字符字符串，随后才说明结束标记的空间要求。这里从一开始区分数组容量与字符串长度。空字符 '\0' 与空指针不是同一个概念。

```
1 #include <stdio.h>
2 #define MAX_STRING 20
3
4 int StringLength(char string[]);
5
6 int main(void)
7 {
8 char input[MAX_STRING]; // Input string
9 int length = 0;
10
11 printf("Input a word (less than 20 characters): ");
12 scanf("%s", input);
13
14 length = StringLength(input);
15 printf("The word contains %d characters\n", length);
16 }
17
18 int StringLength(char string[])
19 {
20 int index = 0;
21
22 while (string[index] != '\0')
23 index = index + 1;
24
25 return index;
26 }
```

图 16.12 计算字符串长度的程序

注意 `scanf` 中使用的 `%s`：它跳过前导空白，读入字符，遇到下一个空白时停止。空格、制表符、换行符、回车符、垂直制表符和换页符都属于这里所说的空白字符。例如，输入 Emma Lazarus 的《新巨人》(*The New Colossus*) 中的两行诗：

```
Not like the brazen giant of Greek fame,
With conquering limbs astride from land to land;
```

只有 `Not` 会存入 `input`，其余文本留待后续读取。再调用一次 `scanf("%s", input)`，便会读入 `like`。读取下一个单词时，`%s` 会自动跳过前面的空白。第 18 章深入讨论 C 的 I/O 时，还会分析这种行为。

这里数组容量为 20，所以最多容纳 19 个文本字符和一个结束标记。如果第一个单词更长会怎样？`scanf` 不知道 `input` 的容量，未指定字段宽度的 `%s` 会继续向后写，直到输入单词结束，因此可能越过数组边界，覆盖相邻存储位置。画出读取前后的栈帧，有助于理解问题。本章习题安排了相关练习，要求分析输入超过数组容量时的情况。<sup>11</sup>

<sup>11</sup>译注：原书误把安全长度说成 20，并把越界后覆盖局部变量的结果说得过于确定。20 个文本字符加结束标记

图 16.13 是稍复杂的例子，复用本章前面开发的函数。程序通过 `scanf` 读入字符串，调用函数反转，再打印结果。`Reverse` 先用 `StringLength` 求长度，再交换首尾字符、第二个与倒数第二个字符，依此类推。交换由图 16.4 的 `NewSwap` 改成字符版本 `CharSwap` 来完成。循环对每一对字符传入各自地址，完成反转。

```
1 #include <stdio.h>
2 #define MAX_STRING 20
3
4 int StringLength(char string[]);
5 void CharSwap(char *firstVal, char *secondVal);
6 void Reverse(char string[]);
7
8 int main(void)
9 {
10 char input[MAX_STRING]; // Input string
11
12 printf("Input a word (less than 20 characters): ");
13 scanf("%s", input);
14
15 Reverse(input);
16 printf("The word reversed is %s.\n", input);
17 }
18
19 int StringLength(char string[])
20 {
21 int index = 0;
22
23 while (string[index] != '\0')
24 index = index + 1;
25
26 return index;
27 }
28
29 void CharSwap(char *firstVal, char *secondVal)
30 {
31 char tempVal; // Temporary location for swapping
32
33 tempVal = *firstVal;
34 *firstVal = *secondVal;
35 *secondVal = tempVal;
36 }
37
38 void Reverse(char string[])
39 {
40 int length;
41
42 length = StringLength(string);
43
44 for (int index = 0; index < (length / 2); index++)
45 CharSwap(&string[index], &string[length - (index + 1)]);
46 }
```

图 16.13 反转字符串的程序

已需 21 个位置；越界访问在 C 中是未定义行为，具体覆盖对象取决于布局。对本例可用 `%19s` 限制一次写入长度，但这本身不等于处理完长单词剩余输入。图中原代码保留。解析文件把 `StringLength` 识别成 `Length`，已依据原图恢复。



C 标准库提供了许多现成的字符串操作，例如复制、连接、比较和求长度等，可以通过包含 `<string.h>` 得到相关函数的声明。

16.3.5 C 中数组与指针的关系

你可能已经注意到，数组名在许多表达式中的作用，与指向同类元素的指针很相似。例如：

```
char word[10];
char *cptr;
cptr = word;
```

这是一段合法而有用的代码，令 `cptr` 指向数组 `word` 的首元素。在此之后，访问第四个字符可以写成 `word[3]`，也可以写成 `*(cptr + 3)`。

不过，二者并不相同。`cptr` 是指针变量，可以重新赋值，例如出现在赋值运算符左边；`word` 是数组对象的名称，不能通过 `word = newArray`；这样的赋值改变整个数组，这种语句会引起编译错误。数组生存期间，它的各元素有确定的存储位置。

表 16.1 列出几组作用相同的表达式。假定 `cptr` 为 `char *`，`word` 为字符数组，且已执行 `cptr = &word[0]`；。表中的下标应在有效范围内。<sup>12</sup>

表 16.1 作用相同的表达式

|                   | 使用指针                     | 使用数组名                    | 使用数组下标                    |
|-------------------|--------------------------|--------------------------|---------------------------|
| 首元素地址             | <code>cptr</code>        | <code>word</code>        | <code>&amp;word[0]</code> |
| 第 0 个元素           | <code>*cptr</code>       | <code>*word</code>       | <code>word[0]</code>      |
| 第 <i>n</i> 个元素的地址 | <code>cptr + n</code>    | <code>word + n</code>    | <code>&amp;word[n]</code> |
| 第 <i>n</i> 个元素    | <code>*(cptr + n)</code> | <code>*(word + n)</code> | <code>word[n]</code>      |

16.3.6 问题求解：插入排序

掌握数组的基础后，可以尝试一个规模较大、也很实用的问题：编写 C 程序，把整数数组按升序排列。也就是说，排列完成后应满足

$$a[0] \leq a[1] \leq a[2] \leq \cdots \leq a[n - 1].$$

这里采用**插入排序**（insertion sort）。排序是计算中的重要基本操作，多年来，人们投入了大量精力理解、分析和改进排序算法。后续课程还会介绍许多其他方法。这里选择插入排序，是因为它与现实中整理物品的方式相似，也足够直观，适合据此编写 C 代码。

用例子最容易说明它。假设要按作者姓名的字母顺序整理书架。概念上，把书分成已排序和未排序两组，最初已排序组为空。每次从未排序组取出一本，插入已排序

<sup>12</sup>译注：这里的等价关系不意味着数组就是指针。数组表达式在多数使用场景会转换为首元素指针，但例如对数组使用 `sizeof` 或一元 `&` 时并不进行这种转换。表中原文“数组地址”实际指首元素地址；`&word` 则是指向整个数组的指针，类型不同。

组中的合适位置。例如，已排序组有 Faulkner、Fitzgerald 和 Steinbeck 的书，要放入 Hemingway 的书，就把它插在 Fitzgerald 与 Steinbeck 之间。不断重复，直到未排序组中的书全部插入。这就是插入排序。

怎样将这种方法用于整数数组？通过系统分解可知，程序的核心是遍历原数组，把每个元素插入另一个已经有序的数组中的正确位置。所有元素处理完后，新数组就包含与原数组相同的数据，只是顺序已经排好。

为了表示未排序和已排序两组元素，可以使用两个数组。不过，也可以在原数组内部同时表示这两组：前半部分是已经排好的元素，余下部分尚未排序。每次选出下一个未排序元素，插入前面的合适位置，直到处理完整个数组。这样节省内存，代码也较紧凑，只是逻辑略复杂一些。

图 16.14 的 InsertionSort 包含嵌套循环。外层逐一处理未排序元素，相当于依次拿起未整理的书。内层在已经排序的部分中寻找插入位置：从右向左检查，把比当前值大的元素向后移动，直到可以放入当前值。

```

1 #include <stdio.h>
2 #define MAX_NUMS 10
3
4 void InsertionSort(int list[]);
5
6 int main(void)
7 {
8
9 int numbers[MAX_NUMS]; // List of numbers to be sorted
10
11 // Get input
12 printf("Enter %d numbers.\n", MAX_NUMS);
13 for (int index = 0; index < MAX_NUMS; index++) {
14 printf("Input number %d : ", index);
15 scanf("%d", &numbers[index]);
16 }
17
18 InsertionSort(numbers); // Call sorting routine
19
20 // Print sorted list
21 printf("\nThe input set, in ascending order:\n");
22 for (int index = 0; index < MAX_NUMS; index++)
23 printf("%d\n", numbers[index]);
24 }
25
26 void InsertionSort(int list[])
27 {
28 int unsorted; // Index for unsorted list items
29 int sorted; // Index for sorted items
30 int unsortedItem; // Current item to be sorted
31
32 // This loop iterates from 1 thru MAX_NUMS
33 for (unsorted = 1; unsorted < MAX_NUMS; unsorted++) {
34 unsortedItem = list[unsorted];
35
36 // This loop iterates from unsorted thru 0, unless
37 // we hit an element smaller than current item
38 for (sorted = unsorted - 1;
39 (sorted >= 0) && (list[sorted] > unsortedItem);
40 sorted--)
41 list[sorted + 1] = list[sorted];
42
43 list[sorted + 1] = unsortedItem; // Insert item
44 }
45 }

```

图 16.14 C 实现的插入排序

仔细看看第 33–43 行的一次外层迭代。第 33 行使 `unsorted` 依次取 1、2、3、...、9。第四次迭代时它为 4。假定数组如下，第 34 行便把 15 保存到 `unsortedItem`：

```

list = [2 16 68 69 15 9 10 7 82 19]
unsorted = 4
unsortedItem = 15

```

这次迭代要把 `list[4]`，即 15，插入已有序的 `list[0]` 至 `list[3]` 之中。

第 38–41 行的内层循环让 `sorted` 从 3 向 0 递减。当找到不大于待插入值的元素，或已经移到最左端之外时，循环停止。

内层循环每次把一个已排序元素复制到后一个位置，为新元素腾出空间。第一次执行第 41 行，把 `list[3]` 复制到 `list[4]` 后，状态为：

```
list = [2 16 68 69 69 9 10 7 82 19]
unsorted = 4
unsortedItem = 15
sorted = 3
```

`list[4]` 原先的 15 已被覆盖，但它的副本仍保存在 `unsortedItem` 中，因此没有丢失。第二次执行第 41 行后：

```
list = [2 16 68 68 69 9 10 7 82 19]
unsorted = 4
unsortedItem = 15
sorted = 2
```

第三次执行第 41 行后：

```
list = [2 16 16 68 69 9 10 7 82 19]
unsorted = 4
unsortedItem = 15
sorted = 1
```

再次检查条件时，`sorted` 为 0，而 `list[0]` 为 2，并不大于 15，所以内层循环结束。随后，第 43 行 `list[sorted + 1] = unsortedItem`；将 15 放入空出的位置：

```
list = [2 15 16 68 69 9 10 7 82 19]
```

现在，有序部分又增加了一个元素。外层循环继续处理剩余元素，直到整个 `list` 排好序。<sup>13</sup>

### 16.3.7 C 数组的常见陷阱

与一些现代语言不同，C 不会自动检查数组访问是否超过边界。C 诞生时，内存与处理能力都很宝贵，程序员必须尽量节省操作开销。因此，C 没有为普通数组访问强制加入范围检查，这有利于实现高效代码，但也把避免越界的责任留给程序员。

表达式 `a[i]` 在语法上可以完全合法，`i` 的值却可能使访问超出数组范围。图 16.15 展示了越界可能导致的令人困惑的结果。输入一个大于数组容量的数，再通过运行时栈分析它，你就会更容易理解：向数组末端之外写入，可能破坏其他数据。<sup>14</sup>

<sup>13</sup>译注：原书把三次移动后的状态误写成“第 43 行执行后”，但第 43 行在内层循环结束后才执行；这里依代码改为第 41 行。原文还将停止条件说成找到“更小”元素，实际遇到相等元素也停止。图中循环注释的范围略宽于实际范围，代码本身保留。

<sup>14</sup>译注：越界读写属于未定义行为；栈帧图可以解释某种实现中观察到的现象，但不能据此断言任何 C 实现都必然出现同样结果。

```
1 #include <stdio.h>
2 #define MAX_SIZE 10
3
4 int main(void)
5 {
6 int index;
7 int array[MAX_SIZE];
8 int limit;
9
10 printf("Enter limit (integer): ");
11 scanf("%d", &limit);
12
13 for(index = 0; index < limit; index++) {
14 array[index] = 0;
15 printf("array[%d] is set to 0\n", index);
16 }
17 }
```

图 16.15 如果输入的数过大，这个 C 程序会因数组越界而出现异常行为。

### 16.3.8 变长数组

前面讨论的数组大小都是固定的，在编译时就已知。按照本书先前的简单栈帧分配方式，编译器需要知道准确大小，才能计算数组及其他变量相对于 R5 的偏移。如果大小未知，就不能原样沿用这套固定偏移方案。

但这一限制有时很不方便。我们常希望根据执行时遇到的实际情况决定数组容量。例如，图 16.14 的插入排序程序固定处理十个数，而我们可能希望它能处理任意给定规模的数据。

C99 引入了**变长数组**：数组大小可以由执行期间求值的整型表达式决定，而不必是编译期常量。下面的片段展示了这种声明：

```
int Calculate(int len)
{
 int data[len];
 /* ... */
}
```

data 的大小在函数执行到声明时才确定。在本书讨论的实现中，它仍可分配在运行时栈上，但需要不同于固定大小数组的分配方法。分配和访问可能需要额外指令，因此这种方便性会带来一定开销。<sup>15</sup>

<sup>15</sup>译注：这里为原书未写完的代码片段补上省略号和右花括号，表示后续函数体省略。变长数组的长度必须为正；C11/C17 中对变长数组的支持是可选的，它也不是标准 C++ 的数组特性。具体分配方式及性能取决于实现。

本书示例通常比较简单，变长数组很适合其中一些场景，后续章节偶尔也会使用它。但变长数组在某次创建后，大小就不能再改变。若需要更一般的可变规模数据结构，可以使用 C 中的动态分配数组或 C++ 的 `vector`；本书最后几章将讨论它们。

### 16.3.9 C 中的多维数组

用智能手机拍照，是一个简单动作，却会引发大量内部活动。一部分涉及物理和光学：光线通过镜头，在 CMOS 图像传感器上成像，可以把它想成手机相机的数字视网膜。另一大部分是数字处理：传感器和相关软件把捕获的信号变成高质量图片。

图片中的每个**像素**（pixel, picture element 的缩写）表示某个位置的红、绿、蓝光强。整个图像就是按二维网格排列的像素集合，每个像素属于特定行和列。例如，一幅图像可以有 1080 行、1920 列，总计约两百万像素。假定每个像素都可以用一个 C 整数表示，整个图像应该怎样表示？

数组很容易推广到图像这样的多维对象。C 的语法如下：

```
int image[1080][1920];
```

这声明一个二维数组：第一维为 1080，表示行数；第二维为 1920，表示列数。访问元素时需要两个下标，一个指定行，一个指定列。例如：

```
image[38][283]
```

访问的是第 39 行、第 284 列的像素。记住，C 的下标从 0 开始。

二维数组也必须映射到一维的内存地址空间。对于一维数组，下标依次增加，地址也依次增加。二维时，可以按列连续排列，也可以按行连续排列。C 和 C++ 使用**行主序**（row-major order）：同一行中相邻列的元素存放在相邻位置，一行结束后接下一行。<sup>16</sup>图 16.16 展示了这种映射。

---

<sup>16</sup>译注：原书此处误写为“每一列”的元素连续，与行主序及图 16.16 都矛盾。这里改为每一行连续，也就是最右侧下标变化最快。

|            |                |
|------------|----------------|
|            |                |
| x0001 EA00 | image[0][0]    |
| x0001 EA01 | image[0][1]    |
| x0001 EA02 | image[0][2]    |
| x0001 EA03 | image[0][3]    |
|            | ⋮              |
| x0001 F17E | image[0][1918] |
| x0001 F17F | image[0][1919] |
| x0001 F180 | image[1][0]    |
| x0001 F181 | image[1][1]    |
|            | ⋮              |
| x0001 F8FE | image[1][1918] |
| x0001 F8FF | image[1][1919] |
| x0001 F900 | image[2][0]    |
| x0001 F901 | image[2][1]    |
|            | ⋮              |

图 16.16 C 中二维数组映射到内存的方式

这幅图像太大，无法放入 LC-3 的 16 位地址空间。LC-3 只有  $2^{16} = 65536$  个存储位置，而图像有  $1080 \times 1920 = 2073600$  个整数元素。因此，为便于说明，图中把地址空间扩展为 32 位，并沿用 一个整数占一个可寻址位置的示意模型。

再看一个简单的图像处理函数。假定每个整数只表示相应像素的亮度，数值越大就越亮，相当于像素的“白色强度”。更一般的表示还会包含颜色，这里先简化为灰度。图 16.17 的 Adjust 将所有像素值除以 2。你预计处理后的图像看起来会怎样？<sup>17</sup>

<sup>17</sup>译注：原图把 Adjust 声明为返回 int，却没有返回语句。若它仅用于原地修改图像，应声明为 void；原图保留。

```
1 #define ROWS 1080
2 #define COLS 1920
3
4 int Adjust(int image[ROWS][COLS])
5 {
6 for (int row = 0; row < ROWS; row++) {
7 for (int col = 0; col < COLS; col++) {
8 image[row][col] = image[row][col] / 2;
9 }
10 }
11 }
```

图 16.17 基于二维数组的简单图像处理函数

多维数组也可以超过二维。增加一组方括号，就能声明三维数组：

```
int dataVolume[40][50][60]; // This declares 120,000 elements
```

三维数组遵循相同规律：最右侧下标对应的元素连续存放，下标从右向左进位。因此，`dataVolume[0][0][59]` 后面紧接着的元素就是 `dataVolume[0][1][0]`。

## 16.4 小结

本章讨论了指针与数组这两种重要的高级语言构件，它们都支持间接访问内存。关键概念如下。

- **指针**。指针变量保存其他存储对象的地址，通过指针可以间接访问和修改这些对象。一个简单用途是向函数传入对象地址，使它能修改调用者的数据。后续章节还会介绍更丰富的用途。
- **数组**。数组由类型相同、在内存中连续排列的元素组成，通过相对于起始位置的下标访问。许多现实对象适合表示为数组，因此它是重要的数据组织形式。例如，字符数组可以保存文本字符串。本章讨论了多种数组操作，包括插入排序，还介绍了大小在执行期间确定的变长数组，以及多维数组。



## 习题

- 16.1** 编写一个 C 函数，形参为一个长度未知、只含单个单词的字符串。函数要按以下规则把它从英语转换成 Pig Latin：取下首字母，移到词尾，再追加 ay。可以假定数组有足够空间容纳新增字符。例如，传入 Hello，函数返回后字符串应为 elloHay，第一个字符应为 e。
- 16.2** 编写 C 程序，持续接收用户输入的数，直到某个数与紧挨着它的前一个数相同为止。然后打印输入的数的个数与总和，均不计最后那个重复数。运行时的提示与输入输出应类似：

```
Number: 5
Number: -6
Number: 0
Number: 45
Number: 45
4 numbers were entered and their sum is 44
```

- 16.3** 编译并运行下面的代码，会输出什么？

```
int x;
int main(void)
{
 int *px = &x;
 int x = 7;

 *px = 4;
 printf("x = %d\n", x);
}
```

译注：原题未列出头文件；独立编译应包含 *stdio.h*，以声明 *printf*。

- 16.4** 编写一个字符串函数，接收 stringA 和 stringB 两个字符串：相同则返回 0；若按字典顺序 stringA 在前，返回 1；若 stringB 在前，返回 2。
- 16.5** 使用习题 16.4 编写的函数，修改插入排序程序，使它对字符串而不是整数排序。

**16.6** 将下面的 C 函数翻译为 LC-3 汇编语言。

```
int main(void)
{
 int a[5], i;
 i = 4;
 while (i >= 0) {
 a[i] = i;
 i--;
 }
}
```

**16.7** 考察下面的程序。注意，ind 是指向另一个指针变量的指针，这在 C 中是合法的。画出 apple++; 刚执行完时的运行时栈，分析程序完成了什么操作。

```
#include <stdio.h>
int main(void)
{
 int apple;
 int *ptr;
 int **ind;
 ind = &ptr;

 *ind = &apple;
 **ind = 123;
 ind++;
 *ptr++;
 apple++;
 printf("%x %x %d\n", ind, ptr, apple);
}
```

译注：原题最后用 %x 打印指针，格式与实参类型不匹配。若实际运行到输出语句，应使用 %p 并将相应实参转换为 void \*。题目要求的栈快照位于该输出之前；原代码保留。

**16.8** 下面的代码调用了函数 `triple`。`triple` 的栈帧最小需要多大?

```
int main(void)
{
 int array[3];
 array[0] = 1;
 array[1] = 2;
 array[2] = 3;
 triple(array);
}
```

译注：原题没有给出 `triple` 的声明或定义。应按本章调用约定讨论最小需求；不能据此确定未给出的函数体实际需要多少局部空间。

**16.9** 编写程序，删除数列中的重复值。例如，输入 5、4、5、5、3，输出应为 5、4、3。

**16.10** 编写程序，求一组数的中位数。中位数位于排序后的中间位置。提示：可能需要先排序。<sup>18</sup>

---

<sup>18</sup>译注：原书称中位数一定是集合中的某个数，且一半的数比它大、一半比它小，这只适用于部分情况。通常，偶数个数据的中位数取中间两项的平均值；有重复值时也应按排序位置定义。作答时应明确采用的约定。

**16.11** 参考下面的 C 程序，回答问题。

```
int FindLen(char *);
int main(void)
{
 char str[10];

 printf("Enter a string : ");
 scanf("%s", str);
 printf("%s has %d characters\n", str, FindLen(str));
}

int FindLen(char * s)
{
 int len=0;
 while (*s != '\0') {
 len++;
 s++;
 }
 return len;
}
```

- a. main 和 FindLen 的栈帧分别有多大?
- b. 输入字符串 apple 时，画出 FindLen 即将返回前的栈内容。
- c. 如果运行程序时输入长度超过十个字符的字符串，栈帧会变成什么样？程序会发生什么？

译注：原题未列出 *stdio.h*，独立编译时应补上。十元素字符数组至多容纳九个文本字符与结束标记，输入十个字符便已越界；c 小题按原文保留。越界后不具有 C 语言保证的确定结果。

**16.12** 下面的代码从键盘读入字符串，并把其中的大写字母转换为小写后打印。然而，代码存在缺陷。请找出问题。

```
#include <stdio.h>
#define MAX_LEN 10

char *LowerCase(char *s);
int main(void)
{
 char str[MAX_LEN];
 printf("Enter a string : ");
 scanf("%s", str);
 printf("Lowercase: %s \n", LowerCase(str));
}

char *LowerCase(char *s)
{
 char newStr[MAX_LEN];
 for (int index = 0; index < MAX_LEN; index++) {
 if ('A' <= s[index] && s[index] <= 'Z')
 newStr[index] = s[index] + ('a' - 'A');
 else
 newStr[index] = s[index];
 }
 return newStr;
}
```

**16.13** 考虑下面的声明。

```
#define STACK_SIZE 100
int stack[STACK_SIZE];
int topOfStack;
int Push(int item);
```

- a. 编写 Push 函数，声明已给出，将 item 的值压入栈顶。如果栈满，无法压入，则返回 1；成功则返回 0。
- b. 编写 Pop 函数，弹出栈顶元素。与 Push 相同，操作失败时返回 1，例如试图从空栈弹出；成功时返回 0。仔细考虑怎样把弹出的值交给调用者。

**16.14** 编写函数，接收用二维整数数组表示的图像，其中各整数表示对应像素的亮度；生成一个直方图，统计各亮度值的出现次数。为简化问题，假定亮度只在 0 至 15 之间，因此直方图只需 16 个计数槽。

- a. 假定图像尺寸固定，可以直接写在代码中，实现该函数。
- b. 假定图像尺寸通过函数参数提供，实现该函数。

# 第十七章 递归

## 17.1 引言

假设要在一叠已经按姓名字母顺序排好的试卷中，找到某个学生的试卷。可以在中间任取一个位置，检查是否匹配。如果找到了，任务便完成；如果没有找到，就根据目标姓名应排在当前姓名之前还是之后，判断只需继续搜索上面或下面的一部分。关键是：我们又可以对一叠更小的试卷重复同样的过程。例如，要找 Mira 的试卷，而选中的位置是 Salina 的试卷，就知道目标如果存在，一定在 Salina 之前，于是继续搜索这一小叠。这样便能较快找到目标，或确定它不存在。

这种方法就是递归：通过求解同一问题的更小版本，解决原问题。这里，原问题是在整叠试卷中查找，子问题是在其中较小的一叠中查找。第 8 章讨论 LC-3 汇编子程序时，我们已经接触过递归。现在编程的抽象层次提高了，值得在 C 语言中重新审视它。

递归和迭代都描述重复进行的计算。递归的优势在于，某些任务的计算过程可以用它清晰而简洁地表达。有些问题的递归解法比相应迭代解法简单得多。不过，递归通常会增加执行开销，因此应当在理解底层代价之后谨慎使用。

本章通过多个例子介绍递归。与第 8 章一样，我们会研究它在 LC-3 上的实现，这次则借助运行时栈。运行时栈的巧妙之处在于：递归函数不需要特殊处理，它们与其他函数以相同方式执行。本章希望让你初步而深入地理解递归，开始在自己的程序中实现递归算法。

## 17.2 什么是递归？

从一个简单例子重温基本思想。调用自身的函数称为递归函数。图 17.1 的 RunningSum 计算从输入参数  $n$  到 1 的整数之和。例如，RunningSum(4) 计算  $4 + 3 + 2 + 1$ ，但采用递归方式。4 的累加和等于 4 加上 3 的累加和；3 的累加和又等于 3 加上 2 的累加和。因此，

$$\text{RunningSum}(n) = n + \text{RunningSum}(n - 1), \quad n > 1.$$

数学上，这样的式子称为**递推方程**（recurrence equation）。为了完成计算，还必须给出初始的基本情况：

$$\text{RunningSum}(1) = 1.$$

于是可以完整展开：

$$\begin{aligned}\text{RunningSum}(4) &= 4 + \text{RunningSum}(3) \\ &= 4 + 3 + \text{RunningSum}(2) \\ &= 4 + 3 + 2 + \text{RunningSum}(1) \\ &= 4 + 3 + 2 + 1.\end{aligned}$$

```
1 int RunningSum(int n)
2 {
3 if (n == 1)
4 return 1;
5 else
6 return (n + RunningSum(n-1));
7 }
```

图 17.1 一个简单的递归函数

C 程序与递推方程以相同方式工作。执行 `RunningSum(4)` 时，函数以实参 3 调用自身；这次调用尚未结束，又调用 `RunningSum(2)`，后者再调用 `RunningSum(1)`。最后一次调用不再递归，而是返回 1。`RunningSum(2)` 因而能够返回  $2 + 1$ ，接着 `RunningSum(3)` 返回  $3 + 2 + 1$ ，最终完成最外层调用。图 17.2 展示这一控制流。

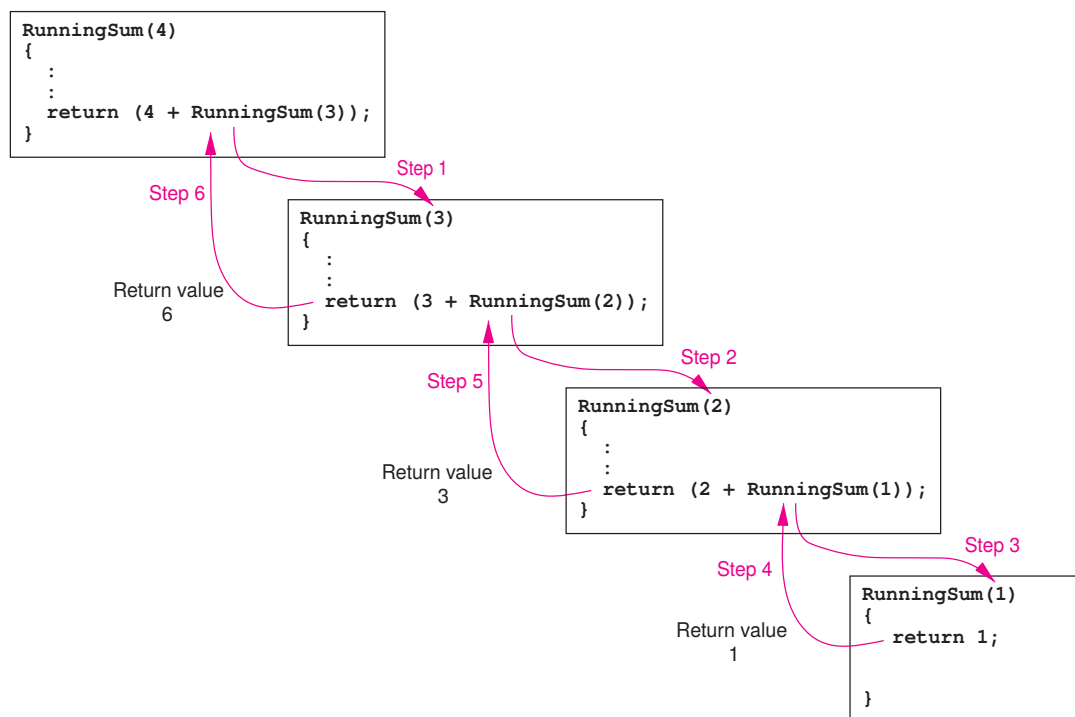


图 17.2 执行 `RunningSum(4)` 时的控制流



## 17.3 递归与迭代

显然，RunningSum 完全可以，也更适合用 for 循环编写，这样比递归版本更直观。这里采用递归，只是为了在容易理解的问题中展示递归调用。

递归与普通的 for、while 迭代之间有对应关系。递归算法都可以改写为迭代形式，必要时由程序显式维护栈。不过，对某些问题，递归版本更简单、更自然。例如，用递推方程定义的问题，本身就适合递归求解。这也是递归成为重要编程技术的原因。判断什么问题适合递归、什么问题更适合迭代，是需要通过实践培养的能力。

递归的便利并非没有代价。可以尝试编写迭代版 RunningSum，比较不同较大  $n$  下的运行时间。方法是在函数开始前和结束后读取时间，例如使用环境提供的 gettimeofday，再绘制不同  $n$  的耗时。若编译器没有消除递归，通常会看到递归版本较慢，因为它承担了迭代版本没有的函数调用开销。某些递归可以由编译器转换为迭代；当递归调用位于函数末尾、返回后不再需要计算时，称为**尾递归**（tail recursion）。<sup>1</sup>

自底向上的学习方法使我们能够看清这些开销，从而更有依据地决定何时使用递归。

## 17.4 汉诺塔

经典的**汉诺塔**（Towers of Hanoi）就是递归解法比迭代更容易表达的问题。它有三根柱子，其中一根套着若干圆盘，上面的盘总比下面的小。目标是把所有盘移到另一根柱子上。移动必须遵守两条规则：一次只能移动一个盘；大盘不能放到小盘上。图 17.3 的五个盘都在柱 1 上，求解就是按规则把它们全部移到另一根柱子。有关汉诺塔的传说称，世界创立时，梵天寺的僧侣受命移动 64 个圆盘，等到他们完成任务，世界就会结束。

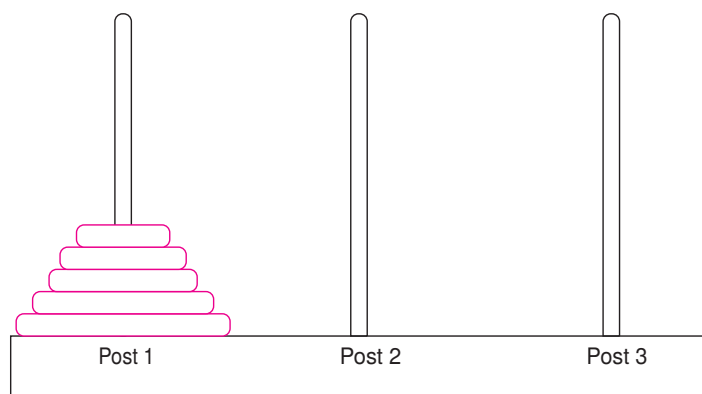


图 17.3 五个圆盘的汉诺塔

怎样编写求解程序？从最终目标倒推：必须先把最大盘从柱 1 移到目标柱，例如

<sup>1</sup>译注：RunningSum 的原始写法在递归返回后还要做加法，并不是直接的尾递归；编译器是否能够进一步转换，取决于优化。gettimeofday 是特定环境提供的接口，不是 ISO C 标准库函数。计时时还应避免整数溢出或栈空间耗尽，以免把错误执行当成性能结果。

柱 3，再把其他盘放到它上面。概念上，这分为三步：先把上面的  $n - 1$  个盘移到中间柱；再把最大盘移到目标柱；最后把  $n - 1$  个盘从中间柱移到目标柱。

一次移动  $n - 1$  个盘并不是合法的单步操作，但它把原问题分解成了两个更小的同类问题。若用  $\text{Move}(n, \text{target})$  表示把  $n$  个盘移到目标柱，就先求解  $\text{Move}(n-1, \text{intermediate})$ ，移动第  $n$  个盘，再求解  $\text{Move}(n-1, \text{target})$ 。这样，对  $n$  个盘的移动就由两次  $n - 1$  规模的递归调用和一次实际移动构成。

与递推方程一样，递归定义需要能够直接求解、终止递归的**基本情况**。这里，移动最小的盘，即盘 1，就是基本情况：它上面没有别的盘，可以直接移动。若没有可到达的基本情况，递归就无法结束，类似迭代中的无限循环。图 17.4 将这一思路直接写成 C 函数。

```
1 // diskNumber is the disk to be moved (disk1 is smallest)
2 // startPost is the post the disk is currently on
3 // endPost is the post we want the disk to end on
4 // midPost is the intermediate post
5 void MoveDisk(int diskNumber, int startPost, int endPost, int midPost)
6 {
7 if (diskNumber > 1) {
8 // Move n-1 disks off the current disk on
9 // startPost and put them on the midPost
10 MoveDisk(diskNumber-1, startPost, midPost, endPost);
11
12 printf("Move disk %d from post %d to post %d.\n",
13 diskNumber, startPost, endPost);
14
15 // Move all n-1 disks from midPost onto endPost
16 MoveDisk(diskNumber-1, midPost, endPost, startPost);
17 }
18 else
19 printf("Move disk 1 from post %d to post %d.\n",
20 startPost, endPost);
21 }
```

图 17.4 求解汉诺塔的递归函数

考察三个盘的求解过程。开始时三个盘都在柱 1 上，如图 17.5。初始调用表示把盘 3 以及它上面的盘移到柱 3，借柱 2 中转：

```
// diskNumber 3; startPost 1; endPost 3; midPost 2
MoveDisk(3, 1, 3, 2)
```

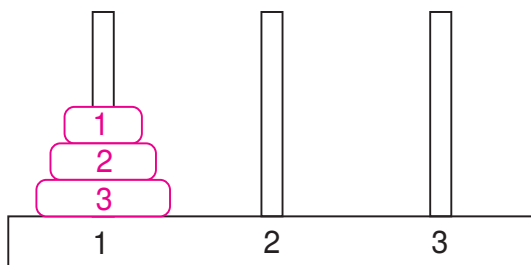


图 17.5 三个盘的汉诺塔：初始状态

为了移开盘 3 上面的盘 1、2，第 10 行产生下面的递归调用，要把它们移到柱 2，借柱 3 中转：

```
// diskNumber 2; startPost 1; endPost 2; midPost 3
MoveDisk(2, 1, 2, 3)
```

要移动盘 2，必须先把盘 1 移到柱 3，所以又在第 10 行调用：

```
// diskNumber 1; startPost 1; endPost 3; midPost 2
MoveDisk(1, 1, 3, 2)
```

这次第 7 行条件不成立，盘 1 可以直接移动。第 19–20 行打印：

```
Move disk 1 from post 1 to post 3.
```

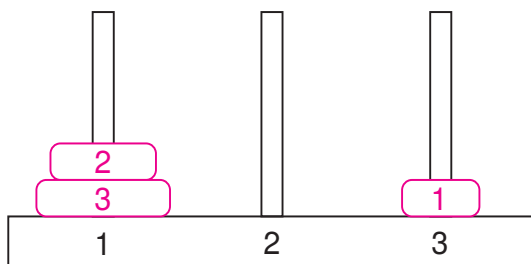


图 17.6 第一次移动后

随后返回 `MoveDisk(2, 1, 2, 3)`。盘 2 上的盘已经移开，于是第 12–13 行打印下一步：<sup>2</sup>

```
Move disk 2 from post 1 to post 2.
```

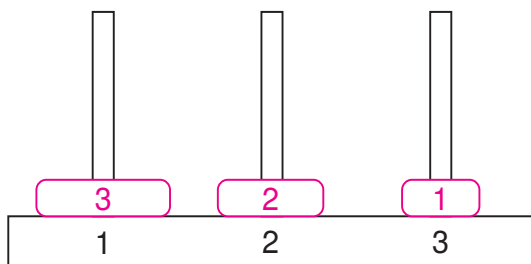


图 17.7 第二次移动后

<sup>2</sup>译注：原书把这里的输出行号写为 19–20，但当前调用的盘号为 2，执行的是 `if` 分支中的第 12–13 行。

接着，第 16 行调用函数，把原先位于盘 2 上面的盘移回盘 2 上：

```
// diskNumber 1; startPost 3; endPost 2; midPost 1
MoveDisk(1, 3, 2, 1)
```

由于只有盘 1，直接打印下面的移动，得到图 17.8 的状态。

```
Move disk 1 from post 3 to post 2.
```

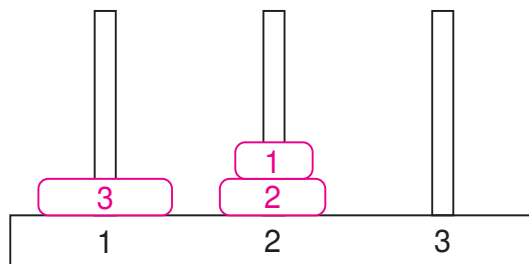


图 17.8 第三次移动后

`MoveDisk(2, 1, 2, 3)` 完成任务，返回 `MoveDisk(3, 1, 3, 2)`。盘 3 上面的盘已经全部移到柱 2，现在可以移动盘 3：

```
Move disk 3 from post 1 to post 3.
```

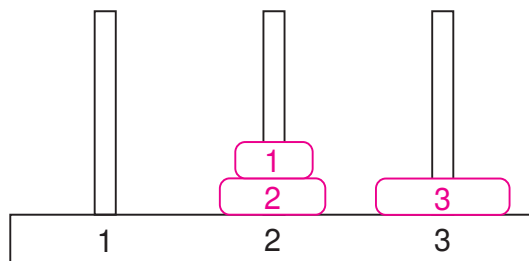


图 17.9 第四次移动后

下一项任务是把盘 2 及其上面的盘从柱 2 移到柱 3，借柱 1 中转。第 16 行产生：

```
// diskNumber 2; startPost 2; endPost 3; midPost 1
MoveDisk(2, 2, 3, 1)
```

为此，先在这次调用的第 10 行把盘 1 从柱 2 移到柱 1：

```
// diskNumber 1; startPost 2; endPost 1; midPost 3
MoveDisk(1, 2, 1, 3)
```

无需再分解，直接移动：

```
Move disk 1 from post 2 to post 1.
```

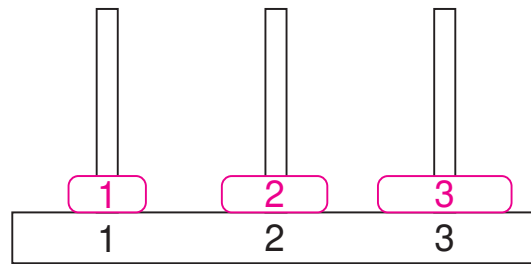


图 17.10 第五次移动后

返回 `MoveDisk(2, 2, 3, 1)` 后, 移动盘 2:

Move disk 2 from post 2 to post 3.

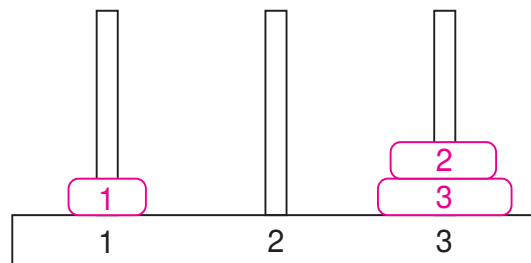


图 17.11 第六次移动后

最后只需把盘 1 放回盘 2 上:

```
// diskNumber 1; startPost 1; endPost 3; midPost 2
MoveDisk(1, 1, 3, 2)
```

这一步立即完成, 打印:

Move disk 1 from post 1 to post 3.

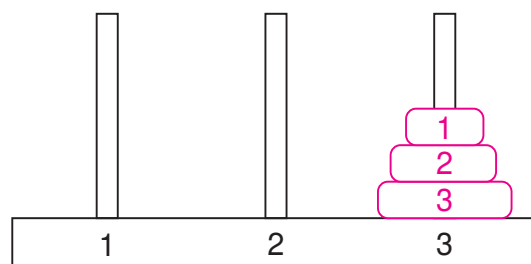


图 17.12 汉诺塔求解完成。

整个递归调用次序如下。缩进表示调用的嵌套关系。<sup>3</sup>

```
MoveDisk(3, 1, 3, 2) // Initial Call
 MoveDisk(2, 1, 2, 3)
 MoveDisk(1, 1, 3, 2)
 MoveDisk(1, 3, 2, 1)
 MoveDisk(2, 2, 3, 1)
 MoveDisk(1, 2, 1, 3)
 MoveDisk(1, 1, 3, 2)
```

试着想一想迭代版本要怎样写，你就会体会到递归表达的简洁。再回到传说：三个盘需要七次移动。如果每次移动用一秒，64 个盘需要多久？如果改用迭代求解，所需移动次数会不同吗？

## 17.5 Fibonacci 数

再看第 8.3.2 节的例子。当时我们说，直接递归计算 Fibonacci 数不是递归的好应用；换成 C 也仍然如此。不过，它值得重新研究：函数足够短，运行时栈的行为却很有代表性，而且可以完整地翻译成 LC-3 汇编，观察运行时栈如何支持递归。

下面的递推方程产生著名的 Fibonacci 数列：

$$f(n) = f(n-1) + f(n-2) \quad (n \geq 2), \quad f(1) = 1, \quad f(0) = 1.$$

第  $n$  项是前两项之和，序列为 1, 1, 2, 3, 5, 8, 13, ...。本书沿用从两个 1 开始、第一项编号为 0 的约定。

约在公元 1200 年，比萨的意大利数学家 Leonardo 推广了这一数列；原书提到，印度数学家可能早在公元前 200 年左右就描述过它。原书将 Fibonacci 这一名称解释为 *filius Bonacci*，即“Bonacci 之子”的缩写。Fibonacci 用这类数列估算兔子的繁殖数量。后来，人们发现它还能描述一些自然现象中的规律，例如螺旋壳的结构、花瓣的排列。相邻两项之比会趋近黄金分割比。

递归实现直接来自方程：Fibonacci( $n$ ) 等于 Fibonacci( $n-1$ ) + Fibonacci( $n-2$ )，基本情况是参数为 0 或 1 时都返回 1。图 17.13 给出了代码。

---

<sup>3</sup>译注：原书第三次移动前的调用及总调用序列误写为 MoveDisk(1, 2, 3, 1)，与圆盘位置矛盾；应为 MoveDisk(1, 3, 2, 1)，这里同步修正实参和注释。原书第五次移动前误引第 16 行，实际是该层调用的第 10 行。调用列表是过程示意，不是完整 C 语句。

```
1 #include <stdio.h>
2
3 int Fibonacci(int n);
4
5 int main(void)
6 {
7 int in;
8 int number;
9
10 printf("Which Fibonacci number? ");
11 scanf("%d", &in);
12
13 number = Fibonacci(in);
14 printf("That Fibonacci number is %d\n", number);
15 }
16
17 int Fibonacci(int n)
18 {
19 int sum;
20
21 if (n == 0 || n == 1)
22 return 1;
23 else {
24 sum = (Fibonacci(n-1) + Fibonacci(n-2));
25 return sum;
26 }
```

图 17.13 递归计算第  $n$  个 Fibonacci 数的 C 函数

译注：原图第 26 行之后还缺少一个右花括号，以闭合函数体；独立编译时需要补上。

现在深入考察底层实现。无论调用来自别的函数还是函数自身，每次调用都会在运行时栈上建立一个新的栈帧，保存这一层调用自己的形参和局部变量。调用完成后，相应空间被回收。若每层递归都共用同一组静态存储位置，后一层就会覆盖前一层还需要使用的值；运行时栈自然地避免了这种冲突。

以 `Fibonacci(3)` 为例，图 17.14 展示了计算过程中的栈快照。本例实现先计算表达式左侧，再计算右侧。<sup>4</sup>

1. 初始调用把 `Fibonacci(3)` 的栈帧压入栈。
2. 为计算左侧 `Fibonacci(2)`，再压入它的栈帧。

<sup>4</sup>译注：原书把先计算左侧说成 C 表达式的一般求值规则。C 并不规定加法两个操作数的求值先后；这里按图 17.15 所展示的实现次序说明。

- 3. 参数 2 尚未达到基本情况，因此先调用 Fibonacci (1)。它直接返回 1。
- 4. 随后调用 Fibonacci (0)，也直接返回 1。Fibonacci (2) 得到 1 + 1 = 2，返回上一层。
- 5. Fibonacci (3) 再计算右侧 Fibonacci (1)，得到 1。
- 6. 最初调用得到 2 + 1 = 3，返回调用者，栈恢复到开始前的状态。

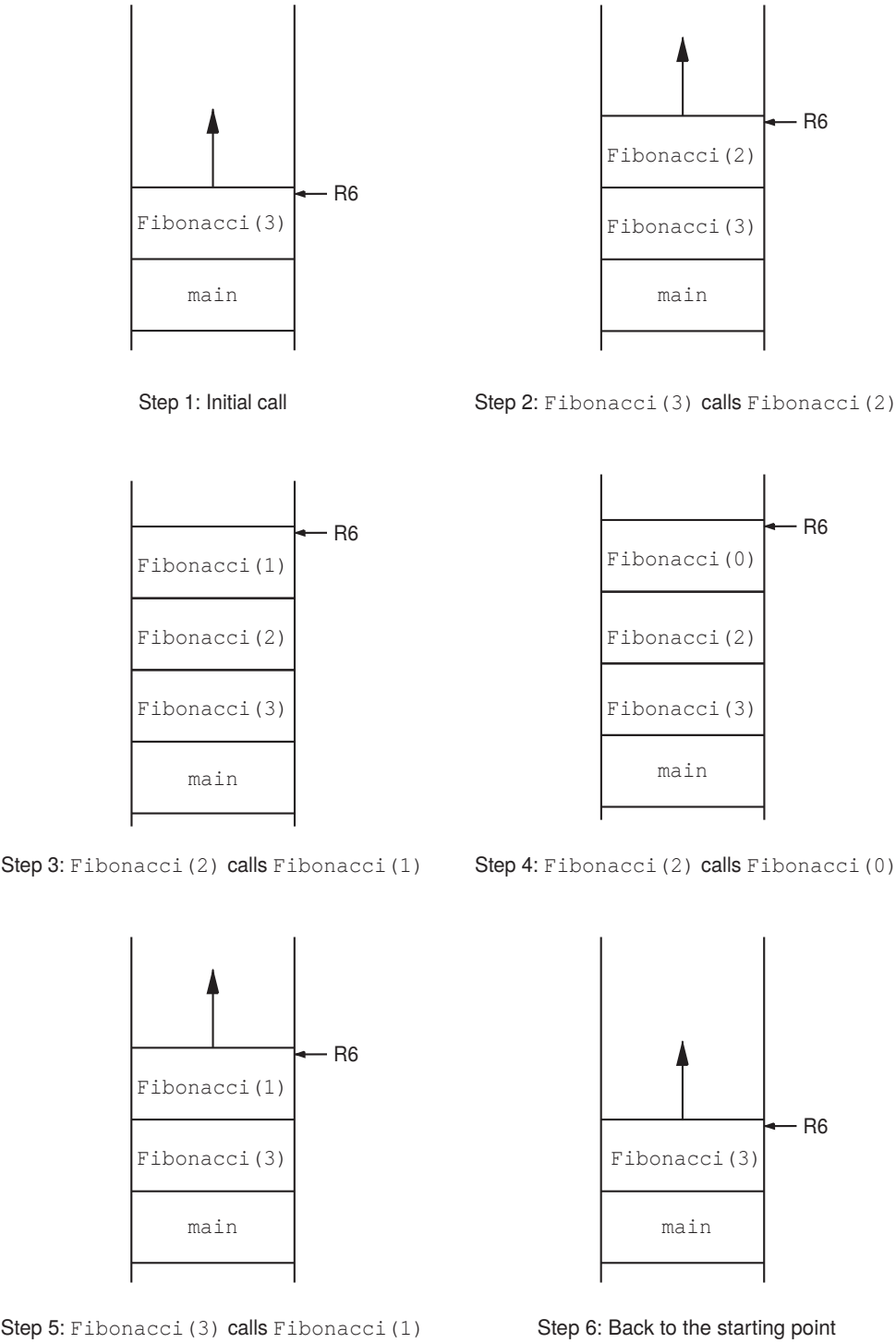


图 17.14 计算 Fibonacci (3) 时的运行时栈快照



用代数方式表示，就是

$$\begin{aligned}\text{Fibonacci}(3) &= \text{Fibonacci}(2) + \text{Fibonacci}(1) \\ &= (\text{Fibonacci}(1) + \text{Fibonacci}(0)) + \text{Fibonacci}(1) \\ &= 1 + 1 + 1 = 3.\end{aligned}$$

调用顺序如下：

```
Fibonacci(3)
 Fibonacci(2)
 Fibonacci(1)
 Fibonacci(0)
 Fibonacci(1)
```

再跟踪 `Fibonacci(4)`，会发现它包含计算 `Fibonacci(3)` 的全部调用，因为  $f(4) = f(3) + f(2)$ 。类似地，`Fibonacci(5)` 又包含计算 `Fibonacci(4)` 的调用。本章习题将要求计算一般情形中的调用次数。与第 8.3.2 节一样，可以比较直接递归与迭代版本的运行时间。递归版本虽然直接来自递推方程、容易编写，却昂贵得多：不仅有调用开销，还有大量迭代版本不需要的重复计算。

图 17.15 给出 `Fibonacci` 函数的 LC-3 汇编实现。运行时栈使递归调用能够与普通调用同样处理。注意，为了计算两次调用结果之和，编译器使用了一个临时存储位置，保存第一次调用的结果。编译复杂表达式时，经常需要这样的临时量，通常分配在栈帧中、程序员声明的局部变量之外。

```
1 Fibonacci:
2 ADD R6, R6, #-2 ; push return value/address
3 STR R7, R6, #0 ; store return address
4 ADD R6, R6, #-1 ; push caller's frame pointer
5 STR R5, R6, #0 ;
6 ADD R5, R6, #-1 ; set new frame pointer
7 ADD R6, R6, #-2 ; allocate space for locals and temps
8
9 LDR R0, R5, #4 ; load the parameter n
10 BRZ FIB_BASE ; check for n == 0
11 ADD R0, R0, #-1 ;
12 BRZ FIB_BASE ; n==1
13
14 LDR R0, R5, #4 ; load the parameter n
15 ADD R0, R0, #-1 ; calculate n-1
16 ADD R6, R6, #-1 ; push n-1
17 STR R0, R6, #0 ;
18 JSR Fibonacci ; call to Fibonacci(n-1)
19
20 LDR R0, R6, #0 ; read the return value at top of stack
21 ADD R6, R6, #-1 ; pop return value
22 STR R0, R5, #-1 ; store it into temporary value
23 LDR R0, R5, #4 ; load the parameter n
24 ADD R0, R0, #-2 ; calculate n-2
25 ADD R6, R6, #-1 ; push n-2
26 STR R0, R6, #0 ;
27 JSR Fibonacci ; call to Fibonacci(n-2)
28
```

图 17.15 Fibonacci 函数的 LC-3 汇编代码，前半部分

```

29 LDR R0, R6, #0 ; read the return value at top of stack
30 ADD R6, R6, #-1 ; pop return value
31 LDR R1, R5, #-1 ; read temporary value: Fibonacci(n-1)
32 ADD R0, R0, R1 ; Fibonacci(n-1) + Fibonacci(n-2)
33 BR FIB_END ; branch to end of code
34
35 FIB_BASE:
36 AND R0, R0, #0 ; clear R0
37 ADD R0, R0, #1 ; R0 = 1
38
39 FIB_END:
40 STR R0, R5, #3 ; write the return value
41 ADD R6, R5, #1 ; pop local variables
42 LDR R5, R6, #0 ; restore caller's frame pointer
43 ADD R6, R6, #1 ;
44 LDR R7, R6, #0 ; pop return address
45 ADD R6, R6, #1 ;
46 RET

```

图 17.15 Fibonacci 函数的 LC-3 汇编代码（续）

原图中的出栈指令需要特别留意：第 21、30 行写成 `ADD R6, R6, #-1`，实际会继续降低栈指针，与“pop return value”的注释相反。<sup>5</sup>

## 17.6 二分查找

本章开头描述了在按字母顺序排列的试卷中查找某份试卷的方法。若每次选择当前范围的中点，就得到**二分查找**（binary search），它能快速查找有序列表中的元素。现在，凭借对递归和数组的理解，我们可以用 C 实现它。

假设要在升序整数数组中查找一个数：找到则返回下标，不存在则返回 -1。先检查中间元素，有三种情况：目标等于中间元素，搜索完成；目标更小，只搜索前半部分；目标更大，只搜索后半部分。后两种情况都可以写成递归调用。

如果目标根本不存在，怎么办？搜索范围会不断缩小，最终变成不包含任何元素的空区间，此时返回 -1。这是递归的一个基本情况。

图 17.16 是具体实现。每次调用 `BinarySearch` 时，除了目标值和数组，还要传入当前子区间的起点与终点。`start` 是第一个待查元素的下标，`end` 是最后一个待查元素的下标。递归调用不断缩小这个范围。

<sup>5</sup>译注：按本书调用约定，子调用返回后，`R6` 指向返回值槽，其后还有一个实参槽。若在这里一并清理，应改为 `ADD R6, R6, #2`。原图保留；虽然函数结尾会通过 `R5` 重置 `R6`，原写法仍会额外占用临时栈空间，不能把它解释为出栈。

```
1 // This function returns the position of 'item' if it exists
2 // between list[start] and list[end], or -1 if it does not.
3 int BinarySearch(int item, int list[], int start, int end)
4 {
5 int middle = (end + start) / 2;
6
7 // Did we not find what we are looking for?
8 if (end < start)
9 return -1;
10
11 // Did we find the item?
12 else if (list[middle] == item)
13 return middle;
14
15 // Should we search the first half of the array?
16 else if (item < list[middle])
17 return BinarySearch(item, list, start, middle - 1);
18
19 // Or should we search the second half of the array?
20 else
21 return BinarySearch(item, list, middle + 1, end);
22 }
```

图 17.16 递归实现二分查找的 C 函数

图 17.17 展示对十一元素数组的查找过程，目标为 109。最初传入完整数组的范围；数组参数本身是首元素地址。后续每次调用都保留同一个数组，只改变查找区间。命中某个中间元素就返回其下标；否则继续缩小，直到区间为空。<sup>6</sup>

---

<sup>6</sup>译注：原书把“只剩一个元素”也直接称为基本情况，但代码仍需比较该元素，未命中时再进入空区间。开篇允许任取位置的描述是分治搜索思路；后文的标准二分查找采用中点。原图中  $(end + start) / 2$  对极大下标可能溢出，实际程序可在有效非负下标范围内用  $start + (end - start) / 2$ ，并先检查空区间。

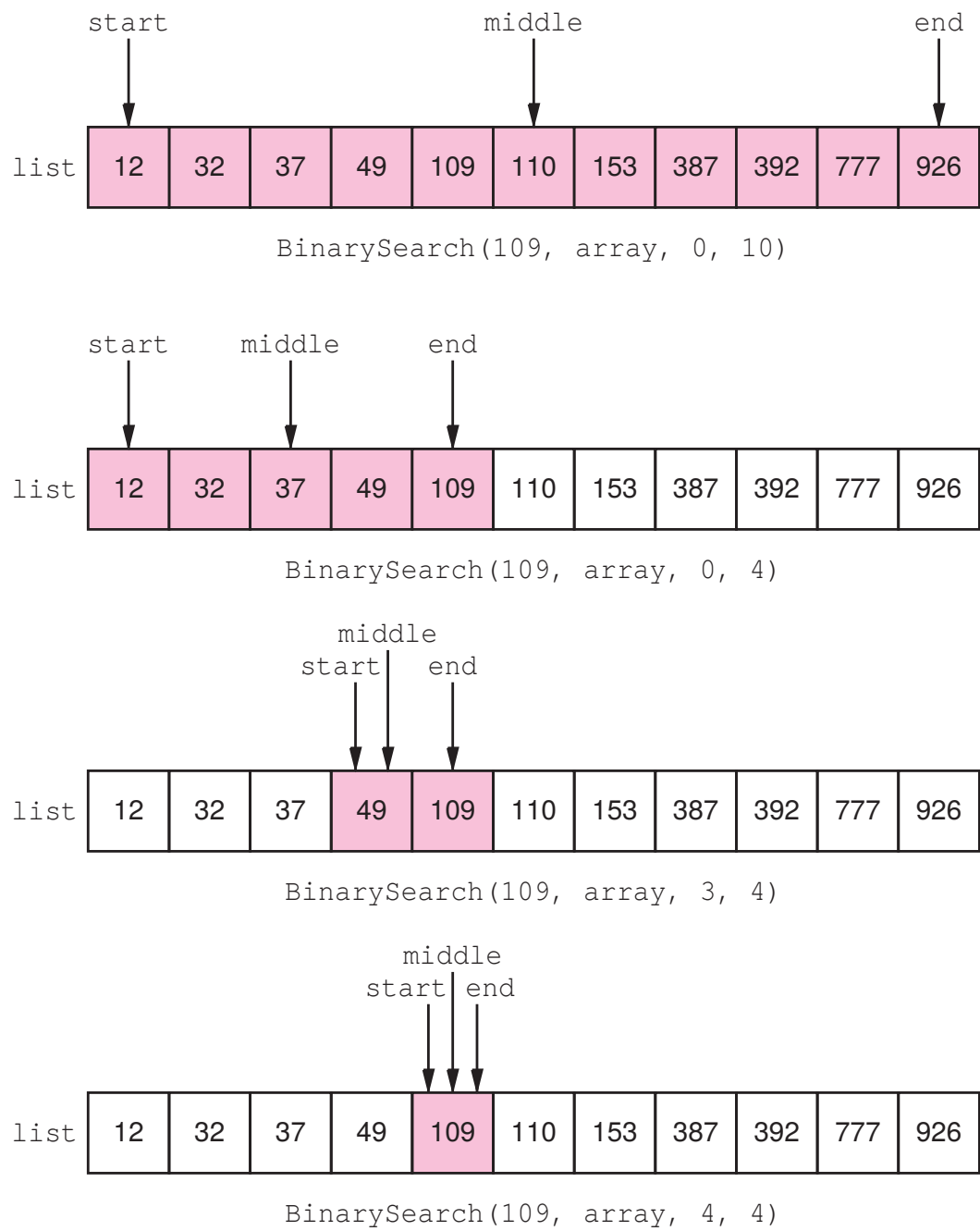


图 17.17 在十一元素数组上执行 BinarySearch, 查找 109

另一种更直接的方法是顺序检查 list[0]、list[1]、list[2] 等，直到找到目标或遍历结束。然而，对于足够大的有序数组，二分查找所需比较次数少得多。后续课程会分析：其运行时间与  $\log n$  成正比，而顺序查找与  $n$  成正比，其中  $n$  是数组大小。这里的对数以 2 为底；例如， $n = 1000000$  时， $\log_2 n \approx 19.93$ 。

这并不是说递归二分查找比迭代二分查找更快。递归形式通常仍有额外调用开销。优势来自二分算法本身：无论用递归还是迭代，它对大型有序数组的查找都远比逐项扫描高效。

17.7 走出迷宫

递归常用于求解游戏和谜题。它能系统地搜索可能的选择，因此适合那些可以通过逐一尝试候选解来求解的问题。例如，数独和迷宫路径搜索，都适合采用递归搜索。

第 8 章已经用 LC-3 汇编编写过一个简单的迷宫求解程序，现在用 C 再看一次。解迷宫有很多方法，递归可能是最容易表达的一种。先决定怎样表示迷宫：与第 8 章一样，使用二维数组，每个元素表示一个位置的状态。字符 X 表示墙，空格字符 ' ' 表示可通行空间，E 表示出口。函数接收迷宫数组和起点坐标，如果存在通路，就寻找一条到出口的路径。

图 17.18 是一个 4×4 迷宫。maze[3][0] 为 'E'，是出口；maze[0][2] 为 'X'，是墙。若起点为 maze[1][2]，则存在到出口的路径。

|   |   |   |   |
|---|---|---|---|
|   | X | X |   |
| X |   |   | X |
|   |   | X |   |
| E | X |   |   |

图 17.18 用二维字符数组表示迷宫的例子

怎样计算这条路径？若从当前位置能到出口，第一步必然走向上、下、左、右相邻位置之一。因此，可以把其中一个邻居作为新位置，递归地问：从这里能否到出口？在新位置，同样的问题再次出现。

搜索过程中可能碰到墙 X 或出口 E，它们都是基本情况。还需要用 V 标记已访问的位置，以便跳过已经检查过的空间，防止来回绕圈。遇到已访问位置也直接返回。

图 17.19 的 ExitMaze 接收一个 MAZE\_HEIGHT 行、MAZE\_WIDTH 列的字符数组，以及当前位置的两个下标 xpos、ypos。函数首先检查坐标是否越界，再检查是否到达出口，或是否碰到墙、已访问位置。否则，先将当前位置标记为已访问，然后递归检查四个邻居。

```
1 #define MAZE_HEIGHT 4
2 #define MAZE_WIDTH 4
3
4 int ExitMaze(char maze[MAZE_HEIGHT][MAZE_WIDTH], int xpos, int ypos)
5 {
6 if (xpos < 0 || xpos >= MAZE_HEIGHT || ypos < 0 || ypos >= MAZE_WIDTH)
7 return 0;
8
9 if (maze[xpos][ypos] == 'E') // Found the Exit!
10 return 1;
11
12 if (maze[xpos][ypos] != ' ') // Space is not empty (possibly X or V)
13 return 0;
14
15 maze[xpos][ypos]='V'; // Mark this space as visited
16
17 // Go Down
18 if (ExitMaze(maze, xpos + 1, ypos)) {
19 maze[xpos][ypos]='P';
20 return 1;
21 }
22
23 // Go Right
24 if (ExitMaze(maze, xpos, ypos + 1)) {
25 maze[xpos][ypos]='P';
26 return 1;
27 }
28
29 // Go Up
30 if (ExitMaze(maze, xpos - 1, ypos)) {
31 maze[xpos][ypos]='P';
32 return 1;
33 }
34
35 // Go Left
36 if (ExitMaze(maze, xpos, ypos - 1)) {
37 maze[xpos][ypos]='P';
38 return 1;
39 }
40
41 // No path to Exit
42 return 0;
43 }
```

图 17.19 递归寻找迷宫出口路径的 C 函数

这是一种**深度优先搜索**（depth-first search）。可以用一幅调用图表示递归过程，每个节点对应一次 ExitMaze 调用，每次最多产生四个子调用。图 17.20 展示了从 maze[1][2] 出发的过程。程序依次尝试向下、向右、向上、向左，而不是同时进行。最初三次调用都立即失败返回，第四次向左进入空白位置，于是继续展开搜索。

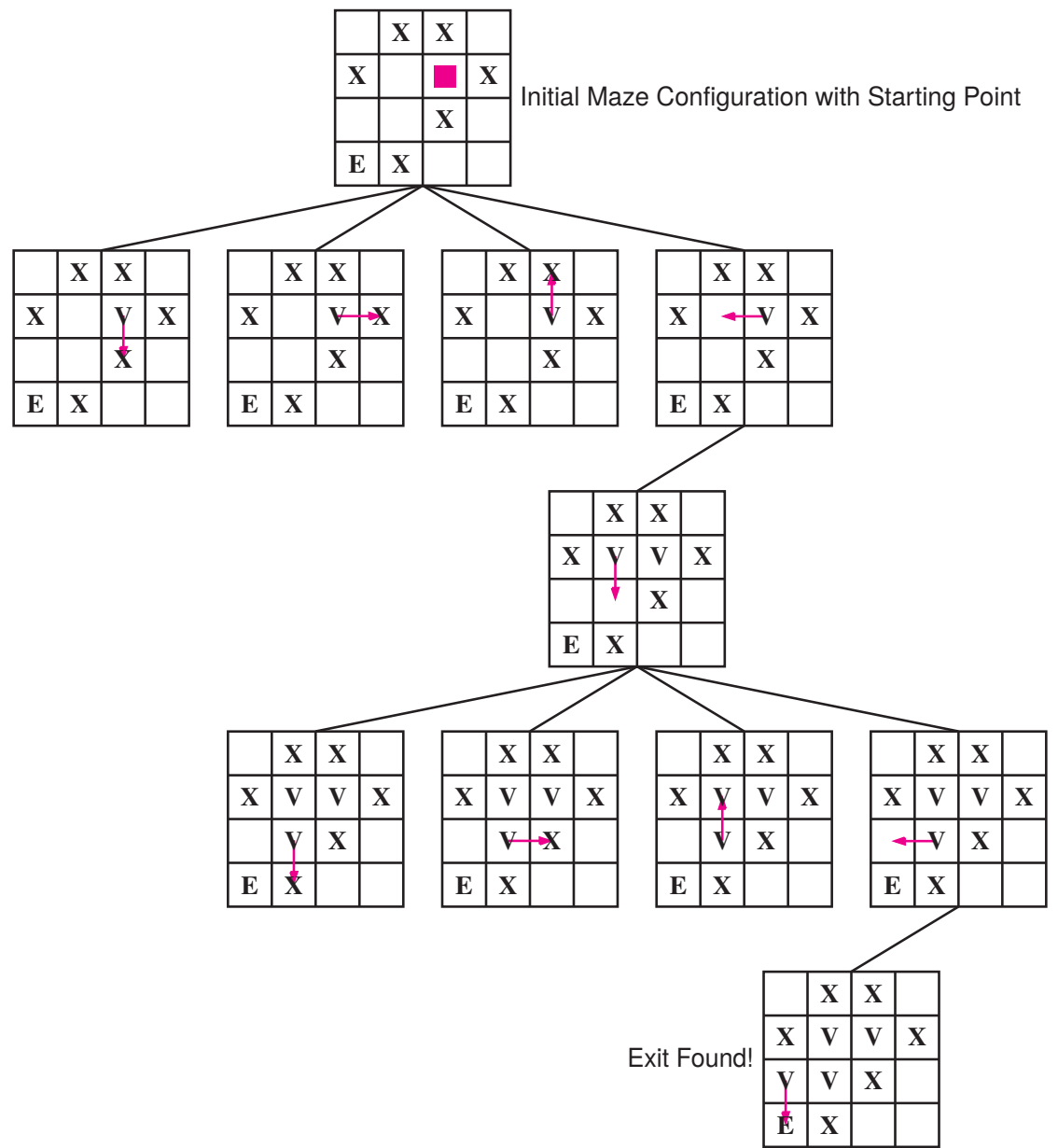


图 17.20 迷宫求解过程中产生的递归调用

当某个子调用找到出口、返回 1 时，当前层把自己的位置改标为 P，再向上返回成功。因此，递归回退后，找到的路径会被标记出来；成功时不会再尝试本层剩余的方向。<sup>7</sup>

17.8 小结

第 8 章在 LC-3 汇编中介绍过递归，本章又在 C 中考察了它。使用递归，既要理解哪些问题适合它，也要理解相关开销。

<sup>7</sup>译注：P 的回溯标记在原程序中明确存在，这里补充其作用。解析文件将第 12 行的空格字符误读为空格字符常量 ' '，原 PDF 实际为 ' '，使用原图恢复。



递归函数通过调用自身求解规模更小的子问题。例如，用较小参数上的同一函数来定义  $f(n)$ ，Fibonacci 数就满足

$$\text{Fibonacci}(n) = \text{Fibonacci}(n - 1) + \text{Fibonacci}(n - 2).$$

为了终止，必须有能够到达的基本情况。对合适的问题，递归能显著简化编程：汉诺塔的递归解法简洁，迭代表达却困难得多。后续课程中，树、图等通过指针组织的数据结构，也常常适合用递归处理。

在底层，递归调用与普通调用以相同方式处理。运行时栈为每次调用分配独立栈帧，使各层调用的数据互不覆盖。这正是递归能够自然运行的基础。

## 习题

**17.1** 回答下列关于本章递归函数的问题，调用次数包含最初的调用。

- a. 调用 `RunningSum(10)` 时，共调用多少次 `RunningSum`?
- b. 调用 `RunningSum(n)` 时，调用次数的一般表达式是什么?
- c. 用 `MoveDisk(4, 1, 3, 2)` 求解四个盘的汉诺塔时，共调用多少次 `MoveDisk`?
- d. 求解  $n$  个盘的汉诺塔时，调用次数的一般表达式是什么?
- e. 调用 `Fibonacci(10)` 时，共调用多少次 `Fibonacci`?
- f. 计算第  $n$  个 `Fibonacci` 数时，调用次数的一般表达式是什么?
- g. `ExitMaze` 的第 12 行会遇到字符 'P' 吗?

**17.2** 对同一个递归函数的各次递归调用，返回地址是否总是相同? 为什么?

**17.3** 求解一个迷宫时，`ExitMaze` 最多会被调用多少次?

**17.4** 下面的函数对 `count(20)` 给出什么结果?

```
int count(int arg)
{
 if (arg < 1)
 return 0;
 else if (arg % 2)
 return(1 + count(arg - 2));
 else
 return(1 + count(arg - 1));
}
```

**17.5 考察下面的 C 程序。**

```
#include <stdio.h>
int Power(int a, int b);
int main(void)
{
 int x, y, z;

 printf("Input two numbers: ");
 scanf("%d %d", &x, &y);
 if ((x > 0) && (y > 0))
 z = Power(x, y);
 else
 z = 0;
 printf("The result is %d.\n", z);
}

int Power(int a, int b)
{
 if (a < b)
 return 0;
 else
 return 1 + Power(a/b, b);
}
```

- a. 分别给出下列输入对应的完整输出：
  - (1) 4 9
  - (2) 27 5
  - (3) -1 3
- b. Power 函数计算什么？
- c. 图 17.21 是调用 Power 后的运行时栈快照，展示了两个栈帧，部分内容已经填入。假定快照取自 Power 中某条 return 语句即将执行之前。填写图中所有问号；对于地址，请用指向相应存储位置的箭头表示。

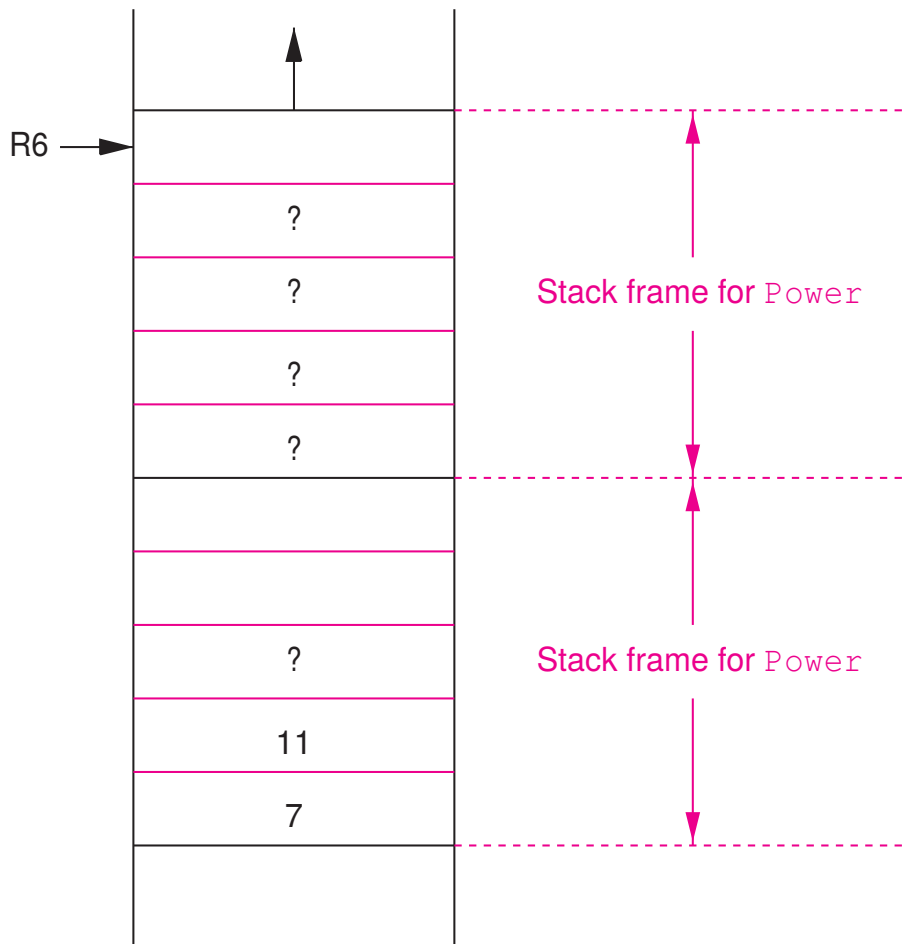


图 17.21 运行时栈中的两个栈帧

**17.6** 考察下面的 C 函数。

```
int Sigma(int k)
{
 int l;
 l = k - 1;
 if (k == 0)
 return 0;
 else
 return (k + Sigma(l));
}
```

- 将它改写为非递归函数。假定调用 `Sigma()` 时，实参总是非负数。
- 运行时栈恰好可以使用连续的 1 KB 存储空间，地址和整数均为 16 位。在栈空间耗尽之前，可以进行多少次递归函数调用？假定无需为临时值分配存储空间。

**17.7** 下面的 C 程序在 LC-3 上编译执行。运行时栈从存储位置 `xFEFF` 开始，向 `xC000` 方向增长，原题称栈最多可占用 16 KB 存储空间。

```
int SevenUp(int x)
{
 if (x == 1)
 return 7;
 else
 return (7 + SevenUp(x - 1));
}

int main(void)
{
 int a;

 printf("Input a number \n");
 scanf("%d", &a);
 a = SevenUp(a);
 printf("%d is 7 times the number\n", a);
}
```

- 能使程序正确运行的最大输入值是多少？解释你的答案。
- 如果运行时栈只能占用 4 KB 存储空间，能使程序正确运行的最大输入值是多少？

译注：原代码的递归调用误写为 `sevenUp`，这里统一为 `SevenUp`。LC-3 按字寻址，题给地址范围与“16 KB”不一致，作答时需明确采用的容量约束，并考虑整数表示范围。独立编译此程序应包含 `stdio.h`。

**17.8** 编写计算第  $n$  个 Fibonacci 数的迭代函数。对多个  $n$  值，绘图比较迭代版本与递归版本的运行时间。为什么递归版本明显较慢？

**17.9** 图 17.16 的二分查找程序搜索升序数组。改写代码，使它能搜索降序数组。

**17.10** 下面是一个非常著名的算法，其递归形式比迭代形式容易表达得多。它最初由古希腊数学家 Euclid 提出。对下列调用，给出函数最终返回的值。

```
int ea(int x, int y)
{
 if (y == 0)
 return x;
 else
 return ea(y, x % y);
}
```

- a. `ea(12, 15)`
- b. `ea(6, 10)`
- c. `ea(110, 24)`
- d. 此函数计算什么？思考如何构造计算相同结果的迭代版本。

**17.11** 编写一个不使用递归函数、与下列 C 程序等价的程序。

```
int M(void);

int main(void)
{
 printf("%d", M());
}

int M(void)
{
 int num, x;
 printf("Type a number: ");
 scanf("%d", &num);
 if (num <= 0)
 return 0;
 else {
 x = M();
 if (num > x)
 return num;
 else
 return x;
 }
}
```

译注：原书将有返回值的 *M* 写成 *void M()*，这里改为 *int M(void)* 并补上前置声明。独立编译应包含 *stdio.h*。

**17.12** 考察下面的递归函数。

```
int func (int arg)
{
 if (arg % 2 != 0)
 return func(arg - 1);
 if (arg <= 0)
 return 1;
 return func(arg / 2) + 1;
}
```

- 是否存在某个 *arg* 值会造成无限递归？如果存在，是什么值？
- 假定 *func* 属于一个程序，其 *main* 函数如下。执行此程序时，共调用多少次 *func*？

```
int main(void)
{
 printf("The value is %d\n", func(10));
}
```

- 程序输出什么值？

**17.13** 幻方是一个  $n \times n$  网格，每个格子包含 1 到  $n^2$  之间的一个整数，各格整数互不相同；每行、每列以及两条对角线上的数之和都相等。给定一个以二维整数数组表示、已经部分填好的幻方，如果存在完整解，就求出它。使用递归技术编写求解程序。

**17.14** 下面的 C 程序输出什么？

```
void Magic(int in);
int Even(int n);

int main(void)
{
 Magic(10);
}

void Magic(int in)
{
 if (in == 0)
 return;
 if (Even(in))
 printf("%i\n", in);
 Magic(in - 1);
 if (!Even(in))
 printf("%i\n", in);
 return;
}

int Even(int n)
{
 if (n % 2) return 1; else return 0;
}
```

译注：原书的 `if (n % 2) return 1 else 0;` 缺少分号和 `return`，这里补齐语法。保留其实际判断逻辑；函数名不能代替对代码的分析。独立编译应包含 `stdio.h`。



# 第十八章 C 中的 I/O

## 18.1 引言

无论是向屏幕、文件，还是通过网络向另一台计算机发送数据，有用的程序总要以某种形式输出结果。大多数程序还需要某种形式的输入。与许多现代程序设计语言一样，C 的核心语言并不直接提供输入输出操作，而是通过一组标准库函数扩展基本语言，完成输入输出 (I/O)。这些函数的行为由 ANSI C 标准明确规定。

本章讨论 C 标准库中支持简单 I/O 的几个函数。putchar 和 printf 向输出设备写入数据，getchar 和 scanf 从输入设备读取数据。更通用的 fprintf 和 fscanf 用于文件 I/O，例如读写磁盘上的文件。本书后半部分已经大量使用 printf 和 scanf，现在将详细考察它们的工作方式。其间，我们还会介绍**可变实参列表**，说明 LC-3 运行时栈上的参数传递机制怎样处理实参数数量不固定的调用。

## 18.2 C 标准库

C 标准库是对 C 程序设计语言的重要扩展。它提供输入输出、字符串处理、数学函数、文件访问，以及各种系统实用功能。这些功能并非只为某个程序服务，而是能被许多程序共同使用。标准库汇集了有用的基础函数，让它们成为构建复杂软件的组成部分。这种通过库提供可复用组件的方式，也是许多当代语言的特点。例如，C++、Java 和 Python 都有类似的标准库，提供基础函数、类型及其他设施。

附录 D.9 简要介绍了一些有用的 C 库函数。库的具体实现通常由底层设备和系统的开发者提供，例如 Android 智能手机系统的开发者，并针对所支持的系统进行优化。

要使用 C 标准库中的函数，必须包含适当的头文件，即 .h 文件。库函数按功能分组，每组有相应的头文件。例如，sin、tan 等数学函数使用 math.h，标准 I/O 函数使用 stdio.h。这些头文件包含函数声明，以及与 I/O 有关的预处理器宏等内容。库头文件并不包含这些库函数的源代码实现。

既然头文件没有实现代码，printf 的机器码又怎样进入应用程序？在形成可执行映像时，程序调用的库函数会被链接进来。包含库函数的目标文件存放在系统中，链接器找到它们，将各个函数的二进制代码与程序连接成一个可执行程序。这称为**静态链接** (static linking)。

库也可以**动态链接** (dynamic linking) 到应用程序中。对于动态链接库 (dynamically linked library, DLL)，库的机器码不直接合入可执行映像，而是在装入或运行时完成所需的链接。与静态链接相比，这可以减小可执行文件的大小，使多个程序共享相同的库代码、减少内存需求，还可以独立于应用程序更新库。

## 18.3 逐字符 I/O

先看 C 库提供的两个最简单的 I/O 函数。getchar 和 putchar 每次处理一个字符。本章沿用 ASCII 字符编码来解释输入输出，类似于 LC-3 的 IN 和 OUT TRAP 服务程序。<sup>1</sup>

### 18.3.1 I/O 流

从概念上说，基于字符的输入输出通过**流**（stream）进行。用户在键盘上输入的一串 ASCII 字符就是输入流的例子。每输入一个字符，就将它加入流的末尾；程序读取键盘输入时，则从流的开头取走字符。同样，程序打印的字符也依次加入输出流的末尾。

流这一抽象进一步分离了生产者与消费者。两者的工作速度往往不同，这种分离就很有用，参见第 8 章。例如，程序要输出数据时，可以先把字符加入输出流，而不必每次都等输出设备显示完前一个字符。C++ 等许多常用语言也提供类似的 I/O 流抽象。<sup>2</sup>

在 C 中，标准输入流名为 stdin，在通常的交互环境中默认连接键盘；标准输出流名为 stdout，默认连接显示设备。getchar 和 putchar 分别操作这两个流。

### 18.3.2 putchar

可以把 putchar 看作与 LC-3 OUT TRAP 服务对应的高级语言函数。它把实参所表示的字符写入 stdout。这里不进行“把一个数转换成十进制数字串”这样的格式转换，而是将该值作为字符输出。下面三个调用都会显示小写字母 h：

```
char c = 'h';
:
putchar(c);
putchar('h');
putchar(104);
```

调用 putchar 与调用普通函数一样，只是函数实现在标准库中。其声明位于 stdio.h，代码在链接阶段与程序关联。<sup>3</sup>

<sup>1</sup>译注：这里是对本书所用系统的说明；C 语言本身并不要求所有实现都采用 ASCII。

<sup>2</sup>译注：流和缓冲能缓解速度差异，但缓冲容量有限，并不意味着任何读写都不会等待。

<sup>3</sup>译注：原书“没有类型转换”的说法指不做数字文本格式化；严格说，putchar 接收 int，写出前将值转换为 unsigned char。片段中的冒号表示省略的程序内容，不是独立 C 语句。

### 18.3.3 getchar

`getchar` 与 LC-3 IN TRAP 服务的共同点是获取输入字符。它从 `stdin` 取出下一个字符并返回其编码值。默认情况下, `stdin` 中的字符来自键盘。下面的片段把返回值赋给变量 `c`:

```
char c;
```

```
c = getchar();
```

译注: `getchar` 的返回类型是 `int`, 以便同时表示所有字符值和 `EOF`。原片段用 `char` 接收, 只适合这里假定成功读到字符的演示; 需要检查文件结束或错误时, 应先用 `int` 接收。与 LC-3 的 `IN` 不同, `C` 不保证 `getchar` 自行打印提示或回显输入。

### 18.3.4 缓冲 I/O

运行图 18.1 的程序, 会发现一个看似奇怪的现象。程序先提示输入第一个字符, 然后等待。只输入一个字符, 例如 `z`, 却看不到第二个提示, 好像 `getchar` 没有接收到按键。直到按下 **Enter** 或 **Return**, 程序才继续。`getchar` 明明一次只读一个字符, 为什么会这样?

```
1 #include <stdio.h>
2
3 int main(void)
4 {
5 char inChar1;
6 char inChar2;
7
8 printf("Input character 1:\n");
9 inChar1 = getchar();
10
11 printf("Input character 2:\n");
12 inChar2 = getchar();
13
14 printf("Character 1 is %c\n", inChar1);
15 printf("Character 2 is %c\n", inChar2);
16 }
```

图 18.1 缓冲输入的例子

原因是键盘输入经过了**缓冲** (buffering)。在常见的交互环境中, 底层系统软件先捕获按键, 将字符保存在一个小数组或队列形式的缓冲区中, 参见第 8.4 节; 按下 **Enter** 后, 整行输入才可供程序读取。**Enter** 本身也在流中留下换行字符。因此, 若输

入 A 后按 Enter, inChar1 得到 A 的 ASCII 编码 65, inChar2 随后得到换行字符的编码 10。

键盘输入采用缓冲有一个实际好处: 用户可以在确认之前修正输入。若打错字符, 可以用 Backspace 或 Delete 编辑, 再按 Enter 提交。<sup>4</sup>

输出流也可以缓冲。图 18.2 使用 sleep 暂停程序, 时间约为实参指定的秒数, 这里是 5 秒。该函数需要包含 unistd.h。

```
1 #include <stdio.h>
2 #include <unistd.h>
3
4 int main()
5 {
6 putchar('a');
7
8 sleep(5);
9
10 putchar('b');
11 putchar('\n');
12 }
```

图 18.2 缓冲输出的例子

在 stdout 采用行缓冲的典型终端环境下, 字符 a 不一定在五秒等待之前就显示出来。程序先把它放入输出缓冲区; 等待结束后写入 b 和换行, 才将缓冲内容送出。这称为**刷新**(flush) 输出。若在第 6 行后立即加入 putchar('\n');, 便可观察到不同的显示时机。

译注: sleep 和 unistd.h 属于 POSIX 环境, 不能作为 ISO C 标准库接口使用。换行是否立即刷新取决于流的缓冲方式; 输出重定向到普通文件时, 常见的是全缓冲。需要主动刷新输出时, 可调用 fflush(stdout)。

虽然缓冲使表面行为复杂了一些, 最终传送字符仍要依赖设备的底层 I/O 功能。本书用 LC-3 的 IN、OUT 等服务解释这一层联系; 输入到达时的中断服务及输出服务, 都与第 8 章讨论的机制有关。<sup>5</sup>

<sup>4</sup>译注: 这里描述的是典型终端的按行输入行为, 通常还涉及操作系统的终端处理。它不是所有设备、终端模式或 C 输入流都必须具有的行为。

<sup>5</sup>译注: C 库缓冲和操作系统、设备缓冲可能位于不同层次。原书将流缓冲概括为中断服务或 OUT 服务完成, 是教学上的简化, 不能据此认为所有 C 库都按 LC-3 服务程序实现。

## 18.4 格式化 I/O

`putchar` 和 `getchar` 足以完成简单 I/O，但如果要把整数、浮点数与字符形式相互转换，逐字符处理就很繁琐。`printf` 和 `scanf` 提供更强的**格式化 I/O** (formatted I/O)，方便地完成这些转换。

### 18.4.1 `printf`

`printf` 向输出流写入格式化文本，允许我们在文字中嵌入程序运行时计算出的值。它负责为输出完成必要的转换。例如，下面的调用打印整数变量 `x` 的值，将其转换成可写入输出流的一串数字字符：

```
int x;

printf("The value is %d\n", x);
```

译注：此处 `x` 的声明与调用只是片段，调用前应给 `x` 赋值。

一般来说，`printf` 按第一个实参指定的格式输出。这个实参是**格式字符串** (format string)，即包含待显示文本的字符串，通过指针传给函数。格式字符串中可以嵌入零个或多个**转换说明** (conversion specification)，用来控制后续实参的输出形式。

转换说明以 `%` 开头。例如，`%d` 把整数输出为十进制数字，`%x` 把无符号整数输出为十六进制数字；原书还用 `%b` 演示二进制输出。这些输出仍然是文本字符。`%c` 输出单个字符，`%s` 输出从给定地址开始连续存放的字符串，此时相应实参应指向字符序列。`%f` 将浮点数输出为浮点格式。若要打印百分号本身，使用 `%%`。更多转换说明见附录 D。<sup>6</sup>

第 11 章已介绍过字符串中的特殊字符：`\n` 表示换行，`\t` 表示制表符。源代码中的这些转义序列以反斜杠开头，可以出现在字符串中的任意位置。若要表示反斜杠本身，写成 `\\`。附录 D 的表 D.1 列出了这些特殊字符。下面是几种格式说明的例子：

```
int a = 102;
int b = 65;
char c = 'z';
char banner[10] = "Hola!";
double pi = 3.14159;

printf("The variable 'a' decimal : %d\n", a);
printf("The variable 'a' hex : %x\n", a);
printf("The variable 'a' binary : %b\n", a);
printf("'a' plus 'b' as character : %c\n", a + b);
printf("Char %c.\t String %s\n Float %f\n", c, banner, pi);
```

译注：保留原示例数值与字符串。`a + b` 等于 167，超出七位 ASCII 范围；对应的 `%c` 不保证显示某个通用可打印字符。

可以把 `printf` 的处理过程理解为逐个检查格式字符串中的字符：普通字符直

---

<sup>6</sup>译注：`%b` 的可用性取决于所用 C 标准和库实现，不能假定本书提到的传统 ANSI C 环境都支持它。这里保留原示例；`%x` 及支持它的 `%b` 要求相应的无符号整数类型，实际编程应按格式传入正确类型。

接写入输出流；遇到转换说明，则按照说明取出下一个实参，转换后输出。流可能带缓冲，所以写入不一定立刻显示。

例如，整数值的位模式若是 0000000001101000，以 %d 输出就得到数字文本 104；按 %c 输出则得到字符 h。转换说明告诉 printf 应当怎样取得并解释相应的实参，但它并不对实际类型进行运行时验证。因此，程序员必须使说明与所传类型相符。<sup>7</sup>

再看下面的调用：

```
printf("The value of nothing is %d\n");
```

其中 %d 没有对应实参。按照本书的栈传参模型，printf 会假定调用者已经放好了相应的值，仍去取得该位置的内容，可能打印出一个垃圾值。<sup>8</sup>

### 18.4.2 scanf

scanf 从输入流读取格式化文本。它与 printf 一样，第一个实参是格式字符串，其后可以有数量不固定的其他实参，处理过程由格式控制。不同之处在于，scanf 的后续实参必须是适当类型的指针。正如第 16 章所述，函数必须能找到原对象的存储位置，才能把输入的新值写进去。

格式字符串既可以包含转换说明，也可以包含需要匹配的普通文字。scanf 的转换说明与 printf 相似，列表见附录 D。它们描述输入中各字段的格式。例如，%d 表示先跳过前导空白，再读取表示十进制整数的字符序列，将其转换为整数，写入对应指针所指的位置。这里的空白包括空格、水平制表符、换行、回车、垂直制表符和换页符。

对于普通标量变量，我们用取地址运算符 & 传入地址。C 仍然按值传递实参，只是所传的值是地址，因此被调用函数可以间接修改原变量。格式中的普通字符则要求与输入匹配。下面的例子将说明这些规则：

```
char name[100];
int month, day, year;
double gpa;

printf("Enter : lastname birthdate grade_point_average\n");
scanf("%s %d/%d/%d %lf", name, &month, &day, &year, &gpa);

printf("\n");
printf("Name : %s\n", name);
printf("Birthday : %d/%d/%d\n", month, day, year);
printf("GPA : %f\n", gpa);
```

译注：原例的 %s 没有限定长度；name 有 100 个元素，实际使用宜写成 %99s，

<sup>7</sup>译注：原书在这里有两处不准确的说明。一是将 \n 说成由 printf 在运行时解析；实际上，字符串字面量中的转义序列在翻译源程序时就已处理，函数收到的是换行字符。二是说程序员可以任意用 %d、%f 解释同一实参；格式与实参类型不匹配会导致未定义行为，不能把它当作一种通用的位模式重解释方法。

<sup>8</sup>译注：严格说，这是未定义行为，“输出垃圾值”只是可能现象，不能保证一定输出某个值或正常返回。保留这段错误代码，是为了说明实参数量必须与格式相符。

为结尾空字符留出空间，并检查返回值后再使用读入的数据。这里保留原代码以便与后面的分析对应。

第一个说明 `%s` 从第一个非空白字符开始读取，到下一处空白之前为止，也就是读取流中的下一个“词”。这些字符写入从 `name` 所指地址开始的存储空间，并自动追加 `'\0'` 作为字符串结束标记。数组表达式 `name` 在这里转换为首元素地址，无需另加 `&`。

下一个说明 `%d` 读取月份。它跳过前导空白，读取十进制整数所需的字符序列，将字符形式转换为二进制整数，存入 `&month` 指定的位置。此后，格式要求输入中的下一个字符恰好是 `/`。这个字符不是转换说明，不会赋给变量；匹配成功后直接消耗掉，继续处理后面的格式。随后的 `%d/%d` 同样读取日期和年份，把转换后的整数分别写入 `day` 和 `year`。

译注：原书说斜杠两侧可以任意有空白，这不符合这里的格式。`%d` 会跳过它自己的前导空白，但格式中的普通 `/` 不会跳过前面的空白。如果希望允许这种输入，应在格式里明确加入空白，例如 `%s %d / %d / %d %lf`。

最后一个字段 `%lf` 读取 `double`。输入可以包含数字、小数点，以及用 `E` 或 `e` 引入的指数，还可以有尾数或指数的正负号；浮点表示见附录 D.2.4。函数将合法的浮点输入序列转换为双精度浮点值，存入 `gpa`。<sup>9</sup>

处理完成后，`scanf` 返回一个整数，表示成功转换并赋值的输入项数。若本例全部成功，返回值为 5；斜杠和空白的匹配不计入这个数。原例没有使用返回值。一般程序应检查它，以免把未成功输入的变量当成有效数据。<sup>10</sup>

输入下面这一行，得到相应输出：

```
Enter : lastname birthdate grade_point_average
Mudd 02/16/69 3.02
```

```
Name : Mudd
Birthday : 2/16/69
GPA : 3.020000
```

译注：原书将 **GPA** 输出列为 `3.02`，但上面的 `%f` 默认打印小数点后六位，因此这里按代码改为 `3.020000`。

原书随后给出下面的分行输入，并声称因为换行属于空白，所以结果相同：

```
Enter : lastname birthdate grade_point_average
Mudd 02
/
16 / 69 3.02
```

但这个结论需要修正：月份 02 后面紧跟换行，而格式要求紧接一个斜杠，所以按原格式会在此停止，返回 2。只有将格式改成允许斜杠前有空白的形式，例如前述带空格的格式，才会读入相同的五项数据，得到相同结果。

<sup>9</sup>译注：`scanf` 中的 `%f` 对应 `float *`，`%lf` 对应 `double *`；这与 `printf` 的浮点实参规则不同，后者的 `%f` 接收 `double`，传入的 `float` 会先提升为 `double`。

<sup>10</sup>译注：返回值不是简单地数格式串中的所有转换说明。例如，带赋值抑制的项不计入返回值。在第一次转换完成之前发生输入失败时，函数返回 EOF。

如果输入中根本没有斜杠，又会怎样？

```
Enter : lastname birthdate grade_point_average
Mudd 02 16 69 3.02
```

此时 name 和 month 已成功赋值，而月份后面的空格不能匹配格式中的斜杠，因此 scanf 返回 2；其余变量不变。未使用的输入不会被自动丢弃，后续读取从这次停止的位置继续。

若接着执行：

```
a = getchar();
b = getchar();
```

则 a 得到空格字符 ' '，b 得到字符 '1'。这里后者是字符的编码值，而不是整数 1。

### 18.4.3 可变实参列表

与此前大多数函数相比，printf 和 scanf 有一个特点：传给它们的实参数量可以变化。需要打印或读取多少项，决定了调用中要传多少个实参。这称为**可变实参列表**（variable argument list）。在本章的普通转换示例中，每个需要实参的转换说明与格式字符串之后的一个实参相对应。例如：

```
printf("Char %c.\t String %s\n Float %f\n", c, banner, pi);
```

格式中有三个转换说明，后面就有三个实参：%c 对应 c，%s 对应 banner，%f 对应 pi。加上格式字符串，共四个实参。同理，按这种简单格式打印五个值时，就需要六个实参。<sup>11</sup>

回顾第 14 章的 LC-3 调用约定：按照调用表达式中从右到左的顺序压入实参。因此，最左边的格式字符串指针总是在调用前最后压栈。执行 JSR 后，printf 或 scanf 刚取得控制权时，就能直接从栈顶取得它。分析格式之后，函数便知道后续应当怎样读取其他实参。如果反过来从左向右压栈，找到格式字符串就麻烦得多。

图 18.3 对比两种压栈顺序。图 (a) 从右向左压栈，无论后面有多少个其他实参，格式字符串指针相对入口处栈指针的偏移始终为 0。图 (b) 从左向右压栈，这个偏移就取决于其他实参占用的空间。想一想，哪种方式更容易编写对应的 LC-3 代码？

<sup>11</sup>译注：这不是任意格式字符串都适用的计数规则。例如，%% 不消耗实参，printf 的星号宽度或精度还会消耗额外实参；scanf 的赋值抑制则不需要目标指针。



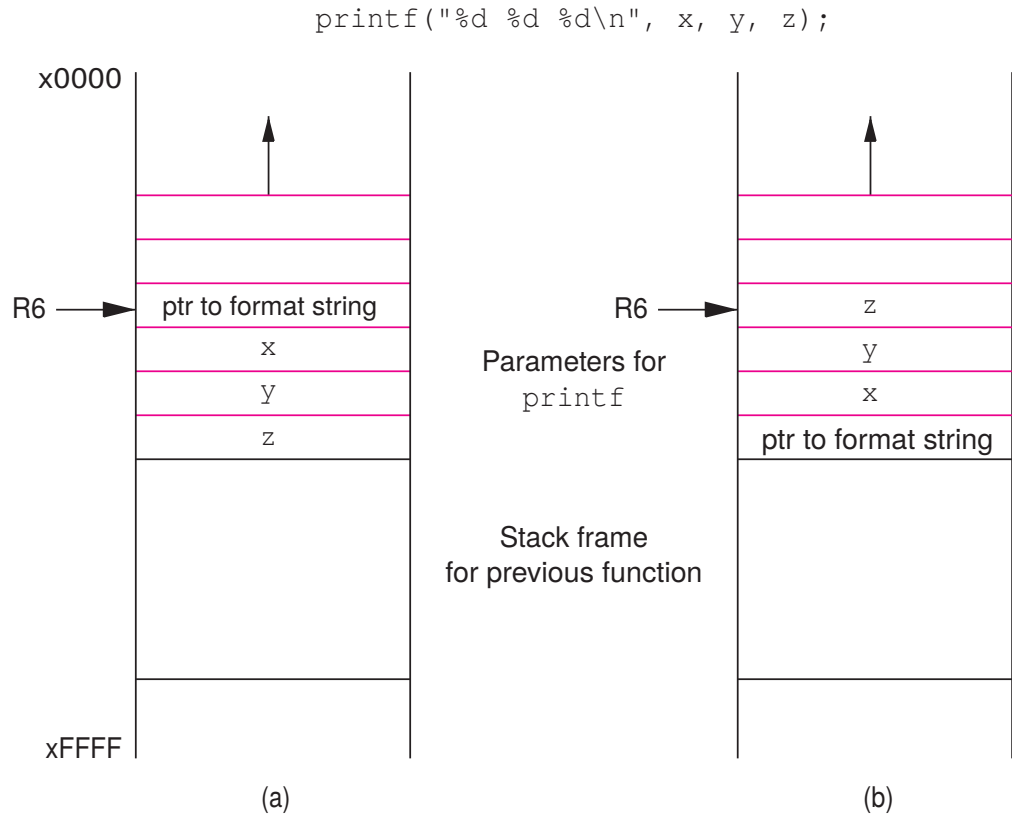


图 18.3 子图 (a) 按从右到左的顺序压入 `printf` 的实参；子图 (b) 按从左到右的顺序压入。

格式字符串本身与程序中的其他字符串字面量一样，存放在用于常量或字面量的存储区域；栈上保存的是指向它的指针。这里讨论的是本书的 LC-3 调用约定，并不是 C 对所有机器都规定的实参布局。

### 18.5 文件 I/O

假设要处理一个大数据集，例如 Apple 股票过去 20 年每天的收盘价。让用户逐项用键盘输入，显然很不方便。我们更希望程序从文件读取数据，必要时再将结果写入另一个文件。如前所述，C 的 I/O 以流为基础；通过流，可以用统一的方式访问文件和设备。

`printf` 和 `scanf` 实际上是更通用的 I/O 函数的特例。它们分别操作标准输出流 `stdout` 和标准输入流 `stdin`；在本书采用的交互环境中，这两个流通常连接显示设备和键盘。

`printf` 的通用版本叫作 `fprintf`，`scanf` 的通用版本叫作 `fscanf`。两者的格式化规则与对应函数相同，主要区别是允许指定要操作的流。例如，可以让 `fprintf` 把输出写到文件系统中某个指定的文件。下面来看具体步骤。

首先，为每个要操作的文件声明一个**文件指针**（file pointer）。文件通常存放在设

备的**文件系统** (file system) 中, 后者管理文档、图片、可执行程序、视频、文本及其他数据的存储与访问。下面声明了一个名为 `infile` 的文件指针:

```
FILE *infile;
```

这表示 `infile` 是指向 `FILE` 类型对象的指针。`FILE` 定义在 `stdio.h` 中, 目前无需了解其内部细节。

声明指针后, 还要把流与文件系统文件关联起来。C 库函数 `fopen` 完成这个工作。它需要两个实参: 要打开的文件名, 以及希望使用的操作模式。例如:

```
FILE *infile;
```

```
infile = fopen("aapl_stock_prices", "r");
```

第一个字符串 `aapl_stock_prices` 是文件名, 第二个字符串 `"r"` 是打开模式。常用模式包括:

- `"r"`: 读取文件。
- `"w"`: 写入文件; 成功打开时, 原有内容会被清空。
- `"a"`: 追加; 保留已有内容, 将新数据写到末尾。
- `"r+"`: 读取和写入。

两个实参都是字符串, 所以示例用双引号包围。本例以读取模式打开 `aapl_stock_prices`。

如果成功, `fopen` 返回与所打开流关联的文件指针; 若失败, 例如文件不存在且模式要求它存在, 则返回空指针 `NULL`。因此, 调用后应检查是否成功:

```
FILE *infile;
```

```
infile = fopen("ibm_stock_prices", "r");
```

```
if (infile == NULL)
 printf("fopen unsuccessful!\n");
```

译注: 此片段的文件名 `ibm_stock_prices` 沿用原书, 其他示例使用 `aapl_stock_prices`。原 *PDF* 的判断为 `==`, 解析文件误识别为赋值运算符 `=`, 这里已恢复。文件指针指向管理流的 `FILE` 对象, 不是文件内容在内存中的起始地址。

文件打开后, 便可用 `fscanf` 和 `fprintf` 读写, 类似于用 `scanf` 和 `printf` 操作标准输入输出。它们的第一个实参是文件指针, 指明操作哪个流; 后面才是格式字符串及其他实参。图 18.4 给出一个例子。

```
1 #include <stdio.h>
2 #define LIMIT10000
3
4 int main()
5 {
6
7 FILE *infile;
8 FILE *outfile;
9 double prices[LIMIT];
10 char answer[10];
11 int i = 0;
12
13 infile = fopen("aapl_stock_prices", "r");
14 outfile = fopen("buy_hold_or_sell", "w");
15
16 if (infile != NULL && outfile != NULL) {
17 // Read the input data
18 while ((fscanf(infile, "%lf", &prices[i]) != EOF) && i < LIMIT)
19 i++;
20
21 printf("%d prices read from the data file", i);
22
23 // Process the data...
24 :
25 :
26 answer = ...
27
28 // Write the output
29 fprintf(outfile, "%s", answer);
30 }
31 else
32 printf("fopen unsuccessful!\n");
33 }
```

图 18.4 执行文件 I/O 的程序示例

这个程序从文本文件 `aapl_stock_prices` 读取股票价格,并向 `buy_hold_or_sell` 写入结果。输入价格以浮点数字文本表示,各项用空白分隔。程序的设计目标是最多处理 10000 项,随后处理读入的数据,生成字符串 `answer` 并写入输出文件。

原例用 `fscanf` 的返回值判断是否到达文件结束。这里需要区分: `EOF` 是由 `stdio.h` 定义的负整数宏,用于表示特定的输入失败情形,并不是从文件中读出的一个“结束字符”。对于图中的单项转换,成功读入一个价格时,返回值为 1;若碰到无法匹配浮点格式的输入,则可能返回 0;在本次转换前就遇到文件结束或读取错误,则返回 `EOF`。

译注：原图第 18 行先执行读取，再检查  $i < LIMIT$ ，因此当  $i$  已等于  $LIMIT$  时，仍可能向数组末尾之外写入。仅检查不等于  $EOF$ ，还会把返回 0 的匹配失败误当成读到一项。应先检查下标，再要求成功读入一项，例如：

```
while (i < LIMIT && fscanf(infile, "%lf", &prices[i]) == 1)
 i++;
```

原图保留了省略的数据处理步骤，`answer = ...` 只是示意；数组不能直接这样赋值，实际代码应把结果字符写入数组。原图也没有列出关闭操作，完整程序应对已成功打开的流调用 `fclose`，包括只有一个文件打开成功时的清理。

读完输入并完成处理后，`fprintf(outfile, "%s", answer)` 将结果字符串写入输出文件。`printf` 等价于将 `stdout` 作为流实参调用 `fprintf`；`scanf` 则等价于将 `stdin` 作为流实参调用 `fscanf`。

## 18.6 小结

本章考察了 C 的输入输出设施。与许多语言一样，C 的核心语法没有直接的 I/O 操作，而是由标准库提供函数，最终再依赖底层机器和系统的 I/O 功能。本书通过 LC-3 的 IN、OUT 等服务，将高级语言中的输入输出与底层字符传输联系起来。应掌握以下几点：

- **通过流进行输入输出。**流提供了有用的抽象：生产者向流中加入数据，消费者从流中读取数据。配合缓冲，双方可以在一定范围内按各自的速度工作。例如，向显示设备输出的程序可以先写入输出流，不必逐字符等待显示完成。
- **四个基本 I/O 函数。**本章较详细地讨论了 `putchar`、`getchar`、`printf` 和 `scanf`。后两个函数使用可变实参列表；本书 LC-3 调用约定采用的压栈顺序，使这类调用容易处理。
- **文件 I/O。**C 通过流统一处理文件与标准输入输出。`printf` 和 `scanf` 是操作标准输出流、标准输入流的特例；`fprintf` 和 `fscanf` 允许显式指定文件指针。使用 `fopen`，可将流与文件系统中的文件关联起来。

## 习题

**18.1** 为下列各项任务编写一个 I/O 函数调用。每项任务都可以通过一次调用完成。

- a. 依次打印一个整数、一个字符串和一个浮点数。
- b. 按 (XXX)-XXX-XXXX 格式打印电话号码。程序内部用三个整数保存该号码。
- c. 按 XXX-XX-XXXX 格式打印学生学号。程序内部用三个字符串保存该号码。
- d. 读取 XXX-XX-XXXX 格式的学生学号，内部保存为三个整数。
- e. 读取一行输入，其中包含姓、名、中间名首字母、年龄和性别。姓名字段之间用逗号分隔；中间名首字母和性别保存为字符，年龄保存为整数。

**18.2** scanf 的返回值表示什么？

**18.3** 为什么对键盘输入流采用缓冲很有用？

**18.4** 当程序试图从输入流读取数据，而流当前为空时，必须发生什么？

**18.5** 为什么下面的代码会打印出一个奇怪的值，例如 1073741824？

```
float x = 192.27163;
printf("The value of x is %d\n", x);
```

译注：题中数值是原书给出的可能现象，不是所有实现都应得到的确定输出。  
保留错误代码供分析。

**18.6** 对于下面的调用，若输入流包含所示内容，input 的值是什么？

```
scanf("%d", &input);
```

输入内容为：

```
This is not the input you are looking for.
```

**18.7** 考察下面的程序。

```
#include <stdio.h>

int main(void)
{
 int x = 0;
 int y = 0;
 char label[10];

 scanf("%d %d", &x, &y);
 scanf("%s", label);

 printf("%d %d %s\n", x, y, label);
}
```

- a. 输入流为 46 29 BlueMoon 时, 打印什么?
- b. 输入流为 46 BlueMoon 时, 打印什么?
- c. 输入流为 111 999 888 时, 打印什么?

**18.8** 编写一个程序, 读入 C 源文件, 删除其中全部空白, 再写入名为 condensed\_program 的文件。**18.9** 编写一个程序, 读取文本文件并进行下列统计。

- a. 文件中的字符串个数。这里, 字符串从一个非空白字符开始, 到空白字符处结束。
- b. 文件中的单词个数。单词从一个字母开始, 例如 a-z 或 A-Z, 到非字母字符处结束。
- c. 文件中不同单词的个数。单词的定义与 (b) 相同; 不同单词的集合不包含重复项。
- d. 按出现次数从多到少列出各单词的频数。也就是说, 分析文件, 统计每个单词出现的次数, 再按从最高到最低的顺序显示。

# 第十九章 C 中的动态数据结构

## 19.1 引言

C 的核心只支持少数几种基本数据类型，例如整数、浮点数、字符和布尔值，并提供加法 +、乘法 \* 等操作这些类型的运算符。我们可以很容易地声明基本类型的变量，以及由它们构成的数组和指向它们的指针。

然而，实际编程所处理的事物，往往不能仅用一个整数、浮点数、字符或布尔值来描述，甚至仅用这些类型的数组也不方便。例如，为飞机机翼建模，或为自动驾驶车辆开发路径规划器，都需要更复杂的类型，才能把现实世界或虚拟世界中的对象映射到计算设备的数字世界。

接下来的两章主要讨论 C 和 C++ 如何支持创建和组织复杂数据类型。基本类型虽然有限，却是构建几乎任意复杂类型的基础。创建这些类型、组织并连接相应的数据，以及定义操作数据的方法，是程序开发的重要部分。

本章先介绍 C 的**结构体**。结构体把多个基本类型的成员组合成一种新类型，便于表示由多种属性共同描述的对象。例如，企业数据库中的员工，可以用包含姓名字符串、职位字符串、部门编号和员工编号的结构体表示。定义这种结构体后，就能像声明一个整数变量那样，声明一个代表员工的完整内存对象。

随后介绍动态内存分配，让对象的生存期不必局限于某次函数调用的运行时栈帧。结合结构体和动态分配，我们可以构造在执行过程中增长或缩小、跨越多次函数调用持续存在的数据结构。这种能力非常有用，是许多软件的基础。

## 19.2 结构体

直接来看一个例子。假设要跟踪某个地理位置周围的飞行器，例如美国 Illinois 州 Urbana 市上空的飞机。每架飞机有多项相关属性：用于识别的注册编号可用字符串表示；高度、经度、纬度和航向可用整数表示；空速可用浮点类型，例如 double 表示。具体应用还可能需要其他信息。本例使用下列各项：

```
char ID[7]; // Max 6 characters
int altitude; // in meters
int longitude; // in tenths of degrees
int latitude; // in tenths of degrees
int heading; // in tenths of degrees
double airSpeed; // in kilometers/hour
```

要跟踪多架飞机，可以为每项属性分别建立一个数组。不过，不难想象，同时维护这些数组、保证各项数据始终对应同一架飞机，会使程序变得繁琐。

C 提供结构体，将 int、char、double 等类型，以及它们的数组和指针，组

合成新的类型。结构体变量的声明与基本类型变量类似，但必须先定义结构体。飞机跟踪程序可以使用：

```
struct flightType {
 char ID[7]; // Max 6 characters
 int altitude; // in meters
 int longitude; // in tenths of degrees
 int latitude; // in tenths of degrees
 int heading; // in tenths of degrees
 double airSpeed; // in kilometers/hour
};
```

这个定义建立了一种包含六个**成员** (member) 的类型。该类型的对象占据连续存储空间，足以容纳全部成员。下面声明一个名为 plane 的变量：

```
struct flightType plane;
```

成员通过“变量名、点号、成员名”的形式访问。例如：

```
struct flightType plane;

plane.airSpeed = 800.00;
plane.altitude = 10000;
```

其中点号，选择结构体的一个成员。

如果 plane 是普通局部变量，本书的编译模型会像其他局部变量一样将它分配在运行时栈上。假定每种基本类型都占一个 LC-3 存储单元，这个对象需要 12 个单元：字符数组占 7 个，四个整数占 4 个，double 占 1 个。<sup>1</sup> 图 19.1 展示包含下列局部声明的函数被调用后，栈帧中的一部分：

```
int x;
struct flightType plane;
int y;
```

---

<sup>1</sup>译注：原书这里误写成“五个整数”，与定义和总数不符，已改为四个。本章将各基本类型都视为占一个单元，是用于说明布局的简化假设，不能当作真实 C 实现中 double 的大小。结构体在实际系统上还可能含有对齐填充。



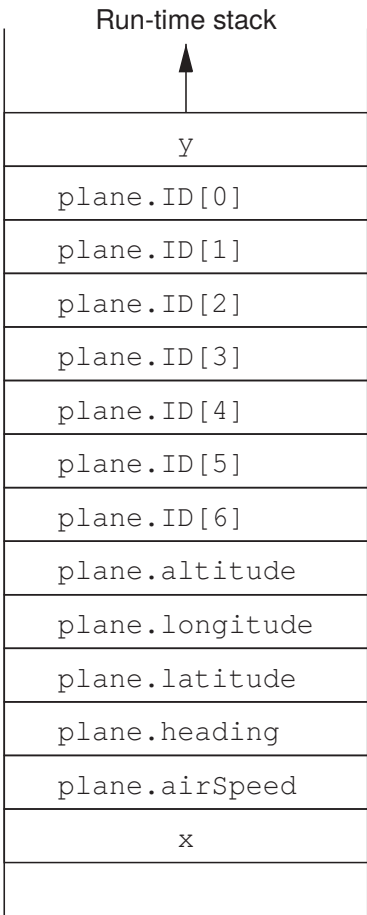


图 19.1 运行时栈中结构体变量的分配

一般形式如下：

```
struct tag {
 type1 member1;
 type2 member2;
 ...
 typeN memberN;
} identifiers;
```

tag 是**结构体标记**，供后续代码引用这种结构体，例如声明变量。成员列表描述结构体的组织方式，语法上是一组成员声明。成员可以是已经定义的其他结构体类型。右花括号与分号之间还可以可选地列出 identifiers，同时声明这种结构体的变量。<sup>2</sup>

<sup>2</sup>译注：原书示意中的最后一个成员声明缺少分号，这里补齐。省略号和 typeN 等是语法示意，不是完整可编译代码。结构体不能直接包含自身类型的对象，但可以包含指向自身类型的指针。

### 19.2.1 typedef

结构体让程序员定义自己的**聚合类型**，typedef 则允许为类型取名字。一般形式是：

```
typedef type name;
```

尽管名称中有“定义”的含义，typedef 实际上为既有类型建立一个同义名称。type 可以是基本类型，也可以是结构体等聚合类型，name 是新的类型别名。例如：

```
typedef int Color;
```

Color 现在是 int 的别名，可以用它声明：

```
Color pixels[200][100];
```

这个数组可以表示一幅  $200 \times 100$  像素的图像。

typedef 对结构体尤其有用。我们可以为前面的飞机结构体取一个简短而有意义的名字：

```
struct flightType {
 char ID[7]; // Max 6 characters
 int altitude; // in meters
 int longitude; // in tenths of degrees
 int latitude; // in tenths of degrees
 int heading; // in tenths of degrees
 double airSpeed; // in kilometers/hour
};
typedef struct flightType Flight;
```

此后就可以写：

```
Flight plane;
```

它等价于先前的 `struct flightType plane;`。typedef 没有增加新的运行时功能，却能使大量使用自定义类型的代码更清晰。恰当的类型名称，可以传达变量名本身未能说明的性质。

### 19.2.2 C 中结构体的实现

结构体建立在 C 的基本机制之上。结构体变量与 int、char、float 变量一样是普通变量，只是内部由更多部分组成。成员仍可通过相应类型的已有运算符操作。下面访问 Flight 对象的 altitude 成员：

```
int x;
Flight plane;
int y;
```

```
plane.altitude = 0;
```

编译器知道结构体布局，因此可以用适当的偏移定位成员。它在符号表中记录每个局部变量相对帧指针 R5 的位置，还记录聚合类型中各字段相对对象起始位置的偏

移。图 19.1 给出了本例的栈布局: `plane` 是声明中的第二个变量, 而 `altitude` 是其中第二个成员。

在本书的布局假设下, 赋值 `plane.altitude = 0`; 对应如下 LC-3 代码:

```
AND R1, R1, #0 ; zero out R1
ADD R0, R5, #-12 ; R0 contains base address of plane
STR R1, R0, #7 ; plane.altitude = 0;
```

先得到 `plane` 的起始地址, 再加上 7 个单元的成员偏移, 便定位到 `altitude`。

### 19.3 结构体数组

继续完善飞机跟踪程序。它的核心数据结构, 应当是 `flightType` 对象的某种集合, 方便我们访问某一时刻 Urbana 上空的全部飞机, 并在飞机起飞、降落、进入或离开空域时添加或删除记录。

一种直接的方法是数组。数组能够把基本类型的对象连续组织起来, 同样也能组织结构体。不过, 我们必须确定数组大小, 无论是编译时固定大小, 还是使用第 16.3.8 节的变长数组。假设空域内不会超过 100 架飞机, 可以声明:

```
Flight aircraft[100];
```

但如果这个假设不成立呢? 暂时先按 100 架处理。此声明分配连续的 100 个结构体, 而不是 100 个整数, 每个结构体包含先前定义的六个成员。`aircraft[12]` 表示第 13 个对象; `aircraft[0]` 表示第一架飞机。访问该飞机的航向可写成 `aircraft[0].heading`。

下面计算全部 100 架飞机的平均空速:

```
double sum = 0;
double averageAirSpeed;

for (int i = 0; i < 100; i++)
 sum = sum + aircraft[i].airSpeed;

averageAirSpeed = sum / 100;
```

译注: 原书循环写成 `plane[i].airSpeed`, 但数组名是 `aircraft`, 这里统一为后者。如果实际上不足 100 架, 就需要另行记录数组中有多少个有效对象, 后面再讨论。

还可以声明指向结构体的指针, 并让它指向数组中的某个对象:

```
Flight *aircraftPtr;

aircraftPtr = &aircraft[34];
```

要访问所指结构体的经度, 可以先解引用, 再使用点号:

```
(*aircraftPtr).longitude
```

`aircraftPtr` 指向一个 `Flight`, `*aircraftPtr` 得到该结构体对象, 随后 `.longitude` 选择成员。这种操作非常常用, 因此 C 提供更直接的箭头运算符:

```
aircraftPtr->longitude
```

`->` 将通过指针访问结构体成员的两步操作合在一起; 它的形状也形象地表示“指向”。

现在增加一项功能: 为每架飞机找出距离它最近的另一架飞机。为此, 需要检查飞机的位置和高度, 并比较距离。图 19.2 中, `NearestNeighbor` 对飞机成对调用 `AirDistance`, 试图为每架飞机找出最短距离。`AirDistance` 并未完整实现; 其意图是利用两架飞机的高度、经度和纬度计算距离。

```

1 #include <stdio.h>
2 #define TOTAL_FLIGHTS 100
3
4 // Structure definition
5 struct flightType {
6 char ID[7]; // Max 6 characters
7 int altitude; // in meters
8 int longitude; // in tenths of degrees
9 int latitude; // in tenths of degrees
10 int heading; // in tenths of degrees
11 double airSpeed; // in kilometers/hour
12 };
13 typedef struct flightType Flight;
14
15 double AirDistance(Flight *aircraftA, Flight *aircraftB)
16 {
17 // This function calculates Euclidean distance
18 // in meters in 3-space of two aircraft given
19 // their altitude, longitude and latitude.
20 return distance;
21 }
22
23 void NearestNeighbor(Flight aircraft[TOTAL_AIRCRAFT])
24 {
25 double minD;
26 Flight *closest;
27
28 for (int i = 0; i < TOTAL_AIRCRAFT; i++) {
29 closestAircraft = NULL; // Initialize
30 minD = MAX_DISTANCE; // Initialize
31 for (int j = 0; j < TOTAL_AIRCRAFT; j++) {
32 if (i != j) {
33 if (AirDistance(&aircraft[i], &aircraft[j]) < minD)
34 closest = &aircraft[j];
35 }
36 }
37 printf("The closest aircraft to %s is %s.\n",
38 aircraft[i].flightNum, closest->flightNum);
39 }
40 }

```

图 19.2 使用 Flight 结构体数组跟踪空中飞机

译注：原图不能直接作为正确的完整程序使用。*TOTAL\_FLIGHTS* 与 *TOTAL\_AIRCRAFT* 不一致，*closestAircraft* 应统一为已声明的 *closest*，*flightNum* 应为成员 *ID*；*MAX\_DISTANCE* 和距离计算尚未定义。找到更小距离时，还必须同步更新 *minD*，否则得到的不一定是最近飞机。访问 *closest* 前也应确认它非空。这里保留原图，说明其算法意图和缺陷。

注意，*NearestNeighbor* 传给 *AirDistance* 的是两个指针，而不是两个

完整结构体。C 可以按值传递结构体，但在本书模型中，传指针减少了压栈数据量：这里只需压入两个地址，而不是为两个 `Flight` 对象复制共 24 个存储单元。

## 19.4 动态内存分配

把固定大小的 `Flight` 数组作为核心数据结构，有几个问题。首先，必须把容量写进代码。飞机较少时虽然能运行，却分配了多余空间，还要区分哪些元素有效、哪些未使用；飞机数量超过容量时，程序就不能正确工作。这种刚性假设使代码容易失效。如果容量能随实际需要调整，会更合适。

其次，空中飞机的集合不断变化：飞机会起飞、降落、进入或离开空域，核心数据结构必须经常添加和删除记录。为了保持数组中的有效元素连续，在中间插入或删除一项，通常要移动其后的元素。随着操作次数增多，这种数据搬移可能拖慢程序，增加设备的工作负担和耗电。

下一节介绍的链接式数据结构将直接应对这些问题。不过，在讨论链接之前，需要先理解**动态内存分配**（dynamic memory allocation）。

在本书的实现模型中，C 程序的数据对象分配在三个区域：运行时栈、全局数据区和堆。普通函数局部变量默认在执行过程中分配到栈上；全局变量位于全局数据区；动态分配的对象位于堆上，由程序逻辑决定何时申请和释放。<sup>3</sup>

从较高层次看，**内存分配器**（memory allocator）管理一片称为堆的存储区域。程序在运行时调用标准库函数 `malloc`，请求某个字节数的连续内存。分配器找到足够的可用空间，将它标记为已分配，并返回起始地址。例如，需要存放 1000 架飞机的数据时，可以计算所需字节数并提出请求；成功后就通过返回的指针访问这片空间。

堆上已分配的内存块会持续存在，直到程序显式调用与分配器配套的 `free` 释放它。释放后，这片空间可供后续请求重新使用。图 19.3 与图 12.7 相同，展示包括堆在内的 LC-3 存储布局。

---

<sup>3</sup>译注：这是本书对存储布局的概括。局部 `static` 对象等具有不同生存期；全局变量能否从其他源文件访问，还受声明和链接属性约束。C 本身不要求所有实现都采用同一物理布局。

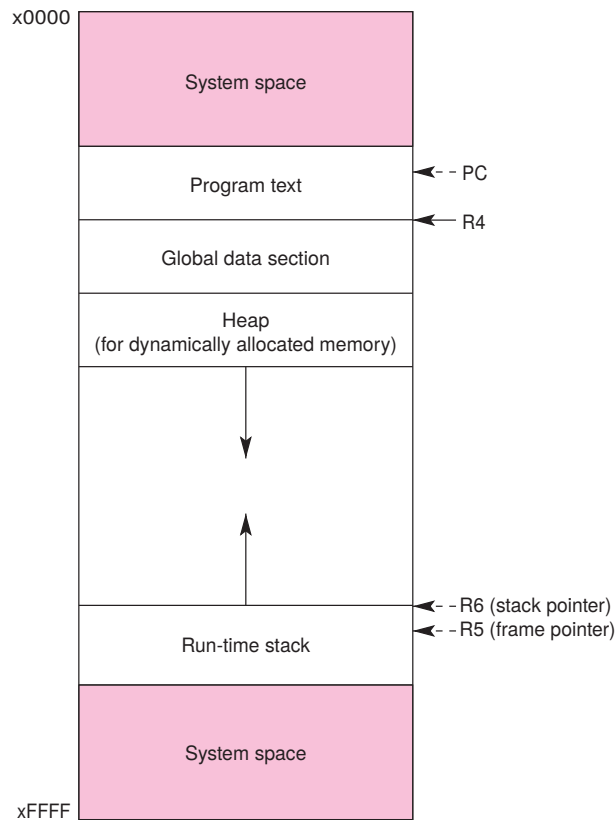


图 19.3 包含堆区域的 LC-3 存储器映射

在图示模型中，堆与栈相向增长。栈的占用与当前调用深度及各栈帧大小有关，堆的占用则取决于尚未释放的动态分配。<sup>4</sup>

普通栈上对象在相应代码块退出后结束生存期；堆对象则由程序员控制释放时机。这种额外控制，使我们能够构造跨越多个函数、在程序较长执行阶段中持续存在的数据结构。

19.4.1 动态大小数组

malloc 和 free 的接口都很简单：各接收一个实参。它们内部的实现可能很复杂，留待后续课程讨论。先用动态大小数组说明用法：

```
int numAircraft;
Flight *planes;

printf("Total number of aircraft?");
scanf("%d", &numAircraft);
planes = malloc(24 * numAircraft);
```

malloc 按实参给出的字节数申请连续空间，成功则返回指向该区域的指针。这里申请 24 \* numAircraft 字节，numAircraft 是要容纳的飞机数量。

为什么是 24？前面采用的教学模型中，Flight 有 7 个字符、4 个整数和 1 个

<sup>4</sup>译注：free 让内存可被分配器重用，并不保证每次调用都使堆边界缩回，或立即把空间交还操作系统。

double, 每个占一个两字节 LC-3 单元, 总共 12 个单元, 即 24 个八位字节。但不同系统上各基本类型的大小不同, 不能把 24 写死。原书举例说, 在一种 PC 上, 成员大小可能合计为 31 字节。<sup>5</sup>

C 的 sizeof 运算符可取得对象或类型在当前实现中的大小, 使程序不必依赖某个固定数字。例如, sizeof(Flight) 给出一个 Flight 所占的 C 字节数, 包含需要的填充。利用它, 可以编写在不同系统之间更容易移植的代码。<sup>6</sup>

请求无法满足时, malloc 返回空指针 NULL, 所以应检查是否成功。它的返回类型是通用对象指针 void \*, 使同一函数可以为整数、字符、结构体等不同对象分配空间。原书在示例中使用显式类型转换, 把返回值转换为 Flight \*。一般的类型转换语法为:

```
var = (newType) expression;
```

于是, 结合大小计算、转换和失败检查, 得到:

```
int numAircraft;
Flight *planes;

printf("Total number of aircraft?");
scanf("%d", &numAircraft);
planes = (Flight *) malloc(sizeof(Flight) * numAircraft);
if (planes == NULL) {
 printf("Error in malloc...\n");
 :
 :
}
```

译注: 在 C 中, void \* 可以隐式转换为对象指针, 因此这里的强制转换可写但不是必需; 原书关于“不转换会降低可移植性”的说法不适用于 C。使用分配函数应包含 stdlib.h。实际程序还需检查输入成功、数量有效, 以及数量乘以对象大小是否溢出; 冒号表示尚未写出的错误处理, 原片段缺少的右花括号已补齐。

因为分配到的空间连续, 既可以用指针记法, 也可以用数组下标记法访问。只要分配成功且 numAircraft >= 30, planes[29] 就表示第 30 架飞机。这正是第 16.3.5 节讨论的下标与指针运算关系。<sup>7</sup>

不再需要这片空间时, 把最初分配得到的指针传给 free, 该内存块便可再次被分配。下一节将给出释放节点的例子。<sup>8</sup>

标准库还有其他分配函数: calloc 分配空间并将所有位初始化为零; realloc 尝试改变既有分配的大小。它们同样需要 stdlib.h。

<sup>5</sup>译注: 31 只是按字符 1、整数 4、double 8 字节相加的结果, 忽略了成员之间和末尾可能存在的对齐填充, 不能据此确定结构体的实际大小。前面的 LC-3 大小也只是本章假设。

<sup>6</sup>译注: sizeof 的单位是 C 的字节, 且 sizeof(char) 恒为 1; 一个字节的位数由实现决定。原书把 LC-3 的物理两字节单元与 C 字节混用, 实际分配应依赖 sizeof。sizeof 通常在编译时确定, 但涉及变长数组时可能需要运行时计算。

<sup>7</sup>译注: 原书写成数量必须“大于 30”, 实际等于 30 时下标 29 也有效。

<sup>8</sup>译注: 释放后不能继续通过旧指针访问对象, 也不能重复释放; free(NULL) 则没有作用。



例如，容量不足时，可调用 `AddMoreAircraft()`，通过 `realloc` 扩大飞机数组；也可以用 `ReduceAircraft()` 缩小过大的分配。<sup>9</sup>

这种在堆上分配、按需要调整大小的**动态数组**，与第 16.3.8 节的变长数组相似，却不相同。普通局部变长数组的生存期受所在代码块限制，而且一次创建后，其长度在该次生存期内不变；动态分配的数组可跨函数持续存在，并通过重新分配调整容量。<sup>10</sup>

回顾本节开头：固定数组的问题包括容量僵硬，以及插入删除时的数据移动。动态分配解决了容量问题，但中间增删仍不方便。现在，我们已经准备好学习用指针连接对象的数据结构了。

## 19.5 链表

**链表** (linked list) 与数组都能表示按顺序排列的一组元素。数组中，除最后一个元素外，每个元素的后继紧接着存放在内存中；链表的后继却不必与当前元素相邻。每个元素保存一个指向后继的指针，逻辑上的顺序由这些指针建立。

链表由若干**节点** (node) 组成。一个节点包含一份数据，例如飞机信息，再加上指向下一个节点的指针。从某个起点出发，沿指针即可依次访问后继。为了用 C 表示这种组织，我们为 `Flight` 增加一个成员 `Flight *next`：

```
// Structure definition
typedef struct flightType Flight;
struct flightType {
 char ID[7]; // Max 6 characters
 int altitude; // in meters
 int longitude; // in tenths of degrees
 int latitude; // in tenths of degrees
 int heading; // in tenths of degrees
 double airSpeed; // in kilometers/hour
 Flight *next; // Pointer to next element
};
```

这里把 `typedef` 移到结构体完整定义之前，这样在定义 `next` 时就能使用简短的 `Flight *`，而不必写 `struct flightType *`。此时结构体尚未完整定义，但声明指向它的指针是允许的。

链表也有起点和终点。通过**头指针** (head pointer) 访问第一个节点，称为头节点；最后一个节点是尾节点，它的 `next` 为 `NULL`，表示没有后继。其作用类似于字符串末尾的结束标记，但空指针与字符 `'\0'` 是不同概念。

---

<sup>9</sup>译注：`realloc` 可能移动整个内存块。应先用临时指针接收结果；对非零新大小的请求，失败时原块仍保留，不能因直接覆盖原指针而丢失它。`calloc` 的全零位也不能普遍等同于所有类型的语义零值。

<sup>10</sup>译注：原书说变长数组在整个程序执行期间大小不变，表述过强。每次重新执行相应声明时，可以按新的值创建；已有的那一个数组不会自行改变长度。栈上数组也可以在生存期内把地址传给其他函数访问。

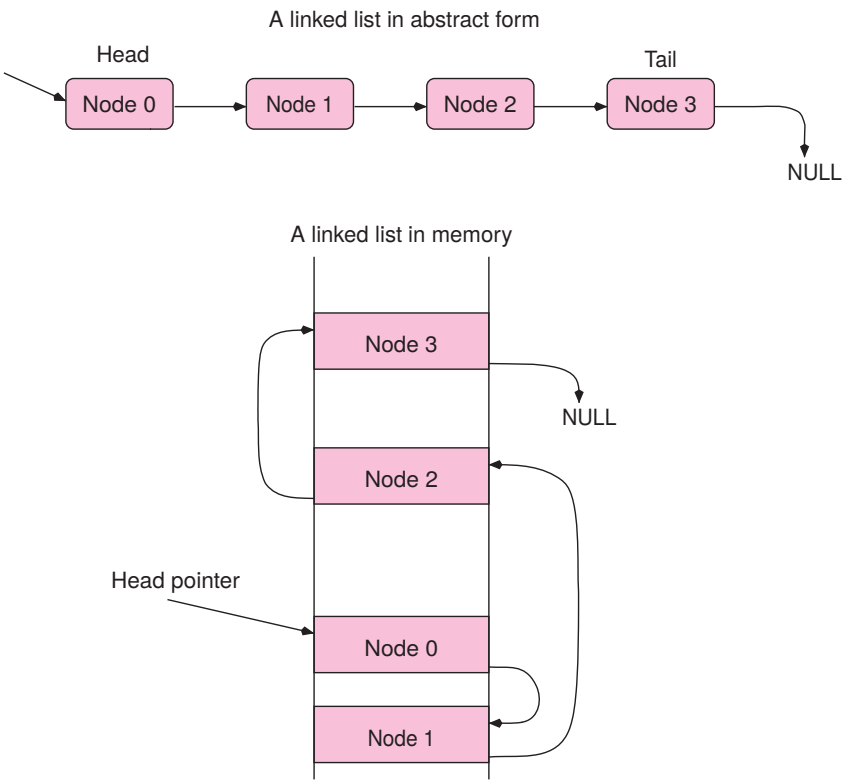


图 19.4 链表的两种表示方式

图 19.4 上方用方框表示节点、箭头表示链接；下方展示节点在内存中可能的实际布局。节点可以分散存放，沿 next 指针仍能恢复它们的线性顺序。给定头指针，就可以从头访问所有节点。

数组与链表有一个根本区别：数组支持按下标直接访问任意元素，例如依次访问第 4、911、45 个位置；普通单链表则要沿链接顺序查找。要从头找到第 29 号节点，通常必须经过第 0、1、2 号等前驱节点。顺序访问确实是链表的一个限制，但它换来了另一项优点。

链表具有动态性：插入或删除节点时，可以让其他节点留在原处，只改变有关链接，无需搬移后面全部数据。动态数组虽然也能调整容量，在保持顺序的中间位置增删一项时，仍可能需要较多的数据移动。

19.5.1 辅助函数

现在改用链表重建飞机跟踪程序。飞机进入空域时，AddFlight 添加记录；离开时，DeleteFlight 删除记录。原来的数组由下面的头指针代替：

```
Flight *airspace = NULL;
```

初始化为 NULL，表示当前链表为空。

先准备这两个操作周围的辅助代码。图 19.5 使用带 next 指针的结构体，并给出创建节点、打印空域和主函数中的调用示例。

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <string.h>
4
5 // Structure definition
6 typedef struct flightType Flight;
7 struct flightType {
8 char ID[7]; // Max 6 characters
9 int altitude; // in meters
10 int longitude; // in tenths of degrees
11 int latitude; // in tenths of degrees
12 int heading; // in tenths of degrees
13 double airSpeed; // in kilometers/hour
14 Flight *next; // Pointer to next element
15 };
16
17 Flight *CreateFlight(char *ID, int altitude, int longitude,
18 int latitude, int heading, double airspeed)
19 {
20 Flight *newFlight;
21
22 newFlight = (Flight *)malloc(sizeof(Flight));
23 strcpy(newFlight->ID, ID);
24 newFlight->altitude = altitude;
25 newFlight->longitude = longitude;
26 newFlight->latitude = latitude;
27 newFlight->heading = heading;
28 newFlight->airSpeed = airspeed;
29 }
30
31 void PrintAirspace(Flight *list)
```

图 19.5 空域跟踪程序的辅助函数，前半部分

```
32 {
33 int count = 1;
34
35 printf("Aircraft in Airspace -----\\n");
36 while (list != NULL) {
37 printf("Aircraft : %d\\n", count);
38 printf("ID : %s\\n", list->ID);
39 printf("Altitude : %d\\n", list->altitude);
40 printf("Longitude: %d\\n", list->longitude);
41 printf("Heading : %d\\n", list->heading);
42 printf("Airspeed : %f\\n", list->airSpeed);
43 printf("-----\\n");
44 count = count + 1;
45 list = list->next;
46 }
47 printf("\\n\\n");
48 }
49
50 int main(void)
51 {
52 Flight *airspace = NULL;
53 Flight *newPlane = NULL;
54
55 newPlane = CreateFlight("ZA123", 1000, 3233,
56 2516, 392, 3493.20);
57 if (AddFlight(newPlane, &airspace) == 0)
58 printf("Successful add of flight %s\\n", newPlane->ID);
59 .
60 .
61 .
62 if (DeleteFlight("ZA123", &airspace) == 0)
63 printf("Successful removal of flight %s\\n", "ZZ");
64 .
65 .
66 .
67 }
```

图 19.5 空域跟踪程序的辅助函数（续）

CreateFlight 接收飞机的各项属性，通过 malloc 创建一个新节点，填写属性，并应当返回新节点的指针。

译注：原图的 CreateFlight 缺少 return newFlight;，且使用分配结果前没有检查失败。完整实现还应初始化 next，并确保 ID 最多六个字符、给结尾空字符留出位置。图中调用添加和删除函数之前，应有对应函数声明。主函数删除的

是 "ZA123", 成功消息却写成 "ZZ", 两者应一致; 原图保留。PrintAirspace 未打印纬度, 是原示例的取舍。

PrintAirspace 以链表头指针为实参, 遍历并打印所有飞机。由于节点数量未知, 使用 while 循环, 直到当前指针为 NULL。它对应于遍历数组的 for 循环, 但推进方式是 `list = list->next;`, 而不是 `i++`。参数 list 是头指针的副本, 改变这个副本不会移动调用者保存的头指针。

main 组织各项操作, 并展示 AddFlight、DeleteFlight 的调用。除 `stdio.h` 外, 还需包含两个标准库头文件: `stdlib.h` 声明 `malloc`、`free` 等分配函数; `string.h` 声明字符串函数, 包括插入和删除时使用的 `strcmp`。

### 19.5.2 向链表添加节点

飞机链表按 ID 排序, 因此每架新飞机都有确定的插入位置。例如, 已有 ID 为 A 和 C 的两个节点, 新节点 B 应插入它们之间, 使顺序成为 A、B、C。

基本操作很简单: 令 B 的 next 指向 C, 再令 A 的 next 指向 B。只需修改两个指针, 就完成了中间插入。比较一下, 有序数组为了腾出中间位置要做多少工作。

先实现这种简单情况, 再逐步补齐其他情况。遍历时寻找第一个 ID 大于 B 的节点: A 小于 B, 继续; C 大于 B, 就在它之前插入。由于 ID 是字符串, 要用标准库函数 `strcmp` 比较:

```
comp = strcmp(stringX, stringY);
```

两个字符串相等时返回 0; 按本书采用的 ASCII 字符顺序, `stringX` 排在 `stringY` 之前时返回负数, 之后时返回正数。这里应比较返回值的符号, 不能假定结果只会是 -1 或 1。

除了定位插入点的 `current`, 还要保留落后一个节点的 `previous`。因为在 C 前插入 B 时, 必须修改它的前驱 A 的 next。`current` 用于查找, `previous` 用于连接新节点。

```
1 int AddFlight(Flight *newPlane, Flight *list)
2 {
3 Flight *previous = NULL;
4 Flight *current = list;
5 int IDcompare;
6
7 while (current != NULL) {
8 IDcompare = strcmp(newPlane->ID, current->ID);
9 // returns 0 if equal
10 // < 0 if newPlane->ID is less than current->ID
11 // > 0 if newPlane->ID is greater than current->ID
12
13 if (IDcompare < 0) {
14 // Add newPlane in between previous and current nodes
15 newPlane->next = current;
16 previous->next = newPlane;
17 return 0;
18 }
19 // Continue traversing thru the list
20 previous = current;
21 current = current->next;
22 }
23 }
```

图 19.6 AddFlight 的初始版本，仅处理向链表中间插入节点的简单情况。

图 19.6 给出基本框架，但还不完整。current 沿链表前进，previous 始终落后一个节点。若 C 不存在、A 不存在，或 A、C 都不存在，会发生什么？此外，若 B 已经存在，又该怎么办？

全部情况可以分成五类：

- **插入中间。** A 和 C 都存在，把 B 接在二者之间，这是刚讨论的情况。
- **空表。** A、C 都不存在，原头指针为 NULL。令它指向 B，并令 B 的 next 为 NULL，链表便只有一个节点。
- **插入尾部。** A 存在而 C 不存在。把 A 的 next 从 NULL 改为 B，并令 B 的 next 为 NULL。
- **插入头部。** C 存在而 A 不存在。让 B 指向 C，再把原来指向 C 的头指针改为指向 B。
- **B 已存在。** 不插入相同 ID 的重复记录，而是返回特殊状态，报告未完成添加。

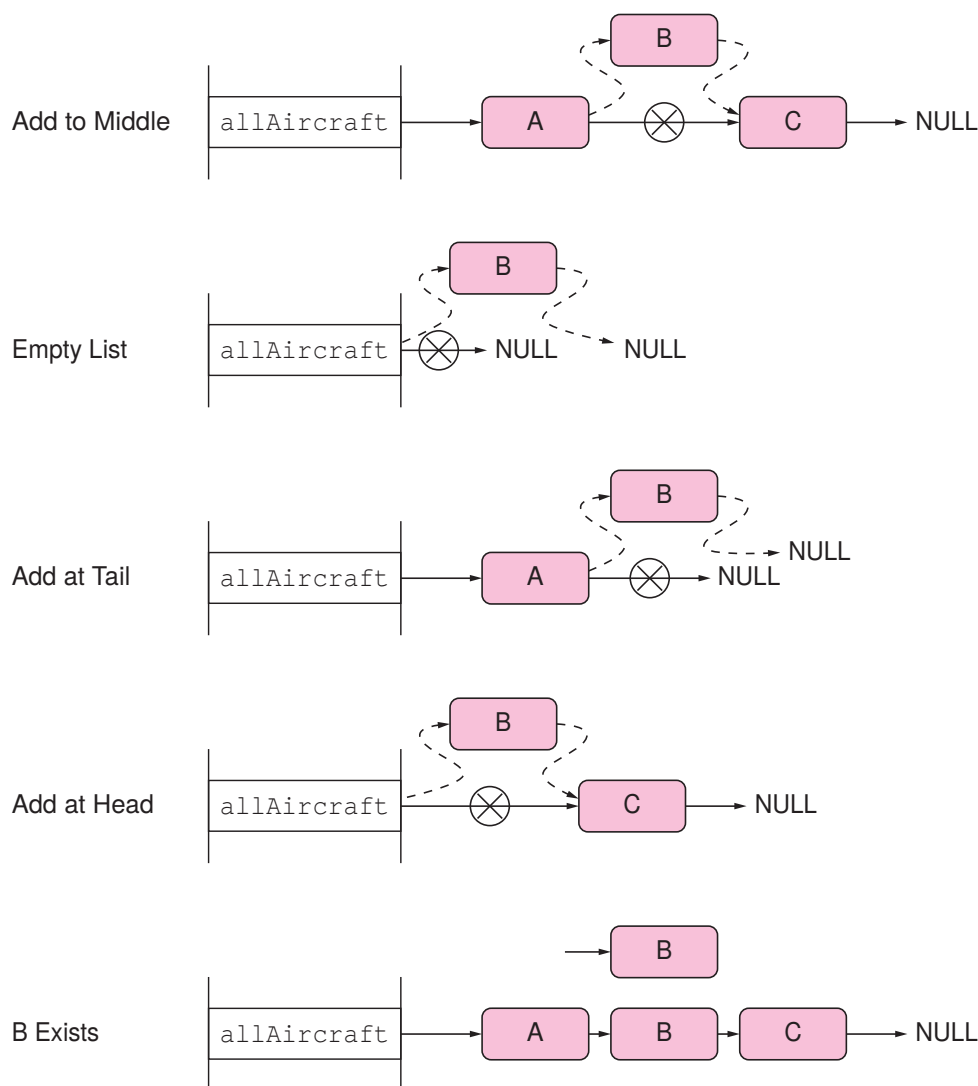


图 19.7 向链表插入节点的五种情况；虚线表示新建立的链接。

译注：图 19.7 中的头指针称为 *allAircraft*，主函数中称为 *airspace*，二者在这里代表同一角色。

必须使最终代码处理所有五种情况。B 已存在时直接返回即可；插入尾部也简单：遍历退出后，当前指针为 NULL，只需让最后的前驱节点指向 B。另两种情况——空表和头部插入——都要求修改调用者的头指针。

为此，需要把头指针变量的地址传给 *AddFlight*，参见第 16.2.3 节。普通指针参数只传递指针值的副本；若要改写原来的 *Flight \** 变量，参数类型就应为指向它的指针：

```
Flight **list;
```

两个星号看起来复杂，但原理与通过 *int \*x* 修改整数变量相同：

```
int *x;
```

```
*x = *x + 1;
```

对头指针解引用，得到的就是调用者保存的那个指针变量。例如，要把头指针推进到下一个节点，可写：

```
*list = (*list)->next;
```

译注：原书写成 `*list = *list->next;`，缺少必要的括号。由于 `->` 的结合优先于一元 `*`，应先用 `(*list)` 取得头节点指针，再访问 `next`。这一操作还要求当前头节点非空。C 仍按值传递参数，这里传递的是头指针变量的地址。

图 19.8 给出覆盖五种情况的版本，处理位置已在代码注释中标出。成功返回 0，重复 ID 返回 -1。



```
1 int AddFlight(Flight *newPlane, Flight **list)
2 {
3 Flight *previous = NULL;
4 Flight *current = *list;
5 int IDcompare;
6
7 while (current != NULL) {
8 IDcompare = strcmp(newPlane->ID, current->ID);
9 // returns 0 if equal
10 // < 0 if newPlane->ID is less than current->ID
11 // > 0 if newPlane->ID is greater than current->ID
12
13 if (IDcompare == 0)
14 return -1; // Exists!
15 }
16 else if (IDcompare < 0) {
17 // Add newPlane in between previous and current nodes
18 newPlane->next = current;
19 if (previous == NULL)
20 *list = newPlane; // Add At Head
21 else
22 previous->next = newPlane; // Add to Middle
23 return 0;
24 }
25 else {
26 // Continue traversing thru the list
27 previous = current;
28 current = current->next;
29 }
30 }
31 newPlane->next = NULL;
32 if (previous == NULL)
33 *list = newPlane; // Empty List
34 else
35 previous->next = newPlane; // Add At Tail
36 return 0;
37 }
```

图 19.8 覆盖各插入情况的 AddFlight 函数

译注：原图第 13 行的 `if (IDcompare == 0)` 后缺少左花括号，与第 15 行的右花括号不配对，应补上。调用者应保证 `list` 和 `newPlane` 有效；若重复 `ID` 导致添加失败，未入链的新节点仍由调用者负责释放，避免泄漏。

### 19.5.3 从链表删除节点

删除函数的框架与添加函数很接近。实参给出待删飞机的 ID，目标可能在中间、头部、尾部，也可能根本不存在。与添加一样，必须覆盖所有情况。

沿链表查找目标。如果找到，让前驱的 next 指向目标的后继，再用 free 释放目标节点。DeleteFlight 成功时返回 0，否则返回 -1。

可以列出与插入相对应的五类情况：删除中间节点、空表、删除尾节点、删除头节点、目标不存在。其中空表与目标不存在都不删除任何节点，返回 -1。删除中间与删除尾部也可用同一条链接赋值处理：尾节点的后继恰好是 NULL。删除头节点则要修改调用者的头指针，因此同样接收 Flight \*\*list。

```
1 int DeleteFlight(char *planeID, Flight **list)
2 {
3 Flight *previous = NULL;
4 Flight *current = *list;
5 int IDcompare;
6
7 while (current != NULL) {
8 IDcompare = strcmp(planeID, current->ID);
9 // returns 0 if equal
10 // < 0 if Plane->ID is less than current->ID
11 // > 0 if Plane->ID is greater than current->ID
12 if (IDcompare == 0) {
13 // Found node to remove!
14 if (previous == NULL)
15 *list = current->next; // Del At Head
16 else
17 previous->next = current->next; // Del from Mid/Tail
18 free(current);
19 return 0;
20 }
21 else if (IDcompare < 0)
22 return -1; // Doesn't Exist
23 else {
24 // Continue traversing thru the list
25 previous = current;
26 current = current->next;
27 }
28 }
29 // Traversed the whole list. Doesn't Exist
30 return -1;
31 }
```

图 19.9 DeleteFlight 函数的完整代码

头节点被删后，新头指针是原头节点的后继；若原表只有一个节点，这个后继就是 NULL，空表情况自然得到处理。节点释放以后，不能再通过指向它的旧指针访问数据。

#### 19.5.4 数组与链表的比较

现在可以用链表替换飞机跟踪程序的数组。不过，选择数据结构总有取舍。链表有优势，也有代价；明智的选择需要同时理解上层操作和底层实现。

先看内存开销。1000 个整数的数组，其元素部分只需 1000 个整数的空间。如果整块数组在堆上分配，分配器通常还要保存少量管理信息，但相对于整个数组往往不大。链表则每个节点都需要后继指针；节点分别动态分配，还会分别产生分配管理开销。若节点本身很小，这些开销可能占总空间的很大比例。

另一方面，数组的预留容量可能大于实际需求。例如，为字符串声明足以容纳最长预期输入的数组，常常比普通输入实际需要的空间多。链表可以按照当前需求逐个分配节点，避免为了固定上限预留全部记录。

再看插入、删除和查找。链表找到操作位置后，只需重新连接少量指针，不必移动其他节点；节点也不必连续存放。数组插入前要有可用容量，为保持顺序通常还要移动元素腾出位置；删除后也常要搬移元素填补空缺。查找方面，普通单链表必须顺序遍历，而有序数组可以采用第 17.6 节的二分查找。<sup>11</sup>

这些差异来自数据的组织方式：数组支持按下标随机访问，链表则通过指针逐个访问。链表在修改连接方面更灵活，却牺牲了直接下标访问能力，也影响了高效查找算法的使用。

数组与链表可以看作数据结构选择中的两端。程序员和计算机科学家还设计了许多变体，针对不同场景组合各自优点。后续的数据结构与算法分析课程会介绍更多选择。

---

<sup>11</sup>译注：链表插入或删除的链接修改是常数次操作，但寻找位置或目标节点仍可能需要线性遍历。数组并非任何增删都要移动全部后继，例如末尾操作或不要求保持顺序的删除可以不同。这里比较的是本例维护有序序列的需求。

## 19.6 小结

本章介绍了三个核心概念：

- **C 中的结构体**。结构体把较基本的数据类型组合为用户定义的聚合类型。这使我们能够直接表达复杂对象，也是理解第 20 章 C++ 对象的重要一步。
- **动态内存分配**。在程序执行过程中增长或缩小的数据结构，需要可控制的内存申请与释放机制。C 标准库通过 `malloc`、`free` 等函数提供这些功能。
- **链表**。结合结构体和动态分配，可以创建按指针连接的节点序列。它与数组一样组织线性数据，但能够在运行时灵活增删节点，适合某些固定数组会浪费空间或频繁搬移数据的场景。通过指针连接对象是基本思想，在哈希表、树和图等更复杂的数据结构中还会经常遇到。

## 习题

**19.1** 下面的程序有错误吗？请解释。

```
struct node{
 int count;
 struct node *next;
};

int main(void)
{
 int data = 0;
 struct node *getdata;
 getdata->count = data + 1;
 printf("%d", getdata->count);
}
```

译注：保留原题代码中的待分析问题；原书未列出 *stdio.h*，独立编译时还应包含该头文件。

**19.2** 下面是一个 C 程序的片段。

```
struct node{
 int count;
 struct node *next;
};

int main(void)
{
 int data = 0;
 struct node *getdata;
 :
 :
 getdata = getdata->next;
 :
 :
}
```

用 LC-3 汇编写出编译器为语句 `getdata = getdata->next;` 生成的指令。

**19.3** 图 19.2 的 NearestNeighbor 对当前空域中的飞机进行成对检查。如果有  $n$  架飞机，共进行多少次检查？

**19.4** 下面的程序在某种计算设备上编译。假定每种基本数据类型——指针、字符、整数和浮点数——都占一个存储单元。

```
typedef struct element_t Element;
struct element_t {
 char name[25];
 int atomic_number;
 float atomic_mass;
};

int NobleElement(Element *t)
{
 if ((t->atomic_number==2) ||
 (t->atomic_number==10) ||
 (t->atomic_number==18) ||
 (t->atomic_number==36) ||
 (t->atomic_number==54) ||
 (t->atomic_number==86))
 return 1;
 else
 return 0;
}

int main(void)
{
 int x, y;
 Element periodic_table[110];
 :
 :
 // Determine if the yth element is noble
 x = NobleElement(____);
 :
 :
}
```

- a. NobleElement 函数的栈帧包含多少个存储单元?
- b. 假定 periodic\_table、x 和 y 是仅有的局部变量, main 栈帧中有多少个单元用于局部变量?
- c. 补全 main 中调用 NobleElement 时留出的空白。

译注: 原书的 `typedef element_t Element;` 缺少 `struct`, 这里补齐; 其余保留题设模型和待填空。

**19.5** 下面的 C 程序编译为 LC-3 机器语言并执行。运行时栈从 xEFFF 开始，用户输入 abac 后按回车。

```
#include <stdio.h>
#define MAX 4

typedef struct rec_t Rec;
struct rec_t {
 char ch;
 Rec *back;
};

int main(void)
{
 Rec *ptr, pat[MAX+2];
 int i = 1, j = 1;
 printf("Pattern: ");
 pat[1].back = pat;
 ptr = pat;
 while ((pat[i].ch = getchar()) != '\n') {
 ptr[++i].back = ++ptr;
 if (i > MAX) break;
 }
 while (j <= i)
 printf("%d ", pat[j++].back - pat);
 // Note the pointer arithmetic here: subtraction
 // of pointers to structures gives the number of
 // structure elements, not the number
 // of memory locations
}
```

- a. 给出程序结束时 main 栈帧中的内容。
- b. 对输入 abac，这个程序的输出是什么？

译注：原书的 `typedef Rec` 和混用的 `rec_t/struct Rec` 无法构成有效的 C 类型声明，这里整理为一致的 `Rec` 别名。但核心语句 `ptr[++i].back = ++ptr;` 对 `ptr` 的读取与修改没有所需的求值次序约束，在 C 中属于未定义行为。原题也可能读取尚未初始化的 `back`。保留这些语句，不擅自替换为另一种算法；因此仅凭题面不能得到唯一、可移植的栈状态和输出，需先明确作者意图与修正规则。

**19.6** 编写 `malloc` 和 `free` 的基本版本。假定每当程序开始时，分配器都会初始化，所管理的堆最初为空、尚未分配。开始时先不考虑效率，也不必考虑重新利用已释放的内存块。先做出能够工作的版本，再逐步改进。

## 第二十章 C++ 入门

### 20.1 C++ 基础

我们可以把编程语言看作构筑数字世界的脚手架。要开发更加复杂、安全而可靠的应用和服务，编程系统——也就是语言和工具——就必须有助于提高程序员的生产率。如果用某种语言编程十分烦琐、过于复杂、容易出错，或者难以扩展，那么能够用它开发的应用就会受到限制，这种语言也不大可能得到广泛使用。许多语言兴起又消失，很少有语言能像 C 和 C++ 一样经久不衰。对于性能要求很高的核心系统代码，两者都是开展大规模软件开发的有效工具。

C++ 的诞生，源于帮助 C 程序员编写更复杂程序的愿望。规模较大的 C 程序容易变成由类型、结构体和函数组成的庞杂集合。C 语言本身没有提供系统组织这些代码实体的机制。随着项目扩大、参与人员增多，缺少组织机制会使代码难以扩展和维护，给软件工程管理带来许多麻烦。

C++ 针对的正是大型程序中的一些根本困难：许多程序员共同定义大量类型、结构体和函数，应该怎样组织它们？C++ 的创造者 Bjarne Stroustrup 在 20 世纪 70 年代末就看到了这些问题，并通过扩展 C，引入了把类型、结构体和函数组织为**类**的正式机制。类是第 19 章所介绍的 C 结构体的自然演进。借助类，代码更容易扩展和复用。大型 C++ 项目可以达到数千万行甚至更多；尤其在要求高性能或高度优化的应用中，C++ 得到了广泛使用。

C++ 是一门内容丰富的语言。它从“带类的 C”逐步发展而来——事实上，最初的名字就是 *C with Classes*——后来又加入了多种高级语言特性。要全面讲解这些特性，恐怕还需要另外 1000 页。不过，不必担心：本章只介绍我们认为最基础、最重要的 C++ 内容。这些知识足以让你在编写较复杂的 C++ 程序时有一个扎实的起点。你可以把本章作为进入 C++ 的第一步，在后续课程中继续探索。

好消息是，前面各章讨论的 C 知识，几乎都可以直接用于 C++。C++ 最初建立在 C 的基础之上；基本类型、变量、代码结构和函数，大多与我们已经学习的 C 相同或相似。如果把第 11–19 章中的 C 改成 C++，其中大部分内容仍然成立。<sup>1</sup>

请记住一个简单的概念：C++ 代码可以编译到与 C 相同的执行模型上。在本书的 C 模型中，程序由一组函数组成，从 main 开始，依照函数调用关系顺序执行；运行时栈用于传递参数和分配局部变量或对象，堆用于动态分配，全局区用于全局分配。同样的框架也适用于本书讨论的 C++ 程序。二进制表示、基本类型、基本结构和执行模型并没有因为改用 C++ 而被推翻。在讨论各种 C++ 特性时，记住这一点会很有帮助。C++ 的许多看似神奇的功能都在编译层实现：编译器替我们承担了更多生成可执行代码的工作，使软件能够使用更高层次的抽象，也使编程更加方便。

---

<sup>1</sup>译注：这里强调两种语言的大量共通之处，并不表示 C 是 C++ 的严格子集。例如，第 19 章介绍的从 `void *` 到其他对象指针的隐式转换，在 C++ 中并不成立；两种语言还有其他兼容性差异。



## 20.2 从 C 到 C++

C++ 推出后很快受到欢迎，部分原因在于，人们把它看作当时最流行的编程语言 C 的改进版本。C++ 保留了 C 的结构概念，同时从软件工程的角度弥补了若干重要不足。它主张围绕程序要操作的对象组织程序，而实现这种组织的机制就是类。我们稍后就会讨论类。

在扩展 C 的过程中，C++ 也改进了一些其他方面。与引入类相比，这些变化没有那么显著，却能让编程更加方便。本节先介绍几项较小但经常使用的改进。本书一贯通过简单代码示例引入重要的新概念，因此，我们直接从图 20.1 的例子开始，逐项看看它与 C 有哪些区别。

### 20.2.1 编译 C++ 代码

图 20.1 的语法和结构看起来很像 C 程序。由于 C++ 建立在 C 的基础上，我们学过的大部分 C 知识都能派上用场。事实上，熟悉 C 的读者只需阅读这段代码，就能相当准确地判断程序的作用。它与第 16 章初次介绍 C 指针时给出的图 16.4 很相似。你可能会对代码中出现三个版本的 swap 感到疑惑，稍后我们会解释原因。

```
1 #include <iostream>
2
3 void swap(int &x, int &y)
4 {
5 int temp = x;
6 x = y;
7 y = temp;
8 }
9
10 void swap(char &x, char &y)
11 {
12 char temp = x;
13 x = y;
14 y = temp;
15 }
16
17 void swap(double &x, double &y)
18 {
19 double temp = x;
20 x = y;
21 y = temp;
22 }
23
24 int main (void)
25 {
26 // Variable declarations
27 int a = 4, b = 5;
28 char c = 'c', d = 'd';
29 double x = 3.14, y = 1.41;
30
31 // Before the swaps
32 std::cout << "a = " << a << " b = " << b << std::endl;
33 std::cout << "c = " << c << " d = " << d << std::endl;
34 std::cout << "x = " << x << " y = " << y << std::endl;
35
36 swap(a, b);
37 swap(c, d);
38 swap(x, y);
39
40 // After the swaps
41 std::cout << "a = " << a << " b = " << b << std::endl;
42 std::cout << "c = " << c << " d = " << d << std::endl;
43 std::cout << "x = " << x << " y = " << y << std::endl;
44 }
```

图 20.1 一个简单的 C++ 代码示例

由于两种语言相近，大多数提供 C 开发环境的集成开发环境（IDE）也支持编译 C++ 代码；其中不少本来就是同时支持 C 的 C++ 开发环境。本书提供的几个 C++ 示例遵循 ISO C++，也就是标准 C++ 的常用编程约定。现在可以在你喜欢的 IDE 中试着编译一些简单的 C++ 代码，例如图 20.1 中的程序。

### 20.2.2 命名空间

**命名空间** (namespace) 在 C 中没有直接对应的机制。它是 C++ 中面向大型软件项目的一项概念，尤其适用于许多人共同开发、代码分布在多个文件中的项目。

团队编写较大程序时，经常会遇到名称冲突：函数、全局变量或结构体类型难免在某处使用了相同名称。要消除冲突，就必须有人修改某些名称，这个过程可能既别扭又费力。若想在 C 中系统地解决这类问题，往往需要采用烦琐的命名约定。

命名空间提供了更好的办法。在 C++ 中，可以创建一个命名空间，使其中的标识符与其他命名空间里的同名标识符相区别。这样，名称就可以专属于某一部分代码，这与作用域的概念很相似（见 12.2.3 节）。对大型代码库来说，这种能力非常重要。

刚开始学习 C++ 时，默认的全局命名空间通常就够用了；它也最接近 C 程序员熟悉的情形。在大多数入门程序中，我们创建的实体都放在全局命名空间中。例如，图 20.1 中的 swap 函数就属于全局命名空间。

我们也可以使用其他命名空间中的类型、函数和全局值。例如，C++ 标准库的许多实体定义在 std 命名空间中。使用它们时，可以在标识符前加上 std::。图 20.1 中多次出现了这种写法，表示这些标识符定义在另一个命名空间，而我们正在当前代码中使用它们。C++ 中的 :: 称为**作用域解析运算符**，它使我们能够明确指出某个标识符来自哪个命名空间。讨论类时，我们还会再次遇到它。

### 20.2.3 输入和输出

继续观察那些带有 std:: 的输出语句。与 C 相比，C++ 为常见的格式化 I/O 提供了一种更方便的写法。其中，cin 是标准输入流对象，作用类似于 C 的 stdin；cout 是标准输出流对象，作用类似于 stdout。这些流对象由第 1 行包含的 iostream 头文件声明。

执行基本格式化 I/O 时，除了使用 printf 和 scanf，C++ 还可以使用运算符从 cin 提取数据，或者向 cout 插入数据。>> 是流提取运算符，<< 是流插入运算符。例如，第 32–34 行和第 41–43 行使用 <<，把字符串和变量插入标准输出流 cout。还可以使用 endl 插入换行。各项内容按插入输出流的顺序，逐字符显示在标准输出设备上。这些名称都属于 C++ 标准库的 std 命名空间，因此示例中都加上了 std:: 前缀。变量插入输出流时，会根据其类型转换为相应的字符表示；例如，在示例所用的默认格式下，整数会转换为十进制数字字符。<sup>2</sup>

输入的方式类似，使用标准输入流 std::cin 和提取运算符 >>。例如，下面的代码从 cin 读取一个整数，赋给变量 input：

```
int input;
std::cout << "Provide an integer value: ";
std::cin >> input;
```

尽管 C++ 的 I/O 写法与 C 很不一样，底层功能却相近。我们把 <<、>> 与 cout、

---

<sup>2</sup>译注：原书把 endl 称为常量，实际它是流操纵器，除了写入换行，还会刷新输出流。输出也不要求底层字符编码一定是 ASCII；这里延续本书所用的字符表示模型。

`cin` 组合使用，代替了这里原本会使用的 `printf` 和 `scanf`。编译器会把这些操作转换为适当的函数调用，并传入相应参数。因此，这种写法在很大程度上是为了方便程序员。最终仍需调用 I/O 功能，并通过系统调用——例如本书模型中的 `TRAP`——完成实际输入输出。

### 20.2.4 传递引用

在 C 中通过指针让函数修改调用者的数据，有时相当棘手。回想第 19 章的链表代码：为了在空表中添加飞机，或把新飞机添加到表头，`AddFlight` 必须接收头指针的地址，而不是仅接收头指针的值。因此需要使用指向指针的指针 `Flight **list`，并在函数内部小心使用解引用运算符，才能修改正确的内存位置。在 C 中改用这种间接访问方式时，通常必须同时修改函数和调用它的各处代码。<sup>3</sup>

C++ 可以直接在函数定义中把形参声明为引用。看图 20.1 第 3 行：`swap` 的两个形参名前都有 `&`，说明它们是整数变量的引用，而不是独立的整数值副本。于是，在 `swap` 内部，可以用访问普通变量的语法访问原变量——本例中是 `main` 栈帧里的变量——不必像 C 指针版本那样显式解引用。编译器会根据标识符表示的是引用还是普通值，生成相应代码。

例如，第 6 行的赋值：

```
x = y; // Equivalent to *x = *y in C
```

可翻译为以下 LC-3 指令：

```
LDR R0, R5, 4 ; Load x, which is a reference
LDR R1, R5, 5 ; Load y, which is a reference
LDR R2, R1, 0 ; Load *y
STR R2, R0, 0 ; Perform *x = *y
```

这又展示了 C++ 如何把 C 中较烦琐、不易阅读的写法变得直接。若要把某个参数改为引用，通常只需调整函数的声明与定义，而不必在各个调用点另加取地址操作。编译器根据函数声明，决定在每个调用点应传入值还是用于访问原对象的引用。程序员不必显式写出底层的地址传递。<sup>4</sup>

### 20.2.5 函数重载

现在解释图 20.1 为什么有三个同名的 `swap` 函数。你可能已经注意到，除了形参和内部临时变量的类型不同，它们的代码几乎一样，执行的操作也相同。

C++ 允许定义多个同名函数，只要它们的形参类型列表有所区别，包括类型的排列次序。本例的三个 `swap` 分别接收两个整数引用、两个字符引用和两个 `double`

<sup>3</sup>译注：C 的参数传递机制仍然是传值，传入的只是地址值。这里所谓“传递引用”是借助指针达到间接修改原对象的效果，区别于下文 C++ 的引用参数。

<sup>4</sup>译注：本例是可修改的左值引用参数，调用时仍须提供能够绑定到该引用的实参；改变形参类型并不保证原先所有调用都继续合法。图中的 LC-3 指令说明一种实现方式，并非 C++ 对所有机器的调用约定作出的规定。

引用。编译器在每个调用点根据实参的类型，选择实际调用的版本；也就是让实参列表与合适的形参列表相匹配。

这种能力称为**函数重载**（function overloading）。一个函数名可以对应形参不同的多个版本。可以把其实实现想象成一种编译器执行的名称处理：按照形参出现的次序，把类型信息附加到函数名上。例如，`swap_int_int` 与 `swap_char_char` 就是不同的名称。这个过程在编译时自动完成，无须程序员手动处理。<sup>5</sup>

函数重载有什么用？仍以 `swap` 为例，我们希望无论操作哪种类型，都能使用通用的交换功能：交换两个整数、两个字符或两个 `double`。借助重载，就能保留直观的名称 `swap`，同时为各种基本类型提供不同版本，如图 20.1 所示。

还可以再进一步。三个版本执行同样的底层操作，只是类型不同。为了复用代码、减少源代码重复，我们也可以使用**模板**，使这段代码不依赖某个特定类型。20.4 节将讨论模板。

### 20.2.6 动态分配

在 C 中，我们使用 `malloc` 从堆上动态分配一块内存；用完之后，再用 `free` 把它归还给堆，供以后重新分配。`malloc` 和 `free` 都是 C 标准库提供的函数。

C++ 则可以使用 `new` 和 `delete` 运算符完成动态分配与释放：

```
1 int *p;
2
3 p = new int;
4 :
5 :
6 delete p;
```

第 3 行用 `new` 分配足以存放一个整数的空间，第 6 行用 `delete` 释放 `p` 所指的對象。与 C 的库函数不同，`new` 和 `delete` 是语言中的运算符；用于类类型时，创建和销毁对象还会涉及构造函数和析构函数。20.3.3 节将进一步介绍。<sup>6</sup>

### 20.2.7 编译为机器代码

本节介绍了从 C 到 C++ 的一小组重要变化。还有许多概念可以讨论，不过我们的目的不是全面覆盖语言，而是让你能够着手开发规模较小、功能却不弱的 C++ 程序。把这些内容与前面学过的 C 知识结合起来，你已经掌握了相当一部分 C++ 基础。

在本书的模型中，C++ 与 C 使用相同的运行时框架。C++ 程序也可以编译成一组函数。图 20.1 中的代码对应四个用户定义函数，以及它所需的库功能。`main` 为各个涉及 `std::cout` 的格式化输出语句调用相应的输出功能，然后依次调用整数版本

<sup>5</sup>译注：这里的名称只是示意，实际名称修饰规则取决于编译器及其 ABI。仅返回类型不同，不能构成函数重载；存在多种匹配可能时，还需要依照重载决议规则选择，无法唯一选择就会产生编译错误。

<sup>6</sup>译注：原书把第 6 行误说成使用 `free`，并把构造函数讨论误引到 20.3.2 节，这里已修正。`delete p` 释放的是所指对象，不是指针变量本身，也不会自动把 `p` 置空。`new int` 未给整数提供初始值。使用数组形式 `new[]` 时，应与 `delete[]` 配对；不要与 `malloc/free` 混用。

的 swap、字符版本的 swap 和 double 版本的 swap，再执行后续输出。与 C 一样，这些高层写法最终都由机器能够执行的操作实现。

## 20.3 类

C++ 相对 C 最初的一项重要改进，就是引入了类。结构体允许程序员定义自己的类型，类则进一步发展了这一思想：除了数据成员，还可以包含函数成员、建立层次关系，并提供明确的接口。使用类声明的内存实体，可以看作带有更丰富操作的**对象**。这些思想在早于 C++ 的其他语言中已经出现，而 C++ 把它们融入了当时很流行的 C。

类有十分明确的实用价值。C 程序通常把类型、值和函数并列组织在一起。程序越复杂，这些实体越容易混杂成团，妨碍后续开发。类首先是一种组织机制，使相互关联的功能能够放在一起，从软件工程角度带来明显好处。后来许多成功的语言，例如 Java，也提供了类的支持，这并不令人意外。

|                                                                                                                |                                                                                                                      |
|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| <pre>// C++ code class Triangle {     double sideA;     double sideB;     double sideC; };  Triangle t1;</pre> | <pre>// C code struct Triangle {     double sideA;     double sideB;     double sideC; };  struct Triangle t2;</pre> |
|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|

图 20.2 比较 C++ 与 C 中的三角形类型

我们从图 20.2 中一个简单的类 Triangle 开始。像第 19 章使用结构体一样，我们也可以用类定义自己的类型。Triangle 包含描述某个三角形的成员，这里就是三条边的长度。它与 C 结构体的写法非常相似。图中两个版本都创建了新类型：C++ 的 Triangle 类让我们可以声明对象 t1，C 的 struct Triangle 则让我们可以声明 t2。本节所说的类对象，就是以类为类型的内存实体。<sup>7</sup>

### 20.3.1 方法

再向对象的世界深入一步。我们可以让某些函数与 Triangle 对象关联起来。这些函数针对该类的具体对象执行操作，脱离对象就没有这里所需的含义。这样的函数称为**方法**（method），也就是成员函数。对三角形来说，我们可能希望计算它的面积和周长。

现在给类添加两个函数：一个计算周长，另一个计算面积。稍后还会再添加一个。图 20.3 给出了扩充后的定义，其中新增了第 5、6、7 行。第 5 行的含义下一小节再讲。第 6、7 行把两个方法声明为类的成员；两者都返回 double，形参列表都为空。

<sup>7</sup>译注：C 和 C++ 标准中的“对象”一词比这里更广，也包括整数、数组等具有存储的实体，并不限于类的实例。这里沿用原书面向类的介绍语境。

第 10–13 行定义 `perimeter`，第 15–21 行定义 `area`。它们与 C 中的函数定义相似，但有一个关键区别：这里的两个方法针对某个具体的 `Triangle` 对象调用。稍后我们会进一步解释。目前，可以先把它们看作与已经熟悉的 C 函数类似的成员函数。

```
1 class Triangle {
2 double sideA;
3 double sideB;
4 double sideC;
5 public:
6 double area();
7 double perimeter();
8 };
9
10 double Triangle::perimeter()
11 {
12 return sideA + sideB + sideC;
13 }
14
15 double Triangle::area()
16 {
17 double s = perimeter() / 2;
18
19 // Heron's Formula
20 return sqrt(s*(s - sideA)*(s - sideB)*(s - sideC));
21 }
```

图 20.3 含有两个方法的 `Triangle` 类

定义中，方法名前的 `Triangle::` 告诉 C++ 编译器，这个方法属于 `Triangle` 类，也就是处于该类的作用域中。`perimeter` 把三条边的长度相加。不过，是哪个三角形的三条边呢？`area` 调用 `perimeter` 求得半周长，再结合三条边，利用海伦公式计算面积。那么，这里调用的是哪个对象的方法，访问的又是哪一个对象的边呢？

图 20.3 只定义了类及其方法，还没有创建任何对象。接下来创建一个 `Triangle` 对象并使用它。图 20.4 是 `main` 函数的一部分，其中声明了 `Triangle` 类型的对象 `t1`。在本书的栈帧模型中，`t1` 分配在 `main` 的栈帧里，占用足以容纳数据成员的空间；这里就是三个表示边长的 `double`。就这些数据成员而言，这与图 20.2 中 C 结构体所需存储的内容相同。方法的代码不会因此在每个对象里各复制一份。

```
1 int main(void)
2 {
3 Triangle t1;
4 double p1;
5 :
6 :
7 p1 = t1.perimeter();
8 :
9 }
```

图 20.4 C++ 中的方法调用

第 7 行对对象 `t1` 调用 `perimeter` 方法。它可以翻译为与 C 普通函数调用相似的低层汇编代码：压入参数、建立栈帧、使用 JSR 转移控制、使用 RET 返回，等等。

然而，与普通 C 函数不同，`perimeter` 需要关联一个 `Triangle` 对象，如第 7 行所示。在类外直接调用未限定的 `perimeter` 或 `area`，会产生编译错误。这里的方法是一种与特定类类型联系在一起的函数。

`perimeter` 需要知道要操作哪个对象，才能计算结果。如果在汇编层它只是普通子程序，而且显式形参列表为空，又如何知道目标是哪一个三角形呢？例如，执行第 7 行时，如何知道应该处理 `t1`？为实现这一点，调用这种成员函数时，会把目标对象的地址作为额外的隐式参数传进去。在 C++ 中，这个隐式对象指针称为 `this` 指针，方法内部可以使用它。

因此，在本书的调用模型中，方法调用可以像 C 函数调用一样实现，不过还需要把 `this` 指针作为隐式实参压栈。图 20.5 给出了对应图 20.4 第 7 行的 LC-3 示意代码，以及 `perimeter` 方法的实现。在方法内部，访问 `Triangle` 数据成员的操作，都以 `this` 指针为基准。<sup>8</sup>

---

<sup>8</sup>译注：这里讨论普通的非静态成员函数。静态成员函数没有 `this` 指针；实际编译器也可能用寄存器传递隐式对象参数。原文把第 7 行的调用误称为 `area`，应为 `perimeter`。



```

1 perimeter
2 ADD R6, R6, #-1 ; Allocate spot for the return value
3
4 ADD R6, R6, #-1 ;
5 STR R7, R6, #0 ; Push R7 (Return address)
6
7 ADD R6, R6, #-1 ;
8 STR R5, R6, #0 ; Push R5 (Caller's frame pointer)
9
10 ADD R5, R6, #-1 ; Set frame pointer for perimeter
11
12 LDR R0, R5, #-4 ; Load this pointer
13 AND R1, R1, #0 ; Zero out R1
14 LDR R2, R0, #0 ; Load this->sideA
15 ADD R1, R1, R2 ; Add to perimeter
16 LDR R2, R0, #1 ; Load this->sideB
17 ADD R1, R1, R2 ; Add to perimeter
18 LDR R2, R0, #2 ; Load this->sideC
19 ADD R1, R1, R2 ; Add to perimeter
20
21 STR R1, R5, #3 ; Write it in return value slot
22 ADD R6, R5, #1 ; Pop local variables
23
24 LDR R5, R6, #0 ; Pop the frame pointer
25 ADD R6, R6, #1 ;
26
27 LDR R7, R6, #0 ; Pop the return address
28 ADD R6, R6, #1 ;
29 RET
30 ...
31 main
32 ...
33 ADD R0, R5, #0 ; Load address of t1 into R0
34 ADD R6, R6, #-1 ;
35 STR R0, R6, #0 ; Push &t1
36 JSR perimeter
37
38 LDR R0, R6, #0 ; Load the return value from perimeter
39 STR R0, R5, #1 ; p1 = t1.perimeter();
40 ADD R6, R6, #2 ; Pop return value and argument
41 ...

```

图 20.5 与图 20.4 方法调用相对应的 LC-3 示意代码

译注：原图保留，但它不能直接作为前述 double 程序的正确机器代码。按图中压栈顺序，this 位于被调用者的 R5+4，第 12 行的 #-4 应为 #4。图中用单个存储单元表示每条边和返回值，并用整数 ADD 求和，这适合单字整数的简化示意，不能实现 double 加法。真实翻译还需处理浮点表示、运算和返回值所需空间。调用者第 33、39 行的局部对象偏移也必须依据实际栈帧布局重新确定；按前文局部变量向低地址分配的约定，不能直接套用这里的 R5+1 作为 p1 的位置。

### 20.3.2 访问说明符

C++ 类的一个好处，是鼓励在软件开发中使用抽象。抽象贯穿本书，因为它是数字系统体系结构的基础：用定义清楚的接口隐藏某一层的复杂细节，就能为更高一层提供构件。例如，相互连接的 MOS 晶体管构成 NOT 门，而 NOT 门是一种更方便组合的抽象，可以参与构造加法器。在 C 中，函数是实现抽象的主要手段：函数可以完成复杂功能，却只通过参数等简单接口与外部交互。一旦理解了功能，就可以暂时隐藏其实现细节。

C++ 利用类扩展并规范了这一思想。类有明确的接口，而接口由类定义中的**访问说明符**（access specifier）来规定。成员可以允许外部访问，成为接口的一部分；也可以只供内部使用，成为实现的一部分。使用某个类时，我们就能把注意力放在接口上，不必了解全部内部细节。

C++ 有多种访问说明符，这里着重讨论 public 和 private。私有成员供类内部使用，公有成员则既可以在内部访问，也可以从外部访问。

图 20.3 第 5 行使用了 public，因此后面的 area 和 perimeter 都是公有成员。在能够使用该类的代码位置，可以通过对象调用它们。这两个成员恰好是方法，但数据成员也可以声明为公有。相反，sideA、sideB 和 sideC 前没有访问说明符；使用 class 定义类时，默认访问权限是私有。因此，这些边长对类外代码隐藏起来，只能通过类提供的操作访问。在目前的接口中，我们可以对 Triangle 对象调用 area 和 perimeter；其余内容则是被抽象隐藏的实现细节。<sup>9</sup>

public 和 private 主要是编译期检查规则，在这一点上与作用域相似；它们不是每次运行到成员访问时再检查的权限。如果从类外、又没有相应访问权限的代码访问私有成员，编译器就会报错。例如，下面第 7 行直接访问私有成员 sideA，会导致编译错误：

```
1 int main(void)
2 {
3 Triangle t1;
4 double a1;
5 :
6 :
7 t1.sideA = 10; // This will generate an error
8 a1 = t1.area();
9 :
10 }
```

### 20.3.3 构造函数

既然前面的代码不能直接给私有边长赋值，那么如何初始化 Triangle 对象呢？可以定义一个**构造函数**（constructor）。它是类的一种特殊成员函数，在创建该类对象

---

<sup>9</sup>译注：更完整地说，类的成员和被授权的友元可以访问私有成员；访问权限属于类，并非仅限于访问当前对象自己的私有成员。这里尚不讨论友元。C++ 的 struct 默认成员访问权限为公有，class 则为私有。

时被隐式调用。例如，函数执行到局部对象的声明时，就会为该对象建立存储，并进行相应的构造。

下面在 `Triangle` 类上进一步扩充，展示构造函数的语法。构造函数与类同名，不声明返回类型，也不返回一个值。第 6 行声明构造函数，第 11 行给出定义。它接收三个 `double`，用于初始化新建三角形的三条边。

```
1 class Triangle {
2 double sideA;
3 double sideB;
4 double sideC;
5 public:
6 Triangle(double a, double b, double c);
7 double area();
8 double perimeter();
9 };
10
11 Triangle::Triangle(double a, double b, double c)
12 {
13 sideA = a;
14 sideB = b;
15 sideC = c;
16 }
```

无论对象按本书模型分配在运行时栈上，还是通过动态分配建立在堆上，都需要进行相应的构造。下面创建两个对象。第一个对象 `t1` 是局部变量，分配在运行时栈中；创建它时，会隐式调用 `Triangle::Triangle`。

第 2 行声明了指向 `Triangle` 的指针变量 `t2`，但还没有让它指向有效的对象。第 4 行使用 `new` 为对象分配空间（见 20.2.6 节），随后调用 `Triangle` 的构造函数，根据传入的实参初始化内部数据成员。

```
1 Triangle t1(2.0, 2.0, 3.0);
2 Triangle *t2;
3
4 t2 = new Triangle(4.2, 7.8, 10.2);
```

构造函数由编译器安排隐式调用，这与 C 的做法有所不同。C 的设计倾向于给程序员较多控制权，使源代码与硬件较为接近。C++ 对这一取向作了一定调整，希望由此建立更便于扩展的代码组织方式。

C++ 允许在构造函数中安排各种所需操作。实际使用时，构造函数主要负责初始化类对象的内部数据和状态。C++ 还提供与之对应的**析构函数**（`destructor`），用于对象销毁时的清理工作，例如通过 `delete` 销毁动态对象，或者退出局部对象所在的块时。析构函数的详细讨论留待后续学习。

图 20.6 给出了这个连续示例的完整代码。

```
1 #include <iostream>
2 #include <math.h>
3
4 class Triangle {
5 double sideA;
6 double sideB;
7 double sideC;
8 public:
9 Triangle(double a, double b, double c);
10 double area();
11 double perimeter();
12 };
13
14 Triangle::Triangle(double a, double b, double c)
15 {
16 sideA = a;
17 sideB = b;
18 sideC = c;
19 }
20
21 double Triangle::perimeter()
22 {
23 return sideA + sideB + sideC;
24 }
25
26 double Triangle::area()
27 {
28 double s = perimeter() / 2;
29
30 // Heron's Formula
31 return sqrt(s*(s - sideA)*(s - sideB)*(s - sideC));
32 }
33
34 int main (void)
35 {
36 Triangle t1(2.0, 2.0, 3.0);
37 Triangle *t2;
38
39 t2 = new Triangle(4.2, 7.8, 10.2);
40
41
42 std::cout << "Tri 1 Area " << t1.area() << std::endl;
43 std::cout << "Tri 1 Perim " << t1.perimeter() << std::endl;
44
45 std::cout << "Tri 2 Area " << t2->area() << std::endl;
46 std::cout << "Tri 2 Perim " << t2->perimeter() << std::endl;
47 }
```

**图 20.6** 一个使用 C++ Triangle 类的完整示例

译注：图中没有在结束使用后执行 `delete t2;`，实际程序应释放该动态对象；局部指针离开作用域不会自动删除它所指向的对象。此外，示例的构造函数只赋值，不检查三条边是否为正并满足三角形不等式。图 20.3、20.4 是逐步讲解的片段，调用面积或周长前必须先正确初始化边长，不能把未初始化的默认对象直接用于计算。

### 20.3.4 进阶主题

到这里，我们只是初步接触了 C++ 的类。本章已经介绍的是使用最频繁的核心概念。结束这一主题之前，再简要看看下一步：**继承** (inheritance)。

项目逐步成熟、规模不断扩大时，程序员定义的类型往往也越来越多。新加入的类可能与已有类大体相似，却又略有不同。结果是，类定义、方法和函数中出现了大量近似代码。为减少这种重复，C++ 的类机制支持继承，使一个类可以从另一个类派生出来。

仍以三角形为例。随着程序扩展，我们也许还想处理四边形和五边形，并建立能够包含不同多边形的数据结构。可以想象，这些类会有许多共同代码，只在具体方法的实现上有所差异。例如，它们计算面积和周长的方法各不相同。

在这种情况下，可以先建立 Polygon 基类，再从中派生三角形类 Triangle、四边形类 Quadrilateral 和五边形类 Pentagon。把共有的功能和数据成员放在 Polygon 中，三个派生类继承这些共同部分，只需补充各自特有的代码。

这样的层次结构让程序员能够建立丰富的类族，复用一個类中的代码，创建与它相似但又不同的派生类。这种机制就是继承。它可以避免为每种相近类型都重新定义结构、重新编写一套支持函数所造成的重复。

与前面讨论的类机制一样，继承关系由编译器参与处理。程序员构造从其他类派生的复杂类型，编译器把它们组织为数据成员和方法。最终，仍要落实为数据在内存中的布局，以及调用适当方法的机器指令。<sup>10</sup>

## 20.4 容器与模板

创建新类型时——无论是 C 的 struct，还是 C++ 的 class——我们很容易同时考虑要用哪种数据结构存放它。例如，如果希望把前一节的 Triangle 对象组织成链式结构，就可能在类定义中加入保存链接的数据成员。若以后决定换一种数据结构，就得重新修改类或结构体。若代码的不同地方同时需要两种组织形式，还可能不得不复制类型定义，造成冗余。

**容器**把数据类型与数据结构分离开来。我们可以独立定义对象的类型，再选择容纳这种对象的数据结构。两者的设计约束各不相同，能够分开考虑，会显著减轻编程负担。概括地说，容器就是一种可用于容纳不同类型对象的、灵活的数据结构实现。

C++ 通过**标准模板库** (Standard Template Library, STL) 提供常用容器。许多有经验的程序员把 STL 看作 C++ 最有用的部分之一。借助预先实现的容器及相关组件，程序员无须编写大量代码，就能建立可调整大小的动态数组、链表、栈、队列和映射等数据结构。<sup>11</sup>

<sup>10</sup>译注：原文用“编译器构造”和 JSR 概括这一过程，是面向本章简单示例的说明。继承还可以配合虚函数实现运行时选择方法；涉及这种动态分派时，不一定是目标在编译期固定的直接调用。

<sup>11</sup>译注：STL 通常指标准库中以容器、迭代器和算法等为核心的一组设施，不等同于整个 C++ 标准库；栈和队列在其中以容器适配器的形式提供。

### 20.4.1 vector

我们重点介绍一种 STL 容器: `vector`。经历过 C 数组的种种限制之后, 它的用处应该很容易理解。`vector` 可以方便地调整元素数量, 也就是实现动态数组。19.4.1 节讨论过这一概念: C 对动态数组的直接支持相当有限。要完整实现动态数组, 每增加一种元素类型, 往往还要编写相应的支持代码, 给程序员增加负担。

```
1 int main (void)
2 {
3 std::vector<Triangle> triVector(10); // Vector of 10 tris
4 std::vector<int> intVector(20); // Vector of 20 ints
5
6 for (int i = 1; i < intVector.size(); i++)
7 intVector[i] = intVector[i - 1] + i;
8
9 // Print out vector
10 std::cout << "intVector[" << intVector.size() << "] = ";
11 for (int i = 1; i < intVector.size(); i++)
12 std::cout << intVector[i] << " ";
13 std::cout << std::endl;
14
15 // Let's resize!
16 intVector.resize(50);
17
18 // Print out vector
19 std::cout << "intVector[" << intVector.size() << "] = ";
20 for (int i = 1; i < intVector.size(); i++)
21 std::cout << intVector[i] << " ";
22 std::cout << std::endl;
23 }
```

图 20.7 使用 C++ 标准模板库中 `vector` 容器的示例

`vector` 提供了丰富的数组操作, 并能用于我们自己创建的类型。图 20.7 声明了两个 `vector`: 一个以 `Triangle` 为元素类型, 另一个以 `int` 为元素类型。注意第 3、4 行的新声明语法: `std::` 表示使用标准库命名空间, `vector` 是所使用的容器类模板, 尖括号中指定元素类型。

译注: 原图是片段, 需要包含 `<vector>` 和 `<iostream>`, 并提供 `Triangle` 定义。若直接沿用图 20.6, 则 `triVector(10)` 无法编译, 因为该类只有三个参数的构造函数, 没有可供此处创建十个元素使用的默认构造函数。可以为类设计有效的默认构造方式, 或在构造容器时明确提供初始元素; 这里保留原图并指出前提。

示例中的 `intVector(20)` 使二十个整数元素初始为 0, 因此第一个循环能够从第 0 项开始累加。`resize(50)` 保留已有的二十项, 并把新增整数元素初始化为 0。两

个打印循环都从下标 1 开始，因此都没有打印第 0 项。索引的有效范围仍是从 0 到 `size() - 1`，容器并不会让任意下标访问都变得合法。

STL 的 `vector` 提供了多种现成方法，表 20.1 简要列出了其中一些。因此，只需很少的代码，就能得到具有一组丰富功能的数据结构；程序员主要需要指定希望存放的类型。

表 20.1 C++ STL 的 `vector` 类中一些有用的方法

| 方法                       | 说明       | 示例                                           |
|--------------------------|----------|----------------------------------------------|
| <code>size()</code>      | 返回元素数量   | <code>intVector.size();</code>               |
| <code>resize()</code>    | 改变元素数量   | <code>intVector.resize(newSize);</code>      |
| <code>insert()</code>    | 插入元素     | <code>intVector.insert(where, value);</code> |
| <code>erase()</code>     | 删除元素     | <code>intVector.erase(where);</code>         |
| <code>push_back()</code> | 在末尾添加新元素 | <code>intVector.push_back(value);</code>     |
| <code>front()</code>     | 访问第一个元素  | <code>y = intVector.front() - 1;</code>      |
| <code>back()</code>      | 访问最后一个元素 | <code>intVector.back() = 3;</code>           |

译注：表中的 `where` 表示迭代器位置，并非直接传入整数下标。`front()` 和 `back()` 要求容器非空；`size()` 是当前元素数量，不是已经预留的存储容量。

## 20.4.2 模板

C++ 的容器类使用**模板**构建。模板是一种语言机制，使我们能够编写不局限于某个具体类型的代码。例如，可以把图 20.1 中的 `swap` 改写为一个函数模板，用单份定义减少重复。编译器根据模板实参，生成所需的具体版本。编写模板属于较进阶的 C++ 内容；相比之下，日常程序中使用标准模板库提供的容器十分常见。

从本书自下而上的视角来看，模板、容器和 STL 再次展示了编译器的作用。C++ 源代码让编译器组合适当的类型和操作，最终生成与 C 程序同样由机器指令组成的代码。换句话说，这些特性中的许多操作，都能用具有相同作用的 C 代码来理解。<sup>12</sup>

<sup>12</sup>译注：这并不意味着容器操作不产生运行时工作；动态分配、元素构造与复制、扩容和遍历等，仍可能在程序执行期间发生。模板还要求所使用的类型支持相应操作，并不是所有模板都适用于任意类型。

## 20.5 小结

最后，总结本章介绍的几个关键 C++ 概念。

- **从 C 到 C++**。C++ 最初是为了提高 C 程序员的生产率而创建的。在本书所采用的底层框架中，两种语言都把代码翻译为机器指令，程序从 `main` 开始调用各个函数，运行时栈用于参数、返回值和局部数据，堆用于动态分配。C++ 还提供了更方便的格式化 I/O、函数重载和引用参数等写法。
- **类**。C++ 的类发展了 C 结构体的思想，把函数也组织为成员，也就是方法，并提供明确的接口和继承机制。类还带来隐式操作，例如创建类对象时调用其构造函数。
- **容器**。把数据类型与数据结构分开考虑，是一种有力的组织方式。C++ 的容器让我们能够这样做；标准库提供了多种数据结构，可以容纳程序中定义的对象。本章介绍的 `vector` 可用于实现数组，并提供调整大小、插入和删除元素等现成功能，比普通 C 数组更丰富。C++ 容器类使用模板构建。



## 习题

**20.1** 修改图 20.1 的代码，添加一个用于布尔类型的 swap 函数。

**20.2** 修改图 20.1 的代码，添加一个交换两个整数指针 (int \*) 的 swap 函数。

**20.3** 下面的代码会调用哪个版本的 funcA?

```
int funcA(int a); // Version A
int funcA(int a, int b); // Version B
int funcA(int a, int b, int c); // Version C

// Which funcA will get called here?
x = funcA(x, y, 2);
```

**20.4** 画出图 20.4 第 3、4 行分配的对象和变量的内存布局。

**20.5** 给 Triangle 类添加一个方法，计算以 sideC 为底边时三角形的高。

**20.6** 如果删除图 20.6 的第 8 行，最终会产生什么结果？

**20.7** 把第 19 章图 19.5、19.8、19.9 中跟踪飞机的代码改写为等效的 C++ 版本，使用类代替结构体。

**20.8** 编写 C++ 代码，定义一个与图 20.6 的 Triangle 类相似的四边形类，并提供计算面积和周长的方法。

**20.9** 扩展习题 20.8 的代码，使其能够处理五边形。

**20.10** 编写 LC-3 代码，实现图 20.6 第 36 行的局部对象分配。

**20.11** 考虑以下代码：

```
int main (void)
{
 std::vector<int> intVector(0); // Vector of 0 ints
 intVector.push_back(1);
 intVector.push_back(2);
 intVector.push_back(3);
}
```

a. 画出代码执行结束时 intVector 的内存布局。

b. intVector 分配在内存的什么位置？

译注：原书把三个 *push\_back* 都误写为 *pushback*，这里已恢复标准成员名；编译时还需包含 `<vector>`。本题的“执行结束”应理解为最后一次插入完成、离开 *main* 作用域之前。具体实现中的内部字段和容量增长策略并不由本题唯一规定，可画逻辑布局并注明采用的实现假设。

**20.12** 编写使用 vector 的 C++ 程序，从键盘读入一个字符串，再将其逆序输出。

**20.13** 编写使用 `vector` 的 C++ 程序，从键盘读入一行文本，并删除其中重复的单词。例如，输入为：

```
This line of this line text text contains no copied copied
↵ no words.
```

输出应为：

```
This line of text contains no copied words.
```

# 附录 A LC-3 指令集体系结构

## A.1 概述

LC-3 的指令集体系结构 (ISA) 定义如下。

**内存地址空间。**地址为 16 位，对应  $2^{16}$  个存储单元，每个单元包含一个 16 位字。地址编号从 0 (x0000) 到 65,535 (xFFFF)，既标识内存单元，也标识存储器映射 I/O 设备寄存器。图 A.1 给出了为特殊用途保留的区域。

x0000 至 x2FFF 是特权存储器，只能在管理模式 (PSR[15]=0) 下访问。x3000 至 xFDFF 可供用户模式的程序和数据使用。xFE00 至 xFFFF 用于 I/O 设备寄存器及特殊的处理器内部寄存器，同样只允许管理模式访问；从访问控制的角度看，这些地址也属于特权存储器。

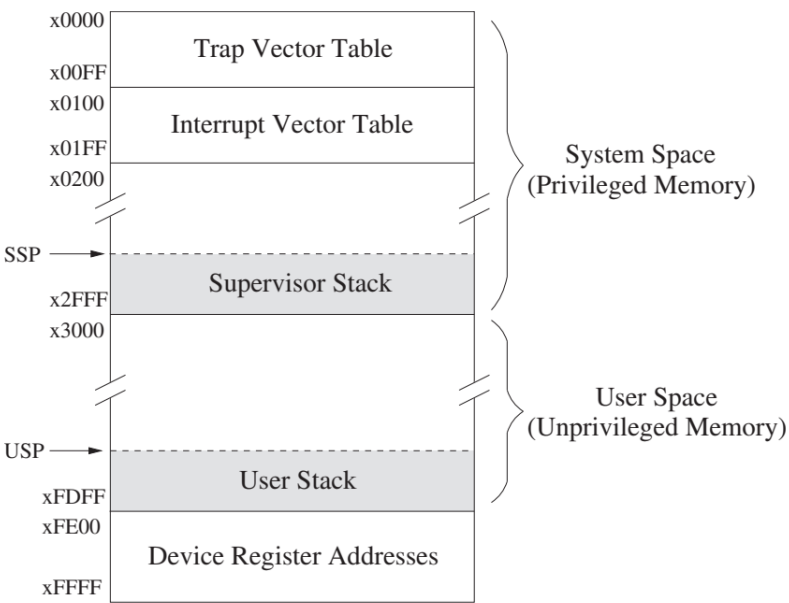


图 A.1 LC-3 的内存映射

**存储器映射 I/O。**输入输出由装入和存储指令 (LD/ST、LDI/STI、LDR/STR) 完成，用 xFE00 至 xFFFF 范围内的地址指定设备寄存器。表 A.1 列出本书 LC-3 已定义的 I/O 设备寄存器、处理器内部寄存器及分配给它们的地址。

表 A.1 设备寄存器地址分配

| 地址    | I/O 寄存器名称      | 功能                       |
|-------|----------------|--------------------------|
| xFE00 | 键盘状态寄存器 (KBSR) | 就绪位 (位 15) 表示键盘是否已收到新字符。 |
| xFE02 | 键盘数据寄存器 (KBDR) | 位 [7:0] 保存最近键入的字符。       |

| 地址     | I/O 寄存器名称      | 功能                               |
|--------|----------------|----------------------------------|
| xFE04  | 显示状态寄存器 (DSR)  | 就绪位 (位 15) 表示显示设备是否可以接收下一个待显示字符。 |
| xFE06  | 显示数据寄存器 (DDR)  | 写入位 [7:0] 的字符将显示在屏幕上。            |
| xFFFFC | 处理器状态寄存器 (PSR) | 保存当前进程的特权模式、优先级和条件码。             |
| xFFFFE | 机器控制寄存器 (MCR)  | 位 15 是时钟使能位, 清零后停止处理指令。          |

**位编号。**所有量的位从右向左编号, 从位 0 开始。一个存储单元内容的最左位为位 15。

**指令。**指令宽度为 16 位。位 [15:12] 指定操作码, 即要执行的操作; 位 [11:0] 提供执行该操作所需的其他信息。各条指令详见 A.2 节。

**非法操作码异常。**操作码 1101 尚未定义。如果指令的位 [15:12] 为 1101, 就产生非法操作码异常。处理过程见 A.3 节。

**程序计数器。**PC 是一个 16 位寄存器, 保存下一条待处理指令的地址。

**通用寄存器。**共有八个 16 位通用寄存器, 编号为 000 至 111, 即 R0 至 R7。

**条件码。**N、Z、P 是三个 1 位寄存器, 分别表示负、零、正。装入指令 LD、LDI、LDR 和运算指令 ADD、AND、NOT 把结果写入通用寄存器, 同时按结果作为 16 位二进制补码整数的符号设置条件码: 负数使 N=1、Z=P=0; 零使 Z=1、N=P=0; 正数使 P=1、N=Z=0。其余指令不按运算结果重新设置条件码。<sup>1</sup>

**中断处理。**I/O 设备能够中断处理器, 机制见 A.3 节。

**优先级。**LC-3 支持八个优先级, PL7 最高, PL0 最低。当前进程的优先级由 PSR[10:8] 指定。

**处理器状态寄存器 (PSR)。**这是一个 16 位寄存器, 保存当前进程的状态信息。其中已有七个位被定义: PSR[15] 指定特权模式, PSR[10:8] 指定优先级, PSR[2:0] 保存条件码, 分别为 N、Z、P。

**管理模式。**LC-3 有管理模式 (有特权) 和用户模式 (无特权) 两种运行模式。中断服务例程和陷阱服务例程, 即系统调用, 均在管理模式下执行。PSR[15]=0 表示管理模式, PSR[15]=1 表示用户模式。

**特权模式异常。**RTI 只能在管理模式下执行; 若在用户模式下尝试执行它, 就产生特权模式异常, 处理过程见 A.3 节。

**访问控制违规 (ACV) 异常。**如果用户模式进程试图访问特权存储器——系统空间内的单元, 或 xFE00 至 xFFFF 范围内的设备寄存器——就会产生 ACV 异常, 处理过程见 A.3 节。

**管理栈。**这是系统空间内的一块区域, 通过管理栈指针 SSP 访问。当 PSR[15]=0 时, 栈指针 R6 就是 SSP; 处理器处于用户模式时, SSP 保存在 Saved SSP 中。

<sup>1</sup>译注: 这里的分类指依据写入数据结果执行 setcc() 的指令。RTI 会恢复整个 PSR, 因此也会恢复其中的条件码。本书第三版的 LEA 不设置条件码, TRAP 与 RTI 使用管理栈; 不要与其他版本或模拟器的约定混淆。

**用户栈。**这是用户空间内的一块区域,通过用户栈指针 **USP** 访问。当  $\text{PSR}[15]=1$  时, **R6** 就是 **USP**; 处理器处于管理模式时, **USP** 保存在 **Saved USP** 中。

## A.2 指令集

**LC-3** 的指令集精简而实用。每条 16 位指令包含位 [15:12] 中的操作码, 以及指定其他执行信息的 12 位。图 A.2 汇总了 15 种已定义操作码和各条指令其余字段的格式; 第 16 种四位操作码保留给以后使用。

后面逐条说明指令。表 A.2 解释其中使用的记号。对每条指令, 给出汇编格式、16 位编码、操作描述、中文说明和示例; 必要时还有补充说明。

|                  | 15   | 14 | 13 | 12 | 11           | 10         | 9 | 8         | 7 | 6 | 5       | 4    | 3 | 2   | 1 | 0 |
|------------------|------|----|----|----|--------------|------------|---|-----------|---|---|---------|------|---|-----|---|---|
| ADD <sup>+</sup> | 0001 |    |    |    | DR           |            |   | SR1       |   |   | 0       | 00   |   | SR2 |   |   |
| ADD <sup>+</sup> | 0001 |    |    |    | DR           |            |   | SR1       |   |   | 1       | imm5 |   |     |   |   |
| AND <sup>+</sup> | 0101 |    |    |    | DR           |            |   | SR1       |   |   | 0       | 00   |   | SR2 |   |   |
| AND <sup>+</sup> | 0101 |    |    |    | DR           |            |   | SR1       |   |   | 1       | imm5 |   |     |   |   |
| BR               | 0000 |    |    |    | n            | z          | p | PCoffset9 |   |   |         |      |   |     |   |   |
| JMP              | 1100 |    |    |    | 000          |            |   | BaseR     |   |   | 000000  |      |   |     |   |   |
| JSR              | 0100 |    |    |    | 1            | PCoffset11 |   |           |   |   |         |      |   |     |   |   |
| JSRR             | 0100 |    |    |    | 0            | 00         |   | BaseR     |   |   | 000000  |      |   |     |   |   |
| LD <sup>+</sup>  | 0010 |    |    |    | DR           |            |   | PCoffset9 |   |   |         |      |   |     |   |   |
| LDI <sup>+</sup> | 1010 |    |    |    | DR           |            |   | PCoffset9 |   |   |         |      |   |     |   |   |
| LDR <sup>+</sup> | 0110 |    |    |    | DR           |            |   | BaseR     |   |   | offset6 |      |   |     |   |   |
| LEA              | 1110 |    |    |    | DR           |            |   | PCoffset9 |   |   |         |      |   |     |   |   |
| NOT <sup>+</sup> | 1001 |    |    |    | DR           |            |   | SR        |   |   | 111111  |      |   |     |   |   |
| RET              | 1100 |    |    |    | 000          |            |   | 111       |   |   | 000000  |      |   |     |   |   |
| RTI              | 1000 |    |    |    | 000000000000 |            |   |           |   |   |         |      |   |     |   |   |
| ST               | 0011 |    |    |    | SR           |            |   | PCoffset9 |   |   |         |      |   |     |   |   |
| STI              | 1011 |    |    |    | SR           |            |   | PCoffset9 |   |   |         |      |   |     |   |   |
| STR              | 0111 |    |    |    | SR           |            |   | BaseR     |   |   | offset6 |      |   |     |   |   |
| TRAP             | 1111 |    |    |    | 0000         |            |   | trapvect8 |   |   |         |      |   |     |   |   |
| reserved         | 1101 |    |    |    |              |            |   |           |   |   |         |      |   |     |   |   |

图 A.2 完整 LC-3 指令集的格式；“+”表示依据结果修改条件码的指令。

表 A.2 记号约定

|         |                 |
|---------|-----------------|
| 记号      | 含义              |
| xNumber | 十六进制数，例如 xF2A1。 |
| #Number | 十进制数，例如 #793。   |

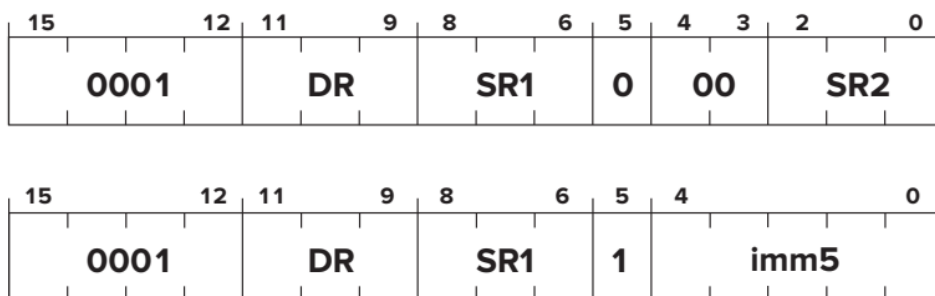
| 记号           | 含义                                                                                                       |
|--------------|----------------------------------------------------------------------------------------------------------|
| bNumber      | 二进制数, 例如 b10011。                                                                                         |
| A[l:r]       | 数据 A 中从左侧位 l 到右侧位 r 的字段。例如 PC 为 0011001100111111 时, PC[15:9] 为 0011001, PC[2:2] 为 1。l 与 r 相同时通常写作 PC[2]。 |
| BaseR        | 基址寄存器, R0 至 R7 之一, 由指令位 [8:6] 指定; 与六位偏移量一起计算 LDR、STR 的基址加偏移地址, 或单独指定 JMP、JSRR 的目标地址。                     |
| DR           | 目的寄存器, R0 至 R7 之一, 用于接收结果。                                                                               |
| imm5         | 指令位 [4:0] 中的五位立即数, 按五位二进制补码整数解释, 使用前符号扩展到 16 位, 范围为 -16 至 15。                                            |
| INTV         | 随中断事件提供的八位值, 用于确定中断服务例程入口。它作为相对中断向量表起始地址的偏移; 对应表项保存服务例程的起始地址。范围为 0 至 255。                                |
| LABEL        | 汇编语言中用名称而非 16 位地址标识某个存储单元的符号。                                                                            |
| mem[address] | 给定地址处的内存内容。                                                                                              |
| offset6      | 位 [5:0] 中的六位有符号二进制补码整数, 用于基址加偏移寻址。先符号扩展到 16 位, 再与基址寄存器内容相加, 范围为 -32 至 31。                                |
| PC           | 程序计数器, 保存下一条待取指令地址的 16 位寄存器。例如, 地址 A 处的指令若不是控制指令, 在其执行阶段 PC 为 A+1, 表示下一条指令位于 A+1。                        |
| PCOffset9    | 位 [8:0] 中的九位有符号二进制补码整数, 用于 PC 加偏移寻址。先符号扩展到 16 位, 再加到已递增的 PC 上, 范围为 -256 至 255。                           |
| PCOffset11   | 位 [10:0] 中的十一位有符号二进制补码整数, 用于 JSR。先符号扩展到 16 位, 再加到已递增的 PC 上, 得到子程序入口, 范围为 -1024 至 1023。                   |
| PSR          | 处理器状态寄存器, 16 位, 保存当前进程状态。已有七个位被指定用途: 位 15 为特权模式, 位 [10:8] 为优先级, 位 [2:0] 分别为 N、Z、P。                       |
| Saved SSP    | 保存的管理栈指针。用户模式下 R6 是 USP, SSP 放在此寄存器中; 从用户模式切换到管理模式时, 把 R6 保存到 Saved USP, 再把 Saved SSP 装入 R6。             |
| Saved USP    | 保存的用户栈指针。管理模式, USP 保存在此寄存器中, 另见 Saved SSP。                                                               |
| setcc()      | 根据写入 DR 的结果设置 N、Z、P 条件码。                                                                                 |
| SEXT(A)      | 对 A 作符号扩展, 复制最高位直至扩展为 16 位。例如, A 为 110000 时, 结果为 1111 1111 1111 0000。                                    |
| SP           | 当前栈指针, 即 R6。两种特权模式各有一个栈: PSR[15]=0 时 SP 是 SSP, PSR[15]=1 时 SP 是 USP。                                     |
| SR, SR1, SR2 | 源寄存器, R0 至 R7 之一, 用于提供源操作数。                                                                              |
| SSP          | 管理栈指针。                                                                                                   |
| trapvect8    | 指令位 [7:0] 中的八位值, 用于 TRAP。按无符号数零扩展到 16 位后, 得到保存对应陷阱服务例程入口地址的内存单元地址, 范围为 0 至 255。                          |
| USP          | 用户栈指针。                                                                                                   |
| ZEXT(A)      | 在 A 左侧补零, 将其扩展为 16 位。例如, A 为 110000 时, 结果为 0000 0000 0011 0000。                                          |

## ADD: 加法

### 汇编格式

```
ADD DR, SR1, SR2
ADD DR, SR1, imm5
```

### 编码



### 操作

```
if (bit[5] == 0)
 DR = SR1 + SR2;
else
 DR = SR1 + SEXT(imm5);
setcc();
```

### 说明

若位 5 为 0, 第二个源操作数取自 SR2; 若为 1, 则把 imm5 符号扩展到 16 位, 作为第二个源操作数。两种情况下都与 SR1 的内容相加, 并把结果写入 DR。根据结果为负、零或正设置条件码。

### 示例

```
ADD R2, R3, R4 ; R2 ← R3 + R4
ADD R2, R3, #7 ; R2 ← R3 + 7
```

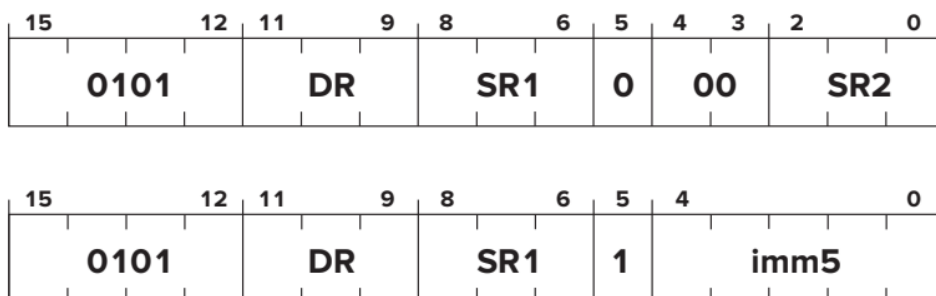


## AND: 按位逻辑与

### 汇编格式

```
AND DR, SR1, SR2
AND DR, SR1, imm5
```

### 编码



### 操作

```
if (bit[5] == 0)
 DR = SR1 AND SR2;
else
 DR = SR1 AND SEXT(imm5);
setcc();
```

### 说明

若位 5 为 0，第二个源操作数取自 SR2；若为 1，则取符号扩展到 16 位的 imm5。将它与 SR1 的内容逐位作 AND，结果写入 DR，并根据结果作为二进制补码整数时为负、零或正设置条件码。

### 示例

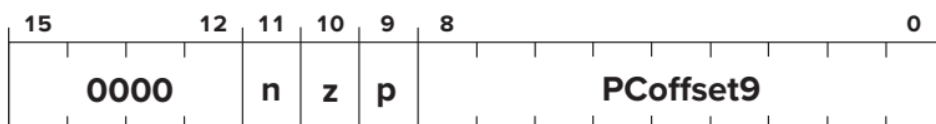
```
AND R2, R3, R4 ;R2 ← R3 AND R4
AND R2, R3, #7 ;R2 ← R3 AND 7
```

## BR: 条件分支

### 汇编格式

```
BRn LABEL BRzp LABEL
BRz LABEL BRnp LABEL
BRp LABEL BRnz LABEL
BR LABEL BRnzp LABEL
```

### 编码



### 操作

```
if ((n AND N) OR (z AND Z) OR (p AND P))
 PC = PC + SEXT(PCoffset9);
```

### 说明

测试位 [11:9] 指定的条件码。位 11 为 1 时测试 N，否则不测试 N；位 10 控制是否测试 Z，位 9 控制是否测试 P。只要被测试的条件码中有一个为 1，就把符号扩展后的 PCoffset9 加到已递增的 PC 上，转到所得地址。

### 示例

```
BRzp LOOP ; Branch to LOOP if the last result was zero or
 ↪ positive.
BR NEXT ; Unconditionally branch to NEXT.
```

### 补充说明

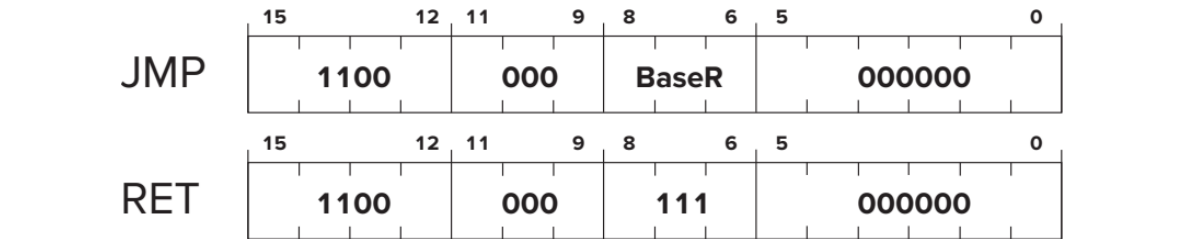
汇编操作码 BR 与 BRnzp 同义，即无条件转到目标地址。这里使用的是已递增的 PC。

JMP / RET：跳转与子程序返回

汇编格式

JMP BaseR  
RET

编码



操作

PC = BaseR;

说明

无条件跳到基址寄存器内容指定的位置；位 [8:6] 选择基址寄存器。

示例

JMP R2 ; PC ← R2  
RET ; PC ← R7

补充说明

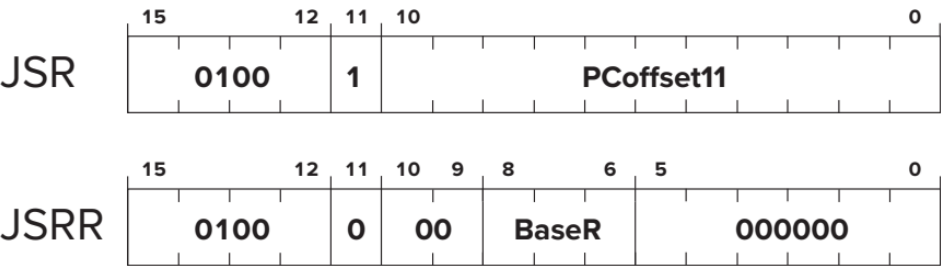
RET 是 JMP 的特殊情形，通常用于从子程序返回：把 R7 的内容装入 PC。R7 保存调用指令之后那条指令的地址，构成返回调用者的链接。

JSR / JSRR：跳转到子程序

汇编格式

JSR LABEL  
JSRR BaseR

编码



操作

```
TEMP = PC;
if (bit[11] == 0)
 PC = BaseR;
else
 PC = PC + SEXT(PCoffset11);
R7 = TEMP;
```

说明

先把已递增的 PC 保存到临时位置，再把子程序第一条指令的地址装入 PC，当前指令完成后即无条件转到那里。位 11 为 0 时，从基址寄存器取得入口；位 11 为 1 时，将位 [10:0] 符号扩展后加到已递增的 PC 上，得到入口。最后把临时保存的值装入 R7，作为返回调用者的链接。

示例

```
JSR QUEUE ; Put the address of the instruction following JSR into
↪ R7;
; Jump to QUEUE.
JSRR R3 ; Put the address of the instruction following JSRR into
↪ R7;
; Jump to the address contained in R3.
```

补充说明

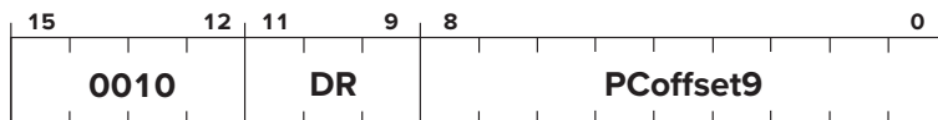
操作中的 PC 是已递增的 PC。

## LD: 加载

### 汇编格式

```
LD DR, LABEL
```

### 编码



### 操作

```
if (computed address is in privileged memory AND PSR[15] == 1)
 Initiate ACV exception;
else
 DR = mem[PC + SEXT(PCoffset9)];
 setcc();
```

### 说明

将位 [8:0] 符号扩展到 16 位，再添加到已递增的 PC 上，得到地址。若它属于特权存储器且 PSR[15]=1，则启动 ACV 异常；否则把该地址处的内存内容装入 DR，并依据载入值为负、零或正设置条件码。

### 示例

```
LD R4, VALUE ; R4 <- mem[VALUE]
```

### 补充说明

操作中的 PC 是已递增的 PC。

## LDI: 间接加载

### 汇编格式

```
LDI DR, LABEL
```

### 编码



### 操作

```
if (either computed address is in privileged memory AND PSR[15]
 ↪ == 1)
 Initiate ACV exception;
else
 DR = mem[mem[PC + SEXT(PCoffset9)]];
 setcc();
```

### 说明

将位 [8:0] 符号扩展后加到已递增的 PC 上，先得到一个地址。该单元保存的内容才是待加载数据的地址。用户模式下，只要两次访问的任一地址属于特权存储器，就启动 ACV 异常；否则加载数据，并依据其符号设置条件码。

### 示例

```
LDI R4, ONEMORE ; R4 <- mem[mem[ONEMORE]]
```

### 补充说明

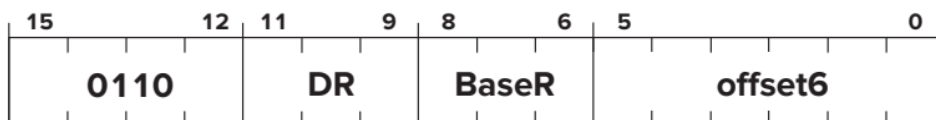
操作中的 PC 是已递增的 PC。

## LDR: 基址加偏移加载

### 汇编格式

```
LDR DR, BaseR, offset6
```

### 编码



### 操作

```
If (computed address is in privileged memory AND PSR[15] == 1)
 Initiate ACV exception;
else
 DR = mem[BaseR + SEXT(offset6)];
 setcc();
```

### 说明

将位 [5:0] 符号扩展到 16 位，再加到位 [8:6] 指定的寄存器内容上。若结果地址属于特权存储器且 PSR[15]=1，启动 ACV 异常；否则把该地址处的内容装入 DR，并依据载入值为负、零或正设置条件码。

### 示例

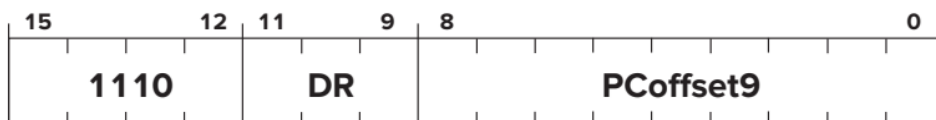
```
LDR R4, R2, #-5 ; R4 <- mem[R2 - 5]
```

## LEA：加载有效地址

## 汇编格式

LEA DR, LABEL

## 编码



## 操作

$$DR = PC + SEXT(PCoffset9);$$

## 说明

将位 [8:0] 符号扩展到 16 位，再加到已递增的 PC 上，把所得地址装入 DR。

## 示例

```
LEA R4, TARGET ; R4 <- address of TARGET.
```

## 补充说明

LEA 计算地址，但不读取该地址处的内存；装入 DR 的是地址本身。操作中的 PC 是已递增的 PC。



**NOT: 按位取反****汇编格式**

```
NOT DR, SR
```

**编码****操作**

```
DR = NOT(SR);
setcc();
```

**说明**

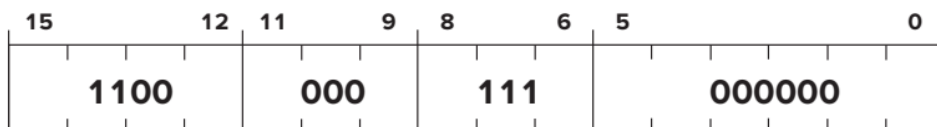
对 **SR** 的内容逐位取反，写入 **DR**。依据结果作为二进制补码整数时为负、零或正设置条件码。

**示例**

```
NOT R4, R2 ; R4 ← NOT(R2)
```

**RET：从子程序返回****汇编格式**

RET

**编码****操作**

PC = R7;

**说明**

将 R7 的内容装入 PC，通常用于返回先前执行 JSR 或 JSRR 的调用者。

**示例**

RET ; PC &lt;- R7

**补充说明**

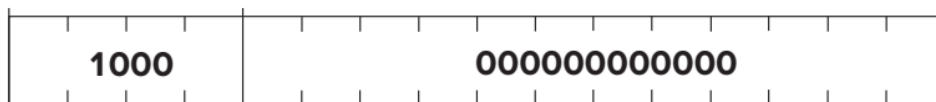
RET 是 JMP 的一个特定编码，另见 JMP。

## RTI: 从陷阱或中断返回

### 汇编格式

RTI

### 编码



### 操作

```

if (PSR[15] == 1)
 Initiate a privilege mode exception;
else
 PC = mem[R6]; R6 is the SSP, PC is restored
 R6 = R6+1;
 TEMP = mem[R6];
 R6 = R6+1; system stack completes POP before saved PSR is
 ↪ restored
 PSR = TEMP; PSR is restored
 if (PSR[15] == 1)
 Saved SSP=R6 and R6=Saved USP;

```

### 说明

如果处理器处于用户模式，产生特权模式异常；若处于管理模式，就从管理栈弹出栈顶两个元素，分别装入 PC 和 PSR。PSR 恢复后，若将返回用户模式，则把 SSP 保存到 Saved SSP，并将 Saved USP 装入 R6。

### 示例

```
RTI ; PC, PSR <- top two values popped off stack.
```

### 补充说明

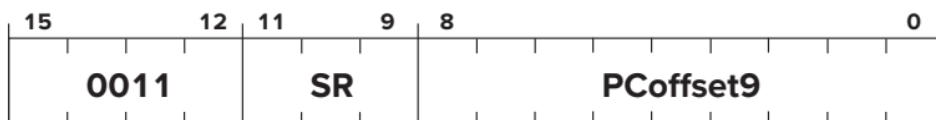
RTI 是中断和陷阱服务例程的最后一条指令，把控制权交回原程序。启动服务例程时，先把原程序的 PSR 和 PC 压入管理栈，再把服务入口装入 PC，例程以管理特权执行。返回时弹出这两个值，恢复 PC 和 PSR。对于中断，PC 恢复为中断发生时尚待执行的指令地址；对于异常，恢复为引发异常的指令或其下一条指令，取决于是否需要重新执行；对于陷阱，恢复为调用程序中 TRAP 之后的指令地址。中断和陷阱通常恢复原来的 PSR；异常则可能恢复原值，也可能使用经处理修改的值，取决于异常性质。另见 A.3 节。

## ST: 存储

### 汇编格式

```
ST SR, LABEL
```

### 编码



### 操作

```
if (computed address is in privileged memory AND PSR[15] == 1)
 Initiate ACV exception;
else
 mem[PC + SEXT(PCoffset9)] = SR;
```

### 说明

若计算出的地址属于特权存储器且  $PSR[15]=1$ ，启动 ACV 异常。否则，将 SR 的内容写入由已递增的 PC 加上符号扩展后的位 [8:0] 所得到的地址。

### 示例

```
ST R4, HERE ; mem[HERE] <- R4
```

### 补充说明

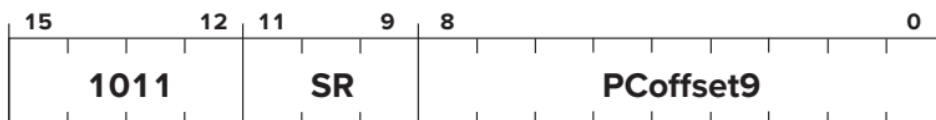
操作中的 PC 是已递增的 PC。

## STI: 间接存储

### 汇编格式

```
STI SR, LABEL
```

### 编码



### 操作

```
if (either computed address is in privileged memory AND PSR[15]
 ↪ == 1)
 Initiate ACV exception;
else
 mem[mem[PC + SEXT(PCoffset9)]] = SR;
```

### 说明

若两次内存访问的任一地址属于特权存储器且  $PSR[15]=1$ ，启动 ACV 异常。否则，先把位 [8:0] 符号扩展并加到已递增的 PC 上；该地址处的内容给出最终存储地址，把 SR 的内容写入那个单元。

### 示例

```
STI R4, NOT_HERE ; mem[mem[NOT_HERE]] <- R4
```

### 补充说明

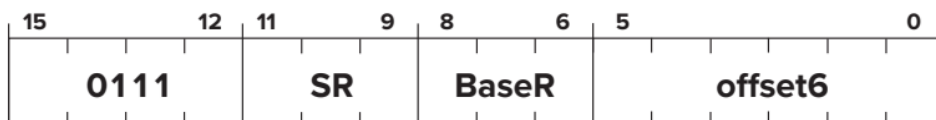
操作中的 PC 是已递增的 PC。

## STR: 基址加偏移存储

### 汇编格式

STR SR, BaseR, offset6

### 编码



### 操作

```
if (computed address is in privileged memory AND PSR[15] == 1)
 Initiate ACV exception;
else
 mem[BaseR + SEXT(offset6)] = SR;
```

### 说明

若计算地址属于特权存储器且 PSR[15]=1, 启动 ACV 异常。否则, 将 SR 的内容写入由符号扩展后的位 [5:0] 加上位 [8:6] 指定寄存器的内容所得到的地址。

### 示例

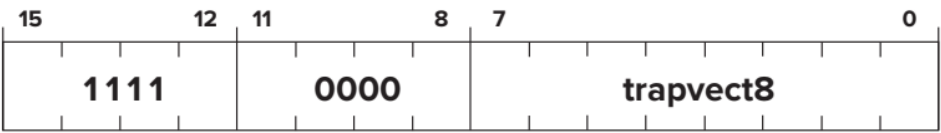
```
STR R4, R2, #5 ; mem[R2+5] <- R4
```

TRAP：系统调用

汇编格式

TRAP trapvector8

编码



操作

```
TEMP=PSR;
if (PSR[15] == 1)
 Saved USP=R6 and R6=Saved SSP;
PSR[15]=0;
Push TEMP,PC on the system stack
PC = mem[ZEXT(trapvect8)];
```

说明

若原程序处于用户模式，先保存用户栈指针，并装入管理栈指针。然后把原 PSR 和已递增的 PC 压入管理栈，使服务例程最后执行 RTI 时能够返回原程序中 TRAP 之后的指令。把 trapvector8 零扩展为 16 位，以此访问陷阱向量表，将该单元保存的系统调用入口地址装入 PC。

示例

```
TRAP x23 ; Directs the operating system to execute the IN system
↪ call.
; The starting address of this system call is contained in
; memory location x0023.
```

补充说明

x0000 至 x00FF 共 256 个单元构成陷阱向量表，保存相应系统调用的入口地址。表 A.3 说明了向量 x20 至 x25 对应的服务。操作中的 PC 是已递增的 PC。

未使用的操作码

汇编格式

无。

编码



操作

Initiate an illegal opcode exception.

说明

遇到非法操作码时，产生非法操作码异常。

补充说明

操作码 1101 尚未定义，保留给以后使用。若当前指令位 [15:12] 为 1101，就产生非法操作码异常，处理过程见 A.3 节。



表 A.3 陷阱服务例程

| 陷阱向量 | 汇编名称  | 说明                                                                                                                             |
|------|-------|--------------------------------------------------------------------------------------------------------------------------------|
| x20  | GETC  | 从键盘读取一个字符，不在控制台回显。其 ASCII 码复制到 R0，高八位清零。                                                                                       |
| x21  | OUT   | 将 R0[7:0] 中的字符输出到控制台显示器。                                                                                                       |
| x22  | PUTS  | 向控制台输出 ASCII 字符串。字符从 R0 指定的地址开始存放在连续内存单元中，每单元一个字符；遇到内容为 x0000 的单元时停止。                                                          |
| x23  | IN    | 显示提示并从键盘读入一个字符，同时在控制台回显；将其 ASCII 码复制到 R0，高八位清零。                                                                                |
| x24  | PUTSP | 向控制台输出 ASCII 字符串。字符从 R0 指定的地址开始连续存储，每个单元存放两个字符，先输出位 [7:0]，再输出位 [15:8]。若字符总数为奇数，最后一个字符所在单元的高八位为 x00，不作为字符输出。遇到内容为 x0000 的单元时停止。 |
| x25  | HALT  | 停止执行，并在控制台输出一条消息。                                                                                                              |

A.3 中断与异常处理

第 9 章已经详细讨论：正在运行的程序之外发生的事件可以中断处理器，中断驱动 I/O 就是常见例子。程序本身在运行中引起的异常事件，也可以打断正常执行。例如，程序中出现尚未使用的操作码，就是一种“内部”事件。

每个能够打断处理器的事件，都关联一个八位向量，用于选择中断向量表的某个表项。这张表有 256 项，起始地址为 x0100，占据 x0100 至 x01FF。每项保存处理相应事件的服务例程起始地址，服务例程在管理模式下执行。

其中一半，即 x0100 至 x017F 的 128 项，对应运行中的程序自身引发的事件。处理这些事件的例程称为异常服务例程，因为异常事件会妨碍程序正常执行。另一半，即 x0180 至 x01FF，对应程序之外的事件，例如 I/O 设备请求；其处理例程称为中断服务例程。

A.3.1 中断

本书的 LC-3 系统目前只规定了一种能够请求中断的 I/O 设备：键盘。它的中断优先级为 PL4，提供的中断向量为 x80。

I/O 设备能够中断处理器，须同时满足：它需要服务，中断使能位 IE 已置位，其请求优先级高于其他竞争事件，也高于当前程序的优先级。

假设某个优先级低于 4 的程序正在执行，此时有人按下键盘。如果 KBSR 的 IE 位为 1，就在当前指令周期结束时中断该程序，并按以下步骤启动服务例程：

1. 把被中断进程的 PSR 保存到 TEMP。
2. 把特权模式设置为管理模式，即  $\text{PSR}[15]=0$ 。
3. 把优先级设置为中断设备的 PL4，即  $\text{PSR}[10:8]=100$ 。
4. 如果被中断进程原来处于用户模式，把 R6 保存到 Saved USP，再把 SSP 装入 R6。
5. 把 TEMP 和被中断进程的 PC 依次压入管理栈。
6. 键盘提供八位中断向量，本例为  $\text{x}80$ 。
7. 处理器将它扩展为中断向量表内对应的 16 位地址  $\text{x}0180$ 。
8. 把  $\text{x}0180$  中的内容装入 PC，也就是键盘中断服务例程第一条指令的地址。

随后，处理器开始执行中断服务例程。

例程最后执行 RTI，从管理栈弹出栈顶两个元素，分别装入 PC 和 PSR。根据恢复后的  $\text{PSR}[15]$ ，为 R6 选择适当的栈指针，然后从被中断程序原先暂停的位置继续执行。

### A.3.2 异常

本书的 LC-3 ISA 规定三种异常：特权模式违规、非法操作码和访问控制违规 (ACV)。用户模式下尝试执行 RTI 会引发特权模式违规；尝试执行操作码为 1101 的指令会引发非法操作码异常；用户模式下访问特权存储器——系统空间或  $\text{xFE}00$  至  $\text{xFFFF}$  的寄存器地址——会引发 ACV 异常。

异常在被检测到后立即处理，其启动过程与中断很相似：

1. 把引发异常的进程的 PSR 保存到 TEMP。
2. 设置管理模式，即  $\text{PSR}[15]=0$ 。
3. 如果该进程原来处于用户模式，把 R6 保存到 Saved USP，再把 SSP 装入 R6。
4. 把 TEMP 和该进程的 PC 依次压入管理栈。
5. 异常提供八位向量：特权模式违规为  $\text{x}00$ ，非法操作码为  $\text{x}01$ ，ACV 为  $\text{x}02$ 。
6. 处理器把向量扩展为中断向量表内的对应地址  $\text{x}0100$ 、 $\text{x}0101$  或  $\text{x}0102$ 。
7. 将该表项中保存的服务例程入口地址装入 PC。

处理器随后开始执行相应的异常服务例程。具体处理取决于异常类型以及操作系统采用的策略。

在许多情况下，异常服务例程能够纠正问题，然后继续执行原程序。此时，例程最后执行 RTI，从管理栈弹出 PC 和 PSR，在问题修正后恢复执行。如果异常原因足够严重，服务例程也可能终止这个程序，不再让它继续运行。

异常处理与中断处理还有一个区别：服务例程执行期间的优先级。处理异常时，通常不改变优先级。优先级表示程序需要执行的紧迫程度；本书规定的这些异常——特权模式违规、非法操作码或用户模式访问特权存储器——本身不会改变原程序的紧迫程度。

## 附录 B 从 LC-3 到 x86

你已经知道，LC-3 的 ISA 明确规定了一道接口：一边是机器语言程序员或编译器生成的代码，另一边是 LC-3 微体系结构所能接受和处理的内容。它规定内存地址空间与寻址单位、寄存器的数量和大小、指令格式、操作码、表示信息的数据类型，以及确定操作数位置的寻址方式。

PC 中微处理器的 ISA 同样规定编译器与微体系结构之间的接口。本附录讨论采用 x86 的 PC，而不是 LC-3。Intel 于 1978 年推出 x86 家族的第一个成员 8086，它采用 16 位寄存器和数据操作，与 LC-3 有相似之处；分段寻址形成的物理地址则为 20 位。后来，x86 扩展到 64 位，并通过向量扩展支持 128、256 和 512 位向量操作。为了继续运行大量使用 32 位地址和数据的旧程序，x86 同时支持 64 位和 32 位运行环境。<sup>1</sup>

从 8086 开始，Intel 不断推出 x86 ISA 的实现，例如 386 (1985)、486 (1989)、Pentium (1993)、Pentium Pro (1995)、Pentium II (1997)、Pentium 4 (2000)、代号 Nehalem 的第一代 Core i7-9xx (2008)、代号 Haswell 的第四代 Core i7-4xxx (2013)，以及代号 Coffee Lake 的第八代 Core i7-8086K (2018)。<sup>2</sup>

x86 比 LC-3 复杂得多：操作码、数据类型和寻址方式更多，内存结构和指令编码也更复杂。不过，它们仍由同样的基本要素构成。

你已经花了许多时间借助 LC-3 理解计算。有些读者也许希望学习一种实际使用的 ISA。一种办法是让 Intel 之类的公司批量制造 LC-3 处理器，让 Dell 之类的公司把它装入 PC，再让 Microsoft 之类的公司把 Windows NT 编译成 LC-3 指令。显然，更容易的办法就是阅读本附录。

尽管 x86 十分复杂，我们仍选择它作为例子，因为它是一种使用极为广泛的 ISA。本附录不会给出完整规范——那需要单独一本书，也需要比目前更深入的操作系统、编译器和计算机系统知识。若要全面了解，可阅读 *Intel Architecture Software Developer's Manual*。这里仅介绍与应用程序相关的若干特征，并以你已经熟悉的 LC-3 为参照，帮助你体会 x86 的丰富性。

---

<sup>1</sup>译注：原书将 8086 的年份写成 1979，已据 Intel 历史资料修正为 1978。模式名称并不意味着所有操作数都只能采用该宽度；64 位模式仍可使用较小操作数，实际实现的有效线性地址位数也可能少于 64。

<sup>2</sup>译注：原书 Pentium 和 Pentium IV 分别列为 1992、2001；这里按家族首次推出年份修正为 1993、2000。参见 Intel 处理器历史速查表。这份列表保留原书的历史范围，不作为当前产品清单。

## B.1 LC-3 的特征与对应的 x86 特征

### B.1.1 指令集

指令集由若干指令组成，每条指令包含操作码及零个或多个操作数，操作数数量由操作决定。每个操作数都是按其数据类型编码的数据元素，位置则通过寻址方式确定。

LC-3 主要支持一种基本运算数据类型、15 种操作码，以及 PC 相对、间接和基址加偏移这三种访问内存的寻址方式，分别见 LD/ST、LDI/STI 和 LDR/STR。x86 则有十多种数据类型、大量操作码和二十多种寻址形式，具体数量取决于统计方法。<sup>3</sup>

#### B.1.1.1 数据类型

数据类型是信息的一种表示，而 ISA 为这种表示提供相应的操作码。LC-3 支持的基本算术类型只有 16 位二进制补码整数。实际应用的需求更广：科学计算需要浮点数，多媒体需要其他成组数据表示，长期运行的商业程序可能使用压缩十进制；不同应用对数值范围和精度的要求也不同。

因此，x86 提供操作多种数据类型的指令，例如 8、16、32 位整数，32、64 位浮点数，以及 64、128、256、512 位多媒体数据。图 B.1 列出其中一些类型。

---

<sup>3</sup>译注：这里列出的是 LC-3 的内存操作数寻址方式；寄存器和立即数等操作数形式并未因此被排除。

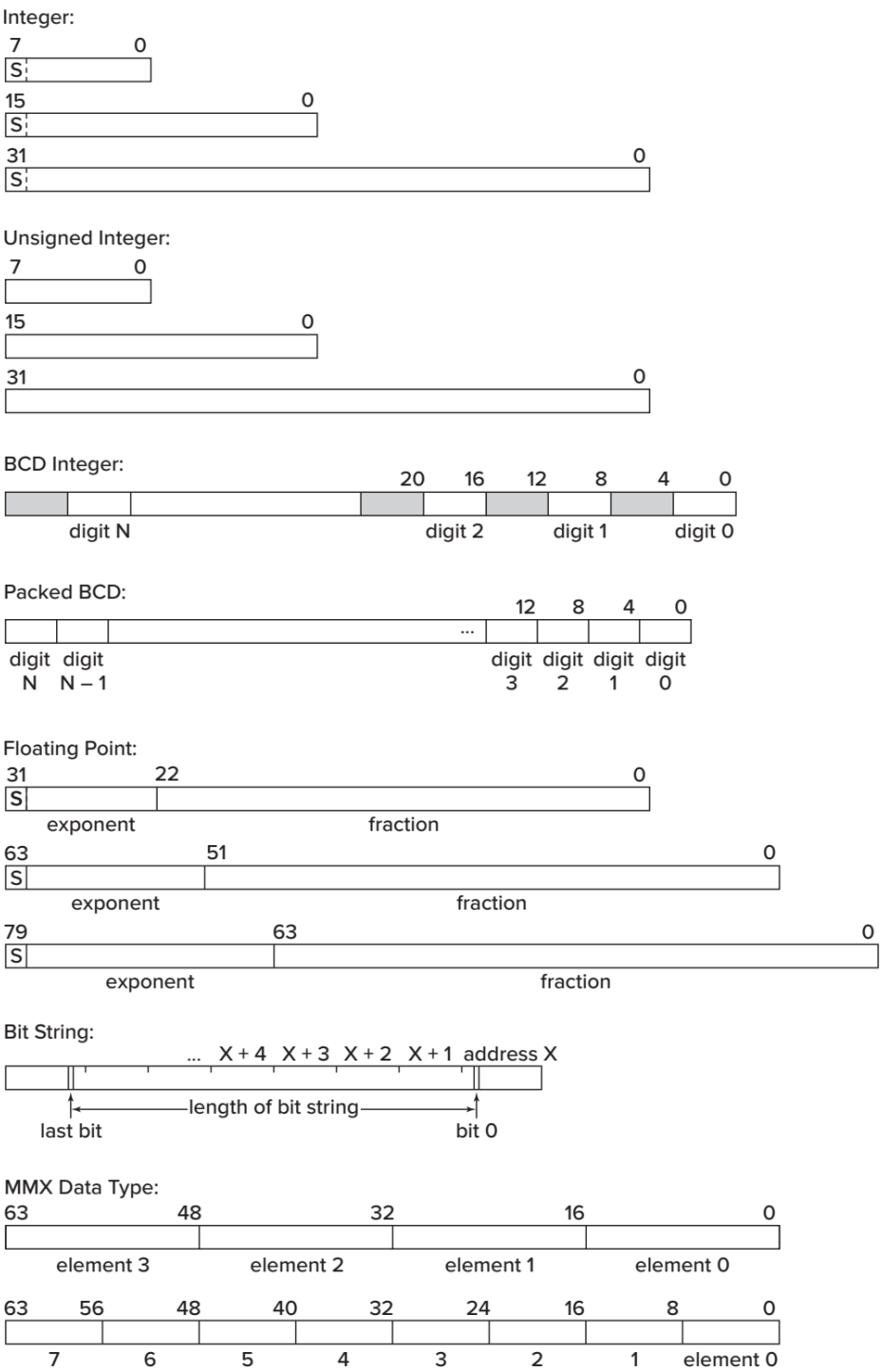


图 B.1 x86 数据类型示例

B.1.1.2 操作码

LC-3 有 15 种操作码，x86 的指令形式则多得多。回想三类基本指令：运算、数据传送和控制。运算指令处理信息；数据传送指令把信息从一处搬到另一处，包括输入输出；控制指令改变指令流。

实际机器还需要第四类功能，因为用户程序是在操作系统管理的系统中执行，而

不是孤立运行的。这些指令涉及安全、系统管理、硬件性能监测等，超出一般应用程序的关注范围。本附录暂不介绍它们，但你应知道它们存在，后续学习会逐渐接触。这里集中讨论前三类。

**运算。**LC-3 的运算指令是 ADD、AND 和 NOT。其中只有 ADD 进行算术运算：减法需要先取负再相加，乘法可以用循环反复相加实现，但对实际微处理器来说，这往往太耗时。x86 因此提供 SUB、MUL、DIV、INC、DEC、ADC 等指令，分别用于减、乘、除、递增、递减和带进位加法。

扩展可处理整数的长度是一项有用能力。软件可以把长整数拆成若干部分，逐段运算。ADC 将两个操作数和先前运算保留的进位一起相加，特别适合这种多精度运算。

x86 还为不同数据类型提供不同指令。例如，多媒体 MMX 指令中经常使用**饱和运算**。以图像灰度为例，假定用数字 0 至 9 表示从白到黑的程度。原灰度为 7，再增加 5。如果只保留普通加法的个位，结果是 2 并产生进位，反而比原先更亮，这就不合适。饱和加法达到上限 9 后停在那里，所以  $7 + 5 = 9$ ，对于非负增量  $n$ ， $9 + n = 9$ 。x86 提供对压缩数据执行这种运算的指令，原书以 PADDS 表示这一类饱和加法。

科学计算需要浮点操作，例如 FADD、FMUL、FSIN 和 FSQRT。LC-3 只有 AND 和 NOT 两种逻辑操作，虽然足以构造任意逻辑表达式，直接实现仍可能费时。x86 还提供 OR、XOR、AND-NOT，以及面向不同类型的逻辑操作。此外还有置位、清零、类型转换、移位和循环移位等。表 B.1 给出部分例子。

**表 B.1** x86 ISA 的运算指令

| 指令           | 说明                                                                  |
|--------------|---------------------------------------------------------------------|
| ADC $x, y$   | 将 $x$ 、 $y$ 和 CF 中保留的进位相加，结果写入 $x$ 。                                |
| MUL $x$      | 在这里的 32 位操作数形式中，将 EAX 与 $x$ 相乘，把 64 位结果存入 EDX:EAX。                  |
| SAR $x$      | 将 $x$ 算术右移 $n$ 位，结果仍存入 $x$ ； $n$ 可以是 1、立即数或 CL 中的计数。                |
| XOR $x, y$   | 对 $x$ 、 $y$ 作按位异或，结果存入 $x$ 。                                        |
| DAA          | 压缩十进制数相加后，AL 内两个 BCD 数位可能因二进制在 15 后而非 9 后进位而不正确；本指令调整这两个 BCD 数位。    |
| FSIN         | 计算浮点寄存器栈顶 ST(0) 的正弦，并用结果替换 ST(0)。                                   |
| FADD         | 执行浮点加法。原表描述的“栈顶两项相加并使栈深减一”对应弹栈形式 FADDP；普通 FADD 不作该弹栈。               |
| PANDN $x, y$ | 对 MMX 数据执行按位 AND-NOT，结果为 $(\text{NOT } x) \text{ AND } y$ ，写入 $x$ 。 |
| PADDS $x, y$ | 对压缩 MMX 元素作有符号饱和加法，结果写入 $x$ 。                                       |

译注：原表将 FSIN 说成先弹出再压入，实际它直接替换栈顶值；FADD 与 FADDP 的弹栈行为也须区别。SAR 的完整语法需要移位计数。PADDS 是原书的概括写法，实际

助记符还指定元素宽度，例如 *PADD SB*、*PADD SW*。*DAA* 等部分传统指令在 64 位模式下不可用。

**数据移动。**LC-3 的 LD、LDI、LDR、ST、STI、STR 在内存或内存映射设备寄存器与 R0–R7 之间复制数据。x86 没有直接对应 LC-3 双重间接访问的单条 LDI/STI，但有对应其他加载、存储功能的指令，还有更多操作。例如 XCHG 交换两个位置的值；PUSHA 保存一组通用寄存器；IN/OUT 在 I/O 端口与处理器之间移动数据；CMOVcc 仅在条件成立时复制数据。表 B.2 列出一些例子。

**表 B.2** x86 ISA 的数据传送指令

| 指令                        | 说明                                                        |
|---------------------------|-----------------------------------------------------------|
| MOV <i>x</i> , <i>y</i>   | 把 <i>y</i> 中的值复制到 <i>x</i> 。                              |
| XCHG <i>x</i> , <i>y</i>  | 交换 <i>x</i> 和 <i>y</i> 中的值。                               |
| PUSHA                     | 将一组通用寄存器压栈保存。                                             |
| PUSH                      | 把一个寄存器的值压入栈顶。                                             |
| POP                       | 从栈顶弹出一个值到寄存器。                                             |
| MOVS                      | 把 DS:ESI 指向的元素复制到 ES:EDI 指向的位置，再按 DF 所定方向及元素大小调整 ESI、EDI。 |
| REP MOVS                  | 在计数非零时重复 MOVS，每次递减 ECX，直至为零；预先设置计数与地址即可复制字符串。             |
| LODS                      | 将 DS:ESI 指向的元素装入相应宽度的累加器，并按 DF 调整 ESI。                    |
| INS                       | 从 DX 指定的 I/O 端口读入数据，存入 ES:EDI 指定的内存位置，并相应调整 EDI。          |
| CMOVZ <i>x</i> , <i>y</i> | 若 ZF=1，将 <i>y</i> 的值复制到 <i>x</i> ；否则不修改目的值。               |
| LEA <i>x</i> , <i>y</i>   | 把 <i>y</i> 表示的有效地址写入 <i>x</i> ，作用与 LC-3 同名指令相近。           |

译注：本表采用传统 32 位示例。原书将 *INS* 的目的写成累加器，那是 *IN* 的行为，这里已修正；*MOVS* 的方向由 *DF* 决定，不能一律递增。*CMOV* 条件不成立并不保证内存源完全不被访问。*PUSHA* 的 16 位形式与 *PUSHAD* 的 32 位形式须区别，且不适用于 64 位模式。

**控制。**LC-3 的五类控制操作码为 BR、JSR/JSRR、JMP、RTI 和 TRAP。x86 有实现相近功能的指令，也有其他控制指令，见表 B.3。

**表 B.3** x86 ISA 的控制指令

| 指令             | 说明                                   |
|----------------|--------------------------------------|
| Jcond <i>x</i> | 根据 cond 指定的条件判断；若成立，把目标地址装入 IP。      |
| JMP <i>x</i>   | 把目标地址 <i>x</i> 装入 IP，作用与 LC-3 的跳转相近。 |
| CALL <i>x</i>  | 把返回地址压栈，再转到 <i>x</i> 指定的入口。          |



| 指令     | 说明                                                     |
|--------|--------------------------------------------------------|
| RET    | 弹出返回地址并装入 IP。                                          |
| LOOP x | 递减计数器;若不为零则跳转。这里的 32 位地址形式使用 ECX。                      |
| INT n  | n 指定中断描述符表中的入口，处理器由此转到操作系统的相应服务例程；其用途与 LC-3 的 TRAP 相近。 |

译注：原表在 *LOOP* 上附加了 *ZF=1* 的条件，那对应 *LOOPE/LOOPZ*，普通 *LOOP* 不检查 *ZF*。以上是功能层面的对照，不表示 x86 具有所有 LC-3 同名指令或相同编码。

B.1.1.3 二地址与三地址

LC-3 通常称为三地址 ISA，这反映了 ADD 显式指定三个操作数位置：两个源操作数和一个结果位置。即使一个源和目的使用同一寄存器，也都要写出，例如：

```
ADD R1, R1, R2
```

其中 R1 同时是源寄存器和目的寄存器。

x86 的传统整数运算主要采用二地址形式；一些 SSE/AVX 等扩展的形式不同。由于加法仍需两个源和一个结果，二地址形式必须让一个源位置同时接收结果。例如，对应的 16 位 x86 指令为：

```
ADD AX, BX
```

AX 和 BX 都是源，AX 同时是目的。原先 AX 中的源值被覆盖；若后面还需要它，就必须在运算前另行保存。

B.1.1.4 内存操作数

两种 ISA 的一个重要区别，在于运算指令能从哪里取得操作数。LC-3 的寄存器运算从寄存器取得源值，并把结果写回寄存器；立即数形式还可直接使用指令中的常数。x86 的许多运算指令则可以把一个操作数放在内存中，并把结果写回内存，也就是在一条指令中完成读取、运算和写回。

LC-3 必须先用装入指令读取内存，运算后再用存储指令写回。具有这种限制的体系结构称为**加载—存储体系结构**（load-store ISA）；x86 不属于这一类。

B.1.2 内存

LC-3 有  $2^{16}$  个存储单元，每单元 16 位，因此地址空间为 16 位，寻址单位也是 16 位。x86 则按八位字节寻址。一个 32 位数据元素需要连续四个单元 X、X+1、X+2、X+3，整体地址记作 X。x86 采用小端序：X 保存位 [7:0]，X+1 保存位 [15:8]，X+2 保存位 [23:16]，X+3 保存位 [31:24]。

LC-3 根据指令和相应寻址方式计算地址；x86 可供选择的形式更多，B.2 节将进一步讨论。x86 还具有**分段**机制，在本附录所示的保护模式模型中，寻址计算得到的值是段内偏移，需要结合段信息才能访问内存。这也有助于阻止不允许的访问，其保护细节留待后续学习。

图 B.2 对比两种机器的寄存器加偏移寻址，操作都是从内存加载到通用寄存器：LC-3 使用 LDR，x86 使用 MOV。x86 的例子访问 DS 段，通过 DS 中的 16 位选择子取得段描述符，后者包含段基址、段界限和保护信息。段内偏移与段基址相加，得到图中的地址。<sup>4</sup>

---

<sup>4</sup>译注：图 B.2 是 32 位分段模型，所得是线性地址，若启用分页还需进一步转换。64 位模式下的分段规则不同，不能将此图直接推广为所有 x86 访存过程。

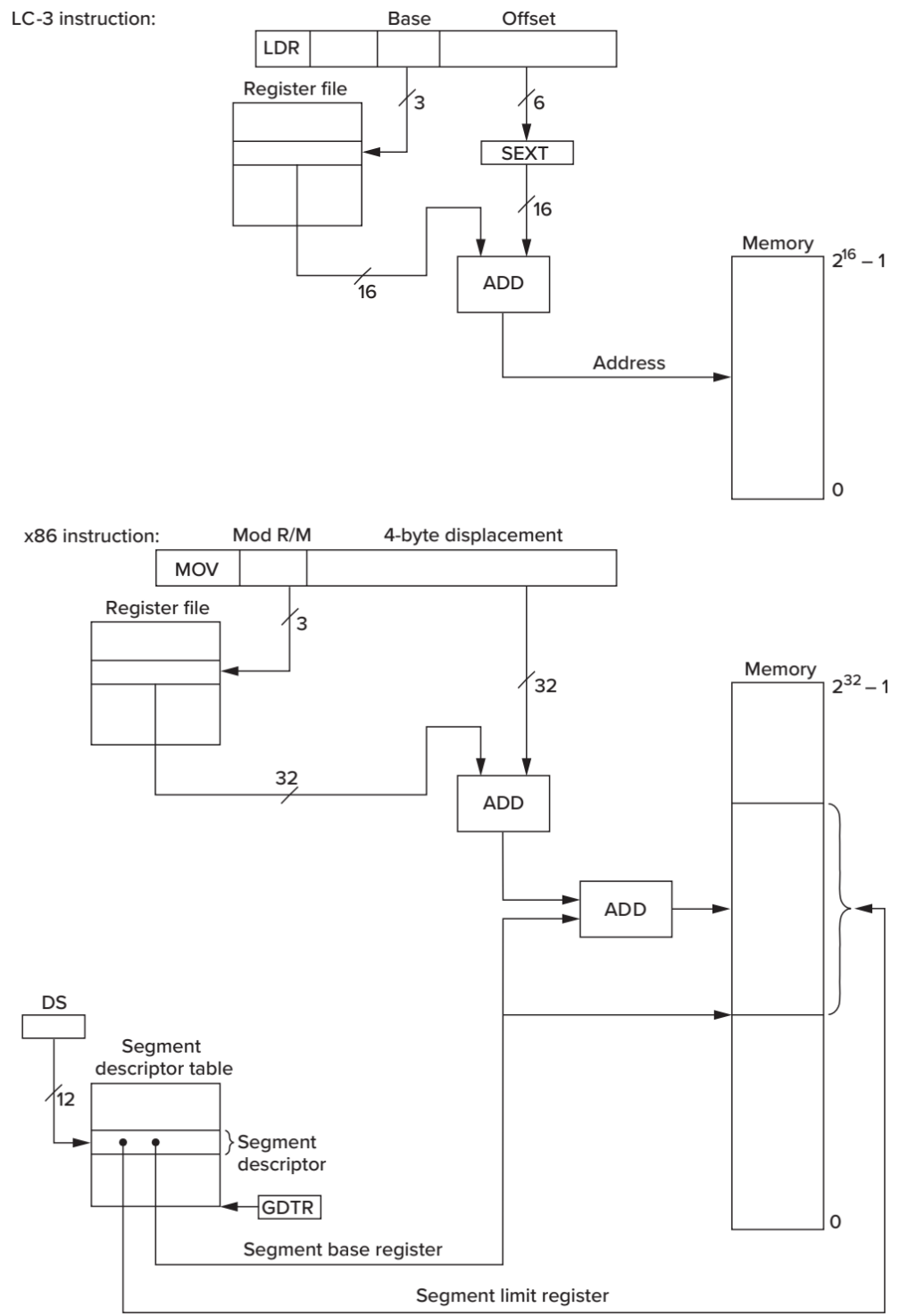


图 B.2 LC-3 与 x86 的寄存器加偏移寻址

B.1.3 内部状态

LC-3 的内部状态包括八个 16 位通用寄存器、16 位 PC，以及保存特权模式、优先级和 N、Z、P 的 16 位 PSR。x86 中与本节相关的可见状态包括应用程序可见寄存器、64 位指令指针 RIP、64 位 RFLAGS，以及 16 位段寄存器。

B.1.3.1 应用程序可见寄存器

图 B.3 列出一部分 x86 应用程序可见寄存器。64 位模式下有十六个通用寄存器：RAX、RBX、RCX、RDX、RSP、RBP、RSI、RDI，以及 R8 至 R15，每个宽 64 位。32 位模式使用 EAX、EBX、ECX、EDX、ESP、EBP、ESI、EDI，对应相关寄存器的低 32 位。<sup>5</sup>

指令还可使用对应低 16 位的 AX、BX 等，以及低八位 AL、BL、CL、DL 和位 [15:8] 对应的 AH、BH、CH、DH。用于 SSE/AVX 等操作的 SIMD 寄存器有 128 位 XMM、256 位 YMM 和 512 位 ZMM；图中展示了支持相应扩展时编号 0–31 的寄存器视图。实际可用数量、宽度和编码受处理器扩展与运行模式约束。

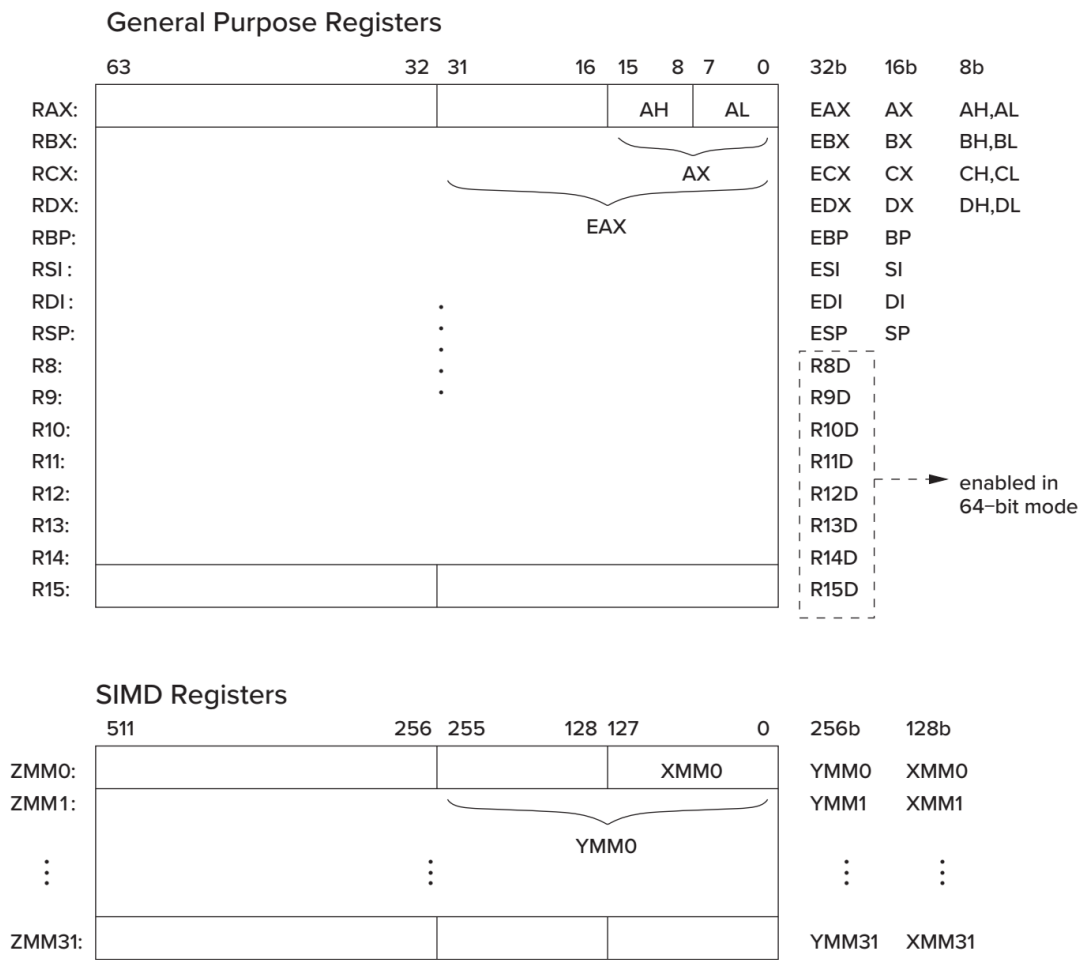


图 B.3 部分 x86 应用程序可见寄存器

B.1.3.2 系统寄存器

LC-3 的 PC 和 PSR 在 x86 中都有相近功能，x86 还提供其他系统相关寄存器。图 B.4 列出其中一些可见部分。

<sup>5</sup>译注：原文将 RSI、ESI 误写为 RCI、ECI，已按图 B.3 修正。

**指令指针。**RIP 相当于 LC-3 的程序计数器；64 位模式下宽 64 位，32 位环境中使用 EIP。

**RFLAGS。**SF（符号标志）和 ZF（零标志）分别对应 LC-3 的 N 和 Z。x86 没有独立对应 P 的正数条件码；知道 N 和 Z，就能推导是否为正。LC-3 仍提供 P，是为了方便汇编程序员和编译器编写者。x86 把这些一位状态及其他标志集中在 RFLAGS 中。

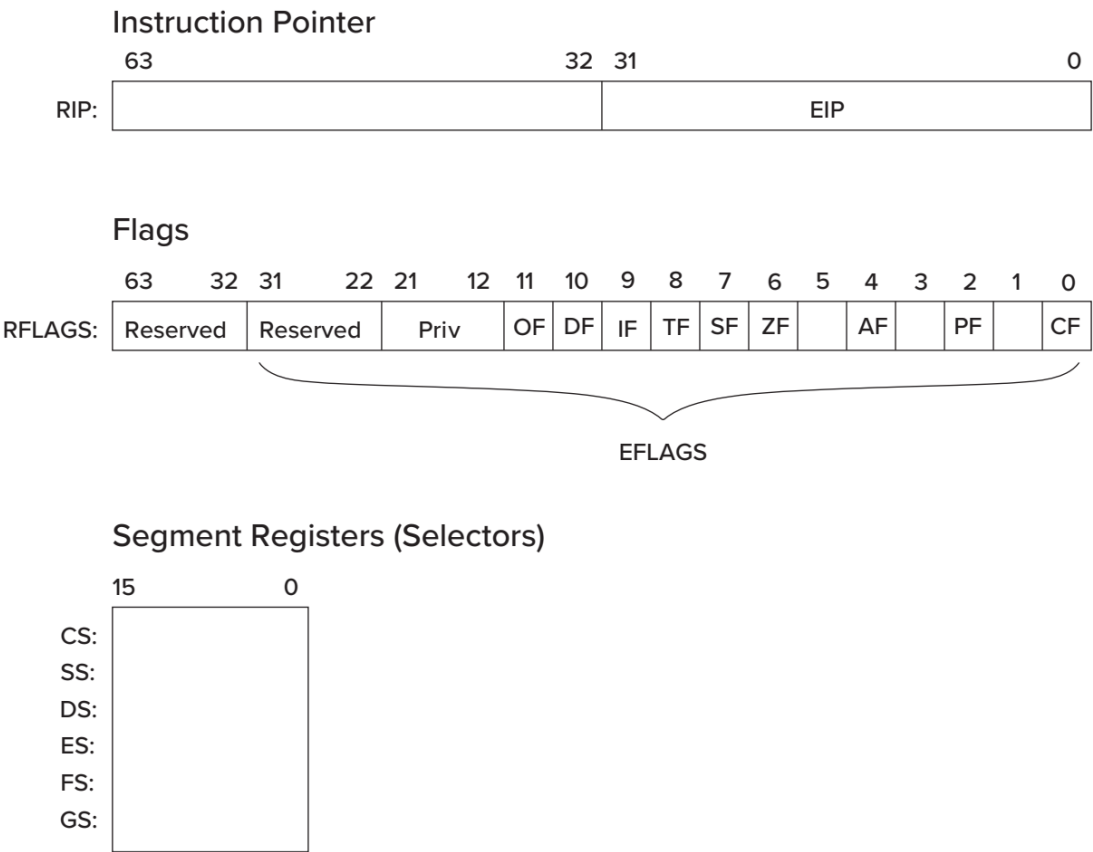


图 B.4 x86 系统相关寄存器

CF 保存相关运算产生的进位，配合 ADC 可以用软件处理长于机器基本整数宽度的数。OF 表示有符号运算结果超出相应位宽所能表示的范围，另见 2.5.3 节关于溢出的讨论。

DF 指定字符串操作的方向。DF=0 时，索引递增，从低地址向高地址处理；DF=1 时，索引递减，从高地址向低地址处理。<sup>6</sup>

IF 和 TF 通常不作为普通应用数据状态处理，但它们对应的功能你已经熟悉。IF 类似中断使能控制：置位时可响应外部可屏蔽中断，清零时禁止这类响应；它并不屏蔽所有种类的事件。TF 用于单步调试，作用类似 LC-3 模拟器的单步模式，但它属于 ISA。启用时，每执行完一条适用指令就引发调试事件，使调试器能够检查状态；清零时不因该单步条件产生事件。<sup>7</sup>

**段寄存器。**在这里讨论的保护模式分段模型中，指令算出的地址是相对段基址的

<sup>6</sup>译注：原书将两种方向写反，已依据 Intel 手册第 1 卷的 DF 说明修正。  
<sup>7</sup>译注：原书说 TF 会让处理器“停止”，这里明确为调试异常及调试器接管，不是执行 HLT。

偏移。段基址等信息保存在段描述符表中。六个活动段分别是代码段 CS、栈段 SS，以及数据段 DS、ES、FS、GS，通过对应段寄存器中的选择子引用。图 B.4 展示这些段寄存器。

B.2 x86 指令的格式与说明

LC-3 指令固定为 16 位，位 [15:12] 总是操作码，其余 12 位满足相应操作的需要。x86 指令长度不固定，一条指令可以包含很多信息。图 B.5 展示传统编码的主要组成部分。<sup>8</sup>

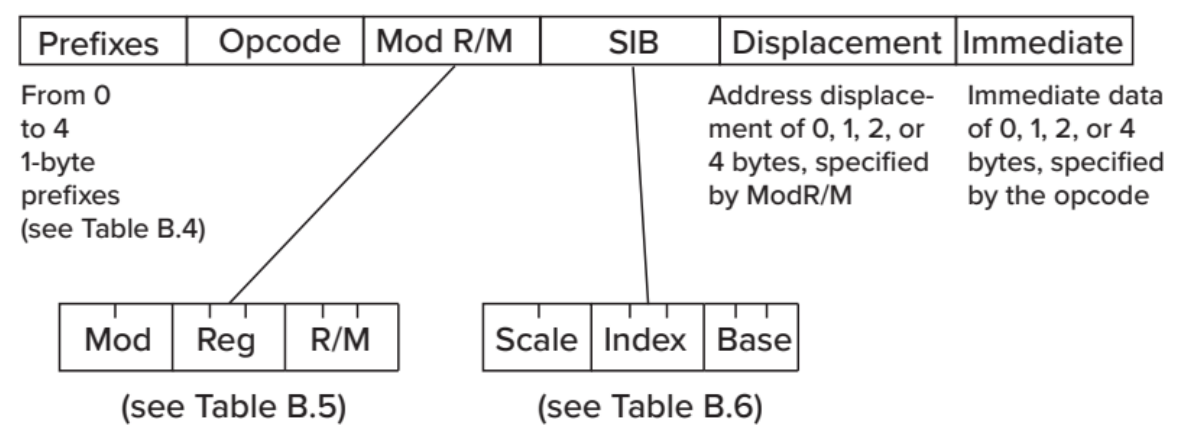


图 B.5 x86 指令格式

关键部分是操作码，以及需要时出现的 ModR/M 字节。前者指定执行何种操作，后者说明如何取得操作数。某些寻址方式还需要寄存器、1、2 或 4 字节位移，以及可选 SIB 字节内的附加寄存器信息。若操作码规定使用立即数，它也说明用于保存立即数的字节数；立即数通常位于指令末尾。

指令对地址大小、操作数大小、所用段等具有默认约定，开头的一个或多个前缀可以改变这些约定。下面分别讨论各部分。

B.2.1 前缀

前缀为处理指令提供附加信息，覆盖其通常的解释。本节的前缀分为四组：锁定与重复、段覆盖、操作数大小覆盖、地址大小覆盖。图中按每组选取一个前缀来示意，共零至四个。表 B.4 说明各组用途。

<sup>8</sup>译注：原书写成 1–16 字节，应为最多 15 字节。图 B.5 主要展示传统编码，并非包含 REX、VEX、EVEX 等所有扩展形式的完整规范。

表 B.4 x86 ISA 的传统前缀

| 前缀                                                           | 说明                                                    |
|--------------------------------------------------------------|-------------------------------------------------------|
| xF0 (LOCK)                                                   | 对允许使用它的内存读—改—写操作提供原子性保证。                              |
| xF2, xF3<br>(REP/REPE/REPNE)                                 | 重复字符串操作，次数由计数器指定；某些比较或扫描形式还依据 ZF 提前停止。                |
| x2E (CS)、x36 (SS)、x3E<br>(DS)、x26 (ES)、x64 (FS)、<br>x65 (GS) | 用指定段代替指令默认的段。                                         |
| x66                                                          | 操作数大小覆盖。在这里的 16/32 位模型中，使原本使用 32 位数据的指令改用 16 位，或反向切换。 |
| x67                                                          | 地址大小覆盖。在这里的 16/32 位模型中，使 32 位地址与 16 位地址相互切换。          |

译注：原书将 *LOCK* 描述为独占全部共享内存，范围过宽；它保障指定操作的原子性。重复前缀是否检查 *ZF* 取决于具体指令，*REP MOVS* 不检查 *ZF*。64 位模式中的大小覆盖规则须另行区分。

### B.2.2 操作码

操作码字节可以不止一个字节，其中包含执行操作、操作数来自寄存器还是内存、操作数大小、是否有立即数以及立即数大小等信息。如果不需要全部 ModR/M 位指定寻址信息，位 [5:3] 还可以与操作码字节组合，进一步区分操作。ModR/M 字节将在下一小节介绍。

### B.2.3 ModR/M 字节

ModR/M 字节可以为一个或两个操作数提供寻址信息。两个操作数可以都在寄存器中，或一个在寄存器、一个在内存中；只有一个操作数时，它可以在寄存器或内存中。

该字节基本分成两部分：位 [7:6] 与 [2:0] 共同构成一部分，位 [5:3] 构成另一部分。若位 [7:6] 为 00、01 或 10，第一部分指定内存操作数的寻址形式；若为 11，位 [2:0] 直接指定寄存器。位 [5:3] 在双操作数形式中指定另一个寄存器；某些其他形式不需要它表示寄存器时，可作为操作码扩展，用来区分八种操作。表 B.5 给出例子。

表 B.5 ModR/M 字节示例

| Mod | Reg | R/M | 有效地址  | 寄存器 | 说明                          |
|-----|-----|-----|-------|-----|-----------------------------|
| 00  | 011 | 000 | [EAX] | EBX | EAX 保存内存操作数地址，EBX 保存寄存器操作数。 |

| Mod | Reg | R/M | 有效地址         | 寄存器 | 说明                                                       |
|-----|-----|-----|--------------|-----|----------------------------------------------------------|
| 01  | 010 | 000 | disp8[EAX]   | EDX | 将指令的位移字节与 EAX 相加得到地址，寄存器操作数在 EDX。                        |
| 10  | 000 | 100 | disp32[-][-] | EAX | 四字节位移与 SIB 指定的基址及索引计算结果相加，另一操作数在 EAX。                    |
| 11  | 001 | 110 | ESI          | ECX | 双操作数形式中两者在 ESI、ECX；若 Reg 是操作码扩展，则操作数在 ESI，001 用于进一步指定操作。 |

B.2.4 SIB 字节

如果操作数来自内存，ModR/M 指定计算地址所需的信息。某些寻址形式需要的内容超过一个 ModR/M 字节所能表达，因此还包括一个 SIB 字节，如表 B.5 第三项。

SIB 是 scaled-index-base 的缩写，表示比例、索引和基址。它给出缩放因子，并指定索引寄存器及基址寄存器，计算  $scale \times index + base$ 。基址或索引可以省略，缩放因子也可以为 1。表 B.6 列出若干解释。

表 B.6 SIB 字节示例

| Scale | Index | Base | 计算                   | 说明                              |
|-------|-------|------|----------------------|---------------------------------|
| 00    | 011   | 000  | EBX+EAX              | EBX 与 EAX 相加，再与 ModR/M 指定的位移组合。 |
| 01    | 000   | 001  | $2 \times EAX + ECX$ | EAX 乘 2，再加 ECX，随后加入相应位移。        |
| 01    | 100   | 001  | ECX                  | 此编码不使用索引，仅把 ECX 与位移相加。          |
| 10    | 110   | 010  | $4 \times ESI + EDX$ | ESI 乘 4，再加 EDX，随后加入相应位移。        |

B.2.5 位移量

如果 ModR/M 指定地址计算需要位移，指令就包含相应的 1、2 或 4 字节位移字段，其大小由操作码或寻址编码确定。图 B.6 展示源操作数的地址计算：前缀 x26 覆盖默认段，选择 ES；ModR/M 与 SIB 指定，将四字节位移加到基址 ECX 和乘 4 的索引 EBX 上。



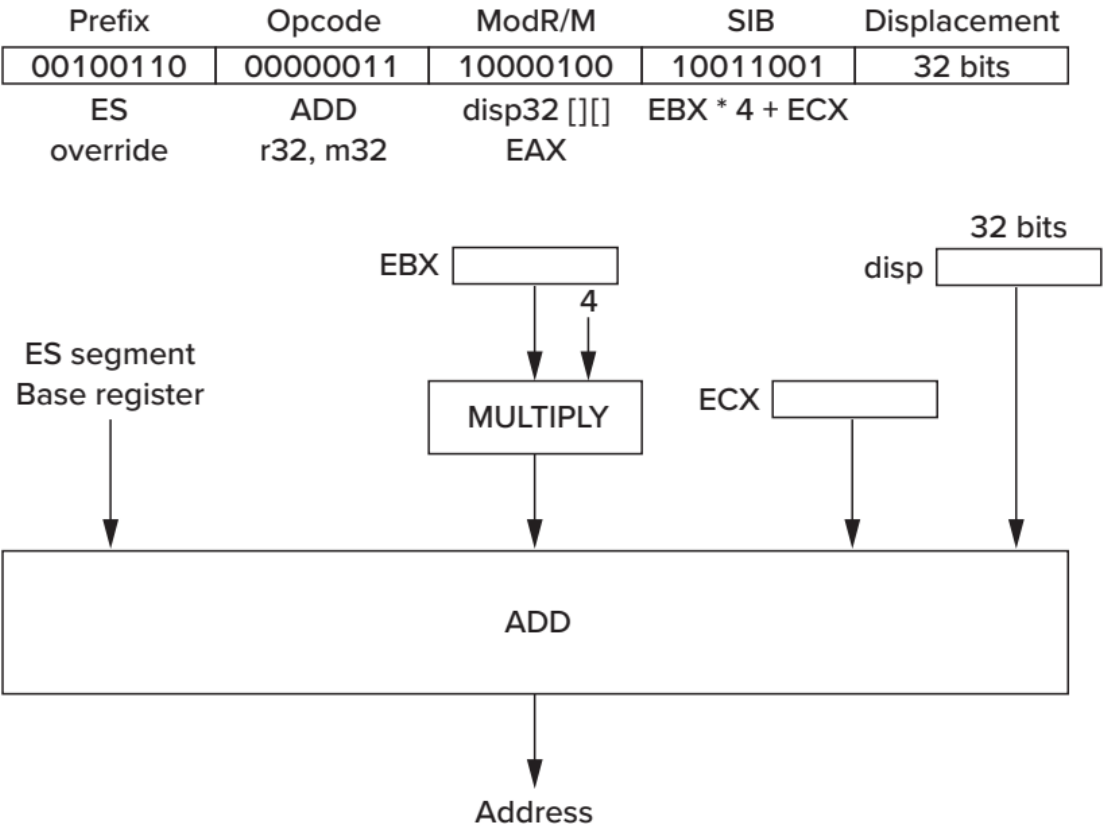


图 B.6 Base+ScaledIndex+disp32 寻址计算

B.2.6 立即数

LC-3 的 ADD 和 AND 可以在位 5 为 1 时使用指令内的小立即数。x86 同样允许把立即数直接放入指令。若操作码规定使用立即数，就同时决定它所占的字节数。在这里的编码示例中，立即数可以占 1、2 或 4 字节；当某种形式用较小字段表示较宽的有符号操作数时，先符号扩展再运算。

图 B.7 展示 ADD EAX, \$5 的立即数用法，作用类似 LC-3 的 ADD R0, R0, #5。

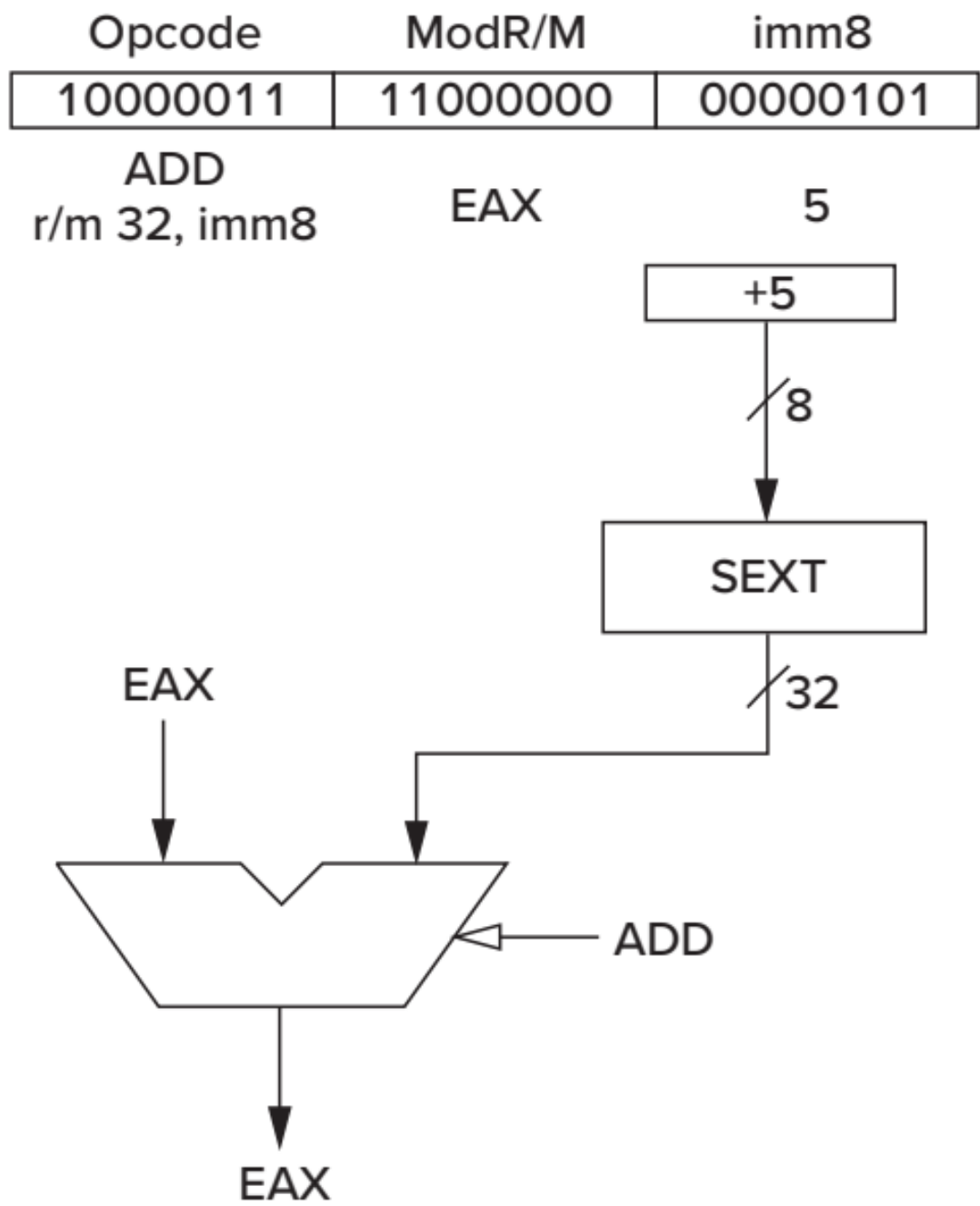


图 B.7 32 位模式下的 x86 示例：ADD EAX, \$5

译注：这里的立即数写法沿用原图，具体汇编器语法可能不同。上述字段长度说明面向本附录的传统示例；不能据此断言所有 64 位指令都只含这些长度的立即数字段。

B.3 一个例子

最后通过第 14 章已反复讨论的问题，把本附录串起来：给定包含字母、数字和标点的输入字符串，把所有小写字母转换为大写。图 B.8 给出 C 程序，图 B.9 给出带

注释的 LC-3 汇编版本，图 B.10 给出假定在 32 位模式下运行的 x86 版本。为了便于阅读，图中使用汇编表示，而不是机器码。

```
#include <stdio.h>

void UppcaseString(char inputString[]);

int main (void)
{
 char string[8];

 scanf("%s", string);
 UppcaseString(string);
}

void UppcaseString(char inputString[])
{
 int i = 0;

 while(inputString[i]) {
 if (('a' <= inputString[i]) && (inputString[i] <= 'z'))
 inputString[i] = inputString[i] - ('a' - 'A');
 i++;
 }
}
```

图 B.8 大小写转换程序的 C 源代码

```

; uppercase: converts lower- to uppercase
.ORIG x3000
MAIN LEA R6, STACK
READCHAR ADD R1, R6, #3
 IN ; read in input string: scanf
 OUT
 STR R0, R1, #0
 ADD R1, R1, #1
 ADD R2, R0, x-A
 BRnp READCHAR
 ADD R1, R1, #-1
 STR R2, R1, #0 ; put in NULL char to mark the "end"
 ADD R1, R6, #3 ; get the starting address of the string
 STR R1, R6, #14 ; pass the parameter
 STR R6, R6, #13
 ADD R6, R6, #11
 JSR UPPERCASE
 HALT
UPPERCASE STR R7, R6, #1
 AND R1, R1, #0
 STR R1, R6, #4
 LDR R2, R6, #3
CONVERT ADD R3, R1, R2 ; add index to starting addr of string
 LDR R4, R3, #0
 BRz DONE ; Done if NULL char reached
 LD R5, a
 ADD R5, R5, R4 ; 'a' <= input string
 BRn NEXT
 LD R5, z
 ADD R5, R4, R5 ; input string <= 'z'
 BRp NEXT
 LD R5, asubA ; convert to uppercase
 ADD R4, R4, R5
 STR R4, R3, #0
NEXT ADD R1, R1, #1 ; increment the array index, i
 STR R1, R6, #4
 BRnzp CONVERT
DONE LDR R7, R6, #1
 LDR R6, R6, #2
 RET
a .FILL #-97
z .FILL #-122
asubA .FILL #-32
STACK .BLKW 100
 .END

```

图 B.9 大小写转换程序的 LC-3 汇编代码

```

.386P
.model FLAT

_DATA SEGMENT ; The NULL-terminated scanf format
$SG397 DB '%s', 00H ; string is stored in global data space.
_DATA ENDS

_TEXT SEGMENT

_string$ = -8 ; Location of "string" in local stack
_main PROC NEAR
 sub esp, 8 ; Allocate stack space to store "string"
 lea eax, DWORD PTR _string$[esp+8]
 push eax ; Push arguments to scanf
 push OFFSET FLAT:$SG397
 call _scanf

 lea ecx, DWORD PTR _string$[esp+16]
 push ecx ; Push argument to UppcaseString
 call _UppcaseString

 add esp, 20 ; Release local stack space
 ret 0
_main ENDP

_inputString$ = 8 ; "inputString" location in local stack
_UppcaseString PROC NEAR
 mov ecx, DWORD PTR _inputString$[esp-4]
 cmp BYTE PTR [ecx], 0
 je SHORT $L404 ; If inputString[0]==0, skip the loop
$L403: mov al, BYTE PTR [ecx] ; Load inputString[i] into AL
 cmp al, 97 ; 97 = 'a'
 jl SHORT $L405
 cmp al, 122 ; 122 = 'z'
 jg SHORT $L405
 sub al, 32 ; 32 = 'a' - 'A'
 mov BYTE PTR [ecx], al
$L405: inc ecx ; i++ %$
 mov al, BYTE PTR [ecx]
 test al, al
 jne SHORT $L403 ; Loop if inputString[i] != 0
$L404: ret 0
_UppcaseString ENDP
_TEXT ENDS
END

```

图 B.10 大小写转换程序的 x86 汇编代码

译注：原图保留，但这些版本不是完全等价、可直接互换的成品。图 B.8 对八字节数组使用没有宽度限制的 `scanf("%s", string)`，存在越界风险，应限制到七个字符并检查返回值；`%s` 也只读一个以空白分隔的词。图 B.9 逐字符读取至换行，未作同样的长度限制，并采用另一套栈帧布局。其 `IN` 后又调用 `OUT`，在本书定义的回显规则下会重复回显；还须确认所用系统调用保存寄存器的约定。图 B.10 为传统 32 位工具链输出，依赖相应汇编器、C 库和调用环境，不能直接作为现代 64 位程序链接运行。示例的转换思路应与这些环境和边界条件分开理解。

## 附录 C LC-3 的微体系结构

第 4 章和第 5 章介绍了指令周期，以及 LC-3 的各条指令在这个周期中必须完成的工作。微体系结构的任务就是实现这些工作。本附录给出 LC-3 的一种具体实现。一个 ISA 可以有许多不同的微体系结构；如何在它们之间作出选择，是更深入的计算机体系结构课程所讨论的问题。

### C.1 概述

微体系结构由数据通路和控制结构组成。数据通路实际处理信息；控制结构在每个时钟周期产生控制信号，指挥数据通路，并确定下一个周期应当做什么。例如，若时钟周期约为 0.33 ns，时钟频率就约为 3 GHz。这里的频率只是说明时钟周期的例子。

图 C.1 展示了二者的关系。在本实现中，每个周期共有 52 位控制信息，其中 42 位控制数据通路，另外 10 位参与选择下一周期的控制信息。下一组信号取决于以下信息：当前的控制信息、正在处理的指令、特权模式及访存是否越权、分支条件是否成立、是否存在更高优先级的外部中断，以及内存操作是否完成。

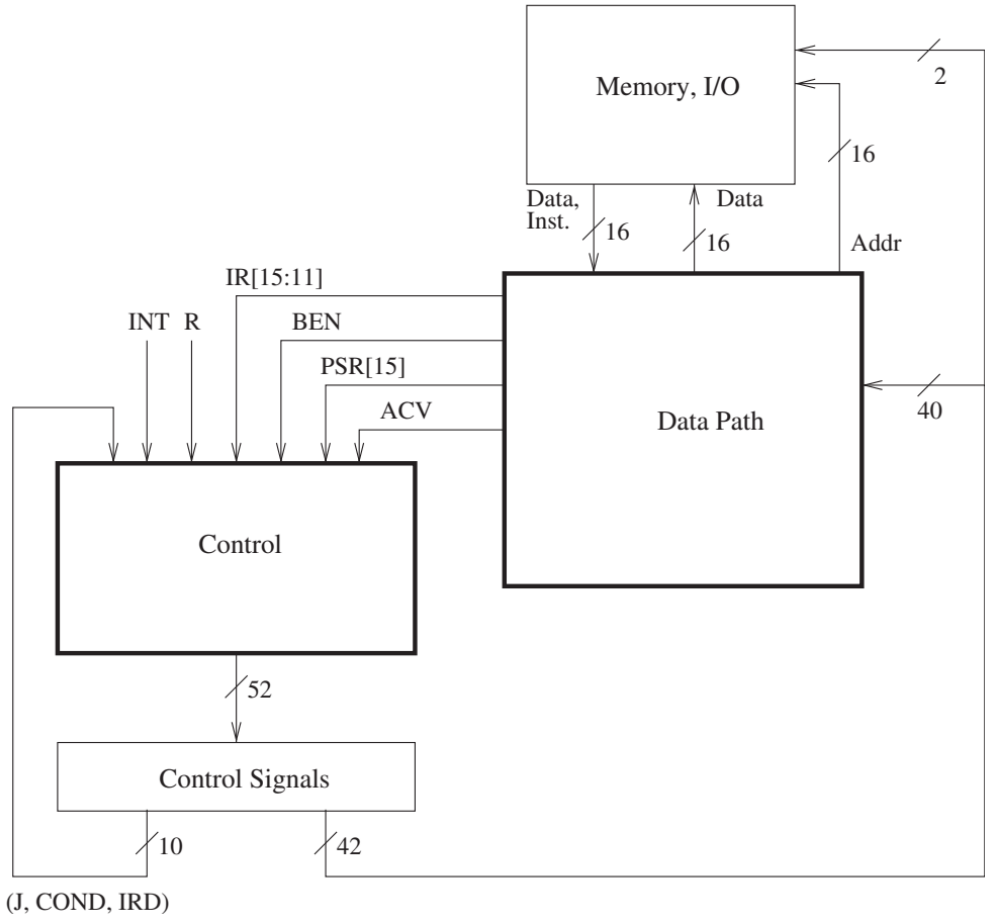


图 C.1 LC-3 数据通路与控制结构之间的关系

具体来说，需要使用以下信号：

1. 当前微指令中的  $J[5:0]$ 、 $COND[2:0]$  和  $IRD$ ，用于确定后继状态。
2.  $inst[15:12]$  指定操作码； $inst[11]$  用于区分 **JSR** 与 **JSRR**。
3.  $PSR[15]$  表示特权模式； $ACV$  表示访问控制违规。当用户模式下的程序试图访问特权存储器时， $ACV$  置位，阻止这次访问。
4.  $BEN$  表示 **BR** 的分支条件是否成立。
5.  $INT$  表示优先级高于当前进程的外部设备正在请求服务。
6.  $R$  表示内存操作已经完成。

## C.2 状态机

在任一时钟周期，LC-3 的行为由 52 位控制信号和另外 10 位信息共同决定。后者是  $inst[15:11]$ 、 $PSR[15]$ 、 $ACV$ 、 $BEN$ 、 $INT$  和  $R$ 。42 位数据通路控制信号

决定本周期的信息处理；其余 10 位控制信号与这些附加信息一起决定下一周期所需的控制信号。

我们把一组 52 位控制信号所规定的控制结构状态简称为状态。用结点表示各个状态，用有向弧表示状态之间的转移，就得到完整描述微体系结构行为的状态机。图 C.2 与图 C.7 合起来给出了本实现的状态机。每进入一个状态，就在其中执行一个时钟周期；若转移回自身，同一状态便会再执行一个周期。结点内写明数据通路在该周期执行的操作，有向弧则表示下一周期的去向。



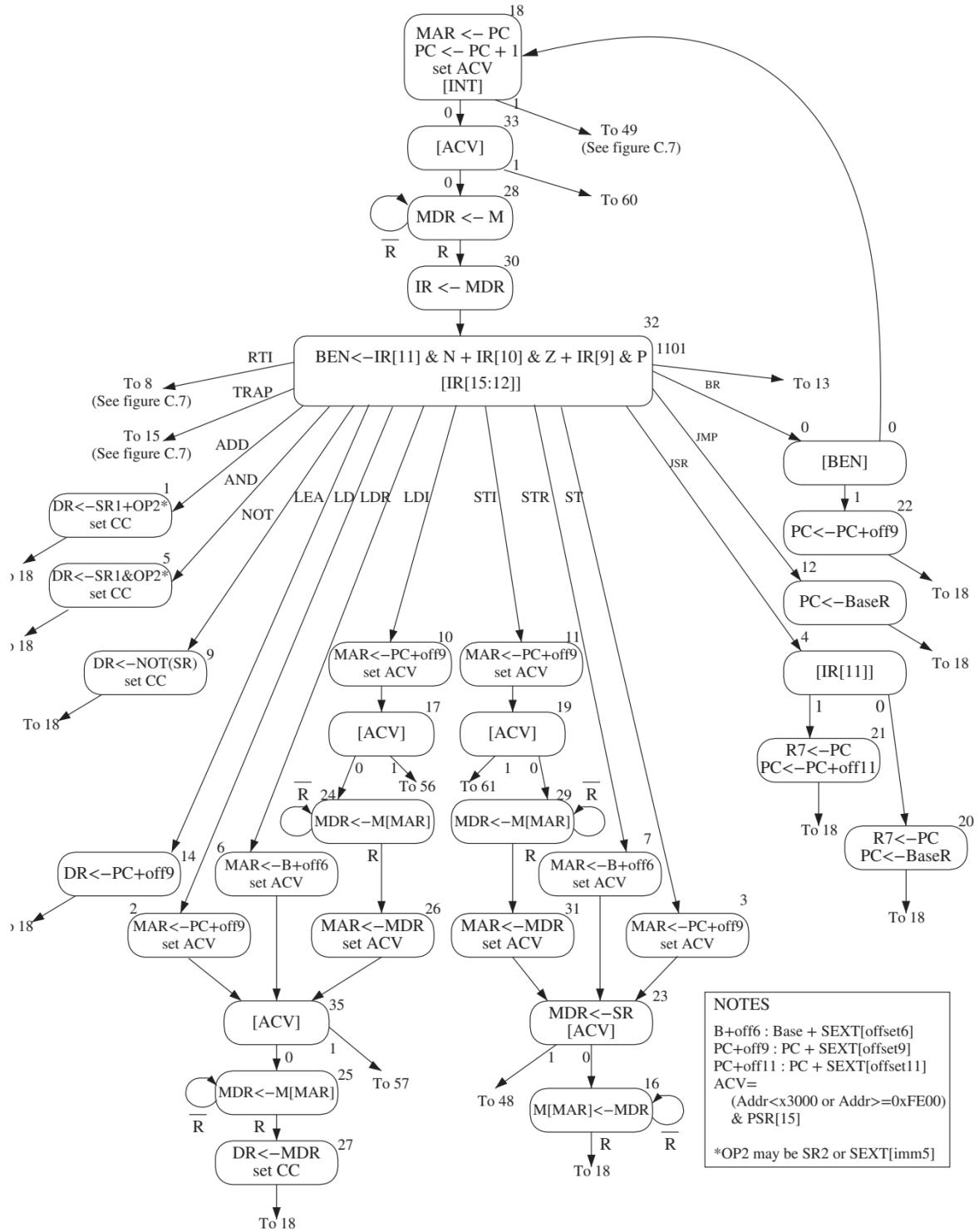


图 C.2 LC-3 状态机；中断、异常和 TRAP 的其余状态见图 C.7。

先看每条指令都要经历的 **FETCH** 阶段。状态 18 把 PC 的内容送入 MAR，并将 PC 加 1，为下一条指令做好准备。如果该地址属于特权存储器，而 PSR[15]=1 表明当前处于用户模式，就设置 ACV。如果没有中断请求，即 INT=0，控制转入状态 33；INT 为 1 的情况在 C.7 节讨论。

状态 33 检查 ACV。没有访问控制违规时转到状态 28，从 MAR 指定的内存位置

读取指令；发生违规时转到状态 60。状态 28 在内存尚未完成读取时反复执行。R 变为 1 后，控制转到状态 30，把 MDR 的内容装入 IR。状态 32 对指令译码，随后依据操作码选择执行路径。每条指令沿自己的路径完成余下的指令周期，最后返回状态 18，开始下一次取指。

完整状态机有 59 个不同的状态，因此用 6 位就可以为每个状态编号。例如，十进制状态号 18 对应二进制 010010。图 C.2 画出 31 个状态，并以 8、13、15、48、49、57、60 这 7 个状态号指出通往图 C.7 的连接；图 C.7 画出其余 28 个状态，其中包含这 7 个入口。译码处由 4 位操作码选择 16 种可能的后继状态；图 C.2 中部分路径汇集在指向图 C.7 的连接中。

### C.3 数据通路

图 C.3 是本实现的数据通路，也是第 5 章图 5.18 的扩充版本。它包含处理数据所需的功能部件、保存信息的寄存器，以及连接它们的总线和导线。图中的控制信号决定各部件在一个周期中如何工作。

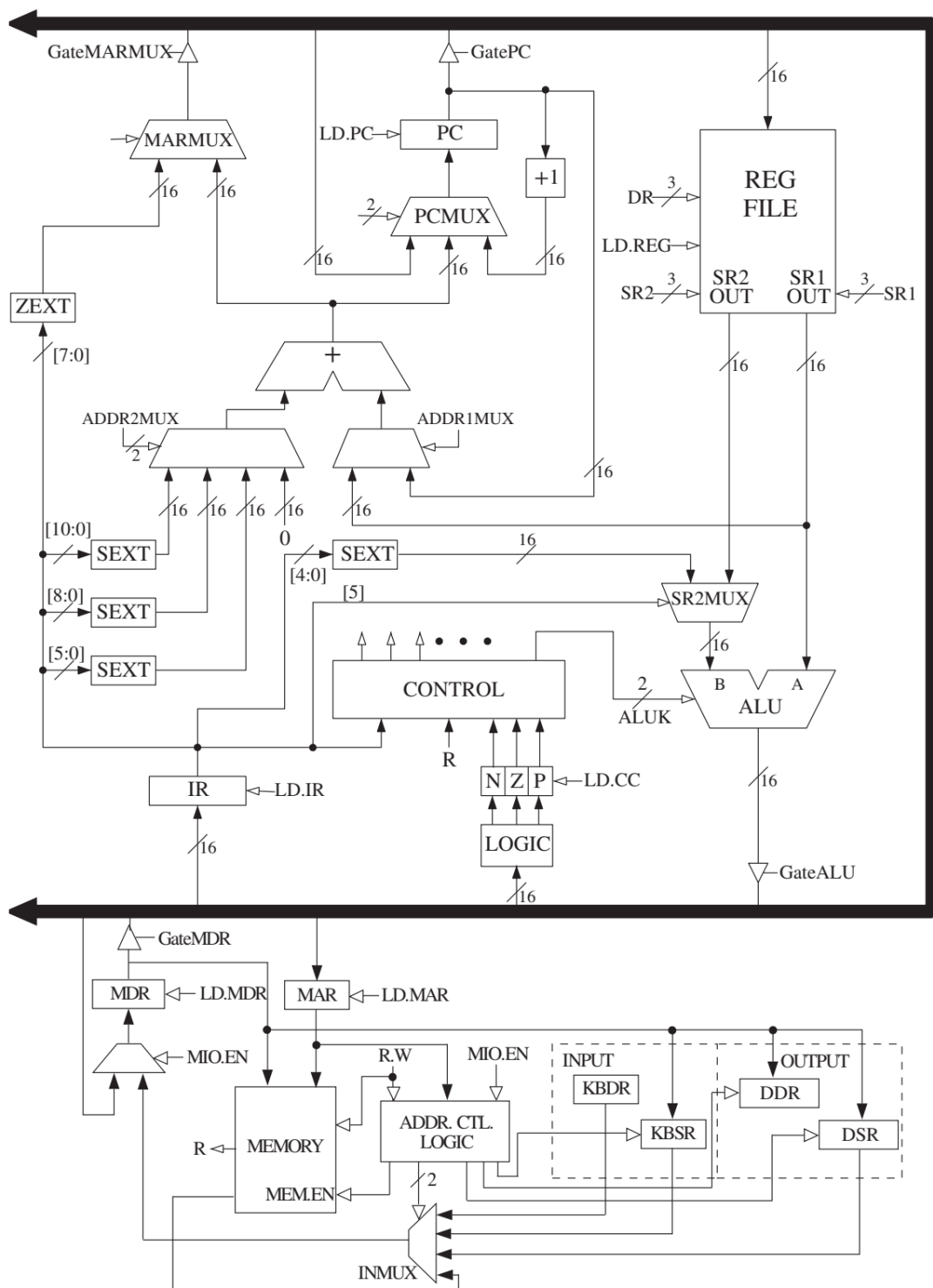


图 C.3 LC-3 数据通路

表 C.1 列出了数据通路的 42 位控制信号。为了便于阅读，表中采用取值的符号名称，而不是二进制编码。例如，ALUK 有 2 位，可以选择 ADD、AND、NOT，或者直接把输入 A 送到输出的 PASSA。PCMUX 也有 2 位，用来选择 PC 的输入。LD.PC 只有 1 位，决定本周期是否装载 PC。当前状态所对应的这 42 位信息共同控制数据通路中的各个部件，实现状态结点内列出的操作。

表 C.1 数据通路控制信号

| 信号          | 位数 | 取值及含义                                                                            |
|-------------|----|----------------------------------------------------------------------------------|
| LD.MAR      | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.MDR      | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.IR       | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.BEN      | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.REG      | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.CC       | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.PC       | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.Priv     | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.Priority | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.SavedSSP | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.SavedUSP | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.ACV      | 1  | NO / LOAD; 不装载 / 装载                                                              |
| LD.Vector   | 1  | NO / LOAD; 不装载 / 装载                                                              |
| GatePC      | 1  | NO / YES; 不驱动 / 驱动总线                                                             |
| GateMDR     | 1  | NO / YES; 不驱动 / 驱动总线                                                             |
| GateALU     | 1  | NO / YES; 不驱动 / 驱动总线                                                             |
| GateMARMUX  | 1  | NO / YES; 不驱动 / 驱动总线                                                             |
| GateVector  | 1  | NO / YES; 不驱动 / 驱动总线                                                             |
| GatePC-1    | 1  | NO / YES; 不驱动 / 驱动总线                                                             |
| GatePSR     | 1  | NO / YES; 不驱动 / 驱动总线                                                             |
| GateSP      | 1  | NO / YES; 不驱动 / 驱动总线                                                             |
| PCMUX       | 2  | PC+1 / BUS / ADDER; 选择 PC 加 1、总线或地址加法器输出                                         |
| DRMUX       | 2  | 11.9 / R7 / SP; 选择 IR[11:9]、R7 或 R6                                              |
| SR1MUX      | 2  | 11.9 / 8.6 / SP; 选择 IR[11:9]、IR[8:6] 或 R6                                        |
| ADDR1MUX    | 1  | PC / BaseR                                                                       |
| ADDR2MUX    | 2  | ZERO / offset6 / PCOffset9 / PCOffset11; 分别选择 0 或 IR[5:0]、IR[8:0]、IR[10:0] 的符号扩展 |
| SPMUX       | 2  | SP+1 / SP-1 / Saved SSP / Saved USP; 选择增减后的栈指针或保存的栈指针                            |
| MARMUX      | 1  | 7.0 / ADDER; 选择 IR[7:0] 的零扩展或地址加法器输出                                             |
| TableMUX    | 1  | x00 / x01                                                                        |
| VectorMUX   | 2  | INTV / Priv.exception / Opc.exception / ACV.exception                            |
| PSRMUX      | 1  | individual settings / BUS; 分别设置各字段或使用总线值                                         |
| ALUK        | 2  | ADD / AND / NOT / PASSA                                                          |
| MIO.EN      | 1  | NO / YES; 不访问 / 访问内存或 I/O                                                        |
| R.W         | 1  | RD / WR; 读 / 写                                                                   |
| Set.Priv    | 1  | 0 / 1; 管理模式 / 用户模式                                                               |

## C.4 控制结构

状态机规定了控制结构：每个周期既要提供数据通路所需的控制信号，又要确定如何从当前状态转到后继状态。图 C.4 给出了这里采用的控制结构。实现方法有很多；

评价和选择它们需要一门完整的计算机体系结构课程。这里采用一种直接的微程序实现。

每个状态所需的 42 位数据通路控制信息与 10 位后继状态控制信息合起来称为一条微指令。每条微指令占据一种特殊存储器——控制存储器——中的一个 52 位单元。59 个状态各对应一条微指令，用 6 位地址加以选择。地址就是状态号：例如，状态 18 所需的 52 位控制信息存放在控制存储器的地址 18 中。

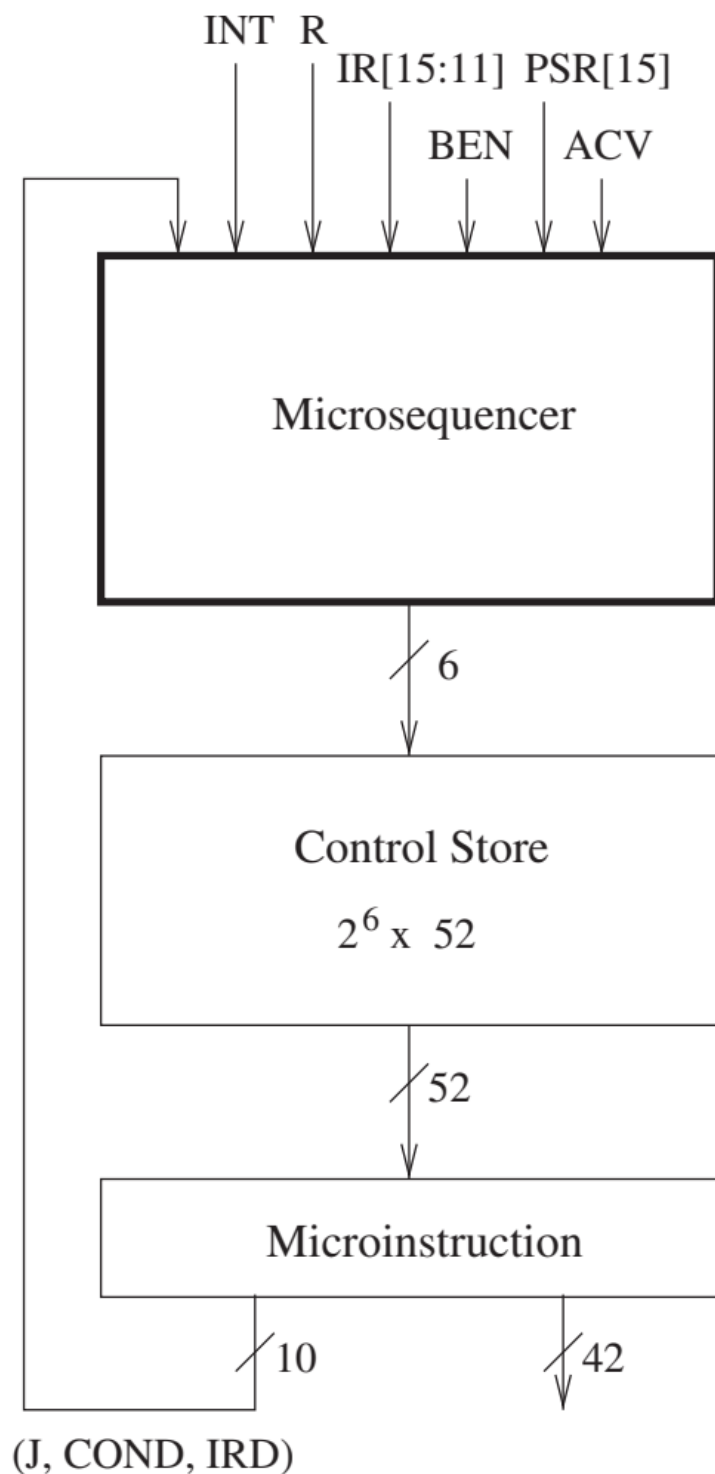


图 C.4 LC-3 的微程序控制结构

表 C.2 微定序器控制信号

| 信号   | 位数 | 功能或取值                                                       |
|------|----|-------------------------------------------------------------|
| J    | 6  | 后继状态地址的基础位模式。                                               |
| COND | 3  | 0: 无条件; 1: 内存就绪; 2: 分支; 3: 寻址方式; 4: 特权模式; 5: 中断; 6: 访问控制违规。 |
| IRD  | 1  | 为 1 时按指令操作码选择后继状态; 为 0 时使用 J 和 COND 指定的条件。                  |

表 C.2 说明了决定后继状态的 10 位控制信息。微定序器的任务是产生下一条微指令在控制存储器中的地址，逻辑见图 C.5。当 IRD 为 1 时，把两个 0 与 IR[15:12] 拼接起来作为地址。因此，ADD 对应状态 1，而未定义的操作码 1101 对应状态 13，转入非法操作码异常处理路径。

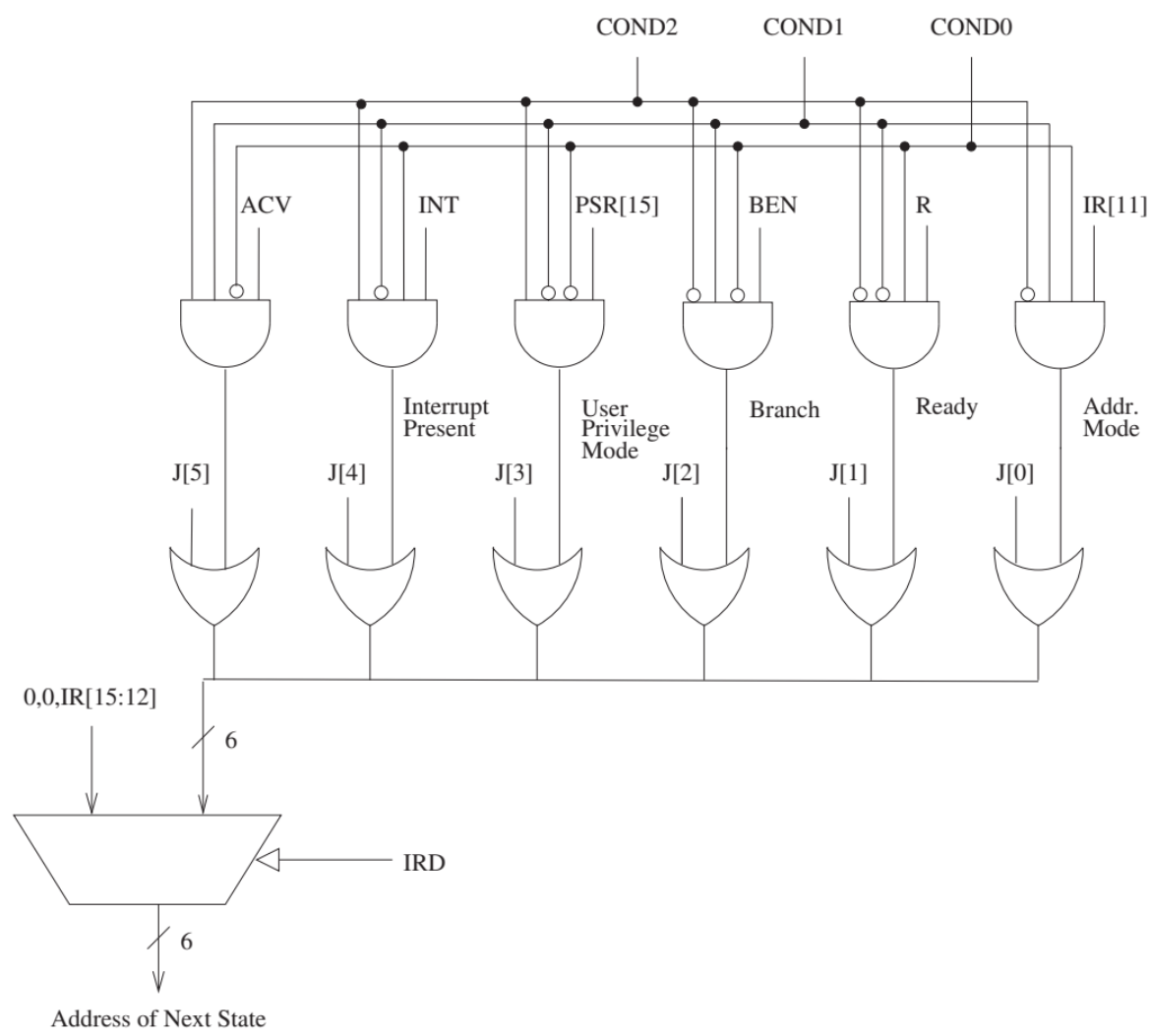


图 C.5 微定序器

DR、SR1、BEN、INT、ACV 和 R 也是数据通路或微定序器需要的信号，但不属

于表 C.1 和 C.2 中的控制位。图 C.6 给出生成 DR、SR1、BEN 和 ACV 的附加逻辑。INT 来自正常指令处理之外的事件，表示应中断当前程序并处理该事件。第 9 章已经介绍了中断机制，C.7 节进一步解释其控制流程。

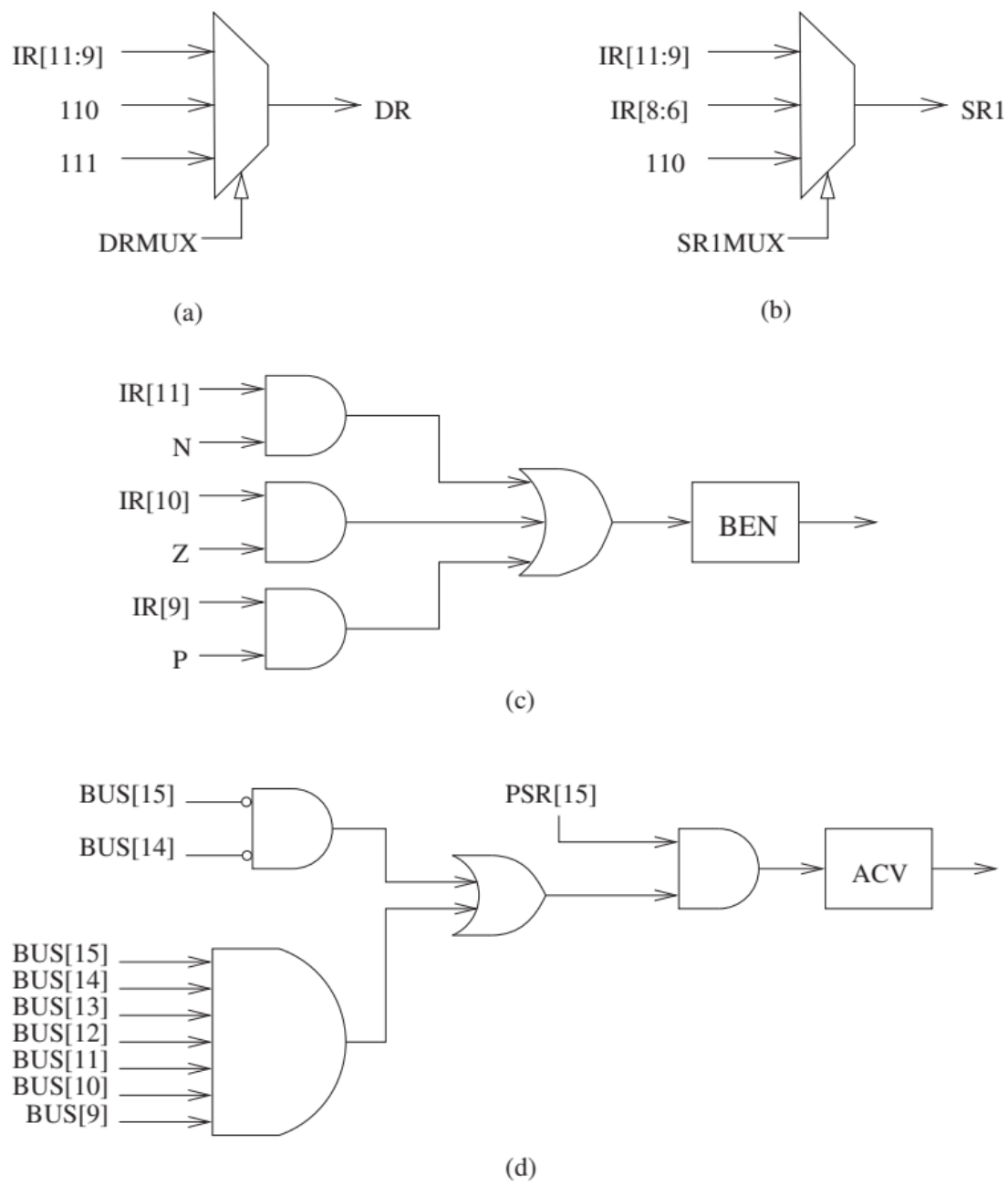


图 C.6 生成控制信号所需的附加逻辑

R 由内存产生，使 LC-3 能够配合读写需要多个时钟周期的内存工作。假设一次读取要用 5 个周期：MAR 中已经有地址，微指令通过 MIO.EN/YES 和 R.W/RD 发出读取请求后，直到第 5 个周期，正确的数据才能送入 MDR。

因此，在取指过程中，状态 28 必须执行 5 次才能进入状态 30。前 4 个周期 R 为 0，微定序器继续选择状态 28；第 5 个周期 R 为 1，选择状态 30。如果不等读取完成就转移，状态 30 就会把 MDR 中无效的内容装进 IR。状态 28 的微定序器控制位如下：

```

IRD/0 ; NO
COND/001 ; Memory Ready
J/011100

```

R 为 0 时，后继地址是 011100；R 为 1 时，地址变为 011110，即状态 30。COND 必须取 001，才能使 R 通过相应的四输入 AND 门，参与后继地址的形成。

## C.5 TRAP 指令

每条指令从译码状态 32 出发，沿自己的路径执行，最终返回状态 18。这里以图 C.7 中的 TRAP 为例。TRAP 将 PSR 和 PC 压入系统栈，把陷阱服务例程的起始地址装入 PC，然后在特权存储器中执行该服务例程。

TRAP 的操作码是 1111，所以状态 32 之后进入状态 15。状态 15 将 Table 装入 x00，将 PC 加 1，并将 PSR 送入 MDR，准备压栈；随后进入状态 47。Table 最终用来形成陷阱向量表项地址的高 8 位。

状态 47 将 IR[7:0] 装入 8 位 Vector 寄存器，将 PSR[15] 清零，进入管理模式，并依据原来的特权模式转入状态 37 或 45。若原来在用户模式，状态 45 先把用户栈指针保存到 Saved USP，再从 Saved SSP 恢复管理栈指针，然后进入状态 37。若原来已在管理模式，就不需要状态 45。

状态 37 和 41 把 PSR 压入系统栈；状态 43、46 和 52 把返回地址压栈。状态 43 送入 MDR 的是 PC 减 1 的结果。这是中断和异常路径所需的公共操作；TRAP 在状态 15 额外将 PC 加 1，正好抵消这里的减 1，使保存的返回地址仍指向 TRAP 后面的指令。<sup>1</sup>

接下来，状态 54 将 Table 与 Vector 拼接成陷阱向量表项地址，装入 MAR；状态 53 读取表项到 MDR；状态 55 将 MDR 装入 PC。TRAP 的指令周期至此结束，状态机回到状态 18，开始取陷阱服务例程的第一条指令。

服务例程以 RTI 返回。RTI 的 4 位操作码为 1000，因此状态 32 之后进入状态 8。状态 8、36 和 38 从系统栈弹出并恢复 PC；状态 39、40、42 和 34 弹出并恢复 PSR。若原程序处于用户模式，最后还要在状态 59 保存管理栈指针并恢复用户栈指针。<sup>2</sup>

## C.6 存储器映射 I/O

LC-3 用与普通内存读写相同的数据传送指令完成 I/O。每个设备寄存器都有一个内存地址；有效地址指向输入设备寄存器的装入指令完成输入，指向输出设备寄存器的存储指令完成输出。例如，在图 C.2 的状态 25 中，若 MAR 为 xFE02，送入 MDR 的是 KBDR 中最近键入的字符；若 MAR 是普通内存地址，则数据来自内存。

无需为存储器映射 I/O 修改图 C.2 的状态机。图 C.3 中的地址控制逻辑

<sup>1</sup>译注：原文称状态 43 将 PC 减 1；按状态图，其具体操作是  $MDR \leftarrow PC - 1$ ，并非在该状态改写 PC。

<sup>2</sup>译注：原文将 RTI 的操作码称为“eight-bit opcode 1000”，已按 LC-3 的 4 位操作码修正。



ADDR.CTL.LOGIC 负责判定究竟访问内存还是设备寄存器。其真值表见表 C.3，输入是 MAR、表示本周期是否访问内存或 I/O 的 MIO.EN，以及表示读写方向的 R.W。

表 C.3 地址控制逻辑的真值表

| MAR   | MIO.EN | R.W | MEM.EN | IN.MUX | LD.KBSR | LD.DSR | LD.DDR |
|-------|--------|-----|--------|--------|---------|--------|--------|
| xFE00 | 0      | R   | 0      | x      | 0       | 0      | 0      |
| xFE00 | 0      | W   | 0      | x      | 0       | 0      | 0      |
| xFE00 | 1      | R   | 0      | KBSR   | 0       | 0      | 0      |
| xFE00 | 1      | W   | 0      | x      | 1       | 0      | 0      |
| xFE02 | 0      | R   | 0      | x      | 0       | 0      | 0      |
| xFE02 | 0      | W   | 0      | x      | 0       | 0      | 0      |
| xFE02 | 1      | R   | 0      | KBDR   | 0       | 0      | 0      |
| xFE02 | 1      | W   | 0      | x      | 0       | 0      | 0      |
| xFE04 | 0      | R   | 0      | x      | 0       | 0      | 0      |
| xFE04 | 0      | W   | 0      | x      | 0       | 0      | 0      |
| xFE04 | 1      | R   | 0      | DSR    | 0       | 0      | 0      |
| xFE04 | 1      | W   | 0      | x      | 0       | 1      | 0      |
| xFE06 | 0      | R   | 0      | x      | 0       | 0      | 0      |
| xFE06 | 0      | W   | 0      | x      | 0       | 0      | 0      |
| xFE06 | 1      | R   | 0      | x      | 0       | 0      | 0      |
| xFE06 | 1      | W   | 0      | x      | 0       | 0      | 1      |
| other | 0      | R   | 0      | x      | 0       | 0      | 0      |
| other | 0      | W   | 0      | x      | 0       | 0      | 0      |
| other | 1      | R   | 1      | mem    | 0       | 0      | 0      |
| other | 1      | W   | 1      | x      | 0       | 0      | 0      |

*x* 表示无关项，other 表示表中四个设备地址以外的地址。

对于装载操作，MDR 的数据可以来自内存、KBDR、KBSR 或 DSR；地址控制逻辑为 INMUX 提供选择信号。对于存储操作，MDR 的数据可以写入内存、KBSR、DDR 或 DSR；地址控制逻辑产生相应的使能信号。表中列出的设备范围与本图的数据通路一致。

## C.7 中断与异常控制

要补全 LC-3 状态机，还需要处理中断启动、中断返回，以及 ISA 规定的三种异常。中断和异常都暂停正在执行的程序，将其 PSR 和 PC 压入系统栈，从中断向量表取得服务例程的起始地址，并把这个地址装入 PC。主要区别在于触发原因：中断来自外部事件，异常来自正在执行的指令。这里的三种异常是用户模式下执行 RTI、遇到非法操作码，以及访问控制违规。

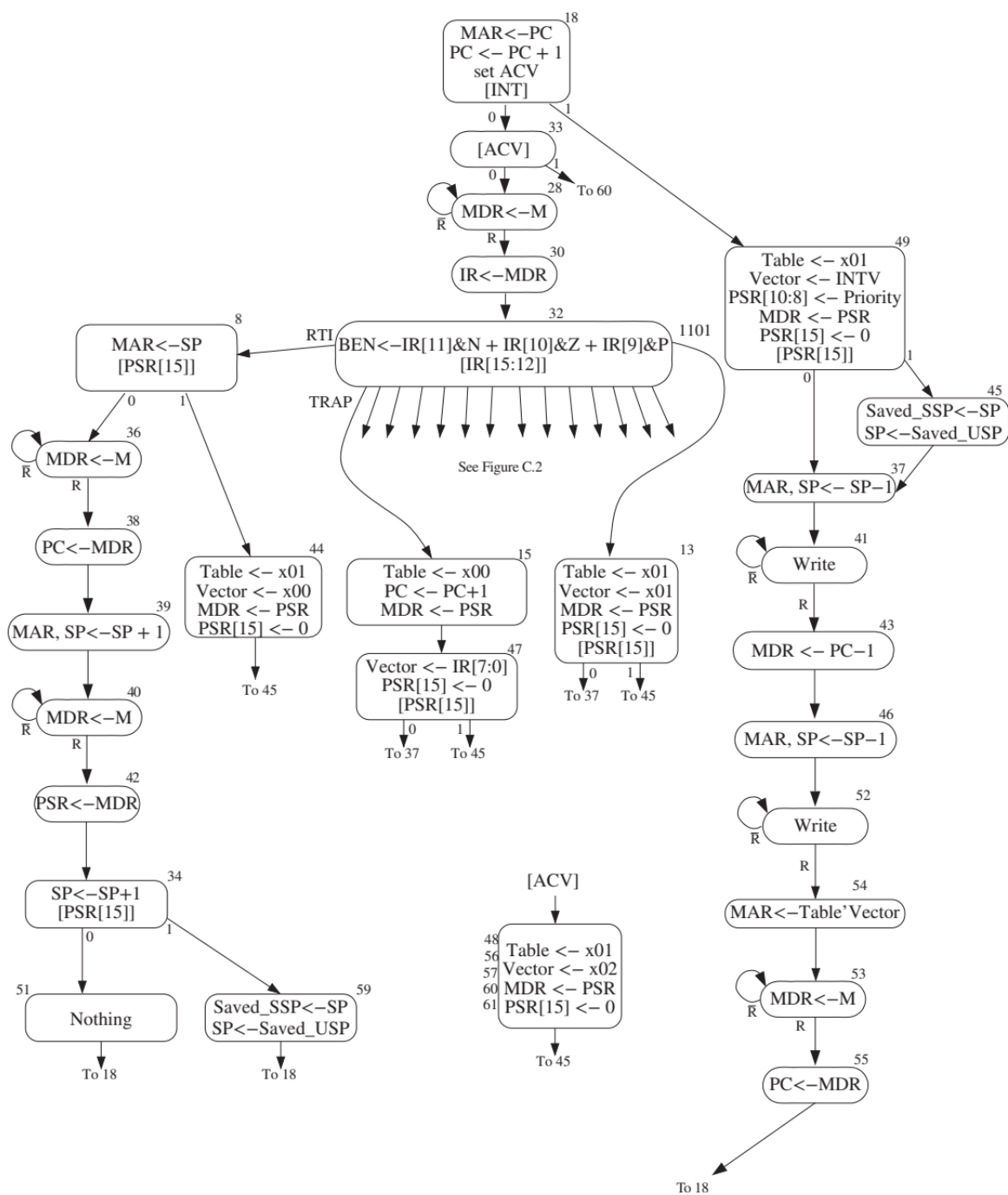
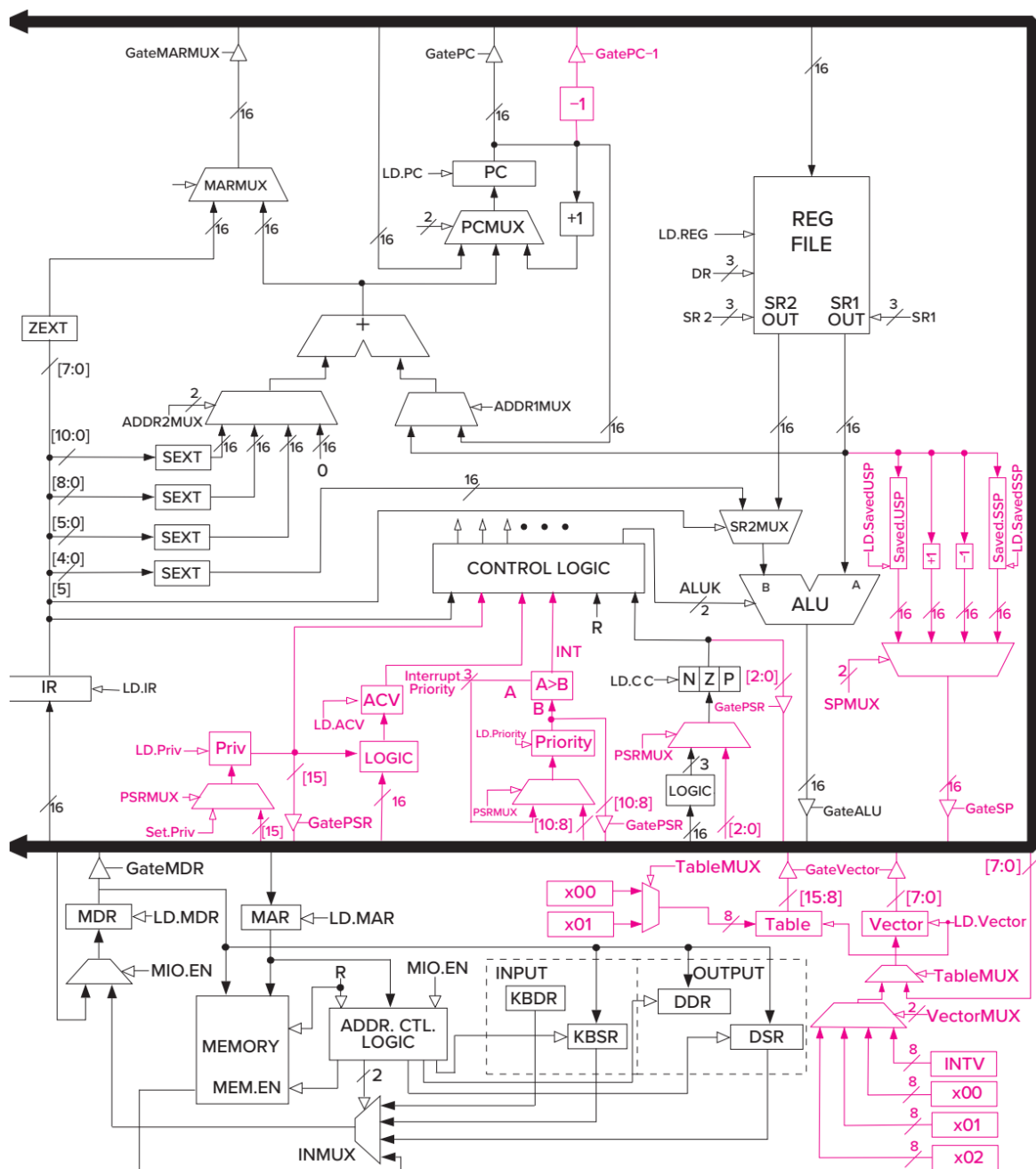


图 C.7 中断、异常、TRAP 和 RTI 的状态



**图 C.8** 加入中断控制所需结构的 LC-3 数据通路

C.7.1 节介绍启动中断所需的处理流程，C.7.3 节介绍启动异常所需的处理流程。

### C.7.1 启动中断

程序执行期间，外部事件可以请求中断，以便抢占正常的指令处理。只有请求的优先级高于当前进程时，才置位 INT。设备还提供中断向量和优先级。处理器必须保存被中断程序的状态，以便服务结束后继续执行。

本实现仅在状态 18 检查 INT。这样，前一条指令已经完整执行，而刚开始的取指可以在真正读取指令之前暂停。状态 18 的微定序器字段为 IRD=0、COND=101、

J=100001。INT 为 0 时进入状态 33；INT 为 1 时设置后继地址的第 4 位，进入 110001，即状态 49。

状态 49 将旧 PSR 送入 MDR，将 PSR[15] 清零进入管理模式，将设备优先级装入 PSR[10:8]，把中断向量装入 Vector，并把 Table 设为 x01。此外，还要根据旧 PSR 的特权模式选择后继状态。对应字段为 IRD=0、COND=100、J=100101：原来在管理模式时转到状态 37；原来在用户模式时转到状态 45。

R6 充当栈指针，但用户栈与系统栈是两个不同的栈。若中断发生在用户模式，状态 45 将 R6 保存到 Saved USP，再把 Saved SSP 装入 R6，随后进入状态 37。若原来已经在管理模式，R6 本来就指向系统栈，可以直接进入状态 37。

状态 37 将栈指针减 1，并把新的栈顶地址装入 MAR；状态 41 把 MDR 中的旧 PSR 写入栈，等待 R 表明写入完成。状态 43 把 PC 减 1 的结果装入 MDR。这里要抵消状态 18 中为取指而进行的 PC 加 1，使中断返回后能重新取出尚未执行的指令。状态 46 再次将栈指针减 1 并设置 MAR；状态 52 将返回地址写入栈。

状态 54 将 Table 与 Vector 拼成中断向量表项地址；状态 53 读取服务例程的起始地址；状态 55 把这个地址装入 PC。回到状态 18 后，处理器开始执行中断服务例程。

### C.7.2 从中断返回

中断服务例程的最后一条指令是 RTI。它必须以与压栈相反的顺序恢复 PC 和 PSR。译码后进入状态 8，将栈指针装入 MAR，并检查当前特权模式。只有管理模式允许执行这条返回路径；用户模式的情况见下一小节。

状态 36 从栈读取 PC 到 MDR，状态 38 将其装入 PC。状态 39 将栈指针加 1，并把新地址装入 MAR；状态 40 读取保存的 PSR；状态 42 恢复 PSR；状态 34 再将栈指针加 1，完成两次弹栈。

状态 34 根据恢复后的特权模式作出选择。其字段为 IRD=0、COND=100、J=110011。若恢复的是管理模式，则进入不执行额外操作的状态 51；若恢复的是用户模式，则进入状态 59，保存当前管理栈指针并恢复用户栈指针。两条路径随后都回到状态 18，继续被中断的程序。

### C.7.3 启动异常

异常与中断采用相同的基本保存和转移机制。处理器将 PSR 和 PC 压入系统栈，再从相应向量表项取得异常服务例程的起始地址。不过，异常由当前指令触发，服务它的紧迫程度与当前程序相同，因此不改变当前优先级。

#### C.7.3.1 特权模式异常

若用户模式下的程序试图执行 RTI，状态 8 转到状态 44。状态 44 将 Table 设为 x01，Vector 设为 x00，把旧 PSR 送入 MDR，并清除 PSR[15]。这样，最终使用的向量表项地址就是 x0100。

与中断入口状态 49 相比，这里既不改变优先级，也不必再判定旧特权模式：能够触发这种异常，就已经知道原程序处于用户模式。因此状态 44 直接转到状态 45，切换栈指针，再沿状态 37、41、43 等公共路径保存 PSR 和 PC，并装入服务例程地址。

### C.7.3.2 非法操作码异常

LC-3 没有为操作码 1101 定义指令。若译码遇到这个操作码，就从状态 32 进入状态 13，启动非法操作码异常。处理器仍然保存 PSR 和 PC，并转入相应服务例程。

状态 13 与状态 44 类似，但 Vector 取  $\times 01$ ，所以向量表项位于  $\times 0101$ 。此外，非法操作码既可能出现在用户模式，也可能出现在管理模式，因此必须检查旧特权模式：管理模式直接转到状态 37，用户模式先转到状态 45 切换栈指针。这里同样保持原优先级。后续压栈和取得服务例程地址的过程与中断入口相同。

### C.7.3.3 访问控制违规 (ACV) 异常

用户模式下访问特权存储器会引发 ACV 异常。状态机在每个相关访存路径上检查这一条件，检查状态为 17、19、23、33 和 35。若检测到违规，它们分别转入状态 56、61、48、60 和 57。

这五个状态执行相同的入口操作：Table 取  $\times 01$ ，Vector 取  $\times 02$ ，MDR 保存旧 PSR，PSR[15] 清零。除向量值不同外，这与状态 44 一样；ACV 服务例程的地址存放在  $\times 0102$ 。由于已知原程序处于用户模式，接下来先进入状态 45 切换到系统栈，然后保存 PSR 和 PC，并转入异常服务例程。

## C.8 控制存储器

图 C.9 给出了完成 LC-3 微程序实现所需的控制存储器格式。每一行对应一个状态所需的 52 位控制信号。原书特意将具体内容留空，让读者自己填写；答案可向任课教师索取。这里保留这些空白。

[illegible]

图 C.9 控制存储器的规格

# 附录 D C 程序设计语言

## D.1 概述

本附录是一份面向 C 语言初学者的 ANSI C (C18) 参考手册。它涵盖了语言的相当一部分内容，其中也有正文没有讨论的特性，供使用本书的课程在编程时快速查阅。各条目简要说明一种 C 语言特性，并在适当之处给出示例。<sup>1</sup>

## D.2 C 语言的约定

先介绍 C 程序的词法元素，以及程序员编写 C 程序时采用的一些约定。

### D.2.1 源文件

通常把 C 程序分成两类文件：扩展名为 `.c` 的源文件和扩展名为 `.h` 的头文件。源文件也称为 `dot-c` 文件，保存一组相关函数的 C 代码。例如，管理栈数据结构的函数可以放在 `stack.c` 中。每个 `.c` 文件分别编译成目标文件，再由链接器把这些目标文件链接成可执行映像。

### D.2.2 头文件

头文件通常保存函数、变量、结构体和类型的声明，以及预处理器宏，而不是可执行的 C 语句。惯例是让一个头文件与实现其中声明的项目的源文件相对应。例如，若用 `stdio.c` 代表实现 `printf`、`scanf`、`getchar` 和 `putchar` 的源文件，那么 `stdio.h` 就保存这些函数的声明。从另一个 `.c` 文件调用这些函数时，应通过 `#include` 包含 `stdio.h`，以取得正确的声明。实际标准库未必把这些实现集中在一个同名源文件中。

### D.2.3 注释

C 支持以 `//` 开始、延续到行末的注释，也支持用 `/*` 和 `*/` 包围的块注释；早期 C 只支持后一种。块注释不能嵌套。在字符串或字符常量中出现的这些字符序列属于文字内容，不会启动注释。<sup>2</sup>

---

<sup>1</sup>译注：本附录保持原书的 C18 范围。原文个别说明沿用了较早版本 C 的规则，另有少量技术错误；影响理解或编译的地方已在相应位置注明。

<sup>2</sup>译注：原文笼统地说“注释中的注释不合法”。准确地说，块注释不支持嵌套；行注释中的另一个 `//` 只是注释内容，不会因此产生语法错误。

## D.2.4 字面量

C 程序可以包含整数、浮点数、字符、字符串，以及枚举常量。这些常量可以用作变量的初始化值，也可以出现在表达式中。

### D.2.4.1 整数

整数字面量可以采用十进制、八进制或十六进制。以 0 开头的整数按八进制解释，以 0x 或 0X 开头的整数按十六进制解释；十六进制允许数字 0–9 和字母 a–f，也允许大写 A–F。其余形式按十进制解释。前面加一元负号可以表示负值。

后缀 l 或 L 用来指定 long 类型，u 或 U 用来指定无符号类型；具体类型还取决于进制和数值是否可表示。long 与 unsigned 见 D.3.2 节。下面前三个例子都表示 87，最后两个例子分别采用无符号整数和 long 整数形式。

```
87 // 87 in decimal
0x57 // 87 in hexadecimal
0127 // 87 in octal
-24 // -24 in decimal
-024 // -20 in octal
-0x24 // -36 in hexadecimal
87U
87L
```

### D.2.4.2 浮点数

十进制浮点常量由整数部分、小数点和小数部分组成；整数部分和小数部分可以省略其中之一，但不能同时省略。前面加负号表示负值。

```
1.613123
.613123
1. // expresses the number 1.0
-.613123
```

也可以使用指数形式：在数值后接 e 或 E，再接一个整数指数，表示前面的值乘以 10 的该次幂。指数可以为负；使用指数时可以省略小数点。

```
6.023e23 // 6.023 * 10^23
454.323e-22 // 454.323 * 10^(-22)
5E13 // 5.0 * 10^13
```

没有后缀的浮点常量类型是 double，即双精度浮点数。后缀 f 或 F 指定 float，即单精度浮点数；后缀 l 或 L 指定 long double，见 D.3.2 节。

### D.2.4.3 字符

用单引号包围一个字符，例如 'c'，就得到字符常量，其值与计算机采用的字符编码有关。LC-3 采用 ASCII。表 D.1 列出一些需要用转义序列表示的特殊字符。八进



制和十六进制形式还可直接给出字符编码：ASCII 字符 S 的十进制编码为 83，可以写成 '\123' 或 '\x53'。<sup>3</sup>

表 D.1 C 中的特殊字符

| 字符             | 转义序列  |
|----------------|-------|
| 换行             | \n    |
| 水平制表           | \t    |
| 垂直制表           | \v    |
| 退格             | \b    |
| 回车             | \r    |
| 换页             | \f    |
| 响铃             | \a    |
| 反斜杠            | \\    |
| 问号             | \?    |
| 单引号            | \'    |
| 双引号            | \"    |
| 八进制编码（1-3 位）   | \ooo  |
| 十六进制编码（至少 1 位） | \xhhh |

D.2.4.4 字符串字面量

字符串字面量用双引号包围，末尾自动附加终止字符 '\0'。普通字符串字面量是具有静态存储期的 char 数组，在多数表达式中转换为指向首字符的指针。

```
char greeting[10] = "bon jour!";
printf("This is a string literal");
```

字符串字面量可以初始化字符数组，也可用于需要 char \* 的位置，例如相应的函数实参。不过，它不能用于对整个数组赋值。以下代码在 C 中不合法：

```
char greeting[10];
greeting = "bon jour!";
```

**译注：**原文直接把字符串字面量的类型写成 char \*，混淆了数组类型及其常见的指针转换。尝试修改字符串字面量的内容会产生未定义行为；上例初始化得到的独立数组 greeting 则可以修改。

D.2.4.5 枚举常量

枚举类型（见 D.3.1 节）关联着一组枚举项，也称枚举常量。它们是 int 类型的符号常量，具体值由枚举声明的枚举项列表确定。

<sup>3</sup>译注：原文八进制例子 '\0123' 多写了一位。C 的八进制转义最多含三个八进制数字，故这里修正为 '\123'。普通单字符整数常量在 C 中的类型是 int。

### D.2.5 格式

C 允许较自由的排版。程序员可以在适当的词法元素之间添加空格、制表符和换行，在语句与声明之间安排版面。常见风格包括充分缩进控制结构、一致地对齐花括号，以及添加有帮助而不妨碍阅读的注释。正文的 C 代码给出了许多示例。空白不能任意插入一个标识符或运算符内部。

### D.2.6 关键字

以下保留字在 C 中具有特殊含义，用于基本类型、类型修饰、控制结构体和其他语言特性。它们不能用作程序员自行命名的变量、函数等标识符。

|          |          |          |                |
|----------|----------|----------|----------------|
| auto     | extern   | short    | while          |
| break    | float    | signed   | _Alignas       |
| case     | for      | sizeof   | _Alignof       |
| char     | goto     | static   | _Atomic        |
| const    | if       | struct   | _Bool          |
| continue | inline   | switch   | _Complex       |
| default  | int      | typedef  | _Generic       |
| do       | long     | union    | _Imaginary     |
| double   | register | unsigned | _Noreturn      |
| else     | restrict | void     | _Static_assert |
| enum     | return   | volatile | _Thread_local  |

## D.3 类型

C 中的表达式、函数和对象都有类型。类型规定了如何解释值。例如，若变量  $\kappa$  的类型为 `int`，它所代表的位模式就解释为有符号整数。C 提供语言直接支持的基本类型，也允许在这些类型的基础上构造其他类型。

### D.3.1 基本数据类型

C 的类型系统包含 `int`、`float`、`double`、`char`、`_Bool`，以及复数、虚数相关的 `_Complex` 和 `_Imaginary` 等类型说明符。各种类型的具体大小和值域与实现有关。<sup>4</sup>

#### D.3.1.1 `int`

`int` 的值解释为有符号整数。常见计算机采用 32 位二进制补码表示，其范围为  $-2,147,483,648$  到  $+2,147,483,647$ ，包括两端。这里描述的是常见实现，并非 C18 对所有实现规定的唯一表示。

---

<sup>4</sup>译注：原文说这些类型在所有实现中都存在，过于绝对。C11/C18 允许实现不支持复数类型；虚数类型也不是所有实现都提供的必备特性。

### D.3.1.2 float

`float` 表示单精度浮点数。它通常但不一定采用 IEEE 单精度格式：总长 32 位，其中 1 位符号、8 位采用偏移量 127 的指数，以及 23 位小数部分。参见 2.7.1 节。

### D.3.1.3 double

`double` 表示双精度浮点数，通常也采用 IEEE 格式。它与 `float` 的具体差别由系统决定，但双精度提供的精度不能低于单精度。在许多机器上，`double` 占 64 位。

### D.3.1.4 char

字符类型保存字符的编码；本书讨论的系统使用 ASCII，见附录 E。`char` 必须足以表示基本执行字符集中的字符，其大小为 1 个 C 字节。`short int` 的大小不能小于 `char`。字符类型、各种整数类型和枚举类型统称整数类型；`float` 和 `double` 则属于浮点类型。

### D.3.1.5 枚举类型

枚举类型允许用符号名称表示值。例如，希望一种类型使用 `Penguin`、`Riddler`、`CatWoman` 和 `Joker` 这四个名称，可以写成：

```
// Specifier
enum villains { Penguin, Riddler, CatWoman, Joker };
// Declaration
enum villains badGuy;
```

变量 `badGuy` 的类型为 `enum villains`。枚举项是实际具有整数值的符号常量。默认情况下，第一项为 0，后面每项比前一项加 1，因此四项依次是 0、1、2、3。程序员也可显式指定某项的值：

```
// Specifier
enum villains { Penguin = 3, Riddler, CatWoman, Joker };
```

此时 `Penguin` 为 3，`Riddler` 为 4，依此类推。

## D.3.2 类型修饰

通过某些说明符或限定符，可以改变基本类型的有符号性、大小，或限制对象的修改。<sup>5</sup>

---

<sup>5</sup>译注：原书将本节项目统称为 `type qualifiers`。严格按 C 术语，`signed`、`unsigned`、`long`、`short` 是类型说明符，`const` 才是类型限定符。这里用“类型修饰”保留原书的分组。

### D.3.2.1 signed、unsigned

int 和 char 可以配合 signed 或 unsigned 使用。普通 int 默认有符号；普通 char 的有符号性由实现决定。例如，在采用 32 位二进制补码整数的机器上，signed int 的范围是 -2,147,483,648 到 +2,147,483,647，而 unsigned int 的范围是 0 到 4,294,967,295。

```
signed int c; // the signed modifier is redundant
unsigned int d;
signed char j; // forces the char to be interpreted
 // as a signed value
unsigned char k; // the char will be interpreted as an
 // unsigned value
```

### D.3.2.2 long、short

long 和 short 可用来选择整数类型的大小，例如 short int 和 long int。C 并未规定相邻两种类型必须相差多少，只规定以下关系：

```
sizeof(char) <= sizeof(short int) <= sizeof(int) <= sizeof(long
↪ int)
```

有些支持 64 位数据的机器采用 64 位 long int 和 32 位 int，但并非所有 64 位平台都这样。各整数类型的范围可通过标准头文件 <limits.h> 中的宏取得。在许多 UNIX 系统中，该头文件位于 /usr/include。

```
short int q;
long int p;
unsigned long int r;
```

long 还可与 double 组合为 long double，在实现支持时提供更高精度或更大范围。float 的值集合包含于 double 的值集合，而后者包含于 long double 的值集合。

```
double x;
long double y;
```

**译注：**原文声称 short 也可修饰 double，不符合 C 语法；short double 不是标准 C 类型。

### D.3.2.3 const

const 表示不能通过相应的左值修改对象，例如：

```
const double pi = 3.14159;
```

这类信息也可能帮助编译器优化。通常应在定义只读对象时给出合适的初值。<sup>6</sup>

---

<sup>6</sup>译注：原文说所有 const 变量“必须显式初始化”，并非语言要求。例如，具有静态存储期的 const 对象若未显式初始化，仍会零初始化；extern 声明也不要求带初始化式。

### D.3.3 存储类

本节主要讨论自动存储期和静态存储期。自动对象属于某个块，例如函数体；块的一次执行结束后，对象的生存期也随之结束。函数内的普通局部变量默认具有自动存储期，通常放在运行时栈中，见 14.3.1 节。

静态存储期对象在整个程序执行期间存在。定义在所有块之外的全局变量属于这一类；函数中的局部变量也可用 `static` 声明，使其值在多次调用之间保留。例如：

```
int Count(int x)
{
 static int y;
 y++;
 printf("This function has been called %d times.", y);
 return y;
}
```

`Count` 的活动记录出栈时，`y` 不会丢失。编译器通常把这种变量放在静态数据区，每次调用都更新同一个 `y`。没有显式初始化式的静态对象会零初始化；自动对象则不能在尚未取得确定值时读取。<sup>7</sup>

`register` 可以用于自动对象，提示编译器该对象经常被访问，适合分配到寄存器中。编译器可依据自身分析忽略这一建议。函数和变量还可以用 `extern` 声明，常用于引用在另一目标模块中定义、随后链接到一起的项目；定义也可能位于同一翻译单元的其他位置。

### D.3.4 派生类型

指针、数组、结构体和联合等类型可以在已有类型的基础上构造。结构体和联合使程序员能够把不同类型组织成新的类型。

#### D.3.4.1 数组

数组是在内存中连续排列的一组同类型对象。如果类型 `T` 的数组首元素位于地址 `X`，下一个元素就位于相隔 `sizeof(T)` 字节的位置。下标从 0 开始，`list[0]` 是第一个元素。下面用整数常量表达式给出数组长度：

```
char string[100]; // Declares array of 100 characters
int data[20]; // Declares array of 20 integers
```

常量表达式不一定是单个字面量。C 也有变长数组机制，不过它不是这里的讨论重点，在 C11/C18 实现中可以不提供。用方括号内的整数表达式访问元素：

```
data[0] // Accesses first element of array data
data[i + 3] // The variable i must be an integer
string[x + y] // x and y must be integers
```

---

<sup>7</sup>译注：原例声明返回 `int` 却未返回值，这里补上 `return y;`。形参 `x` 保留原样；该例没有使用它。

编译器不必检查或生成代码检查下标是否越界，责任在程序员。对于上例，`string[x+y]` 要求  $0 \leq x + y < 100$ 。<sup>8</sup>

### D.3.4.2 指针

指针可以保存对象的地址。声明时在标识符前加 `*`，指针类型说明它指向的对象类型：

```
int *v; // v points to an integer
```

C 只允许一组受限制的指针操作。兼容类型的指针可以赋值；把空指针常量 `0` 赋给指针会得到空指针。指针可以加减整数；在同一数组范围内，两个适当类型的指针还可以相减或作大小比较。运算也可涉及数组末尾后一位置的指针，但不能解引用该位置。强制转换可以显式转换某些类型，却不会自动使越界访问或任意地址访问成为合法操作。

### D.3.4.3 结构体

结构体是一种聚合类型，由各自具有类型的成员组成。一般形式如下，省略号代表未展开的成员：

```
struct tag_id {
 type1 member1;
 type2 member2;
 ...
 typeN memberN;
};
```

这些成员分别名为 `member1`、`member2`，直到 `memberN`，类型分别为 `type1`、`type2`，直到 `typeN`。成员可采用适当的基本类型或构造类型，包括其他结构体类型。可选的标记 `tag_id` 用来声明该结构体类型的变量：

```
struct tag_id x;
```

在同一作用域中，即使两个结构体的成员完全相同，分别用不同标记声明的结构体类型仍然不同。也可在说明结构体时直接声明变量：

```
struct point {
 int x;
 int y;
} firstPoint;
// declares an array of structure type variables
struct point image[100];
```

这里一起声明了 `firstPoint`，随后利用标记 `point` 声明结构体数组 `image`。更多内容见 19.2 节。

---

<sup>8</sup>译注：原文写成 “100 or less”，会错误地允许下标 100；长度为 100 的数组最后一个下标是 99。

#### D.3.4.4 联合

结构体为多个成员分别保留存储空间；联合则让若干预先规定类型的成员共享存储空间，可以在程序的不同阶段保存不同类型的值。例如：

```
union u_tag {
 int ival;
 double fval;
 char cval;
} joined;
```

joined 最终保存的是位。用 joined.ival、joined.fval 或 joined.cval 分别选择整数、双精度浮点数或字符成员。编译器为联合分配足以容纳最大成员并满足对齐要求的空间。共享空间不意味着写入某个成员后，其他成员都能作为独立值同时保留。

#### D.3.5 typedef

typedef 为已有类型创建别名，尤其适合为程序员定义的类型提供名称。一般形式为：

```
typedef type name;
```

type 可以是基本类型、枚举类型或其他适当类型，name 是合法标识符。选择得当的类型名称能够进一步说明对象的用途，改善可读性。例如：

```
typedef enum {coffee, tea, water, soda} Beverage;
Beverage drink; // Declaration uses previous typedef

typedef struct {
 int xCoord;
 int yCoord;
 int color;
} Pixel;
Pixel bitmap[1024*820]; // Declares an array of pixels
```

### D.4 声明

对象是用来保存值的存储区域，变量是有名字的对象。使用对象之前，必须先声明，使编译器知道其类型、名称和存储特征，进而生成正确的机器代码。函数也要在调用前声明，以说明返回值类型以及参数的类型、数量和顺序。

#### D.4.1 变量声明

变量声明可概括为：

```
[storage-class] [type-qualifier] {type} {identifier} [=
↪ initializer] ;
```

花括号表示必需部分，方括号表示可选部分，并非实际要输入的符号。`storage-class` 可以是 D.3.3 节中的 `static` 等存储类说明符；`type-qualifier` 表示适用的限定符；`type` 可以是基本类型、枚举类型、数组、指针、结构体或联合类型。

普通标识符可以由字母、数字和下划线组成，第一个字符必须是字母或下划线。大小写不同，例如 `sum` 与 `Sum` 是不同名称；名称也不能是关键字。以下名称在词法上各不相同：

```
blue
Blue1
Blue2
blue
bluE
primary_colors
primaryColors
```

标识符可很长，但可移植程序仍须考虑实现保证的有效字符数。C18 至少保证内部标识符和宏名的前 63 个初始字符有效，外部标识符至少保证 31 个。<sup>9</sup>

自动对象的初始化式可以使用已取得有效值的表达式。静态存储期对象的初始化式则必须满足常量表达式或相应初始化规则。一个声明也可以用逗号列出多个声明符及各自的初始化式。以下为若干例子：

```
static long unsigned int k = 10UL;
register char l = 'Q';
int list[100];
struct node_type n; // Declares a structure variable
```

声明可以放在块的开头，其可见范围受所在块限制；也可以位于所有函数之外，形成文件作用域声明。全局变量的名称从相应声明处开始可见，并可能被内层同名声明遮蔽。C99 及之后还允许在块内交错放置声明和语句，不必全部集中到块开头。参见 12.2.3 节。

## D.4.2 函数声明

函数声明说明返回类型，以及函数希望从调用者接收的参数类型、数量和顺序。概括形式为：

```
{type} {function-id} ([type1] [, type2], ... [, typeN]);
```

这里仍用花括号表示必需部分、方括号表示可选部分。返回类型可为基本类型、枚举、结构体、联合、指针或 `void`，但不能是数组类型；不返回值时使用 `void`。`function-id` 是合法名称，后面的参数类型用逗号分隔，也可附上参数名称。例如：

<sup>9</sup>译注：原文对一般变量只提到 31 个字符，未区分内部与外部名称。下划线开头的名称还受保留标识符规则限制，例如文件作用域下的 `_blue_` 应避免用作用户名称。



```
int Average(int numbers[], int howMany);
```

此声明可用于计算整数数组平均值的函数。若函数确实没有参数，应写 (void)；在 C18 中，单独的空括号声明 f() 不提供参数类型和数量的信息。

## D.5 运算符

本节按所执行的操作对 C 运算符分组介绍。

### D.5.1 赋值运算符

C 支持多种赋值运算符，最基本的是简单赋值 =。所有赋值运算符都从右向左结合。一般形式为：

```
{left-expression} = {right-expression}
```

左操作数必须是可修改左值，不能是函数、const 限定的不可修改对象或整个数组，但可以是数组元素、结构体对象或联合对象。赋值后，左侧对象取得右侧表达式的值。两边通常具有相同类型；若是允许转换的算术类型，则先把右值转换为左侧类型。其他赋值运算符包括：

```
+= -= *= /= %= &= |= ^= <<= >>=
```

它们将运算与赋值合并。一般而言，A op= B 的效果类似 A = A op (B)，例如 x += y 类似 x = x + y；区别是复合赋值对左操作数只求值一次。示例见 12.3.2 和 12.6.3 节。

### D.5.2 算术运算符

C 的二元算术运算符为：

```
+ - * / %
```

它们分别执行加、减、乘、除和取余，通常作用于 int、double、float、char 等算术类型。混合类型按 D.5.11 节的规则转换；% 的两个操作数必须是整数类型，例如 int、char 或枚举类型。

加减也可用于数组范围内的指针运算。若 ptr 的类型为 type \*，ptr+1 产生下一个元素的地址，即在字节地址上前进 sizeof(type) 字节。<sup>10</sup>

C 还支持一元 + 与 -。一元负号产生操作数的相反数；一元正号产生经过适用的整数提升后的值，主要用于与负号保持形式上的对称。更多示例见 12.3.3 节。

---

<sup>10</sup>译注：原文称 ptr+1 等价于 ptr+sizeof(type)，混淆了字节地址变化与 C 指针运算。后一个 C 表达式实际前进 sizeof(type) 个元素。

D.5.3 位运算符

以下位运算符只能作用于整数类型，不能直接作用于浮点数：

& | ^ ~ << >>

左移和右移运算符将左操作数分别向左或向右移动右操作数指定的位数。若移位数为负，或大于等于提升后左操作数类型的位宽，行为未定义。例如，对 32 位操作数移动 32 位已经不合法。表 D.2 以  $x=186$ 、 $y=6$  为例。

表 D.2 C 的位运算符

| 符号 | 操作     | 示例           | 结果    |
|----|--------|--------------|-------|
| &  | 按位 AND | $x \& y$     | 2     |
|    | 按位 OR  | $x   y$      | 190   |
| ~  | 按位 NOT | $\sim x$     | -187  |
| ^  | 按位 XOR | $x \wedge y$ | 188   |
| << | 左移     | $x << y$     | 11904 |
| >> | 右移     | $x >> y$     | 2     |

**译注：**表中按位取反的有符号结果采用本书的二进制补码表示。原文移位数上限漏掉了“等于位宽”这一情况，已修正。有符号左移还受可表示性限制；C18 中负的有符号数右移结果由实现定义。

D.5.4 逻辑运算符

逻辑运算符适合构造包含多个条件的表达式，例如测试条件 A 和条件 B 是否同时成立。逻辑 AND 的两个操作数不必同类型；若二者都非零，结果为 1，否则为 0。逻辑 OR 在至少一个操作数非零时为 1，否则为 0。一元逻辑 NOT 将零变成 1，将非零变成 0。

逻辑 AND 与逻辑 OR 都采用短路求值：若仅计算左操作数就能确定结果，就不计算右操作数。例如  $(x \ || \ y++)$  中，若  $x$  非零，便不会计算  $y++$ ，也不会产生对  $y$  自增的副作用。表 D.3 仍取  $x=186$ 、 $y=6$ 。

表 D.3 C 的逻辑运算符

| 符号 | 操作     | 示例             | 结果 |
|----|--------|----------------|----|
| && | 逻辑 AND | $x \ \&\& \ y$ | 1  |
|    | 逻辑 OR  | $x \    \ y$   | 1  |
| !  | 逻辑 NOT | $!x$           | 0  |

### D.5.5 关系运算符

用于比较的运算符有：

`== != > >= < <=`

它们分别测试相等、不等、大于、大于等于、小于和小于等于。关系成立时结果是整数 1，否则是 0。算术类型混合时按 D.5.11 节转换。指针也可比较，但大小比较主要适用于同一数组等标准允许的情形；相等与不等比较的适用范围更广，也可用于判定空指针。严格术语中，`==` 与 `!=` 属于相等性运算符，其余四个属于关系运算符。

### D.5.6 自增与自减运算符

`++` 与 `--` 将操作数增加或减少 1，都有前缀和后置形式。前置形式的表达式值是修改后的值：

```
int x = 4;
int y;
y = ++x;
```

执行后 `x` 和 `y` 都为 5。后置形式的表达式值是修改前的值，同时仍产生自增或自减的副作用：

```
int x = 4;
int y;
y = x++;
```

执行后 `x` 为 5，`y` 为 4。这些运算符也可以作用于指针，每次前进或后退一个元素，见 D.5.2 节。

### D.5.7 条件表达式运算符

条件运算符的形式为：

`{expressionA} ? {expressionB} : {expressionC}`

若 `expressionA` 非零，则选择并计算 `expressionB`；否则选择并计算 `expressionC`。整个表达式的值来自被选择的分支，并按条件运算符的类型规则转换。例如：

```
w = x ? y : z;
```

`x` 非零时把 `y` 的相应值赋给 `w`，否则使用 `z`。与逻辑 AND、OR 一样，条件运算符不会计算未选中的分支，见 D.5.4 节。

### D.5.8 指针、数组和结构体运算符

下面的运算符用于地址操作和构造类型。

### D.5.8.1 取地址运算符

一元 `&` 取得操作数的地址，常用于变量、数组元素或结构体成员。它也可用于函数指示符；位字段和声明为 `register` 的对象不能按这种方式取地址。

### D.5.8.2 解引用运算符

一元 `*` 与取地址相配合，指示指针所指的对象。例如：

```
int *p;
int x = 5;
p = &x;
*p = *p + 1;
```

`*p` 指示 `x`。它出现在赋值左侧时是用于指定存储位置的左值；在这里的右侧表达式中则取得 `x` 的值。执行后 `x` 为 6。

### D.5.8.3 数组引用

方括号 `[]` 内的整数表达式指定数组下标。下面演示对数组 `list` 的引用：

```
int x;
int list[100];
x = list[x + 10];
```

**译注：**该片段只说明语法，不能直接作为正确的运行示例。执行赋值之前，必须先给 `x` 和所读取的数组元素设置有效值，并保证 `x+10` 在 0-99 内。原片段省略了这些前提。

### D.5.8.4 结构体和联合成员引用

点运算符，直接访问结构体或联合对象的成员。例如：

```
struct pointType {
 int x;
 int y;
};
typedef struct pointType Point;
Point pixel;
pixel.x = 3;
pixel.y = pixel.x + 10;
```

`pixel` 是结构体变量，成员用点运算符访问。<sup>11</sup>箭头运算符 `->` 则通过指向结构体或联合的指针访问成员，把解引用和成员选择结合在一起：

<sup>11</sup>译注：原文 `typedef pointType Point;` 缺少 C 中必需的 `struct`，已补全。

```
Point pixel;
Point *ptr;
ptr = &pixel;
ptr->x = ptr->x + 1;
```

此时 `ptr` 指向 `pixel`。这个独立片段同样要求事先初始化 `pixel.x`，才能读取它并加 1。

### D.5.9 sizeof

`sizeof` 返回保存指定类型对象所需的字节数。例如 `sizeof(int)` 返回一个 `int` 占用的字节数；若操作数是数组，则返回整个数组的大小。

```
int list[45];
struct example_type {
 int valueA;
 int valueB;
 double valueC;
};
typedef struct example_type Example;
...
sizeA = sizeof(list); /* 45 * sizeof(int) */
sizeB = sizeof(Example); /* Size of structure */
```

结果类型为 `size_t`。结构体的大小可能包括对齐填充，不能一般地只把成员大小相加。

### D.5.10 求值次序

理解一个表达式时，先按括号、运算符优先级与结合性确定它的分组。优先级较高的运算符结合得更紧；同一优先级的运算符由结合性确定分组方向。例如加法左结合，`2+3+4` 分组为 `(2+3)+4`。表 D.4 按优先级从高到低列出运算符。

**译注：**原文把分组规则当成了实际求值顺序。优先级和结合性并不一般地规定操作数或函数调用先后执行的次序；短路运算符等另有明确的顺序保证。原表还拆分了本应同组的后置运算符，并把条件运算符的结合方向写反。表 D.4 按 C 语法重新分组，补列逗号运算符。

表 D.4 运算符优先级，由高到低

| 组  | 结合性  | 运算符及说明                                               |
|----|------|------------------------------------------------------|
| 1  | 从左向右 | () [] . -> ++ --; 函数调用、下标、成员、后置自增/自减                 |
| 2  | 从右向左 | ++ -- + - ! ~ * & sizeof _Alignof; 前置运算、一元运算、解引用、取地址 |
| 3  | 从右向左 | (type); 强制类型转换                                       |
| 4  | 从左向右 | * / %; 乘、除、取余                                        |
| 5  | 从左向右 | + -; 加、减                                             |
| 6  | 从左向右 | << >>; 移位                                            |
| 7  | 从左向右 | < > <= >=; 关系比较                                      |
| 8  | 从左向右 | == !=; 相等性比较                                         |
| 9  | 从左向右 | &; 按位 AND                                            |
| 10 | 从左向右 | ^; 按位 XOR                                            |
| 11 | 从左向右 | ; 按位 OR                                              |
| 12 | 从左向右 | &&; 逻辑 AND                                           |
| 13 | 从左向右 | ; 逻辑 OR                                              |
| 14 | 从右向左 | ? :; 条件运算                                            |
| 15 | 从右向左 | = += -= *= /= %= <<= >>= &= ^=  =; 赋值与复合赋值           |
| 16 | 从左向右 | ,; 逗号运算                                              |

D.5.11 类型转换

考虑 A op B。结果类型取决于 A、B 的类型以及运算符 op 的规则。即使操作数类型相同，结果也不一定保持该类型，例如比较运算符产生 int 类型的 0 或 1。

混合算术类型运算会应用通常算术转换。大体上，较小整数类型先提升到 int 或 unsigned int；若有浮点操作数，则转换到适用的浮点类型。例如 double 与 int 相加，结果为 double。涉及有符号和无符号整数时，还要依据整数转换等级和值域，不能只看类型占多少字节。

```
char i;
int j;
float x;
double y;
i * j // This expression is an integer
j + 1 // This expression is an integer
j + 1.0 // This expression is a double
i + 1.0 // This expression is a double
x + y // This expression is a double
i + j + x + y // This is a double
```

**译注：**原文把含 1.0 的两个结果写成 float；无后缀 1.0 是 double，已更正对应英文注释。上述表达式仅用于说明类型，若实际求值，变量须先初始化。

运算符自身也会限制类型。例如取余% 只接受整数类型，char 等会先作整数提升，浮点操作数会导致编译诊断。浮点数转换为整数时丢弃小数部分；如果截断后的值不

能由目标整数类型表示，行为未定义。这种转换可由赋值或显式强制转换等引起，而不是通常算术转换主动把混合运算中的浮点数降为整数。

#### D.5.11.1 强制类型转换

程序员可以显式要求转换：

```
(new-type) expression
```

expression 按相应规则转换为 new-type。沿用前面的变量：

```
j = (int) x + y; // This results in conversion of
 // double into an integer
```

这里先将 x 转为 int，与 double 类型的 y 相加时又转换为 double，最后赋给 j 时再转换为 int。因此，强制转换只作用于紧随其后的 x，并不作用于整个和。

### D.6 表达式与语句

C 程序的工作由函数体内的表达式和语句描述。

#### D.6.1 表达式

表达式由常量、变量、运算符和函数调用等合法组合而成。其分组遵循 D.5.10 节的优先级和结合性规则，结果类型则由各部分类型和运算规则决定，见 D.5.11 节。例如：

```
a * a + b * b
a++ - c / 3
a <= 4
q || integrate(x)
```

本节这些只含 int 算术值的常见表达式产生 int 结果；不过，不能据此把“所有操作数都是 int”当成适用于所有运算符的通则，例如 sizeof 的结果类型另有规定。

#### D.6.2 语句

在表达式后加分号，就得到表达式语句。它常用于修改变量或产生其他副作用。通常，一条语句完成后继续执行顺序上的下一条语句；到达函数体末尾则结束这次函数调用。

```
c = a * a + b * b; // Two simple statements
b = a++ - c / 3;
```

把相关语句放在花括号内，就组成复合语句，也称块。在需要一条语句的位置，通常可以用一个块代替：

```
{ // One compound statement
 c = a * a + b * b;
 b = a++ - c / 3;
}
```

## D.7 控制

控制结构通过条件执行或重复执行，改变默认的顺序执行流程。以下格式中的 `statement` 等名称是语法占位符。

### D.7.1 if

```
if (expression)
 statement
```

`expression` 可以是适用的算术类型或指针类型。若它的值非零，就执行 `statement`；`statement` 可以是简单语句，也可以是复合语句。

```
if (x < 0)
 a = b + c; // Executes if x is less than zero
```

更多示例见 13.2.1 节。

### D.7.2 if-else

```
if (expression)
 statement1
else
 statement2
```

`expression` 非零时执行 `statement1`，否则执行 `statement2`。二者均可作为简单语句或复合语句。

```
if (x < 0)
 a = b + c; // Executes if x is less than zero
else
 a = b - c; // Otherwise, this is executed.
```

更多示例见 13.2.2 节。



### D.7.3 switch

```
switch (expression) {
case const-expr1:
 statement1A
 statement1B
 ...
case const-expr2:
 statement2A
 statement2B
 ...
case const-exprN:
 statementNA
 statementNB
 ...
}
```

`switch` 的控制表达式必须具有整数类型，后面通常是复合语句。各 `case` 标签对应整数常量表达式，同一个 `switch` 中不能出现重复的 `case` 值。计算控制表达式后，若它与某个 `case` 相符，就从该标签之后开始执行。

可选的 `default` 标签处理所有 `case` 都不匹配的情况；若没有 `default` 且没有匹配项，`switch` 体不执行。以下示例中的 `break` 使控制离开 `switch`，见 D.7.7 节。

```
int k;
k = getchar();
switch (k) {
case '+':
 a = b + c;
 break; // break causes control to leave switch
case '-':
 a = b - c;
 break;
case '*':
 a = b * c;
 break;
case '/':
 a = b / c;
 break;
}
```

**译注：**原例用 `char` 接收 `getchar`，这里改为 `int`，以保留 EOF 与所有 `unsigned char` 值的区别。除法分支仍要求 `c` 非零，并满足相应类型的可表示性条件。更多 `switch` 示例见 13.5.1 节。

### D.7.4 while

```
while (expression)
```

```
statement
```

**while** 是迭代结构。若 **expression** 非零，就执行 **statement**，然后再次判断 **expression**，而不是直接进入循环之后的语句。这个过程重复到 **expression** 为 0。循环体可为简单语句或复合语句。下面循环 100 次：

```
x = 0;
while (x < 100) {
 printf("x = %d\n", x);
 x = x + 1;
}
```

更多示例见 13.3.1 节。

### D.7.5 for

```
for (initializer; term-expr; reinitializer)
 statement
```

**initializer** 在循环开始前只执行一次。每次迭代前计算 **term-expr**；非零时继续，为零时退出。每次迭代先执行循环体 **statement**，再计算 **reinitializer**。下面循环 100 次：

```
for (x = 0; x < 100; x++) {
 printf("x = %d\n", x);
}
```

**译注：**原例更新式误写为大写 **X++**，已与循环变量 **x** 统一。C18 的 **initializer** 也可以采用适当的声明。更多示例见 13.3.2 节。

### D.7.6 do-while

```
do
 statement
while (expression);
```

**do-while** 与 **while** 类似，但先执行循环体，再判断 **expression** 是否非零，以决定是否继续。因此循环体至少执行一次。下面同样循环 100 次：

```
x = 0;
do {
 printf("x = %d\n", x);
 x = x + 1;
}
while (x < 100);
```

更多示例见 13.3.3 节。

### D.7.7 break

```
break;
```

**break** 只能用于循环或 **switch**，退出包围它的最内层循环或 **switch**，并转到该结构之后。它常用于在正常终止条件成立之前退出循环。例如：

```
for (x = 0; x < 100; x++) {
 ...
 if (error)
 break;
 ...
}
```

执行 **break** 后立即离开这个 **for** 循环。更多示例见 13.5.2 节。

### D.7.8 continue

```
continue;
```

**continue** 只能用于循环，跳过本次迭代中循环体余下的部分。对于 **while** 和 **do-while**，接着判断循环条件；对于 **for**，先执行更新式，再判断条件。以下例子遇到 **continue** 时先执行 **x++**，再检查 **x<100**，决定是否开始下一轮：

```
for (x = 0; x < 100; x++) {
 ...
 if (skip)
 continue;
 ...
}
```

更多示例见 13.5.2 节。

### D.7.9 return

```
return expression;
```

**return** 将控制交还给调用当前函数的函数。**expression** 提供返回值，并转换为函数声明的返回类型。例如：

```
return x + y;
```

不返回值的 **void** 函数可以使用 **return**；。执行到函数体末尾也会返回；对于需要返回值的普通非 **void** 函数，若调用者使用了这种未提供的返回值，行为未定义。**main** 是一个特例：到达其末尾等价于返回 0。

## D.8 C 预处理器

编译之前，预处理器按程序员给出的指令处理源文本。最常用的功能是宏替换 `#define` 与文件包含 `#include`。预处理指令不是 C 语句，不要求也不应习惯性地末尾补分号。

### D.8.1 宏替换

`#define` 指示预处理器用一组预处理记号替换另一记号。例如：

```
#define A B
```

作为独立标识符出现的 A 会替换成 B；APPLE 中的字母 A 不会被替换，字符串 "A" 中的 A 也不会。替换文本延续到行末；需要跨行时可在行末使用反斜杠续行。

宏也可以带参数，参数表紧接宏名，二者之间不能插入空白。例如：

```
#define REMAINDER(X, Y) ((X) % (Y))
```

调用这个宏时给出两个实参：

```
valueC = REMAINDER(valueA, valueB + 15);
```

预处理器将 X、Y 分别替换成对应实参，得到：

```
valueC = ((valueA) % (valueB + 15));
```

定义中包围 X 和 Y 的括号十分必要；省略后，运算符优先级可能改变原本想要的含义。宏看起来像函数调用，但展开本身不引入普通函数调用的开销。<sup>12</sup>

### D.8.2 文件包含

`#include` 将指定文件的内容纳入当前源文件，常用于包含多个源文件共用的宏和声明。两种常见形式是：

```
#include <stdio.h>
#include "program.h"
```

尖括号形式通常从系统配置的头文件搜索位置查找，适合标准库和系统头文件。双引号形式通常先从当前源文件相关的目录查找，再采用实现规定的后续搜索方式。<sup>13</sup>

<sup>12</sup>译注：原说明误将宏名写成 COPY，并把形参 X、Y 写成 A、B，已统一为示例中的名称。

<sup>13</sup>译注：原文把这两种查找位置说成固定目录规则；实际搜索方法由实现规定，这里保留常见做法并说明这一点。

## D.9 部分标准库函数

ANSI C 标准库提供大量实用函数，执行 I/O、动态内存分配等工作。借助标准库，程序可以在符合相应标准环境的平台之间移植。这里选取一些常用函数。<sup>14</sup>

### D.9.1 I/O 函数

调用标准 I/O 函数的源文件应包含 `<stdio.h>`。

#### D.9.1.1 getchar

```
int getchar(void);
```

`getchar` 从标准输入 `stdin` 读取下一个字符，并把该字符作为 `unsigned char` 转换为 `int` 后返回；遇到文件结束或读取错误时返回 `EOF`。其作用类似 LC-3 的输入 `TRAP`，但不会自行显示输入提示横幅。

许多系统对终端输入采用缓冲方式：键入的字符先由系统保存，按下 `Enter` 后才把整行交给输入流。这是常见的终端行为，而不是 `getchar` 本身必须逐次等待 `Enter` 的规则。

#### D.9.1.2 putchar

```
int putchar(int c);
```

`putchar` 把 `c` 转为 `unsigned char` 后写入标准输出，作用类似 LC-3 的 `OUT`。若标准输出连接屏幕，字符最终显示在屏幕上；采用缓冲输出时，可能要等缓冲区刷新后才出现。面向终端的行缓冲通常在输出换行时刷新。

**译注：**原文将返回类型写成 `void`，已改为 `int`。成功时返回写出的字符，失败时返回 `EOF`。

#### D.9.1.3 scanf

```
int scanf(const char *restrict format, ...);
```

`scanf` 接收格式字符串和相应的指针实参。格式字符串决定如何解释输入字段。例如 `%d` 跳过适用的前导空白，读取十进制整数形式，将其转换为整数并存入下一个指针实参所指的对象。表 D.5 列出本书使用的转换说明。

需要赋值的转换应有类型匹配的目标指针。返回值是成功匹配并赋值的项目数；若在完成第一次转换之前发生输入失败，则返回 `EOF`。

<sup>14</sup>译注：原文说每种 ANSI C 安装都具有全部这些函数；严格地说，这里主要针对宿主实现，自由站立实现只需提供标准规定的较小子集。下列函数声明中的类型错误已修正，完整声明应由对应标准头文件提供。

表 D.5 scanf 转换说明

| 说明     | 输入的解释                         | 对应目标指针         |
|--------|-------------------------------|----------------|
| %d     | 有符号十进制整数                      | int *          |
| %i     | 十进制，或前导 0 的八进制、前导 0x/0X 的十六进制 | int *          |
| %o     | 八进制整数                         | unsigned int * |
| %x     | 十六进制整数                        | unsigned int * |
| %u     | 无符号十进制整数                      | unsigned int * |
| %c     | 字符；不自动追加终止字符                  | char *         |
| %s     | 连续非空白字符，末尾追加 \0               | char *         |
| %f, %e | 浮点数                           | float *        |
| %lf    | 双精度浮点数                        | double *       |

**译注：**原声明中没有类型的 \*ptr1 不是合法函数原型，已改为标准可变参数形式。对%s 应给出不超过目标容量减 1 的字段宽度；例如 8 字节数组使用%7s，并检查返回值。

D.9.1.4 printf

```
int printf(const char *restrict format, ...);
```

printf 把格式字符串中的普通字符写到标准输出，并按转换说明解释后续实参，把转换结果插入输出。例如%d 将对应整数实参转换为十进制数字输出。表 D.6 列出常见说明。每项需要实参的转换都应提供类型匹配的值。成功时返回输出的字符数，发生错误时返回负值。

表 D.6 printf 转换说明

| 说明     | 输出形式                      |
|--------|---------------------------|
| %d, %i | 有符号十进制整数。                 |
| %o     | 无符号八进制整数。                 |
| %x, %X | 无符号十六进制整数，分别使用 a–f 或 A–F。 |
| %u     | 无符号十进制整数。                 |
| %c     | 单个字符，实参按整数提升后传入。          |
| %s     | 输出以 \0 终止的字符串，不输出终止字符本身。  |
| %f     | 十进制小数形式的浮点数，实参为 double。   |
| %e, %E | 指数形式的浮点数，实参为 double。      |
| %p     | 指针，实参应为 void *，具体表示由实现规定。 |

## D.9.2 字符串函数

标准库提供多种字符串操作函数。C 字符串是以空字符终止的字符数组；调用这些函数的源文件应包含 `<string.h>`。下面介绍两个例子。

### D.9.2.1 strcmp

```
int strcmp(const char *stringA, const char *stringB);
```

`strcmp` 比较 `stringA` 与 `stringB`。相等时返回 0；若 `stringA` 按字典序大于 `stringB`，返回正值；较小时返回负值。这里的顺序依据第一个不同字符按 `unsigned char` 解释后的值，不是自然语言词典的区域排序规则。返回值只保证符号，不保证恰好是 1 或 -1。

### D.9.2.2 strcpy

```
char *strcpy(char *restrict stringA, const char *restrict
→ stringB);
```

`strcpy` 把 `stringB` 中的所有字符，包括末尾空字符，复制到 `stringA`，并返回 `stringA`。目标必须具有足够空间，且源与目标区域不能重叠。<sup>15</sup>

## D.9.3 数学函数

标准数学函数需要 `<math.h>`。下面这些函数的参数和返回值均为 `double`：

```
double sin(double x); // sine of x, expressed in
→ radians
double cos(double x); // cosine of x, expressed in
→ radians
double tan(double x); // tan of x, expressed in radians
double exp(double x); // exponential function, e^x
double log(double x); // natural log of x
double sqrt(double x); // square root of x
double pow(double x, double y); // x^y -- x to the y power
```

## D.9.4 实用函数

`<stdlib.h>` 声明内存分配、数据转换、排序及其他实用函数。

### D.9.4.1 malloc

如 19.4 节所述，`malloc` 从可用存储空间分配一块指定大小的内存：

---

<sup>15</sup>译注：原文为 `strcmp`、`strcpy` 省略了相应的 `const` 限定，已补全；`strcpy` 没有原文所暗示的错误返回机制，调用者必须满足容量和不重叠的前提。

```
void *malloc(size_t size);
```

参数给出所需字节数。`size_t` 是一种无符号整数类型，也是 `sizeof` 结果的类型。成功时返回所分配区域的指针；请求无法满足时返回 `NULL`。新分配区域的内容不会自动初始化。

#### D.9.4.2 free

```
void free(void *ptr);
```

`free` 释放 `ptr` 所指的先前动态分配的区域，使其可被重新使用。非空参数必须是相应分配函数返回且尚未释放的有效指针，不能是块内偏移后的地址，也不能重复释放。`free(NULL)` 不执行操作。

#### D.9.4.3 rand 和 srand

`rand` 产生伪随机数序列，而不是真正随机的序列。同一实现中，用相同种子重新初始化会得到相同序列，例如每次用种子 10 都能重现结果。改变种子用于改变序列，但标准不保证不同种子必然产生不同序列，也不保证不同实现具有相同序列。

```
int rand(void);
```

`rand` 返回 0 到 `RAND_MAX`（包括两端）的伪随机整数，`RAND_MAX` 至少为 32767。用 `srand` 设置种子：

```
void srand(unsigned int seed);
```



# 附录 E 常用表格

## E.1 常用数值词头

表 E.1 数值词头

| 数量         | 二进制近似    | 词头    | 缩写    | 词源                 |
|------------|----------|-------|-------|--------------------|
| $10^{24}$  | $2^{80}$ | yotta | Y     | 希腊语“八”：okto        |
| $10^{21}$  | $2^{70}$ | zetta | Z     | 希腊语“七”：hepta       |
| $10^{18}$  | $2^{60}$ | exa   | E     | 希腊语“六”：hexa        |
| $10^{15}$  | $2^{50}$ | peta  | P     | 希腊语“五”：pente       |
| $10^{12}$  | $2^{40}$ | tera  | T     | 希腊语“怪物”：teras      |
| $10^9$     | $2^{30}$ | giga  | G     | 希腊语“巨人”：gigas      |
| $10^6$     | $2^{20}$ | mega  | M     | 希腊语“大”：megas       |
| $10^3$     | $2^{10}$ | kilo  | k     | 希腊语“千”：chilioi     |
| $10^{-3}$  | —        | milli | m     | 拉丁语“千”：milli       |
| $10^{-6}$  | —        | micro | $\mu$ | 希腊语“小”：mikros      |
| $10^{-9}$  | —        | nano  | n     | 希腊语“侏儒”：nanos      |
| $10^{-12}$ | —        | pico  | p     | 西班牙语“少量”：pico      |
| $10^{-15}$ | —        | femto | f     | 丹麦语、挪威语“十五”：femten |
| $10^{-18}$ | —        | atto  | a     | 丹麦语、挪威语“十八”：atten  |
| $10^{-21}$ | —        | zepto | z     | 希腊语“七”：hepta       |
| $10^{-24}$ | —        | yocto | y     | 希腊语“八”：okto        |

**译注：**表 E.1 保留原书采用的词头范围及词源说明。“常用二进制近似”不表示 SI 词头的精确定义；例如 kilo 严格表示  $10^3$ ，而  $2^{10} = 1024$ 。二进制容量可用 Ki、Mi、Gi 等二进制词头明确区分。本表是原书的历史范围，未增补后来引入的 SI 词头。

## E.2 标准 ASCII 码

表 E.2 标准 ASCII 表

| 字符  | 十进制 | 十六进制 | 字符 | 十进制 | 十六进制 | 字符 | 十进制 | 十六进制 | 字符  | 十进制 | 十六进制 |
|-----|-----|------|----|-----|------|----|-----|------|-----|-----|------|
| nul | 0   | 00   | sp | 32  | 20   | @  | 64  | 40   | `   | 96  | 60   |
| soh | 1   | 01   | !  | 33  | 21   | A  | 65  | 41   | a   | 97  | 61   |
| stx | 2   | 02   | "  | 34  | 22   | B  | 66  | 42   | b   | 98  | 62   |
| etx | 3   | 03   | #  | 35  | 23   | C  | 67  | 43   | c   | 99  | 63   |
| eot | 4   | 04   | \$ | 36  | 24   | D  | 68  | 44   | d   | 100 | 64   |
| enq | 5   | 05   | %  | 37  | 25   | E  | 69  | 45   | e   | 101 | 65   |
| ack | 6   | 06   | &  | 38  | 26   | F  | 70  | 46   | f   | 102 | 66   |
| bel | 7   | 07   | '  | 39  | 27   | G  | 71  | 47   | g   | 103 | 67   |
| bs  | 8   | 08   | (  | 40  | 28   | H  | 72  | 48   | h   | 104 | 68   |
| ht  | 9   | 09   | )  | 41  | 29   | I  | 73  | 49   | i   | 105 | 69   |
| lf  | 10  | 0A   | *  | 42  | 2A   | J  | 74  | 4A   | j   | 106 | 6A   |
| vt  | 11  | 0B   | +  | 43  | 2B   | K  | 75  | 4B   | k   | 107 | 6B   |
| ff  | 12  | 0C   | ,  | 44  | 2C   | L  | 76  | 4C   | l   | 108 | 6C   |
| cr  | 13  | 0D   | -  | 45  | 2D   | M  | 77  | 4D   | m   | 109 | 6D   |
| so  | 14  | 0E   | .  | 46  | 2E   | N  | 78  | 4E   | n   | 110 | 6E   |
| si  | 15  | 0F   | /  | 47  | 2F   | O  | 79  | 4F   | o   | 111 | 6F   |
| dle | 16  | 10   | 0  | 48  | 30   | P  | 80  | 50   | p   | 112 | 70   |
| dc1 | 17  | 11   | 1  | 49  | 31   | Q  | 81  | 51   | q   | 113 | 71   |
| dc2 | 18  | 12   | 2  | 50  | 32   | R  | 82  | 52   | r   | 114 | 72   |
| dc3 | 19  | 13   | 3  | 51  | 33   | S  | 83  | 53   | s   | 115 | 73   |
| dc4 | 20  | 14   | 4  | 52  | 34   | T  | 84  | 54   | t   | 116 | 74   |
| nak | 21  | 15   | 5  | 53  | 35   | U  | 85  | 55   | u   | 117 | 75   |
| syn | 22  | 16   | 6  | 54  | 36   | V  | 86  | 56   | v   | 118 | 76   |
| etb | 23  | 17   | 7  | 55  | 37   | W  | 87  | 57   | w   | 119 | 77   |
| can | 24  | 18   | 8  | 56  | 38   | X  | 88  | 58   | x   | 120 | 78   |
| em  | 25  | 19   | 9  | 57  | 39   | Y  | 89  | 59   | y   | 121 | 79   |
| sub | 26  | 1A   | :  | 58  | 3A   | Z  | 90  | 5A   | z   | 122 | 7A   |
| esc | 27  | 1B   | ;  | 59  | 3B   | [  | 91  | 5B   | {   | 123 | 7B   |
| fs  | 28  | 1C   | <  | 60  | 3C   | \  | 92  | 5C   |     | 124 | 7C   |
| gs  | 29  | 1D   | =  | 61  | 3D   | ]  | 93  | 5D   | }   | 125 | 7D   |
| rs  | 30  | 1E   | >  | 62  | 3E   | ^  | 94  | 5E   | ~   | 126 | 7E   |
| us  | 31  | 1F   | ?  | 63  | 3F   | _  | 95  | 5F   | del | 127 | 7F   |

E.3 2 的幂

表 E.3 2 的幂

| 数量       | 十进制值          | 常用缩写 |
|----------|---------------|------|
| $2^1$    | 2             | —    |
| $2^2$    | 4             | —    |
| $2^3$    | 8             | —    |
| $2^4$    | 16            | —    |
| $2^5$    | 32            | —    |
| $2^6$    | 64            | —    |
| $2^7$    | 128           | —    |
| $2^8$    | 256           | —    |
| $2^9$    | 512           | —    |
| $2^{10}$ | 1,024         | 1K   |
| $2^{11}$ | 2,048         | 2K   |
| $2^{12}$ | 4,096         | 4K   |
| $2^{13}$ | 8,192         | 8K   |
| $2^{14}$ | 16,384        | 16K  |
| $2^{15}$ | 32,768        | 32K  |
| $2^{16}$ | 65,536        | 64K  |
| $2^{17}$ | 131,072       | 128K |
| $2^{18}$ | 262,144       | 256K |
| $2^{19}$ | 524,288       | 512K |
| $2^{20}$ | 1,048,576     | 1M   |
| $2^{30}$ | 1,073,741,824 | 1G   |
| $2^{32}$ | 4,294,967,296 | 4G   |

译注：表 E.3 的 K、M、G 保留原书的二进制数量缩写惯例。原表把  $2^{19}$  的十进制值误印为 544,288，已修正为 524,288。

## 附录 F 部分习题的解答

部分习题的解答可在本书网站上找到：

<http://www.mhhe.com/patt3>

# 术语表

本表汇集前言、第 1–20 章及附录共用的术语，用于统一译名及英文保留规则。它不是原书索引。

| 英文                           | 统一译法或保留形式     | 使用说明                  |
|------------------------------|---------------|-----------------------|
| bit                          | 位             | 首次可标注 bit；与 byte 区分   |
| byte                         | 字节            | 不译为“位”                |
| word                         | 字             | 按机器数据单位理解，不能固定等同于两个字节 |
| data type                    | 数据类型          |                       |
| abstraction                  | 抽象            |                       |
| level of abstraction         | 抽象层次          |                       |
| instruction                  | 指令            |                       |
| instruction set architecture | 指令集体系结构（ISA）  | 引入后可使用 ISA            |
| microarchitecture            | 微体系结构         | 与 ISA 区分              |
| register                     | 寄存器           | 具体名称保留，如 R0、R7        |
| program counter              | 程序计数器（PC）     | PC 保留                 |
| instruction register         | 指令寄存器（IR）     | IR 保留                 |
| memory address register      | 存储器地址寄存器（MAR） | MAR 保留                |
| memory data register         | 存储器数据寄存器（MDR） | MDR 保留                |
| memory                       | 存储器           | 指内存空间、内存分配时依语境用“内存”   |
| address                      | 地址            | 地址值保持原样               |
| address space                | 地址空间          |                       |
| data path / datapath         | 数据通路          |                       |
| control unit                 | 控制单元          |                       |
| arithmetic and logic unit    | 算术逻辑单元（ALU）   | ALU 保留                |
| logic gate                   | 逻辑门           |                       |
| combinational logic          | 组合逻辑          |                       |
| sequential logic             | 时序逻辑          |                       |
| finite state machine         | 有限状态机         | 首次可标注 FSM             |
| von Neumann model            | 冯·诺依曼模型       |                       |
| machine language             | 机器语言          |                       |
| assembly language            | 汇编语言          |                       |
| assembler                    | 汇编器           |                       |

| 英文                               | 统一译法或保留形式   | 使用说明             |
|----------------------------------|-------------|------------------|
| compiler                         | 编译器         |                  |
| operating system                 | 操作系统        | 首次可标注 OS         |
| input/output                     | 输入/输出 (I/O) | 引入后可保留 I/O       |
| interrupt                        | 中断          |                  |
| subroutine                       | 子程序         | 与 function 按原文区分 |
| function                         | 函数          | 具体函数名保留          |
| pointer                          | 指针          |                  |
| array                            | 数组          |                  |
| stack                            | 栈           |                  |
| heap                             | 堆           |                  |
| stack frame                      | 栈帧          |                  |
| run-time stack                   | 运行时栈        |                  |
| recursion                        | 递归          |                  |
| iteration                        | 迭代          |                  |
| debugging                        | 调试          |                  |
| breakpoint                       | 断点          |                  |
| condition codes                  | 条件码         | N、Z、P 等名称保留      |
| opcode                           | 操作码         | 操作码实际值及助记符保持原样   |
| C / C++                          | C / C++     | 语言名称保留           |
| LC-3 / x86                       | LC-3 / x86  | 架构名称保留           |
| ASCII                            | ASCII       | 保留，不音译           |
| CPU                              | CPU         | 需要展开解释时用“中央处理器”  |
| ADD / AND / NOT / TRAP           | 原样保留        | 指令助记符，其他指令同理     |
| printf / scanf 等                 | 原样保留        | 函数及 API 名称       |
| deterministic system             | 确定性系统       | 相同初态和相同输入产生相同响应  |
| information hiding               | 信息隐藏        |                  |
| bottom-up / top-down             | 自底向上 / 自顶向下 | 学习方法与设计方法依语境区分   |
| stepwise refinement              | 逐步求精        |                  |
| deconstruct                      | 解构          | 本书指从抽象回到组成部分     |
| universal computational device   | 通用计算设备      |                  |
| analog machine / digital machine | 模拟机器 / 数字机器 |                  |
| Turing machine                   | 图灵机         |                  |
| universal Turing machine         | 通用图灵机       |                  |
| Turing's thesis                  | 图灵论题        | 不译为已获证明的“定理”     |

| 英文                        | 统一译法或保留形式                | 使用说明                                       |
|---------------------------|--------------------------|--------------------------------------------|
| levels of transformation  | 转换层次                     |                                            |
| natural language          | 自然语言                     |                                            |
| mechanical language       | 机械语言                     | 按本书原词，不与 machine language（机器语言）混同          |
| ambiguity                 | 歧义                       |                                            |
| algorithm                 | 算法                       |                                            |
| definiteness              | 确定性                      | 算法语境下指每一步表述精确，区别于 deterministic system 的含义 |
| effective computability   | 有效可计算性                   | 算法步骤可执行                                    |
| finiteness                | 有穷性                      | 过程保证终止                                     |
| operand                   | 操作数                      |                                            |
| addressing mode           | 寻址方式                     | 如 PC+offset                                |
| implementation            | 实现                       | 不与接口规范混同                                   |
| tradeoff                  | 取舍 / 权衡                  | 名词一般用“取舍”，动词一般用“权衡”                        |
| trap                      | 陷阱                       | 指令名 TRAP 保留                                |
| trap service routine      | 陷阱服务例程                   |                                            |
| Trap Vector Table         | Trap Vector Table（陷阱向量表） | 作为表的专名时保留英文                                |
| privilege                 | 特权                       |                                            |
| re-entrant                | 可重入                      |                                            |
| polling                   | 轮询                       |                                            |
| activation record         | 活动记录                     | 与栈帧的关系依上下文说明                               |
| variable-length array     | 变长数组                     | 与泛指大小可变的数组区分                               |
| structure (C)             | 结构体                      | 非 C 类型语境中一般译为“结构”                          |
| container                 | 容器                       | 类名 vector 等保留                              |
| Standard Template Library | 标准模板库（STL）               |                                            |
| PSR                       | PSR                      | 展开时为“处理器状态寄存器”（Processor Status Register）  |
| RISCV                     | RISCV                    | 本书原文拼写，样章保留                                |
| bit pattern               | 位模式                      | 一组位的具体取值；不预设数据类型                           |
| binary digit              | 二进制数字                    | bit 的全称                                    |
| unsigned integer          | 无符号整数                    |                                            |
| signed integer            | 有符号整数                    |                                            |

| 英文                                     | 统一译法或保留形式            | 使用说明                        |
|----------------------------------------|----------------------|-----------------------------|
| signed-magnitude                       | 原码                   | 首次注明英文，避免与泛指的原<br>始编码混淆     |
| 1's complement                         | 反码                   | 整数表示方式；逐位取反操作依<br>语境译为“取反”  |
| 2's complement                         | 补码                   | 整数表示方式；求相反数时为<br>“取补码”      |
| positional notation                    | 位值记数法                | 各位的权值由位置决定                  |
| sign bit                               | 符号位                  |                             |
| sign-extension / SEXT                  | 符号扩展 / SEXT          | 保持原数值，向高位复制符号位              |
| carry / carry-in / carry-out           | 进位 / 输入进位 / 输出进位     |                             |
| overflow                               | 溢出                   | 区分无符号与补码溢出的判定条<br>件         |
| logical variable                       | 逻辑变量                 |                             |
| truth table                            | 真值表                  |                             |
| AND / OR / NOT / XOR                   | AND / OR / NOT / XOR | 运算符保留；中文释义为与、或、<br>非、异或     |
| inclusive-OR / exclusive-OR            | 包含性的或 / 异或           | 后者简称 XOR                    |
| complement / invert                    | 取反 / 反相              | 逐位操作与“取补码”区分                |
| mask / bit mask                        | 掩码 / 位掩码             |                             |
| bit vector                             | 位向量                  | 每位独立表示某个逻辑性质                |
| clear                                  | 清零                   | 将变量或位设为 0                   |
| DeMorgan's laws                        | 德摩根定律                |                             |
| floating point                         | 浮点数 / 浮点             | 数据类型；float 名称保留             |
| normalized form / normalized<br>number | 规格化形式 / 规格化数         |                             |
| subnormal number                       | 次正规数                 | 与规格化数区分                     |
| exponent field                         | 指数域                  |                             |
| fraction field                         | 小数域                  | 不把整个有效数字都称为小数域              |
| excess code / bias                     | 偏置表示 / 偏置值           | excess-127 等具体形式保留          |
| precision / range                      | 精度 / 范围              |                             |
| hexadecimal                            | 十六进制                 | x 前缀和 A-F 数字符号保留            |
| transistor                             | 晶体管                  |                             |
| P-type / N-type                        | P 型 / N 型            |                             |
| gate / source / drain                  | 栅极 / 源极 / 漏极         | 晶体管端子；logic gate 则译为逻<br>辑门 |
| open circuit / short circuit           | 开路 / 短路              | 在开关模型中分别对应断开与导<br>通         |



| 英文                             | 统一译法或保留形式      | 使用说明                   |
|--------------------------------|----------------|------------------------|
| MOS / CMOS                     | MOS / CMOS     | 首次展开中文名称，后续保留缩写        |
| inverter                       | 反相器            | 即 NOT 门、非门             |
| NAND / NOR gate                | NAND 门 / NOR 门 | 首次给出与非门、或非门            |
| decoder                        | 译码器            | decoder 不译为编码器         |
| multiplexer / mux              | 多路选择器 / mux    | 图内 MUX 等名称保持原样         |
| select signal / select line    | 选择信号 / 选择线     |                        |
| asserted                       | 有效 / 置为有效      | 不机械等同于 1；本章所述信号为高电平有效  |
| full adder / half adder        | 全加器 / 半加器      |                        |
| one-bit adder                  | 一位加法器          | 本章正文用于称呼全加器            |
| programmable logic array / PLA | 可编程逻辑阵列 (PLA)  |                        |
| logical completeness           | 逻辑完备性          |                        |
| storage element                | 存储元件           |                        |
| latch / R-S latch              | 锁存器 / R-S 锁存器  | 与触发器严格区分               |
| gated D latch                  | 门控 D 锁存器       |                        |
| transparent latch              | 透明锁存器          |                        |
| quiescent state                | 静止状态           | 锁存器保持已存值的状态            |
| write enable / WE              | 写使能 / WE       |                        |
| addressability                 | 寻址粒度           | 每个可寻址位置的位数，不是地址位数或地址空间 |
| byte-addressable               | 按字节寻址          |                        |
| word line / bit line           | 字线 / 位线        |                        |
| nibble                         | 半字节            | 四位                     |
| state / state diagram          | 状态 / 状态图       |                        |
| current state / next state     | 当前状态 / 下一状态    |                        |
| state transition / arc         | 状态转换 / 弧       |                        |
| synchronous / asynchronous     | 同步 / 异步        |                        |
| clock / clock cycle            | 时钟 / 时钟周期      |                        |
| flip-flop                      | 触发器            | 不与 latch（锁存器）混用        |
| master/slave flip-flop         | 主从触发器          | 主锁存器和从锁存器的图内名称保留       |
| timing diagram                 | 时序图            |                        |
| propagation delay              | 传播延迟           |                        |
| processing unit                | 处理单元           | 与中央处理器 CPU 的整体概念区分     |
| word length                    | 字长             | ALU 处理的基本数据宽度          |

| 英文                                      | 统一译法或保留形式       | 使用说明                     |
|-----------------------------------------|-----------------|--------------------------|
| peripheral                              | 外围设备            | 也简称外设                    |
| instruction pointer                     | 指令指针            | 与程序计数器的用途对应              |
| keyboard data register / KBDR           | 键盘数据寄存器 (KBDR)  | 寄存器名称保留                  |
| keyboard status register / KBSR         | 键盘状态寄存器 (KBSR)  | 寄存器名称保留                  |
| display data register / DDR             | 显示数据寄存器 (DDR)   | 本书 LC-3 I/O 语境           |
| display status register / DSR           | 显示状态寄存器 (DSR)   | 寄存器名称保留                  |
| operate instruction                     | 运算指令            | 如 ADD、AND、NOT            |
| data movement instruction               | 数据传送指令          | 如 LD；不译为数据移动指令           |
| control instruction                     | 控制指令            | 改变指令执行顺序                 |
| source / destination operand            | 源操作数 / 目的操作数    | 区分源值和目的位置                |
| instruction format                      | 指令格式            | 位域及其含义                   |
| immediate / imm                         | 立即数 / imm       | 位域名保留                    |
| initialize                              | 初始化             | 例如将寄存器初始化为 0             |
| load / store                            | 装入 (读) / 存储 (写) | 按对象区分装入寄存器与读取存储器         |
| offset / PC+offset                      | 偏移量 / PC+offset | 与自增后的 PC 相加；偏移量按指令规定符号扩展 |
| instruction cycle                       | 指令周期            | 完成一条指令的整个步骤序列，与时钟周期区分    |
| machine cycle                           | 机器周期            | 本章模型中对应一个时钟周期            |
| FETCH / DECODE                          | 取指令 / 译码        | 指令周期阶段；原图阶段名保留           |
| EVALUATE ADDRESS /<br>FETCH OPERANDS    | 计算地址 / 取操作数     | 指令周期阶段                   |
| EXECUTE / STORE<br>RESULT               | 执行 / 存储结果       | 指令周期阶段                   |
| conditional branch                      | 条件分支            | 指令名 BR 保留                |
| clock generator / crystal<br>oscillator | 时钟发生器 / 晶体振荡器   |                          |
| RUN latch                               | RUN 锁存器         | RUN 保留，不与一般寄存器混用         |
| trap vector                             | 陷阱向量            | 八位编码标识所请求的陷阱服务           |
| system call                             | 系统调用            |                          |
| pipeline                                | 流水线             | 指令处理语境                   |
| general purpose register / GPR          | 通用寄存器 (GPR)     | LC-3 的 R0 至 R7           |
| register file                           | 寄存器组            | 不译为寄存器文件                 |
| literal                                 | 字面量             | 指令或源代码中直接写出的值            |
| PC-relative mode                        | PC 相对寻址         | 以自增后的 PC 为基准             |

| 英文                                  | 统一译法或保留形式                 | 使用说明                   |
|-------------------------------------|---------------------------|------------------------|
| indirect mode                       | 间接寻址                      | 先读出操作数地址，再访问操作数        |
| Base+offset mode                    | 基址加偏移量寻址 / Base+offset 寻址 | 位域 BaseR、offset6 保留    |
| base register / BaseR               | 基址寄存器 / BaseR             |                        |
| effective address                   | 有效地址                      | 实际访问操作数所用的地址           |
| zero extension / ZEXT               | 零扩展 / ZEXT                | 在高位补零                  |
| loop body                           | 循环体                       |                        |
| sentinel                            | 哨兵                        | 用特定值标记终止条件             |
| sentinel method                     | 哨兵法                       | 与计数器法对应                |
| tri-state device / tri-state buffer | 三态器件 / 三态缓冲器              | 第三种输出状态为高阻态            |
| high impedance                      | 高阻态                       |                        |
| structured programming              | 结构化程序设计                   |                        |
| systematic decomposition            | 系统化分解                     |                        |
| sequential construct                | 顺序结构                      |                        |
| conditional construct               | 条件结构                      |                        |
| iterative construct                 | 迭代结构                      |                        |
| trace                               | 跟踪记录                      | 动作译为跟踪；PC、MAR、MDR 名称保留 |
| single-stepping                     | 单步执行                      |                        |
| debugger                            | 调试器                       |                        |
| module                              | 模块                        |                        |
| corner case                         | 边角情况                      | 易被遗漏的特殊输入情况            |
| infinite loop                       | 无限循环                      |                        |
| privileged memory                   | 特权存储器                     | 第 6 章先引入名称，后续解释        |
| assembly                            | 汇编                        | 指汇编器进行的翻译过程            |
| mnemonic                            | 助记符                       | 如 ADD、NOT，保留具体助记符      |
| symbolic address                    | 符号地址                      |                        |
| label                               | 标号                        | 如 AGAIN、NUMBER，保留具体标号  |
| reserved word                       | 保留字                       |                        |
| comment                             | 注释                        | 代码内注释保留英文，正文中的说明翻译     |
| pseudo-op                           | 伪操作                       | .ORIG 等名称保留            |
| assembler directive                 | 汇编指示                      | 伪操作的正式名称               |
| symbol table                        | 符号表                       |                        |

| 英文                         | 统一译法或保留形式   | 使用说明                           |
|----------------------------|-------------|--------------------------------|
| location counter / LC      | 位置计数器 (LC)  | 汇编时使用, 与执行时的 PC 区分             |
| executable image           | 可执行映像       |                                |
| object file                | 目标文件        |                                |
| object module              | 目标模块        |                                |
| link                       | 链接          | 将目标模块组合为可执行映像                  |
| linker                     | 链接器         |                                |
| circular stack             | 循环栈         | 容量有限, 回绕时可覆盖最早的值               |
| palindrome                 | 回文          | 正向和反向读取相同的字符串                  |
| abstract data type         | 抽象数据类型      | 本章沿用原书引入方式                     |
| data structure             | 数据结构        |                                |
| procedure                  | 过程          | 本章作为子程序的别称                     |
| library                    | 库           |                                |
| library routine            | 库例程         |                                |
| caller                     | 调用者         |                                |
| callee                     | 被调用者        |                                |
| return linkage             | 返回链接        | 由 R7 保存的返回地址                   |
| callee save                | 被调用者保存      |                                |
| caller save                | 调用者保存       |                                |
| Last In, First Out / LIFO  | 后进先出 (LIFO) |                                |
| First In, First Out / FIFO | 先进先出 (FIFO) |                                |
| push                       | 压栈          | PUSH 标号与代码保留英文                 |
| pop                        | 弹栈          | POP 标号与代码保留英文                  |
| stack pointer              | 栈指针         | 本书 LC-3 约定使用 R6                |
| stack protocol             | 栈协议         |                                |
| underflow                  | 下溢          | 此处指从空栈或空队列取出元素                 |
| base case                  | 基本情况        | 递归终止的直接求解情况                    |
| row major order            | 行主序         |                                |
| breadcrumb                 | 访问标记        | 沿用“面包屑”的比喻;<br>BREADCRUMB 标号保留 |
| queue                      | 队列          |                                |
| front                      | 队首          | FRONT 指针名保留                    |
| rear                       | 队尾          | REAR 指针名保留                     |
| wrap-around                | 回绕          |                                |
| character string           | 字符串         |                                |
| personnel record           | 人事记录        |                                |

| 英文                             | 统一译法或保留形式    | 使用说明                 |
|--------------------------------|--------------|----------------------|
| priority / priority level / PL | 优先级          | 与特权彼此独立              |
| supervisor privilege           | 管理特权         |                      |
| Supervisor mode                | 管理模式         |                      |
| User mode                      | 用户模式         |                      |
| system space                   | 系统空间         | 寄存器名保留               |
| user space                     | 用户空间         |                      |
| I/O page                       | I/O 页        |                      |
| Master Control Register / MCR  | 主控制寄存器 (MCR) |                      |
| supervisor stack               | 管理栈          | Saved SSP 等寄存器名保留    |
| user stack                     | 用户栈          |                      |
| Supervisor Stack Pointer / SSP | 管理栈指针 (SSP)  |                      |
| User Stack Pointer / USP       | 用户栈指针 (USP)  |                      |
| memory-mapped I/O              | 存储器映射 I/O    | Saved USP 等寄存器名保留    |
| handshaking                    | 握手           |                      |
| ready bit                      | 就绪位          |                      |
| interrupt-driven I/O           | 中断驱动 I/O     |                      |
| echo                           | 回显           | 本章指等待用户输入时显示的提示      |
| prompt                         | 提示信息         |                      |
| control character              | 控制字符         |                      |
| service call                   | 服务调用         |                      |
| interrupt enable / IE          | 中断使能         | 位名称译为中断使能位；区分设备与全局使能 |
| priority encoder               | 优先编码器        |                      |
| instruction boundary           | 指令边界         |                      |
| vectored interrupt             | 向量中断         |                      |
| interrupt vector               | 中断向量         | 本节指不可被中断的指令序列        |
| Interrupt Vector Table         | 中断向量表        |                      |
| atomic unit                    | 原子单元         |                      |
| data type conversion           | 数据类型转换       |                      |
| three-address machine          | 三地址机器        |                      |
| two-address machine            | 二地址机器        |                      |
| zero-address machine           | 零地址机器        |                      |
| stack machine                  | 栈机器          |                      |
| simulation                     | 模拟           |                      |

| 英文                                       | 统一译法或保留形式    | 使用说明                     |
|------------------------------------------|--------------|--------------------------|
| leading zero                             | 前导零          |                          |
| robustness                               | 健壮性          | 程序对无效输入等情况的处理能力          |
| high-level programming language          | 高级编程语言       |                          |
| variable                                 | 变量           | 变量名保留英文                  |
| variable declaration                     | 变量声明         |                          |
| control structure                        | 控制结构         |                          |
| statement                                | 语句           |                          |
| portability                              | 可移植性         |                          |
| maintainability                          | 可维护性         |                          |
| interpretation                           | 解释           | 执行方式称解释执行                |
| interpreter                              | 解释器          |                          |
| compilation                              | 编译           | 可指包括预处理和链接的整体流程          |
| virtual machine                          | 虚拟机          |                          |
| sandbox                                  | 沙箱           | 解释器不必然具有安全隔离能力           |
| class                                    | 类            | C++ 类型构造；类名保留            |
| template                                 | 模板           | C++ 语境                   |
| code segment                             | 代码片段         | 本章指非完整程序的示例，区别于存储布局中的代码段 |
| source file                              | 源文件          |                          |
| source code                              | 源代码          |                          |
| object code                              | 目标代码         |                          |
| preprocessor                             | 预处理器         |                          |
| preprocessor directive                   | 预处理指令        | #define、#include 保留      |
| analysis                                 | 分析           | 编译过程的分析阶段                |
| synthesis                                | 综合           | 编译过程的综合阶段                |
| parsing                                  | 语法分析         | parse 依句式译为解析            |
| register allocation                      | 寄存器分配        |                          |
| instruction scheduling                   | 指令调度         |                          |
| integrated development environment / IDE | 集成开发环境 (IDE) |                          |
| function definition                      | 函数定义         |                          |
| free-format                              | 自由格式         | 仍须遵守语法规则                 |
| white space                              | 空白           | 包括空格、制表符和换行              |
| header file                              | 头文件          | 文件名保留                    |
| macro                                    | 宏            | 按预处理记号展开                 |

| 英文                    | 统一译法或保留形式 | 使用说明                      |
|-----------------------|-----------|---------------------------|
| format string         | 格式字符串     | 内容按原代码保留                  |
| format specification  | 格式说明      | %d、%x、%c 等保留              |
| standard input        | 标准输入      |                           |
| standard output       | 标准输出      |                           |
| newline character     | 换行符       | 代码中写作 \n                  |
| operator              | 运算符       | 区别于一次具体运算 operation       |
| memory object         | 存储器对象     |                           |
| type specifier        | 类型说明符     | int、char 等保留              |
| initializer           | 初始化器      |                           |
| boolean value         | 布尔值       | true、false 保留             |
| identifier            | 标识符       | 实际名称保留英文                  |
| scope                 | 作用域       | 名称可见的代码范围，不等同于存储期         |
| local variable        | 局部变量      |                           |
| global variable       | 全局变量      |                           |
| block                 | 块         | 花括号括起的代码区域                |
| indeterminate value   | 不确定值      | 不将未初始化值当作安全的随机数           |
| storage duration      | 存储期       | 与作用域区分                    |
| expression            | 表达式       |                           |
| compound statement    | 复合语句      |                           |
| assignment operator   | 赋值运算符     | = 保留                      |
| arithmetic operator   | 算术运算符     |                           |
| integer remainder     | 整数余数      | % 不译作整数除法                 |
| precedence            | 优先级       | 运算符分组规则，不等同于操作数求值先后       |
| associativity         | 结合性       | 区别于算术结合律                  |
| bitwise operator      | 位运算符      |                           |
| unary operator        | 一元运算符     |                           |
| binary operator       | 二元运算符     |                           |
| relational operator   | 关系运算符     |                           |
| logical operator      | 逻辑运算符     |                           |
| increment / decrement | 自增 / 自减   | 指 C 的 ++、-；一般操作依语境可称递增、递减 |
| prefix / postfix      | 前置 / 后置   | 运算符语境                     |
| global data section   | 全局数据区     |                           |
| frame pointer         | 帧指针       | 本书 LC-3 C 约定使用 R5         |

| 英文                           | 统一译法或保留形式     | 使用说明                     |
|------------------------------|---------------|--------------------------|
| program text                 | 程序代码区         | 存储器映射语境                  |
| qualifier                    | 限定符           | const 保留                 |
| literal constant             | 字面常量          |                          |
| symbolic value               | 符号值           | 以名称表示的值                  |
| compound assignment          | 复合赋值          | +=、-= 等保留                |
| conditional expression       | 条件表达式         | ? : 保留                   |
| type cast                    | 类型转换          | 显式转换语境，可称强制类型转换          |
| iteration construct          | 迭代结构          | 与 iterative construct 同义 |
| decision construct           | 判定结构          | 本章指 if 和 if-else；与条件结构相应 |
| sentinel condition           | 哨兵条件          | 用于结束循环的事件或条件             |
| sentinel-controlled loop     | 哨兵控制循环        |                          |
| counter-controlled loop      | 计数器控制循环       |                          |
| nested loop                  | 嵌套循环          |                          |
| iteration variable           | 迭代变量          |                          |
| test condition               | 测试条件          | 循环与条件结构语境                |
| case label                   | case 标签       | case、default 关键字保留       |
| constant expression          | 常量表达式         |                          |
| series expansion             | 级数展开          |                          |
| prime number                 | 素数            |                          |
| tab character                | 制表符           | C 转义序列为 \t               |
| function declaration         | 函数声明          |                          |
| function prototype           | 函数原型          | 带有形参类型信息的函数声明            |
| function call                | 函数调用          |                          |
| argument / actual argument   | 实参 / 实际参数     | 调用时提供的值或表达式              |
| parameter / formal parameter | 形参 / 形式参数     | 函数内接收实参值的变量              |
| formal parameter list        | 形式参数列表 / 形参列表 |                          |
| return value                 | 返回值           | 与返回地址区分                  |
| function body                | 函数体           |                          |
| pass by value                | 传值            | C 的参数传递方式                |
| calling convention           | 调用约定          |                          |
| preamble code                | 序言代码          | 函数建立栈帧的准备代码              |
| bookkeeping information      | 管理信息          | 栈帧中的保存信息，如返回地址、帧指针       |
| Pythagorean triple           | 勾股数组          | 满足勾股关系的三个正整数             |
| brute force                  | 穷举法           | 本章遍历候选整数的所有组合            |



| 英文                                              | 统一译法或保留形式        | 使用说明                            |
|-------------------------------------------------|------------------|---------------------------------|
| testing                                         | 测试               | 通过试验暴露软件错误                      |
| syntactic error / syntax error /<br>parse error | 语法错误             |                                 |
| semantic error                                  | 语义错误             | 本章指语法可接受但行为不符合意图的错误             |
| algorithmic error                               | 算法错误             | 求解方法本身有误                        |
| specification                                   | 规格               | 软件应满足的输入、输出及行为要求                |
| specification error                             | 规格错误             | 需求定义、传达或理解有误                    |
| control-flow error                              | 控制流错误            |                                 |
| initialization error                            | 初始化错误            |                                 |
| off-by-one error                                | 差一错误             | 循环边界、计数或容量等相差 1                 |
| black-box testing                               | 黑盒测试             | 根据规格检验外部行为                      |
| white-box testing                               | 白盒测试             | 根据内部结构组织测试                      |
| checker program                                 | 校验程序             | 独立实现，用于核对待测程序结果                 |
| assertion                                       | 断言               | 与硬件信号的 asserted 区分              |
| source-level debugger                           | 源代码级调试器          |                                 |
| conditional breakpoint                          | 条件断点             |                                 |
| watchpoint                                      | 监视点              | 根据数据访问、变化或附加条件暂停执行              |
| modular design                                  | 模块化设计            |                                 |
| defensive programming                           | 防御式编程            |                                 |
| pointer variable                                | 指针变量             | 保存指针值的变量                        |
| address operator                                | 取地址运算符           | C 的一元 &                         |
| indirection operator /<br>dereference operator  | 间接访问运算符 / 解引用运算符 | C 的一元 *                         |
| null pointer                                    | 空指针              | 不指向任何对象或函数的特殊指针值                |
| null character                                  | 空字符              | 字符值为 0，与空指针区分                   |
| null-terminated string                          | 以空字符结尾的字符串       | 结尾标记占一个字符位置                     |
| array index / subscript                         | 数组下标 / 下标        | C 数组从 0 开始编号                    |
| base address                                    | 基地址              | 本章指数组首元素的地址                     |
| call by reference / pass by<br>reference        | 引用调用 / 传递引用      | C 中传入指针值，参数机制仍为传值；C++ 可直接声明引用参数 |
| insertion sort                                  | 插入排序             |                                 |
| multidimensional array                          | 多维数组             |                                 |
| pixel                                           | 像素               | picture element 的缩写             |

| 英文                               | 统一译法或保留形式   | 使用说明                  |
|----------------------------------|-------------|-----------------------|
| recurrence equation              | 递推方程        | 用较小参数上的函数值定义当前值       |
| recursive function               | 递归函数        | 调用自身的函数               |
| tail recursion                   | 尾递归         | 递归调用之后没有待执行的计算        |
| Towers of Hanoi                  | 汉诺塔         | 保留程序标识符 MoveDisk      |
| Fibonacci number                 | Fibonacci 数 | 按原书约定 $f(0)=f(1)=1$   |
| Golden Ratio                     | 黄金分割比       | 相邻 Fibonacci 数之比的极限   |
| binary search                    | 二分查找        | 在有序区间中按中点缩小范围         |
| depth-first search               | 深度优先搜索      | 优先沿一条分支深入搜索           |
| static linking                   | 静态链接        | 库代码合入可执行映像            |
| dynamic linking                  | 动态链接        | 装入或运行时完成所需链接          |
| dynamically linked library (DLL) | 动态链接库 (DLL) | 保留缩写 DLL              |
| stream                           | 流           | I/O 的顺序数据抽象           |
| buffer                           | 缓冲区         | 存储待处理数据的区域；区别于电路缓冲器   |
| buffering                        | 缓冲          | 暂存输入或输出数据             |
| flush                            | 刷新          | 将输出缓冲内容交付给底层环境        |
| line buffering                   | 行缓冲         | 换行等条件触发传送             |
| full buffering                   | 全缓冲         | 缓冲区填满等条件触发传送          |
| formatted I/O                    | 格式化 I/O     | 依据格式进行文本和数值转换         |
| conversion specification         | 转换说明        | 与较宽泛的“格式说明”按上下文区分     |
| variable argument list           | 可变实参列表      | 函数调用的实参数量可变           |
| file pointer                     | 文件指针        | 指向 FILE 对象，不是文件内容地址   |
| file system                      | 文件系统        | 管理文件存储与访问             |
| end of file (EOF)                | 文件结束 (EOF)  | EOF 是宏表示的返回值，不是文件中的字符 |
| aggregate type                   | 聚合类型        | 如结构体和数组               |
| member                           | 成员          | 结构体或类中的组成项            |
| structure tag                    | 结构体标记       | struct 后的类型标记         |
| type alias                       | 类型别名        | typedef 为已有类型提供同义名称   |
| dynamic memory allocation        | 动态内存分配      | 由程序在运行时申请对象存储空间       |
| memory allocator                 | 内存分配器       | 管理动态内存申请与回收           |
| deallocation                     | 释放          | 结束分配对象的生存期并使空间可重用     |

| 英文                         | 统一译法或保留形式 | 使用说明                          |
|----------------------------|-----------|-------------------------------|
| dynamic array              | 动态数组      | 可通过重新分配调整容量；与变长数组区分           |
| linked list                | 链表        | 通过指针连接节点的线性数据结构               |
| node                       | 节点        | 数据结构中的一个组成单位                  |
| head pointer               | 头指针       | 指向链表首节点的指针；空表时为空指针            |
| tail node                  | 尾节点       | 单链表最后一个节点，后继为空指针              |
| traversal                  | 遍历        | 依次访问数据结构中的元素                  |
| random access              | 随机访问      | 可直接按下标访问位置，不表示随机挑选            |
| sequential access          | 顺序访问      | 沿序列逐项访问                       |
| namespace                  | 命名空间      | C++ 中用于组织名称的作用域；std 保留        |
| global namespace           | 全局命名空间    | 未置于具名命名空间中的全局实体所属空间           |
| scope resolution operator  | 作用域解析运算符  | C++ 的::                       |
| stream insertion operator  | 流插入运算符    | 输出流语境中的 «                     |
| stream extraction operator | 流提取运算符    | 输入流语境中的 »                     |
| stream manipulator         | 流操纵器      | endl 等名称保留                    |
| function overloading       | 函数重载      | 同名函数具有不同的形参类型列表               |
| reference                  | 引用        | C++ 类型语境；与通过指针间接访问区别          |
| object                     | 对象        | 类语境中指实例；语言标准中的对象范围更广          |
| method / member function   | 方法 / 成员函数 | 类中定义的函数；method 对应“方法”         |
| access specifier           | 访问说明符     | public、private、protected 保留英文 |
| public member              | 公有成员      | 允许类外具有相应可见性的代码访问              |
| private member             | 私有成员      | 受类的访问控制规则保护                   |
| constructor                | 构造函数      | 创建类对象时执行初始化                   |
| destructor                 | 析构函数      | 销毁类对象时执行清理                    |
| default constructor        | 默认构造函数    | 能够不提供实参调用的构造函数                |
| inheritance                | 继承        | 从已有类建立派生类的机制                  |
| base class                 | 基类        | 被继承的类                         |

| 英文                           | 统一译法或保留形式      | 使用说明                         |
|------------------------------|----------------|------------------------------|
| derived class                | 派生类            | 从基类派生而来的类                    |
| vector                       | vector         | C++ 容器类名保留；解释为可调整大小的动态数组     |
| iterator                     | 迭代器            | 表示容器中的位置；不等同于整数下标            |
| microinstruction             | 微指令            | 控制存储器中的一组控制位                 |
| microprogrammed control      | 微程序控制          | 以控制存储器实现控制结构                 |
| control store                | 控制存储器          | 保存微指令，区别于主存储器                |
| microsequencer               | 微定序器           | 产生下一条微指令地址                   |
| access control violation     | 访问控制违规（ACV）    | 用户模式访问特权存储器引发的异常             |
| illegal opcode exception     | 非法操作码异常        | LC-3 的未定义操作码 1101            |
| privilege mode exception     | 特权模式异常         | 用户模式执行 RTI                   |
| binary-coded decimal         | 二进制编码的十进制（BCD） | 十进制数字分别编码                    |
| packed BCD                   | 压缩 BCD         | 每个十进制数字占四位                   |
| saturation arithmetic        | 饱和运算           | 超范围结果截至可表示边界                 |
| segment selector             | 段选择子           | x86 段寄存器保存的选择信息              |
| segment descriptor           | 段描述符           | 描述段的基址、界限和属性                 |
| segment override prefix      | 段覆盖前缀          | 覆盖默认段选择                      |
| displacement                 | 位移量            | x86 有效地址中的常量位移               |
| scaled index                 | 比例索引           | 乘以比例因子的索引寄存器值                |
| direction flag               | 方向标志（DF）       | x86 字符串操作的增减方向               |
| carry flag                   | 进位标志（CF）       | x86 标志位                      |
| overflow flag                | 溢出标志（OF）       | x86 标志位                      |
| SIMD                         | SIMD           | 单指令多数据；缩写保留                  |
| MMX                          | MMX            | x86 多媒体扩展名称保留                |
| enumerated type              | 枚举类型           | enum 声明的类型                   |
| enumerator                   | 枚举项            | C18 中是 int 类型常量              |
| union                        | 联合             | union 类型，共享成员存储空间            |
| modifiable lvalue            | 可修改左值          | 赋值左操作数所需条件                   |
| integer promotion            | 整数提升           | 较小整数类型提升为 int 或 unsigned int |
| usual arithmetic conversions | 通常算术转换         | 混合算术类型操作的转换规则                |
| short-circuit evaluation     | 短路求值           | 未必求值右操作数或未选中的分支              |
| static storage duration      | 静态存储期          | 对象存在于整个程序执行期间                |
| automatic storage duration   | 自动存储期          | 对象生存期与块执行关联                  |

| 英文                      | 统一译法或保留形式 | 使用说明                     |
|-------------------------|-----------|--------------------------|
| storage-class specifier | 存储类说明符    | static、extern、register 等 |
| type qualifier          | 类型限定符     | const 等                  |
| lexicographical order   | 字典序       | strcmp 按首个不同字节比较         |
| numerical prefix        | 数值词头      | SI 十进制词头与二进制词头应区分        |